Informatica® Reference 360
October 2026

Reference 360

Informatica Reference 360 Reference 360
October 2026

© Copyright Informatica LLC 2019, 2026

This software and documentation are provided only under a separate license agreement containing restrictions on use and disclosure. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC.

U.S. GOVERNMENT RIGHTS Programs, software, databases, and related documentation and technical data delivered to U.S. Government customers are "commercial computer software" or "commercial technical data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, duplication, disclosure, modification, and adaptation is subject to the restrictions and license terms set forth in the applicable Government contract, and, to the extent applicable by the terms of the Government contract, the additional rights set forth in FAR 52.227-19, Commercial Computer Software License.

Portions of this software and/or documentation are subject to copyright held by third parties. Required third party notices are included with the product.

The information in this documentation is subject to change without notice. If you find any problems in this documentation, report them to us at infa_documentation@informatica.com.

Informatica products are warranted according to the terms and conditions of the agreements under which they are provided. INFORMATICA PROVIDES THE INFORMATION IN THIS DOCUMENT "AS IS" WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING WITHOUT ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND ANY WARRANTY OR CONDITION OF NON-INFRINGEMENT.

Publication Date: 2026-10-30

Table of Contents

Preface 13

Informatica Resources 13

- Informatica Documentation 13
- IDMC web site 13
- Intelligent Data Management Cloud Communities 13
- IDMC Marketplace 13
- Informatica Knowledge Base 14
- IDMC Trust Center 14
- Informatica Global Customer Support 14

Chapter 1: Introducing Reference 360 15

Key concepts 15

- System reference data 15
- Reference data sets 16
- Code lists 19
- Code values 23
- Crosswalks 23
- Hierarchies 25
- Attributes 26
- Dictionaries 31
- Business IDs 32
- Batch Jobs 32
- Workflows 33
- Versions 34
- Comparisons 35
- History 37
- Relationships 38
- Data quality rule associations 39
- Search 40
- Internal IDs and aliases 40
- Localization Languages code list 41

My Services page 41

Home page 42

Explore page 43

Customize the brand 46

Keyboard shortcuts 46

Chapter 2: Getting started with Reference 360 49

Reference 360 licenses 49

- Reference Data Management Basic Edition 49

SaaS Edition - Reference 360 license . . . 50
Users, groups, and roles. . . . 50
Reference 360 roles. . . . 50
Stakeholder roles. . . . 53
Guidelines for assigning roles. . . . 55
Guidelines for restricting access to assets. . . . 55
Creating an Informatica Intelligent Cloud Services user. . . . 56
Creating a user group. . . . 57
Managing privileges for custom user roles . . . 57
Configure privileges for custom roles to access the Security page. . . . 58
Prerequisites. . . . 58
Rules and guidelines for managing privileges for custom user roles. . . . 59
Configure privileges for system reference data. . . . 60
Configure privileges to access reference data assets. . . . 61
Configure privileges to access a code list. . . . 65
Configure privileges for multiple code lists. . . . 72
Configure privileges to access a crosswalk. . . . 72
Configure privileges for multiple crosswalks. . . . 75
Configure privileges to access a hierarchy. . . . 76
Configure privileges for multiple hierarchies. . . . 78
Configure privileges to view the history of records. . . . 79
Configure workflow privileges for custom user roles. . . . 80
Configure privileges to manage multilingual assets. . . . 85
SAML single sign-on. . . . 85
SAML single sign-on requirements. . . . 87
Single sign-on restrictions. . . . 87
User management with SAML single sign-on. . . . 87
Configuring SAML single sign-on for IDMC. . . . 88
Configuring provider settings and mapping attributes. . . . 91
Downloading the service provider metadata. . . . 95
Reference data configuration process. . . . 96
Best practices to configure reference data . . . 97
Migrate Reference 360 assets. . . . 97
Best practices to migrate assets. . . . 98
Guidelines to import an existing asset based on identifiers. . . . 98
Migrating reference data assets. . . . 98

Chapter 3: Manage system reference data. . . . 100

Adding values to system reference data. . . . 100
Editing values of system reference data. . . . 101
Deleting values of system reference data. . . . 101

Chapter 4: Manage reference data sets. 102

Creating reference data sets. 102
Step 1. Create a reference data set. 102
Step 2. Define a reference data set. 103
Step 3. Assign stakeholders. 105
Step 4. View the history of a reference data set. 106
Deleting reference data sets. 107
Considerations for deleting reference data sets. 108

Chapter 5: Manage code lists. 109

Creating code lists. 109
Step 1. Create a code list. 110
Step 2. Define the structure, attributes, and display settings of code lists. 111
Step 3. Assign stakeholders. 116
Viewing the history feed of a code list. 117
Viewing code lists at a point in time. 119
Associate code list attributes with Data Governance and Catalog assets. 119
Managing Data Quality dictionaries in code lists. 120
Create a dictionary. 120
Synchronize data in dictionaries. 120
Re-create the dictionary. 121
Editing code lists. 121
Add multilingual code values to existing code lists. 121
Deleting code lists. 122
Considerations for deleting code lists. 122

Chapter 6: Manage code values. 123

Adding code values. 123
Rules and guidelines for adding code values with dependent reference data attributes. 125
Creating child code values. 126
Editing code values. 127
Viewing history of a code value. 128
Cloning code values. 130
Moving code values. 130
Exporting code values. 130
Deleting code values. 132
Considerations for deleting code values. 132
Purging code values. 133
Rules and guidelines for purging code values. 133
Configuring privileges to purge code values. 133
Performance considerations for purging code values. 134
Purging code values. 134

Chapter 7: Manage crosswalks. 135

Step 1. Create a crosswalk. 135
Step 2. Configure value mappings. 137
Step 3. Assign stakeholders. 138
Viewing history of a crosswalk. 139
Exporting value mappings of crosswalks. 140
Managing Data Quality dictionaries in crosswalks. 141
Create a dictionary. 141
Synchronize data in a dictionary. 142
Re-create the dictionary. 142
Deleting crosswalks. 142
Considerations for deleting crosswalks. 143

Chapter 8: Manage multilingual assets. 144

Rules and guidelines for managing multilingual assets. 144
Localization Languages code list. 147
Adding language code values. 148
BCP 47 code format. 149
Viewing language code values. 150
Deleting language code values. 151

Chapter 9: Import data. 152

Import code lists and crosswalks. 152
Import multilingual code values into code lists. 152
Import related code values into a hierarchy. 153
Rules and guidelines to import data. 154
Import process. 155
Step 1. Upload a file. 155
Step 2. Map fields. 156
Step 3. Preview and import data. 157
Field mapping. 157

Chapter 10: Manage hierarchies. 159

Creating hierarchy models. 159
Step 1. Create a hierarchy. 160
Step 2. Define the hierarchy model. 161
Step 3. Assign stakeholders. 162
Creating hierarchies. 163
Viewing the history of a hierarchy. 165
Viewing hierarchies at a point in time. 165
Exporting hierarchies. 166
Deleting hierarchies. 167

Chapter 11: Manage attributes. 168

Deleting attributes. 168

Considerations for deleting attributes. 169

Basic rule associations. 169

Guidelines for basic rule associations. 170

Add basic rule associations. 171

Changing the execution order of basic rule associations. 172

Advanced rule associations. 172

Rules and guidelines for advanced rule associations. 172

Add advanced rule associations. 173

Redeploying advanced rule associations. 174

Configuring display attributes. 174

Chapter 12: Manage workflows. 176

Configuring workflows. 177

Reviewing tasks. 178

Chapter 13: Manage jobs. 180

Defining reference data import jobs. 180

Reference data import process. 180

Prerequisites for importing reference data. 180

Rules and guidelines for reference data import jobs. 181

Import reference data. 181

Defining reference data export jobs. 182

Prerequisites for exporting reference data. 182

Rules and guidelines for reference data export jobs. 183

Export types. 183

Export reference data. 184

Monitoring jobs. 185

My jobs page. 185

Viewing specific job details. 186

Creating a job schedule. 187

Modifying and rerunning reference data import and export jobs. 188

System-generated jobs. 188

Stopping reference data import and file import jobs. 189

Monitoring reference data import jobs. 190

Extract. 190

Transform. 190

Load. 191

Index for Search. 191

Monitoring the reference data export jobs. 192

Stage. 192

Extract. . . . 193
Monitoring Reference 360 Draft jobs. . . . 193
Commit Changelist. . . . 193
History, Publish Events. . . . 194
Index for Search. . . . 194
Generate Tokens. . . . 194

Chapter 14: Monitor Reference 360. . . . 195

Configure access to monitor Reference 360. . . . 195
Monitor usage statistics for Reference 360. . . . 195

Chapter 15: Reference 360 REST API. . . . 197

Session IDs. . . . 198
Asset IDs. . . . 199
REST API versions. . . . 200
configuration. . . . 204
Get authorization mode configuration. . . . 206
Update authorization mode configuration. . . . 206
Get approval mode configuration. . . . 207
Update approval mode configuration. . . . 208
Get approval implementation mode. . . . 208
Update approval implementation mode. . . . 209
model version 3. . . . 210
Export model (v3). . . . 210
Import model (v3) for adding new assets and updating existing assets. . . . 224
Import model (v3) for adding new assets. . . . 238
Get details of an import model job (v3). . . . 253
import version 1. . . . 259
Import code values (v1). . . . 259
Import value mappings (v1). . . . 261
Import hierarchy relationships (v1). . . . 263
Get import job status (v1). . . . 266
Get failed import job report (v1). . . . 267
import version 2. . . . 269
Import code values (v2). . . . 269
Import value mappings (v2). . . . 272
Import hierarchy relationships (v2). . . . 275
import version 3. . . . 278
Import code values (v3). . . . 278
Import crosswalk values (v3). . . . 281
Import hierarchy relationships (v3). . . . 284
Get import job status (v3). . . . 287
Get failed import job report (v3). . . . 288

export version 1. . . . 290
Export code values to a CSV file (v1). . . . 290
Export code values to JSON format (v1). . . . 292
Export value mappings to a CSV file (v1). . . . 293
Export value mappings to JSON format (v1). . . . 295
export version 2. . . . 296
Export code values to a CSV file (v2). . . . 296
Export code values to JSON format (v2). . . . 298
Export value mappings to a CSV file (v2). . . . 300
Export value mappings to JSON format (v2). . . . 301
Export hierarchies to a CSV file (v2). . . . 302
Export hierarchies to JSON format (v2). . . . 304
Filter criteria for export version 2. . . . 306
export version 3. . . . 309
Export code lists at a point in time to JSON format (v3). . . . 309
Export code lists at a point in time to CSV format (v3). . . . 311
Export value mappings at a point in time to JSON format (v3). . . . 314
Export value mappings at a point in time to CSV format (v3). . . . 316
Export incoming crosswalk mappings to CSV format (v3). . . . 318
Export outgoing crosswalk mappings to CSV format (v3). . . . 320
Export value mappings to CSV format with display attributes in the header (v3). . . . 322
Export value mappings to CSV format with code list names in the header (v3). . . . 324
Filter criteria for export version 3 and 4. . . . 326
export version 4. . . . 329
Export code lists at point in time to JSON format (v4). . . . 329
Export code lists at point in time to CSV format (v4). . . . 332
Export value mappings at point in time to JSON format (v4). . . . 335
Export value mappings at point in time to CSV format (v4). . . . 336
Export incoming crosswalk mappings (v4). . . . 338
Export outgoing crosswalk mappings (v4). . . . 340
Export value mappings to CSV format with display attributes in the header (v4). . . . 342
Export value mappings to CSV format with code list names in CSV header (v4). . . . 344
Export hierarchies to a CSV file (v4). . . . 346
Export hierarchies to JSON format (v4). . . . 348
audit trail version 1. . . . 350
Get history of specified assets by time range (v1). . . . 350
audit trail version 2. . . . 353
Get history of specified assets by time range (v2). . . . 353
rds version 1. . . . 357
Get reference data sets (v1). . . . 357
Get reference data set details (v1). . . . 358
Get history of a reference data set by time range (v1). . . . 361

Get code lists (v1). . . . 366
rds version 2. . . . 368
Get reference data sets (v2). . . . 368
Get reference data set details (v2). . . . 370
Get history of a reference data set by time range (v2). . . . 373
Get code lists (v2). . . . 378
codelists version 1. . . . 381
Get code list details (v1). . . . 381
Get history of a code list by time range (v1). . . . 384
Enable auto-generation of Code field values for a code list (v1). . . . 389
Get the status of a code list if it's enabled with auto-generation of Code field values (v1). . . . 390
Get code value details (v1). . . . 391
Get history of code values of a code list (v1). . . . 392
Move a code value (v1). . . . 396
Create a code value (v1). . . . 397
Update a code value (v1). . . . 399
Delete code values (v1). . . . 401
Search code values (v1). . . . 403
Synchronize Data Quality dictionaries with code values of a code list. . . . 406
Export a subset of code values in a hierarchical code list (v1). . . . 406
Get history of code value hierarchies by time range (v1). . . . 412
Get crosswalks for a code list (v1). . . . 416
Get latest modified code values by time range (v1). . . . 417
Get modified code value relationships in hierarchy by time range (v1). . . . 419
Get modified code lists by time range (v1). . . . 421
codelists version 2. . . . 423
Unlock a code list (v2). . . . 423
Create a code value (v2). . . . 424
Update a code value (v2). . . . 426
Move a code value (v2). . . . 428
Delete code values (v2). . . . 430
Get code value details (v2). . . . 432
Get latest modified code values by time range (v2). . . . 433
Get modified code value relationships in hierarchy by time range (v2). . . . 435
Get modified code lists by time range (v2). . . . 437
Get code list details (v2). . . . 439
Get history of a code list by time range (v2). . . . 444
Get crosswalks for a code list (v2). . . . 449
Get history of code value hierarchies by time range (v2). . . . 451
Get history of code values of a code list (v2). . . . 455
Search code values (v2). . . . 460
Export code value hierarchy to JSON format (v2). . . . 462

Update cursor settings of the subset of code values (v2). . . . 464
codelists version 3. . . . 466
Unlock a code list (v3). . . . 466
crosswalks version 1. . . . 466
Unlock a crosswalk (v1). . . . 466
Get crosswalk (v1). . . . 467
Get history of a crosswalk by time range (v1). . . . 468
Get value mappings for a code value (v1). . . . 471
Delete duplicate mappings of a crosswalk (v1). . . . 472
Get job details of a crosswalk cleanser job (v1). . . . 472
Synchronize a Data Quality dictionary with value mappings of a crosswalk 474
crosswalks version 2. . . . 475
Unlock a crosswalk (v2). . . . 475
Get crosswalk details (v2). . . . 475
Get value mappings for a code value (v2). . . . 477
Delete duplicate mappings of a crosswalk (v2). . . . 479
Get job details of a crosswalk cleanser job (v2). . . . 480
Get history of value mappings of a crosswalk by time range (v2). . . . 482
Delete value mappings of a crosswalk (v2). . . . 484
crosswalk version 3. . . . 487
Get history of a crosswalk by time range (v3). . . . 487
Get value mappings for a code value (v3). . . . 491
Delete value mappings of a crosswalk by mapped source and target code values (v3). . . . 493
hierarchies version 1. . . . 495
Get hierarchies (v1). . . . 495
Get hierarchy details (v1). . . . 496
Get hierarchy model relationships (v1). . . . 497
Get history of a hierarchy by time range (v1). . . . 499
Get history of hierarchy relationships by time range (v1). . . . 502
hierarchies version 2. . . . 505
Get hierarchy details (v2). . . . 505
Get hierarchy model relationships (v2). . . . 506
Get history of a hierarchy by time range (v2). . . . 509
Get history of hierarchy relationships by time range (v2). . . . 513
external localization. . . . 516
Export localization languages. . . . 517
Import localization languages. . . . 519
asset optimization modelling. . . . 522
Enable the asset optimization option. . . . 522
Get asset modeling job status. . . . 523
alias generation 525
Generate aliases for reference data assets 525

Get alias generation job status. . . . 526
enums. . . . 529
Get system reference data values. . . . 529
Add system reference data values. . . . 531
Update system reference data values. . . . 532
Delete system reference data values. . . . 534
Using REST APIs to import and export. . . . 534
REST APIs for import and export. . . . 535
Filter criteria. . . . 535
Importing code values. . . . 538
Importing value mappings. . . . 540
Exporting code values. . . . 541
Exporting filtered code values. . . . 542
Exporting value mappings. . . . 545
Using REST APIs to manage hierarchies. . . . 545
REST APIs to manage hierarchies. . . . 546
Managing hierarchies. . . . 546

Chapter 16: Glossary. . . . 551

Index. . . . 553

Preface

Use the MDM - Reference 360 help to learn how to create, manage, and govern reference data in Reference 360. Learn how to create reference data sets, code lists, and code values to organize reference data important to your business.

Informatica Resources

Informatica provides you with a range of product resources through the Informatica Network and other online portals. Use the resources to get the most from your Informatica products and solutions and to learn from other Informatica users and subject matter experts.

Informatica Documentation

Use the Informatica Documentation Portal to explore an extensive library of documentation for current and recent product releases. To explore the Documentation Portal, visit https://docs.informatica.com.

If you have questions, comments, or ideas about the product documentation, contact the Informatica Documentation team at infa_documentation@informatica.com.

IDMC web site

You can access the Informatica® Intelligent Data Management Cloud™ (IDMC) web site at http://www.informatica.com/cloud.

Intelligent Data Management Cloud Communities

Use the IDMC Community to discuss and resolve technical issues. You can also find technical tips, documentation updates, and answers to frequently asked questions.

To access the IDMC Community, see Master Data Management SaaS Community.

Developers can learn more and share tips at the Cloud Developer community:

https://network.informatica.com/community/informatica-network/products/cloud-integration/cloud-developers

IDMC Marketplace

Visit the Informatica Marketplace to try and buy Data Integration Connectors, templates, and mapplets:

https://marketplace.informatica.com/

Informatica Knowledge Base

Use the Informatica Knowledge Base to find product resources such as how-to articles, best practices, video tutorials, and answers to frequently asked questions.

To search the Knowledge Base, visit https://search.informatica.com. If you have questions, comments, or ideas about the Knowledge Base, contact the Informatica Knowledge Base team at KB_Feedback@informatica.com.

IDMC Trust Center

The IDMC Trust Center provides information about Informatica security, compliance, and privacy practices.

You can access the trust center at https://trust.informatica.com/.

Informatica Global Customer Support

You can contact a Global Support Center through the Informatica Network or by telephone.

To find online support resources on the Informatica Network, click **Contact Support** in the IDMC Help menu to go to the **Cloud Support** page. The **Cloud Support** page includes system status information and community discussions. Log in to Informatica Network and click **Need Help** to find additional resources and to contact Informatica Global Customer Support through email.

The telephone numbers for Informatica Global Customer Support are available from the Informatica web site at https://www.informatica.com/services-and-training/support-services/contact-us.html.

CHAPTER 1

Introducing Reference 360

Informatica® Reference 360 is a cloud-based service that organizations use to create, manage, and govern reference data. Reference data is a subset of master data that is used throughout your organization. Common types of reference data are countries, currency codes, and cost centers.

Applications in your organization typically refer to the same reference data by different code values. You can manage the different code value representations by creating crosswalks. Crosswalks contain value mappings between code values in one code list and code values in another code list. Crosswalks allow you to translate between the different code value representations used by each application.

When you manage your reference data, you see improvements in the quality of reporting, compliance with regulations, and the trustworthiness of reference data. It also helps reduce operational overhead, such as additional manual effort and QA effort to resolve issues with your reference data.

Reference 360 provides the following functionality:

- Create, manage, and govern reference data
- Create crosswalks to translate how different applications represent the same business term
- Approve changes to assets before they become part of the published reference data
- View historical information about your reference data assets
- Localization of user interface labels and error messages to Japanese, French, German, Spanish, and Brazilian Portuguese.
- Enhanced accessibility for all users. Our Accessibility Conformance Report (ACR) based on Voluntary Product Accessibility Template (VPAT) outlines the accessibility conformance in detail. To download the VPAT for Reference 360, visit Accessibility Compliance.

Key concepts

System reference data

System reference data is a group of values that provide information about your reference data. New organizations don't contain system reference data values.

You can add the values that you want to appear as system reference data. Users can then apply the values to their reference data sets, code lists, code values, and crosswalks. For example, you might add Low, Medium, High, and Critical as values to the Priority system reference data.

You can add values to the following system reference data:

- Application. Application that uses an asset, such as Customer 360 and Enterprise.
- Confidentiality. Confidentiality level of an asset, such as confidential, internal, restricted, and public.
- Domain. Area or grouping to describe an asset, such as finance, geography, and social.
- Priority. Priority of an asset, such as critical, high, medium, and low.
- Status. State of an asset in the life cycle, such as active, inactive, draft, merged, and open.

Related Topics:

- "Manage system reference data" on page 100

Reference data sets

All reference data in Reference 360 is categorized under reference data sets. A reference data set is a logical grouping of reference data, such as country codes, currency codes, or cost centers.

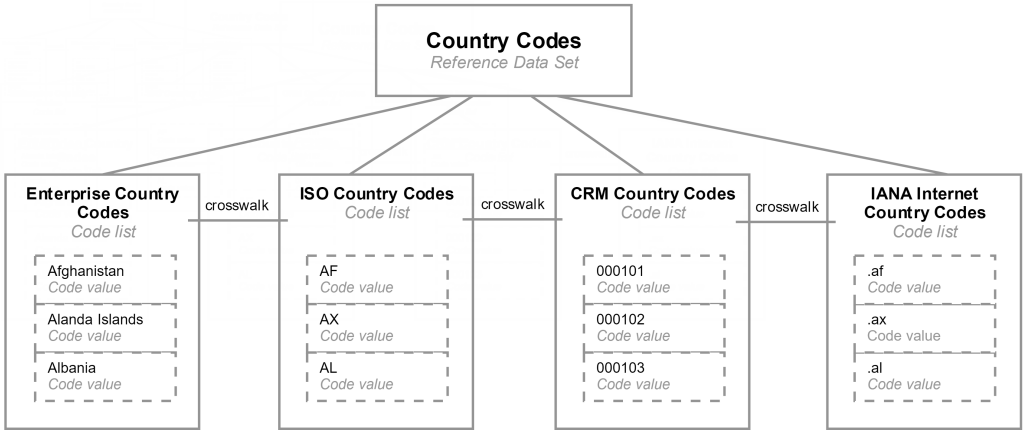
Reference data sets contain code lists. Reference data sets act as both a container and template for the code lists.

When you create a reference data set, you configure its structure definition and attributes. You cannot change the structure definition and attributes of the reference data set after creation. When you add code lists to a reference data set, the code lists inherit the structure definition and attributes from the reference data set. If you do not define the structure definition of a reference data set, you can still define the structure definition of the code lists.

The following table lists the actions you can perform on a reference data set after creation:

Action	Allowed
Delete the reference data set	Yes
Modify the general properties	Yes
Modify the structure definition	No
Modify existing attributes	No
Create additional attributes	Yes
Create required attributes	Yes
Delete attributes	Yes
Modify the display settings	Yes
Modify the stakeholders	Yes

For example, you might have three different applications, each with a slightly different list of country codes. In Reference 360, you create a Country Codes reference data set. Then you create three code lists in the Country Codes reference data set, one for each of the applications that contain country code values. The last step is to create crosswalks between the code lists so that you can translate how each application represents the same code value.



Related Topics:

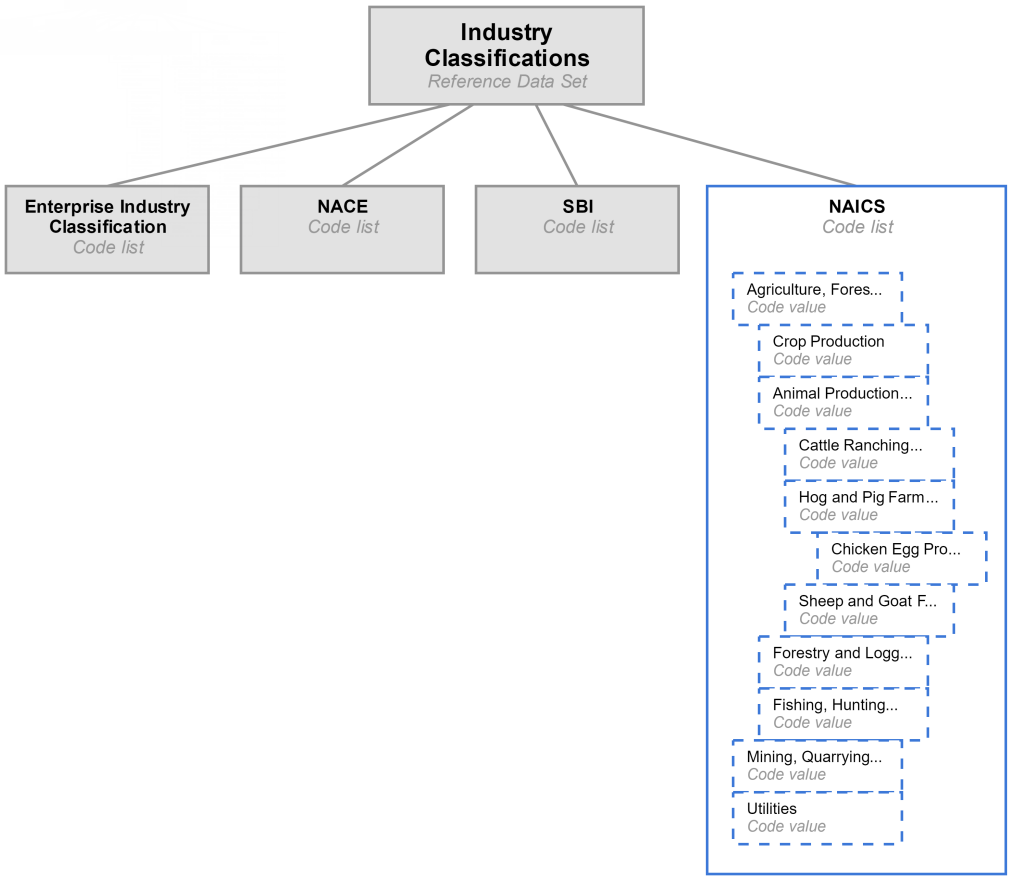
- "Creating reference data sets" on page 102

Hierarchical reference data sets

A hierarchical reference data set allows users to model hierarchical data structures. When you create a code list in a hierarchical reference data set, the code list inherits the hierarchical structure definition. In the code list, you can arrange the code values into levels to create hierarchies. For example, you might create a hierarchical reference data set for industry classification systems or cost centers.

After you create a hierarchical reference data set, you cannot change the structure definition. If you create a code list in a hierarchical reference data set, the code list must use the inherited structure definition.

For example, you want to create an Industry Classifications reference data set. When you create the set, you define the structure definition as hierarchical because you know that industry classifications contain hierarchical code values. In the reference data set, you create an Enterprise Industry Classification code list, a NACE code list, a SBI code list, and a NAICS code list. In the NAICS code list, you arrange the code values into a hierarchy to reflect the hierarchical structure of this externally mandated list.



Related Topics:

- "Hierarchical code lists" on page 21

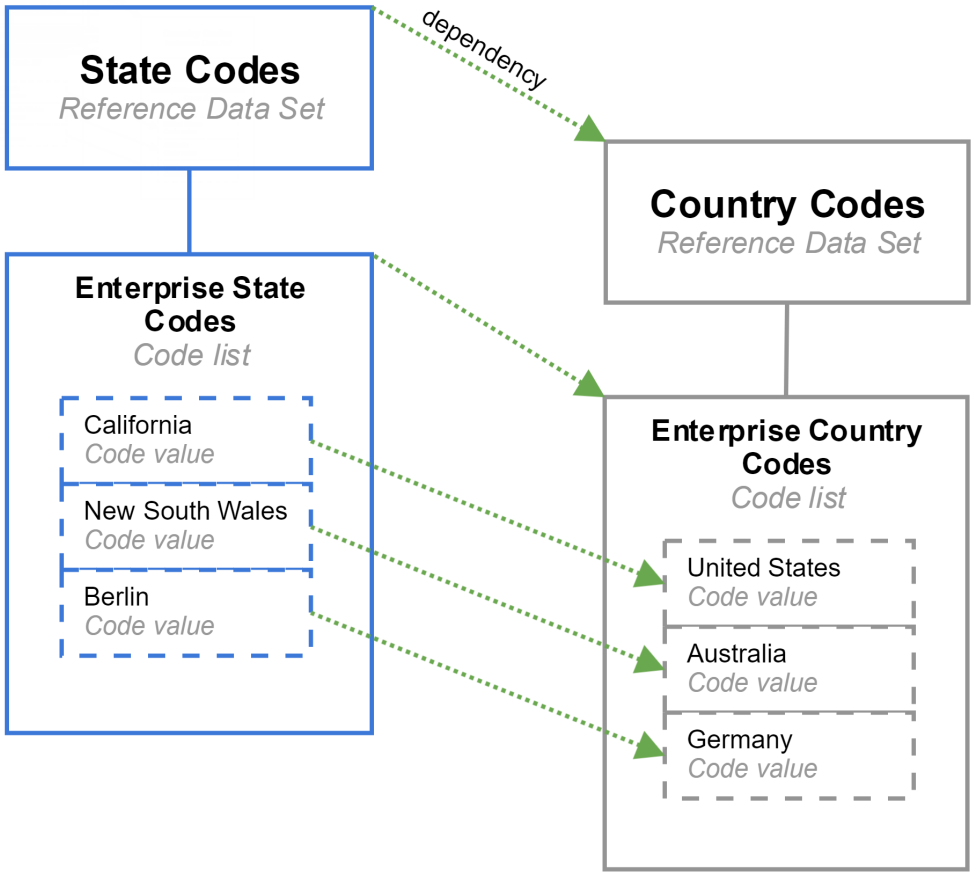
Dependent reference data sets

A dependent reference data set depends on another reference data set and passes down its dependency to its code lists.

When you configure a reference data set as dependent, the reference data set that it depends on is referred to as the parent dependency. Reference data sets can have only one parent dependency. When you create code lists in a dependent reference data set, the code lists inherit the same parent dependency.

After you create a dependent reference data set, you cannot change the structure definition. If you create a code list in a dependent reference data set, the code list must use the inherited structure definition.

For example, you have a Country Codes reference data set. When you create a State Codes reference data set, you define the structure definition as dependent and configure the Country Codes reference data set as the parent dependency. Then when you create code lists in the State Codes reference data set, the code lists inherit the dependency on the Country Codes reference data set. For each code list in the State Codes reference data set, you can choose the code list dependency. A code list dependency defines the dependent relationship between one code list and another. You might want the Enterprise State Codes code list to depend on the Enterprise Country Codes code list.



Related Topics:

- "Dependent code lists" on page 22

Code lists

Code lists are containers for code values. The code values in a code list reflect the code values that are used in a source application. Code lists from different applications might use different code values for the same business term.

A code list inherits the structure definition and attributes from the reference data set. If a reference data set does not have a structure definition, you can configure the structure definition of the code list.

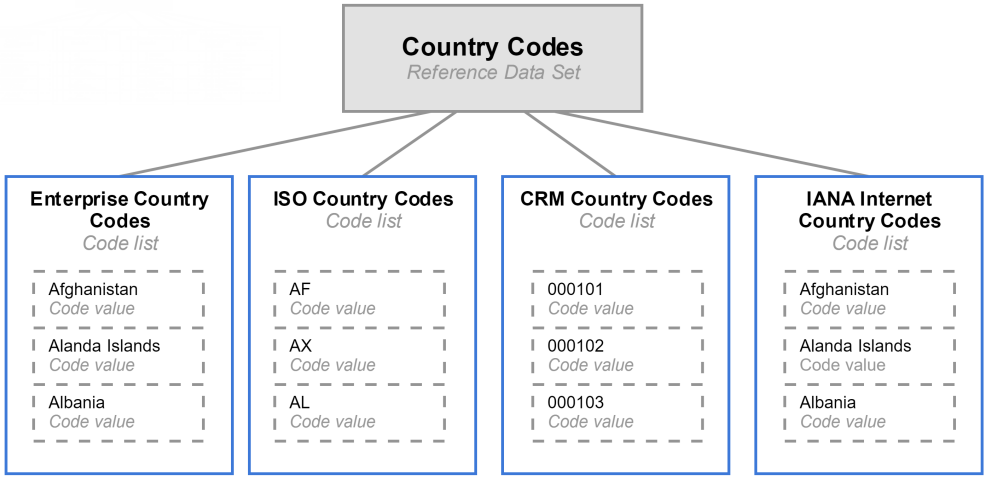
You can provide additional details for a code list, such as additional description, external URLs, and notification recipients.

The following table lists the actions you can perform on a code list after creation:

Action	Allowed
Delete the code list	Yes
Modify the general properties	Yes

Action	Allowed
Add external URLs	Yes
Modify external URLs	Yes
Delete external URLs	Yes
Add notification recipients	Yes
Modify notification recipients	Yes
Delete notification recipients	Yes
Modify the inherited structure definition	No
Modify the inherited attributes	No
Create additional attributes	Yes
Create required attributes	Yes
Delete attributes	Yes
Modify the display settings	Yes
Modify the stakeholders	Yes

For example, you might have a Country Codes reference data set to categorize all the country codes used in your organization. This reference data set might contain an Enterprise Country Codes code list for country code values that your organization uses for enterprise reporting. You might also have code lists for industry standard lists of values, such as ISO Country Codes and IANA Internet Country Codes. You might also have code lists for systems that store similar code values, such as country codes in your CRM system.



Related Topics:

- "Creating code lists" on page 109
- "Crosswalks" on page 23

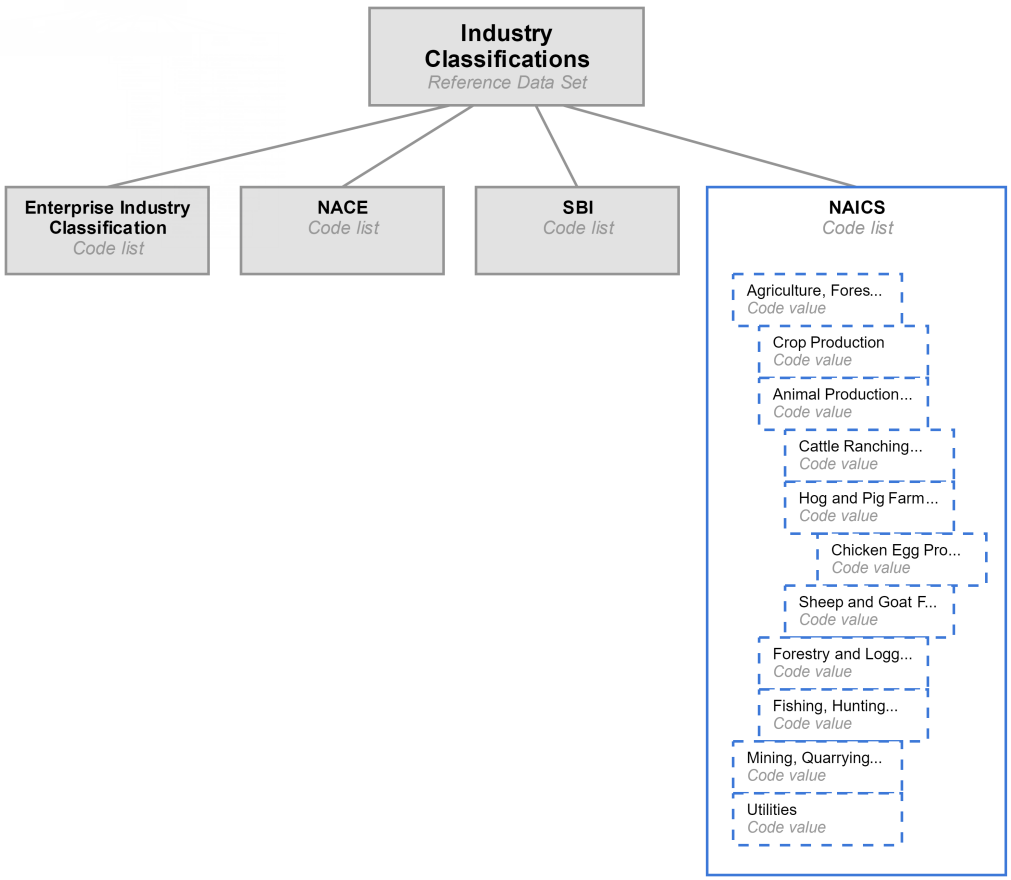
Hierarchical code lists

A hierarchical code list supports hierarchical data structures. You can arrange its code values into levels to create hierarchies.

If you create a code list in a hierarchical reference data set, the code list inherits the hierarchical structure definition from the reference data set. If you create a code list in a reference data set without a hierarchical structure definition, you can configure the structure definition of the code list as hierarchical. After you create a hierarchical code list, you cannot change its structure definition.

For more information about actions that are allowed after code list creation, see "Code lists" on page 19.

For example, you have an Industry Classification reference data set that does not support hierarchical data structures. If you need to create a NAICS code list in the reference data set that supports hierarchies, you configure the structure definition of the code list as hierarchical.



Related Topics:

- "Hierarchical reference data sets" on page 17

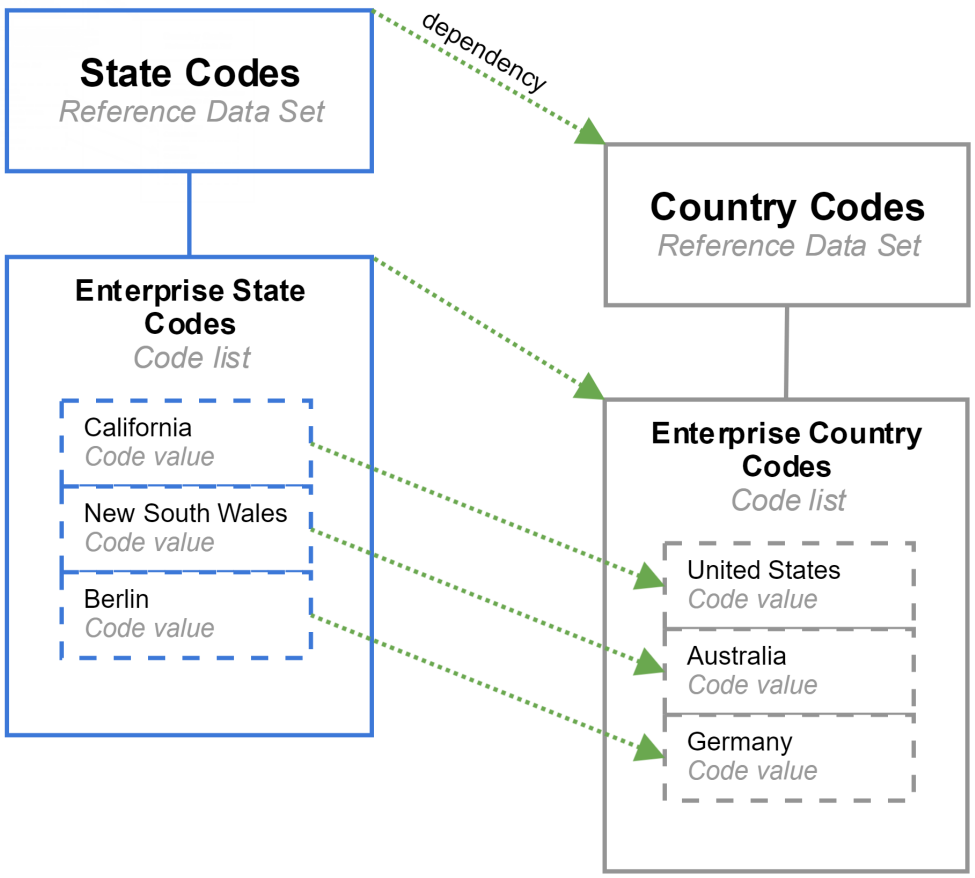
Dependent code lists

A dependent code list contains code values that depend on code values in another code list. The dependency exists between code lists that belong to different reference data sets.

If you create a code list in a dependent reference data set, the code list inherits the parent dependency from the dependent reference data set. If you create a code list in a reference data set without a dependent structure definition, you can define the code list as dependent. After you create a dependent code list, you cannot change the dependency.

For more information about actions that are allowed after code list creation, see "Code lists" on page 19.

For example, you have a Country Codes reference data set. When you create a State Codes reference data set, you define the structure definition as dependent and configure the Country Codes reference data set as the parent dependency. Then when you create code lists in the State Codes reference data set, the code lists inherit the dependency on the Country Codes reference data set. For each code list in the State Codes reference data set, you can choose the code list dependency. A code list dependency defines the dependent relationship between one code list and another. You might want the Enterprise State Codes code list to depend on the Enterprise Country Codes code list.



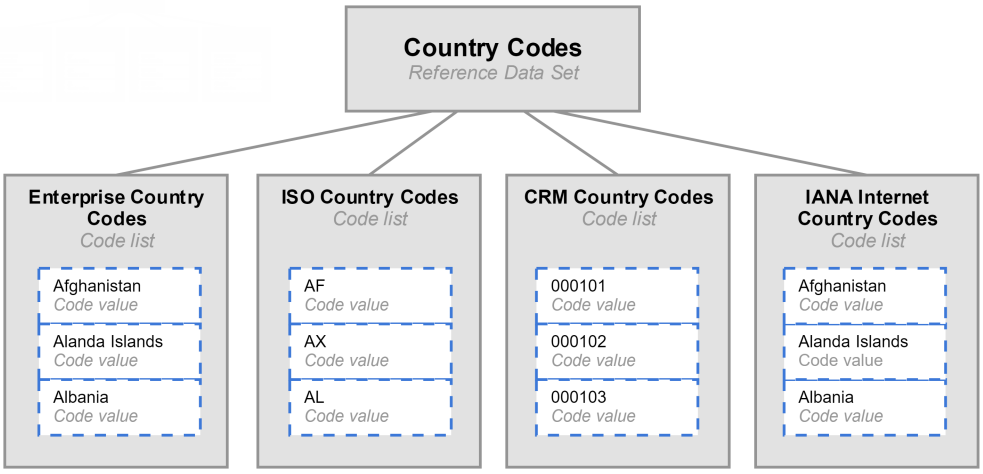
Related Topics:

- "Dependent reference data sets" on page 18

Code values

A code value is a unique value, such as a business term, code, or lookup value. Code values are organized into code lists. Each code list contains code values from a singular source application or industry standard list.

For example, you might have a Country Codes reference data set, and within it is an Enterprise Country code list. In the Enterprise Country code list, you import enterprise country codes values that your organization uses for enterprise reporting. The code list includes code values such as Afghanistan, Aland Islands, and Albania. You might also have a ISO Country Codes code list, a CRM Country Codes code list, and a IANA Internet Country Codes code list. Each of these code lists contain their own code values.



Code values consist of attributes. Each code value consists of a Name attribute and a Code attribute. For example, you might create a code value for the US country code. You define the Name attribute as US and the Code attribute as 001. For more information, see "Attributes" on page 26.

Related Topics:

- "Manage code values" on page 123

Crosswalks

A crosswalk is a visual representation of a one-way relationship between code values in a pair of code lists. A reference data set can contain many code lists, and each code list contains a variation of the same type of code values. Crosswalks provide a way to translate between the different variations each code list uses.

You can create crosswalks for code lists that belong to the same reference data set or different reference data sets. When you create a crosswalk, you configure a source code list and a target code list. You create one-way value mappings between code values in the source code list and code values in the target code list. Value mappings provide a way to translate the code values in the source code list to code values in the target code list. The crosswalk and the value mappings are associated with the source code list.

If code values in a pair of code lists are equivalent, you must create two crosswalks. When you create the crosswalks, use the same pair of code lists in reversed order as the source and target code lists. You must

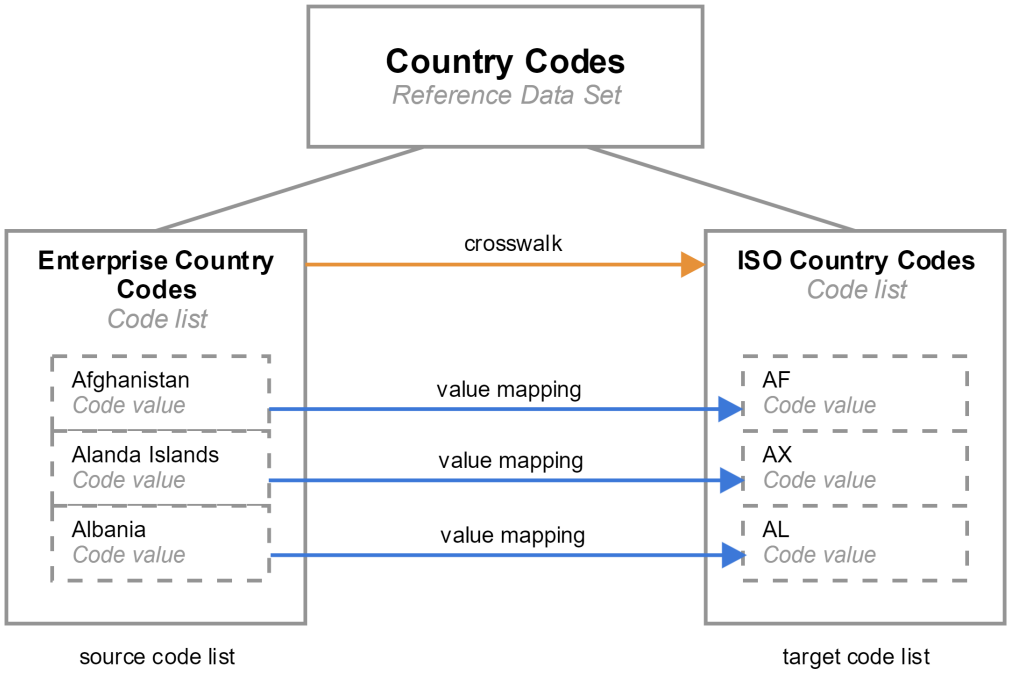
create two crosswalks because each code list can be used as a source code list and crosswalks are stored with the source code list. You cannot create multiple crosswalks for the same source and target code lists.

For example, your organization might have a Country Codes reference data set with code lists for enterprise country codes and ISO country codes. You might create the following crosswalks to map code values in the code lists:

1. Create a crosswalk with the Enterprise Country Codes code list as the source code list and the ISO Country Codes code list as the target code list.
2. Map the code value "Afghanistan" in the Enterprise Country Codes code list to the code value "AF" in the ISO Country Codes code list. The value mapping shows that the code value "Afghanistan" can be translated to the "AF" code value.
3. Create a crosswalk with the ISO Country Codes code list as the source code list and the Enterprise Country Codes code list as the target code list.
4. Map the code value "AF" in the ISO Country Codes code list to the code value "Afghanistan" in the Enterprise Country Codes code list. The value mapping shows that the code value "AF" can be translated to the "Afghanistan" code value.

After you create a crosswalk, the crosswalk is associated to the source code list as an outgoing crosswalk and to the target code list as an incoming crosswalk. Use the **Explore** panel to view code lists and the associated outgoing and incoming crosswalks. An outgoing crosswalk appears below the source code list and shows the source code values that map to the target code values. An incoming crosswalk appears below the target code list and shows the source code values that map to the target code values.

The following diagram shows a sample crosswalk:



Related Topics:

- "Manage crosswalks" on page 135
- "Code lists" on page 19

Hierarchies

After you create reference data sets, code lists, and code values, you can create hierarchies. A hierarchy shows how code values are related to one another. You can arrange code values above, below, or at the same level as other code values.

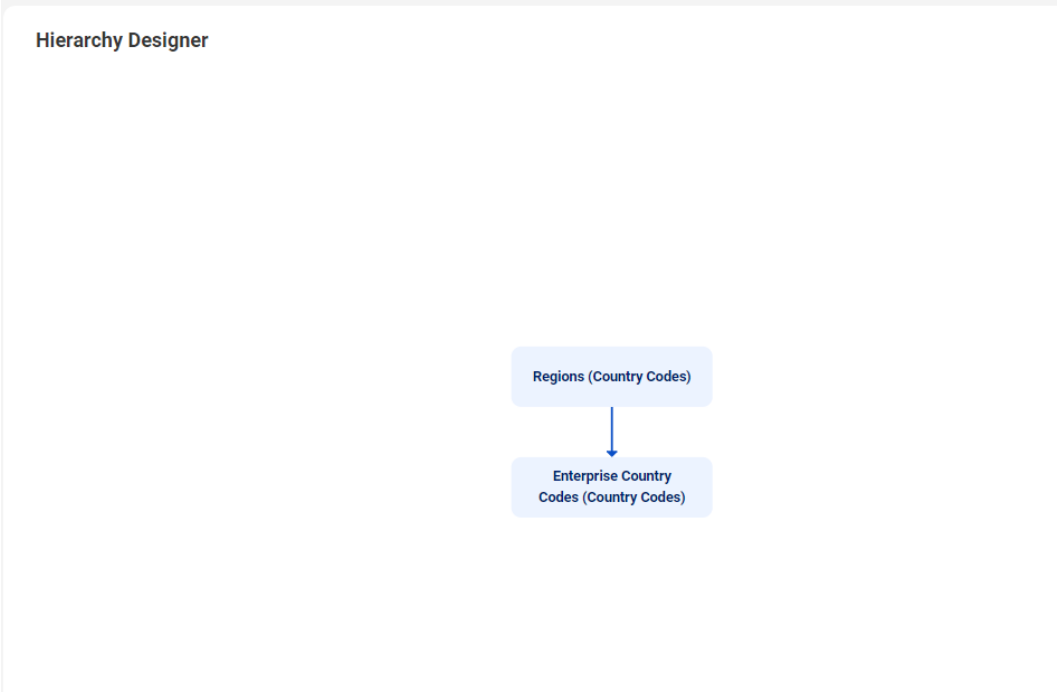
Code values in a hierarchy are top-level nodes or child nodes. Each hierarchy must contain at least one code value as the top-level node. Top-level nodes are code values at the highest level in a hierarchy and don't have any code values above them. Child nodes are arranged below other code values and can have other code values below them.

The hierarchy relationships that you can create depend on the hierarchy model. A hierarchy model consists of hierarchical relationships between code lists. You define a code list as the top-level code list and then define levels by adding code lists and relationships between the code lists. Then based on the hierarchy model, you can add code values as nodes in the hierarchy.

For example, your organization needs to track country codes by regions. You might create the following hierarchy model:

1. Create the Locations hierarchy asset.
2. Add the Regions code list as the top-level node.
3. Create a relationship from the Regions code list to the Enterprise Country Codes code list.

The following image shows a sample hierarchy model:



With the hierarchy model defined, you might create the following hierarchy:

1. Add the North America code value as the top-level code list.
2. Add the USA code value as a child node of the North America code value.
3. Add the Canada code value as a child node of the North America code value.

The following image shows a sample hierarchy:



The relationship between the code values in a hierarchy are termed as related code values.

For example, in the preceding sample hierarchy, the Regions code list contains two code values, such as North America and South America; and the Enterprise Country Codes code list contains two code values, such as Canada and USA. In the hierarchy model, the hierarchical relationship connects the two code lists, and the code values in the parent and child code lists are related to each other.

Note: You must be assigned the Planner role to work with hierarchies. For more information, see "Users, groups, and roles" on page 50.

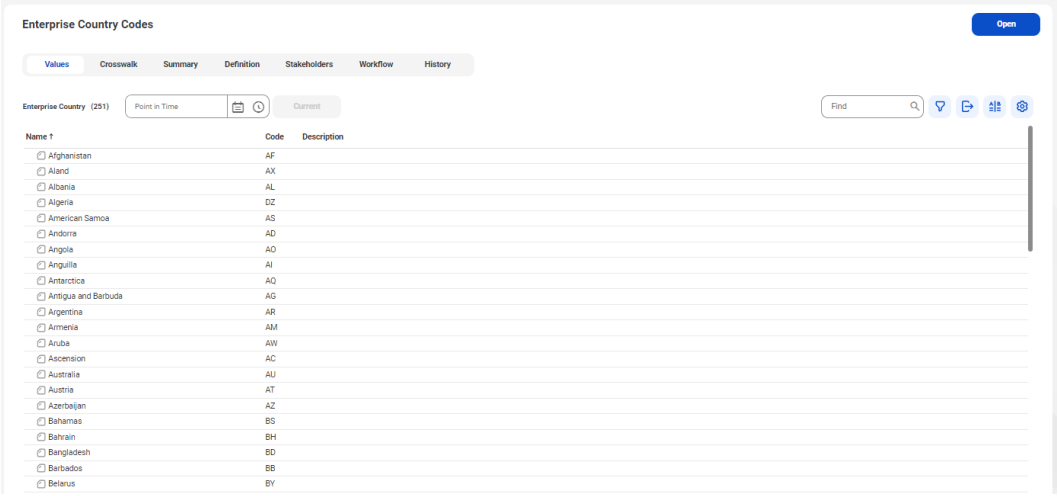
Attributes

An attribute is a component of a code value. When you create reference data sets and code lists, you define attributes that code values must consist of. Then when you add code values, you enter a value for each attribute.

By default, the attributes defined for all code values are the Name, Code, and Description attributes. The Name and Code attributes are required attributes, so you cannot delete these attributes. For each code value, you must enter a value for these attributes. The Description attribute is not required, so you can delete the Description attribute if you do not need it.

For example, you might have a code list for Enterprise Country Codes. When you add a code value for the US country code, you enter the `United States of America` value in the Name attribute. You enter the `US` value in the Code attribute. This means that the US country code value consists of the `United States of America` and the `US` attribute values.

The following image shows the Enterprise Country Codes code list with values in the Name attribute and Code attribute for each code value:



You can set default values for predefined and custom attributes in a code list. When you set a default value, you don't have to add the same set of values multiple times.

For example, if you have to add the office location for all the employees in the Employee Details code list, you can add the office location as a default value for the Location attribute. When you add code values, the Location attribute displays the default value.

You can reorder the attributes in a code list to change the sequence in which they appear.

You can associate code list attributes with Data Governance and Catalog assets, such as business terms, metrics, technical data elements, and manual data elements. This association helps eliminate redundant business terms, standardize terminology, and improve data accuracy across services.

For more information about associating code list attributes with Data Governance and Catalog assets, see "Associate code list attributes with Data Governance and Catalog assets" on page 119.

Custom attributes

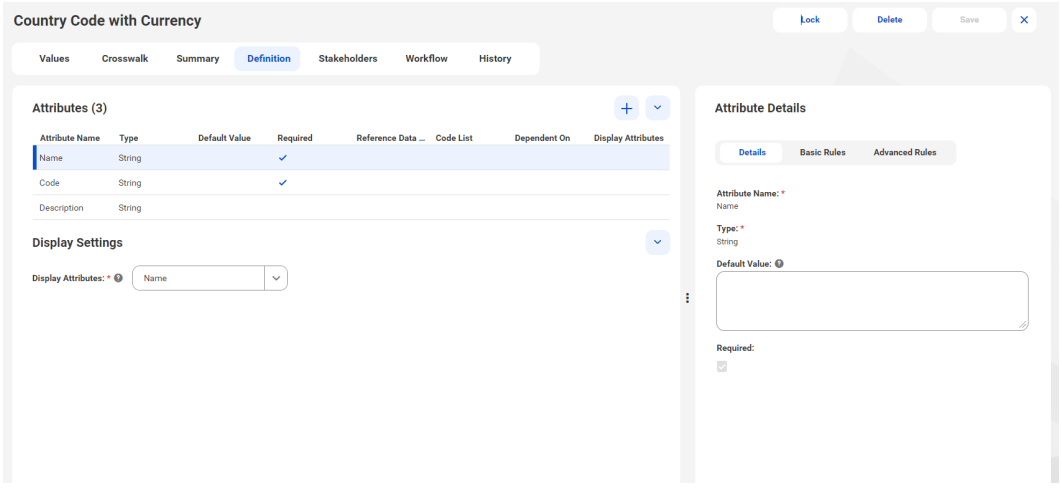
You can define additional attributes for your code values. Your custom attributes can be required or optional attributes.

When you create a reference data set or code list, you can create custom attributes and define them as required or optional. If you define custom required attributes for reference data sets, the code lists inherit the attributes from the reference data set. Then when you create code values in the code lists, the code values must contain values in the custom required attributes. Also, when you create code lists, you can configure custom required attributes in addition to the inherited attributes from the reference data set.

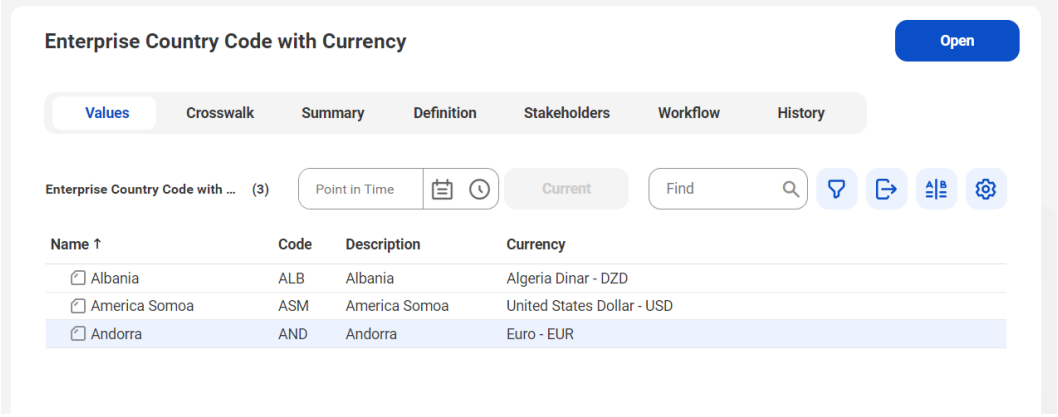
Note: You can only define custom required attributes when you create a reference data set or code list. For more information about allowed actions, see "Reference data sets" on page 16 and "Code lists" on page 19.

For example, you create a Country Codes with Currency code list. You create the optional Population and Currency custom attributes. When you create code values in the Country Codes with Currency code list, you must enter values in the required Name and Code attributes. You can choose to enter values for the Population and Currency attributes.

The following image shows a sample of attributes defined for a code list:



The following image shows sample attribute values for code values in the Country Code with Currency code list:



Attribute data types

When you configure custom attributes, you can define data types, such as string, integer, decimal, date, boolean, and reference data for the attribute values.

You can define a scale for decimal data type attributes. The default scale is `4`.

Note: When you migrate a code list that contains a decimal attribute, the scale doesn't get updated in the target organization. To update the scale, delete the decimal attribute from the target organization and then import the code list.

When you add an integer data type, the minimum value is `-2147483648`, and the maximum value is `2147483647`. When you define reference data as the data type, the values depend on the referenced reference data asset.

The Reference Data data type supports lookup reference data. Use the Reference Data data type to include the code values from another reference data set and code list. For more information, see "Reference data attributes" on page 28.

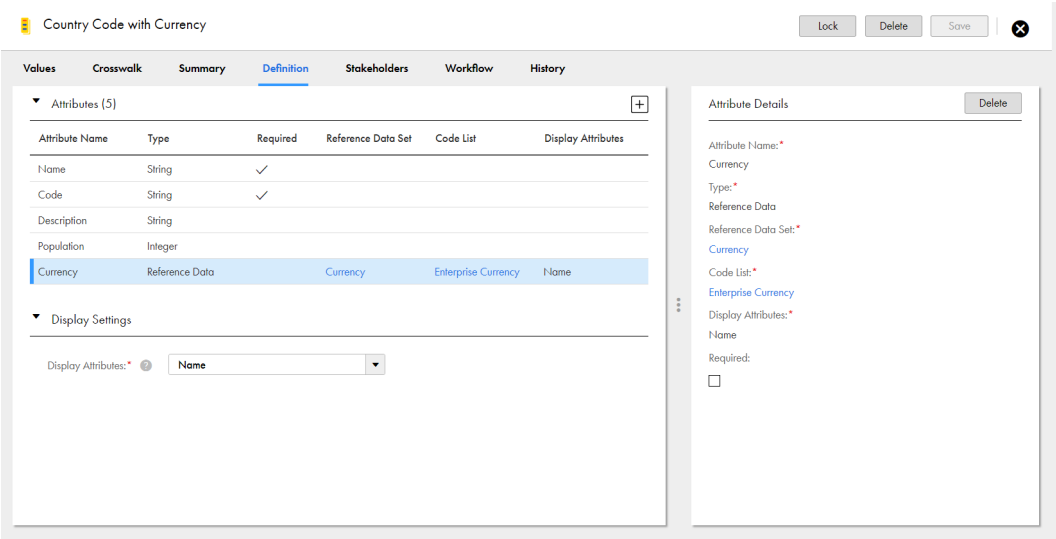
Reference data attributes

You can create custom attributes that support reference data from other assets. These reference data attributes allow you to use code values in other assets as an attribute value for code values in the code list.

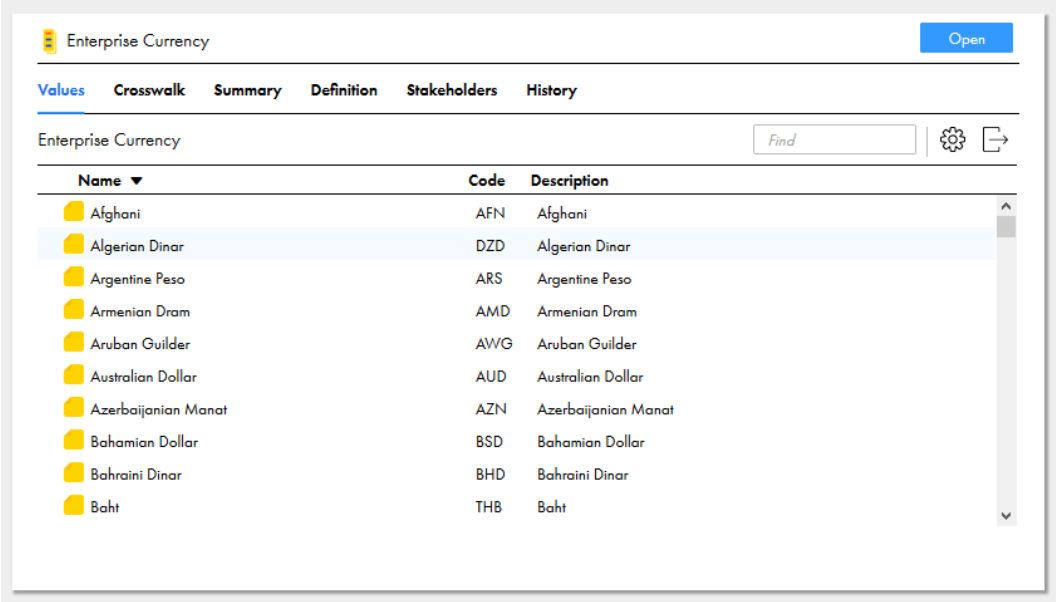
When you create reference data attributes, you configure the reference data set and code list that you want to use reference data from. You also specify the attributes that you want to appear as the display attributes to represent the code values from the selected code list and reference data set.

If you don't specify a reference data attribute as a required attribute, you can leave the reference data attribute empty when you add code values.

The following image shows the Currency attribute configured as a data type - Reference Data:

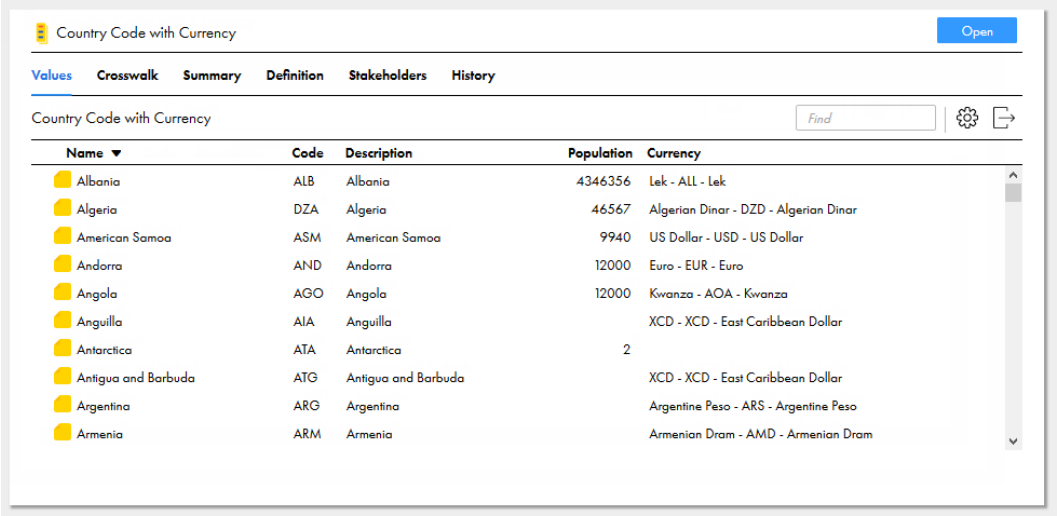


The following image shows an example of the code values in the Enterprise Currency code list that are used as reference data attributes:



When you add code values to the Country Codes with Currency code list, you can select reference data in the Enterprise Currency code list to use as the Currency attribute value.

The following image shows the `Euro - EUR - Euro` code value from the Enterprise Currency code list used as the Currency attribute value of the `Andorra` code value:



When the reference data set and code list of a reference data attribute depend on the code list of existing reference data attributes, you can set dependency between them.

For example, to manage the employee details, your organization has added the Employee Details code list. The details include the birth and work locations of the employees.

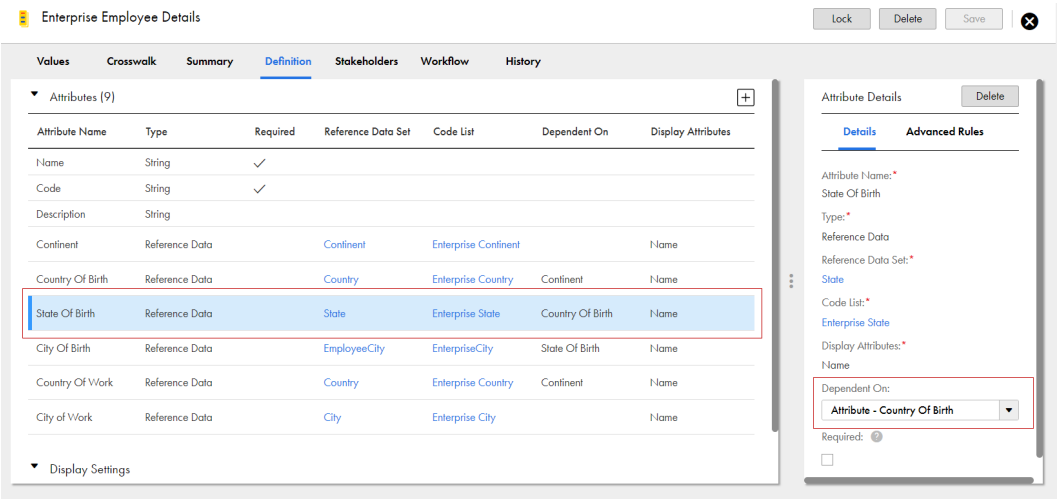
If John was born in Ontario and works in Bavaria, you can add reference data attributes, such as Continent of Birth, Country of Birth, State of Birth. To add the details of the work location, you can add similar reference data attributes.

Your organization has defined reference data sets, such as Continent, Country, and State. The State reference data set depends on the Country reference data set, and the Country reference data set depends on the Continent reference data set.

After you add the Continent of Birth reference data attribute, you can add the Country of Birth reference data attribute. When you add the Country of Birth reference data attribute in the Employee Details code list, you can view its dependency with the Continent of Birth reference data attribute. You can select the Continent of Birth on the **Dependent On** field.

Note: You can set up to five levels of dependency for reference data attributes.

The following image shows the dependency set for the reference data attributes in the Employee Details code list:



When you add the employee details for John, if you select North America as the Continent of Birth, the Country of Birth reference data attribute displays only the countries added to North America.

If you select Europe as the Continent of Work, the Country of Work reference data attribute displays only the countries in Europe.

You can view the values because there's an existing dependency between the code values in the code list.

Note: You can't enable a reference data attribute as required if the reference data attribute that it depends on isn't required.

Related Topics:

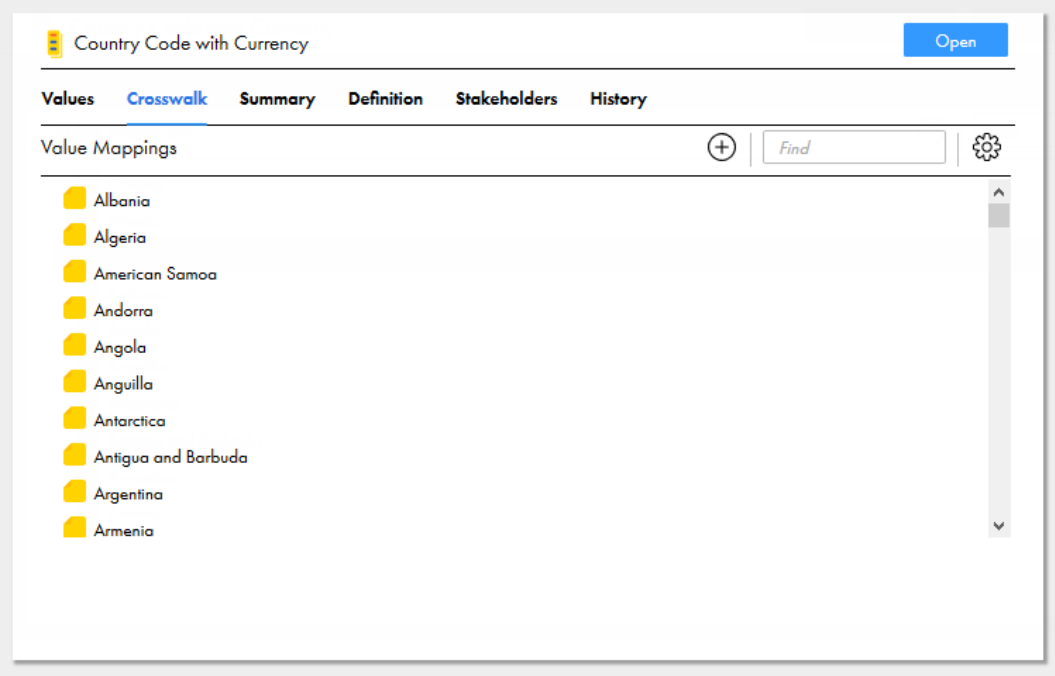
- "Exporting filtered code values" on page 542

Display attributes

Display attributes are the attributes that represent the code values in the asset throughout Reference 360. You configure display attributes for reference data sets and code lists. You can configure multiple attributes as display attributes.

For example, code values in the Country Code with Currency code list might consist of the Name, Code, Description, Population and Currency attributes. You can configure the Name attribute as the display attribute. Later when you create a crosswalk to map code values in the code list, the attribute values in the Name attribute appear for you to map.

The following image shows an example of the values in the Name attribute that represent the code values in the code list:



Dictionaries

Dictionaries are Data Quality assets that are used to standardize and validate data values.

You can create dictionaries for Data Quality based on attributes of code lists or value mappings of crosswalks. After you create a dictionary in Reference 360, the dictionary appears in Data Quality. You must

then synchronize data to populate the dictionaries in Data Quality with code values or value mappings. You can view these dictionaries in Data Quality but can edit them only in Reference 360.

For more information about dictionaries, see *Dictionaries* in the Data Quality help.

Creating dictionaries in Reference 360 eliminates duplication of reference data in Data Quality and ensures that Reference 360 serves as the single, trusted source of standardized reference data for dictionaries.

For example, if the ISO Country Codes code list in Reference 360 contains ISO country codes, you can create dictionaries in the code list and use the codes to standardize data values in Data Quality.

Business IDs

Business IDs are unique sequential values for code values in a code list. You can use business IDs as identifiers in applications that use Reference 360.

When you create reference data sets or code lists, you can enable business IDs in code lists. You can configure business IDs in numeric or alphanumeric formats. When you add code values to a code list, Reference 360 generates business IDs sequentially for the code values in the specified format. You can't update the business ID configuration after you save the code list.

You can select one of the following formats for business IDs:

- Alphanumeric. Use 10 to 24 characters, starting with a prefix that consists of three characters. The prefix can contain uppercase letters and numbers. For example, RDM4AB78W2.
- Numeric. Use 10 to 24 numbers including a prefix of three digits. For example, 9994567123. You can enter an offset value. The business ID of code values created by Reference 360 starts at the offset value, excluding the prefix. For example, with a length of 10, an offset value of 1000, and a prefix of 999, the starting business ID is 9990001000. Reference 360 imports code values with the prefix 999 if their business IDs are lower than the offset value and rejects them if they are equal to or greater than it.

If you enable business IDs for code lists, you can import code values with their business IDs from a CSV file. You can also export code values with their business IDs to a CSV or JSON file.

Batch Jobs

You can define, execute, schedule, and monitor batch jobs in Reference 360.

The batch jobs use the taskflows defined in Cloud Data Integration. Each taskflow can run multiple mapping tasks in a sequential order or in parallel. Each mapping task includes a mapping that maps the source fields with the fields of the Reference 360 assets. To connect to the Reference 360 assets, use the Business 360 connector, and create a connection in Administrator.

For example, if you want to import enterprise country codes from a flat file, create a mapping with a flat file connection as the source transformation and the Business 360 connection as the target transformation. You can then define field mappings between the source and target fields, create a mapping task that uses the mapping, and then create a taskflow and run the taskflow. For more information, see Taskflows in the Data Integration help.

Use Administrator to create a connection for the source transformation and a Business 360 connection for the target transformation. For more information about the Business 360 Connector, see Business 360 Connector in the Data Integration help.

Workflows

When you propose changes to a code list, a crosswalk, or a hierarchy, you can send the changes for approval through an approval workflow. An approval workflow consists of one or more linked approval tasks to review and approve your changes.

To propose changes, you can lock the code list, crosswalk or hierarchy. When you lock the code list, the crosswalk or the hierarchy, a draft version is created and you prevent other users from making changes. You can import, create, update, or delete code values in a code list, value mappings in a crosswalk, and create, update, or delete parent or child code values in a hierarchy. Your edits remain in the draft, and you can choose to continue editing later. The presence of a lock icon beside the code list, the crosswalk, or the hierarchy indicates to other users that you are working on it.

After you finish editing the code list, the crosswalk or the hierarchy, you can send your draft for approval. This action triggers an approval workflow and generates an approval task. Users responsible for approving the task receive notifications. As the code list, the crosswalk or the hierarchy progresses through the approval workflow, it remains locked.

Note: To use workflows, you must add values to the Priority system reference data. For more information, see "Adding values to system reference data" on page 100 or "Add system reference data values" on page 531.

Users assigned the following roles can send their proposed changes for approval or directly publish their changes without approval:

- Reference 360 Primary Owner role and the Primary Owner stakeholder role for a code list.
- Reference 360 Business Steward role and the Business Steward stakeholder role for a crosswalk.
- Reference 360 Planner role and the Planner stakeholder role for a hierarchy.

For more information about roles, see "Users, groups, and roles" on page 50

Related Topics:

- "Manage workflows" on page 176

Notifications

When users send an approval request, cancel an approval request, or perform a task action, the users responsible for the asset receive a notification. The notification alerts users that a change requires their review, a review is no longer required, or that a task action was performed by another user. Notifications appear in the user's inbox and in an email notification.

The following table lists which users receive inbox notifications following task events:

Event	Requester	Assignees
User assigns a task	-	Yes
User approves, rejects, or sends back a task	Yes	-

The following table describes the email notifications users receive following task events:

Event	Requester	Assignees
User sends approval request	Notification confirms that they successfully sent the approval request.	Notification of approval request that requires their review.
User cancels approval request	Notification confirms that they successfully canceled the approval request.	Notification of canceled approval request that no longer requires their review.
User approves, rejects, or sends back a task	Notification of the action performed on the approval request.	The assignee who performed the action receives a notification that confirms their action. All other assignees receive a notification that another user performed an action on the approval request.

Tasks

Tasks are requests to review and approve changes to code lists, crosswalks, and hierarchies. A task is created when a user sends their draft code list, crosswalk, or hierarchy for approval. Tasks are assigned to users who are responsible for reviewing and approving changes to the code list, crosswalk, or hierarchy. A task helps to manage changes to the code list, crosswalk, or hierarchy.

Use the **Workflow Inbox** tab to view and take actions on tasks. You can also send your draft code lists, crosswalks, or hierarchies for approval.

The **Workflow Inbox** tab consists of a **Task** panel and a **Task Details** panel. Use the **Task** panel to view all tasks available to you. In the **Task Details** panel, view the details of a task that you selected, compare the proposed changes against the active version, and take action on the task.

You can view the following task details:

- Assignee
- Submit date
- Priority
- Due date
- Comments

Versions

As an asset moves through a workflow, different versions of the asset are created. The version that is available depends on where the asset is in the workflow.

When you view an asset, you are viewing the active version of the asset. To propose changes to an asset, you create a draft version of the asset. When you are done making changes, you send your draft for approval. If approved, the draft version becomes the active version. If rejected, the draft version is discarded and the active version of the asset is available.

When you view an asset that was sent for approval, you can see the changes pending approval. You can compare the pending changes with the active version. If you are the last reviewer responsible for approving the draft, you can approve the changes while viewing the pending changes.

The following table describes the available versions for an asset:

Version	Description	Available to
Active	Active version of the asset that contains approved reference data.	All users viewing an asset without proposed changes.
Draft	Draft version of the asset that contains proposed changes.	Users proposing changes to an asset.

If you are a reviewer, you can compare the draft version and active version to see moved, deleted, and added reference data. For more information, see "Comparisons" on page 35.

Comparisons

You can compare the draft version of a code list or a crosswalk with the active version to compare proposed changes with the active version. You can also compare code lists and hierarchies to identify similarities and differences.

Compare changes

You can compare the draft version of a code list or a crosswalk with the active version. You can view the changes to code values and value mappings and evaluate whether the changes are correct. You can compare versions when you work on a draft, send a draft for approval, or review an approval task.

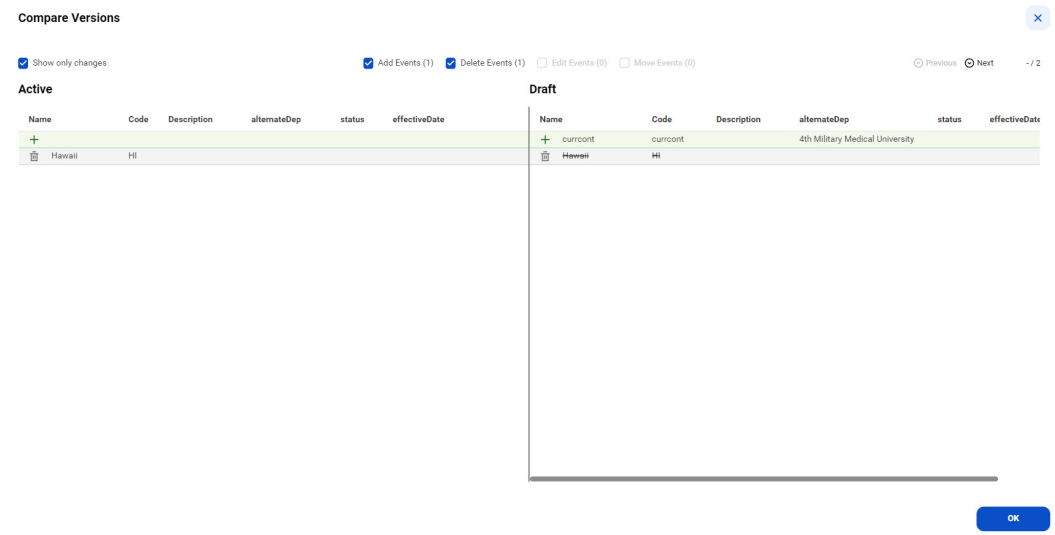
The **Compare Versions** view displays both unchanged and changed code values in a code list and value mappings in a crosswalk. The **Compare Versions** view shows changes to the language-specific values of localized attributes. When you edit or review a draft, you might want to view only changed code values or value mappings. You can also filter to view added, edited, moved, and deleted code values or value mappings only.

When you compare moved code values, you can navigate to the new location of the moved code value. You can navigate back to the original location to evaluate whether the move to the new location is correct.

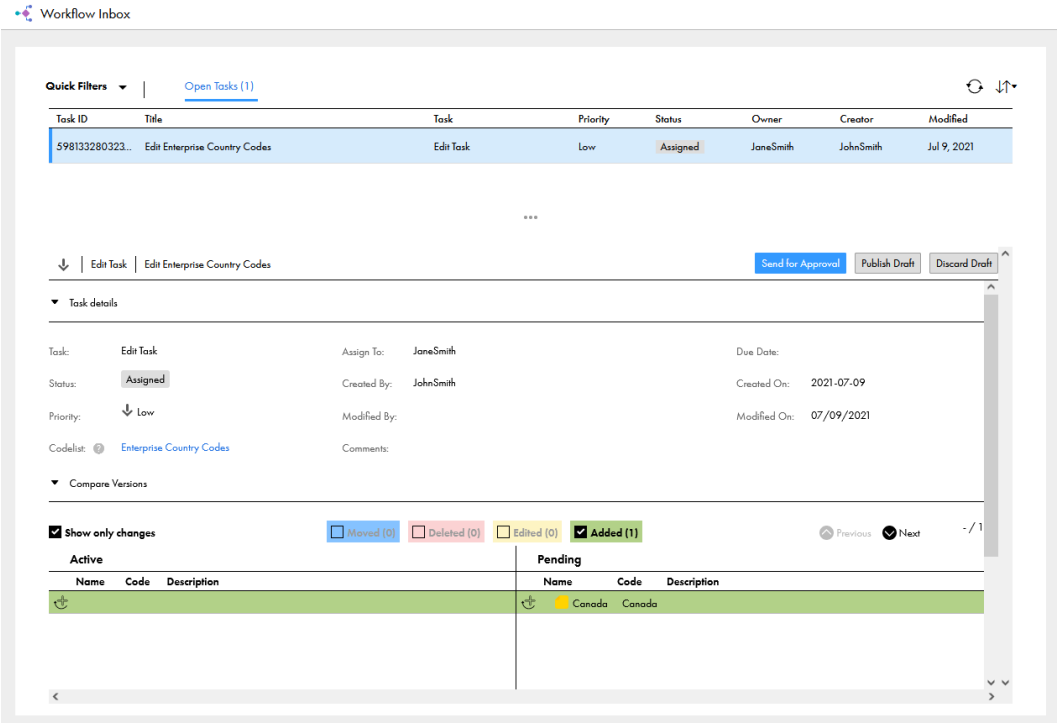
The following table lists the changes to code lists and crosswalks and the highlights:

Change in a code list	Change in a crosswalk	Highlight
Deleted code value	Removed value mapping	Red
Edited code value	-	Yellow and italicized
Moved code value	-	Blue
New code value	New value mapping	Green

The following image shows an example of the **Compare Versions** dialog box that appears when you compare your draft with the active version:



The following image shows the **Compare Versions** section of a task:



Compare lists

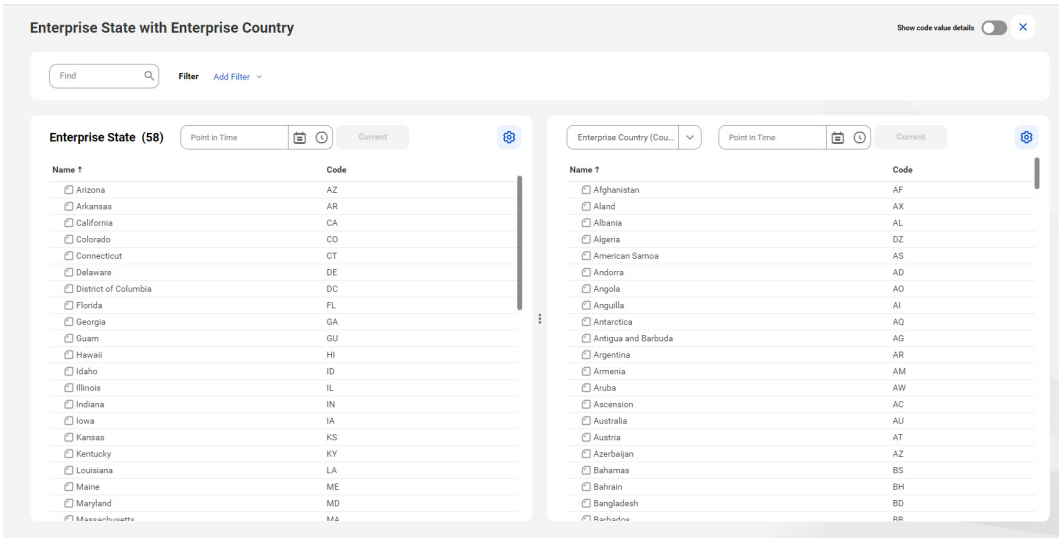
You can see a side-by-side view of code lists and hierarchies to identify similarities and differences between code values. You can choose to compare the same or different types of assets. For example, you might want to compare the code values in a code list with the code values in another code list, or the code values in a code list with the code values in a hierarchy. The side-by-side view displays attribute values in their default language in columns. You can select additional languages to view the respective columns.

To use the side-by-side view to display the information that you want to compare, perform the following tasks:

- To find specific code values, add filter criteria or enter search terms to find specific code values.

- To display the details of selected code values in each list, including summary, history, and relationship information, enable the **Show code value details** option.
- To choose which attribute columns appear and rearrange the attribute columns, configure the column settings.
- To view the code values that existed in code lists and hierarchies at a point in time in the past, select a specific date and time.

The following image shows a sample comparison between the Enterprise State and Enterprise Country code lists:



History

The **History** tab displays a log of changes to an asset or code value. You can view historical information about a reference data set, a code list, a crosswalk, a hierarchy, or a code value.

When you view the **History** tab, you can see the following information:

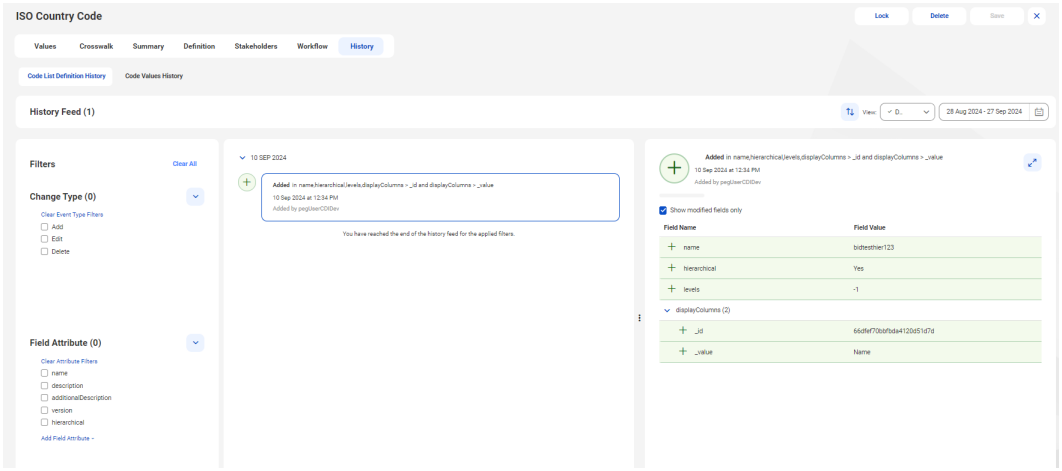
Asset	Historical information
Reference data set	Changes to properties, status, and definition.
Code list	Changes to properties, status, definition, and code values
Crosswalk	Changes to properties, status, definition, and value mappings.
Code value	Changes to the code value.

The **History** tab displays the following historical information:

- The field that was changed
- The value that was changed
- The new value
- The event that occurred
- The user who modified the value
- The date and time the value was modified

- The user who approved the changed value

The following image shows the **History** tab for a sample code list:



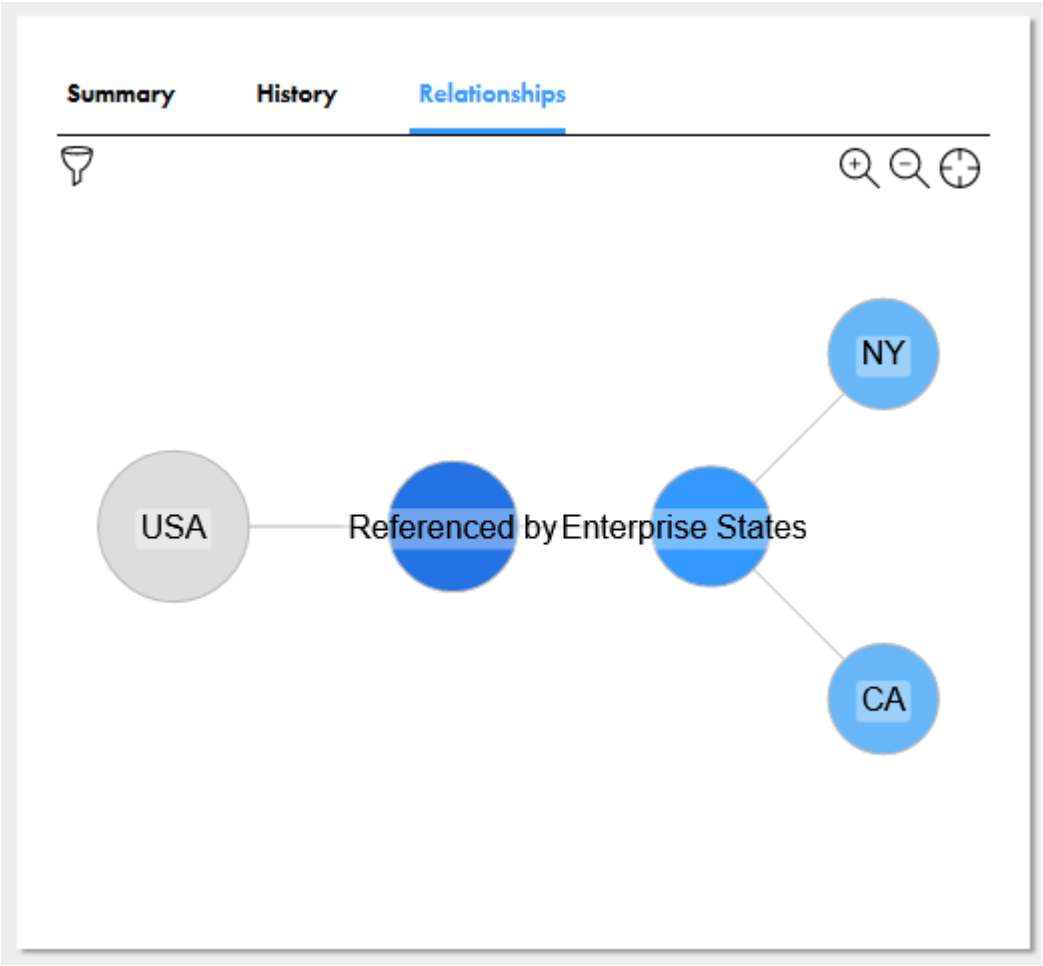
Relationships

You can view relationships between code values in code lists, crosswalks, and hierarchies. Use the **Relationships** tab to view the relationships for a code value in a graph.

The following table lists the relationships that the graph displays:

Asset Type	Relationships
Dependent code list	The code value that the selected code value is dependent on. The dependent code values of the selected code value.
Hierarchical code list	The parent code value of the selected code value. The child code values of the selected code value.
Crosswalk	The code value that the selected code value is mapped to. The code value that the selected code value is mapped from.
Hierarchy	The parent code value of the selected code value. The child code values of the selected code value.

The following image shows the relationships for the USA code value:



Data quality rule associations

Create data quality rule associations to ensure that code values meet your business standards. A basic rule association is based on a simple condition-based rule. An advanced rule association is based on a Cloud Data Quality rule specification. You can create basic and advanced rule associations for string, integer, decimal, date, and reference data attributes of code lists, and advanced rule associations for boolean attributes of code lists.

When users add or edit code values that do not meet your business standards, they receive inline validation errors.

For example, in the Country Codes code list, you might want the Code attribute to require a minimum of three characters. When users create code values in the code list, if they enter the USA value in the Codes attribute, then the value is valid. If they enter the US value, then the value is invalid and they receive inline validation errors.

For example, in the Health Insurers code list, you might have the Name, Code, and Third-Party Administrator attributes. You configure a concatenate rule for the Code attribute to populate the Code attribute with the values in the Name and Third-Party Administrator attributes. If the Name attribute value is Acme Inc. and the Third-Party Administrator attribute value is General Insurer, the value populated for the Code attribute is Acme Inc., General Insurer.

Warning: If you create or edit rules for a code list containing code values, some of the code values might become invalid. To find the invalid code values, edit each code value to trigger a validation check.

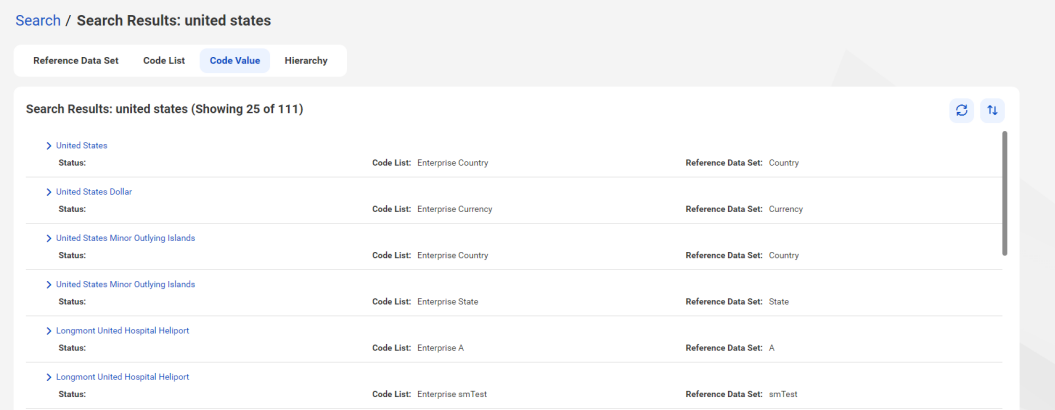
Search

You can search for reference data in your organization and filter the search results by reference data sets, code lists, code values, or hierarchies. The search results are grouped into tabs by asset type. The hierarchy details are not available for users with a Reference Data Management Basic Edition license.

Use the search box in the application header to perform a keyword search. Any reference data that match the keywords appears in the search results.

When code values appear in the search results, you can open the code lists and reference data sets that contain the code values from the search results.

The following image shows a sample **Search Results** page with a list of code values:



Internal IDs and aliases

Internal ID is a unique identifier and alias is an alternative identifier of a reference data asset. You can use internal IDs and aliases as identifiers in applications that use Reference 360.

When you create a reference data asset, you can specify a unique internal ID for the reference data asset. You can configure internal IDs with alphanumeric characters and underscores, limited to a maximum of 45 characters. You can't update the internal ID after you save an asset. You can't specify internal IDs for assets that were created before the April 2025 release.

When you create or update a reference data asset, you can also specify an alias. You can configure aliases with alphanumeric characters and underscores, limited to a maximum of 70. After you save an asset, you can edit the alias.

Note: Effective in the April 2025 release, internal IDs of predefined assets start with an underscore. However, internal IDs and aliases for custom assets and aliases for predefined assets can't begin with an underscore.

For more information about using the identifiers of assets to import the assets, see *Import model for adding new assets and updating existing assets*.

To generate alias for the existing predefined assets that don't have internal IDs, run the alias generation API.

For more information about alias generation API, see “alias generation ” on page 525.

Localization Languages code list

Use the Localization Languages code list, which is a predefined code list, to maintain and display reference data attribute values in multiple languages so that users across your organization can work with assets in their preferred language.

Preview Notice: Effective in the October 2026 release, the ability to manage multilingual assets is available for preview.

Preview functionality is supported for evaluation purposes but is unwarranted and is not supported in production environments or any environment that you plan to push to production. Informatica intends to include the preview functionality in an upcoming release for production use, but might choose not to in accordance with changing market or technical circumstances. For more information, contact Informatica Global Customer Support.

Reference 360 enables you to store and display reference data in more than one language. You can configure the languages that your organization supports, set attributes as multilingual, and then enter and view the values of those attributes in each configured language.

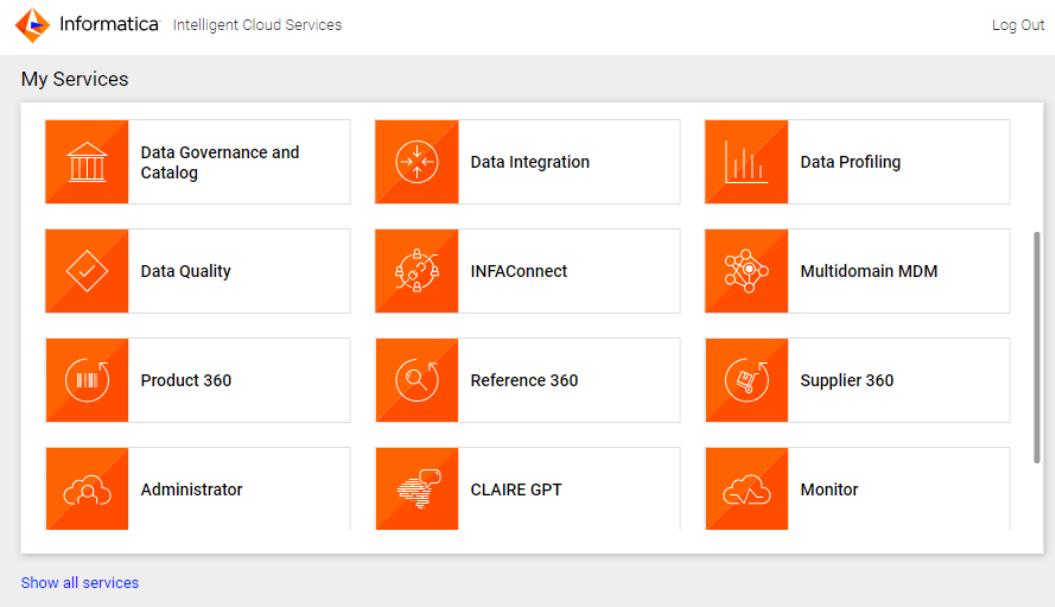
The languages available to your organization are defined in the Localization Languages code list, which also identifies the default language. When you set a string attribute in a reference data set or code list as multilingual, Reference 360 displays a separate value for the attribute in each configured language. If a value isn't available in the preferred language, Reference 360 displays the value in the default language.

For more information about the Localization Languages code list, see Chapter 8, “Manage multilingual assets” on page 144.

My Services page

When you log in to Informatica Intelligent Cloud Services, the **My Services** page displays the services that your organization is licensed to use and any common services that are available under the same license, such as Administrator. If your organization has trial licenses for additional services, the page also displays those services.

The following image shows a sample **My Services** page:

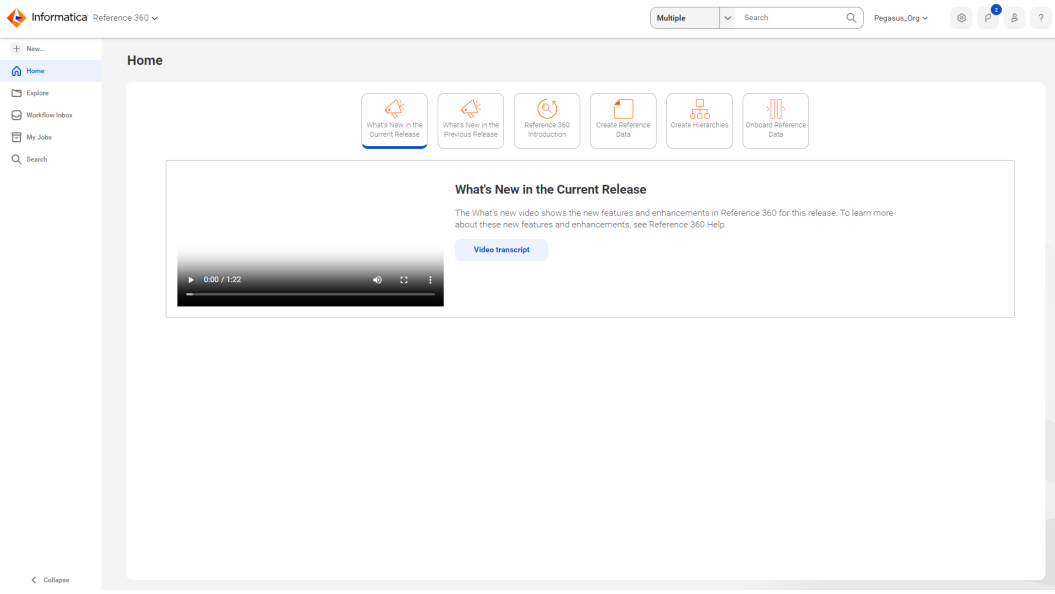


To use Reference 360, click **Reference 360**.

Home page

After you log in to Reference 360, the **Home** page appears.

The following image shows the Home page:

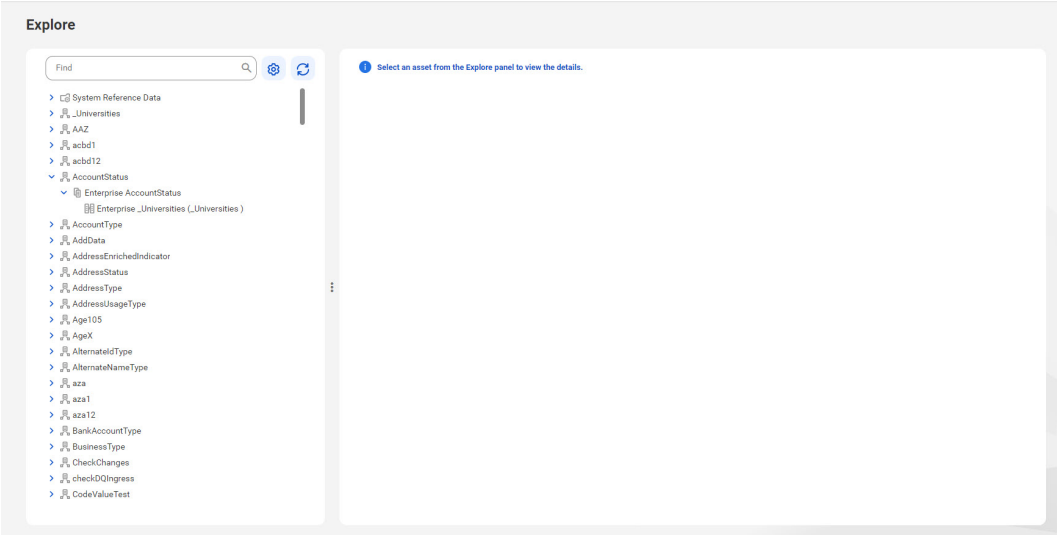


You can access the quick start videos and their transcripts on the **Home** page.

Explore page

Use the **Explore** page to find and work with your assets in Reference 360.

The following image shows an example **Explore** page:



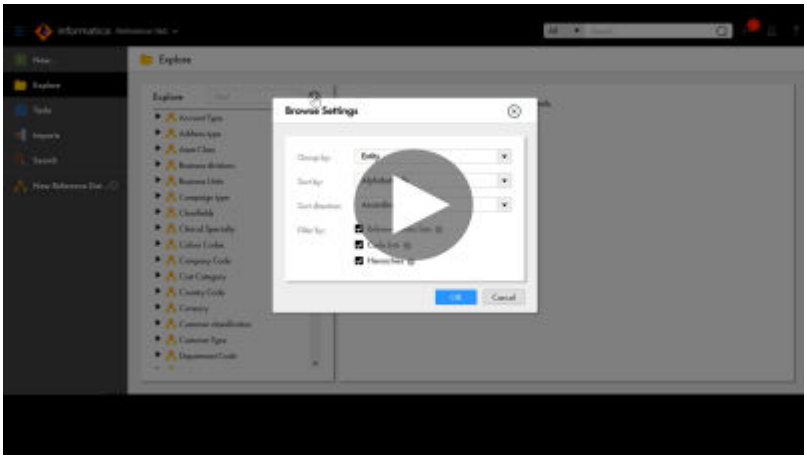
Finding assets on the Explore page

Use any of the following methods to find your assets on the **Explore** page:

- Group assets. View assets grouped by entity, status, domain, priority, and approver. To group assets, click **Browse Settings**, and then select the group by option.
- Sort assets. View assets alphabetically, by priority, or by status. You can also further sort by ascending or descending order. To sort assets, click **Browse Settings**, and then select the sort options.
- Filter the assets on the page. To view reference data sets, code lists, crosswalks, hierarchies, or all assets, click **Browse Settings**, and then select the filter by options.

  Note: The **Browse Settings** page does not display hierarchy assets for users with a Reference Data Management Basic Edition license.
- Search for assets. To search all assets in the organization, enter a name in the search box.

The following video shows you how to find assets on the Explore page:



Refreshing assets on the Explore page

To view the updated assets list in Reference 360, click the **Refresh** icon on the **Explore** panel.

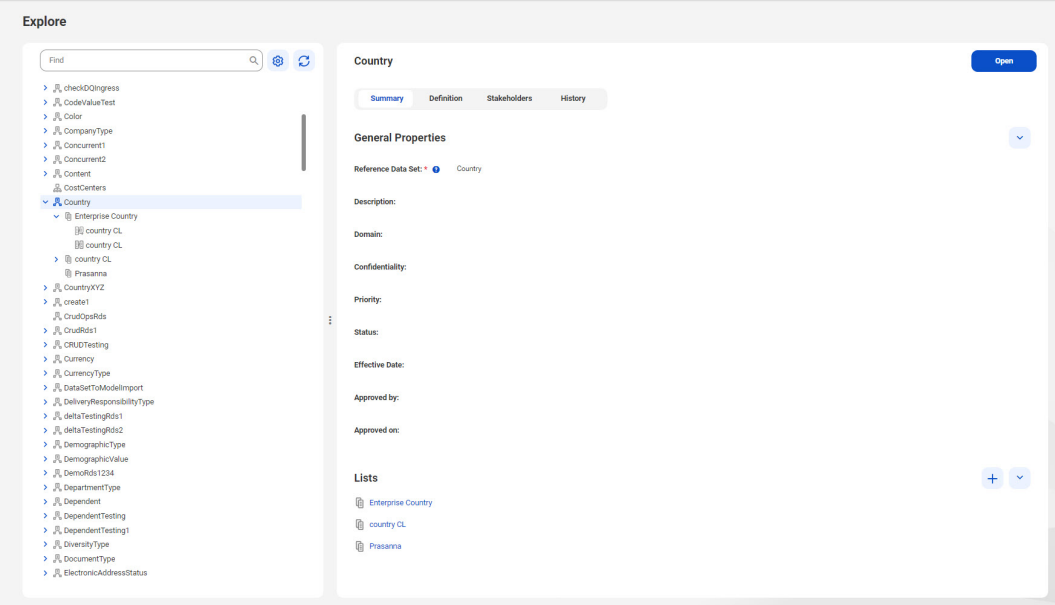
Note: The assets that other users created appear only after you refresh the **Explore** panel.

Working with assets on the Explore page

You can view and edit assets on the **Explore** page. The **Explore** panel displays all assets. The details panel displays the asset details, such as the summary, definition, stakeholders, and history. To edit an asset, select an asset in the **Explore** panel, and then click **Open** in the details panel.

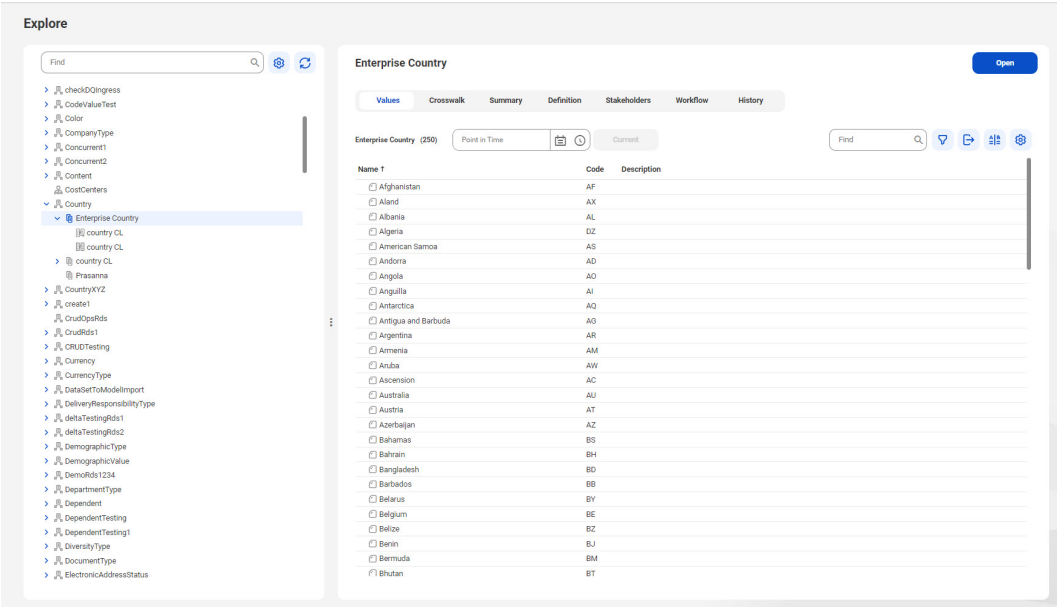
In the **Explore** panel, when you select a reference data set, you can view the associated code lists.

The following image shows a sample reference data set:



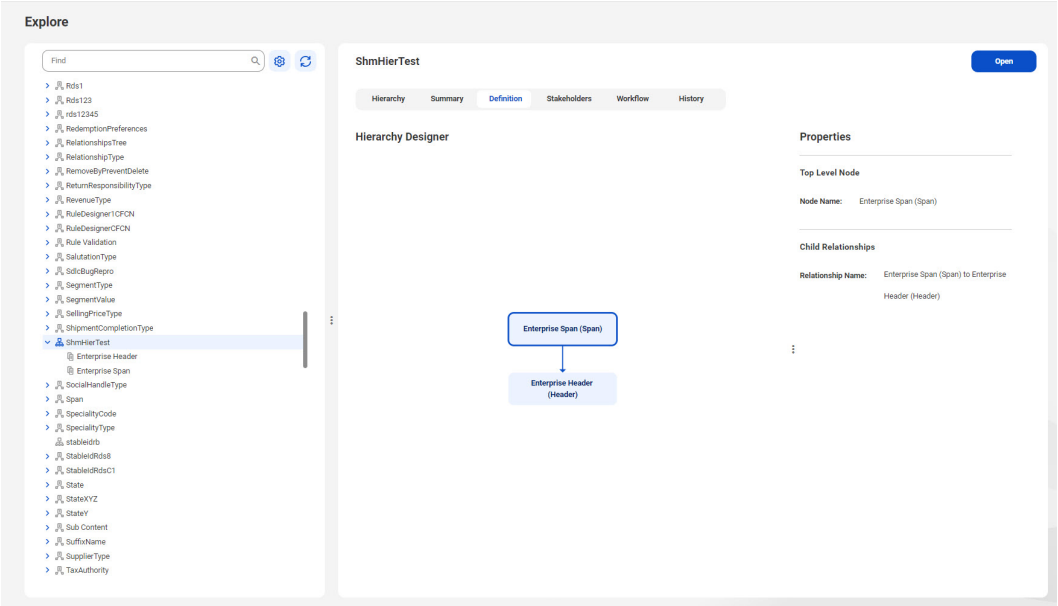
In the **Explore** panel, when you view a code list, you can see the associated outgoing and incoming crosswalks. In the details panel, you can view the code values in the code list and access all crosswalks associated to the code list.

The following image shows a sample code list:



In the **Explore** panel, when you select a hierarchy, you can see the code lists associated to the hierarchy. In the details panel, you can view the hierarchy model on the **Definition** tab and the code values in the hierarchy on the **Hierarchy** tab.

The following image shows a sample hierarchy:



Assets preserve the current view even when you navigate to another workspace or switch between assets in the **Explore** panel.

When you select a reference data set, code list, crosswalk, or hierarchy from the **Explore** panel, click the **History** tab, and then expand the **Explore** panel, the **History** tab displays the history feed similar to the history feed of code values. You can view the **History details** page by clicking the **Expand** icon. For more information, see "Viewing history of a code value" on page 128.

Note: The **Explore** panel retains its state when you navigate to the **Explore** page from a different workspace. The **Explore** panel resets to the default state when you perform the following actions:

- create or edit an asset to include the new asset or changes to the asset.
- delete an asset to exclude the asset.
- create a draft of a code list, which displays a lock icon against the draft code list.
- publish or discard the draft of a code list to save or discard the changes to the code list, which releases the lock icon that appears against the code list.

Customize the brand

You can customize the logo and favicon that appear on your business application.

Use Administrator to configure custom branding for your parent organization and apply them to your sub-organizations. You can also configure custom branding for each sub-organization.

To change the branding, you must have the **Custom Logo & Color Themes** license. For more information about licenses, see Licenses in the Administrator help.

Use PNG, JPG, and GIF formats for the custom logo, and PNG format for the favicon.

For more information about configuring the custom branding, see Configuring custom branding for an organization in the Administrator help. For more information about the logo and favicon guidelines, see Logo and favicon guidelines in the Administrator help.

Note: Reference 360 does not support custom theme setting.

Keyboard shortcuts

You can use keyboard shortcuts to navigate and work with the Reference 360 user interface. Press the Tab key to navigate across the application and Shift+Tab keys to navigate backwards across the application. A solid line appears around the selected element indicating that the element is in focus. The navigation order of objects is from top to bottom and left to right.

The following user interface elements support the keyboard shortcuts:

- Explore panel.
- Values tab of code lists.
- Value Mappings tab of crosswalks.
- Hierarchy tab of hierarchies.
- comparison view of code values in code lists and hierarchies.
- Select the value dialog box that's used to select assets for reference data attributes of code lists.

The following table lists the available keyboard shortcuts:

Action	Shortcut key
Navigate across elements	Tab
Navigate backwards across elements	Shift+Tab
Perform the selected operation	Enter
Open the calendar	Alt+Down arrow
Select a value or clear the selection	Space
Select multiple nodes	Shift+Space
Expand a node	Shift+Right arrow
Collapse a node	Shift+Left arrow
Decrease the width of a table header	Alt+Left arrow
Increase width of a table header	Alt+Right arrow
Scroll left	Ctrl+Left arrow
Scroll right	Ctrl+Right arrow
Navigate to the upper cell in a table	Up arrow
Navigate to the lower cell in a table	Down arrow
Navigate to the left cell in a table	Left arrow
Navigate to the right cell in a table	Right arrow
Delete a node	Delete or Shift+D
Add a new node	Shift+N
Cut one or multiple nodes	For Windows: Ctrl+X For Mac: Cmd+X
Paste the cut nodes	For Windows: Ctrl+V For Mac: Cmd+V
Close a dialog box	Escape
In the comparison view, navigate to the next row that underwent changes	N
In the comparison view, navigate to the previous row that underwent changes	P

Action	Shortcut key
Navigate to the first cell in a table	For Windows: Home For Mac: Fn+Left arrow
Navigate to the last cell in a table	For Windows: End For Mac: Fn+Right arrow

CHAPTER 2

Getting started with Reference 360

You can set up your organization in just a few steps.

Step 1. Create users or configure SAML

All Reference 360 users must have an Informatica Intelligent Cloud Services user account. Use the Administrator service to create Informatica Intelligent Cloud Services user accounts or configure SAML.

Step 2. Add system reference data values

System reference data contains a group of values that provide information about your reference data, such as the application, confidentiality, domain, priority, or status of an asset. You can add the values that you want to appear in the system reference data. For more information about adding values to the system reference data, see "Adding values to system reference data" on page 100 or "Add system reference data values" on page 531.

Note: To use workflows, you must add values to the Priority system reference data. For more information about workflows, see "Workflows" on page 33.

Reference 360 licenses

Licenses determine the subscription plan and unlock access to the Reference 360 features and assets.

Reference 360 uses the following types of licenses:

- Reference Data Management Basic Edition. Provides access to create and manage reference data assets when you ingress data to Reference 360.
- SaaS Edition - Reference 360. Provides complete access to Reference 360 assets and advanced functionalities.

Reference Data Management Basic Edition

The Integration Components for Business 360 license package includes the Reference Data Management Basic Edition license. The Reference Data Management Basic Edition license provides limited access to Reference 360 features.

The Reference Data Management Basic Edition license for Reference 360 includes the following capabilities:

- Create and manage reference data assets.

- Import and export reference data assets using CSV files and batch jobs.

The Reference Data Management Basic Edition license for Reference 360 doesn't include workflows, hierarchies, stakeholders, advanced rule associations, or API support.

SaaS Edition - Reference 360 license

The SaaS Edition - Reference 360 license provides complete access to Reference 360 features without any restrictions.

Users, groups, and roles

A user is an individual account in Informatica Intelligent Cloud Services. Reference 360 users must have a Informatica Intelligent Cloud Services user account. You create and manage users in the Administrator service.

A group is a collection of users who have something in common, such as working in the same team. Add users to groups to efficiently manage privileges for a collection of users.

A role is a collection of privileges that you assign to users and groups to allow access to Reference 360 and provide a collection of privileges in Reference 360.

Assign the following types of roles to provide privileges for Reference 360:

Reference 360 Roles

Reference 360 roles are pre-defined Informatica Intelligent Cloud Services roles that provide service-specific access to Reference 360 and global privileges in Reference 360. Use the Administrator service to assign Reference 360 roles.

For more information, see "Reference 360 roles" on page 50.

Stakeholder Roles

Stakeholder roles are pre-defined roles that provide asset-specific privileges in Reference 360. When you create assets in Reference 360, assign stakeholder roles to users and groups for each asset.

For more information, see "Stakeholder roles" on page 53.

For more information about Informatica Intelligent Cloud Services users, user groups, and roles, see the *Administrator* help.

Reference 360 roles

Reference 360 roles define a set of privileges that a user has while working in MDM - Reference 360. The privileges apply to all assets. To allow a user to access MDM - Reference 360, an administrator must assign at least one of the Reference 360 roles to the users or to their user group.

The following list describes the Reference 360 roles:

Reference 360 Administrator

Reference 360 Administrators configure the Reference 360 environment. Administrators can purge code values of code lists along with their relationships and historical data in Business 360 Console.

Note: Users with access to Business 360 Console must use Business 360 Console exclusively to purge reference data assets and avoid using other user interface components.

To purge reference data in Business 360 Console, users must navigate to **Global Settings** and use the **Purge Data** option.

Reference 360 Planner

Planners create hierarchy assets, define hierarchy models, and import hierarchy relationships. Planners can delete hierarchies that are no longer needed. Planners can also assign the Planner role to other users for the hierarchy assets.

Reference 360 Primary Owner

Primary Owners create and define reference data structures such as reference data sets and code lists. Primary Owners can delete code lists and propose changes to code values in code lists. The proposed changes must be approved by Business Stewards.

Reference 360 Business Steward

Business Stewards are subject matter experts for reference data, who create and manage code values in code lists and value mappings in crosswalks. They review and approve changes proposed by other users, and can also submit their own changes for approval. Business Stewards can directly publish their changes if they are added as approvers in the approval workflow.

Reference 360 Stakeholder

Stakeholders propose changes to code values. The proposed changes must be approved by Business Stewards.

Reference 360 Business Analyst

Business Analysts view and analyze assets. Business Analysts cannot propose changes to assets.

Reference 360 User

By default, users with this role can't access any assets. To allow users to access a specific asset, such as a code list or crosswalk, the users require stakeholder roles for the asset.

The following table lists the privileges of the Reference 360 roles:

Function	Planner	Primary Owner	Business Steward	Stakeholder	Business Analyst	User
System reference data	Read	-	Create Read Update Delete	Read	Read	-
Reference data set	Read	Create Read Update Delete	Read	Read	Read	-
Reference data set structure definition	Read	Create Read	Read	Read	Read	-
Reference data set attributes	Read	Create Read Update Delete	Read	Read	Read	-

Function	Planner	Primary Owner	Business Steward	Stakeholder	Business Analyst	User
Code list	Read	Create Read Update Delete	Read	Read	Read	-
Code list structure definition	Read	Create Read	Read	Read	Read	-
Code list attributes	Read	Create Read Update Delete	Read	Read	Read	-
Code list draft	Read	Propose changes	Propose changes Approve changes Publish changes	Propose changes	Propose changes Approve changes	-
Crosswalk	Read	Read	Create Read Update Delete	Read	Read	-
Crosswalk value mappings	Read	Read	Create Read Update Delete	Read	Read	-
Hierarchy	Create Read Update Delete Assign stakeholders Import hierarchy relationships	Read	Read	Read	Read	-
Search and explore	Permitted	Permitted	Permitted	Permitted	Permitted	-
Export	Permitted	Permitted	Permitted	Permitted	Permitted	-
Import	-	Permitted	Permitted	Permitted	-	-
Direct import	-	-	Permitted	-	-	-

Function	Planner	Primary Owner	Business Steward	Stakeholder	Business Analyst	User
Localization Languages code list	Read	Read	Read	Read	Read	Read
Code value of Localization Languages code list	Read	Create Read Update Delete	Read	Read	Read	Read

Note: After you create an asset, you might not be able to edit some definition settings. For more information, see "Reference data sets" on page 16 and "Code lists" on page 19.

For more information about Informatica Intelligent Cloud Services users, user groups, and roles, see the *Administrator* help.

Stakeholder roles

Stakeholder roles define a set of privileges for each asset in Reference 360. An asset might be a reference data set, code list, or crosswalk. Users with the Reference 360 Primary Owner role are responsible for assigning stakeholder roles to users and groups for each asset.

The following list describes the stakeholder roles that you can assign to users and groups for an asset:

Planner

Planners define the hierarchy model and import hierarchy relationships to hierarchy assets. Planners can also assign the Planner role to other users for the hierarchy assets.

Primary Owner

Primary Owners create and define the asset. Primary Owners can propose changes to the asset, but their changes must be approved by Business Stewards.

Business Steward

Business Stewards are subject matter experts for reference data, who create and manage code values in code lists and value mappings in crosswalks. They review and approve changes proposed by other users, and can also submit their own changes for approval. Business Stewards can directly publish their changes if they are added as approvers in the approval workflow.

Stakeholder

Stakeholders propose changes to the asset. The proposed changes must be approved by Business Stewards.

Business Analyst

Business Analysts view and analyze the asset. Business Analysts cannot propose changes to the asset.

The following table lists the privileges that are assigned to users with a stakeholder role:

Function	Planner	Primary Owner	Business Steward	Stakeholder	Business Analyst
Reference data set	-	Read Update Delete	Read	Read	Read
Reference data set structure definition	-	Read Update Delete	Read	Read	Read
Reference data set attributes	-	Create Read Update Delete	Read	Read	Read
Code list	-	Read Update Delete	Read	Read	Read
Code list structure definition	-	Read Update Delete	Read	Read	Read
Code list attributes	-	Create Read Update Delete	Read	Read	Read
Code list draft	-	Propose changes	Propose changes Approve changes Publish changes	Propose changes	-
Crosswalk	-	Read	Read Update Delete	Read	Read
Crosswalk value mappings	-	Read	Create Read Update Delete	Read	Read

Function	Planner	Primary Owner	Business Steward	Stakeholder	Business Analyst
Hierarchy	Create Read Update Delete Assign stakeholders Import hierarchy relationships	Read	Read	Read	Read
Search and explore	-	Permitted	Permitted	Permitted	Permitted
Export	-	Permitted	Permitted	Permitted	Permitted
Import	-	Permitted	Permitted	Permitted	-
Direct import		-	-	-	-

Note: After you create an asset, you might not be able to edit some definition settings. For more information, see "Reference data sets" on page 16 and "Code lists" on page 19.

Note: The Planner stakeholder role is the only stakeholder role that you can assign to hierarchy assets.

Guidelines for assigning roles

To provide users with access to MDM - Reference 360, administrators must assign at least one Reference 360 role to users or groups to which the users belong. Then primary owners can assign stakeholder roles for assets to users. Role privileges are cumulative. Users who have multiple roles or who belong to groups receive the combined privileges associated with each role.

When administrators assign Reference 360 roles to users or groups, assign the role with the minimum number of privileges that are required for each user to work with Reference 360. Assign stakeholder roles only to those users or groups that work with an asset.

For example, you assign John the Reference 360 User role. He cannot view any assets in Reference 360. John is the subject matter expert for the Enterprise Language code list, so you assign John the Business Steward stakeholder role for the code list. The role allows him to create, manage, and approve code values in the Enterprise Language code list. He can also directly publish his changes to the code list. For all other assets, John's Reference 360 User role applies.

Guidelines for restricting access to assets

You can restrict a user's access to assets and only grant access to the assets that the user needs to view and work on. To restrict a user's access, assign the user the Reference 360 User role.

To grant privileges for only the assets that a user needs to view and work on, you assign the user a stakeholder role for the code list or crosswalk. The stakeholder role that you assign depends on the privileges that you want to grant the user.

When you grant privileges to code lists or crosswalks, the user gets access to the related assets.

Note: You must be assigned the Reference 360 Primary Owner role or the Primary Owner stakeholder role for the asset and the related assets to which you want to grant, change, or revoke access.

For example, assign Jane Smith the Reference 360 User role to restrict her access to assets. Then you assign Jane the Business Analyst stakeholder role for the Enterprise Language code list. The role allows her to view and analyze the Enterprise Language code list and the Languages reference data set.

The following table lists the assets for which you can assign stakeholders, and the assets and related assets to which users get access:

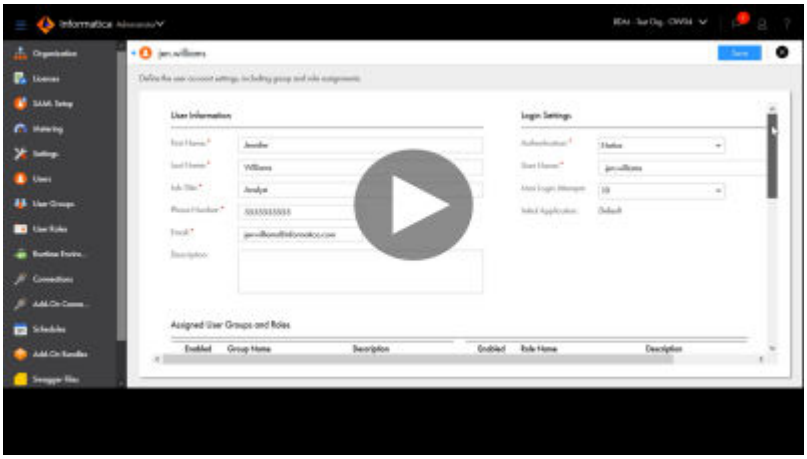
Stakeholder for	Access to
Code list	- The code list to which you granted access. - The reference data set to which the code list is associated.
Code list with a dependency	- The code list to which you granted access. - The reference data set to which the code list is associated. - The parent code list. - The reference data set to which the parent code list is associated.
Code lists with Reference Data attributes	- The code list to which you granted access. - The code lists referenced as Reference Data attributes. - The reference data sets to which the referenced code lists are associated.
Crosswalk	- The crosswalk to which you granted access. - The source and target code lists used in the crosswalk. - The reference data sets to which the code lists are associated.

Creating an Informatica Intelligent Cloud Services user

All Reference 360 users must have an Informatica Intelligent Cloud Services user account. You use the Administrator service to create user accounts and assign Reference 360 roles to users.

For more information about roles, see *User Administration* in the Administrator help.

The following video shows you how to create an Informatica Intelligent Cloud Services user:



1. In the service menu, click **Reference 360**.
2. In the **My Services** window, click **Administrator**.
3. Click **Users**.
4. Click **Add User**.
5. Enter the user information and login settings.

6. In the **Assigned User Groups and Roles** section, select the user groups and roles that you want to assign to the user.

   Assign Reference 360 roles to users to provide access privileges to Reference 360.
   For more information, see "Reference 360 roles" on page 50.

7. Click **Save**.

Creating a user group

Create a user group when multiple users in your organization need to perform the same tasks and need the same access rights for different types of assets. Group members can perform tasks and access assets based on the roles that you assign to the group. You use the Administrator service to create user groups and assign Reference 360 roles to groups.

1. In the service menu, click **Reference 360**.
2. In the **My Services** window, click **Administrator**.
3. Click **User Groups**.
4. Click **Add Group**.
5. Enter a group name and optional description.
6. In the **Assigned Roles** section, select the Reference 360 roles that you want to assign to the group.

   For more information, see "Reference 360 roles" on page 50.

7. To assign a user to the group, move the user from the **Available Users** list to the **Assigned Users** list.

   You can also assign a user to a group when you create or edit a user.

8. Click **Save**.

   Note: When you rename a user group in **Administrator**, the **Stakeholders** tab in Reference 360 doesn't show the updated name. Remove the user group from the **Stakeholders** tab, add it again with the updated name, and reassign its stakeholder roles.

Managing privileges for custom user roles

You can create custom user roles and configure their privileges in your organization to control access to all reference data assets, individual assets, records, attributes, and workflows in Reference 360. For example, you can allow a custom user role to view code lists but not edit or delete them.

Unlike system-defined roles, which have predefined access privileges that you can't change, custom user roles aren't predefined. You can create custom user roles based on specific requirements of your organization. You can also control specific tasks that a custom user role can perform. For example, you can authorize a custom user role to import code values into Reference 360.

In Administrator, you can create a custom user role and assign permissions, and then fine-tune the permissions on the Security page of Business 360 Console. The authorizations that you configure in Administrator work in conjunction with the authorizations that you configure on the Security page.

Preview Notice: Effective in the October 2026 release, the ability to manage privileges for custom user roles is available for preview. Preview functionality is supported for evaluation purposes but is unwarranted and is not supported in production environments or any environment that you plan to push to production. Informatica intends to include the preview functionality in an upcoming release for production use, but might choose not to in accordance with changing market or technical circumstances. For more information, contact Informatica Global Customer Support.

Configure privileges for custom roles to access the Security page

As an organization administrator, you can create custom roles to access the Security page in Business 360 Console. Users with these roles can configure privileges for other custom user roles to control access to reference data assets, attributes, code values, and workflows in Reference 360. You can also configure privileges for the custom user roles directly.

1. In Administrator, click **User Roles**.
2. Click the custom role for which you want to configure privileges.
3. In the **Services** field, select **MDM Configuration**.
4. On the **Features** tab, enable the **Configuration and Authoring** and **Manage Authorization configurations** features.
5. Click **Save**.

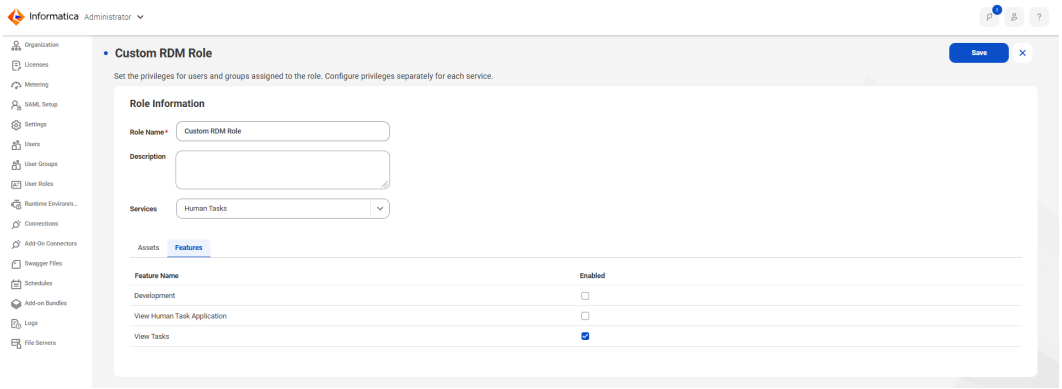
Prerequisites

Before you configure privileges for custom user roles in Business 360 Console, assign relevant privileges for the custom user roles in Administrator.

The following table lists the privileges that you can configure for the custom user roles in Administrator:

Service	Assets/Features	Set to Enabled
Reference 360	Features	Reference Data Management. Allows you to assign privileges for all the reference data assets available in Reference 360 on the **Security** page of Business 360 Console.
Human Tasks	Features	**View Tasks**. Indicates whether the user can access the assigned tasks in the workflow inbox of Reference 360. A user must have this privilege to submit drafts of reference data assets for approval.

The following image shows the role details page in Administrator:



Rules and guidelines for managing privileges for custom user roles

Consider the following rules and guidelines when you manage privileges for custom user roles in your organization:

- You can't delete system-defined roles, but you can delete custom user roles.
- After you configure privileges for custom user roles on the Security page in Business 360 Console, don't rename the custom user roles in Administrator. If you rename custom user roles, Business 360 Console removes all the configured privileges.
- Custom user role names are case-sensitive.
- If a user is assigned a custom user role along with a predefined user role in Reference 360, the predefined role overrides the custom user role.
- If you create a custom user role in Administrator with a name identical to any predefined user role in Reference 360, Business 360 Console doesn't display the custom user role on the Security page.
- When you change privileges for all or for individual reference data assets, users with custom user roles must wait at least 10 minutes before they use the search box in the Reference 360 header to search for assets. Reference 360 applies privilege changes to keyword search after a short delay. After at least 10 minutes, the asset options available for search, the search results, and the tabs in the search results reflect the updated privileges. If users search for assets immediately after the changes, the search returns inaccurate results.
- When you bulk edit privileges for code lists and assign a dependent privilege without its required privilege, Reference 360 displays an error icon next to the required privilege without a validation error message.
- Users with a custom user role can delete code list attributes configured with rule associations even if the role doesn't have the delete privilege for the Data Quality metadata.
- Users with a custom user role can't view the Value Mappings tab of a crosswalk if the source or target code list contains localized attributes.
- Users with a custom user role can view the display attributes of a referenced or dependent code list even if the role doesn't have the read privilege for the attributes on the Metadata tab of the code list.
- A custom user role that has the create privilege only on the Name and Code fields of a code list can't create code values when other fields have default values or when the Status or Effective Date field has a value.
- When users with a custom user role open the side-by-side view of a code list or hierarchy, Reference 360 displays the code values that existed at a specific point in time, even if the role doesn't have the privilege to view the history of records.
- Users with a custom user role can view and select rule specifications for an advanced rule association of a code list attribute even if the role doesn't have the read privilege for the Rule Specification asset in Administrator.
- Users with a custom user role can modify the Required or Default Value setting of a code list attribute even if the role doesn't have the update privilege for the attributes on the Metadata tab of the code list.
- A custom user role that has only the read privilege for the attributes on the Metadata tab of a code list can update the display attribute of the code list.
- Users with a custom user role can't update the default value of an attribute configured with an advanced rule association, even if the role has create and update privileges for the attribute on the Metadata tab of the code list.
- When you create a reference data asset, Reference 360 doesn't list custom user roles in the **Approved by** field. The field lists only the predefined roles.
- When you import code values into a code list that has localized attributes, the **Map Fields** page of the **Import Values** wizard doesn't list the localized fields.

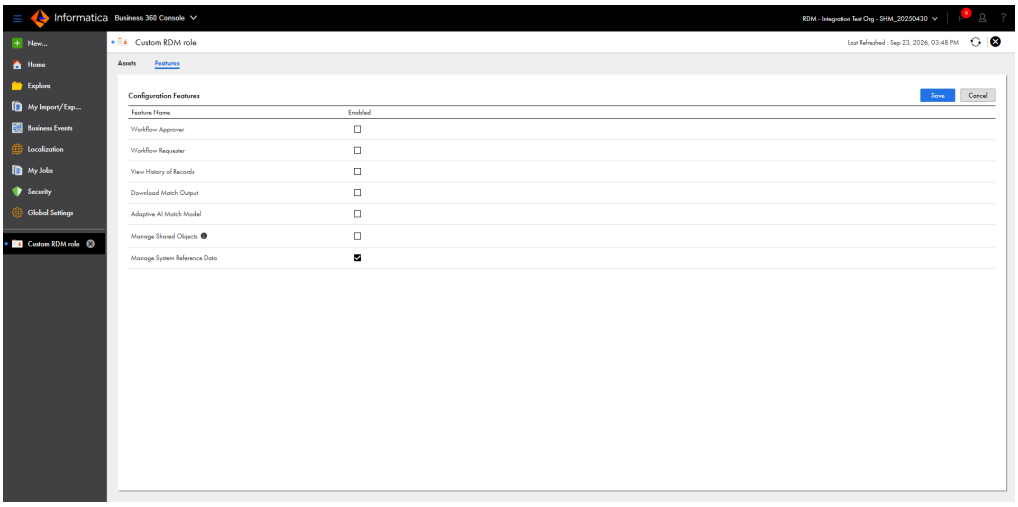
- Users with a custom role can modify the details of code list attributes even if the role doesn't have the update privilege for attributes on the Metadata tab of the code list.
- Users with a custom user role can lock a code list even if the role doesn't have create, update, and delete privileges for code values on the Records tab of the code list. The users can also import code values into the code list even if the **File Import** option is not enabled for the role.
- In an organization that provides access control of reference data assets for custom user roles, if a custom user role doesn't have privileges to access workflows, the Workflow tab and the My Tasks page show a message that a Reference Data Management license is required instead of a message that the user role doesn't have the required workflow privileges. To access workflows, contact your administrator to assign the custom user role with the required workflow permission, such as Workflow Requester or Workflow Approver.
- In an organization that provides access control of reference data assets for custom user roles, when you move a hierarchy value, Reference 360 verifies the privileges for cut and paste of the value separately. To cut a value, you need the read and delete privileges on the parent relationship of the value, but you don't need the create privilege. To paste a value, you need the read and create privileges on the child relationship of the target value, but you don't need the delete privilege.
  You can drag a hierarchy value if you have the delete privilege on the parent relationship of the value. You can drop the value only on target values where you have the create privilege on the child relationship.
- When you assign the read privilege for code values of a code list, assign the privilege to at least one attribute of the code list. You can't save the privileges for a custom user role that has the read privilege for code values but not for any attribute.
- You can restrict the read privilege for any attribute of a code list, including the Name and Code attributes. Users with the custom user role view only the attributes for which the role has the read privilege. For example, if a custom user role has the read privilege for the Code attribute but not the Name attribute, users with the role view only the code of each code value.
- Restricting privileges for a required attribute doesn't make the attribute optional. Required attributes, such as Name and Code, remain required when users add code values. To add code values to a code list, the custom user role needs the read and create privileges for all required attributes of the code list.
- To view a hierarchy, assign to the custom user role the read privilege on the relationship at the first level of the hierarchy. The role doesn't need the read privilege on every relationship in the hierarchy. If the custom user role has the read privilege on some relationships, users with the role view only the code values in those relationships. To view code values in a hierarchy, the custom user role also needs the read privilege on the code values and on at least one attribute of each code list in the hierarchy. The hierarchy displays only the attributes for which the custom user role has the read privilege for predefined and custom code lists.

Configure privileges for system reference data

You can configure privileges for custom user roles to access system reference data. These privileges determine whether a custom user role can create, read, update, and delete values for system reference data.

1. In Business 360 Console, on the **Security** page, click the custom user role for which you want to configure privileges.
2. On the **Features** tab, select **Manage System Reference Data**.

The following image shows the Features tab:



3. Click **Save**.

Configure privileges to access reference data assets

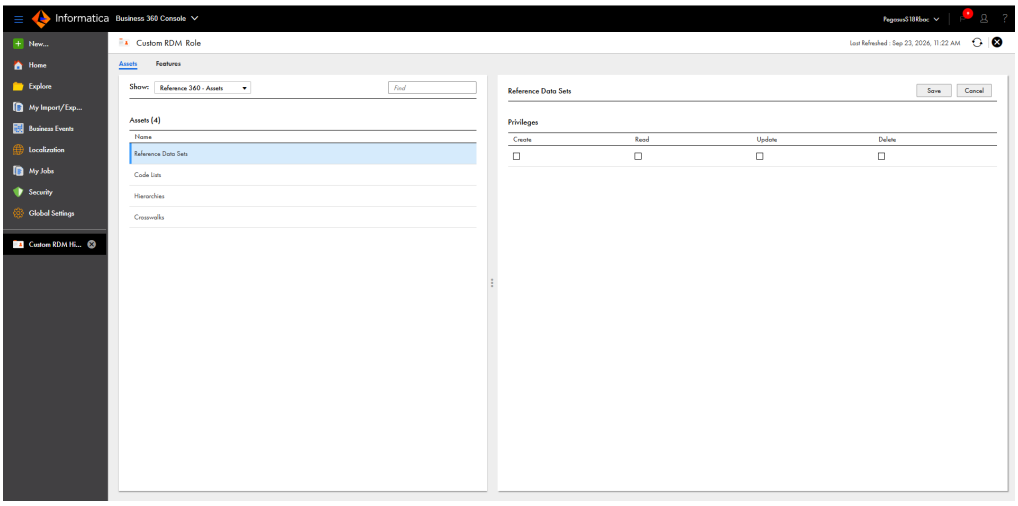
You can configure privileges for custom user roles to access reference data assets. These privileges control whether a custom user role can create, read, update, and delete reference data sets, code lists, crosswalks, and hierarchies.

1. In Business 360 Console, on the **Security** page, click the custom user role for which you want to configure privileges.

   The user role details page lists all the assets in the organization.

2. On the **Assets** tab, select **Reference 360 - Assets** from the **Show** list.

   The left pane displays all the reference data assets.
   The following image shows the list of reference data assets:



3. Select the asset for which you want to configure privileges.

   The right pane displays the privileges for the selected asset.

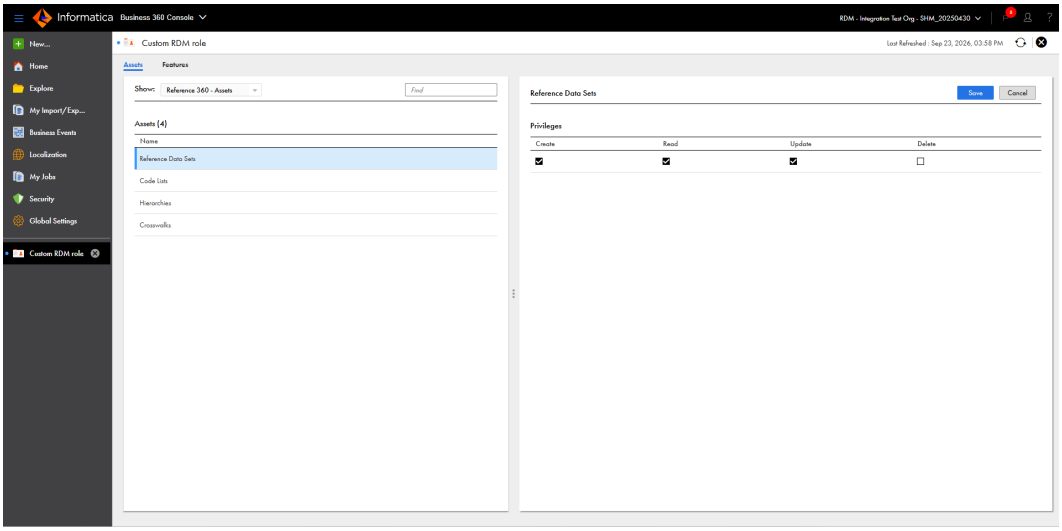
4. Select the permissions in the **Privileges** section.

   For more information about the privileges, see "Configure privileges for reference data sets" on page 62, "Configure privileges for code lists" on page 63, "Configure privileges for crosswalks" on page 63, and "Configure privileges for hierarchies" on page 64.

Configure privileges for reference data sets

Assign privileges to specify whether custom user roles can create, read, update, and delete reference data sets. The privileges that you assign determine whether a custom user role has complete access, partial access, or no access to the reference data sets.

The following image shows the privileges that you can assign to reference data sets:



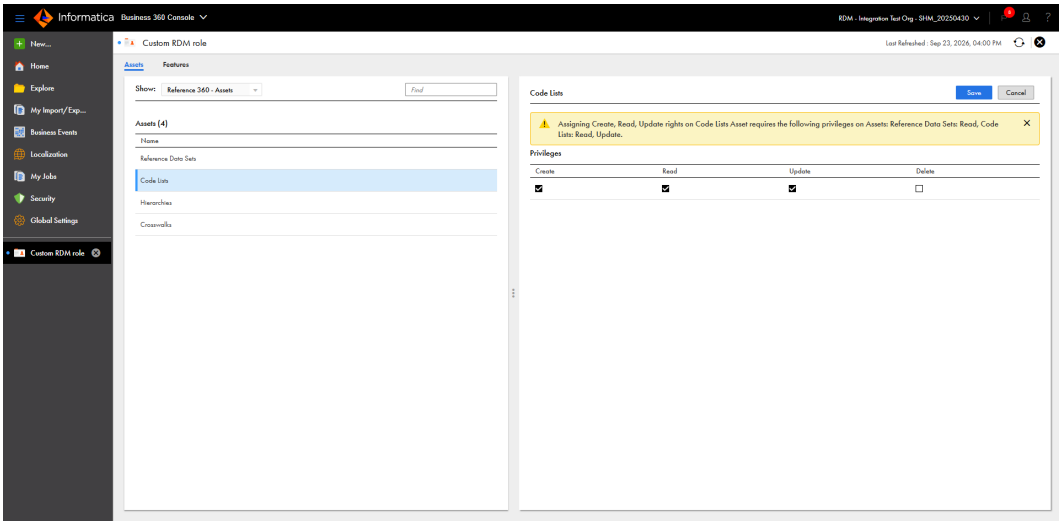
The following table describes the actions that the custom user role can perform based on the privileges assigned to reference data sets:

Privilege	Action
Create	Create reference data sets. Assign the read privilege for reference data sets before you assign the create privilege for reference data sets.
Read	View reference data sets and their properties.
Update	Edit the properties of reference data sets. Assign the read privilege for reference data sets before you assign the update privilege for reference data sets.
Delete	Delete reference data sets. Assign the read and update privileges for reference data sets before you assign the delete privilege for reference data sets.

Configure privileges for code lists

Assign privileges to specify whether custom user roles can create, read, update, and delete code lists. The privileges that you assign determine whether a custom user role has complete access, partial access, or no access to the code lists.

The following image shows the privileges that you can assign to code lists:



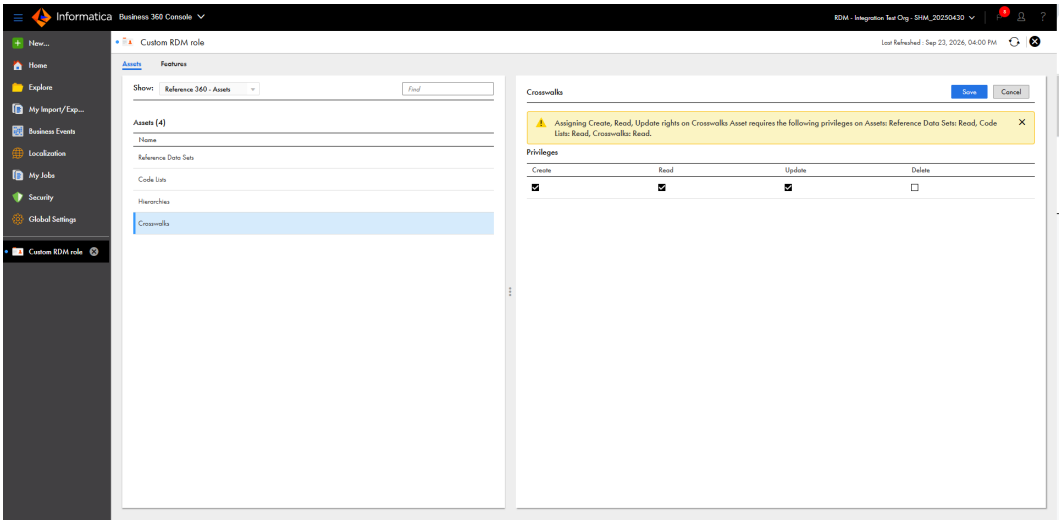
The following table describes the actions that the custom user role can perform based on the privileges assigned to code lists:

Privilege	Action
Create	Create code lists. Assign the read privilege for reference data sets and code lists before you assign the create privilege for code lists.
Read	View code lists and their properties. Assign the read privilege for reference data sets before you assign read privilege for code lists.
Update	Edit the properties of code lists. Assign the read privilege for reference data sets and code lists before you assign the update privilege for code lists.
Delete	Delete code lists. Before you assign the delete privilege for code lists, assign the following privileges: - The read privilege for reference data sets. - The read and update privileges for code lists.

Configure privileges for crosswalks

Assign privileges to specify whether custom user roles can create, read, update, and delete crosswalks. The privileges that you assign determine whether a custom user role has complete access, partial access, or no access to the crosswalks.

The following image shows the privileges that you can assign to crosswalks:



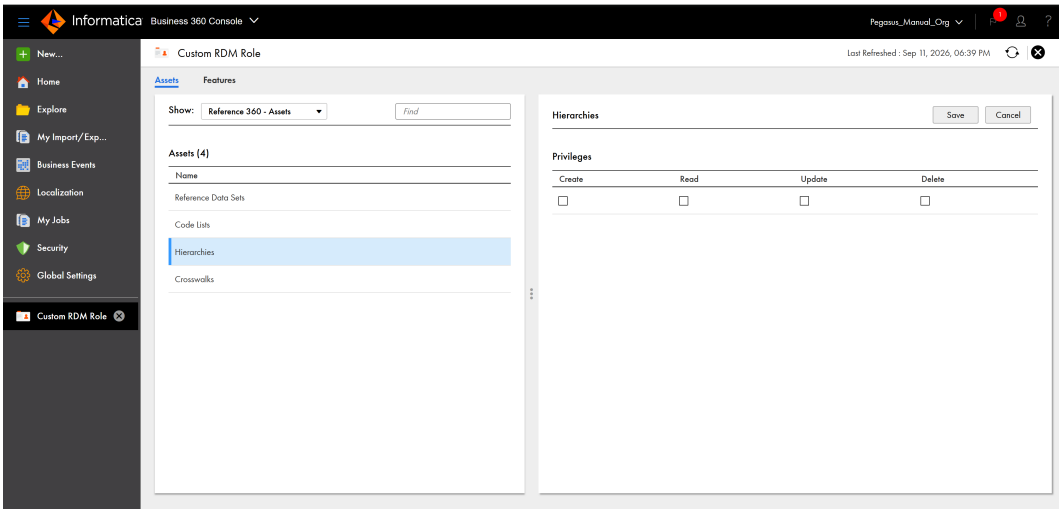
The following table describes the actions that the custom user role can perform based on the privileges assigned to crosswalks:

Privilege	Action
Create	Create crosswalks. Assign the read privilege for reference data sets, code lists, and crosswalks before you assign the create privilege for crosswalks.
Read	View crosswalks and their properties. Assign the read privilege for reference data sets and code lists before you assign the read privilege for crosswalks.
Update	Edit the properties of crosswalks. Assign the read privilege for reference data sets, code lists, and crosswalks before you assign the update privilege for crosswalks.
Delete	Delete crosswalks. Before you assign the delete privilege for crosswalks, assign the following privileges: - The read privilege for reference data sets, code lists, and crosswalks. - The read and update privileges for crosswalks.

Configure privileges for hierarchies

Assign privileges to specify whether custom user roles can create, read, update, and delete hierarchies. The privileges that you assign determine whether a custom user role has complete access, partial access, or no access to the hierarchies.

The following image shows the privileges that you can assign for hierarchies:



The following table describes the actions that the custom user role can perform based on the privileges assigned for hierarchies:

Privilege	Action
Create	Create hierarchies. Before you assign the read privilege for hierarchies, assign the following privileges: - The read privilege for reference data sets and code lists. - The read and update privileges for hierarchies.
Read	View hierarchies and their properties. Before you assign the read privilege for hierarchies, ensure the custom user role has the read privilege for reference data sets.
Update	Edit hierarchies and their properties. Before you assign the update privilege for hierarchies, ensure the custom user role has the read privilege for reference data sets, code lists, and hierarchies.
Delete	Delete hierarchies. Before you assign the delete privilege for hierarchies, assign the following privileges: - The read privilege to reference data sets and code lists. - The read and update privileges for hierarchies.

Configure privileges to access a code list

You can configure privileges for custom user roles to control access to the metadata and data of a code list. These privileges determine whether a custom user role can create, read, update, and delete code list attributes, basic and advanced rule associations for those attributes, and code values. You can also control which specfic attributes the custom user role can access.

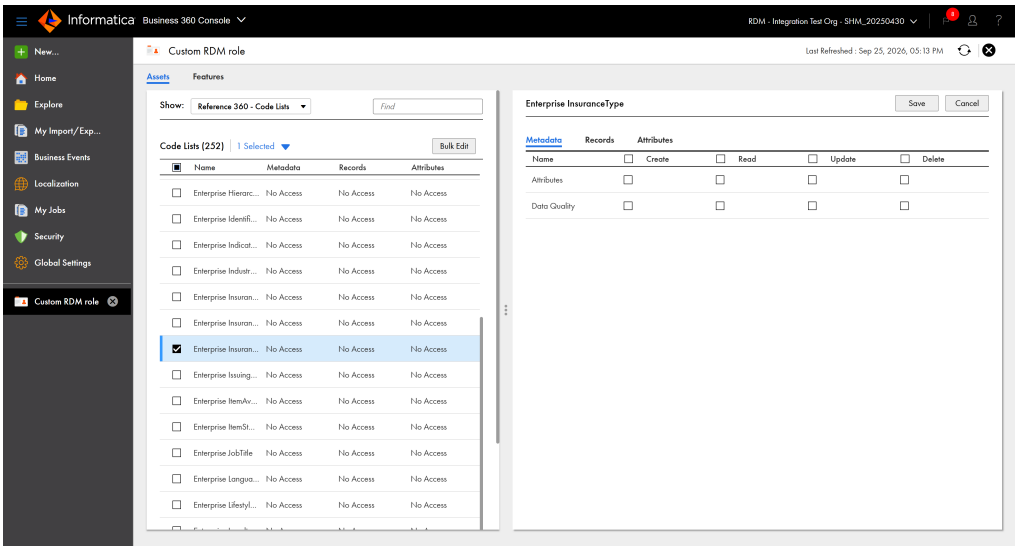
1. In Business 360 Console, on the **Security** page, click the custom user role for which you want to configure privileges.

   The user role details page lists all the assets in the organization.

2. On the **Assets** tab, select **Reference 360 - Code Lists** from the **Show** list.

   The left pane displays the list of code lists.

The following image shows the list of code lists available in the organization:

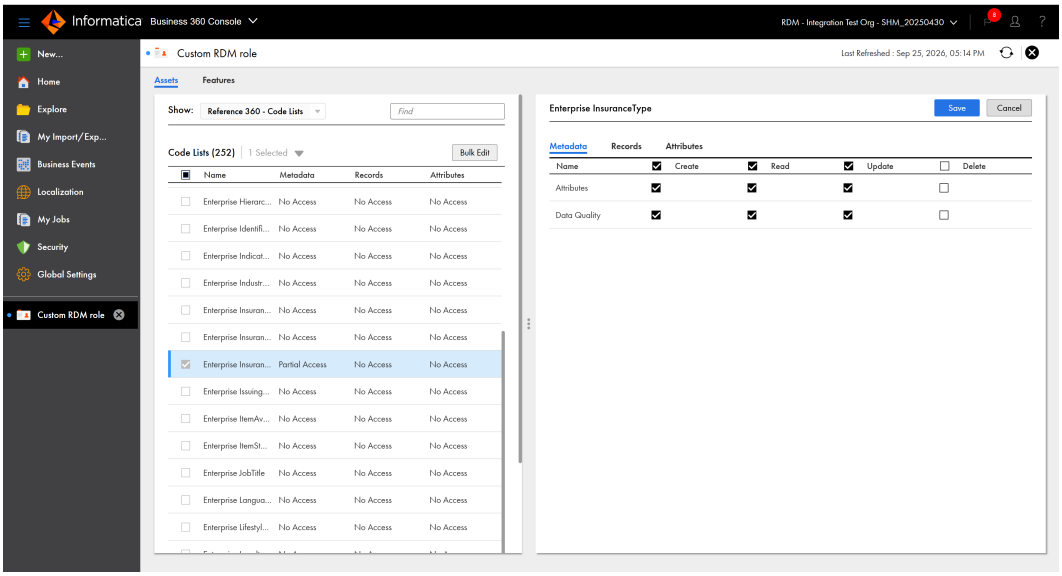


3. Select the code list for which you want to configure metadata and data privileges.
4. Select the permissions on the following tabs:
   - Metadata. Specifies whether the custom user role can create, read, update, and delete attributes in the code list, and basic and advanced rule associations for the code list attributes. For more information about configuring metadata privileges, see "Configure metadata privileges for a code list" on page 66.
   - Records. Specifies whether the custom user role can create, read, update, and delete code values in the code list. You can also specify whether the custom user role can import code values to the code list. For more information about configuring record privileges, see "Configure record privileges for a code list" on page 69.
   - Attributes. Specifies whether the custom user role can create, read, update, and delete values for specific attributes in the code list. For more information about configuring attribute privileges, see "Configure attribute privileges for a code list" on page 70.

Configure metadata privileges for a code list

The metadata privileges specify whether custom user roles can create, read, update, and delete attributes and configure basic and advanced rule associations for them.

The following image shows the metadata privileges for a custom user role:



You can assign one or more privileges for the following metadata:

- "Attribute privileges" on page 67
- "Data Quality privileges" on page 68

Note: Assign read privilege to reference data sets and code lists before you assign other privileges. Without the read privilege, other privileges might not behave as expected.

Attribute privileges

You can configure attribute privileges for custom user roles on the Metadata tab of a code list. The privileges determine whether a custom user role can create, read, update, and delete attributes in the code list. Based on the assigned privileges, the custom user role has complete access, partial access, or no access to the attributes of the code list.

The following table describes the actions that the custom user role can perform based on the assigned attribute privileges:

Privilege	Action
Create	Create custom attributes. Before you assign the create privilege for attributes, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read privilege for attributes.
Read	View predefined and custom attributes. Before you assign the read privilege for attributes in the code list, assign the read privilege for reference data sets and code lists.

Privilege	Action
Update	Edit the details of the predefined and custom attributes. Before you assign the update privilege for attributes in the code list, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read privilege for attributes.
Delete	Delete custom attributes. Before you assign the delete privilege for attributes in the code list, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read and update privileges for attributes. You can delete code list attributes configured with rule associations even if you don't have the delete privilege for the Data Quality metadata.

Data Quality privileges

You can configure data quality privileges for custom user roles on the Metadata tab of a code list. The privileges determine whether a custom user role can create, read, update, and delete basic and advanced rule associations for attributes. Based on the assigned privileges, the custom user role has complete, partial, or no access to the rule associations.

Prerequisites

Before you configure the data quality privileges, complete the following prerequisites:

- Assign data quality privileges for the custom user role in Administrator. For more information about data quality privileges, see Prerequisites in the Business 360 Console help.
- Assign at least read privilege to the attributes on the Metadata tab of a code list. Without the read privilege, the custom user role can't view the Basic Rules tab and the Advanced Rules tab for the selected attribute.

The following table describes the actions that the custom user role can perform based on the assigned data quality privileges:

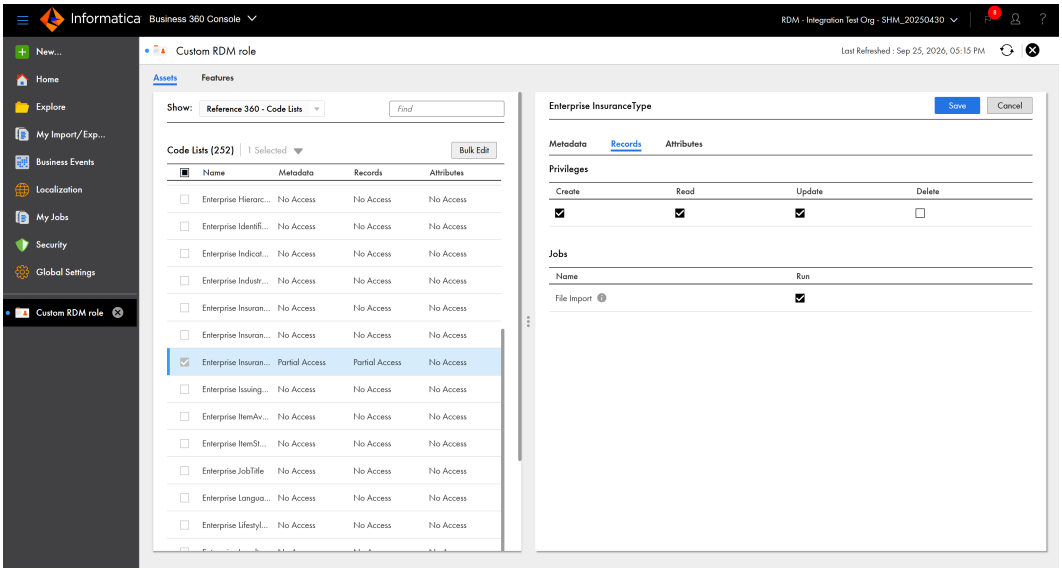
Privilege	Action
Create	Add basic and advanced rule associations. Before you assign the create privilege for rule associations, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read and update privileges for attributes. - The read privilege for rule associations.
Read	View basic and advanced rule associations on the Basic Rules and Advanced Rules tabs for the selected attribute. Before you assign the read privilege for rule associations, assign the read privilege for reference data sets, code lists, and attributes.

Privilege	Action
Update	Edit basic and advanced rule associations, and change the order of execution of basic rule associations for the selected attribute. Before you assign the update privilege for rule associations, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read and update privileges for attributes. - The read privilege for rule associations.
Delete	Delete basic rule associations. Before you assign the delete privilege for rule associations, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read and update privileges for attributes. - The read and update privileges for rule associations. **Note:** You can disable the advanced rule associations, but you can't delete them. To delete the advanced rule associations, set the delete privilege for Data Quality in the Administrator.

Configure record privileges for a code list

The record privileges specify whether custom user roles can create, read, update, and delete code values in the code list. The privileges also determine whether a custom user role can import code values into the code list.

The following image shows the record privileges for a custom user role:



Privileges

You can configure privileges to specify whether custom user roles can create, read, update, and delete code values in the code list.

The following table describes the actions that a custom user role can perform based on the assigned privileges:

Privilege	Action
Create	Create code values in the code list. Before you assign the create privilege for code values, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read privilege for code values. - The workflow requester privilege. - The Business360ProcessExecutor role in Administrator.
Read	View the details of code values in the code list. Before you assign the read privilege for code values in the code list, assign the read privilege for reference data sets and code lists.
Update	Edit the details of code values in the code list. Before you assign the update privilege for code values, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read privilege for code values. - The workflow requester privilege. - The Business360ProcessExecutor role in Administrator.
Delete	Delete code values in the code list. Before you assign the delete privilege for code values, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read and update privileges for code values. - The workflow requester privilege. - The Business360ProcessExecutor role in Administrator.

Jobs

You can configure privileges to specify whether custom user roles can import code values into the code list.

To authorize a custom user role to import code values into the code list, complete the following prerequisites in Business 360 Console:

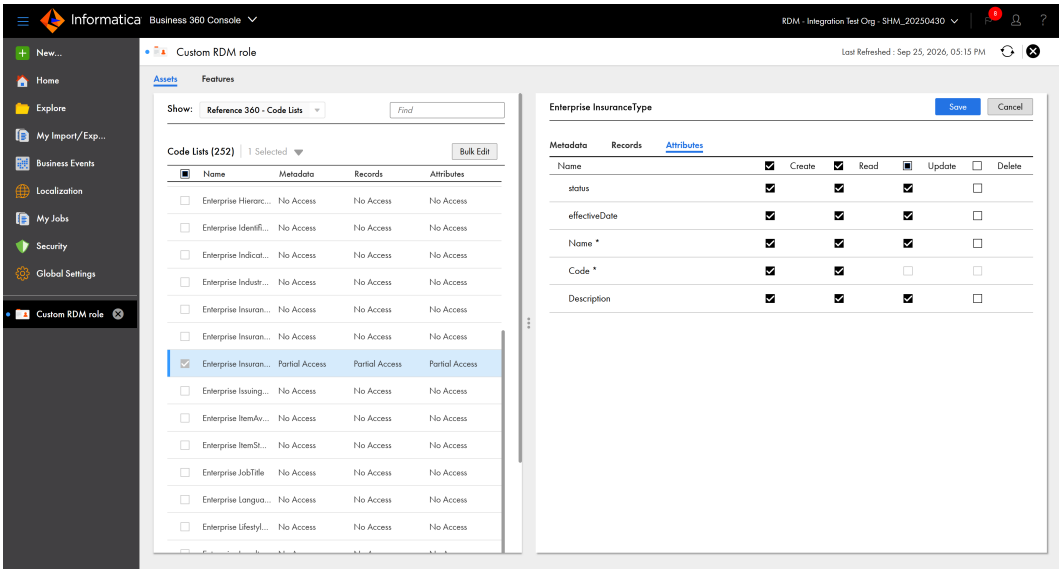
- Assign the read privilege for code lists.
- Assign the read and update privileges to code values of the code list.
- Assign the Workflow Requester privilege for the custom user role. For more information about configuring workflow privileges, see "Configure workflow privileges for custom user roles" on page 80.
- Select **Run** for the File Import job in the **Jobs** section.

Note: To access the **My Jobs** page, assign the Business360ProcessExecutor role in Administrator.

Configure attribute privileges for a code list

The attribute privileges specify whether custom user roles can create, read, update, and delete values for each attribute of the code list. Based on the assigned privileges, the custom user role has complete access, partial access, or no access to the attributes of the code list.

The following image shows the attribute privileges for a custom user role:



The following table describes the actions that a custom user role can perform based on the assigned privileges:

Privilege	Action
Create	Add values to the attributes in the code list. Before you assign the create privilege for the attributes in the code list, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read privilege to the code lists at the record and attribute levels.
Read	View values of the attributes in the code list. Before you assign the read privilege for the attributes in the code list, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read privilege to the code lists at the record and attribute levels.
Update	Edit values of the attributes in the code list. Before you assign the update privilege for the attributes in the code list, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read and update privileges to the code lists at the record and attribute levels. - The workflow requester privilege.
Delete	Delete values of the attributes in the code list. Before you assign the update privilege for the attributes in the code list, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read, update, and delete privileges to the code lists at the record and attribute levels. - The workflow requester privilege.

Configure privileges for multiple code lists

You can configure privileges for custom user roles to control access to the metadata and data of multiple code lists instead of configuring privileges for each code list individually.

Before you configure privileges to control access to the metadata and data of multiple code lists, assign read privilege to reference data sets and code lists. For more information about configuring privileges to reference data sets and code lists, see "Configure privileges for reference data sets" on page 62 and "Configure privileges for code lists" on page 63.

1. In Business 360 Console, on the **Security** page, click the custom user role for which you want to configure privileges.

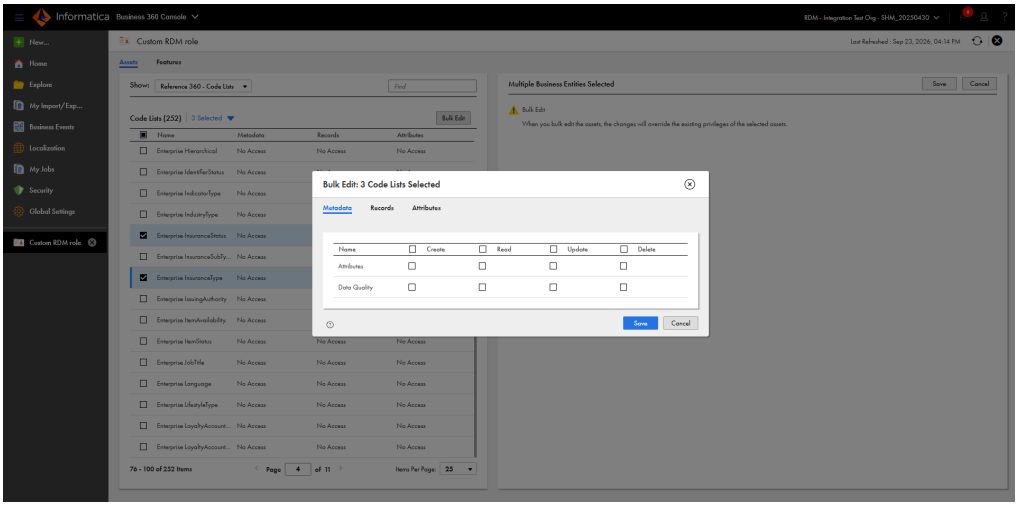
   The user role details page lists all the assets in the organization.

2. On the **Assets** tab, select **Reference 360 - Code Lists** from the **Show** list.

   The left pane displays the list of code lists.

3. Select the code lists for which you want to configure privileges, and click **Bulk Edit**.

   The **Bulk Edit** dialog box appears.
   The following image shows the Bulk Edit dialog box with the privileges that you can assign for the selected code lists:



4. In the **Bulk Edit** dialog box, assign the required privileges for the selected code lists.

   For more information about configuring privileges for code lists, see "Configure privileges to access a code list" on page 65.

   Note: You can't assign privileges to specific attributes of the selected code lists.

5. Click **Save**.

The selected privileges override the privileges of the code lists.

Configure privileges to access a crosswalk

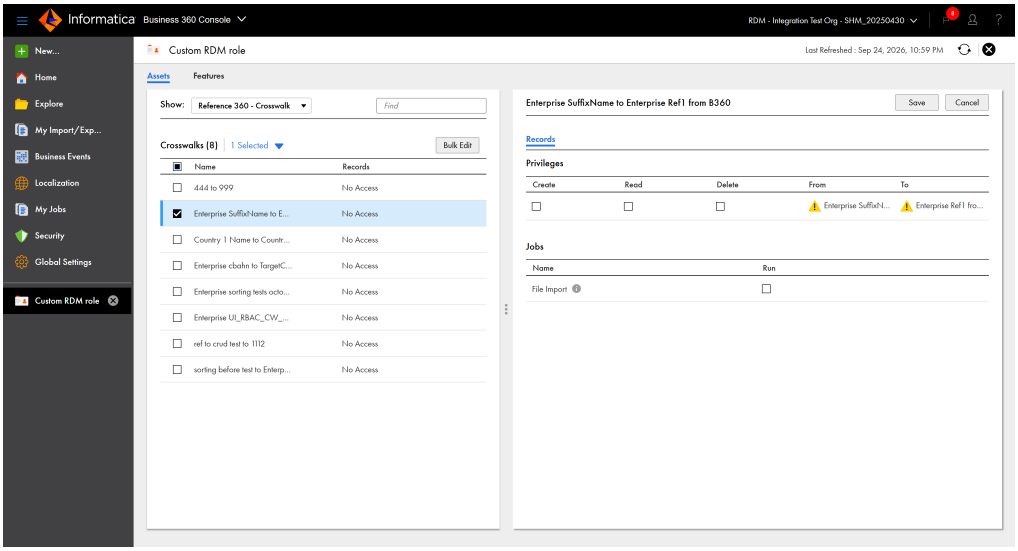
You can configure privileges for custom user roles to access a crosswalk. These privileges control whether the custom user role can create, read, and delete value mappings in the crosswalk. They also control whether the custom user role can import value mappings into the crosswalk.

1. In Business 360 Console, on the **Security** page, click the custom user role for which you want to configure privileges.

   The user role details page lists all the assets in the organization.

2. On the **Assets** tab, select **Reference 360 - Crosswalk** from the **Show** list.

   The left pane displays the list of crosswalks.
   The following image shows the list of crosswalks available in the organization:



3. Select the crosswalk for which you want to configure data privileges.
4. Select the required permissions on the **Records** tab:

   Note: Assign the read privilege to the code lists at the record and attribute levels before you assign other privileges. Without the read privilege, other privileges might not behave as expected. If you don't assign read privileges for the code lists, a warning icon appears before the names of the source and target code lists of the crosswalk.

   - Privileges. Specifies whether the custom user role can create, read, and delete value mappings in the crosswalk.
   - Jobs. Specifies whether the custom user role can import value mappings into the crosswalk.

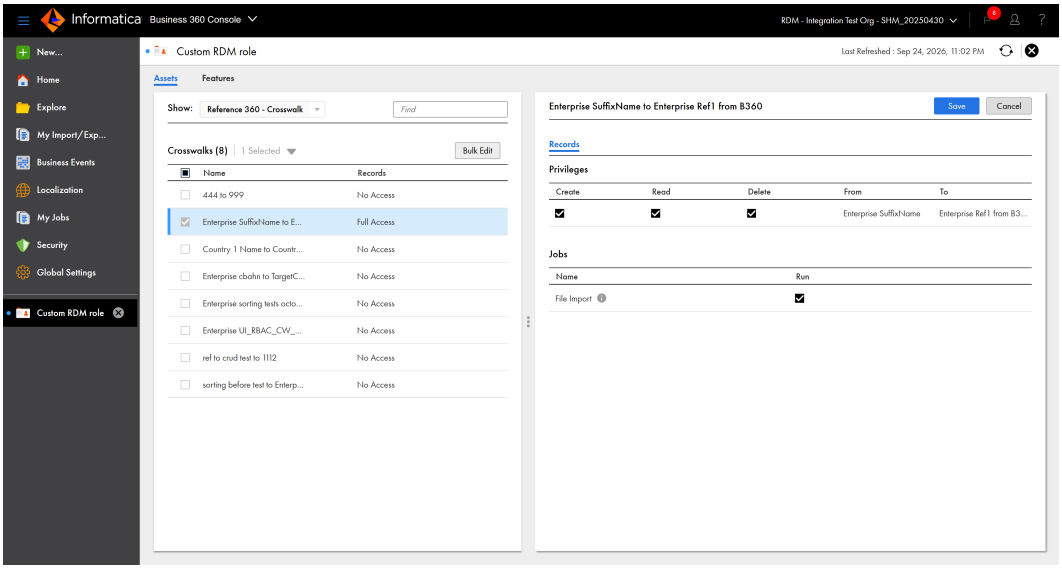
     Note: To import value mappings into the crosswalk, ensure that you assign the required record-level privileges for the code lists.

   For more information about configuring record privileges for a crosswalk, see "Configure record privileges for a crosswalk" on page 73.

Configure record privileges for a crosswalk

The record privileges specify whether custom user roles can create, read, and delete value mappings in a crosswalk. The privileges also determine whether a custom user role can import value mappings into the crosswalk.

The following image shows the record privileges for a custom user role:



Privileges

You can configure privileges to specify whether custom user roles can create, read, and delete value mappings in the crosswalk.

The following table describes the actions that a custom user role can perform based on the assigned privileges:

Privilege	Action
Create	Create value mappings in the crosswalk. Before you assign the create privilege for value mappings in the crosswalk, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read privilege for crosswalks. - The read privilege for code values of source and target code lists. - The workflow requester privilege. - The Business360ProcessExecutor role in Administrator.
Read	View value mappings in the crosswalk. Before you assign the read privilege for value mappings in the crosswalk, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read privilege for crosswalks. - The read privilege for code values of source and target code lists.
Delete	Delete value mappings in the crosswalk. Before you assign the delete privilege for value mappings in the crosswalk, assign the following privileges: - The read privilege for reference data sets. - The read privilege for code lists. - The read privilege for crosswalks. - The read and update privileges for code values of source and target code lists. - The read and update privileges for value mappings. - The workflow requester privilege. - The Business360ProcessExecutor role in Administrator.

Jobs

You can configure privileges to specify whether custom user roles can import value mappings into the crosswalk.

To authorize a custom user role to import value mappings into the crosswalk, complete the following prerequisites in Business 360 Console:

- Assign the read privilege for crosswalks.
- Assign the read and update privileges for code values of source and target code lists.
- Assign the read and update privileges for the value mappings in the crosswalk.
- Assign the Workflow Requester privilege to the custom user role. For more information about configuring workflow privileges, see "Configure workflow privileges for custom user roles" on page 80.
- Select **Run** for the File Import job in the **Jobs** section.

Note: To access the **My Jobs** page, assign the Business360ProcessExecutor role in Administrator.

Configure privileges for multiple crosswalks

You can configure privileges for custom user roles to control access to multiple crosswalks instead of configuring privileges for each crosswalk individually.

Before you configure privileges to control access to multiple crosswalks, assign read privilege to reference data sets, code lists, and crosswalks. For more information about configuring privileges to reference data sets, code lists, and crosswalks, see "Configure privileges for reference data sets" on page 62, "Configure privileges for code lists" on page 63, and "Configure privileges for crosswalks" on page 63.

1. In Business 360 Console, on the **Security** page, click the custom user role for which you want to configure privileges.

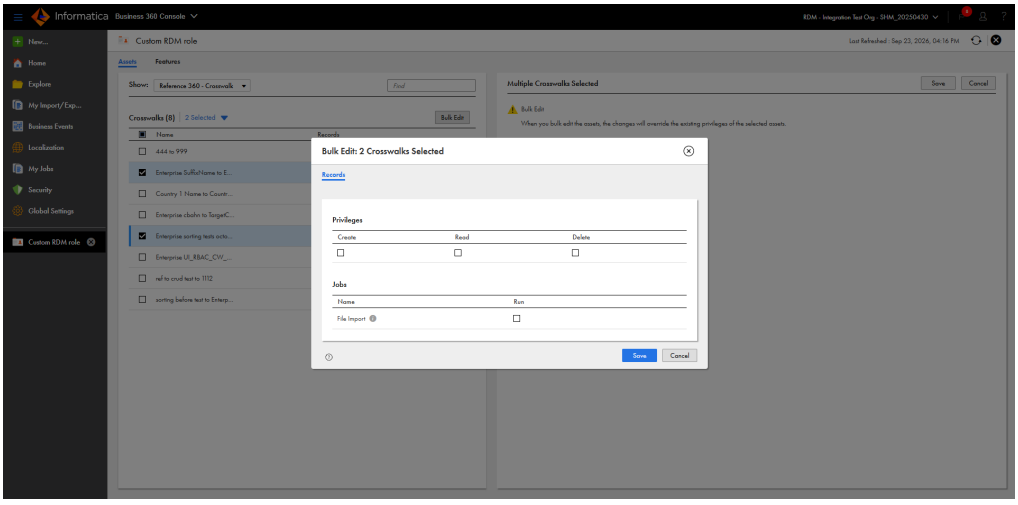
   The user role details page lists all the assets in the organization.

2. On the **Assets** tab, select **Reference 360 - Crosswalk** from the **Show** list.

   The left pane displays the list of crosswalks.

3. Select the crosswalks for which you want to configure privileges, and click **Bulk Edit**.

   The **Bulk Edit** dialog box appears.
   The following image shows the Bulk Edit dialog box with the privileges that you can assign for the selected crosswalks:



4. In the **Bulk Edit** dialog box, assign the required privileges for the selected crosswalks.

   For more information about configuring privileges for crosswalks, see "Configure privileges to access a crosswalk" on page 72.

5. Click **Save**.

The selected privileges override the privileges of the crosswalks.

Configure privileges to access a hierarchy

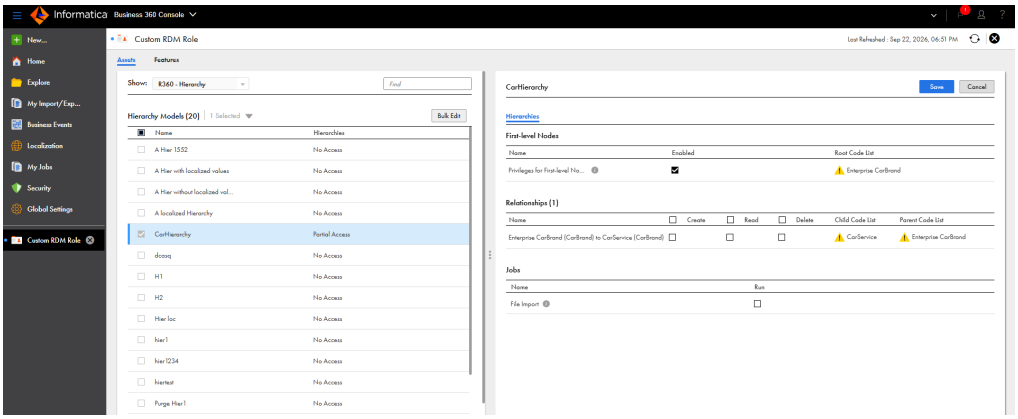
You can configure privileges for custom user roles to control access to a hierarchy. These privileges control the permissions to create, read, update, and delete hierarchies based on the hierarchy model, the first-level nodes, and the relationships within the hierarchies. You can also configure privileges to import hierarchy data.

1. In Business 360 Console, on the **Security** page, click the custom user role for which you want to configure privileges.

   The user role details page lists all the assets in the organization.

2. On the **Assets** tab, select **Reference 360 - Hierarchy** from the **Show** list.

   The left pane displays the list of hierarchy models.
   The following image shows the list of hierarchies available in the organization:



3. Select the hierarchy models for which you want to configure data privileges.

   The right pane displays the privileges for the selected hierarchy model.

4. Select the required permissions in the following sections:

   - First-level Nodes. Specifies whether a user with custom user role can add root records to a hierarchy. For more information about configuring first-level node privileges, see"Configure first-level nodes privileges" on page 77 for a hierarchy.
   - Relationships. Specifies whether a user with custom user role can add and manage the child records for each relationship in the hierarchy. For more information about configuring relationship privileges, see "Configure relationship privileges" on page 77 for a hierarchy
   - Jobs. Specifes whether a user with custom user role can import hierarchy data into Reference 360. For more information about file import jobs, see "Configure privileges for file import " on page 78 for a hierarchy.

   Note: Ensure that you assign at least the read privilege for the hierarchies before you assign other privileges. Without the read privilege, the user with the custom user role cannot view the hierarchies in the business application.

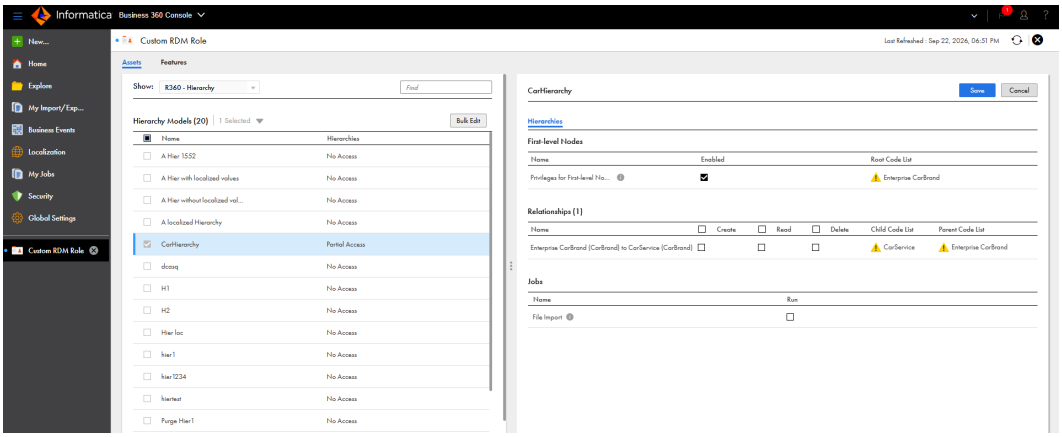
Configure first-level nodes privileges

The privilege specifies whether a user with custom user role has privileges to create, update, and delete for the first-level nodes of the hierarchy.

Select **Privileges for First-level Nodes** to enable privileges for the user to create, update, and delete for the first-level nodes of the hierarchy.

Note: If you don't assign the read privilege for hierarchies , a warning icon appears. For more information about configuring privileges to access reference data assets, see "Configure privileges to access reference data assets" on page 61.

The following image shows the warning icon for the code lists to which the user does not have the read privilege:



Configure relationship privileges

The relationships table lists all the relationships associated with the hierarchy model selected in the left pane and the corresponding privileges for each relationship. These privileges specify the code lists and the relationship type based on which the user with custom user role can add and manage child records for hierarchies.

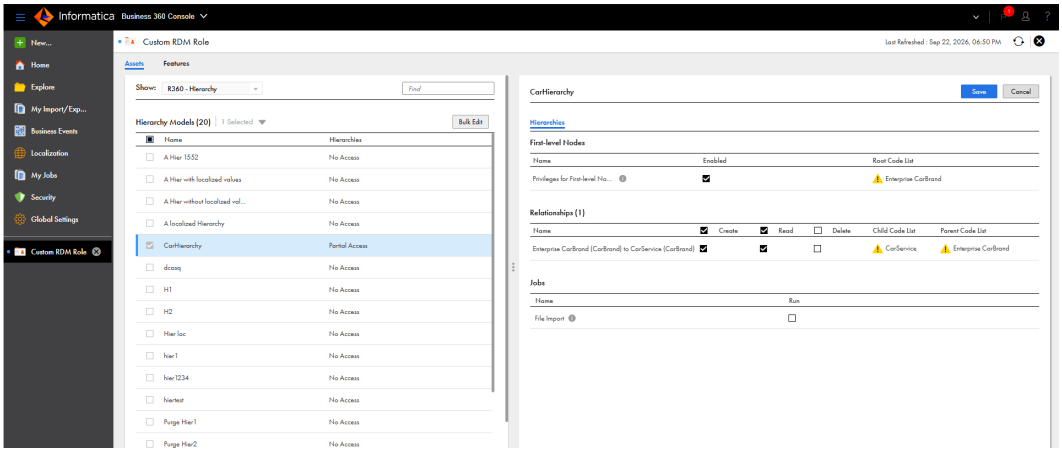
The following table describes the relationship privileges that you can enable:

Privilege	Set to Enabled
Create	Add child nodes to a hierarchy based on the relationship type.
Read	View child nodes of a hierarchy based on the relationship type.
Delete	Delete child nodes of a hierarchy based on the relationship type.

Note: If you use a searchable hierarchy as a hierarchy filter, ensure that you provide read access to all the relationships within the hierarchy. If you only provide read access to some relationships, the hierarchy doesn't appear in the filter criteria.

If you do not assign the read privilege for each code list associated with a relationship, a warning icon appears.

The following image shows the warning icon for the code lists to which the user does not have the read privilege:



Configure privileges for file import

You can configure privileges to specify whether a user with custom user role can import hierarchy data into Reference 360.

To authorize a user to import hierarchy data, complete the following prerequisites in Business 360 Console:

- Assign the read privilege for hierarchies and for each code list associated with a relationship.
- Assign the read privilege for the code values of the code lists associated with a relationship.
- Enable the privilege for the first-level nodes of the hierarchy.
- Assign the create and read privileges for the relationships that you want to import.
- Assign the Workflow Requester privilege for the custom user role. For more information about configuring workflow privileges, see "Configure workflow privileges for custom user roles" on page 80.
- Select **Run** for the File Import job in the **Jobs** section.

Note: To access the **My Jobs** page, assign the Business360ProcessExecutor role in Administrator.

Configure privileges for multiple hierarchies

You can configure privileges for custom user roles to control access to multiple hierarchies instead of configuring privileges for each hierarchy individually.

Before you configure privileges to control access to the metadata and data of multiple hierarchies, assign read privilege to reference data sets and code lists. For more information about configuring privileges to reference data sets and code lists, see "Configure privileges for reference data sets" on page 62 and "Configure privileges for code lists" on page 63.

1. In Business 360 Console, on the **Security** page, click the custom user role for which you want to configure privileges.

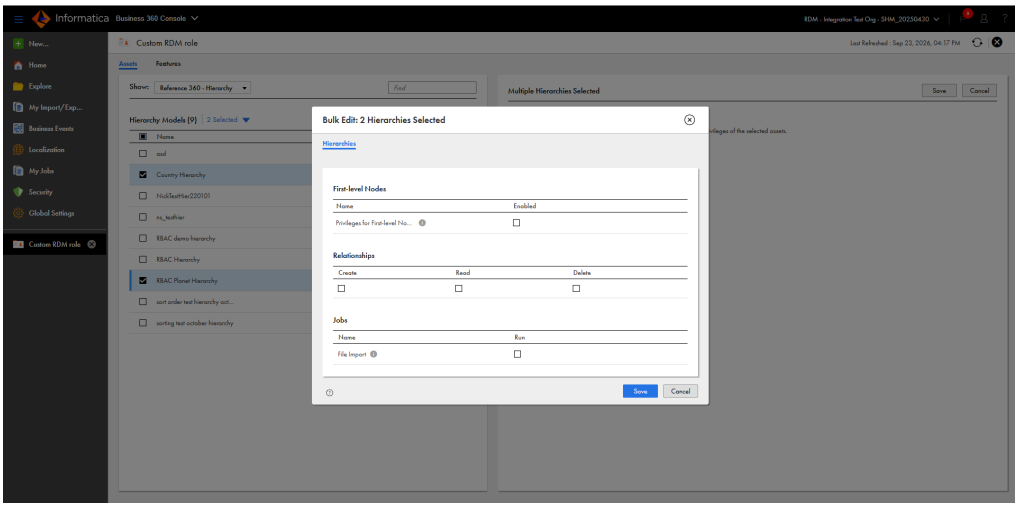
   The user role details page lists all the assets in the organization.

2. On the **Assets** tab, select **Reference 360 - Hierarchy** from the **Show** list.

   The left pane displays the list of hierarchy models.

3. Select the hierarchy models for which you want to configure privileges, and click **Bulk Edit**.

   The **Bulk Edit** dialog box appears.
   The following image shows the Bulk Edit dialog box with the privileges that you can assign for the selected hierarchy models:



4. In the **Bulk Edit** dialog box, assign the required privileges for the selected hierarchy models.

   For more information about configuring privileges for hierarchies, see "Configure privileges to access a hierarchy" on page 76.

   Note: When you assign privileges for multiple hierarchies, you can assign the privileges for all the relationships associated with the hierarchy models and not for individual relationships. To assign privilege for each relationship within the hierarchy model, modify privileges for an individual hierarchy model without using the Bulk Edit option.

5. Click **Save**.

The selected privileges override the privileges of the hierarchies.

Configure privileges to view the history of records

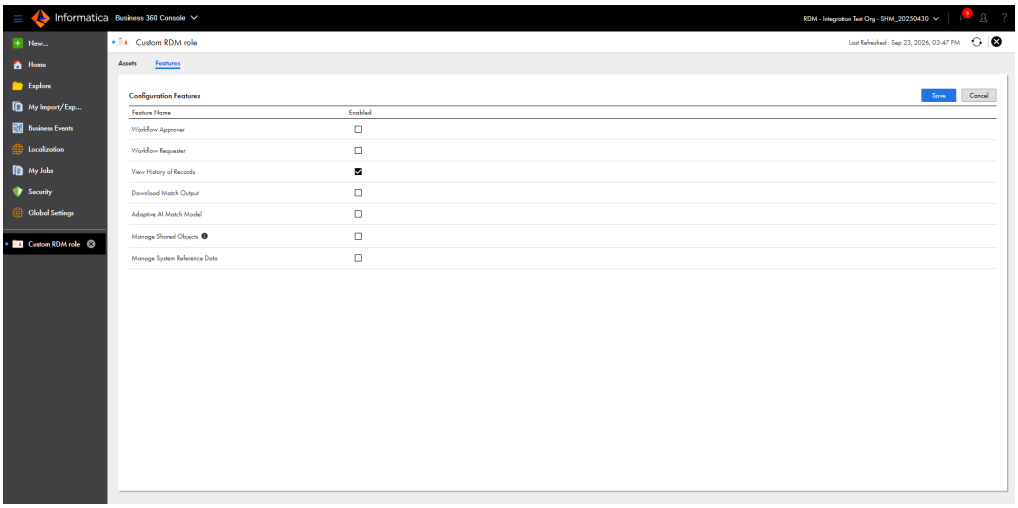
You can configure privileges for custom user roles to view the history of reference data sets, code lists, crosswalks, hierarchies, code values in a code list, and value mappings in a crosswalk.

Before you configure privileges to view the history of records, complete the following prerequisites:

- Assign the read privilege for reference data sets, code lists, crosswalks, and hierarchies.
- Assign the read privilege for code values in a code list.
- Assign the read privilege for value mappings in a crosswalk.
- Assign the read privilege for relationships in a hierarchy.

1. In Business 360 Console, on the **Security** page, click the custom user role for which you want to configure privileges.
2. On the **Features** tab, select **View History of Records**.

The following image shows the Features tab:



3. Click **Save**.

After you configure privileges to view the history of records, the custom user roles can also view the code values that existed at a specific point in time in a code list and a hierarchy.

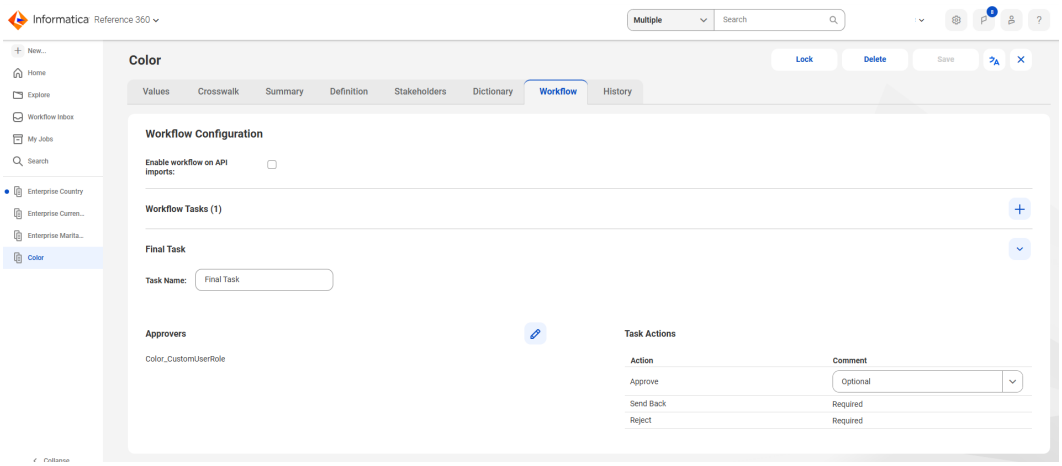
Configure workflow privileges for custom user roles

You can configure workflow privileges for custom user roles to control who can request asset changes and who can approve them.

Requesters perform tasks that trigger workflows in Reference 360, such as creating a draft or sending changes for approval. Approvers review and approve changes to assets.

To view the **Workflow** tab, a user must have a predefined user role other than Reference 360 User, or a custom user role with the workflow privileges that you assign to it.

The following image shows the Workflow tab:



To configure workflow privileges, use Administrator and Business 360 Console.

Rules and guidelines for configuring workflow privileges for custom user roles

Consider the following rules and guidelines when you configure workflow privileges for custom user roles in your organization.

- You can select a user with any predefined user role other than the Reference 360 User role as a workflow approver. If your organization uses custom user roles, select users who have at least one custom role with the Workflow Approver privilege.
- Before you create reference data assets, assign the required predefined user role to at least one user in your organization. If no user has the required role when you create an asset, Reference 360 creates the workflow configuration without an approver and displays an error until you add one.
  The following table lists the role that each asset type requires:

Asset Type	Predefined User Role
Code list	Business Steward
Crosswalk	Business Steward
Hierarchy	Planner

- A workflow is complete when an approval task has at least one eligible approver.
- When the workflow is incomplete, Reference 360 blocks the following actions:
  - Locking of assets, editing, adding, deleting, and cloning of attribute values.
  - Editing data on the Values, Hierarchy, and Crosswalk tab.
  - Importing a draft version of code lists or crosswalks through REST APIs.
- Approvers of a workflow that's in progress remain valid even if their privileges change. Changes that you make to approvers apply only to workflows that start after you make the changes.

Workflow Requester privilege

When you assign the Workflow Requester privilege to a custom user role, users can trigger workflows in Reference 360.

Users with read privilege for an assets and whose only roles are custom user roles with workflow privileges have read-only access to the Workflow tab. They can't add approvers or modify task levels.

After you assign the Workflow Requester privilege to a custom user role, users with the custom user role and the appropriate asset-level and record privileges can create a draft, update attribute values, send changes for approval, and withdraw approval requests.

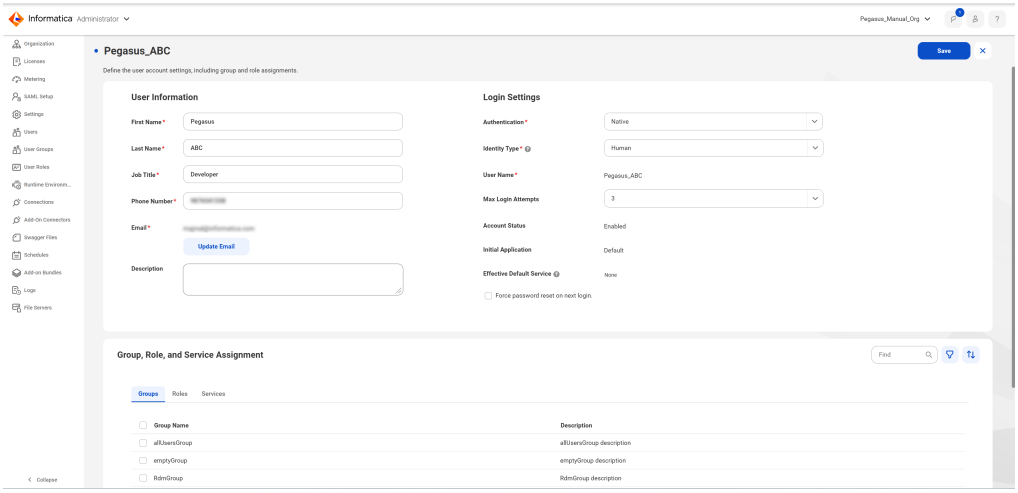
Configure Workflow Requester privilege

You can configure Workflow Requester privileges for custom user roles to trigger workflows.

1. In Administrator, click **Users**.

   The list of available users appears.
2. Select the user to assign user roles.

   The user details page appears.
   The following image displays the user details page:



3. For the user to view the workflow inbox, assign a custom user role and the **Business360ProcessExecutor** role.
4. Click **Save**.
5. Click **User Roles**.

   The user roles page with the list of predefined and custom user roles appears.
6. Select the custom user role you selected in 3.

   The user role details page appears.
7. Select **Human Tasks** in the **Services** list, and enable the **View Tasks** feature on the **Features** tab.

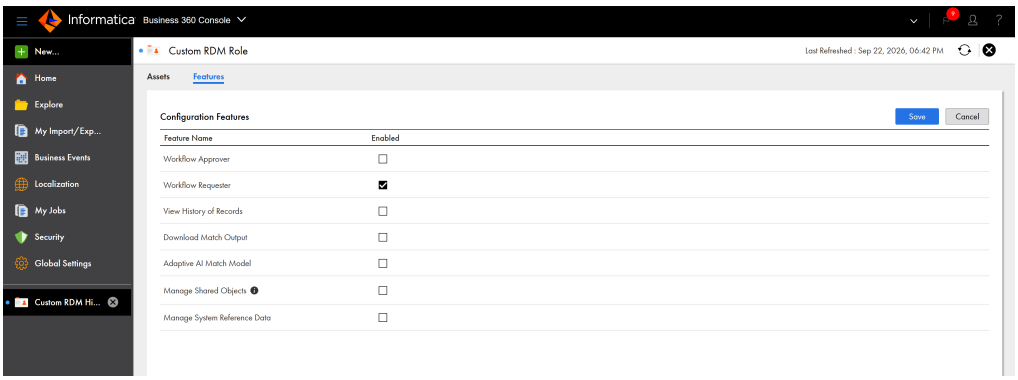
   Users with the custom user role can see and use the Submit and Discard Draft buttons when they view an asset. Users can submit a draft of the asset for approval or discard the draft.

   For more information about the View Tasks privilege, see "Prerequisites" on page 58.
8. Click **Save**.
9. In Business 360 Console, click **Security**, and then click the custom user role you selected in 3.

   The user role details page appears.
10. Select the asset and assign the required privilege for the asset.

    For more information about the privileges, see "Configure privileges to access reference data assets" on page 61"Configure privileges to access a code list" on page 65, "Configure privileges to access a crosswalk" on page 72, and "Configure privileges to access a hierarchy" on page 76.
11. On the **Features** tab, select **Workflow Requester**.

    The following image shows the Features tab on the user role details page:

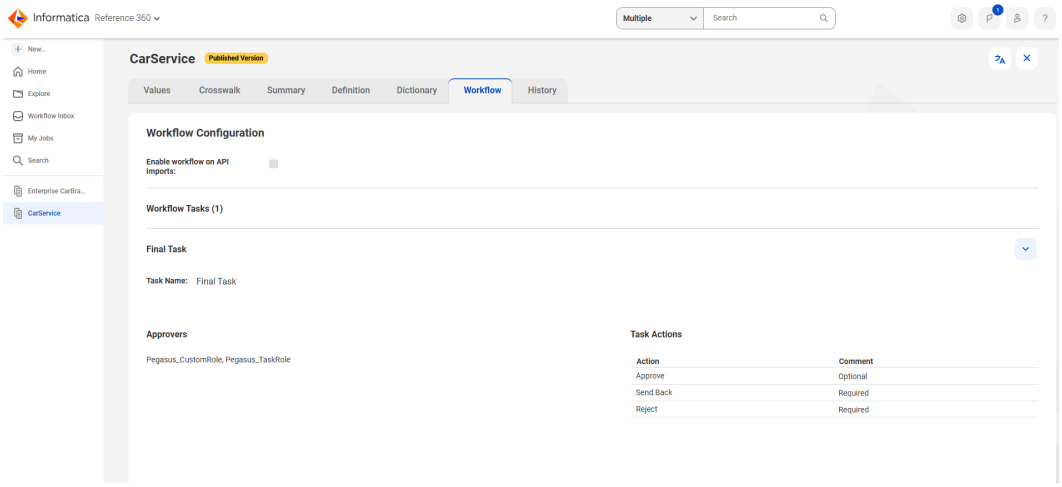


Workflow Approver privilege

When you assign the Workflow Approver privilege to a custom user role, users can review and approve changes to assets.

Custom user roles with the Workflow Approver privilege appear in the **Approvers** list when you change workflow configuration for the assets.

The following image shows the Workflow tab and the list of approvers:



When you create an asset, Reference 360 adds default approvers to the approval task. These approvers are users with Business Steward role for code lists and crosswalks or Planner role for hierarchies. A workflow is complete when an approval task has at least one eligible approver.

You can select a user with any predefined user role other than the Reference 360 User role as a workflow approver. If your organization uses custom user roles, select users who have at least one custom role with the Workflow Approver privilege.

After you assign the Workflow Approver privilege to a custom user role, users with the custom user role can review and approve or reject changes proposed by other users. For more information about reviewing data by approvers, see "Reviewing tasks" on page 178.

To enable the Claim, Approve, Reject, Send Back, and Release buttons in the Reference 360 workflow inbox, after you assign the View Tasks and Workflow Approver privileges for the custom user role, enable the following privileges for the assets:

Asset	Privilege
Code list	- Asset-level read privilege for code lists. - Record-level read privilege for the code list.
Crosswalk	- Asset-level read privilege for crosswalks. - Record-level read privilege for crosswalks. - Record-level read privilege for the source and target code lists of the crosswalk. - Record-level read privilege for the value mappings in the crosswalk.
Hierarchies	- Asset-level read privilege for hierarchies. - Record-level read privilege for the root code list. - Enable privileges for first-level nodes of the hierarchy.

For more information about the View Tasks privilege, see "Prerequisites" on page 58.

Configure privileges for the workflow inbox

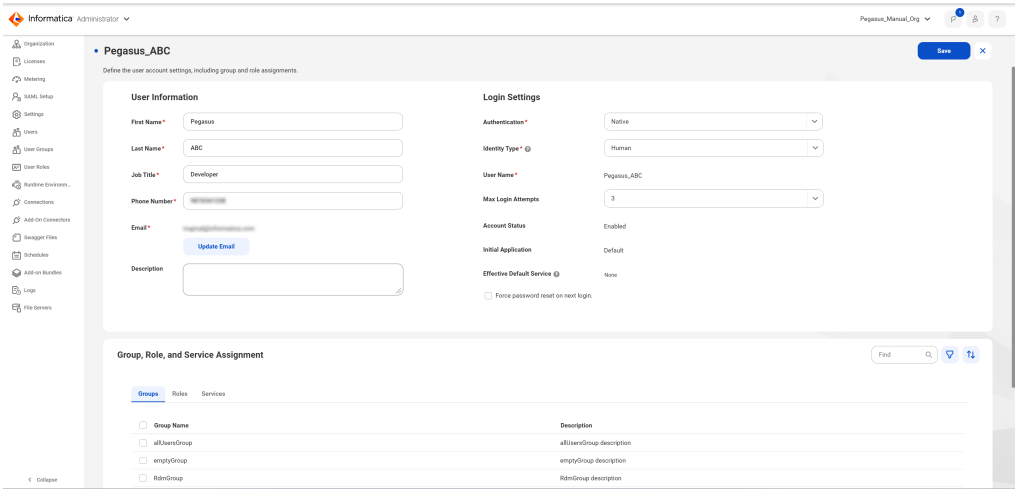
You can configure Workflow Approver privileges for custom user roles. You can also configure specific privileges for approvers to approve, reject, or send tasks back to the requester in the workflow inbox.

1. In Administrator, click **Users**.

   The list of available users appears.

2. Select the user to assign user roles.

   The user details page appears.
   The following image displays the user details page:



3. For the user to view the workflow inbox, assign the custom user role and the **Business360ProcessExecutor** role.
4. Click **Save**.
5. Click **User Roles**.

   The user roles page with the list of predefined and custom user roles appears.

6. Select the custom user role you selected in 3.

   The user role details page appears.

7. Select **Human Tasks** in the **Services** list, and enable the **View Tasks** feature on the **Features** tab.
8. Click **Save**.
9. In Business 360 Console, click **Security**, and then click the custom user role for the user that you selected in 3.

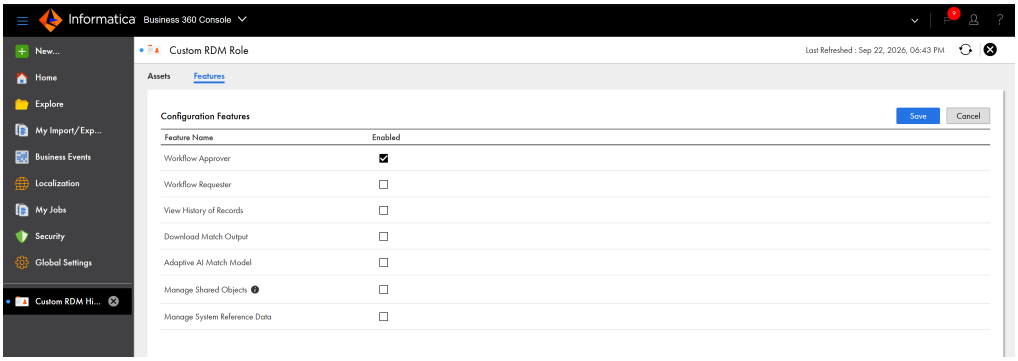
   The user role details page appears.

10. Select the assets for which you want to assign the Workflow Approver privilege.

11. On the **Features** tab, select **Workflow Approver**.

    Note: After you edit the Workflow Approver privilege for the assets, the privilege changes can take at least 10 minutes to take effect. To load the latest Workflow Approver privilege, refresh the Reference 360 application before you configure workflows.

    The following image shows the Features tab on the user role details page:



Revoke workflow privileges

You can revoke workflow privileges for custom user roles in Business 360 Console.

Before you revoke the Workflow Requester or Workflow Approver privilege for a custom user role, ensure that the user has no assigned tasks. You can check if any tasks are assigned to the user in the workflow inbox.

Configure privileges to manage multilingual assets

Custom user roles have read-only access to the Localization Languages code list that stores the languages available for localized data.

When a custom user role has the read privilege for a reference data set and code list, Reference 360 automatically assigns the read privilege for the Localization Languages code list.

To enable a custom user role manage multilingual attributes, assign the read privilege for a reference data set that has multilingual attributes. You can then assign the read privilege for the code list of the reference data set.

For more information about configuring privileges to access reference data sets and code lists that contain multilingual attributes, see "Configure privileges for reference data sets" on page 62, and "Configure privileges for code lists" on page 63.

SAML single sign-on

You can enable single sign-on (SSO) capability so that users can access their organization without the need to enter login information. You can use SSO for user authentication or for both authentication and authorization in an organization. You configure SSO capability for an organization on the **SAML Setup** page.

Single sign-on to IDMC is based on the Security Assertion Markup Language (SAML) 2.0 web browser single sign-on profile. The SAML web browser single sign-on profile consists of the following entities:

Identity provider

An entity that manages authentication information and provides authentication services by using security tokens.

Service provider

An entity that provides web services to principals, for example, an entity that hosts web applications. IDMC is a service provider.

Principal

An end user who interacts through an HTTP user agent.

SAML 2.0 is an XML-based protocol that uses security tokens that contain assertions to pass information about a principal between an identity provider and a service provider. An assertion is a package of information that supplies statements made by a SAML authority. You can find more information about SAML on the Oasis web site: https://www.oasis-open.org

The process that occurs when a user enters the IDMC URL in a browser or launches IDMC through a chiclet differs based on whether the organization uses SAML SSO for authentication only or for both authentication and authorization.

SAML single sign-on for authentication only

When a user signs on to IDMC and the organization uses SAML SSO for user authentication only, the following process occurs:

1. IDMC sends a SAML authentication request to the organization's identity provider.
2. The identity provider confirms the user's identity and sends a SAML authentication response to IDMC. The authentication response includes a SAML token.
3. When IDMC receives the SAML authentication response from the identity provider, it completes the following tasks:
   - If the user exists, IDMC establishes the user session and logs the user in.
   - If the user does not exist and auto-provisioning of users is enabled, IDMC gets the user attributes from the SAML token, creates the user, and assigns the user the default role and the default group, if it is configured. IDMC establishes the user session and logs the user in.
   - If the user does not exist and auto-provisioning of users is disabled, IDMC fails the login.
4. When a user logs out of IDMC or the session times out, IDMC sends a SAML logout request to the identity provider.
5. The identity provider terminates the user session on the identity provider side.

SAML single sign-on for authentication and authorization

When a user signs on to IDMC and the organization uses SAML SSO for authentication and authorization, the following process occurs:

1. IDMC sends a SAML authentication request to the organization's identity provider.
2. The identity provider confirms the user's identity and sends a SAML authentication response to IDMC. The authentication response includes a SAML token.
3. When IDMC receives the SAML authentication response from the identity provider, it completes the following tasks:
   - If the user exists, IDMC gets the user roles, groups, and attributes from the SAML token. It finds the corresponding IDMC user roles and groups, and updates the user roles, if necessary. IDMC establishes the user session and logs the user in.

- If the user does not exist and auto-provisioning of users is enabled, IDMC gets the user roles, groups, and attributes from the SAML token and creates the user. IDMC establishes the user session and logs the user in. If the token contains no SAML role or group information, IDMC fails the login.
- If the user does not exist and auto-provisioning of users is disabled, IDMC fails the login.

4. When a user logs out of IDMC or the session times out, IDMC sends a SAML logout request to the identity provider.
5. The identity provider terminates the user session on the identity provider side.

SAML single sign-on requirements

SAML single sign-on for an IDMC organization requires an appropriate identity provider.

To set up SAML single sign-on for an organization, ensure that the following requirements are met:

- The system uses a SAML 2.0-based identity provider.

  Common identity providers include Microsoft Active Directory Federation Services (AD FS), Okta, SSOCircle, OpenLDAP, and Shibboleth. The identity provider should be configured to use either the DSA-SHA256 or RSA-SHA256 algorithm to generate the signature.
- The IDMC organization has the SAML based Single Sign-On license.
- You have access to the organization as an organization administrator to set up single sign-on.

Single sign-on restrictions

There are some restrictions for SAML single sign-on access to IDMC.

The following restrictions apply to SAML single sign-on access:

- Your license with the identity provider must remain valid to access IDMC through single sign-on.
- If the identity provider is down or IDMC servers can't reach it, users can't log in to IDMC through single sign-on.
- If the identity provider certificate used for SAML single sign-on to IDMC expires, users can't access IDMC through single sign-on.
- If your organization uses trusted IP address ranges, users can't log in to IDMC from an IP address that is not within the trusted IP address ranges.

User management with SAML single sign-on

The following rules apply to users and user accounts when you enable SAML single-sign on for IDMC:

- IDMC stores user information that passes from the identity provider such as first name and email address in the IDMC repository.
- You can create a regular user account with credentials in IDMC after you enable an organization for single sign-on, and the user credentials are saved in the IDMC repository. However, the user must log in to IDMC directly instead of using single sign-on.
- If you delete a user from IDMC, the user is deleted from the IDMC repository. The user is not deleted from the identity provider.

Configuring SAML single sign-on for IDMC

Configure SAML single sign-on (SSO) in IDMC to exchange authentication and authorization information with your identity provider.

Tip: To watch a video that describes the SAML configuration procedure, see https://youtu.be/4DewSNbvJBc?si=VAeGj5-5Oq0HbXtF.

SAML single sign-on requires an identity provider and service provider. Examples of identity providers include: Okta, Microsoft Azure AD, and PingFederate. In this scenario, the service provider is IDMC.

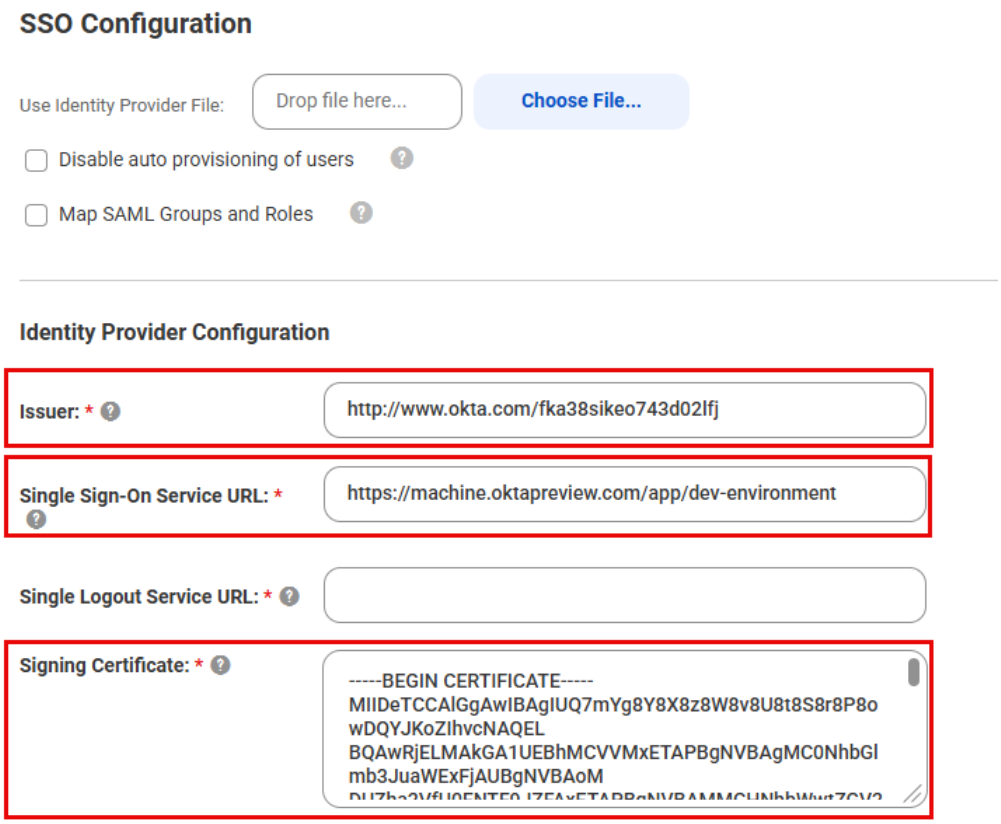
Before you can configure SAML SSO in IDMC, it must already be set up in your identity provider. For information about configuring two common identity providers, see the following articles on the Informatica network:

- HOW TO: Configure SAML SSO in Informatica Cloud using Microsoft Azure AD
- HOW TO: Configure SAML SSO using OKTA in IICS

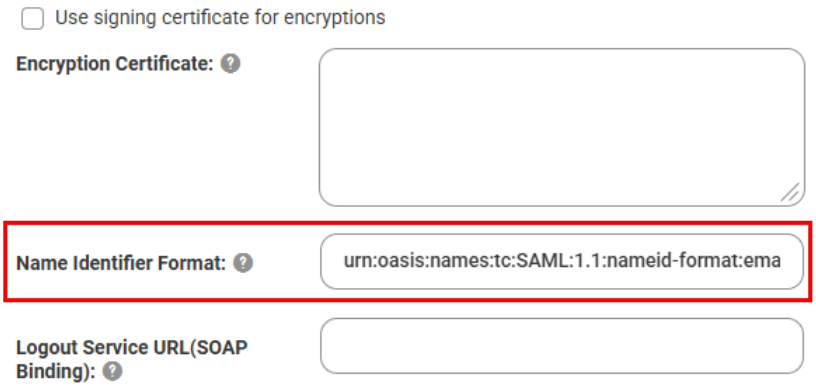
To configure SAML single sign-on for IDMC:

1. Contact your identity provider team for the following information:
   - Issuer
   - Single Sign-on Service URL
   - Signing Certificate
2. Enter the information into the corresponding fields in **Identity Provider Configuration** section of the **SAML Setup** page:
   - **Issuer**
   - **Single Sign-on Service URL**
   - **Signing Certificate**

The following images shows these fields on the **SAML Setup** page:

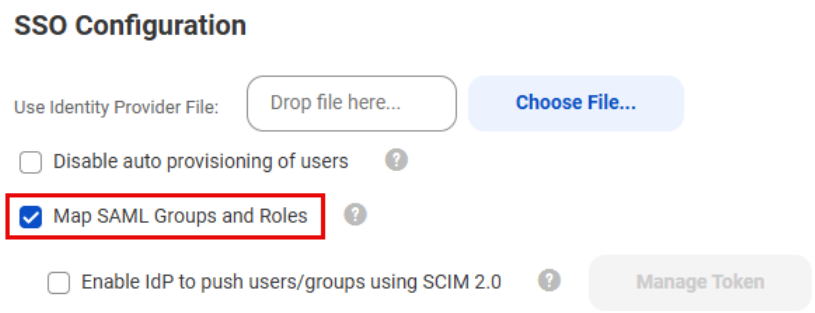


3. Enter the **Name Identifier Format** if possible, otherwise this can be added later. The following image shows the **Name Identifier Format** field:

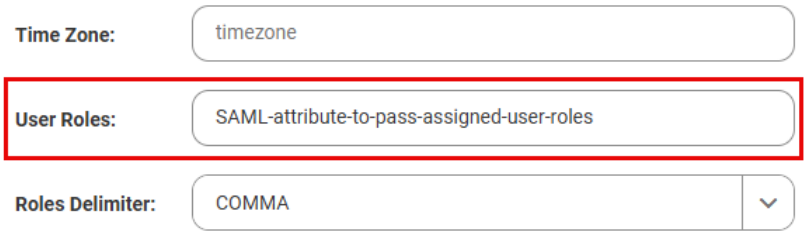


4. Click **Save**.
   IDMC generates the service provider metadata file and a unique token for your organization.
5. Click **Download Service Provider Metadata**. This downloads the file `iics_saml_sp_metadata.xml` to your machine. It contains information that your service provider needs to complete the configuration.
6. In the **Information** dialog box, note the URL for single sign-on access to your IDMC organization. For example:
   `https://dm-us.informaticacloud.com/ma/sso/<organization token>`

7. Send the `iics_saml_sp_metadata.xml` file to the identity provider administrator, along with the URL for single sign-on access.
8. Complete the **SAML Attribute Mapping** section. For more information, see "SAML attribute mapping properties" on page 93.
9. If you want to map SAML roles to specific roles in IDMC, perform the following steps:
   a. Enable **Map SAML Groups and Roles**. The following image shows the **SAML Setup** page with this option enabled:



   b. In the **User Roles** field, enter the SAML attribute used to pass the assigned user roles. The following image shows the **User Roles** field:

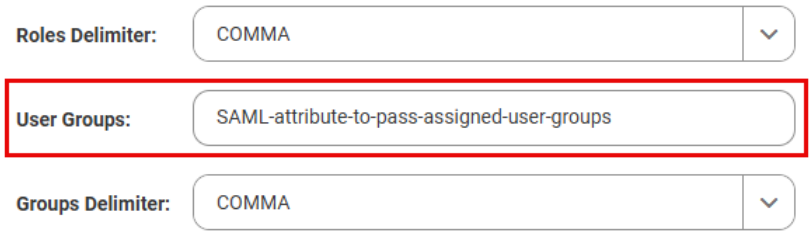


   For example, if "idp_roles" is the name of the attribute from the identity provider used to pass user role information, then enter **idp_roles**.
   c. Map the corresponding IDMC roles in the **SAML Role Mapping** tab. For more information, see "SAML role and group mapping properties" on page 95.
10. If you don't want to map specific roles and instead just use a default role, perform the following steps:
   a. Ensure that **Map SAML Groups and Roles** is not selected.
   b. In the **SAML Role Mapping** tab, scroll to the bottom of the list and select a **Default Role**. The following image shows the **Default Role** field:



   The Administrator can change the roles later.
11. If you want to map SAML groups to specific groups in IDMC, perform the following steps:
   a. Select **Map SAML Groups and Roles**.

b. In the **User Groups** field, enter the SAML attribute used to pass the assigned user groups. The following image shows the **Default Groups** field:



For example, if "idp_groups" is the name of the attribute from the identity provider used to pass user group information, then enter **idp_groups**.

c. Map the corresponding IDMC groups in the **SAML Group Mapping** tab. For more information, see "SAML role and group mapping properties" on page 95.
If there isn't a matching IDMC group, enter a new group name and IDMC will create this group and map it to the SAML group. Groups created this way are read-only in IDMC.

12. If you don't want to map specific groups and instead just use a default group, perform the following steps:
    a. Ensure that **Map SAML Groups and Roles** is not selected.
    b. In the **SAML Group Mapping** tab, scroll to the bottom of the list and select a **Default Group**.
    The Administrator can change the groups later.
13. Complete the remaining fields on the **SAML Setup** page as necessary.

Configuring provider settings and mapping attributes

Configure SAML single sign-on settings and map SAML attributes on the **SAML Setup** page.

1. Log in to IDMC as an organization administrator.
2. In Administrator, select **SAML Setup**.
3. On the **SAML Setup** page, configure the following properties:
    - SSO configuration properties
    - Identity provider configuration properties
    - Service provider settings
    - SAML attribute mapping properties
    - SAML role and group mapping properties (if you use SAML SSO for authentication and authorization)
4. Click **Save**.

    IDMC generates the service provider metadata file. IDMC also generates a unique token for your organization and saves the token to the IDMC repository. The single sign-on URL for your organization includes the token. For example:

    ```
    https://dm-us.informaticacloud.com/ma/sso/<organization token>
    ```

After you save your changes on the **SAML Setup** page, download the service provider metadata, and send it to your identity provider along with the IDMC single sign-on URL.

Identity provider configuration properties

Define identity provider configuration properties on the **SAML Setup** page.

The following table describes the identity provider configuration properties:

Property	Description
Issuer	The entity ID of the identity provider, which is the unique identifier of the identity provider. The Issuer value in all messages from the identity provider to IDMC must match this value. For example: `<saml:Issuer>http://idp.example.com</saml:Issuer>`
Single Sign-On Service URL	The identity provider's HTTP-POST SAML binding URL for the SingleSignOnService, which is the SingleSignOnService element's location attribute. IDMC sends login requests to this URL.
Single Logout Service URL	The identity provider's HTTP-POST SAML binding URL for the SingleLogoutService, which is the SingleLogoutService element's location attribute. IDMC sends logout requests to this URL.
Signing Certificate	Base64-encoded PEM format identity provider certificate that IDMC uses to validate signed SAML messages from the identity provider. **Note:** The identity provider signing algorithm must be either DSA-SHA256 or RSA-SHA256.
Use signing certificate for encryption	Uses the public key in your signing certificate to encrypt logout requests sent to your identity provider when a user logs out from IDMC.
Encryption Certificate	Base64-encoded PEM format identity provider certificate that IDMC uses to encrypt SAML messages sent to the identity provider. Applicable if you do not enable use of the signing certificate for encryption.
Name Identifier Format	The format of the name identifier in the authentication request that the identity provider returns to IDMC. IDMC uses the name identifier value as the IDMC user name. The name identifier cannot be a transient value that can be different for each login. For a particular user, each single sign-on login to IDMC must contain the same name identifier value. To specify that the name identifier is an email address, the Name Identifier Format is as follows: `urn:oasis:names:tc:SAML:1.1:nameidformat:emailAddress`
Logout Service URL (SOAP Binding)	The identity provider's SAML SOAP binding URL for the single logout service. IDMC sends logout requests to this URL.
Logout Page URL	The landing page to which a user is redirected after the user logs out of IDMC. IDMC redirects the logged out user to the landing page in the following ways: - If you specify a logout page URL, IDMC redirects the user to this URL after logout. - If you do not specify a logout page URL, IDMC redirects the user to a default logout page.

Service provider settings

Define the IDMC service provider settings on the **SAML Setup** page.

The following table describes service provider settings:

Property	Description
Informatica Cloud Platform SSO	Displays the single sign-on URL for your organization. IDMC generates this URL.
Clock Skew	Specifies the maximum permitted time, in seconds, between the time stamps in the SAML response from the identity provider and the IDMC clock. Default is 180 seconds (3 minutes).
Name Identifier value represents user's email address	If enabled, IDMC uses the name identifier as the email address. Default is enabled.
Sign authentication requests	If enabled, IDMC signs authentication requests to the identity provider. Default is enabled.
Sign logout requests sent using SOAP binding	If enabled, IDMC signs logout requests sent to the identity provider. Default is enabled.
Encrypt name identifier in logout requests	If enabled, IDMC encrypts the name identifier in logout requests. **Note:** Verify that the identity provider supports decryption of name identifiers before you enable this option. Default is disabled.

SAML attribute mapping properties

The authentication response from the identity provider to IDMC includes user login attributes such as name, email address, and user role. If the identity provider passes user and group information using SCIM 2.0, the authentication response includes additional SCIM attributes such as Display Name, Employee Number, and Organization.

Map the IDMC user fields to corresponding SAML attributes on the **SAML Setup** page.

Note: The attribute format differs based on your identity provider. Refer to the provider documentation for more information.

The following table describes the SAML attribute mapping properties:

Property	Description
Use friendly SAML attribute names	If selected, uses the human-readable form of the SAML attribute name which might be useful in cases in which the attribute name is complex or opaque, such as an OID or a UUID.
First Name	SAML attribute used to pass the user first name.
Last Name	SAML attribute used to pass the user last name.
Job Title	SAML attribute used to pass the user job title.
Email Addresses	SAML attribute used to pass the user email addresses. This property is required.

Property	Description
Emails Delimiter	Delimiter to separate the email addresses if multiple email addresses are passed.
Phone Number	SAML attribute used to pass the user phone number.
Time Zone	SAML attribute used to pass the user time zone.
These four properties are enabled when the **Map SAML Groups and Roles** option is enabled:	
User Roles	SAML attribute used to pass the assigned user roles.
Roles Delimiter	Delimiter to separate the roles if multiple roles are passed.
User Groups	SAML attribute used to pass the assigned user groups.
Groups Delimiter	Delimiter to separate the groups if multiple groups are passed.

The following table describes the additional attributes. These attributes are visible when the **Enable IdP to push users/groups using SCIM 2.0** option is enabled:

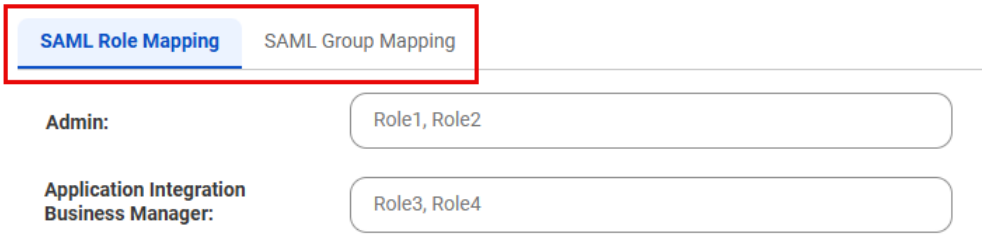
Property	Description
Display Name	SCIM attribute used to pass the user displayName.
Employee Number	SCIM attribute used to pass the enterprise user employeeNumber.
Organization	SCIM attribute used to pass the enterprise user organization.
Department	SCIM attribute used to pass the enterprise user department.
Street Address	SCIM attribute used to pass the user streetAddress.
Locality	SCIM attribute used to pass the user locality.
Region	SCIM attribute used to pass the user region.
Post Code	SCIM attribute used to pass the user postalCode.
Country	SCIM attribute used to pass the user country.
Locale	SCIM attribute used to pass the user locale.
Preferred Language	SCIM attribute used to pass the user preferredLanguage.
ID	SCIM attribute used to pass the user id.
External ID	SCIM attribute used to pass the user externalId. For Azure Active Directory, this is the objectID. For Okta, it is the id.

SAML role and group mapping properties

When you use SAML for authentication only, define a default role and optionally a default user group for new users. When you use SAML for both authentication and authorization, you can map specific SAML role and group names to IDMC role names. You can map multiple SAML roles and groups to a single IDMC role.

Note: For instructions on how to map groups with Azure Active Directory, see the HOW TO: Create a SAML Group Mapping with Azure AD Knowledge article.

When the **Map SAML Groups and Roles** option on the **SAML Setup** page is enabled, you can map specific IDMC roles and groups in the **SAML Role Mapping** and **SAML Group Mapping** tabs, as shown in the following image:



You can map multiple IDMC roles and groups, separated by commas.

When the **Map SAML Groups and Roles** option is disabled, all SAML roles and groups become non-editable, except for the following fields at the end of the list:

Property	Description
Default Role	Default user role for single sign-on users. When auto-provisioning is enabled, new users are assigned this role the first time they sign in to IDMC.
Default User Group	Optional, default user group for single sign-on users. When auto-provisioning is enabled, new users are assigned to this user group the first time they sign in to IDMC.

Downloading the service provider metadata

The identity provider requires the SAML service provider metadata and IDMC URL to complete the SAML single sign-on setup process. After IDMC generates the service provider metadata file, deliver the file and the IDMC URL to the identity provider.

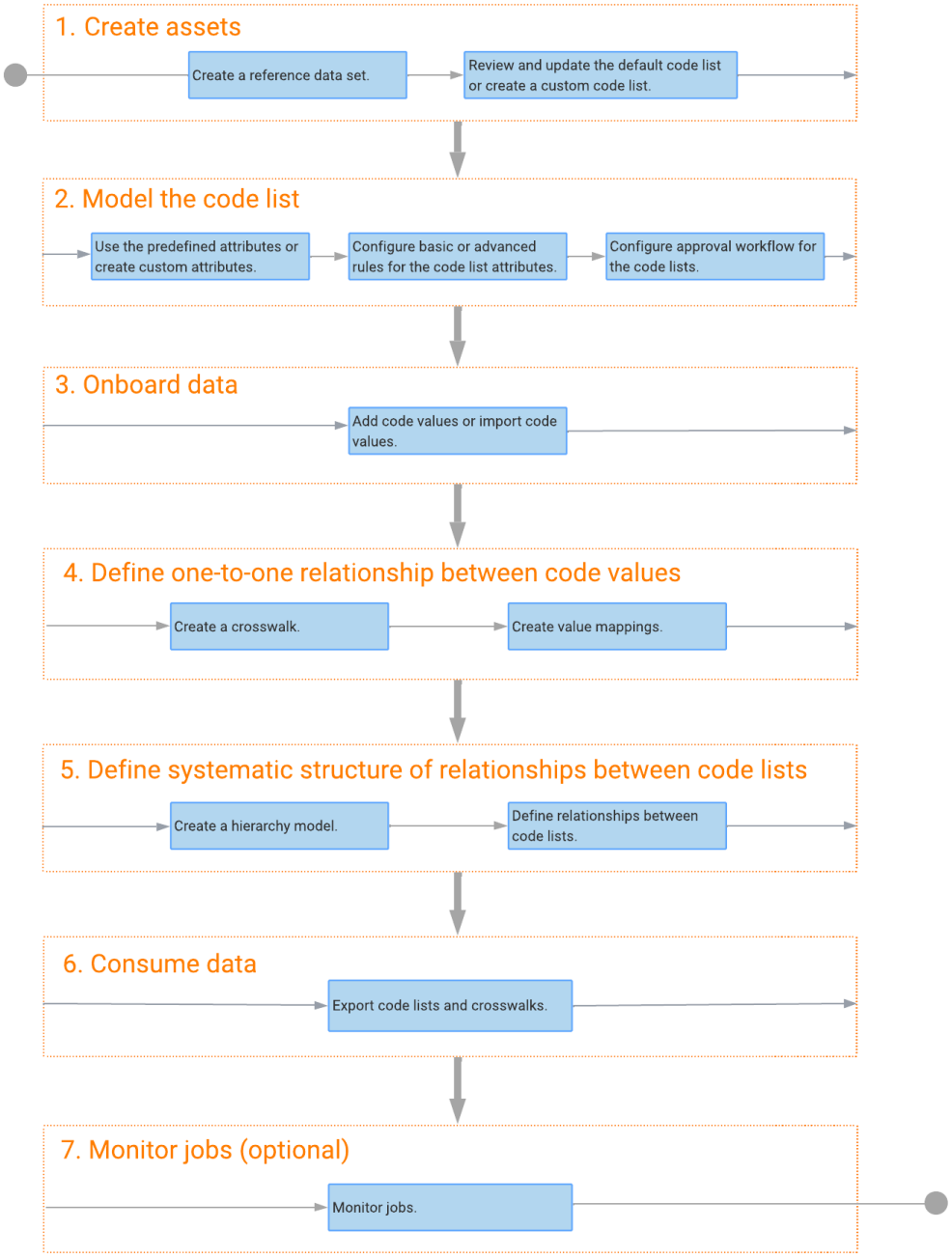
1. On the **SAML Setup** page, click **Download Service Provider Metadata**.

   The service provider metadata file is downloaded to your machine.
2. In the **Information** dialog box, note the URL for single sign-on access to your IDMC organization.
3. Click **OK** to close the **Information** dialog box.
4. Send the metadata file and the IDMC single sign-on URL to your identity provider administrator.

Reference data configuration process

To define and use the reference data, ensure that you follow the steps in the configuration process.

The following image shows the steps in the reference data configuration process:



The following table lists the steps involved in the reference data configuration process:

Step	Sub-steps	Reference Content
Step 1. Create assets	1. Create a reference data set 2. Review and update the Enterprise code list or create a custom code list	- "Creating reference data sets" on page 102 - "Code lists" on page 19 or "Creating code lists" on page 109
Step 2. Model the code list	1. Use the predefined attributes or create custom attributes 2. Configure basic or advanced rules for the code list attributes 3. Configure approval workflow for the code list	- "Attributes" on page 26 or "Custom attributes" on page 27 - "Basic rule associations" on page 169 or "Advanced rule associations" on page 172 - "Configuring workflows" on page 177
Step 3. Onboard data	Add code values or import code values	- "Adding code values" on page 123 - "Import process" on page 155 or "Defining reference data import jobs" on page 180
Step 4. Define one-to-one relationship between code values	1. Create a crosswalk 2. Create value mappings	- "Step 1. Create a crosswalk" on page 135 - "Step 2. Configure value mappings" on page 137
Step 5. Define systematic structure of relationships between code lists	1. Create a hierarchy model 2. Define relationships between code lists	- "Creating hierarchy models" on page 159 - "Creating hierarchies" on page 163
Step 6. Consume data	Export code lists and crosswalks	"Defining reference data export jobs" on page 182
Step 7. Monitor jobs (optional)	Monitor jobs	"Monitoring jobs" on page 185

Best practices to configure reference data

To configure and use reference data, use the following best practices:

- Use unique, business-friendly, and descriptive names for reference data assets, and avoid generic names, such as `ReferenceDataSet1`.
- Organize reference data sets by application or domain, such as finance or healthcare.
- Ensure that each code value within a code list is unique.

Migrate Reference 360 assets

When you migrate reference data assets, MDM SaaS uses internal IDs, aliases, and names to identify Reference 360 assets.

Best practices to migrate assets

To migrate reference data assets seamlessly, use the following best practices:

- Create reference data assets in your development environment first, and then migrate them to higher environments, such as QA and production.
- Ensure that all changes are available in your development environment before you migrate the assets to the production environment.
- Use import and export REST APIs to migrate assets instead of manually creating or updating assets. This approach ensures consistency of internal IDs across environments and seamless downstream integrations.
- If you need to manually create an asset in your production environment, ensure that its internal ID matches the one in your development environment.
- To avoid data loss and system errors, ensure that you don't delete reference data models in QA or production environments.

Guidelines to import an existing asset based on identifiers

Consider the following guidelines when you import an existing asset:

- Ensure that the internal IDs of the imported and existing assets are the same. If they aren't same, Reference 360 creates a new asset.
- If the imported asset doesn't have an internal ID but has an alias, the alias must be the same as the internal ID or alias of the existing asset.
- If the imported asset doesn't have an internal ID or an alias, then the name of imported asset must be the same as the name of the existing asset.
- If an identifier of the imported reference data set is the same as the identifier of an existing reference data set, their default code lists must also match.
- If an identifier of the imported code lists is the same as the identifier of an existing code list, their reference data sets must also match.
- If an identifier of the imported crosswalk is the same as the identifier of an existing crosswalk, their source and target code lists must also match.

Migrating reference data assets

When you migrate reference data assets from one organization to another, Reference 360 uses internal IDs, aliases, and names to identify reference data assets.

When you import an asset, Reference 360 first compares the internal ID of the asset that you import with the internal ID of existing assets. If the internal ID of the asset you import and the internal ID of any existing asset aren’t null and they match, the existing asset is successfully updated.

If the internal IDs don’t match and neither is null, Reference 360 compares the internal ID of the asset that you import with the alias of the existing asset, alias with alias, and then name with name. If any of these comparisons result in a match while internal IDs mismatch, the import fails.

If the internal IDs don’t match but either one of the internal ID is null, Reference 360 compares the internal ID of the asset that you import with the alias of the existing asset. If they match, the existing asset is successfully updated.

If they don’t match, Reference 360 compares the alias of the asset that you import with the internal ID and alias of the existing asset. If any of these values match, the existing asset is successfully updated.

If they don’t match, Reference 360 compares the name of the asset that you import with the name of the existing asset. If they match, the existing asset is successfully updated. If they don’t match, a new asset is created.

The asset migration fails if the names match but meet the following criteria:

- The assets have different aliases and no internal IDs.
- The assets have different internal IDs.

If no matches are detected, then Reference 360 considers the asset as a new asset and creates the asset.

For example, if the source asset is Cost Center with internal ID `int_ccenter` and the target asset is Cost Center Category with the same internal ID `int_ccenter`, Reference 360 compares internal IDs and updates the Alias and Name fields in the target system.

If the source asset is Cost Center with an internal ID `int_ccenter` and alias `ccenter` and the target asset is Cost Center Category with no internal ID and alias `int_ccenter`, Reference 360 compares internal ID and alias and updates the Name field in the target system.

If an asset with the same name but different IDs exists in both source and target, the import fails. Reference 360 considers to create the asset in the target, but a duplication error occurs and the import fails because an asset with the same name exists.

For more information about using the identifiers of assets to import the assets, see *Internal IDs and aliases*.

You can't migrate job definitions created for Reference 360 assets. All Reference 360 assets aren't published and available in Business 360 Console.

CHAPTER 3

Manage system reference data

System reference data contains a group of values that provide information about your reference data.

You can configure system reference data, such as application, confidentiality, domain, priority, and status. You can then use the system reference data when you define other reference data assets.

To manage the system reference data, users must be assigned one of the following roles:

- Reference 360 Administrator
- Reference 360 Primary Owner
- Reference 360 Business Steward
- MDM Designer

For more information about system reference data, see "System reference data" on page 15.

Related Topics:

- "System reference data" on page 15

Adding values to system reference data

You can add values to the system reference data. You can then use these values when you define reference data assets.

1. In the **Explore** panel, select **System Reference Data** and click **Open**.

   The system reference data opens in a new tab.
2. Click the system reference data for which you want to add a value.
3. Click the **Add** icon.

   A blank row appears.
4. To add a value, click the new row and perform the following actions:
   a. In the **Name** field, enter a name.
   b. In the **Key** field, enter a unique identifier for the value.

      You can't change the key for the values after you save the values.
5. Click the **Save** icon.

Editing values of system reference data

You can edit your system reference data values.

1. In the **Explore** panel, select **System Reference Data** and click **Open**.

   The system reference data opens in a new tab.
2. Click the system reference data for which you want to edit a value.
3. Click the **Edit** icon.
4. Click the value that you want to edit.
5. Edit the name as required.

   Note: You can't modify the key for the values.
6. Click the **Save** icon.

Deleting values of system reference data

You can delete the system reference data values that you no longer need.

1. In the **Explore** panel, select **System Reference Data**, and click **Open**.

   The system reference data opens in a new tab.
2. Click the system reference data for which you want to delete a value.
3. Hover over the value that you want to delete, and then click the **Delete** icon.

   A confirmation dialog box appears.
4. Click **Delete**.

   The value is deleted for all the assets.

CHAPTER 4

Manage reference data sets

All reference data in Reference 360 is categorized under reference data sets. You create a reference data set to contain a category of reference data, such as country codes or currency codes. In a reference data set, you create code lists. Code lists contain code values from an application.

For example, you create a Country Codes reference data set to categorize the country codes reference data in your organization. In the reference data set, you create a Sales Country Codes code list and a Marketing Country Codes code list. In the Sales Country Codes code list, you create or import country code values used in your organization's sales application. In the Marketing Country Codes code list, you create or import country code values used in your organization's marketing application.

When you create a reference data set, you define the definition and attributes. Later, when you create code lists, the code lists inherit the structure definition and attributes of the reference data set.

If you no longer need a reference data set, you can delete the reference data set.

For more information, see "Reference data sets" on page 16.

Creating reference data sets

Create reference data sets to categorize the reference data in your organization. Reference data sets contain code lists, and code lists contain code values.

To create a reference data set, perform the following actions:

1. Create a reference data set.
2. Define a reference data set.
3. Assign stakeholders.
4. View the history of a reference data set.

Related Topics:

- "Reference data sets" on page 16

Step 1. Create a reference data set

Create a reference data set, and configure its general properties and status. You can specify internal ID and alias for a reference data set.

1. Click **New** and select **Reference Data Set**.

   The **New Reference Data Set** page appears and displays the **Summary** tab.

2. Configure the general properties fields.

   Note: The domain, confidentiality, and priority are inherited by code lists in the reference data set.

   The following table lists the properties that you can configure for a reference data set:

Field	Description
Reference Data Set	Required. Display name of a reference data set.
Internal ID	Required. Unique identifier of a reference data asset, limited to a maximum of 45 characters. You can use alphanumeric characters and underscores. After you save an asset, you can't edit the internal ID. You can't specify internal IDs for existing assets.
Alias	Alternative identifier for a reference data asset, limited to a maximum of 70 alphanumeric characters. After you save an asset, you can edit the alias.
Description	Short description of the reference data set.
Domain	Area or grouping to describe the code values.
Confidentiality	Confidentiality level of a reference data set and its code lists and crosswalks. The confidentiality levels are confidential, internal, restricted, and public.
Priority	Priority of a reference data set. The priority levels are critical, high, medium, and low.
Status	State of a reference data set in the life cycle.
Effective Date	Date from when the status is effective.
Approved by	User who approved a reference data set.
Approved on	Date of approval.

   If you don't see options in some lists, configure your system reference data values. For more information, see "Adding values to system reference data" on page 100.

Step 2. Define a reference data set

Define the structure, attributes, and display settings of a reference data set. When you create code lists, the code lists inherit the structure definition and attributes from the reference data set.

Important: After you create the reference data set, some actions are restricted to prevent issues with the definition of the crosswalk, such as modifying the structure definition or creating additional required attributes. For more information, see "Reference data sets" on page 16.

1. Click **Definition**.

   The **Definition** tab opens.

2. Optionally, configure the structure definition of the reference data set.

Option	Description
Hierarchical	A hierarchical reference data set can contain hierarchical data structures, and its code lists inherit the hierarchical structure. Code lists that have a hierarchy allow you arrange their code values into levels. For more information, see "Hierarchical reference data sets" on page 17.
Dependent	A dependent reference data set depends on the code values in another reference data set and passes on its dependency to its code lists. For more information, see "Dependent reference data sets" on page 18.

3. Optionally, to configure business IDs for code values in a default code list, select **Enable Business ID** and specify the business ID properties.

   The following table describes the business ID properties:

Property	Description
Display Name of Business ID	Name of the field that displays business IDs. Default is `Business ID`.
Format	Required. The format in which you want the business IDs to be generated. Select one of the following formats: - Alphanumeric. Use 10 to 24 characters, starting with a prefix that consists of three characters. The prefix can contain uppercase letters and numbers. For example, the format RDM4AB78W2 starts with a prefix of three characters, RDM, followed by seven alphanumeric characters. - Numeric. Use 10 to 24 numbers including a prefix of three digits. For example, 9994567123. You can enter an offset value. The business ID of code values created by Reference 360 starts at the offset value, excluding the prefix. For example, with a length of 10, an offset value of 1000, and a prefix of 999, the starting business ID is 9990001000.
Length (including prefix)	Required. Total length of the business IDs.
Prefix	Required. Characters added to the beginning of business IDs.
Offset	Starting value for business IDs that Reference 360 generates. Applicable for numeric business IDs.

4. Optionally, select the attributes that you want to display by default.
   a. Click the **Edit** icon.
   b. Select **Required**.

5. Optionally, configure a custom attribute for the reference data set.
   a. Click **Add**.

      An attribute row appears in the **Attributes** section. In the **Attribute Details** panel, you can configure the details of the attribute.

   b. In the **Attribute Name** field, enter a name.

   c. From the **Type** list, select the data type of the attribute.

      For more information about the type of attributes, see "Attributes" on page 26.

   d. If the reference data attribute depends on the existing reference data attributes, set the dependency in the **Dependent On** field.

      For more information about the dependency between reference data attributes, see "Attributes" on page 26.

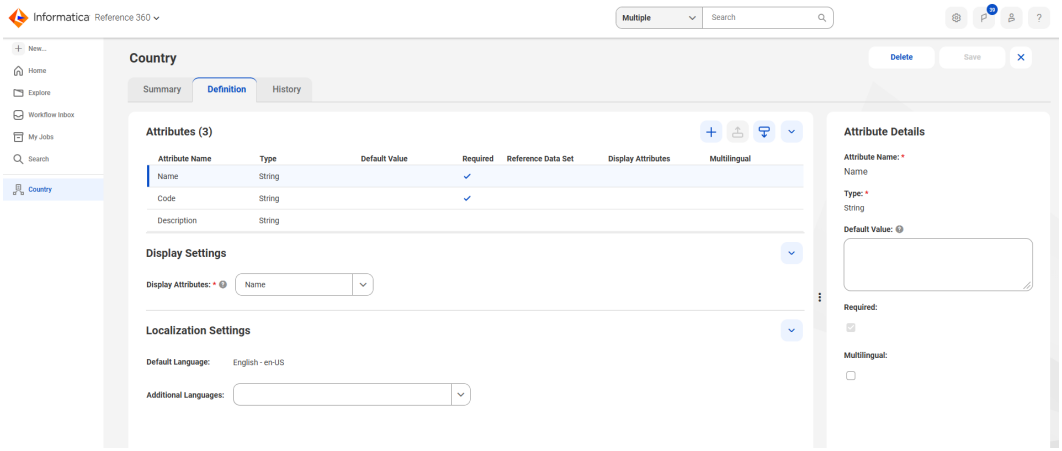
   e. To make the attribute required, select **Required**.

   Note: You can set a new attribute or an existing attribute as required for the existing reference data sets. You cannot set the Name and Code attributes as optional as they are required by default.

6. In the **Localization Settings** section, select the additional languages that you want to configure for the reference data set or its code lists. You can select additional languages from those available in the Localization Languages code list. You can view the default language for the organization defined in the Localization Languages code list.

7. In the **Display Settings** section, select the attribute that you want to display in Reference 360 to represent code values in the reference data set.

The following image shows a sample page to define the structure, attributes, and display settings of the code list:

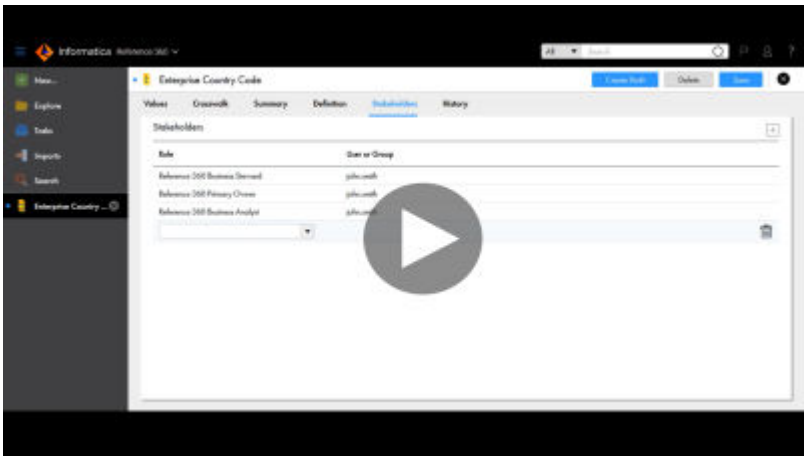


Step 3. Assign stakeholders

Assign stakeholder roles to users who play a role in using, creating, or maintaining the asset. Stakeholder roles determine a user's privileges for the asset.

When a user manages an asset as a stakeholder, they have the combined privileges provided by their stakeholder role and their Reference 360 role. For more information about stakeholder roles, see "Stakeholder roles" on page 53. For more information about assigning roles, see "Guidelines for assigning roles" on page 55.

The following video shows you how to assign stakeholders:



1. Click **Stakeholders**.

   The **Stakeholder** tab opens.
2. Click **Add**.

   A list appears in an empty row.
3. In the **Role** list, select a stakeholder role.

   The **New Stakeholder** dialog box appears.
4. Select a user name and click **Add**.

   The user name of the stakeholder appears in the row.
5. Click **Save**.

When you create a reference data set and assign a stakeholder role to a user or user group, Reference 360 automatically adds an additional Reference 360 Business Analyst stakeholder role to the users and groups.

Step 4. View the history of a reference data set

You can view the changes made to a reference data set in a chronological order. Use the historical data to track the changes made to the reference data set at any point in time.

1. In the **Explore** panel, select a reference data set and click **Open**.

   The reference data set opens in a new tab.
2. To view the change history of the reference data set, click the **History** tab.

   The change history of the reference data set appears.

The following table lists the different user interface elements that you can find on the **History** tab:

User Interface Element	Description
Show/Hide Filter icon	Displays the **Filter** menu. Click **Add Filter** and select the following values based on which you want to filter the history feed: - Date Range. Allows you to specify a time period. You can view the history feed for the specified time period. Use the following predefined time periods: - Today. Displays the changes occurred on the current date. - Last 7 days. Displays the changes occurred in the last seven days. - Last 30 days. Displays the changes occurred in the last 30 days. - Custom. Displays the changes occurred during the selected date range. - Event. Displays the changes based on the change events. You can filter based on a single or multiple event types. - Business Entity Fields. Filters the history feed based on the general properties of the code list. You can filter based on a single property or all properties.
Sort icon	Displays the following values based on which you can sort the history feed: - New to Old. Sorts the changes in reverse chronological order. - Old to New. Sorts the changes in chronological order.
View icon	Groups the history feed based on the selected frequency. Use one of the following frequencies: - Daily. Groups the history feed by day. - Monthly. Groups the history feed by month. - Yearly. Groups the history feed by year.
Expand icon	Displays the event details in full screen.

3. To view all fields in the history feed, clear **Show the updated fields**.

   By default, the **Show the updated fields** check box is selected, and the updated fields appear in the history feed.

Deleting reference data sets

Users with the Reference 360 Primary Owner role can delete reference data sets that are no longer needed.

You cannot delete reference data sets that contain code lists or are referenced by other assets. For more information, see "Considerations for deleting reference data sets" on page 108.

1. Open the reference data set that you want to delete.
2. Click **Delete**.
3. Click **OK**.

Considerations for deleting reference data sets

You can delete reference data sets that you no longer need. You must be assigned the Reference 360 Primary Owner role to delete reference data sets.

You cannot delete the following types of reference data sets:

- Reference data sets that contain code lists
- Reference data sets that are specified as a dependent of another asset
- Reference data sets that are specified as a Reference Data attribute for another asset

Note: Users with the Primary Owner stakeholder role cannot delete reference data sets. For more information, see "Users, groups, and roles" on page 50 .

After you delete a reference data set, you can't reuse its internal ID. If you try to create a reference data set with the same internal ID, either through asset migration or manually reuse the internal ID of a previously deleted reference data set in the API request body during model import or export, the asset migration and import and export models fail.

CHAPTER 5

Manage code lists

Code lists contain a set of code values that you create or import from a source application. You create code lists in reference data sets. A code list inherits its structure definition and attributes from the reference data set.

Later, you can create crosswalks to map code values in a pair of code lists. Crosswalks provide a way to translate between the different code values each application uses for a business term.

You can create validation rules for attributes in code lists to ensure that code values meet your business standards. For example, in the Country Codes code list, you might want the Code attribute for code values to require a minimum of three characters.

You can view the history of code values in code lists. Use the historical data to track the detailed changes made to code lists.

You can view the code values that existed in a code list at a point in time in the past. For example, you can view the code values that existed in the Enterprise Country code list on October 2022.

Note: Reference 360 displays the code values that are available in the database at a point in time in the past.

If you no longer need a code list, you can delete the code list. When you delete a code list, the code list and its code values are removed from Reference 360. You cannot delete a code list that is used in a picklist field of a business entity in a business application, such as Customer 360.

For more information about code lists, see "Code lists" on page 19.

Creating code lists

You create code lists in reference data sets. A code list inherits its structure definition and attributes from the reference data set. You can define additional attributes or display columns, and assign stakeholders to a code list that are different from the reference data set.

Later, you create or import a set of code values from a source system into the code list.

For more information, see "Code lists" on page 19.

To create a code list, perform the following actions:

1. Create a code list.
2. Define a code list.
3. Assign stakeholders.

Related Topics:

- "Code lists" on page 19

Step 1. Create a code list

Create a code list in a reference data set, and configure the general properties, external URLs, notification recipients, and status of the code list. You can specify internal ID and alias for a code list.

Before you begin, you must create a reference data set.

1. Click **New** and select **Code List**.

   The **New Code List** page appears and displays the **Summary** tab.
2. Select a reference data set to associate with this code list.
3. Configure the general properties fields.

   Note: Code lists inherit the domain, confidentiality, and priority from the reference data set.

Field	Description
Name	Required. Display name of a code list. **Note:** Renaming a code list doesn't update the name in its dependent objects, such as Data Integration mapping.
Internal ID	Required. Unique identifier of a code list, limited to a maximum of 45 characters. You can use alphanumeric characters and underscores. After you save an asset, you can't edit the internal ID. Ensure that an internal doesn't start with `ent_` because the internal ID of default code lists start with `ent_`. **Note:** You can't specify internal IDs for existing assets that you created before the April 2025 release.
Alias	Alternative identifier for a code list, limited to a maximum of 70 alphanumeric characters. After you save an asset, you can edit the alias.
Version	Version information of a code list.
Description	Short description of the code list. The description cannot exceed 4000 characters.
Additional Description	Additional description of a code list. You can add additional details of a code list in the Additional Description field.
Application	Source application of the code values.
Domain	Area or grouping to describe the code values.
Confidentiality	Confidentiality level of the code list inherited from the reference data set. The confidentiality levels are confidential, internal, restricted, and public.

Field	Description
Priority	Priority of a code list. The priority levels are critical, high, medium, and low.
Status	State of a code list in the life cycle.
Effective Date	Date from when the status is effective.
Approved by	User who approved a code list.
Approved on	Date of approval.

If you do not see options in some lists, configure your system reference data values. For more information, see "Adding values to system reference data" on page 100.

Note: The **Summary** tab of the code lists displays only the Reference Data Set, Name, and Description fields for users with a Reference Data Management Basic Edition license.

4. In the **External URLs** section, click **Add External URL**.

   A blank row appears.

5. To add an external URL, enter a URL name and a link of an external page for an externally mandated code list.

   The added external URL appears in the row. You can enter up to 4,000 characters for external URL names and links.

6. To remove an external URL, hover over a row, and click **Delete External URL**.

7. To notify external stakeholders about changes to the code list, click **Add Notification Recipient** in the **Notification Recipients** section.

   Note: Ensure that you define a process in Cloud Application Integration to trigger notifications to external stakeholders in Reference 360. For more information about designing processes in Cloud Application Integration, see Designing Processes.

   A blank row appears.

8. To add a notification recipient, enter a name, contact type, such as email or SMS text message, and the respective contact detail.

   The added notification recipient appears in the row. You can enter up to 4,000 characters for notification recipient names and contact details.

9. To remove a notification recipient, hover over a row, and click **Delete Notification Recipient**.

Step 2. Define the structure, attributes, and display settings of code lists

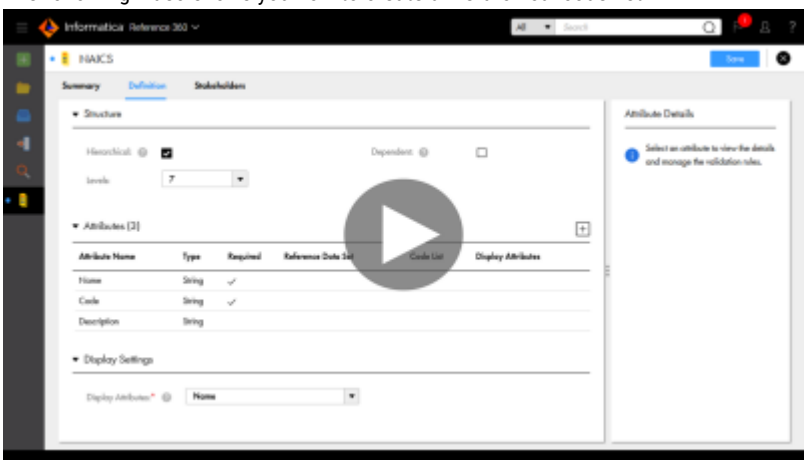
Define the structure, attributes, and display settings of the code list. Code lists inherit their structure and attributes from the reference data set.

Important: After you create the code list, some actions are restricted to prevent issues with the definition of the code list, such as modifying the structure. For more information, see "Code lists" on page 19.

1. Click **Definition**.

   The **Definition** tab opens.

2. If the code list didn't inherit a structure from the reference data set, configure the structure definition.

Option	Description
Hierarchical	A hierarchical code list allows you to arrange code values into levels to create hierarchies. The following video shows you how to create a hierarchical code list:  For more information, see "Hierarchical code lists" on page 21.
Dependent	A dependent code list depends on code values in a code list that belongs to a different reference data set. For more information, see "Dependent code lists" on page 22.

3. Optionally, to configure business IDs for code values in the code list, select **Enable Business ID** and specify the business ID properties.

   The following table describes the business ID properties:

Property	Description
Display Name of Business ID	Name of the field that displays business IDs. Default is `Business ID`.
Format	Required. The format in which you want the business IDs to be generated. Select one of the following formats: - Alphanumeric. Use 10 to 24 characters, starting with a prefix that consists of three characters. The prefix can contain uppercase letters and numbers. For example, RDM4AB78W2. - Numeric. Use 10 to 24 numbers including a prefix of three digits. For example, 9994567123. You can enter an offset value. The business ID of code values created by Reference 360 starts at the offset value, excluding the prefix. For example, with a length of 10, an offset value of 1000, and a prefix of 999, the starting business ID is 9990001000.
Length (including prefix)	Required. Total length of the business IDs.

Property	Description
Prefix	Required. Characters added to the beginning of business IDs.
Offset	Starting value for business IDs that Reference 360 generates. Applicable for numeric business IDs.

4. Optionally, configure a custom attribute for the code list.

   Note: Users with a Reference Data Management Basic Edition license cannot edit the existing attributes, add new attributes, or add rules to the attributes of the code list.

   a. Click **Add**.

      An attribute row appears in the **Attributes** section. In the **New Attribute** panel, you can configure the details of the attribute.

   b. In the **Attribute Name** field, enter a name.

      Note: Ensure that you don't use the display name of the business ID of the code list or the reserved keywords. For a complete list of reserved keywords, see "Reserved keywords" on page 115.

      Reference 360 generates internal field names for the attributes based on the attribute names and displays them in the export file when you export code values.

      For more information about the properties of attribute names and their corresponding internal names, see "Attribute names" on page 116.

   c. From the **Type** list, select the data type of the attribute.

      Note: You can define a scale for the Decimal data type attributes. The supported scale value ranges from 1 to 34. The default scale is 4 and the default precision is 34. Ensure that the scale value that you define is less than the default precision.

   d. Set a default value for the attribute.

      When you add a code value to the code list, the field displays the default value. If you add new rule associations or modify the existing rule associations for the attribute, Reference 360 clears the default value. You can set a default value based on the configured rule association.

   e. To make the attribute required, select **Required**.

   f. To add a validation rule for a string, integer, or decimal attribute type, click **Add Rule** and configure a validation rule.

The following table lists the validation rules that you can configure for each attribute type:

Attribute Type	Rules
String	- Allowed Characters - Not Allowed Characters - Concatenate - Contains - Does Not Contain - Minimum Length - Maximum Length - Starts with - Ends with **Note:** For attributes with a concatenate rule, you can update the values that are used in the concatenated attribute value. However, if you configure a concatenate rule for the Code attribute, you cannot update the values in the concatenated attribute value.
Decimal	- Minimum Value - Maximum Value - Maximum Scale - Maximum Precision
Integer	- Minimum Value - Maximum Value
Date	- Start Date - End Date - Date Range

You can set a new attribute or an existing attribute as required for an existing code list. You can't set the Name and Code attributes as optional as they are required by default.

When you set a new attribute or modify the existing attribute as required, a warning message appears.

After you update a code list definition, save the changes before you add a code value to the code list. Otherwise, the changes are lost, and the **Save** button is disabled. To update the code list again, close and reopen it.

5. In the **Localization Settings** section, select the additional languages that you want to configure for the reference data set or its code lists. You can select additional languages from those available in the Localization Languages code list. You can view the default language for the organization defined in the Localization Languages code list.

   Note: You can't remove additional languages for a code list that already contains multilingual code values.
6. Optionally, in the **Display Settings** section, select the attribute that you want to appear in Reference 360 to represent code values in the code list.
7. To change the order of attributes, select an attribute and click **Move Up** or **Move Down**. You can also drag an attribute to the desired location within the attributes list.

   Reference 360 displays the attributes of the code list in the specified order. You can also view the attributes in the specified order when you import and export code values.

Reserved keywords

When you configure a custom attribute for a code list, ensure that you don't use the following reserved words as the attribute name:

- _localizedContent
- _language
- _members
- _meta
- appEntityType
- approvedById
- approvedByName
- approvedOn
- appType
- Business ID
- businessId
- Code
- Created By
- Created On
- createdBy
- creationDate
- dependency
- Description
- effectiveDate
- Last Updated By
- Last Updated Date
- lastUpdateDate
- Name
- OrgId
- Record State
- Source Primary Key
- Source Record Details
- sourcePKey
- sourceSystem
- status
- updatedBy

Attribute names

Reference 360 generates internal names for attributes based on the attribute names that you specify.

The following table explains how Reference 360 generates internal names based on attribute names:

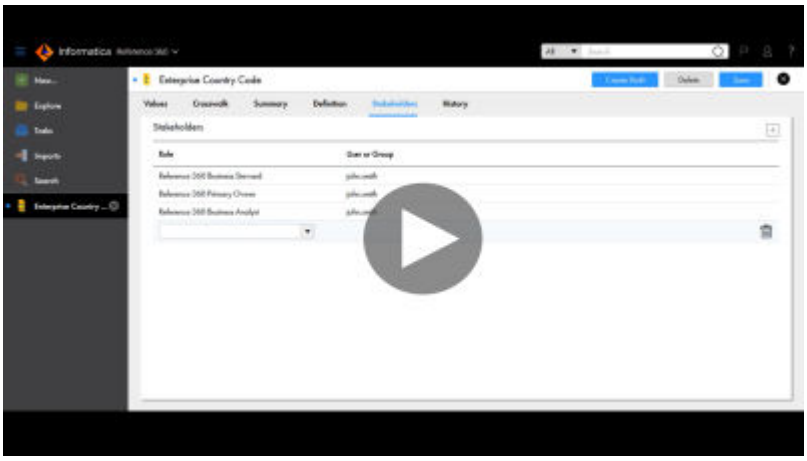
Attribute Names	Internal Name
If attribute names contain only alphanumeric characters	The internal names use the attribute names in lowercase. For example, if an attribute name is Country, the internal name appears as `country`.
If attribute names contain alphanumeric characters with spaces or special characters	The internal names use the attribute names in lowercase without spaces and special characters. If a generated internal name is the same as an existing internal name, Reference 360 appends numbers starting from 1 to the generated internal name. For example, if an attribute name is United States, the internal name appears as `unitedstates`.
If attribute names contain only special characters	The internal names use the following format: `field<n>`, where n can be 1, 11, 111, and so on. For example, if an attribute name is @#$%, the internal name appears as `field1`. **Note:** If a code list contains any existing attribute as `field1`, the internal name appears as `field11`.

Step 3. Assign stakeholders

Assign stakeholder roles to users who play a role in using, creating, or maintaining the asset. Stakeholder roles determine a user's privileges for the asset.

When a user manages an asset as a stakeholder, they have the combined privileges provided by their stakeholder role and their Reference 360 role. For more information about stakeholder roles, see "Stakeholder roles" on page 53. For more information about assigning roles, see "Guidelines for assigning roles" on page 55.

The following video shows you how to assign stakeholders:



1. Click **Stakeholders**.

   The **Stakeholder** tab opens.

2. Click **Add**.

   A list appears in an empty row.

3. In the **Role** list, select a stakeholder role.

   The **New Stakeholder** dialog box appears.

4. Select a user name and click **Add**.

   The user name of the stakeholder appears in the row.

5. Click **Save**.

When you create a reference data set and assign a stakeholder role to a user or user group, Reference 360 automatically adds an additional Reference 360 Business Analyst stakeholder role to the users and groups.

Viewing the history feed of a code list

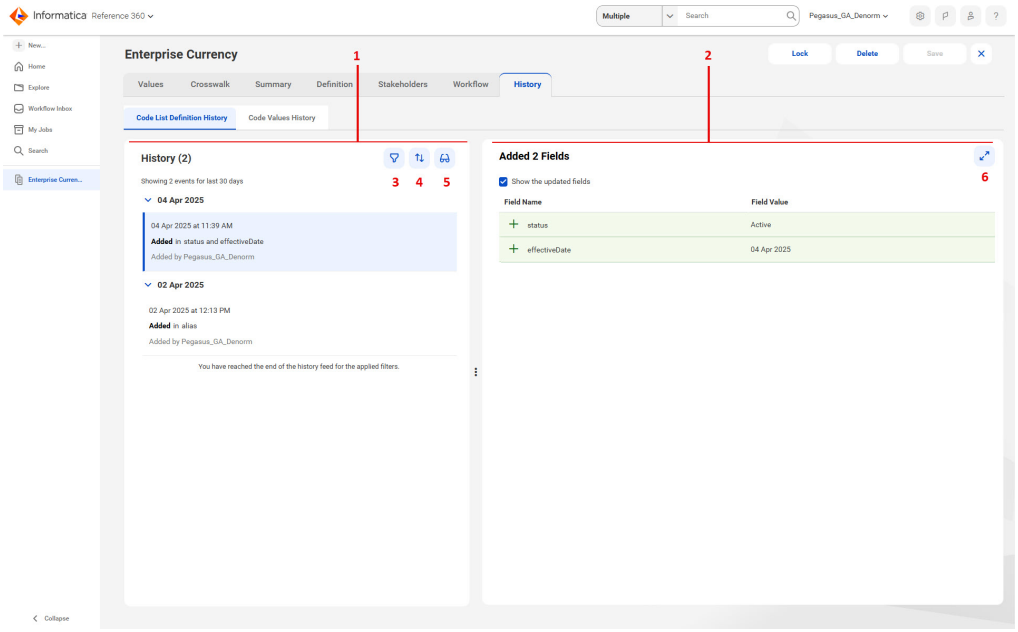
You can view the changes made to a code list in a chronological order. Use the historical data to track the detailed changes made to the code list.

1. In the **Explore** panel, select a code list, and click **Open**.

   The code list opens in a new tab.

2. Click the **History** tab.

   By default, the **Code List Definition History** tab opens and displays the change history of the code list properties.
   The following image shows the change history of a sample code list:



1. History panel
2. Event details panel
3. Show/Hide filter
4. Sort
5. View
6. Expand

The following table lists the different user interface elements that you can find on the **Code List Definition History** tab of the **History** tab:

User Interface Element	Description
Show/Hide Filter icon	Displays the **Filter** menu. Click **Add Filter** and select the following values based on which you want to filter the history feed: - Date Range. Allows you to specify a time period. You can view the history feed for the specified time period. Use the following predefined time periods: - Today. Displays the changes occurred on the current date. - Last 7 days. Displays the changes occurred in the last seven days. - Last 30 days. Displays the changes occurred in the last 30 days. - Custom. Displays the changes occurred during the selected date range. - Event. Displays the changes based on the change events. You can filter based on a single or multiple event types. - Business Entity Fields. Filters the history feed based on the general properties of the code list. You can filter based on a single property or all properties.
Sort icon	Displays the following values based on which you can sort the history feed: - New to Old. Sorts the changes in reverse chronological order. - Old to New. Sorts the changes in chronological order.

User Interface Element	Description
View icon	Groups the history feed based on the selected frequency. Use one of the following frequencies: - Daily. Groups the history feed by day. - Monthly. Groups the history feed by month. - Yearly. Groups the history feed by year.
Expand icon	Displays the event details in full screen.

3. To view all fields in the history feed, clear **Show the updated fields**.

   By default, the **Show the updated fields** check box is selected, and the updated fields appear in the history feed.

4. To view the history of changes to code values of the code list in a table, click the **Code Values History** tab.

   The **Code Values History** tab displays only 2500 rows of history feed. To view the complete history feed, use the audit trail REST API.

Viewing code lists at a point in time

You can view code values that existed in a code list at a point in time in the past.

1. In the **Explore** panel, select a code list and click **Open**.
2. To view the code values at a point in time, select the date and time using the calendar or type the date and time in the **Point in Time** field.

   You can view the code values that existed at a point in time in the past.

   Note: When you view the code list at a point in time in the past, you can't add, sort, filter, import, or export the code values.
3. To revert to the current code values, click **Current**.

Associate code list attributes with Data Governance and Catalog assets

You can associate code list attributes with glossary assets such as business terms and metrics, manual data elements, and technical data elements in Data Governance and Catalog. You can also link data elements to glossaries that are already associated with code list attributes.

For example, you can associate standardized region codes, such as APAC, EMEA, and AMER, that are defined in Reference 360 with the Customer Region business glossary term. When you tag data elements with the business glossary term, Reference 360 enforces the approved codes, ensuring consistent and governed data classification.

Use the Reference 360 and Data Governance and Catalog integration to view data lineage and understand the downstream impact of reference data.

For example, consider the Country Codes reference data that is defined in Reference 360. This reference data flows into downstream assets, such as CRM account records, regional sales dashboards, and GDPR data classification reports. When you update or retire a country code, use the lineage view in Data Governance and Catalog to identify all downstream assets that consume the reference data and assess the impact of the change across reports and business glossary terms.

To associate code list attributes with Data Governance and Catalog assets, you require the stakeholder role for Data Governance and Catalog assets.

For more information about associating code list attributes with Data Governance and Catalog assets, see *Asset Management* in the Data Governance and Catalog help.

Managing Data Quality dictionaries in code lists

You can create dictionaries based on attributes of code lists. The dictionaries that you create in Reference 360 appear in Data Quality in the read-only mode. You can edit the dictionaries only in Reference 360.

To create dictionaries in code lists, you require a Data Quality custom role with all permissions for the Dictionary asset in addition to a Reference 360 Primary Owner role or Primary Owner stakeholder role.

Ensure that the code lists that you use to create dictionaries contain less than 10,000 code values.

Create a dictionary

Create a dictionary in a code list, and configure the properties of the dictionary.

1. In the **Explore** panel, select a code list and click **Open**.
2. Click the **Dictionary** tab.
3. Click **Add**.
4. In the **Create Dictionary** dialog box, specify the following properties:
   a. Enter a name.
   b. Enter a description for the dictionary.
   c. Select the project or folder within which you want to save the dictionary.
   d. Select at least two attributes that you want to add as columns in the dictionary.

      When you select one attribute, Reference 360 duplicates the attribute as a second column in the dictionary.
   e. Click **Create**.

   The dictionary that you create in Reference 360 appear in Data Quality in the read-only mode.

Synchronize data in dictionaries

After you create dictionaries in a code list, synchronize data to populate the dictionaries in Data Quality with the relevant code values. You must synchronize data to reflect any updates to the code values.

1. In the **Explore** panel, select a code list and click **Open**.
2. Click the **Dictionary** tab.

3. Click the **Synchronize** icon.

   Note: You can synchronize up to 10,000 code values in a code list. If the code list contains more than 10,000 code values, Reference 360 disables the Synchronize icon.

Re-create the dictionary

If a dictionary that's associated with the code list is deleted in Data Quality, you can either delete the dictionary or re-create it in Reference 360.

1. In the **Explore** panel, select a code list and click **Open**.
2. Click the **Dictionary** tab.
3. Select the dictionary that was deleted in Data Quality.
4. In the **Details** panel, click the **Re-create** icon.

   The dictionary is re-created in Reference 360. The recreated dictionary appears in Data Quality.

Editing code lists

You can edit the details and code values of a code list.

For more information about editing code values, see "Editing code values" on page 127.

You can edit details, such as general properties, external URLs, notification recipients, attributes, display settings, and stakeholders of a code list. You can't edit the structure definition or attributes that the code list inherits from the reference data set.

For more information about the editable details of code lists, see "Code lists" on page 19.

Add multilingual code values to existing code lists

To add multilingual code values to an existing code list, enable the multilingual property for the required attributes in the code list. You can enable the property for all string type attributes, except for the Code attribute.

For more information about the impact of multilingual assets in Reference 360 and MDM SaaS, see "Rules and guidelines for managing multilingual assets" on page 144.

When you enable the multilingual property for one or more attributes and save the code list, the data in those attributes is deleted, so back up the attribute data before you enable the property.

To back up the existing code values, perform the following actions:

1. Export the code values as a CSV file that is prepared for localization before you enable the property for an attribute. Ensure that you select the **Prepare for Localization** option when you export the code values.
2. Edit the export file and add the multilingual code values.

After you enable the property, import the code values.

For more information about exporting existing code values and importing them as multilingual code values, see "Exporting code values" on page 130 and "Import multilingual code values into code lists" on page 152.

After you import the multilingual code values, add the preferred additional languages to display the attribute values.

Deleting code lists

Users with the Reference 360 Primary Owner role can delete code lists that are no longer needed.

You can't delete code lists that are locked. For more information about the considerations for deleting code lists, see “Considerations for deleting code lists” on page 122.

Note: Users with a Reference Data Management Basic Edition license can't delete default code lists.

1. Open the code list that you want to delete.
2. Click **Delete**.
3. Click **OK**.

Considerations for deleting code lists

You can delete code lists that you no longer need. When you delete a code list, the code list and its code values are removed from Reference 360.

You can't delete the following types of code lists:

- Code lists that are locked.
- Code lists that are specified as a dependent of another asset.
- Code lists that are specified as a reference data attribute for another asset.
- Code lists that are part of value mappings in crosswalks.
- Default code lists that are used as reference data for the picklist fields in Business 360 Console.
- Custom code lists that are associated with source systems in Business 360 Console.

Note: You can't delete the default code lists that are published in Business 360 Console.

After you delete a code list, you can't reuse its internal ID. If you try to create a code list with the same internal ID, either through asset migration or manually reuse the internal ID of a previously deleted code list in the API request body during model import or export, the asset migration and import and export models fail.

CHAPTER 6

Manage code values

A code value is a unique value, such as a business term, code, or lookup value. Code values are organized into code lists. Each code list contains code values from a single source application or industry standard list. You can add code values or import code values into a code list.

You can add or edit code values in a code list by locking the code list or without locking the code list. When you lock the code list, you restrict other users to make concurrent edits to the code list. When you don't lock the code list and edit a code value, other users can edit other code values in the same code list. The edited code value is locked for other users until you publish or discard the draft code list or an approver approves the updated code list.

When you add or edit a code value, you can update values for attributes that are set as output fields of advanced rule associations. After you save the code value, Reference 360 runs all the advanced rule associations and overwrites the output field values based on the input field values that you entered.

For example, consider a rule specification with a condition to copy the input field value to the output field. When the advanced rule association runs, the output field returns an empty value if the input field is empty. You can manually enter a value for the output field.

You can create crosswalks to map code values in a pair of code lists. Crosswalks provide a way to translate between the different code values each application uses for a business term.

For more information, see "Code values" on page 23.

To add code values, perform one of the following actions:

"Adding code values" on page 123

Chapter 9, "Import data" on page 152

Related Topics:

- "Code values" on page 23

Adding code values

Add code values to a code list by locking the code list or without locking the code list.

1. In the **Explore** panel, select a code list and click **Open**.

   The code list opens on a new tab.
2. To restrict other users from editing the code list, click **Lock**.

   A draft version of the code list is created, and the code list is locked for other users.

3. Click **Add Code Value at the Root Level**.

   The **New Code Value** dialog box appears.

4. If the code list is dependent on another code list or a reference data set, select a value in the **Dependency** field.
5. In the **Name** field, enter a name.
6. In the **Code** field, enter a code value.

   When you add a code value, if the reference data type attributes of a code list contain more than 100 values, type a few characters and select one of the matching values. If the reference data type attributes contain less than 100 values, select the value from the list.

   Note: When you search for a reference data attribute value, type a few characters and then press **Enter** or click the **Search** icon to display relevant matching values.

7. Optionally, complete the remaining code value fields.

Field	Description
Description	A description of the code value.
Status	The state of the code value in the life cycle. **Note:** If you don't see options in the list, configure your system reference data values. For more information, see "Adding values to system reference data" on page 100.
Effective Date	The date the status is effective from. **Note:** You can't type a date. To select or modify a date, use the **Calendar** icon. To clear the selection, click the **Remove** icon.

   If you set multilingual attributes for the code list, Reference 360 displays a separate field for the default language and for each additional language. The language is appended to the attribute display name in parentheses, such as Name (en-US) and Name (de-DE).

   Preview Notice: Effective in the October 2026 release, the ability to manage multilingual assets is available for preview.

   Preview functionality is supported for evaluation purposes but is unwarranted and is not supported in production environments or any environment that you plan to push to production. Informatica intends to include the preview functionality in an upcoming release for production use, but might choose not to in accordance with changing market or technical circumstances. For more information, contact Informatica Global Customer Support.

   If you set default values for code list attributes, the default values appear in the fields. You can also modify the default values. However, you can't set default values to multilingual attributes.

   For example, if you have added the City attribute and set a default value as Germany, you can view the default value in the City field. To modify the value, you can remove the value and enter a different value.

   Note: When you set default values, such as 1e or 9e308, for decimal or integer attribute types, Reference 360 doesn't display any value in the **Default Value** field.

8. Click **Save and New** to add additional code values, or click **Save**.

   The new code values appear in the code list. If you skipped step 2, a draft version of the code list is created now.

   Note: Users with a Reference Data Management Basic Edition license can't send the proposed changes for approval.

9. To send your changes for approval, perform the following actions:
   a. Click **Submit** after you finish adding code values to the code list.

      The **Send for Approval** window appears.
   b. Set the priority, due date, and optionally add a comment.
   c. In the **Compare Versions** section, review your proposed changes.
   d. Click **Send for Approval**.

   You receive an email notification confirming that you successfully sent the approval request. Users assigned the Business Steward stakeholder role for the asset or the Reference 360 Business Steward role are notified of the task. The task appears in the **Workflow Inbox** tab.

   Note: Users assigned the Reference 360 Business Steward role and the Business Steward stakeholder role for an asset can send their proposed changes for approval or directly publish their changes without approval.
10. To cancel the approval request, perform the following actions:
    a. Click **Cancel Approval Request**.

       A confirmation dialog box appears.
    b. Click **OK**.

    The code list is reset to the draft state. To unlock the code list, you must discard the draft.

Rules and guidelines for adding code values with dependent reference data attributes

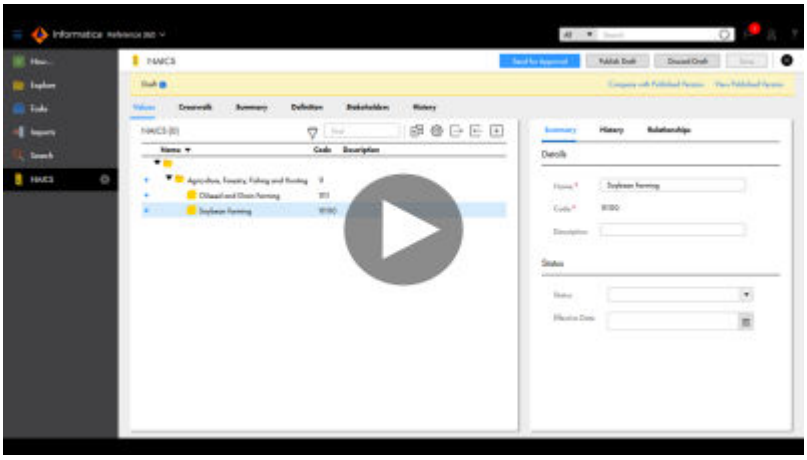
Consider the following rules and guidelines when you add code values with reference data type attributes:

- Reference 360 disables the dependent reference data attributes until you select a value for the reference data attribute that it depends on.
  For example, if the Country of Birth depends on the Continent of Birth, you can't select a value for the Country of Birth attribute until you select a value for the Continent of Birth attribute.
- If you modify the selected value for a reference data attribute, Reference 360 clears the selected values for its respective dependent reference data attributes.
- If you select a hierarchical code list for a reference data attribute that depends on another reference data attribute, Reference 360 displays the values in hierarchical structure.
- After you add default values for reference data attributes set with dependency, if you clear the default value for a reference data attribute, Reference 360 clears the default value for its dependent reference data attributes.
  For example, the Country of Birth attribute depends on the Continent of Birth attribute. You have set default values for the Country of Birth attribute as India and for the Continent of Birth attribute as Asia. When you add a code value, if you clear the default value for the County of Birth attribute, Reference 360 clears default value for the Continent of Birth attribute.

Creating child code values

Create child code values in a hierarchical code list. For example, you might want to add the Electric Power Distribution code value as a child code value of the Utilities code value.

The following video shows you how to create child code values in a hierarchical code list:



1. In the **Explore** panel, select a code list and click **Open**.

   The code list opens on a new tab.
2. Click **Lock**.

   A draft version of the code list is created.
3. Hover over the code value that you want to create a child code value under, and then click **New**.

   The **New Code Value** dialog box appears.
4. To copy all the values from the parent code value, perform the following actions:
   a. Click **Copy Values from Parent Code Value**.
   b. Edit the values that are different from the parent code value.
5. To enter values for the child code value, perform the following actions:
   a. In the **Name** field, enter a name.
   b. In the **Code** field, enter a code value.
   c. Optionally, complete the remaining code value fields.
6. To move code values, drag a code value to a new location. Click **OK**.
7. Create additional child code values.
8. Click **Save**.

   The new code values appear in the code list.

   Note: You can't add a child code value to multiple parent code values in hierarchical code lists.
9. To send your changes for approval, perform the following actions:
   a. Click **Submit** after you finish adding child code values.

      The **Send for Approval** window appears.
   b. Set the priority, due date, and optionally add a comment.
   c. In the **Compare Versions** section, review your proposed changes.

d. Click **Send for Approval**.

You receive an email notification confirming that you successfully sent the approval request. Users assigned the Business Steward stakeholder role for the asset or the Reference 360 Business Steward role are notified of the task. The task appears in the **Workflow Inbox** tab.

Note: Users assigned the Reference 360 Business Steward role and the Business Steward stakeholder role for an asset can send their proposed changes for approval or directly publish their changes without approval.

10. To cancel the approval request, perform the following actions:
    a. Click **Cancel Approval Request**.

       A confirmation dialog box appears.

    b. Click **OK**.

    The code list is reset to the draft state. To unlock the code list, you must discard the draft.

Editing code values

Edit code values in a code list by locking the code list or collaborating with other users without locking the code list.

1. In the **Explore** panel, select a code list and click **Open**.

   The code list opens on a new tab.
2. To restrict other users from editing the code list, click **Lock**.

   A draft version of the code list is created, and the code list is locked for other users.
3. On the **Values** tab, click the code value that you want to edit.

   The code value details panel displays the details of the code value in different tabs.
4. On the Summary tab, click the **Edit** icon.
5. Edit the details as required.
6. Click the **Save** icon.

   When you don't lock the code list, the edited code value appears locked for other users and can't be edited by other users.

   Note: Users with a Reference Data Management Basic Edition license can't send the proposed changes for approval.
7. To send your changes for approval, perform the following actions:
   a. Click **Submit** after you finish editing the code values in the code list.

      The **Send for Approval** window appears.

   b. Set the priority, due date, and optionally add a comment.

   c. In the **Compare Versions** section, review your proposed changes.

   d. Click **Send for Approval**.

   You receive an email notification confirming that you successfully sent the approval request. Users assigned the Business Steward stakeholder role for the asset or the Reference 360 Business Steward role are notified of the task. The task appears in the **Workflow Inbox** tab.

Note: Users assigned the Reference 360 Business Steward role and the Business Steward stakeholder role for an asset can send their proposed changes for approval or directly publish their changes without approval.

8. To cancel the approval request, perform the following actions:
   a. Click **Cancel Approval Request**.

      A confirmation dialog box appears.

   b. Click **OK**.

   The code list is reset to the draft state. To unlock the code list, you must discard the draft.

Viewing history of a code value

You can view the changes made to a code value of a code list in a chronological order. Use the historical data to track the changes made to the code value at a point in time in the past.

1. In the **Explore** panel, select a code list and click **Open**.

   The code list opens in a new tab.

2. On the **Values** tab, select a code value.

   The code value details panel displays the details of the code value in different tabs.

3. To view the change history of the code value, click the **History** tab.

   The change history of the code value appears.

4. On the **History** tab, click the **Expand** icon.

   The **History Details** page appears.

The following table lists the different user interface elements that you can find on the **History Details** page:

User Interface Element	Description
Show/Hide Filter icon	Displays the **Filter** menu. Click **Add Filter** and select the following values based on which you want to filter the history feed: - Date Range. Allows you to specify a time period. You can view the history feed for the specified time period. Use the following predefined time periods: - Today. Displays the changes occurred on the current date. - Last 7 days. Displays the changes occurred in the last seven days. - Last 30 days. Displays the changes occurred in the last 30 days. - Custom. Displays the changes occurred during the selected date range. - Event. Displays the changes based on the change events. You can filter based on a single or multiple event types. - Business Entity Fields. Filters the history feed based on the general properties of the code list. You can filter based on a single property or all properties.
Sort icon	Displays the following values based on which you can sort the history feed: - New to Old. Sorts the changes in reverse chronological order. - Old to New. Sorts the changes in chronological order.
View icon	Groups the history feed based on the selected frequency. Use one of the following frequencies: - Daily. Groups the history feed by day. - Monthly. Groups the history feed by month. - Yearly. Groups the history feed by year.
Expand icon	Displays the event details in full screen.

5. To view the change details, click each of the following tabs:
   - Business Entity Fields. Displays the changes to the fields of code value.
     To view all fields in the history feed, clear **Show the updated fields**.

     By default, the **Show the updated fields** check box is selected, and the updated fields appear in the history feed.
   - Task Details. Displays the task details when the record is associated to an approval workflow.
     - **Change List ID**. A system-generated unique identifier to track the changes made to the code values of the code list.
     - **Requested By**. The user who submitted the changes for approval.
     - **Requested On**. The date when the changes are sent for approval.
     - **Approved By**. The user who approved the changes.
     - **Approved On**. The date of approval.
6. Click **Close**.

Cloning code values

You can clone a code value in a code list.

Note: You can't clone a code value in a hierarchy asset.

1. In the **Explore** panel, select a code list and click **Open**.
2. Select a code value that you want to clone, and then click **Clone**.

   The **Clone Code Value** dialog box appears.

   Note: You can't clone multiple code values that you select.
3. In the **Name** field, modify the name.
4. In the **Code** field, modify the code.
5. Optionally, update the remaining fields.
6. To save the cloned code value, click **Save**, or click **Save and Clone Another** to create additional clones.

   You can view the cloned code values in your code list.

Note: Users with the Reference Data Management Basic Edition license can't send the proposed changes for approval.

If you don't have privileges to publish your proposed changes, send your changes for approval. If you are a Business Steward, you can publish your proposed changes without approval or send your changes for approval. For more information, see "Users, groups, and roles" on page 50.

Moving code values

You can cut and paste code values to move them across nodes in hierarchies and hierarchical code lists. You can also drag the code values across nodes.

1. In the **Explore** panel, select a hierarchy or hierarchical code list, and click **Open**.
2. Select the code values that you want to move across nodes, and then click **Cut**.

   Note: You must lock the code list to view the cut icon.

   The code values are copied to the clipboard.
3. Hover over to the node where you want to paste the code values, and click **Paste**.

   The **Move Code Values** dialog box appears.
4. Review the code values, and click **OK**.

   The code values are removed from the node where you cut them and moved to the node where you pasted them.

Exporting code values

Export code values in a code list to a CSV file. You can configure the format of the data and the attributes that you want to include in the exported file.

1. Open a code list.

2. Optionally, filter code values or search for specific code values in the code list.

   After you filter the code values, Reference 360 displays the complete hierarchy of the matched code values for hierarchical code lists.

   Note: Reference 360 displays the attributes that you selected in the column settings as filters.
3. Optionally, to view the code values at a point in time, select the date and time using the calendar or enter the date and time in the **Point in Time** field.
4. Click **Export Values**.

   The **Export** dialog box appears.
5. On the **General** tab, enter a file name.
6. To export the parent code value for each code value in the hierarchy, select **Include hierarchy**.

   This option is available only for hierarchical code lists.
7. To export the display attributes of the code list, select **Include Display Attributes**.
8. To export the metadata of the code list, select **Include Metadata Attributes**.

   You can export the following metadata attributes:

   - Last updated date
   - Last updated by user
   - Created date
   - Created by user
   - Last updated date of parent relationship
   - Last updated by user of parent relationship

   Note: You can export the last updated date of parent relationship and last updated user of parent relationship attributes only for hierarchical code lists.
9. In the **Values to Export** option, select one of the following options:

   - All. Exports all code values in the code list.
   - Currently Listed. Exports only the filtered set of code values, specific code values based on the search results, or the code values that existed at a specific point in time.
10. Optionally, configure how you want the data to be formatted in the file that you export.

Field	Description
Delimiter	Character used to separate values.
Date Format	Format used for dates.
Decimal Mark	Decimal separator used for numbers.
Thousands Separator	Grouping separator used for numbers.

11. If you export the code values to back up attribute data before enabling the multilingual property, select **Prepare for Localization** in the **Localization** section.

    When an export file isn't prepared for localization, you have to manually add the Language field and its values to the export file.

    For more information about using the exported file for import, see "Import multilingual code values into code lists" on page 152.

12. Optionally, click **Columns**, and then select the attributes to include in the exported file.
    For example, you might want to only include the Name and Code attributes for code values.
13. Click **Export**, and then save or open the exported file.

The export file displays internal field names based on the attribute names that you defined. For more information about internal field names of attributes, see "Step 2. Define the structure, attributes, and display settings of code lists" on page 111.
In the exported CSV file, the number of rows is based on the number of target languages, and the file includes the _language and _localizedContent fields. During import, the import process automatically maps the _language and _localizedContent fields.

The following table lists a sample exported CSV file that includes _language and _localizedContent fields:

_language	_localizedContent.Name	Code	Description	status.status.key	effectiveDate
en-US	Female	F	Identifies a person as female.		18-06-2025
pt-BR	Feminino	F			18-06-2025
en-US	Male	M	Identifies a person as male.		18-06-2025
pt-BR	Masculino	M			18-06-2025
en-US	Unknown	U	Indicates that the gender is unknown or not specified.		18-06-2025
pt-BR	Ignorado	U			18-06-2025

Deleting code values

You can propose to delete code values that you no longer need.

1. In the **Explore** panel, select a code list and click **Open**.
   The code list opens on a new tab.
2. Click **Lock**.
   A draft version of the code list is created.
3. Hover over the code values that you want to delete, and then click **Delete**.
4. Optionally, delete additional code values.

When you are done deleting code values, send your changes for approval.

Considerations for deleting code values

You can delete code values that you no longer need. When you delete a code value, the code value is permanently deleted from Reference 360.

You cannot delete the following types of code values:

- Code values that are defined as a parent code value in a hierarchical code list

- Code values that are used as a Reference Data attribute for another code list
- Code values that are used as a dependency in another code list
- Code values that are part of a value mapping in a crosswalk
- Code values that are part of a hierarchy asset

Purging code values

In Business 360 Console, you can purge code values of code lists along with their relationships and historical data.

When you purge data, you permanently delete data from Reference 360 without deleting data models and other configuration details of your organization. You can purge code values of custom code lists that aren't picklist values of business entities in business applications, such as Customer 360 SaaS.

You can purge data in the development, QA, and production environments. When you purge data, a purge job is automatically created and triggered.

Note: Before you purge code values of code lists in production environments, analyse the data you want to purge. You can't recover the purged data.

For more information about purging data, see Purging data.

Rules and guidelines for purging code values

Before you purge data, consider the following rules and guidelines to prevent data issues:

- No users have logged in to business applications.
- No jobs are running.
- All workflow tasks related to the code list that you want to purge are complete.
- You can purge code values of custom code lists that aren't picklist values of business entities in business applications, such as Customer 360 SaaS.
- Review all assets that depend on the code list you are about to purge and assess their impact.
- If you have only the SaaS Edition - Reference 360 license, you must configure purge privileges for the Reference 360 Administrator user role to purge code values.
  For more information about configuring Reference 360 Administrator privileges, see "Configuring privileges to purge code values" on page 133.
- If you have the SaaS Edition - Reference 360 license along with any MDM SaaS business application edition license, the Reference 360 Administrator user role can't purge code values but the organization Administrator user role can.
- If you have only the SaaS Edition - Reference 360 license and have configured purge privileges for the Reference 360 Administrator user role, you need to use Business 360 Console to purge reference data assets. Ensure you don't perform any other operations in Business 360 Console.

Configuring privileges to purge code values

If you have only the SaaS Edition - Reference 360 license, you must configure purge privileges for the Reference 360 Administrator user role to access Business 360 Console and purge code values.

1. In Administrator, click **User Roles**.

2. Click **Reference 360 Administrator**.
3. In the **Services** field, select **MDM Configuration**.
4. On the **Features** tab, enable the **Configuration and Authoring**, **Execute Business 360 Flows**, and **Manage Job Instances** features.

Performance considerations for purging code values

The performance of data purge depends on the data set, data volume, and POD-level limits.

To optimize performance of data purge, consider the aspects of data, such as volume and complexity, and purging limits.

Complexity of the data set

The performance of purge jobs depends on the size and complexity of your data set, such as the number of relationships and historical data.

Concurrency limits at the POD level

MDM SaaS limits the number of concurrent purge jobs that can run on a POD. As a result, your purge job might be queued if multiple jobs run concurrently on a POD.

Purging code values

You can purge code values that belong to a code list along with their relationships and historical data.

1. In Business 360 Console, click **Global Settings** > **Purge Data**.
2. Click **Start**.
3. In the **Purge Data** dialog box, select **Purge records**.
4. Select a code list.
5. Select one of the following options for code values to purge:
   - Purge all records. Permanently deletes all the code values, that belong to the selected code list along with their relationships and historical data.
   - Purge deleted source records. Permanently deletes only the deleted code values of the selected code list along with their relationships and historical data.
6. Click **Next**.
7. Enter your organization ID, and click **Submit**.

   You can monitor the status of the job on the **My Jobs** page. For more information about monitoring jobs, see *Monitoring Jobs*.

CHAPTER 7

Manage crosswalks

A crosswalk is a visual representation of a one-way relationship between code values in a pair of code lists. A reference data set can contain many code lists, and each code list contains a variation of the same type of code values. Crosswalks provide a way to translate between the different variations each code list uses.

For more information, see "Crosswalks" on page 23.

To create a crosswalk, perform the following actions:

1. Create a crosswalk.
2. Create value mappings.
3. Assign stakeholders.

Related Topics:

- "Crosswalks" on page 23

Step 1. Create a crosswalk

Create a crosswalk to map code values in a source code list to code values in a target code list from same or different reference data sets. Value mappings provide a way to translate code values in one code list to code values in another code list. You can specify internal ID and alias for a crosswalk.

1. Click **New** and select **Crosswalk**.

   The **New Crosswalk** page appears and displays the **Summary** tab.

   Note: You cannot configure the general properties. A crosswalk inherits its general properties from the source and target reference data sets.
2. In the **Source Reference Data Set** field, select the reference data set that contains the code list you want to map.
3. In the **From Code List** field, select the source code list.

   The crosswalk is associated with the source code list.
4. In the **Target Reference Data Set** field, select the reference data set that contains the code list you want to map to the selected source code list.
5. In the **To Code List** field, select the target code list.

   The target code list contains code values that you want to map.
6. Configure the additional properties fields.

Note: Crosswalks inherit the name from the reference data set.

The following table lists the properties that you can configure for a crosswalk:

Additional Property Field	Description
Name	Display name of a crosswalk.
Internal ID	Required. Unique identifier of a crosswalk, limited to a maximum of 45 characters. You can use alphanumeric characters and underscores. After you save an asset, you can't edit the internal ID. **Note:** You can't specify internal IDs for existing assets.
Alias	Alternative identifier for a crosswalk, limited to a maximum of 70 alphanumeric characters. After you save an asset, you can edit the alias.
Description	Short description of the crosswalk. The description cannot exceed 4000 characters.
Confidentiality	Confidentiality level of the code list inherited from the reference data set. The confidentiality levels are confidential, internal, restricted, and public.
Status	State of a crosswalk in the life cycle. **Note:** If you do not see options in the list, configure your system reference data values. For more information, see "Adding values to system reference data" on page 100.
Effective Date	Date from when the status is effective.
Approved by	User who approved a crosswalk.
Approved on	Date of approval.

7. Click **Save**.

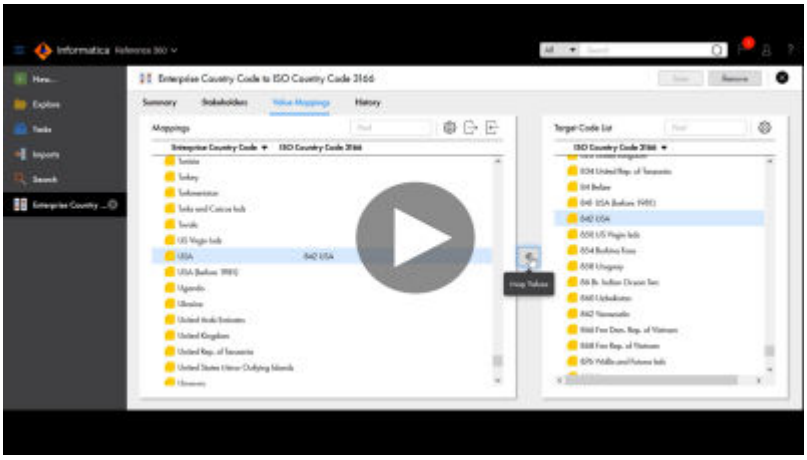
   A new crosswalk is created. The crosswalk name displays the source code list and target code list. If the target reference data set is different from the source reference data set, the crosswalk name displays the name of the target reference data set adjacent to the target code list.
   In the **Explore** panel, you can view the crosswalk name appended with the target reference data set name when the source and target reference data sets are different.

   Note: Users with a Reference Data Management Basic Edition license can create crosswalks only between the default code list and another code list of the same reference data set.

Step 2. Configure value mappings

Map code values in one code list to code values in another code list to indicate that the code values represent the same business term.

The following video shows you how to create value mappings:



1. In the **Explore** panel, select a crosswalk and click **Open**.
2. On the **Value Mappings** tab of the crosswalk, click **Lock**.

   A draft version of the crosswalk is created, and the crosswalk is locked for other users. Also, the target code list appears after you lock the crosswalk.
3. To create a value mapping, perform one of the following actions:
   - In the **Mappings** panel, select a source value that you want to map. In the **Target Code List** panel, select the equivalent code value, and click **Map Values**.
   - Drag a code value from the **Target Code List** panel onto the equivalent code value in the **Mappings** panel.

   The mapping appears in the **Mappings** panel.
4. To create additional value mappings, repeat 3.

   Note: When you search for value mappings of a crosswalk, the search results display only the matching code values of the source code list.
5. To import value mappings, click **Import Mappings**.

   For more information about importing value mappings, see "Import process" on page 155.
6. To remove a value mapping, perform the following actions:
   a. Hover over a value mapping, and click the **Remove** icon.

      The **Remove Value Mappings** dialog box displays the list of available mappings for the specific value mapping.

   b. Select the mappings and click **Remove**.

      To select all the mappings, click **Mappings**.
7. To send your changes for approval, perform the following actions:
   a. Click **Submit** after you finish proposing changes to the crosswalk.

      The **Send for Approval** window appears.

b. Set the priority, due date, and optionally add a comment.

c. In the **Compare Versions** section, review your proposed changes.

d. Click **Send for Approval**.

You receive an email notification confirming that you successfully sent the approval request. Users assigned the Business Steward stakeholder role for the asset or the Reference 360 Business Steward role are notified of the task. The task appears in the **Workflow Inbox** tab.

Note: Users assigned the Reference 360 Business Steward role and the Business Steward stakeholder role for a crosswalk can send their proposed changes for approval or directly publish their changes without approval.

8. To cancel the approval request, perform the following actions:

a. Click **Cancel Approval Request**.

A confirmation dialog box appears.

b. Click **OK**.

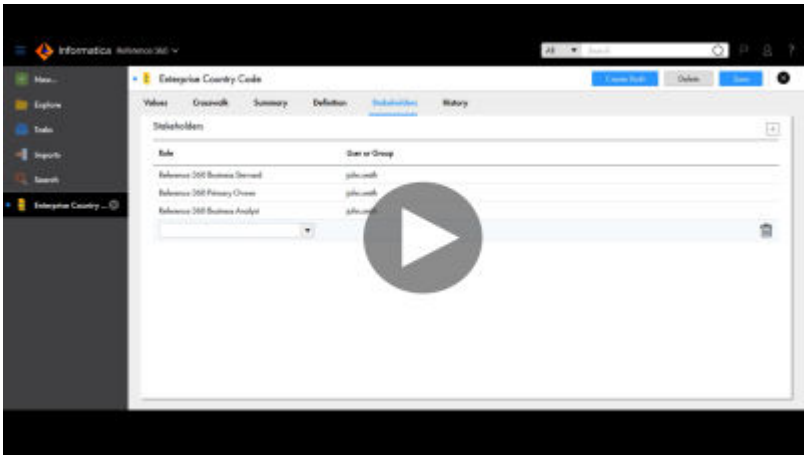
The crosswalk is reset to draft state. To unlock the crosswalk, you must discard the draft.

Step 3. Assign stakeholders

Assign stakeholder roles to users who play a role in using, creating, or maintaining the asset. Stakeholder roles determine a user's privileges for the asset.

When a user manages an asset as a stakeholder, they have the combined privileges provided by their stakeholder role and their Reference 360 role. For more information about stakeholder roles, see "Stakeholder roles" on page 53. For more information about assigning roles, see "Guidelines for assigning roles" on page 55.

The following video shows you how to assign stakeholders:



1. Click **Stakeholders**.

The **Stakeholder** tab opens.

2. Click **Add**.

A list appears in an empty row.

3. In the **Role** list, select a stakeholder role.

The **New Stakeholder** dialog box appears.

4. Select a user name and click **Add**.

   The user name of the stakeholder appears in the row.

5. Click **Save**.

When you create a reference data set and assign a stakeholder role to a user or user group, Reference 360 automatically adds an additional Reference 360 Business Analyst stakeholder role to the users and groups.

Viewing history of a crosswalk

You can view the changes made to a crosswalk in a chronological order. Use the historical data to track the changes made to the crosswalk at a point in time in the past.

1. In the **Explore** panel, select a crosswalk and click **Open**.

   The crosswalk opens in a new tab.

2. Click the **History** tab.

   By default, the **Crosswalk Definition History** tab opens. The **Crosswalk Definition History** tab displays the change history of the crosswalk properties.
   The following table lists the different user interface elements that you can find on the **Crosswalk Definition History** tab of the **History** tab:

User Interface Element	Description
Show/Hide Filter icon	Displays the **Filter** menu. Click **Add Filter** and select the following values based on which you want to filter the history feed: - Date Range. Allows you to specify a time period. You can view the history feed for the specified time period. Use the following predefined time periods: - Today. Displays the changes occurred on the current date. - Last 7 days. Displays the changes occurred in the last seven days. - Last 30 days. Displays the changes occurred in the last 30 days. - Custom. Displays the changes occurred during the selected date range. - Event. Displays the changes based on the change events. You can filter based on a single or multiple event types. - Business Entity Fields. Filters the history feed based on the general properties of the code list. You can filter based on a single property or all properties.
Sort icon	Displays the following values based on which you can sort the history feed: - New to Old. Sorts the changes in reverse chronological order. - Old to New. Sorts the changes in chronological order.

User Interface Element	Description
View icon	Groups the history feed based on the selected frequency. Use one of the following frequencies: - Daily. Groups the history feed by day. - Monthly. Groups the history feed by month. - Yearly. Groups the history feed by year.
Expand icon	Displays the event details in full screen.

3. To view all fields in the history feed, clear **Show the updated fields**.

   By default, the **Show the updated fields** check box is selected, and the updated fields appear in the history feed.

4. To view the history of changes to value mappings of the crosswalk in a table, click the **Value Mapping History** tab.

   The **Value Mapping History** tab displays only 2500 rows of history feed. To view the complete history feed, use the audit trail REST API.

Exporting value mappings of crosswalks

Export value mappings of all crosswalks or any selected crosswalk of a code list to a CSV file.

1. Open a code list.
2. Click **Crosswalk** > **Export Mappings**.

   The **Export** dialog box appears with a warning message indicating that you check the formatting of the exported file.

3. In the **General** section, enter a file name.
4. In the **Crosswalk** section, specify the crosswalks to export.
   a. Select the direction of crosswalks that you plan to export.
   b. Select the crosswalks that you plan to export, or click **Select All**.
5. Optionally, in the **Advanced Export Format** section, configure the format for the exported data.

The following table describes the properties for the exported data:

Property	Description
Character Encoding	Mapping of the characters from a language or group of languages to hexadecimal code. The supported value is `UTF-8`.
Delimiter	Character used to separate the crosswalks. You can use the following delimiters: - Asterisk (*) - Circumflex (^) - Colon (:) - Comma (,) - Pipe (|) - Section (§) - Semicolon (;) - Space - Tab
Mapping Delimiter	Character used to separate the value mappings. You can use the following mapping delimiters: - Asterisk (*) - Circumflex (^) - Colon (:) - Comma (,) - HYPHEN (-) - Pipe (|) - Section (§) - Semicolon (;) - Space - Tab

6. Click **Export**.

Managing Data Quality dictionaries in crosswalks

You can create a dictionary based on value mappings of a crosswalk. The dictionary that you create in Reference 360 appears in Data Quality in the read-only mode. You can edit the dictionary only in Reference 360.

To create dictionaries in crosswalks, you require a Data Quality custom role with all permissions for the Dictionary asset in addition to a Reference 360 Business Steward role or Business Steward stakeholder role.

Ensure that the crosswalks that you use to create dictionaries contain less than 10,000 value mappings.

Create a dictionary

Create a dictionary in a crosswalk, and configure the properties of the dictionary.

1. In the **Explore** panel, select a crosswalk and click **Open**.
2. Click the **Dictionary** tab.

3. Click **Add**.
4. In the **Create Dictionary** dialog box, specify the following properties:
   a. Enter a name.
   b. Enter a description for the dictionary.
   c. Select the project or folder within which you want to save the dictionary.
   d. The **All Mappings** option is selected by default to add the value mappings of the crosswalk as columns in the dictionary.
   e. Click **Create**.

   The dictionary that you create in Reference 360 appear in Data Quality in the read-only mode.

Synchronize data in a dictionary

After you create a dictionary in a crosswalk, synchronize data to populate the dictionary in Data Quality with the relevant value mappings. You must synchronize data to reflect any updates to the value mappings.

1. In the **Explore** panel, select a crosswalk and click **Open**.
2. Click the **Dictionary** tab.
3. Click the **Synchronize** icon.

   Note: You can synchronize up to 10,000 value mappings in a crosswalk. If the crosswalk contains more than 10,000 value mappings, Reference 360 disables the Synchronize icon.

Re-create the dictionary

If a dictionary that's associated with the crosswalk is deleted in Data Quality, you can either delete the dictionary or re-create it in Reference 360.

1. In the **Explore** panel, select a crosswalk, and click **Open**.
2. Click the **Dictionary** tab.
3. Select the dictionary that was deleted in Data Quality.
4. In the **Details** panel, click the **Re-create** icon.

   The dictionary is re-created in Reference 360. The recreated dictionary appears in Data Quality.

Deleting crosswalks

Users with the Reference 360 Business Steward role can delete crosswalks that are no longer needed.

You can't delete crosswalks that are locked. For more information about the considerations for deleting crosswalks, see “Considerations for deleting crosswalks” on page 143.

1. Open the crosswalk that you want to delete.
2. Click **Delete**.
3. Click **OK**.

Considerations for deleting crosswalks

You can delete crosswalks that you no longer need. When you delete a crosswalk, the crosswalk and its value mappings are removed from Reference 360.

You can't delete the following types of crosswalks:

- Crosswalks that are locked.
- Crosswalks that contain relationships between default code lists that are published and custom code lists associated with source systems in Business 360 Console.
- Crosswalks that are part of a taskflow that the reference data import and export jobs use.

After you delete a crosswalk, you can't reuse its internal ID. If you try to create a crosswalk with the same internal ID, either through asset migration or manually reuse the internal ID of a previously deleted crosswalk in the API request body during model import or export, the asset migration and import and export models fail.

CHAPTER 8

Manage multilingual assets

You can manage and display attribute values in multiple languages. When you enable multiple languages, users across your organization can view and work with reference data assets in their preferred language.

The languages available to your organization are defined in the Localization Languages code list. This predefined code list identifies each language by a BCP 47 language code and specifies the default language for your organization. When you set a string attribute of a reference data set or code list as multilingual, you can view the multilingual values in each preferred language that you add to the Localization Languages code list. If a value isn't available in your preferred language, Reference 360 displays the value in the default language.

You can add languages to the Localization Languages code list manually through the user interface or use the external localization REST API to import the language code values into the code list. For more information about the external localization REST API, see "external localization" on page 516.

To maintain reference data assets in multiple languages, configure the Localization Languages code list with the languages that your organization uses. Set the required string attributes as multilingual and enter a value for each attribute in every configured language.

Preview Notice: Effective in the October 2026 release, the ability to manage multilingual assets is available for preview.

Preview functionality is supported for evaluation purposes but is unwarranted and is not supported in production environments or any environment that you plan to push to production. Informatica intends to include the preview functionality in an upcoming release for production use, but might choose not to in accordance with changing market or technical circumstances. For more information, contact Informatica Global Customer Support.

Rules and guidelines for managing multilingual assets

Before you create multilingual assets, review how they work in Reference 360 and MDM SaaS.

Configuring multilingual attributes

Consider the following rules and guidelines when you configure multilingual attributes:

- Before you set an attribute as multilingual, remove all rule associations configured for the attribute.
- After you enable the multilingual property and save the asset, you can't disable the property or add rule associations to the attribute.

- You can set only string attributes as multilingual, such as Name and Description. You can't set the Code attributes as multilingual.
- You can't set default values for multilingual attributes. When you set an attribute as multilingual, Reference 360 clears the default value that you previously set for the attribute.
- Don't use the reserved words _language and _localizedContent as attribute names. For a complete list of reserved keywords, see "Reserved keywords" on page 115.
- After you set attributes in a code list as multilingual, complete all the export and import model updates for the code list. You can then reindex the code list so that search results include the localized data. To reindex the code list, contact Informatica Global Customer Support.
- After you optimize assets in your organization, you can't enable the multilingual property on any reference data assets in Reference 360. For more information about asset optimization, see "asset optimization modelling" on page 522 and Enhancing Reference 360 UI Performance for Large Data Sets.

Managing display languages in a code list

Consider the following rules and guidelines when you manage display languages in a code list:

- You can't remove an additional language for a code list if a code value contains data in that language. To remove the language, first purge or clear the values in the code list. You can then remove the language and reimport the data. Therefore, when you configure the code list, select the additional languages based on your anticipated business needs.
- Reference 360 uses the browser language preferences to determine which language displays picklist values in the business application. To display picklist values in a specific language, move that language to the top of the language preferences in your browser, add the corresponding BCP 47 language code to the Localization Languages code list, and maintain values for it.
- In MDM SaaS business applications, multilingual assets appear as picklist field values in your browser locale language. If a value is not available in that language, the business application displays it in the default language of the reference data. MDM SaaS business applications use the following rules to match languages:
  - A browser language matches any language code that starts with it. For example, `de` matches `de`, `de-LI`, and `de-AT`, but `de-LI` matches only `de-LI` and doesn't match `de`. If your browser languages are `de-IN` and then `de`, and the code list contains `de-LI` but not `de-IN` or `de`, Reference 360 shows values in `de-LI`.
  - When you select a regional language in your browser, the browser also checks its base language. For example,if you select `de-LI` and the code list contains `de` but not `de-LI`, Reference 360 shows values in `de`. If the code list contains both `de-LI` and `de`, Reference 360 shows values in `de-LI`.
    To show values in a specific regional language, add the regional language BCP 47 code to the Localization Languages code list and maintain values for it.
- When you view an attribute value in a language for which no value exists, Reference 360 displays an empty value for that language. For example, you set the Status attribute as multilingual and store values in English (en-US) and French (fr-FR). When you choose to view the value in Japanese (ja-JP), Reference 360 returns an empty value because Reference 360 stores no Japanese value for the attribute.
- When you configure a combination of display attributes, such as a concatenation of the Name, Code, and Description, to represent code values in other assets, and if a multilingual attribute has no stored value, Reference 360 shows an empty string for each field that has no value in the preferred language. For example, you might want the Code, Name, and Description attributes to represent code values in the Country Codes code list. You might have a code value with the Name as `Canada`, the Code as `124`, and no stored value for the Description attribute in the preferred language. When the code value appears in other assets, it appears as `124-Canada-`. For more information about configuring display attributes, see "Configuring display attributes" on page 174.

Viewing code values and history

Consider the following rules and guidelines when you view code values and their change history:

- You can't sort code values based on multilingual attributes on the Values tab of a multilingual code list.
- The change history of a code value with multilingual attributes shows values in the browser language you used when you created or updated the code value. If you change your browser language later, earlier history events continue to show values in the original language. For example, if you update a code value while your browser uses Japanese, the history event continues to show Japanese values after you switch to English.
- When you create a code value in a multilingual code list, the default value for a required reference attribute isn't displayed. Reference 360 skips validation and saves the code value even though the attribute appears empty. However, the default value shows up after you save the code value.

Exporting and importing multilingual data

Consider the following rules and guidelines when you export and import multilingual assets:

- When you export a code list at a point in time that precedes localization, the export file contains only the Code attributes, and you can't export localized attributes, such as Name and Description.
- When you enable the multilingual property for one or more attributes and save the code list, the data in those attributes is deleted, so back up the attribute data before you enable the property. Export the code values to a CSV file that is prepared for localization, so that you don't have to add language rows manually. Edit the export file, add multilingual code values, and then use it as the source file for import after you enable the property.

  For more information about exporting and reimporting code values, see "Exporting code values" on page 130 and "Import multilingual code values into code lists" on page 152.
- In the exported CSV file, the number of rows is based on the number of target languages, and the file includes the _language and _localizedContent fields. During import, the import process automatically maps the _language and _localizedContent fields.
  The following table describes a sample exported CSV file that includes _language and _localizedContent fields:

_language	_localizedContent.Name	Code	Description	status.status.key	effectiveDate
en-US	Female	F	Identifies a person as female.		18-06-2025
pt-BR	Feminino	F			18-06-2025
en-US	Male	M	Identifies a person as male.		18-06-2025
pt-BR	Masculino	M			18-06-2025
en-US	Unknown	U	Indicates that the gender is unknown or not specified.		18-06-2025
pt-BR	Ignorado	U			18-06-2025

- When you export a code list that contains multilingual attributes, Reference 360 ignores sort filters applied to multilingual attribute columns. The exported data doesn't reflect the specified order.

- During file import, you can import data for a language code that isn't in the Localization Languages code list. The import doesn't require a value for the default language. The import lets you skip the value for the default language though a value for the default language is required. For example, you import a code value with the language code `fr-FR`, which isn't in the Localization Languages code list but is a valid BCP 47 code. If `en-US` is the default language, Reference 360 requires a value for the Name (en-US) attribute. However, the import doesn't require it, and the import completes successfully because the language code is valid.

Using multilingual picklist values in MDM SaaS business applications

Consider the following rules and guidelines when you use multilingual picklist values in MDM SaaS business applications:

- When you change a multilingual picklist field value, the history event displays the Code attribute value instead of the Name attribute value of the reference data.
- When you filter search results, the Filters panel displays values in your browser locale or in the default language of the picklist if not localized for your browser locale. For example, if a field is localized into German with English as the default language, a French-locale browser displays the English values.
- When you sort records, use auto-suggest, or view the search results, picklist values appear in the default language configured for the field, regardless of browser locale.
- When you export records that have localized picklist fields, the exported data contains only the codes for those fields, not their values.
- After you localize reference data, reindex the records in each business entity that uses the reference data so that you can search for records by using the localized data in picklist fields.
  For more information about searching for records with localized picklist values, see *Manage records* in the *Customer 360* help.
- When you retrieve business entity records through the Business 360 REST APIs, use the optional `Accept-Language` request header to get picklist values in a specific language. If you don't specify the language in the header, or if no value exists in the requested language, the APIs return values in the default language of the reference data.
  For more information about retrieving localized picklist field values using Business 360 REST APIs, see *Business 360 REST API Reference*https://www4.informatica.com/onlinehelp/default.aspx?product=IICS/prod&ver=b360&lang=en&contextID=MULTILINGUAL_PICKLIST in Business 360 Console help.

  For more information about search indexes, see *Defining generate search indexes or match keys job* in Business 360 Console help.

Localization Languages code list

The Localization Languages code list is a predefined code list that defines the set of languages available in your organization to display reference data and also the default language that applies at the organization level.

You can view the Localization Languages code list in the Explore panel.

You can add your preferred languages to the Localization Languages code list based on the Best Current Practice (BCP) 47 language code, and edit the code list to add or delete language code values. When you add a language to this code list, the language becomes available to display the attributes of your reference data assets in that language.

The Localization Languages code list includes code values for each language identified by a BCP 47 language code. For example, if the Localization Languages code list contains the code fr-FR, you can choose to view the attributes in French. For more information about BCP 47 language code, see "BCP 47 code format" on page 149.

You can't add custom attributes, define rules, or configure crosswalks, dictionaries, stakeholders, or history for the code list in the Localization Languages code list.

Adding language code values

You can add language code values to the Localization Languages code list with or without locking the code list.

1. In the **Explore** panel, select the **Localization Languages** code list and click **Open**.

   The code list opens in a new tab. The **Values** tab lists the code values.

2. To restrict other users from editing the code list, click **Lock**.

   A draft version of the code list is created, and the code list is locked for other users.

3. Click **Add Code Value at the Root Level**.
4. On the **New Code Value** dialog box, enter a name in the mandatory name field.
5. Optionally, you can enter the name of the language for each of the supported languages you choose to display the attributes in.
6. In the **Code** field, enter a code value.

   For more information about the list of supported languages and their codes, see "BCP 47 code format" on page 149.

7. Click **Save and New** to add additional code values, or click **Save**.

   The new code values appear in the code list. If you skipped step 2, a draft version of the code list is created now.

   Note: Users with a Reference Data Management Basic Edition license can't send the proposed changes for approval.

8. To send your changes for approval, perform the following actions:
   a. Click **Submit** after you finish adding code values to the code list.

      The **Send for Approval** dialog box appears.

   b. Set the priority, due date, and optionally add a comment.
   c. In the **Compare Versions** section, review your proposed changes.
   d. Click **Send for Approval**.

   You receive an email notification confirming that you successfully sent the approval request. Users assigned the Business Steward stakeholder role for the asset or the Reference 360 Business Steward role are notified of the task. The task appears in the **Workflow Inbox** tab.

   Note: Users assigned the Reference 360 Business Steward role and the Business Steward stakeholder role for an asset can send their proposed changes for approval or directly publish their changes without approval.

9. To cancel the approval request, perform the following actions:
   a. Click **Cancel Approval Request**.

      A confirmation dialog box appears.
   b. Click **OK**.

   The code list is reset to the draft state. To unlock the code list, you must discard the draft.

BCP 47 code format

Reference 360 uses BCP 47 language codes to identify the languages that you add to the Localization Languages code list.

Each language that your organization uses is represented by a BCP 47 language code. You enter these codes as code values in the Localization Languages code list, and Reference 360 uses them to store and retrieve the language-specific values of multilingual attributes.

For example, English (United States) is denoted as en-US and English (United Kingdom) as en-GB.

A language tag consists of hyphen-separated subtags in the following order:

1. Language. Two or three lowercase letters, such as en for English.
2. Region. Two uppercase letters for country or three digits for the area, such as ES for Spain or 013 for Central America.

The following table describes example BCP 47 language codes and the languages and regions that they represent:

BCP 47 Language Code	Language	Region	Description
de-AT	German	Austria	Austrian German
de-CH	German	Switzerland	Swiss German
de-DE	German	Germany	Standard German as spoken in Germany
en-AU	English	Australia	Australian English
en-CA	English	Canada	Canadian English
en-GB	English	United Kingdom	British English
en-IE	English	Ireland	Irish English
en-IN	English	India	Indian English
en-NZ	English	New Zealand	New Zealand English
en-US	English	United States	US English
en-ZA	English	South Africa	English South Africa
es-AR	Spanish	Argentina	Argentine Spanish

BCP 47 Language Code	Language	Region	Description
es-CL	Spanish	Chile	Chilean Spanish
es-CO	Spanish	Colombia	Colombian Spanish
es-ES	Spanish	Spain	Castilian Spanish as spoken in Central-Northern Spain
es-MX	Spanish	Mexico	Mexican Spanish
es-US	Spanish	United States	American Spanish
fr-BE	French	Belgium	Belgian French
fr-CA	French	Canada	Canadian French
fr-CH	French	Switzerland	"Swiss" French
fr-FR	French	France	Standard French as spoken in France
ja-JP	Japanese	Japan	Japanese as spoken in Japan
pt-BR	Portuguese	Brazil	Brazilian Portuguese
pt-PT	Portuguese	Portugal	European Portuguese as written and spoken in Portugal

You can also check your browser settings to view the list of supported languages.

Viewing language code values

You can view language code values in each configured language for the Localization Languages code list. Reference 360 displays a separate column for each language and appends the language code to the attribute display name in parentheses, such as Name (en-US).

1. In the **Explore** panel, select the **Localization Languages** code list and click **Open**.

   The code list opens in a new tab. The **Values** tab lists the code values.
2. Click the **Values** tab to view the list of code values.
3. Click the Translate icon to select the languages in which you want to view the code values from the language list.

   Reference 360 displays a separate column for each selected language.
4. To add languages to display the code values, on the **Definition** tab, in the **Localization Settings** section, add your preferred additional language to show in the list of asset display languages.
5. On the **Values** tab, select a code value.

   The code value Details panel displays the details of the code value in different tabs.

Deleting language code values

You can delete language code values that you no longer need.

1. In the **Explore** panel, select the **Localization Languages** code list and click **Open**.

   The code list opens in a new tab. The **Values** tab lists the code values.

2. Click **Lock**.

   A draft version of the code list is created.

3. Select the code value that you want to delete, and then click **Delete**.

   Note: You can't delete a code value that is set as a default langauge for the organization. To delete the code value, assign a different default language and then delete it.

When you are done deleting code values, send your changes for approval.

CHAPTER 9

Import data

You can import code lists, crosswalks, and related code values into a hierarchy from a CSV file into Reference 360.

Import code lists and crosswalks

When you import code lists and crosswalks, the import process parses the import data and automatically maps the columns of the source file with the target fields. The automatically mapped fields display confidence indicators to indicate the accuracy of the mapping. For the fields with low confidence scores or incorrect field mappings, you can manually map the source columns with the required target fields.

When you set default values for code list attributes and import code lists that don't contain any values for the attributes, Reference 360 displays the default value for the empty attributes. If you import a code list that contains values for the attributes, the import process retains the values in the source file.

If you enable business IDs for code lists, you can specify business IDs for the code values when you import new code values through REST APIs or the user interface.

Note: Ensure that business IDs that you specify don't contain special characters.

Import multilingual code values into code lists

To import multilingual code values into code lists, use a CSV file containing the multilingual code values. You can also use an export file that was prepared for localization to import multilingual code values.

Note: When **Prepare for Localization** isn't enabled before you export the code values, you have to manually add the Language field and its values to the export file. Otherwise, import fails. You can also add one row for each target language and code value to the file.

For more information about exporting code values and adding multilingual code values to existing code lists, see "Exporting code values" on page 130 and "Add multilingual code values to existing code lists" on page 121.

The following table shows the source file format for the Item Status code list, with the Name and Description code values localized into French and Spanish:

Language	Name	Code	Description
en-US	In Stock	in_stock	The item is in stock.
fr-FR	En stock	in_stock	L'article est en stock.
es-ES	En stock	in_stock	El artículo está en stock.
en-US	Out of Stock	out_of_stock	The item is out of stock.
fr-FR	En rupture de stock	out_of_stock	L'article est en rupture de stock.
es-ES	Agotado	out_of_stock	El artículo está agotado.

The source file lists multilingual code values against their respective target languages. Each code value is entered in each target language.

Import related code values into a hierarchy

You can import related code values into a hierarchy in Reference 360.

For example, a location hierarchy is defined with two parent-child relationships between the Country to State code list and State to State Code code lists. If you want to import 1000 related code values into a hierarchy, you can import the source file that contains the required relationship details with the parent and child code values.

The source file that you import must contain the code values of the parent and child code lists. Reference 360 generates the source primary key of relationships with the related parent and child code values. To import the code values, map the source primary key fields from your source file to the target fields.

Note: You can't update existing relationships in a hierarchy because each relationship contains source primary keys with unique codes of the parent and child code values.

The following table lists a sample format of the source primary keys of the relationships along with the related parent and child code values:

TopLevel_spkey	CountrytoState_parent	CountrytoState_child	StatetoStatecode_parent	StatetoStateCode_child
Germany	Germany	Berlin	Berlin	10243
India	India	Delhi	Delhi	110010
Australia	Australia	Sydney	Sydney	2001

The import process verifies the mapping and ensures that the source file contains the same relationships and code values that exist in the hierarchy model. If the source file contains different relationships and code values, the import process fails.

Rules and guidelines to import data

Consider the following rules and guidelines when you import data:

- You can import CSV files of up to 20 MB at a time.
- When you import code values into a code list, ensure that the CSV file doesn't contain leading or trailing spaces in the Code field values. Reference 360 uses the Code field values as source primary keys for code values. If the Code field values contain leading or trailing spaces, Reference 360 ignores these spaces in the source primary keys. When you update the code values, Reference 360 fails to create relationships because of mismatches between the Code field values and the source primary keys.
- When you import code values into a code list, ensure that the CSV file doesn't contain special characters, such as / \ % " . ; and any non-printable ASCII characters in the Code field values. Reference 360 uses the Code field values as source primary keys for code values and might drop these characters in the source primary keys, which causes a mismatch between the Code field values and the source primary keys.
- When you import code values into a hierarchical code list, don't add a child code value to multiple parent code values in hierarchical code lists.
- Don't import code values with cyclic relationships into a hierarchical code list. If you import code values that contain cyclic relationships, you can't export the code values from the code list.
- If you want to update an attribute of a code value, enter values for all the attributes of the code value in the input file. The import process doesn't retain the values of attributes that don't contain values in the input file.
- The source file that you use for import must contain relationships and code values that exist in the hierarchy model.
- When you import a CSV file to update the existing relationship between the code values in a hierarchy, the import fails.
  Each code value contains a unique code that's considered as the source primary key for the relationship between the parent and child code values.
- When you import decimal values, consider the following guidelines:
  - The values must conform to the default precision and the defined scale.
    The default precision is 34. The scale value ranges from 1 to 34.
  - The total number of digits before the decimal point must be less than or equal to the difference between the precision and the scale.

  The following table describes the sample success and failure situations when you import decimal values:

Precision	Scale	Decimal Value	Result
34	4	3456.827	Imports the decimal value because the value is within the default precision and the defined scale.
34	32	646.789	Rejects the value because the number of digits before the decimal point isn't less than or equal to the difference between the precision and the scale.
34	5	15.345678	Rejects the value because the number of digits after the decimal point isn't within the defined scale.
34	34	1.345	Rejects the value because the difference between the precision and the scale is 0. If you import 0.789, the import process still rejects the value.

- When a user with a custom user role imports code lists and crosswalks using the user interface or the REST APIs, the records must go through workflow approval. The user can import a CSV file into a draft asset through the REST APIs and send the draft for workflow approval.
- A user with only a custom user role can import a draft version of the hierarchy through the user interface. The user can't import hierarchies through the REST APIs.

Import process

Your user role must have permission to import data.

To import data, perform the following tasks:

1. Upload a CSV file to import.
   When you configure a rule association for the code field in a new code list, you can leave the Code column blank in the CSV file. Reference 360 generates values for the code field based on the rule association. If you provide values in the Code column of the CSV file, Reference 360 overwrites the imported values with the values the rule generates.
2. Verify the automatically mapped fields. Manually map the fields that have incorrect mapping to ensure that the columns of the source file match the target fields.
   When you import hierarchies, map the source columns with the required target fields. The automatically mapped fields display confidence indicators to indicate the accuracy of the mappings. If fields have low confidence scores or incorrect mappings, manually map the source columns with the required target fields.
3. Preview and verify the field mapping and import the file.

Step 1. Upload a file

To import data into Reference 360, upload a CSV file with code values or value mappings.

Review your CSV file and note from which line data in the CSV file starts. In the **Import Data Starting From Line** field, enter the line number from where you want to import data.

1. From the **Explore** page, search and open the code list, crosswalk, or hierarchy.
2. Click the **Import Values** icon.

   The **Upload File** page of the **Import Values** wizard appears.
3. To select a file to upload, drag a CSV source file, or click **Browse** and select the source file.

   The first 10 rows of records appear in the preview section.
4. In the **File Import Settings** section, verify the system suggested settings based on your data, and update any incorrect settings.

The following table describes the available import settings:

Field	Description
Delimiter	A character that represents the break between data values in the import file. Select a predefined delimiter or select **Other** to define a custom delimiter.
Text Qualifier	Symbols used in the file to enclose a string.
Encoding Type	Unicode encoding type.
Import Data Starting From Line	Line number in the file from where you want to import data. Use this field to exclude headers. Default is 2.
Contains column headers	Indicates whether the source file includes column headers.
Column Header Row	Header row number in the file.
Regional settings	Indicates whether you want to configure date and time formats.
Date Pattern	Format of the date fields in the file.
Date Time Pattern	Format of the date and time fields in the file. The supported time zone is in UTC (Coordinated Universal Time).
Decimal Separator	A character used as a decimal separator. Default is period (.).
Thousand Separator	A character used as a thousand separator. Default is comma (,).

5. Click **Next**.

   The **Map Fields** page appears.

Step 2. Map fields

After you select the file to import, the import process automatically maps the possible columns to the suitable target fields. You can either use the system suggested mapping or modify the mapping as required.

When you map fields to import data, ensure that you don't map the following fields because these fields are ignored during the load step of the reference data import job:

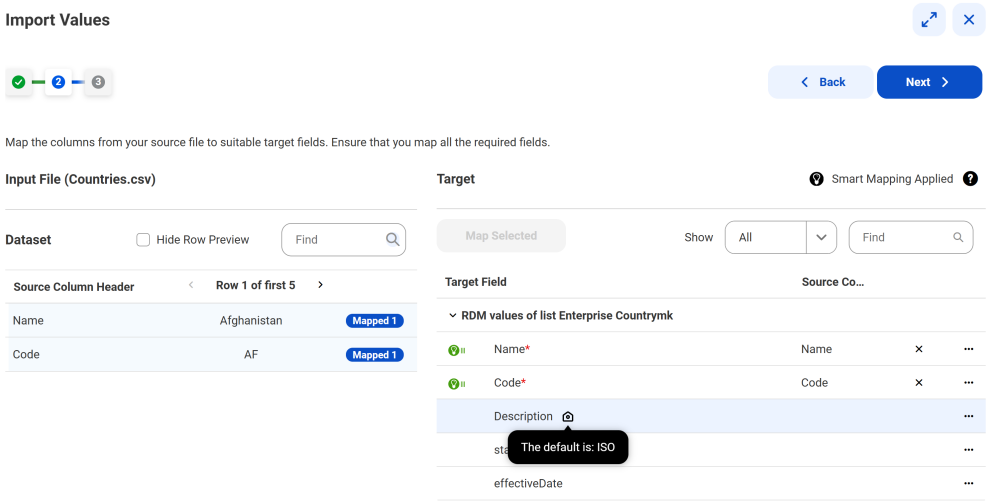
- createdBy
- updatedBy
- creationDate
- lastUpdateDate

1. On the **Map Fields** page of the **Import Values** wizard, verify the system suggested mapping of fields.
2. To manually map any field, perform one of the following tasks:
   - Drag a source column to a target field.
   - Select a source column and a corresponding target field, and click **Map Selected**.

   Note: When you import related code values into a hierarchy, manually map the source primary key columns from the source file to the target fields.

The import process maps the field accordingly.

3. To view the default value configured for a target field, hover over its information icon and decide whether to map a source value or let the field auto-populate with the default.



4. Verify the mapping of all fields and ensure that all the required fields are mapped.
5. Click **Next**.

   The **Preview and Import File** page appears.

Step 3. Preview and import data

After you map the fields, use the preview section to review and verify the mapping, and then import the data.

1. On the **Preview and Import File** page of the **Import Values** wizard, verify the mapping.

   If the preview does not return desired results, you can go back to the **Map Fields** page to map the fields again.
2. To import the file, click **Import**.

   The import process starts. You get a notification after the import process is complete.

You can view the status of the import on the **My Jobs** page. If the import job fails, click the **Download** icon in the **Status** column of the failed job to view the error report. For more information, see "My jobs page" on page 185.

Field mapping

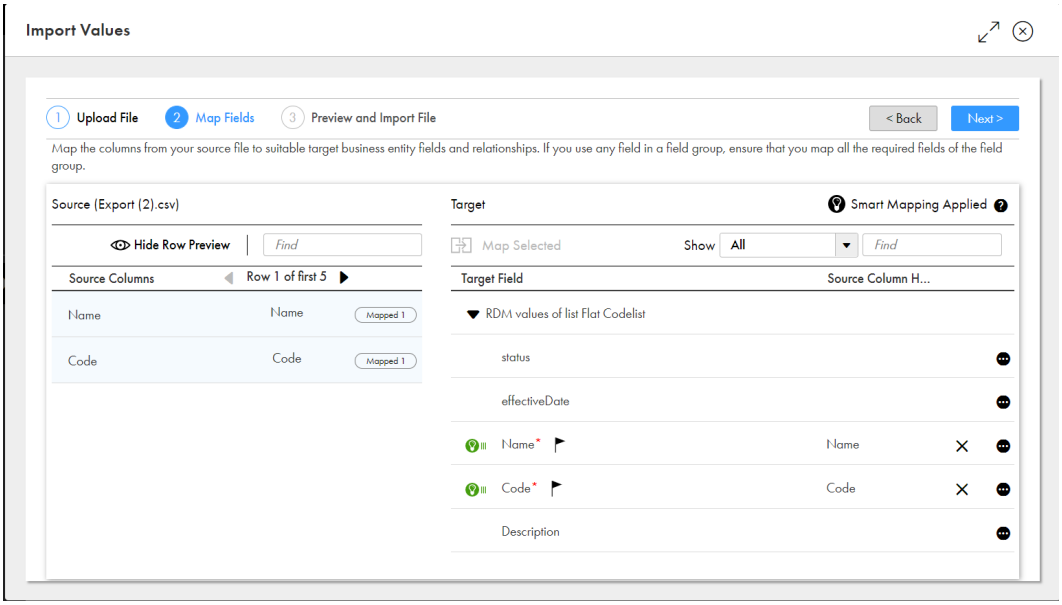
After you upload the source file, the import process automatically maps the columns of your source file to suitable target fields. For each mapped field, a confidence indicator indicates the confidence level of the field mapping.

If you don't agree with the system suggested field mapping, manually map the source columns with the appropriate target fields. When you map manually, a **User Mapped** icon appears against the mapped field.

The following table describes the confidence indicators:

Confidence Indicator	Description
	Indicates a high level of confidence in the field mapping.
	Indicates a medium level of confidence in the field mapping. If you don't agree with the mapping, delete the mapping and modify the field mapping as required.
	Indicates a low level of confidence in the field mapping. If you don't agree with the mapping, delete the mapping and modify the field mapping as required.

The following image shows some mapped fields with confidence indicators:



CHAPTER 10

Manage hierarchies

Create hierarchies to show how code values are related to other code values. First, define the hierarchy model, and then add code values to the hierarchy to define relationships between code values.

For example, you might create a location hierarchy. First, you define the hierarchy model. You define the Region code list as the top-level code list. Then you create a parent-child relationship from the Region code list to the Enterprise Country Codes code list. Based on this hierarchy model, in the hierarchy, you create a hierarchy relationship from the North America code value to the United States code value. You create a hierarchy relationship from the North America code value to the Canada code value.

You can extend a hierarchy model that contains data by adding nodes and relationships to the nodes of the hierarchy model. You can add multiple child nodes and a recursive relationship to any node. After you extend and save the hierarchy model, you cannot delete the nodes or relationships.

You can use the details panel to view the details of code values in hierarchies. The panel displays the summary, history, and relationships for a code value.

You can view the history of code values in hierarchies. Use the historical data to track the detailed changes made to a hierarchy.

You can view the code values that existed in a hierarchy at a point in time in the past. For example, you can view code values that existed in the Locations hierarchy on April 2023.

Note: Reference 360 displays the code values that are available in the database at a point in time in the past.

You can also export the code values in a hierarchy to a CSV file.

If you no longer need a hierarchy, you can delete the hierarchy.

Note: You must be assigned the Planner role to work with hierarchies. For more information, see "Users, groups, and roles" on page 50.

For more information, see "Hierarchies" on page 25.

Creating hierarchy models

Create a hierarchy model to define hierarchy relationships between code lists. Later, you can add code values from the code lists to define hierarchy relationships.

To create a hierarchy model, perform the following actions:

1. Create a hierarchy asset.
2. Define the hierarchy model.
3. Assign stakeholders.

Step 1. Create a hierarchy

Create a hierarchy and configure the general properties and status. You can specify internal ID and alias for a hierarchy.

Note: You must be assigned the Planner role to work with hierarchies. For more information, see "Users, groups, and roles" on page 50.

1. Click **New** and select **Hierarchy**.

   The **New Hierarchy** page appears and displays the **Summary** tab.

2. Configure the general properties fields.

   Note: The domain, confidentiality, and priority are inherited by code lists in the reference data set.

   The following table lists the properties that you can configure for a hierarchy:

General Property Field	Description
Name	Required. Display name of a hierarchy.
Internal ID	Required. Unique identifier of a hierarchy, limited to a maximum of 45 characters. You can use alphanumeric characters and underscores. After you save an asset, you can't edit the internal ID. **Note:** You can't specify internal IDs for existing assets.
Alias	Alternative identifier for a hierarchy, limited to a maximum of 70 alphanumeric characters. After you save an asset, you can edit the alias.
Version	The version information of a hierarchy.
Description	Short description of a hierarchy.
Application	The source application of a hierarchy.
Domain	An area or grouping to describe a hierarchy.
Confidentiality	The confidentiality level of a hierarchy.
Priority	The priority of a hierarchy asset. The priority levels are critical, high, medium, and low.
Status	The state of a hierarchy in the life cycle.
Effective Date	The date from when the status is effective. **Note:** You can't type a date. To select or modify a date, use the **Calendar** icon. To clear the selection, click the **Remove** icon.
Approved by	The user who approved a hierarchy.
Approved on	The date of approval. **Note:** You can't type a date. To select or modify a date, use the **Calendar** icon. To clear the selection, click the **Remove** icon.

Note: If you do not see options in some lists, configure your system reference data values. For more information, see "Adding values to system reference data" on page 100.

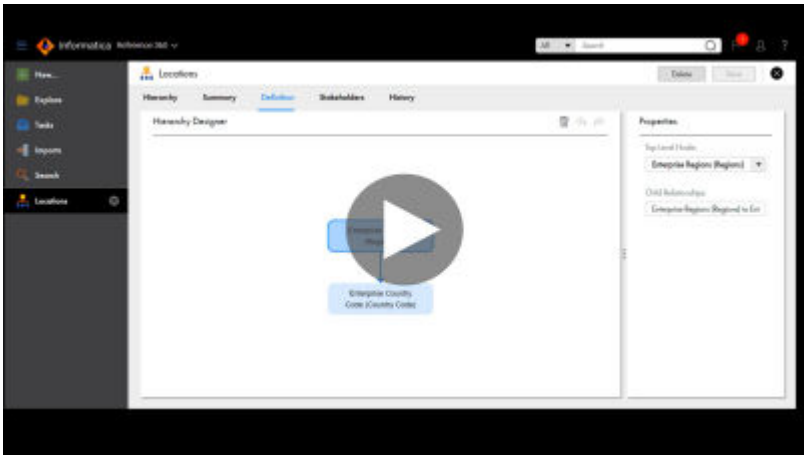
3. Click **Save**.

   The **Definition** tab appears and displays an undefined node in the Hierarchy Designer.

Step 2. Define the hierarchy model

Define the top level node and then add child nodes to the hierarchy model. You can add flat and hierarchical code lists as nodes to different levels of the hierarchy model. For example, you might define the Enterprise Regions code list as the top level node. Then you create a relationship from the Enterprise Regions code list to the Enterprise Country Codes code list.

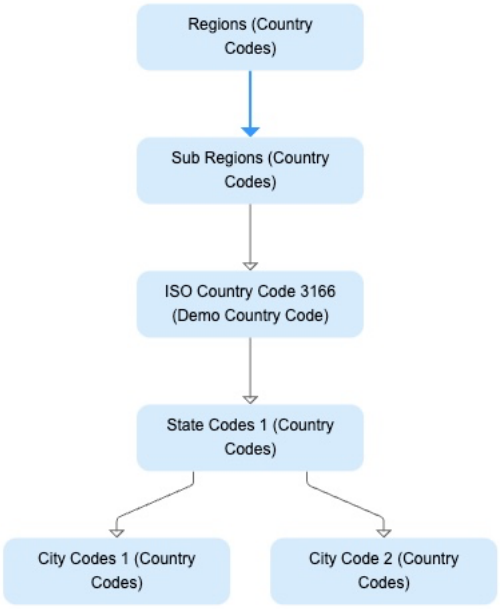
The following video shows you how to define the hierarchy model:



1. In the **Properties** panel, select a code list as the top level node for the hierarchy model.

   The undefined node name is replaced with the code list you selected.
2. In the Hierarchy Designer, hover over the node and click **Add Child**.

   A child node appears in the Hierarchy Designer.
3. In the **Node** field, select a code list to create a relationship.
   The following image shows a relationship from the top level node to the child node:



The undefined node name is replaced with the code list you selected.

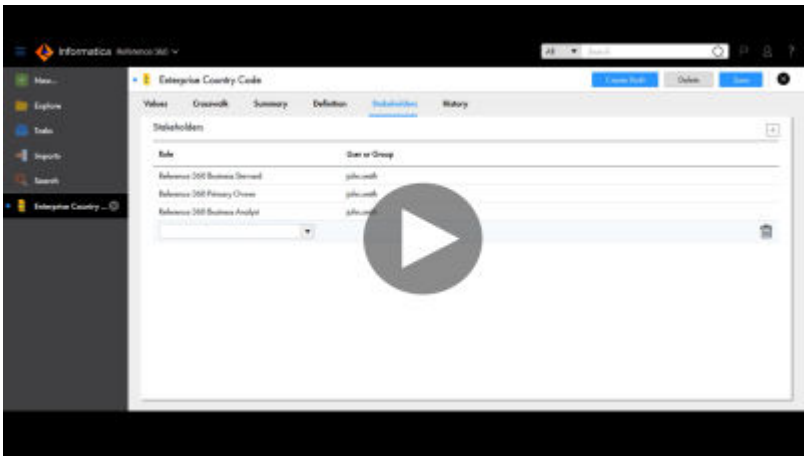
4. Optionally, add additional child nodes.
5. Click **Save**.

Step 3. Assign stakeholders

Assign stakeholder roles to users who play a role in using, creating, or maintaining the asset. Stakeholder roles determine a user's privileges for the asset.

When a user manages an asset as a stakeholder, they have the combined privileges provided by their stakeholder role and their Reference 360 role. For more information about stakeholder roles, see "Stakeholder roles" on page 53. For more information about assigning roles, see "Guidelines for assigning roles" on page 55.

The following video shows you how to assign stakeholders:



1. Click **Stakeholders**.

   The **Stakeholder** tab opens.
2. Click **Add**.

   A list appears in an empty row.
3. In the **Role** list, select a stakeholder role.

   The **New Stakeholder** dialog box appears.
4. Select a user name and click **Add**.

   The user name of the stakeholder appears in the row.
5. Click **Save**.

When you create a reference data set and assign a stakeholder role to a user or user group, Reference 360 automatically adds an additional Reference 360 Business Analyst stakeholder role to the users and groups.

Creating hierarchies

You can add, move, and remove code values in the hierarchy to define hierarchy relationships between code values.

Before you create a hierarchy, define the hierarchy model. The hierarchy relationships that you can create and the code values that you can use depends on the hierarchy model.

1. In the **Explore** panel, select a hierarchy and click **Open**.

   The hierarchy opens in a new tab.
2. Click **Lock**.

   A draft version of the hierarchy is created, and the hierarchy is locked for other users.
3. Click **Hierarchy**.

   The hierarchy page appears.

4. To add code values as top-level nodes, perform the following actions:
   a. Click **Add Hierarchy Value at the Top Level**.

      The **Add Code Value** dialog box appears.
   b. To view all available code values, in the **Search** field, enter *.
   c. Select the code values that you want to add, and click **Add**.
5. To add code values as child nodes, perform the following actions:
   a. Hover over the top-level node for which you want to add a child node, and click **Add**.

      The **Add Code Value** dialog box appears.
   b. To view all available code values, in the **Search** field, enter *.
   c. Select the code values that you want to add, and click **Add**.
6. To move a node, select a code value and drag the code value to a new location.
7. To remove a node, select a code value and click **Remove**.
8. To send your changes for approval, perform the following actions:
   a. Click **Submit** after you finish adding code values to the hierarchy.

      The **Send for Approval** window appears.
   b. Set the priority, due date, and optionally add a comment.
   c. In the **Compare Versions** section, review your proposed changes.
   d. Click **Send for Approval**.

   You receive an email notification confirming that you successfully sent the approval request. Users assigned the Business Steward stakeholder role for the asset or the Reference 360 Business Steward role are notified of the task. The task appears in the **Workflow Inbox** tab.

   Note: Users assigned the Reference 360 Planner role and the Planner stakeholder role for a hierarchy can send their proposed changes for approval or directly publish their changes without approval.
9. To cancel the approval request, perform the following actions:
   a. Click **Cancel Approval Request**.

      A confirmation dialog box appears.
   b. Click **OK**.

   The hierarchy is reset to the draft state. To unlock the hierarchy, you must discard the draft.

The following image shows an example hierarchy:



Note: When you sort a hierarchy that contains multiple child nodes in a parent node, the code values are sorted based on the category of child nodes. For example, if a parent node has two categories of child nodes such as CRM country codes and ISO country codes, sorting is applied on each child node category.

Viewing the history of a hierarchy

You can view the changes made to a hierarchy in a chronological order. Use the historical data to track the detailed changes made to the hierarchy at a point in time in the past.

1. In the **Explore** panel, select a hierarchy and click **Open**.

   The hierarchy opens in a new tab.

2. To view the change history of the hierarchy, click the **History** tab.

   The change history of the hierarchy appears.
   The following table lists the different user interface elements that you can find on the **History** tab:

User Interface Element	Description
Show/Hide Filter icon	Displays the **Filter** menu. Click **Add Filter** and select the following values based on which you want to filter the history feed: - Date Range. Allows you to specify a time period. You can view the history feed for the specified time period. Use the following predefined time periods: - Today. Displays the changes occurred on the current date. - Last 7 days. Displays the changes occurred in the last seven days. - Last 30 days. Displays the changes occurred in the last 30 days. - Custom. Displays the changes occurred during the selected date range. - Event. Displays the changes based on the change events. You can filter based on a single or multiple event types. - Business Entity Fields. Filters the history feed based on the general properties of the code list. You can filter based on a single property or all properties.
Sort icon	Displays the following values based on which you can sort the history feed: - New to Old. Sorts the changes in reverse chronological order. - Old to New. Sorts the changes in chronological order.
View icon	Groups the history feed based on the selected frequency. Use one of the following frequencies: - Daily. Groups the history feed by day. - Monthly. Groups the history feed by month. - Yearly. Groups the history feed by year.
Expand icon	Displays the event details in full screen.

3. To view all fields in the history feed, clear **Show the updated fields**.

   By default, the **Show the updated fields** check box is selected, and the updated fields appear in the history feed.

Viewing hierarchies at a point in time

You can view code values that existed in a a hierarchy at a point in time in the past.

1. In the **Explore** panel, select a hierarchy and click **Open**.

2. To view the code values at a point in time, select the date and time using the calendar or type the date and time in the **Point in Time** field.

   You can view the code values that existed at a point in time in the past.

   Note: When you view the hierarchy at a point in time in the past, you can't add, sort, filter, import, or export the code values.
3. To revert to the current code values, click **Current**.

Exporting hierarchies

Export code values in a hierarchy to a CSV file to analyze the hierarchies. You can choose how to format the data, and the attributes and the code value levels to include in the exported file.

1. In the **Explore** panel, select a hierarchy and click **Open**.

   The hierarchy displays the **Definition** tab.
2. Click **Hierarchy**.

   The **Hierarchy** page displays the code values in the hierarchy.
3. Optionally, filter code values or search for specific code values in the hierarchy.

   After you filter the code values in the hierarchy, Reference 360 displays the complete hierarchy of the matched code values.

   Note: Reference 360 displays the attributes that you selected in the column settings as filters.
4. Click **Export**.

   The **Export** dialog box displays a warning message indicating that you check the formatting of the exported file.
5. On the **General** tab, enter a file name.
6. Optionally, configure how you want the data to be formatted in the file that you export.

Field	Description
Delimiter	Character used to separate values.
Date Format	Format used for dates.
Decimal Mark	Decimal separator used for numbers.
Thousands Separator	Grouping separator used for numbers.

7. Optionally, on the **Columns** tab, select the code value levels and attributes that you want to export and the column order of the data.
8. Click **Export**, and then save or open the exported file.

Deleting hierarchies

Users with the Reference 360 Primary Owner role can delete hierarchies that are no longer needed.

You can't delete hierarchies that are locked.

1. Open the hierarchy that you want to delete.
2. Click **Delete**.
3. Click **OK**.

After you delete a hierarchy, you can't reuse its internal ID. If you try to create a hierarchy with the same internal ID, either through asset migration or manually reuse the internal ID of a previously deleted hierarchy in the API request body during model import or export, the asset migration and import and export models fail.

CHAPTER 11

Manage attributes

An attribute is a component of a code value. When you create reference data sets and code lists, you define the attributes of the code values. By default, all code values consist of the Name and Code attributes.

You can configure custom attributes. For example, in an Organizational Chart reference data set, you might want code values to include a Title attribute and Location attribute, in addition to the Name and Code attributes.

You can set default values for attributes in your code list. When you add a code value, the attribute displays the default value.

When you configure reference data attributes, if the configured reference data and code list of the attribute depend on any of the code list of existing reference data attributes, you can set dependency between the reference data attributes. If you don't want to set any dependency, you can select **None** in the **Dependent On** field.

If you've configured reference data attributes in your code list, by default, Reference 360 displays **None** in the **Dependent On** field. Based on your requirement, you can set dependency for the existing reference data attributes.

You can create rule associations for attributes to ensure that code values meet your business standards. For example, in the Country Codes code list, you might want the Code attribute for code values to require a minimum of three characters.

When you add or modify a rule association for attributes set with default values and save the rule association, Reference 360 clears the default value. After you save the rule association, you can set a default value based on the configured rule.

If you no longer need an attribute, you can delete the attribute.

For more information about attributes, see "Attributes" on page 26. For more information about rules, see "Data quality rule associations" on page 39.

Deleting attributes

Users with the Reference 360 Primary Owner role or the Primary Owner stakeholder role can delete attributes that are no longer needed. When you delete an attribute, the attribute is no longer a component of code values in the asset.

There are some attributes that you cannot delete. For more information, see "Considerations for deleting attributes" on page 169.

1. Open the reference data set or code list that contains attributes that you want to delete.

2. Click **Definition**.

   The **Definition** tab opens.
3. Hover over the attribute that you want to delete and then click **Delete**.
4. Click **OK**.
5. Click **Save**.

   Note: If the asset fails to save, the reasons why the save failed appear in a dialog box.

Considerations for deleting attributes

You can delete attributes of code values that you no longer need as components of code values. When you delete attributes, the attributes are no longer components of code values. You must be a Primary Owner to delete attributes.

You can delete attributes in reference data sets or empty code lists. You can also delete attributes that are required.

You cannot delete the following types of attributes:

- Default attributes such as Name and Code
- Attributes that are used as display attributes for the asset
- Attributes that are used as display attributes in dependent reference data sets or dependent code lists
- Attributes that are used as Reference Data attributes
- Attributes in a locked code list

Basic rule associations

You can use a basic rule association to validate or transform the attribute values based on the selected predefined function. A validation function validates whether data matches the specified condition. For example, you can set a maximum length condition for a field value. A transformation function transforms data. For example, the uppercase function converts lowercase values to uppercase.

You can group the predefined functions into the following categories:

- Validation. Validates whether data matches the specified condition. When you use a validation function, you must configure a validation message or use the default validation message.
- Transformation. Transforms data.

Note: Attributes that are configured with transformation functions of basic rule associations appear in read-only mode. Reference 360 generates values for these attributes based on the basic rule associations, and you can't edit these values.

The following table lists the rules that you can configure for each attribute type:

Attribute Type	Rules and Descriptions
String	- **Maximum Length** - Validates whether the length of a string is less than or equal to the specified length. - **Minimum Length** - Validates whether the length of a string is greater than or equal to the specified length. - **Starts With** - Validates whether a string starts with the specified string. - **Ends With** - Validates whether a string ends with the specified string. - **No Spaces** - Validates whether a string does not contain spaces. - **Contains** - Validates whether a value contains the specified string in the exact order. - **Does Not Contain** - Validates whether a value does not contain the specified string in the exact order. - **Allowed Characters** - Validates whether a value contains the specified allowed characters. - **Not Allowed Characters** - Validates whether a value contains the specified characters that are not allowed. - **Regular Match** - Validates whether a string matches with the specified regular expression pattern. - **External Validation** - Validates the field data based on an external validation rule that is associated to a Cloud Application Integration process. The Cloud Application Integration process uses external APIs to validate the data. - **Concatenate** - Concatenates two or more strings. - **Concatenate With Spaces** - Concatenates two or more strings with spaces in between. - **Uppercase** - Converts lowercase string characters to uppercase. - **Lowercase** - Converts uppercase string characters to lowercase. - **Title Case** - Capitalizes the first character in each word of a string and converts all other characters to lowercase. - **Left Trim** - Removes spaces from the beginning of a string. - **Right Trim** - Removes spaces at the end of a string. - **Replace** - Replaces a character set in a string with another character set.
Decimal	- **Minimum Value** - Validates whether a numerical value is greater than or equal to the specified value. - **Maximum Value** - Validates whether a numerical value is less than or equal to the specified value. - **Maximum Decimal Scale** - Validates whether the number of decimal places is less than or equal to the specified value. - **Maximum Precision** - Validates whether the total number of digits in a decimal number is less than or equal to the specified value.
Integer	- **Minimum Value** - Validates whether a numerical value is greater than or equal to the specified value. - **Maximum Value** - Validates whether a numerical value is less than or equal to the specified value. - **Maximum Precision** - Validates whether the total number of digits in a decimal number is less than or equal to the specified value.
Date	- **Date Range** - Validates whether a date is in between two specified dates. - **Greater Than Date** - Validates whether a date is greater than the specified date. - **Less Than Date** - Validates whether a date is less than the specified date. - **No Future Date** - Validates whether a date is not a future date.

Guidelines for basic rule associations

Consider the following guidelines when you define basic rule associations:

- When you configure a basic rule association for a reference data field or a field that uses a reference data field, the rule displays only the code field of the referenced code value.

- When you configure a basic rule association for the Code field, you must consider the following guidelines:
  - Ensure that the rule output doesn't include leading or trailing spaces. For example, to concatenate two fields that include leading and trailing spaces, configure basic rule associations, such as left trim and right trim, to eliminate these spaces.
  - Date fields that you might use must be in the YYYY-MM-DD format.
- Basic rule associations with external validation functions validate the code values that you add or update through the user interface and REST API.

Add basic rule associations

You can add basic rule associations for code list attributes. When users add or edit code values in a code list that does not meet the business standards, they receive inline validation errors.

1. Open a code list.
2. Click **Definition**.

   The **Definition** tab opens.
3. In the **Attributes** section, select the attribute to add a rule.

   The **Attribute Details** panel displays the details of the attribute on the **Details** tab.
4. On the **Basic Rules** tab, click the **Add** icon.

   A form appears.
5. In the **General** section, enter a unique name for the rule association.
6. Optionally, enter a description.
7. To validate and transform inputs fields that are empty, enable **Run rule association even if all input fields are empty**.

   Note: When you add a code value, the fields configured with rules, such as `Starts With`, `Ends With`, `Contains`, `Date Range`, `Greater Than Date`, `Less Than Date`, and `No Future Date` display validation error messages if the input fields are empty.
8. On the **Condition** tab, perform the following tasks:
   a. Select a function.
   b. For the input field, select an option from the list.
   c. If you select **Select a Field**, select an attribute from the list.
   d. If you select **Enter a Value**, type a value for validation.
9. If you select a validation function, on the **Validation** tab, enter an error message or use the default validation message to display when the validation fails.
10. To verify the created rule, perform the following tasks:
    a. Click **Test**.

       The **Test** dialog box appears.
    b. Enter values in the **Input** column, and click **Run**.

       The validation result appears in the **Output** column.
    c. Click **Reset** to reset the values in the **Input** and **Output** columns.
    d. Click **Close**.
11. Click **Save**.

Changing the execution order of basic rule associations

When you have multiple basic rule associations for an attribute, you can set their order of execution to obtain clean and correct data.

1. On the **Basic Rules** tab, click **Change Order**.

   The **Change Order** dialog box appears.

2. To change the execution order of rule associations, drag a rule association or select a rule association and click **Move Up** or **Move Down**.
3. Click **OK**.

   The list of rule associations with the updated order of execution appears.

Advanced rule associations

An advanced rule association is an association between attribute values and a Cloud Data Quality rule specification. You can use the predefined rule specifications or create custom rule specifications in Cloud Data Quality. For more information about rule specifications in Cloud Data Quality, see *Rule specification assets* in Cloud Data Quality.

For example, you work for an organization as a Governance Lead. To retire the old general ledger accounts created before 2012, you configure advanced rule associations based on the account creation date. These rule associations update the status of accounts created before 2012 to Retired.

To learn how to create rule specifications in Cloud Data Quality and validate reference data using the rule specifications in Reference 360, see the following video:

https://www.youtube.com/watch?v=k-vGsF2KdcQ

Rules and guidelines for advanced rule associations

Consider the following rules and guidelines when you define advanced rule associations:

- When a rule specification is modified in Cloud Data Quality, select the rule specification again or reset the fields of the advanced rule association in Reference 360. Otherwise, the changes to the rule specification aren't effective when the rule is executed.
- If you set an attribute as input and output fields for the same advanced rule association, you can't add values for the fields that use the rule specification, on the **New Code Value** page. To add code values to a code list, you can use reference data import jobs.
- When you configure an advanced rule association for a reference data field or a field that uses a reference data field, the rule considers only the code field of the referenced code value.
- When you re-enable the advanced rule association for the **Code** attribute and update a field other than the input fields, you can't save the code value if the input fields were updated previously.
  To save the code value, perform one of the following actions:
  - Disable the advanced rule association.
  - Revert the changes to the input fields and re-enable the advanced rule association.
- By default, Data Quality uses the Date/Time format to execute rules. When you configure advanced rule associations to validate date fields in Reference 360, the rules consider the date based on the time zone.

For example, when you configure an advanced rule association to check whether the date is earlier than the current date. The rule considers the time zone and determines the date.

- To ensure the success of the comparison operation for the code value field in Reference 360, use integer or string values in the rule statement output of the advanced rule association. If you use a float value in the rule statement output, specify additional decimal values in the **Rule is Valid when Status Is** field.
- An advanced rule association for a transformation function must have rule statement output values for all the conditions in Cloud Data Quality to generate values in the output field.
- After you configure an advanced rule association to code list attributes, specify values for at least one of the input fields to run the rule and display validation error messages for code values.
- In Data Quality, you can create a rule specification to copy the input field value to the output field if the input field starts with `<value>` and set the action to `Do nothing` if the rule statement is not valid. When you use this rule specification in an advanced rule association, it returns an empty value for the output field if the input field value doesn't start with `<value>`. To use the input field value for the output field, edit the rule specification in Data Quality and change the action `Do nothing` to `Use the Input field`.
- When you configure an advanced rule association for a required field, ensure that the field value doesn't depend on the value of another field that is configured with an advanced rule association. Reference 360 doesn't guarantee the execution order of the rules.

Add advanced rule associations

Use Cloud Data Quality rule specifications to add advanced rule associations for code list attributes. You can associate a rule specification as a transformation rule or a validation rule.

1. Open a code list.
2. Click **Definition**.

   The **Definition** tab opens.
3. In the **Attributes** section, select the attribute to add a rule.

   The **Attribute Details** panel displays the details of the attribute on the **Details** tab.
4. On the **Advanced Rules** tab, click the **Add** icon.

   A form appears.
5. Select a rule specification.
   a. Click the asset picker.

      The **Select an Asset** page appears.
   b. Select a rule specification, and click **Select**.
6. Enter a unique name for the rule association.
7. Optionally, enter a description.
8. On the **Input and Output Fields** tab, perform the following tasks:
   a. In the **Input Fields** section, select a business entity field or enter an input value for each rule specification input field.
   b. In the **Output Fields** section, select a business entity field for each rule specification output field if you want to associate a transformation rule.

      If you associate a transformation rule by specifying a value in the **Output Fields** section, the **Validation** tab is disabled.

9. To associate a validation rule, specify values on the **Validation** tab without providing values in the **Output Fields** section.
    a. In the **Rule Status** section, select a rule specification field that contains the validation status.
    b. Enter a validation status that indicates successful validation.

       Note: Ensure that the validation status matches the rule specification output value for successful validation.
    c. In the **Error Status** section, configure the error message by using one of the following ways:
        - Select a rule specification field that contains an error message.
        - Enter a custom error message.
    d. Click **Save**.

Redeploying advanced rule associations

After you modify rule specifications in Cloud Data Quality, you must redeploy the existing advanced rule associations that use the updated rule specifications. If you don't redeploy the advanced rule associations in Reference 360, they might not work as expected.

1. In the **Explore** panel, select a code list and click **Open**.

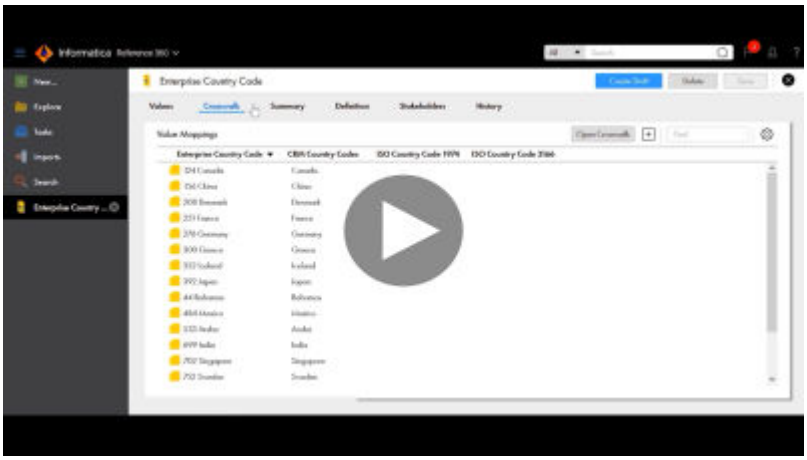
   The code list opens in a new tab.
2. On the **Definition** tab, select the attribute that's configured with an advanced rule association.
3. In the **Attribute Details** section, click **Advanced Rules**.
4. Click **Edit** for the rule association.
5. Click the **Select an Asset** icon.
6. On the **Select an Asset** dialog box, select the same rule association and click **Select**.
7. On the **Input and Output Fields** tab, remap input or output fields to the attribute, and save the rule association.
8. Save the code list.

Configuring display attributes

Configure the attributes that you want to represent code values in other assets. For example, you might want the Code and Name attributes to represent code values in the Country Codes code lists. You might have a

code value with the Name value as `Canada` and the Code value as `124`. When the code value appears in other assets, the code value appears as `124 Canada`.

The following video shows you how to configure display attributes:



1. Open a code list.
2. Click **Definition**.

   The **Definition** page appears.
3. To configure one attribute as the display attribute, from the **Display Attributes** list, select an attribute.
4. To configure multiple attributes as the display attributes, perform the following actions:
   a. From the Display Attributes list, select **Advanced...**.

      The **Advanced Display Attributes** dialog box appears.
   b. From the **Column** list, select an attribute.
   c. To add an additional attribute, click **Add**, and select an attribute.
   d. Repeat step 3 for additional attributes.
   e. Click **OK**.

   Tip: Add the attributes in the order you want the attribute values to appear. For example, you might want the value in the Code attribute to appear before the Name attribute. Select the Code attribute first and then add the Name attribute.

   The display attributes appear in the **Display Attributes** list.
   When you display a combination of multilingual attributes, such as a concatenation of the Name, Code, and Description, and if a multilingual attribute has no stored value, Reference 360 shows an empty string for each field that has no value in the preferred language.
5. Click **Save**.

CHAPTER 12

Manage workflows

You can configure workflows that are triggered when other users edit code lists, crosswalks, or hierarchies. You can also enable an approval workflow for code lists or crosswalks when you import code values or value mappings by using the import version 2 REST API. You can define the number of approval steps, approvers for each approval task, and whether comments are required for task actions.

Users assigned the following roles can configure workflows that are triggered when other users edit assets, review the tasks, and approve or reject the proposed changes:

- Reference 360 Primary Owner role and the Primary Owner stakeholder role for a code list.
- Reference 360 Business Steward role and the Business Steward stakeholder role for a crosswalk.
- Reference 360 Planner role and the Planner stakeholder role for a hierarchy.

The workflow configuration includes the approval tasks, approvers for each approval task, and whether comments are required.

The following table describes the approval workflows you can configure:

Approval Workflow	Description	Activity
One-step	One-step approval workflows contain one approval task. This workflow is the default approval workflow for all code lists or crosswalks.	An approver can approve, reject, or send the changes back to the originator. If the changes are approved, they become part of the active version of the code list or the crosswalk.
Multi-step	Multi-step approval workflows contain multiple approval tasks.	The first approver can approve or send the changes back to the originator. If the first approver approves the changes, the next approver can approve or send the changes back to the originator. The next approver can also approve the changes or send the changes back to the originator. After the task reaches the final approver, the final approver can approve, reject, or send the changes back to the originator. If the changes are approved, they become part of the active version of the code list or the crosswalk.

Note: To enable multi-step workflows, configure the approval implementation mode by using APIs. For more information, see "Update approval implementation mode" on page 209.

If approved, the changes become part of the active version of the code list, crosswalk, or hierarchy. You can view your latest changes and previous changes on the **History** tab of the code list, crosswalk, or hierarchy.

Related Topics:

- "Workflows" on page 33

Configuring workflows

You can configure an approval workflow for code lists, crosswalks, and hierarchies. You can also add approval steps, define approvers for each approval task, and configure whether comments are required for task actions.

Users assigned the following roles can configure an approval workflow:

- Reference 360 Primary Owner role and the Primary Owner stakeholder role for a code list.
- Reference 360 Business Steward role and the Business Steward stakeholder role for a crosswalk.
- Reference 360 Planner role and the Planner stakeholder role for a hierarchy.

By default, one-step approval workflows are configured for all code lists, crosswalks, and hierarchies. To create multi-step approval workflows, you add additional approval tasks to the workflow configuration of a code list, a crosswalk, or a hierarchy.

When you create an asset, Reference 360 adds default approvers to the approval task. These approvers are users with Business Steward role for code lists and crosswalks or Planner role for hierarchies.

1. In the **Explore** panel, select a code list, a crosswalk, or a hierarchy, and click **Open**.
2. Click the **Workflow** tab.

   The **Workflow** page appears.
3. To edit an existing approval task, perform any of the following actions:
   - To edit the approval task name, edit the name in the **Task Name** field.
   - To update the approvers, in the **Approvers** section, click **Edit**. Select or clear approvers and click **OK**.
   - To update whether users are required to leave a comment when they approve the approval task, in the **Task Actions** section, change your selection.
4. To add an approval task to the workflow, perform the following actions:
   a. Click **Add Approval Task**.

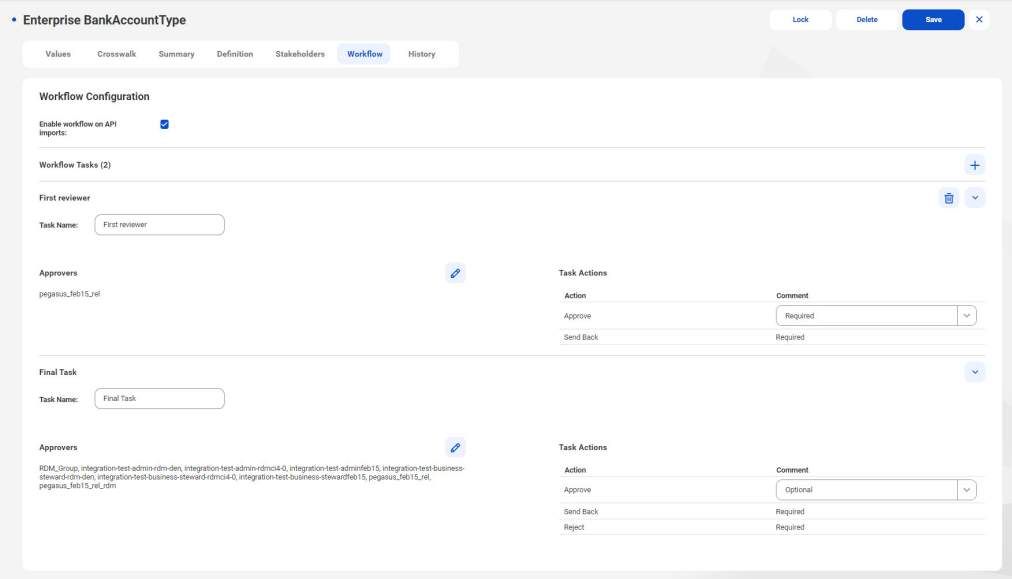
      The approval task appears at the top of the list.
   b. Enter an approval task name.
   c. In the **Approvers** section, click **Add**. Select approvers and click **OK**.
   d. Optional. If you don't want to require users to leave a comment when they approve the approval task, select **Optional**.
5. To add additional approval tasks to the workflow, repeat 4.

   Note: An approver can approve only one approval task within a workflow even if the approver is assigned multiple approval tasks.
6. To rearrange the order of approval tasks, click **Move Up** or **Move Down** for the appropriate approval task.

   The order that the approval tasks appear in the list determines the order that the approval tasks are linked together. The approval task at the top is the first approval task in the workflow.
7. To enable an approval workflow for the code list or crosswalk when you import code values or value mappings by using the import version 2 REST API, select **Enable workflow on API import**.

Reference 360 locks the code list or crosswalk when the approval workflow is in progress. Other users can't perform any action on the locked code list or crosswalk until the workflow is complete. When the approval workflow is in progress, you can discard the draft to unlock the code list or crosswalk.

8. Click **Save**.
   The following image shows a sample workflow configuration of a code list:



Reviewing tasks

You can review and approve or reject changes proposed by other users.

Users assigned the following roles are responsible for reviewing and approving or rejecting changes proposed by other users:

- Reference 360 Primary Owner role and the Primary Owner stakeholder role for a code list.
- Reference 360 Business Steward role and the Business Steward stakeholder role for a crosswalk.
- Reference 360 Planner role and the Planner stakeholder role for a hierarchy.

1. In the **Workflow Inbox** tab or from your notification inbox, select a task.
   The **Task Details** panel displays the details of the task.
2. Review comments in the task.
3. To work on a task, click **Claim**.
   After you claim a task, other users can't work on it.
4. To confirm whether the proposed changes are correct, review the **Compare Versions** section.

5. To resolve the task, select one of the following task actions:

Option	Description
Approve	If you agree with the changes, click **Approve**. The draft and proposed changes become part of the active version.
Reject	If you disagree with the changes and think that the asset does not need to be changed, click **Reject**. Add a comment to explain your decision. The draft and proposed changes are discarded.

6. To send back the task or return the task to the pool of unassigned tasks, select one of the following task actions:

Option	Description
Send Back	If you disagree with some of the changes or think that some changes are missing, click **Send Back**. Add a comment to explain your decision. The draft and proposed changes aren't discarded.
Release	To return the task to the pool of unassigned tasks, click **Release**.

7. Click **OK**.

You receive an email notification confirming your action. Other assignees receive an email notification notifying them that another user performed an action on the approval request. The requester receives a notification in their inbox notifying them that an approver performed an action on the approval request.

CHAPTER 13

Manage jobs

You require specific roles to create a job and define job schedules to run the jobs. Use Reference 360 for file import, reference data import, and reference data export jobs.

To define a job and access the job definitions on the My Jobs page, you require one of the following roles in addition to a Reference 360 role:

- Admin
- Designer
- MDM Designer

To run a job, you require the Job Executor role.

To define job schedules, you require one of the following roles in addition to a Reference 360 role:

- IICS Admin
- Service Consumer

For more information about user roles, see *User administration* in the *Administrator* help.

Defining reference data import jobs

You can define reference data import jobs to import code lists, crosswalks, and hierarchies into Reference 360. When you get started with Reference 360, you need to import your initial data to onboard it.

Reference data import process

The process available to import reference data is Extract, Validate, and Load Data.

1. Extract data from a source file and load it to a staging area.
2. Cleanse and transform the staged data.
3. Load the cleansed data to a target asset.

Prerequisites for importing reference data

Before you import reference data, ensure that you complete the following prerequisites:

1. In Administrator, create a source connection to read data from the source and a Business 360 connection to write to the Reference 360 assets. For more information about creating connections, see *Configuring a connection* in the Administrator help.

2. In Administrator, specify a Secure Agent that is up and running as the runtime environment to run a mapping task. For more information about Secure Agent, see *Runtime environments* in the Administrator help.
3. In Cloud Data Integration, create a mapping with the source connection as the source transformation and the Business 360 connection as the target transformation. Configure the target by selecting the code list from Reference 360 and define field mappings for the source and target fields. Ensure that you create an in-out parameter `jobInstanceId` in the Parameters panel. For more information about creating a mapping, see *Mapping configuration* in the Data Integration help.

   Note: The name of the in-out parameter is case-sensitive.
4. In Cloud Data Integration, create a mapping task by selecting the runtime environment and adding the mapping that you create. For more information about configuring a mapping task, see *Mapping task configuration* in the Data Integration help.
5. In Cloud Data Integration, create the required taskflows. Publish the taskflows to use in your reference data import job. For more information about creating taskflows, see *Taskflows* in the Data Integration help.

Rules and guidelines for reference data import jobs

Consider the following rules and guidelines when you import reference data, such as code lists, crosswalks, and hierarchies, into Reference 360:

- Before you run the reference data import job, ensure that you publish the taskflow in Cloud Data Integration.
- After you add a taskflow in Reference 360, don't rename the code lists, crosswalks, and hierarchies that the Cloud Data Integration mapping uses. If you rename the code lists, crosswalks, and hierarchies, the reference data import job fails.
- After you create a mapping in Cloud Data Integration, if you rename an asset that's associated with the mapping in Reference 360, manually update the mapping in Cloud Data Integration.
- Ensure that you don't rename a code list that's associated with a mapping in a source organization if the target organization has the same code list.
- Ensure that the Code fields don't contain leading or trailing spaces. Reference 360 uses the Code field values as source primary keys for code values. If the Code field values contain leading or trailing spaces, Reference 360 ignores these spaces in the source primary keys. When you update the code values, Reference 360 fails to create relationships because of mismatches between the Code field values and the source primary keys.
- Ensure that the name of an asset you use in a Cloud Data Integration mapping doesn't contain a slash `(/)`. If the asset name contains a slash `(/)`, you can't view the asset in Cloud Data Integration.
- Before you import a child code list, use a separate reference data import job to import its parent code list. If you use the same job to import both the code lists, the load step fails.

Import reference data

Create a reference data import job to import data to Reference 360 data store.

1. Click **New** > **Jobs** > **Reference Data Import** > **Create**.

   The **Reference Data Import** dialog box appears.

2. Specify the following job properties:

Property	Description
Display Name	Name of the job.
Internal ID	A unique job identifier, which is generated based on the display name that you enter. You cannot change the internal ID after you create the job.
Description	Optional. A brief description of the job.
Location	Project or folder within which you want to save the job.

3. Click **OK**.

   The reference data import job page displays the process, description, and source system.

4. To add a taskflow, perform the following tasks:

   a. Click **Add Taskflow**.

      The **Select an Asset** page appears.

   b. Select a taskflow, and click **Select**.

      The taskflow appears in the **Taskflows** section and the associated assets display in the **Assets** section.

5. Click **Save**.

6. To execute the job, click **Run**.

   You can monitor the status of job in the **My Jobs** page.

Defining reference data export jobs

You can define reference data export jobs to export code lists, crosswalks, and hierarchies from Reference 360 data store to an external data source.

Prerequisites for exporting reference data

Before you export data, ensure that you complete the following prerequisites:

1. In Administrator, create a Business 360 connection to read data from Reference 360 and a target connection to write the data to an external data source. For more information about creating connections, see *Configuring a connection* in the Administrator help.

   Note: You cannot use same connections as source and target for reference data export jobs.

2. In Administrator, specify a Secure Agent that is up and running as the runtime environment to run a mapping task. For more information about Secure Agent, see *Runtime environments* in the Administrator help.

3. In Cloud Data Integration, create a mapping with the Business 360 connection as the source transformation and the target connection as the target transformation. Select code lists, crosswalks, or hierarchies from Reference 360 to specify the source object and select **Map all descendants** for the root element to define field mappings. Select **root** as the output group. Ensure that you create an in-out

parameter `jobInstanceId` in the Parameters panel. For more information about creating a mapping, see *Mapping configuration* in the Data Integration help.

Note: The name of the in-out parameter is case-sensitive.

4. In Cloud Data Integration, select the runtime environment to create a mapping task. Add the mapping that you create to the mapping task. For more information about configuring a mapping task, see *Mapping task configuration* in the Data Integration help.
5. In Cloud Data Integration, create the required taskflow. Publish the taskflow to use in the reference data export job. For more information about creating taskflows, see *Taskflows* in the Data Integration help.

Rules and guidelines for reference data export jobs

Consider the following rules and guidelines when you export reference data, such as code lists, crosswalks, and hierarchies, into Reference 360:

- Before you run the reference data export job, ensure that you publish the taskflow in Cloud Data Integration.
- After you create a mapping in Cloud Data Integration, if you rename an asset that's associated with the mapping in Reference 360, manually update the mapping in Cloud Data Integration.
- Ensure that you don't rename a code list that's associated with a mapping in a source organization if the target organization has the same code list.
- Ensure that the name of an asset you use in a Cloud Data Integration mapping doesn't contain a slash `(/)`. If the asset name contains a slash `(/)`, you can't view the asset in Cloud Data Integration.

Export types

You can export code lists, crosswalks, and hierarchies to an external data source. You can choose to export all data or only incremental data after you export all data for the first time.

You can use any one of the following export types:

Standard export

Exports all the records in the first run of a job based on their creation date, and then incrementally exports the records added or updated after the first run based on their last updated date.

For example, you add 30 code values to a code list. During the first run of the job, the job exports 30 code values. After the initial run of the job, you update 15 code values to the code list. During the second run of the job, the job exports the last updated 15 code values.

Custom export

Exports all the data added or modified after the specified date in the first run of the job, and then exports the data that are added or updated after the first run incrementally.

For example, you add 20 code values to a code list after January 18, 2022, 5:00 p.m. During the first run, if you specify January 18, 2022, the job exports the 20 code values added after the specified date. After the initial run of the job, you update 10 code values. During the second run of the job, the job exports the last updated 10 code values.

Export all

Exports all the records in each run of the job based on their creation date.

Export reference data

Before you create a reference data export job, ensure that a taskflow is created and published in Cloud Data Integration. You can create the reference data export job to export data from Reference 360 data store to an external data source.

When you use Cloud Data Integration to export data, you can export the following additional attributes of a code list:

- Last updated date
- Last updated by user
- Created date
- Created by user
- Last updated date of parent relationship
- Last updated by user of parent relationship

Note: You can export the last updated date of parent relationship and last updated user of parent relationship attributes only for hierarchical code lists.

1. Click **New** > **Jobs** > **Reference Data Export** > **Create**.

   The **Reference Data Export** dialog box appears.
2. Specify the following job properties:

Property	Description
Display Name	Name of the job.
Internal ID	A unique job identifier, which is generated based on the display name that you enter. You cannot change the internal ID after you create the job.
Description	Optional. A brief description of the job.
Location	Project or folder within which you want to save the job.

3. Click **OK**.

   The reference data export job page displays the process, description, and export type.
4. Select one of the export types.

   For more information about export types, see "Export types" on page 183.
5. To add a taskflow, perform the following tasks:
   a. Click **Add Taskflow**.

      The **Select an Asset** page appears.
   b. Select a taskflow, and click **Select**.

      The taskflow appears in the **Taskflows** section and the associated assets display in the **Assets** section.

   Note: Ensure that the taskflow is published before you run the reference data export job.
6. Click **Save**.
7. To run the job, click **Run**.

   You can monitor the status of job on the **My Jobs** page.

Monitoring jobs

You can monitor jobs on the **My Jobs** page. This page lists all jobs that are started in Reference 360. You can schedule jobs to ensure that the job executes at a specified time.

To open the **My Jobs** page, click **My Jobs** in the left navigation bar.

My jobs page

The **Imports** tab on the navigation bar is enhanced and renamed to **My Jobs**. You can monitor the jobs instances and job schedules on the **My Jobs** page.

The **My Jobs** page contains the following tabs:

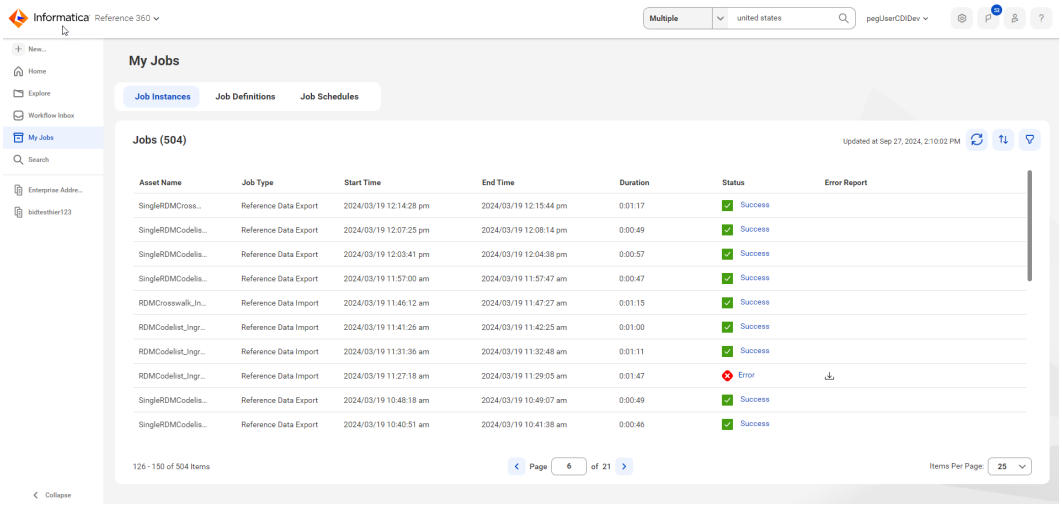
- Job Instances. Lists the job instances that are currently running, failed, stopping, stopped, and successfully completed.
- Job Definitions. Lists all the reference data import and export jobs. You can modify the properties of the jobs and rerun the reference data import and export jobs.
- Job Schedules. Lists the job schedules.

To filter the jobs that appear on the **My Jobs** page, click the **Filter** icon. You can use filters to find specific jobs. To specify a filter, click **Add Field**, and select the property such as job type and status, to filter results.

When you search for a job on the **My Jobs** page, the search results display jobs based on instance ID, started by, job name, and status fields. The instance ID and started by fields do not appear on the **Job Instances** tab of the **My Jobs** page.

You can search for reference data import jobs and reference data export jobs with asset names. You can search for a job schedule only using schedule names.

The following image shows the list of job instances on the **My Jobs** page:



By default, the following properties display for each job instance:

Property	Description
Asset Name	Name of the asset. - For file import jobs and Reference 360 draft jobs, displays the asset name to which data is imported. - For reference data import jobs and reference data export jobs, displays the job name. **Note:** Clicking the asset name of a file import job opens the asset.
Job Type	The type of job that was executed.
Start Time	Date and time that the job started.
End Time	Date and time that the job completed or stopped.
Duration	Time taken for the job to run.
Status	Status of the job instance.

To view the job details, click the status of the job. To download error reports for the failed jobs, click the **Download** icon in the **Status** column of the failed jobs. You can view error messages and a summary of rejected and invalid records in the error reports.

Viewing specific job details

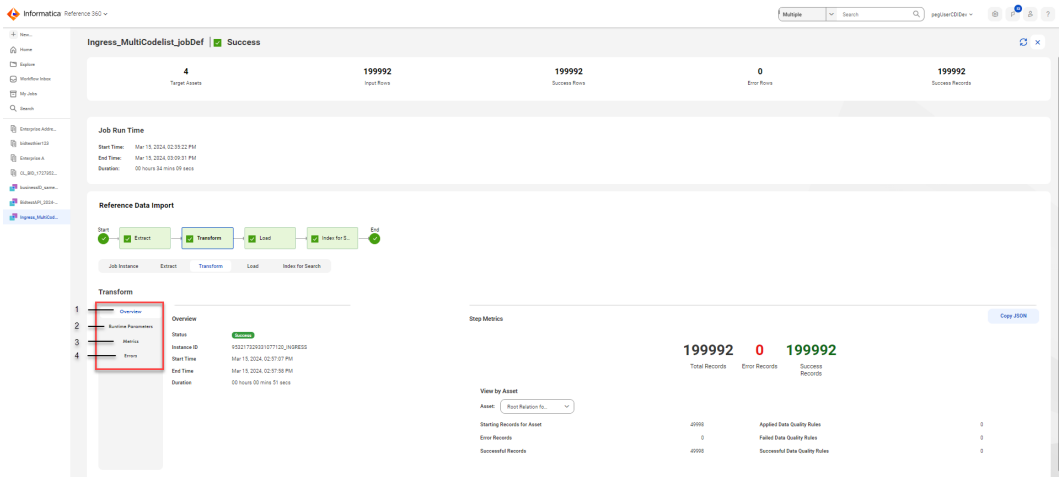
On the **My Jobs** page, click the status of any job to view detailed information about the job. If errors occur during a job execution, you can view the error details in the process steps of the job.

You can view key metrics about the job, such as total records processed, records processed successfully, and failed records. The metrics vary for each job.

You can view the overall data flow process for each job instance. Select a specific process step to view the step level details. The details displayed for the job vary based on the job type.

Note: The number of processed records for reference data import jobs shows the summary of all types of imported records, such as business entity records and relationship records, to identify which record type failed. For example, when you import 5 rows of flat code lists, the **Input Rows** field displays the number of processed records as 10 which includes the business entity records and the relationship records.

The following image shows the details of a reference data import job:



1. Overview. Lists the details of the job instance for each step, such as instance ID, status, start time, end time, and duration.
2. Runtime Parameters. Lists the runtime parameters of the job instance and each step.
3. Metrics. Lists the key metrics related to each step, such as total records processed.
4. Errors. Lists the rejected and invalid records that failed to adhere to data quality rules.

Creating a job schedule

Schedule a job to ensure that the job executes at a specified time. You can also set a frequency to repeat the job at regular intervals. Ensure that you create a job before creating the job schedule.

1. On the **My Jobs** page, click **Job Schedules** > **Add Job Schedule**.

   The **New Job Schedule** page appears.
2. Enter the name of the job schedule.
3. To select a job, perform the following tasks:
   a. Click the asset picker.

      The **Select a Job Definition** page appears.
   b. Select a project, and then select a job definition.
   c. Click **Select**.
4. Select an existing IICS schedule, or add a new schedule. To add a new IICS schedule, perform the following tasks:
   a. Click **New Schedule**.

      The **New Schedule** dialog box appears.
   b. Enter the name and description of the schedule.
   c. Select the start date and time of the schedule.
   d. Select the time zone for the schedule.
   e. To run the job again in a schedule, select a frequency.
   f. Click **Save**.
5. Click **Save** to save the job schedule.

Modifying and rerunning reference data import and export jobs

You can modify the properties of the existing reference data import and export jobs after you create them. You can also rerun the jobs.

1. On the **My Jobs** page, click the **Job Definitions** tab.

   You can view the existing reference data import and export jobs.
2. To rerun the job, click **Run**.

   You can monitor the status of job on the **My Jobs** page.
3. To modify the properties of a job, perform the following tasks:
   a. Click the job.

      You can view the details of the job.
   b. Click **Actions** > **Properties**.
   c. Modify the display name and description of the job.

      Note: You can't modify the location of the job.
   d. Click **OK**.

System-generated jobs

Reference 360 runs system-generated jobs to complete operations that take longer than expected.

Reference 360 draft job

The Reference 360 draft job is a system-generated job that runs when you import large number of code values to a draft code list and publish or discard the draft code list. The code list is locked when the job is triggered.

When operations, such as publishing or discarding a draft code list with large code value changes take longer than expected, the Reference 360 draft job starts to complete these operations. You can view the job details on the **My Jobs** page.

To stop a Reference 360 draft job that is in progress, perform one of the following tasks on the **My Jobs** page:

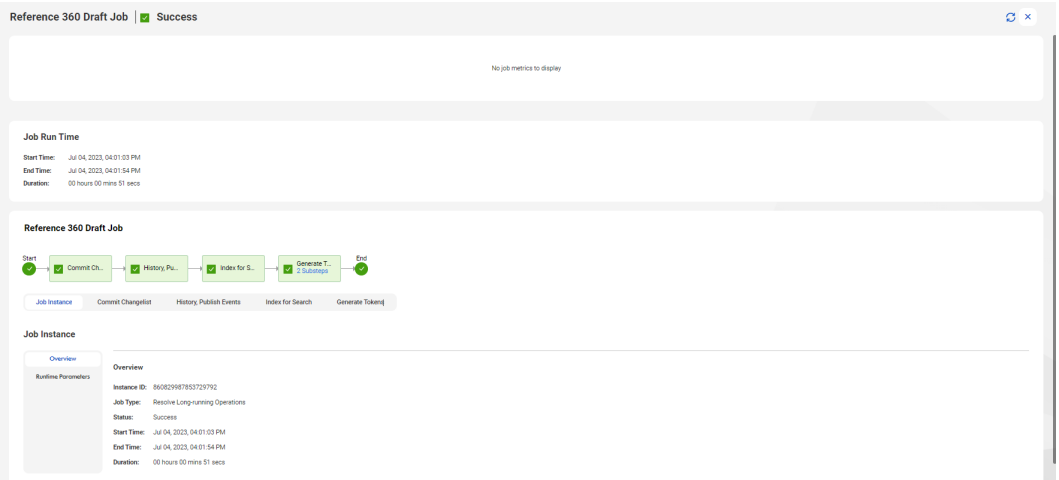
- On the **Actions** menu of the job, click **Stop**.
- Click the status of a job that is running, and then click **Stop** on the job details page.

You can restart a job. A new job is triggered when you restart the job that is stopped.

Viewing Reference 360 draft jobs

After the Reference 360 draft job is run, you can monitor and view the job details.

The following image shows a sample Reference 360 draft job type:



You can view the following steps in a Reference 360 draft job:

- Commit Changelist. Commits large code value changes to a draft code list.

  Note: The status of the code list changes to unlocked after the commit changelist step is complete.
- History, Publish Events. Generates the history feed for the imported code values.
- Index for Search. Indexes the code values for search.
- Generate Tokens. Generates tokens for the code values.

Click each step within a data flow to view more information. The parameters vary based on the step.

Note: You can view the history details of the imported code values only when the history, publish events step is complete.

Stopping reference data import and file import jobs

You can stop reference data import and file import jobs that are in progress. For example, if a job takes a longer time to complete, you might want to stop it. You can't resume a job that is stopped.

To stop a job, perform one of the following tasks on the **My Jobs** page:

- On the **Actions** menu of the job, click **Stop**.
- Click the status of a job that is running, and then click **Stop** on the job details page.

If you stop the reference data import and file import jobs during or after the Load step, the data is partially imported. To find and process this data, contact your administrator.

You can also download error reports for the stopped file import jobs.

Monitoring reference data import jobs

After you run a reference data import job, you can monitor and view the job details on the **My Jobs** page.

You can use a reference data import job to import code lists, crosswalks, and hierarchies to Reference 360. When you get started with Reference 360, you need to ingress your initial data to onboard it.

The reference data import job includes the following steps:

- Extract
- Transform
- Load
- Index for Search

You can view the following key metrics about the reference data import job:

- Input Rows. The number of rows of assets in the CSV file to import.
- Success Rows. The number of rows of assets in the CSV file that are moved to the load step for processing.
- Error Rows. The number of rows of assets in the CSV file that are not processed in the transform step.

Extract

The extract step extracts data from the source system to a staging area. A staging area is a temporary location where the data is stored.

If the reference data import job fails at this step or the step is in the Running status for a long time, go to the **My Jobs** page of Cloud Data Integration to troubleshoot. If the taskflow is in the Suspended status, view the subtasks and select the failed mapping task. Download the sessions log and check for errors.

The following table describes the metrics that you can view for the extract step:

Metric	Description
Total Records	The total number of code values and value mappings that the extract step processed.
Error Records	The number of code values and value mappings that didn't process.
Success Records	The number of code values and value mappings that the step successfully extracted from the source system.
Starting Records for Asset	The total number of code values and value mappings of crosswalks that the extract step processed for the specific asset.

Transform

The transform step applies basic and advanced rule associations to the staged data. The rule associations cleanse and transform the staged data.

If the reference data import job fails at this step or the step is in the Running status for a long time, go to the My Jobs page of Data Integration to troubleshoot. If the taskflow is in the Suspended status, view the subtasks and select the failed mapping task. Download the sessions log and check for errors.

The following table describes the metrics that you can view for the transform step:

Metric	Description
Total Records	The total number of code values and value mappings that the transform step processed.
Error Records	The number of code values and value mappings that failed the data quality rule associations.
Success Records	The number of code values and value mappings that are validated and transformed and then moved to the load step.
Starting Records for Asset	The total number of code values and value mappings that the transform step processed for the specific asset.

Load

The load step loads the cleansed code values and the value mappings into Reference 360.

If the reference data import job fails at this step or the step is in the Running status for a long time, go to the **My Jobs** page of Data Integration to troubleshoot. If the taskflow is in the Suspended status, view the subtasks and select the failed mapping task. Download the sessions log and check for errors.

You can view the following key metrics about the reference data import job for the load step:

- Success Records. The number of assets that are loaded and moved to the index for search step.
- Error Records. The number of assets the load step didn't process.

The following table describes the metrics that you can view for the load step:

Metric	Description
Total Records	The total number of code values and value mappings that the load step processed.
Error Records	The number of code values and value mappings the step didn't process.
Success Records	The number of code values and value mappings that are loaded and moved to the index for search step.
Starting Records for Asset	The total number of code values and value mappings that the load step processed for the specific asset.

Index for Search

The index for search step indexes the code values and value mappings for search.

If the reference data import job fails at this step or the step is in the Running status for a long time, go to the My Jobs page of Data Integration to troubleshoot. If the taskflow is in the Suspended status, view the subtasks and select the failed mapping task. Download the sessions log and check for errors.

The following table describes the metrics for the index for search step:

Metric	Description
Total Records	The total number of code values and value mappings that the index for search step processed.
Error Records	The number of code values and value mappings that the step didn't process because of internal errors
Success Records	The number of code values that are indexed for search.
Starting Records for Asset	The total number of code values and value mappings that the index for search step processed for the specific asset.

Monitoring the reference data export jobs

After you run a reference data export job, you can monitor and view the job details on the **My Jobs** page.

A reference data export job exports data from Reference 360 to other external data sources.

The reference data export job contains the following steps:

- Stage
- Extract

Stage

The stage step loads the code values and value mappings from Reference 360 to a staging collection.

The following table describes the metrics that you can view for the stage step:

Metric	Description
Total Records	The total number of code values and value mappings that the stage step processed. **Note:** The total number of processed records for flat code lists includes only business entity records.
Error Records	The number of code values and value mappings that the stage step didn't process because of internal errors.
Success Records	The number of code values and value mappings that the stage step processed and moved to the extract step.
Starting Records for Asset	The total number of code values and value mappings that the stage step processed for the specific asset.

Extract

The extract step reads and extracts code values and value mappings from the staging collection and writes them to an external data source.

The following table describes the metrics that you can view for the extract step:

Metric	Description
Total Records	The total number of code values and value mappings that the extract step processed.
Error Records	The number of code values and value mappings that the extract step didn't process because of internal errors.
Success Records	The number of code values and value mappings that are exported to an external data source.
Starting Records for Asset	The total number of code values and value mappings that the extract step processed for the specific asset.

Monitoring Reference 360 Draft jobs

The Reference 360 draft job is a system-generated job that runs when you import large number of code values to a draft code list and publish or discard the draft code list. The code list is locked when the job is triggered.

After you run an Reference 360 draft job, you can monitor and view the job details on the **My Jobs** page.

The Reference 360 draft job includes the following steps:

- Commit Changelist
- History, Publish Events
- Index for Search
- Generate Token

Commit Changelist

The commit changelist step commits large code value changes to a draft code list. After the step is complete, the state of the code list changes to unlocked.

The following table describes the metrics that you can view for the commit changelist step:

Metric	Description
Total Records	The total number of code values that the commit changelist step processed.
Error Records	The number of code values that failed to commit changes to the draft code list.

Metric	Description
Success Records	The number of code values that committed changes to the draft code list and moved to the history step.
Starting Records for Asset	The total number of code values that the commit changelist step processed for a specific asset, such as code list.

History, Publish Events

The history step generates the history feed for the imported code values in the code list.

The following table describes the metrics that you can view in the history, publish events step:

Metric	Description
Total Records	The total number of code values tha the history step processed.
Error Records	The number of code values that failed to generate the history feed.
Success Records	The number of code values that generated the history feed and moved to the index for search step.
Starting Records for Asset	The total number of code values that the history step processed for a specific asset, such as code list.

Index for Search

The index for search step indexes the code values for search.

The following table describes the metrics that you can view for the index for search step:

Metric	Description
Total Records	The total number of code values that the index for search step processed.
Error Records	The number of code values that weren't indexed.
Success Records	The number of code values that were indexed and moved to the next step.
Starting Records for Asset	The total number of code values that the step processed for a specific asset, such as code list.

Generate Tokens

The generate tokens step isn't applicable for Reference 360 and is reserved for future use.

CHAPTER 14

Monitor Reference 360

You can use Operational Insights to view the usage statistics, such as the total number of code values in an organization and the number of code values in each code list.

To monitor Reference 360 in Operational Insights, your user role requires access to Operational Insights.

Configure access to monitor Reference 360

To monitor Reference 360 in Operational Insights, configure access to Operational Insights for your user role.

1. In Administrator, click **User Roles**.
2. Select the custom user role to which you want to provide access for monitoring Reference 360.
3. From the **Services** list on the role details page, select **Operational Insights**.
4. Click the **Features** tab.
5. Select **Operational Insights - view**.
6. From the **Services** list, select **MDM Configuration**.
7. Select **Configuration and Authoring**.
8. Click **Save**.

Monitor usage statistics for Reference 360

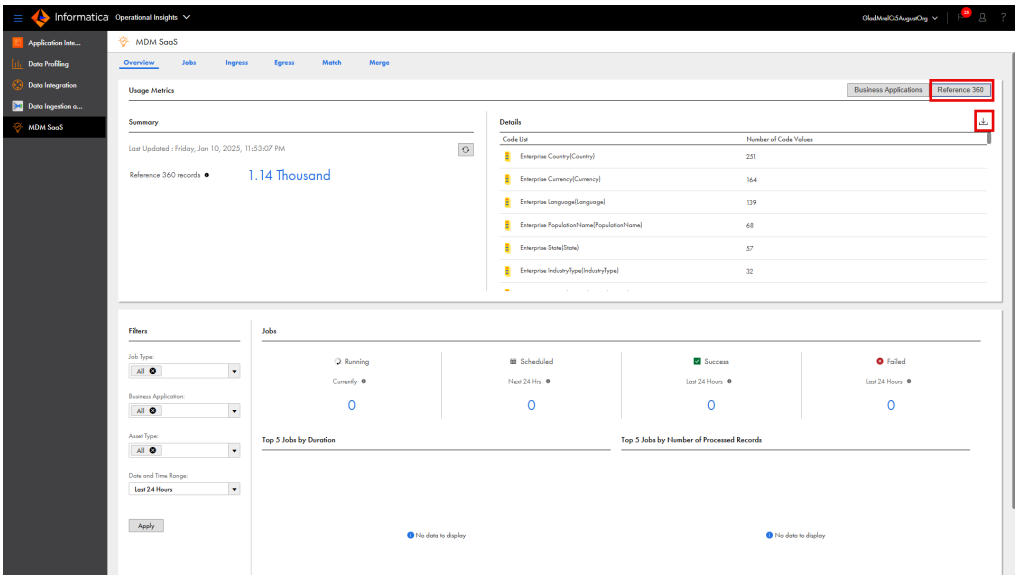
You can view the total number of code values in an organization and the number of code values in each code list.

1. In Operational Insights, click **MDM SaaS**.

   The **Overview** page shows the usage statistics of business applications by default.
2. Click **Reference 360**.

   The **Summary** section shows the total number of code values in an organization, and the **Details** section shows the number of code values in each code list.

The following image shows the usage statistics of Reference 360:



3. To download the metrics as a CSV file, click **Download**.
4. To view the updated count of code values, perform the following steps:
   a. In the **Summary** section, click the **Refresh** icon.

      The **Summary Update Request** dialog box appears.

   b. Click **OK**.

   The refresh MDM SaaS usage metrics job runs to update the count of code values. You can monitor and view the job details on the My Jobs page. If the job fails due to a timeout error, you can still view the updated count of code values. However, if the job fails due to other reasons, contact Informatica Global Customer Support.

   Note: You can refresh the metrics only once a week. For existing organizations, the records count is displayed as 0 until the refresh MDM SaaS usage metrics job completes successfully for the first time.

CHAPTER 15

Reference 360 REST API

Use the Reference 360 REST APIs to interact with your reference data. For example, you can export or import reference data sets, import code values and value mappings, or retrieve a list of assets and their details.

When you use Reference 360 REST APIs, note the following rules:

- Use the following base URL:

```
<serverUrl>/rdm-service/external/v1/<API name>
```

- Use the following request header format:

```
<METHOD> <serverUrl>/<URI> HTTP/<HTTP version>
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

For example, you might use the following request to export the model for reference data sets:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/model/export
HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
   "referenceDataSetIds":[
      "56dbde1afe4d8257b6d7735e",
      "a83fa4bda81df711caca4e71"
   ]
}
```

To access the REST API documentation in the OpenAPI format, use the following URL format:

```
https://{POD region}.informaticacloud.com/rdm-service/api-docs
```

The following sample URL uses `usw3-mdm.dm-us` as the POD region value:

```
https://<usw3-mdm.dm-us>.informaticacloud.com/rdm-service/api-docs
```

You can also access the REST API documentation in the Swagger UI with the following URL format:

```
https:://<host>:<port>/rdm-ui/swagger
```

Session IDs

To authenticate your Reference 360 REST API requests, you must get a session ID and add it to the header of every request. The Informatica Intelligent Cloud Services (IICS) Identity Service issues the session ID.

To log in and authenticate, you can use Login API V3, Login API V2 or Login API V1. After you log in, you get a session ID that you must use to authenticate API requests. Ensure that you use only the IDS-SESSION-ID header for authentication, not the Authorization header.

Note: If your session ID expires, log in again to get a new session ID.

Login API V3

To get a session ID using Login API V3, submit the following POST request with your credentials:

```
POST https://dm-us.informaticacloud.com/saas/public/core/v3/login
Accept: application/json
Content-Type: application/json

{
     "username": "<username>",
     "password": "<password>",
}
```

The login response includes the session ID and base URL that you need to include in subsequent REST API calls.

Use values from the following fields in the REST API calls:

- sessionId. REST API session ID that you include in the header for version 3 REST API calls. The session ID expires after 30 minutes of inactivity. After the session ID expires, log in again to continue working with the REST API.
- baseApiUrl. Base URL that you use for subsequent requests during this session.

Login API V2

To get a session ID using Login API V2, submit the following POST request with your credentials:

```
POST  https://dm-us.informaticacloud.com/ma/api/v2/user/login
Accept: application/json
Content-Type: application/json

{
     "username": "<username>",
     "password": "<password>",
}
```

Use the base URL and session ID returned in the response for subsequent requests during this session.

Login API V1

To get a session ID using Login API V1, submit the following POST request with your credentials:

```
POST https://dm-us.informaticacloud.com/identity-service/api/v1/Login
Accept: application/json
Content-Type: application/json

{
  "username": "<username>",
  "password": "<password>",
}
```

The response returns a `sessionId`.

For example, you might receive the following response:

```
{
  ...
```

```
  "sessionId": "95H4irGZd6glXndROHbniS",
  "sessionExpireTime": "2025-01-01T00:00:00.000Z",
  ...
}
```

To authenticate your requests, add `IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX` in the request header.

The following example shows how `IDS-SESSION-ID` is used in the request header:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/model/export
HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: 95H4irGZd6glXndROHbniS

{
  "referenceDataSetIds":[
    "56dbde1afe4d8257b6d7735e",
    "a83fa4bda81df711caca4e71"
  ]
}
```

Asset IDs

Each asset has an unique identifier. Some Reference 360 REST APIs require you to specify the ID of the asset. You can identify the ID of an asset in Reference 360 or by using REST APIs.

Reference 360

In Reference 360, the ID of an asset appears in the URL when you open a reference data set, code list, crosswalk, or hierarchy.

When you open an asset, you see the following URL format:

```
https://use4-mdm.dm-us.informaticacloud.com/rdm-ui/#/<asset type>/<asset ID>/edit
```

For example, the asset ID for the following code list is `5d38987dbc49de0001113db3`:

```
https://use4-mdm.dm-us.informaticacloud.com/rdm-ui/#/codelist/5d38987dbc49de0001113db3/
edit
```

REST APIs

The following table describes the REST APIs that you can use to identify the ID of an asset:

REST API	Description
List reference data sets	Retrieves a list of reference data sets with their ID. With the ID of a reference data set, you can use the List code lists REST API to retrieve a list of code lists.
List code lists	Retrieves a list of code lists with their IDs for the specified reference data set. With the ID of a code list, you can use the List crosswalks REST API to retrieve a list of crosswalks.
List crosswalks	Retrieves a list of crosswalks associated with the code list and the ID of each crosswalk.
List hierarchies	Retrieves a list of hierarchies with their ID.

REST API versions

You can use different versions of REST APIs to perform tasks.

The following table describes REST APIs, their versions, and the functionalities provided by each version:

REST API	version	Description
configuration	version 1	Configures the Reference 360 organization with settings, such as the approval mode configuration, approval implementation mode, and the rule validation mode. For more information about the configuration REST API, see "configuration" on page 204.
model	version 3	Exports or imports reference data sets, including their code lists and crosswalks, and hierarchies. For more information about the model version 3 REST API, see " model version 3" on page 210.
import	version 1	Imports code values, value mappings, and hierarchy relationships, and retrieves the status of an import job and the detailed report for a failed import job based on asset IDs. For more information about the import version 1 REST API, see "import version 1" on page 259.
import	version 2	Imports code values, value mappings, and hierarchy relationships with additional optional fields based on asset IDs. For more information about the import version 2 REST API, see "import version 2" on page 269.
import	version 3	Imports code values, value mappings, and hierarchy relationships with additional optional fields based on internal IDs or aliases of assets. For more information about the import version 3 REST API, see "import version 3" on page 278.
export	version 1	Exports code values from a code list and value mappings from a crosswalk to JSON and CSV formats based on asset IDs. For more information about the export version 1 REST API, see "export version 1" on page 290.

REST API	version	Description
export	version 2	Exports code values from a code list, value mappings from a crosswalk, and relationships within a hierarchy to JSON and CSV formats based on asset IDs. For more information about the export version 2 REST API, see "export version 2" on page 296.
export	version 3	Use the REST API to perform the following actions based on asset IDs: - Export code values from a code list and value mappings from a crosswalk to JSON and CSV formats. - Export code values from a code list and value mappings from a crosswalk that were available at a specific point in time to JSON and CSV formats For more information about the export version 3 REST API, see "export version 3" on page 309.
export	version 4	Use the REST API to perform the following actions based on the internal IDs or aliases of assets: - Export code values from a code list and value mappings from a crosswalk to JSON and CSV formats. - Export code values from a code list and value mappings from a crosswalk that were available at a specific point in time to JSON and CSV formats For more information about the export version 4 REST API, see "export version 4" on page 329.
audit trail	version 1	Retrieves the history of the specified assets based on asset IDs. For more information about the audit trail version 1 REST API, see "audit trail version 1" on page 350.
audit trail	version 2	Retrieves the history of the specified assets based on internal IDs or aliases of assets. For more information about the audit trail version 2 REST API, see "audit trail version 2" on page 353.

REST API	version	Description
rds	version 1	Retrieves the details for a reference data set, and code lists of a reference data set based on the asset ID of the reference data set. For more information about the rds version 1 REST API, see "rds version 1" on page 357.
rds	version 2	Retrieves the details for a reference data set, and code lists of a reference data set based on the internal ID or alias of the reference data set. For more information about the rds version 2 REST API, see "rds version 2" on page 368.
codelists	version 1	Use the REST API to perform the following actions based on the asset ID: - Retrieve details of code lists and code values. - Create, update, delete, search, and export code values directly without creating draft changes or sending changes for approval. - Move code values within hierarchical code lists without locking the code list. - Retrieve change events, crosswalks, and relationships for specified time ranges. You can also synchronize Data Quality dictionaries with code values of a code list based on the internal ID or alias of the code list. For more information about the codelists version 1 REST API, see "codelists version 1" on page 381.

REST API	version	Description
codelists	version 2	Use the REST API to perform the following actions based on the internal ID or alias of the code list: - Retrieve and manage code lists and code values. - Create, update, delete, search, and export code values directly without creating draft changes or sending changes for approval. - Move code values within hierarchical code lists without locking the code list. - Retrieve change events, crosswalks, and summary information associated with a code list. You can also unlock code lists locked by other users based on the asset ID. For more information about the codelists version 2 REST API, see "codelists version 2" on page 423.
codelists	version 3	Unlocks a code list that's locked by another user based on the internal ID or alias of the code list. For more information about the codelists version 3 REST API, see "codelists version 3" on page 466.
crosswalks	version 1	Use the REST API to perform the following actions based on the asset ID: - Retrieve crosswalk details, value mappings, and change events for a specific time range. - Unlock crosswalks that other users have locked. - Run crosswalk cleanser jobs to remove duplicate mappings and retrieve job details. You can also synchronize a Data Quality dictionary with value mappings of a crosswalk based on the internal ID or alias of the crosswalk. For more information about the crosswalks version 1 REST API, see "crosswalks version 1" on page 466.

REST API	version	Description
crosswalks	version 2	Use the REST API to perform the following actions based on the internal ID or alias of the crosswalk: - Retrieve crosswalk details, value mappings, and change events for a specific time range. - Unlock crosswalks that other users have locked. - Run crosswalk cleanser jobs to remove duplicate mappings and retrieve job details. - Delete value mappings by source and target code values. For more information about the crosswalks version 2 REST API, see "crosswalks version 2" on page 475.
crosswalks	version 3	Use the REST API to perform the following actions based on the internal ID or alias of the crosswalk: - Retrieve crosswalk details, value mappings, and change events for a specific time range. - Delete value mappings by mapped source and target code values. For more information about the crosswalks version 3 REST API, see "crosswalk version 3" on page 487.
hierarchies	version 1	Retrieves hierarchies, hierarchy details, and hierarchy model relationships based on the asset ID of the hierarchy. For more information about the hierarchies version 1 REST API, see "hierarchies version 1" on page 495.
hierarchies	version 2	Retrieves a list of hierarchies, hierarchy details, and hierarchy model relationships based on the internal ID or alias of the hierarchy. For more information about the hierarchies version 2 REST API, see "hierarchies version 2" on page 505.

configuration

Use this resource to configure the Reference 360 organization with settings, such as the approval mode for Business Stewards, approval implementation mode, and the rule validation mode in Reference 360.

Note: To use the configuration resource, you must be assigned the Reference 360 Administrator role.

Approval mode configuration

The approval mode determines whether approval is required for changes to code values and value mappings, and whether direct import is enabled.

The following table describes the supported approval modes for users assigned the Business Steward role:

Approval Mode	Description
DIRECT_PUBLISH_AND_APPROVAL	- Can choose to send their changes for approval or publish their drafts. - Can directly import code values and value mappings.
DIRECT_PUBLISH	- Must publish their drafts. - Can directly import code values and value mappings.
APPROVAL	- Must send their changes for approval. - Can't directly import code values and value mappings.

Approval implementation mode

The approval implementation mode determines whether the Workflow Inbox and multi-step approval workflows are enabled.

Note: Before you update the approval implementation mode, resolve all tasks and close all draft code lists. Existing tasks and draft code lists will be discarded. After you update the approval implementation mode to `PLATFORM`, you cannot revert back to the `RDM` approval implementation mode.

The following table describes the supported approval implementation modes:

Approval Implementation Mode	Description
RDM	Enables the Tasks interface and uses a pre-defined one-step approval workflow. The default approval implementation mode is `RDM`.
PLATFORM	Enables the Workflow Inbox interface and multi-step approval workflows.

Rule validation mode

The rule validation mode determines the type of rules that you can configure for attributes in a code list.

The following table describes the rule validation modes:

Rule Validation Mode	Description
RDM	Allows configuring the legacy rules in Reference 360.
PLATFORM	Allows configuring the basic rule associations based on the enhanced data validation framework.

Note: After you change the rule validation mode to `PLATFORM`, you cannot revert back to the `RDM` rule validation mode.

Get authorization mode configuration

Retrieves the authorization mode that Reference 360 uses to control user access to reference data assets.

Note: Effective in the October 2026 release, the ability to retrieve the authorization mode configuration is available for preview. Preview functionality is supported for evaluation purposes but is unwarranted and is not supported in production environments or any environment that you plan to push to production. Informatica intends to include the preview functionality in an upcoming release for production use, but might choose not to in accordance with changing market or technical circumstances. For more information, contact Informatica Global Customer Support.

GET request

To get the authorization mode configuration, submit a GET request with the following URI:

```
/rdm-service/external/v1/configuration/authorizationMode
```

GET response

The response contains the authorization mode.

The following table describes the mode:

Field	Description
mode	Name of the authorization mode. You can use one of the following modes: - STAKEHOLDER. Grants access at the individual asset level. Users with the Organization Administrator role assign predefined roles to users or user groups, and users with the Primary Owner role assign those users or user groups as stakeholders for a specific code list, crosswalk, or hierarchy. - RBAC. Grants access through roles. Users with the Organization Administrator role define custom user roles and grant those roles create, read, update, and delete privileges to reference data assets.

GET example

To retrieve the authorization mode, you might use the following request:

```
GET  https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/configuration/
authorizationMode HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the authorization mode:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 31

{
        "mode":"RBAC"
}
```

Update authorization mode configuration

Updates the authorization mode configuration.

Note: Effective in the October 2026 release, the ability to update the authorization mode configuration is available for preview.

Preview functionality is supported for evaluation purposes but is unwarranted and is not supported in production environments or any environment that you plan to push to production. Informatica intends to include the preview functionality in an upcoming release for production use, but might choose not to in

accordance with changing market or technical circumstances. For more information, contact Informatica Global Customer Support.

PUT request

To update the authorization mode configuration, submit a PUT request with the following URI specifying the authorization mode:

```
/rdm-service/external/v1/configuration/authorizationMode?mode=RBAC
```

Use the following parameter in the URI and the request body to specify the authorization mode:

Parameter	Description
mode	Authorization mode. Value is STAKEHOLDER or RBAC.

PUT response

The request returns a 204 no content response.

PUT example

To update the authorization mode configuration, you might use the following request:

```
PUT https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/configuration/
authorizationMode?mode=RBAC HTTP/1.1
Host: localhost:8080
Content-Type: application/json

mode=RBAC
```

The following sample response shows a successful update with no content returned:

```
HTTP/1.1 204 No Content
```

Get approval mode configuration

Retrieves the approval mode configuration.

GET request

To get the approval mode configuration, submit a GET request with the following URI:

```
/rdm-service/external/v1/configuration/approval
```

GET response

The response contains the approval mode.

The following table describes the field:

Field	Description
mode	Approval modes. The following modes are available: - DIRECT_PUBLISH_AND_APPROVAL - DIRECT_PUBLISH - APPROVAL

GET example

To retrieve the approval mode, you might use the following request:

```
GET  https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/configuration/
approval HTTP/1.1
```

```
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the approval mode:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 31

{
        "mode" : "DIRECT_PUBLISH"
}
```

Update approval mode configuration

Updates the approval mode configuration.

PUT request

To update the approval mode configuration, submit a PUT request with the following URI and specify the approval mode:

```
/rdm-service/external/v1/configuration/approval?mode=<approval mode>
```

Use the `mode` parameter in the URI and request body to specify the approval mode configuration.

Parameter	Description
mode	Approval modes. Values are `DIRECT_PUBLISH_AND_APPROVAL`, `DIRECT_PUBLISH`, or `APPROVAL`.

PUT response

A 204 no content response is returned.

PUT example

To update the approval mode configuration, you might use the following request:

```
PUT https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/configuration/
approval?mode=DIRECT_PUBLISH_AND_APPROVAL HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded

mode=DIRECT_PUBLISH_AND_APPROVAL
```

The following sample response shows the no content response:

```
HTTP/1.1 204 No Content
```

Get approval implementation mode

Retrieves the approval implementation mode.

GET request

To get the approval implementation mode, submit a GET request with the following URI:

```
/rdm-service/external/v1/configuration/approvalImplementation
```

GET response

The response contains the approval implementation mode.

The following table describes the field:

Field	Description
mode	Approval implementation mode. The following modes are available: - RDM. Enables the Tasks interface and uses a pre-defined one-step approval workflow. - PLATFORM. Enables the Workflow Inbox interface and multi-step approval workflows.

GET example

To retrieve the approval implementation mode, you might use the following request:

```
GET  https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/configuration/
approvalImplementation HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the approval implementation mode:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 20

{
   "mode":"RDM"
}
```

Update approval implementation mode

Updates the approval implementation mode.

Note: Before you update the approval implementation mode, resolve all tasks and close all draft code lists. Existing tasks and draft code lists will be discarded. After you update the approval implementation mode to PLATFORM, you cannot revert back to the RDM approval implementation mode.

PUT request

To update the approval implementation mode, submit a PUT request with the following URI and specify the approval implementation mode:

```
/rdm-service/external/v1/configuration/approvalImplementation?mode=<mode>
```

Use the mode parameter in the URI and request body to specify the approval mode configuration.

Field	Description
mode	Approval implementation mode. The following modes are available: - RDM. Enables the Tasks interface and uses a pre-defined one-step approval workflow. - PLATFORM. Enables the Workflow Inbox interface and multi-step approval workflows.

PUT response

A 204 no content response is returned.

PUT example

To update the approval implementation mode, you might use the following request:

```
PUT https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/configuration/
approvalImplementation?mode=PLATFORM HTTP/1.1
Content-Type: application/x-www-form-urlencoded
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX
```

```
mode=PLATFORM
```

The following sample response shows the no content response:

```
HTTP/1.1 204 No Content
```

model version 3

Use this resource to export and import reference data sets and hierarchies.

Note: To use the model resource, you must be assigned the Reference 360 Administrator role.

The model resource exports and imports reference data sets and hierarchies, including the following objects:

- Basic and advanced rule associations of code list attributes
- Crosswalks
- User groups in the stakeholder configuration of assets
- Workflow configurations that contain workflow tasks with user group assignments
- Dictionaries

The model resource doesn't export and import the data in code lists or crosswalks, such as code values and value mappings.

When you migrate reference data sets and hierarchies, the model resource doesn't export the rule specifications for advanced rule associations that are configured for code list attributes. The model resource includes only the names and location of the rule specifications in Cloud Data Quality to identify the rule specification details.

Before you import the exported model, ensure that you migrate the rule specifications from Cloud Data Quality to the same locations in your target organization. If the target organization doesn't contain the rule specifications, the import model imports the fields that are configured with the advanced rule associations without the rule. You can also view the details of the objects that were ignored during the import in the import model report.

When you import code lists with attributes that contain both default values and advanced rule associations, the import fails. To resolve this issue, remove the default values from the code list attributes in the source organization.

To export and import data, use the export and import resources.

Export model (v3)

Exports reference data sets and hierarchies.

Note: When you create a crosswalk between code lists of different reference data sets without specifying the IDs of the source and target reference data sets, the crosswalk export fails without a warning message.

POST request

To export reference data sets and hierarchies based on internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v3/model/export
```

The export model API might time out if the export process takes longer than 5 minutes.

Use the following attributes in the request body to specify the reference data sets and hierarchies to export:

Field	Type	Description
referenceDataSetIds	Array	Comma-separated list of IDs of reference data sets to export. If you don't specify IDs, all reference data sets are exported. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
referenceDataSetInternalIds	Array	Comma-separated list of internal IDs of reference data sets to export.
referenceDataSetAliases	Array	Comma-separated list of aliases of reference data sets to export.
hierarchyIds	Array	Comma-separated list of IDs of hierarchies to export. If you don't specify IDs, all hierarchies are exported. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
hierarchyInternalIds	Array	Comma-separated list of internal IDs of hierarchies to export.
hierarchyAliases	Array	Comma-separated list of aliases of hierarchies to export.

POST response

The response contains the exported assets.

The following table describes the attributes in the response body:

Field	Type	Description
version	String	Version of the export file. The latest version is `4.1` to reflect the addition of multilingual reference data. **Note:** The version of the export file is updated based on the enhancements to the export and import model version 3 REST APIs.
referenceDataSets	-	Includes details about the reference data set.
id	String	ID of the reference data sets. For more information, see "Asset IDs" on page 199.
name	String	Name of the reference data set.
internalId	String	Internal ID of the reference data set.
alias	String	Alias of the reference data set.
description	String	Description of the reference data set.
hierarchical	Boolean	Indicates whether code lists in the reference data set inherit the hierarchical structure definition.
levels	Number	Number of hierarchical levels supported in the code lists associated with the reference data set. If hierarchical levels are not provided, value is `1`. If hierarchical levels are unlimited, value is `-1`.

Field	Type	Description
defaultList	String	ID of the default code list.
domain	String	Optional. Domain of the reference data set.
confidentiality	String	Optional. Confidentiality of the reference data set.
priority	String	Optional. Priority of the reference data set.
status	String	Optional. Status of the reference data set.
codeValueFields	-	Includes the attribute definition for code values in the reference data set.
name	String	Name of the field.
origin	String	Origin of the definition of the field.
datatype	String	Datatype of the field. Values are `String`, `Integer`, `Decimal`, `Boolean`, `Date`, or `Reference Data`.
mandatory	Boolean	Indicates whether the attribute is required.
localized	Boolean	Indicates whether the attribute is multilingual.
localizationDef	Object	Includes the definition of the target languages that the code list depends on.
targetLanguages	Array	List of preferred additional languages to display the attribute.
dependencyDef	-	Includes the definition of the asset specified as the dependency.
termId	String	ID of the asset specified as the dependency.
assetStakeholders	-	Stakeholders associated to the asset.
codeLists	-	Includes details about the code list.
id	String	ID of the code lists. For more information, see "Asset IDs" on page 199.
termId	String	ID of the reference data set to which the code list is associated.
name	String	Name of the asset.
description	String	Description of the asset.
hierarchical	Boolean	Indicates whether code lists in the reference data set inherit the hierarchical structure definition.
levels	Number	Number of hierarchical levels supported in the code lists associated with the reference data set. If hierarchical levels are not provided, value is `1`. If hierarchical levels are unlimited, value is `-1`.
codeValueFields	-	Includes the attribute definition for code values in the reference data set.

Field	Type	Description
name	String	Name of the field.
origin	String	Origin of the definition of the field.
datatype	String	Datatype of the field. Values are `String`, `Integer`, `Decimal`, `Boolean`, `Date`, or `Reference Data`.
mandatory	Boolean	Indicates whether the attribute is required.
businessIdDef	-	Optional. Includes the details of the business IDs configured for the code values.
type	String	Format in which the business IDs are generated. Values are `NUMERIC` or `ALPHANUMERIC`.
prefix	String	Characters added to the beginning of business IDs. Values are `NUMERIC` or `ALPHANUMERIC`.
length	Number	Total length of the business IDs.
offset	String	Starting value for the business IDs that Reference 360 generates. Applicable for numeric business IDs.
label	String	Display name for the business ID field.
dependencyDef	-	Includes the definition of the asset specified as the dependency.
termId	String	ID of the asset specified as the dependency.
displayColumns	Array	Display columns used as labels for code values associated with the dependent asset.
dqValidationInfo	-	Includes details about the basic and advanced rule associations of code list attributes.
assetStakeholders	-	Stakeholders associated to the asset.
assetWorkflowConfiguration	-	Workflow configuration of the asset.
dictionaries	-	Includes the details of dictionaries configured for the code list.
dictionaryConfigurationId	String	Identifier of the dictionary.
name	String	Name of the dictionary.
description	String	Description of the dictionary.
location	String	The project or folder within which the dictionary is located.
dataSourceConfiguration	String	Code list attributes that are added as columns in the dictionary.
crosswalks	-	Details about the crosswalk.
id	String	ID of the crosswalks. For more information, see "Asset IDs" on page 199.

Field	Type	Description
description	String	Description of the asset.
status	String	Status of the asset.
sourceCodelistId	String	ID of the source code list to which the crosswalk is associated.
targetCodelistId	String	ID of the target code list.
assetStakeholders	-	Stakeholders associated to the asset.
enums	-	Includes details about enum groups and entries.
key	String	ID of the system reference data value.
label	String	Label for the system reference data value.
dictionaries	-	Includes the details of dictionary configured for the crosswalk.
dictionaryConfigurationId	String	Identifier of the dictionary.
name	String	Name of the dictionary.
description	String	Description of the dictionary.
location	String	The project or folder within which the dictionary is located.
dataSourceConfiguration	String	Value mappings of the crosswalk added as columns in the dictionary.
hierarchies	-	Includes details about the hierarchies.
codeListRelations	-	Code list relations defined by the hierarchies.

POST example

To export reference data sets and hierarchies, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/model/export
HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
{
  "referenceDataSetIds": [
    "645ab8251c45ce8e99b99daf",
    "64089cd1d1cb171040bc679c"
  ],
  "referenceDataSetInternalIds": [
    "country_int"
  ],
  "referenceDataSetAliases": [
    "country_als"
  ],
  "hierarchyIds": [
    "4cde04c990465a2efa8a7e8b"
  ],
  "hierarchyInternalIds": [
    "hierarchy1_int"
  ],
  "hierarchyAliases": [
```

```
    "hierarchy1_als"
  ]
}
```

The following sample response shows the exported model of reference data sets and hierarchies:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 6076
{
  "version": "4.1",
  "referenceDataSets": [
    {
      "id": "645ab8251c45ce8e99b99daf",
      "name": "Country",
      "internalId": "country_int",
      "alias": "country_als",
      "description": "desc",
      "hierarchical": false,
      "levels": 1,
      "defaultList": "a7d999842c11b52a53865bbe",
      "codeValueFields": [
        {
          "name": "Name",
          "labels": [
            {
              "language": "en",
              "value": "Name"
            }
          ],
          "origin": "TERM",
          "datatype": "String",
          "mandatory": true,
          "position": 0
        },
        {
          "name": "Code",
          "labels": [
            {
              "language": "en",
              "value": "Code"
            }
          ],
          "origin": "TERM",
          "datatype": "String",
          "mandatory": true,
          "position": 0,
          "localized": true
        }
      ],
      "localizationDef": {
        "targetLanguages": [
          "en-US",
          "de-DE",
          "fr-FR"
        ]
      },
      "assetStakeholders": {
        "stakeholders": [
          {}
        ]
      }
    },
    {
      "id": "64089cd1d1cb171040bc679c",
      "name": "rds2",
      "description": "desc",
      "hierarchical": false,
      "levels": 1,
      "defaultList": "b8f577b04efe19d099fde755",
      "codeValueFields": [
```

```
      {
        "name": "Name",
        "labels": [
          {
            "language": "en",
            "value": "Name"
          }
        ],
        "origin": "TERM",
        "datatype": "String",
        "mandatory": true,
        "position": 0
      },
      {
        "name": "Code",
        "labels": [
          {
            "language": "en",
            "value": "Code"
          }
        ],
        "origin": "TERM",
        "datatype": "String",
        "mandatory": true,
        "position": 0
      },
      {
        "name": "fieldDef",
        "origin": "TERM",
        "datatype": "Reference",
        "mandatory": false,
        "relatedTermId": "645ab8251c45ce8e99b99daf",
        "displayColumns": [
          "name"
        ],
        "position": 0
      }
    ],
    "dependencyDef": {
      "termId": "645ab8251c45ce8e99b99daf",
      "displayColumns": [
        "name"
      ]
    },
    "assetStakeholders": {
      "stakeholders": [
        {}
      ]
    }
  }
],
"codeLists": [
  {
    "id": "b4e6823f84eed45eda84b2c1",
    "termId": "64089cd1d1cb171040bc679c",
    "name": "SapCountry",
    "description": "desc",
    "hierarchical": false,
    "codeValueFields": [
      {
        "name": "Name",
        "labels": [
          {
            "language": "en",
            "value": "Name"
          }
        ],
        "origin": "TERM",
        "datatype": "String",
        "mandatory": true,
        "position": 0
```

```
  },
  {
    "name": "Code",
    "labels": [
      {
        "language": "en",
        "value": "Code"
      }
    ],
    "origin": "TERM",
    "datatype": "String",
    "mandatory": true,
    "position": 0
  }
],
"assetWorkflowConfiguration": {
  "assetApiWorkFlowEnabled": false,
  "assetWorkFlowTasks": [
    {
      "taskName": "TestTaskSendback",
      "taskAction": {
        "approve": "REQUIRED",
        "reject": "REQUIRED",
        "sendBack": "REQUIRED"
      },
      "approvers": {
        "userGroups": [
          {
            "name": "testgroup2"
          }
        ]
      }
    },
    {
      "taskName": "TestTaskReject",
      "taskAction": {
        "approve": "OPTIONAL",
        "sendBack": "REQUIRED"
      },
      "approvers": {
        "userGroups": [
          {
            "name": "testgroup1"
          }
        ]
      }
    },
    {
      "taskName": "TestTakApprove",
      "taskAction": {
        "approve": "REQUIRED",
        "sendBack": "REQUIRED"
      },
      "approvers": {
        "userGroups": [
          {
            "name": "testgroup"
          }
        ]
      }
    }
  ]
},
"rdsName": "rds2",
"dqValidationInfo": {
  "enabled": false
},
"assetStakeholders": {
  "stakeholders": [
    {}
  ]
```

```
  }
},
{
  "id": "d36800d6fcf20b4b91a20f0e",
  "termId": "645ab8251c45ce8e99b99daf",
  "name": "rds1_cl1_name",
  "internalId": "rds1_cl1_int",
  "alias": "rds1_cl1_als",
  "description": "desc",
  "hierarchical": false,
  "codeValueFields": [
    {
      "name": "Name",
      "labels": [
        {
          "language": "en",
          "value": "Name"
        }
      ],
      "origin": "TERM",
      "datatype": "String",
      "mandatory": true,
      "position": 0
    },
    {
      "name": "Code",
      "labels": [
        {
          "language": "en",
          "value": "Code"
        }
      ],
      "origin": "TERM",
      "datatype": "String",
      "mandatory": true,
      "position": 0
    }
  ],
  "businessIdDef": {
    "type": "ALPHANUMERIC",
    "prefix": "RDM",
    "length": 10,
    "label": "modelExportBusinessId"
  },
  "assetWorkflowConfiguration": {
    "assetApiWorkFlowEnabled": false,
    "assetWorkFlowTasks": [
      {
        "taskName": "TestTaskSendback",
        "taskAction": {
          "approve": "REQUIRED",
          "reject": "REQUIRED",
          "sendBack": "REQUIRED"
        },
        "approvers": {
          "userGroups": [
            {
              "name": "testgroup2"
            }
          ]
        }
      },
      {
        "taskName": "TestTaskReject",
        "taskAction": {
          "approve": "OPTIONAL",
          "sendBack": "REQUIRED"
        },
        "approvers": {
          "userGroups": [
            {
```

```
                "name": "testgroup1"
              }
            ]
          }
        },
        {
          "taskName": "TestTakApprove",
          "taskAction": {
            "approve": "REQUIRED",
            "sendBack": "REQUIRED"
          },
          "approvers": {
            "userGroups": [
              {
                "name": "testgroup"
              }
            ]
          }
        }
      ]
    },
    "rdsName": "Country",
    "dqRulesFrsLocation": {
      "5sciIEK0QBKftMdqHqTCeT": {
        "name": "RuleSpecName",
        "project": "Default",
        "folder": "ExampleFolder"
      }
    },
    "dictionaries": {
      "entries": [
        {
          "dictionaryConfigurationId": "5fe35898fe9c74f150c6ef0d_b3c4d798_533b7e98-
acd9-4018-9835-dc47ff5d7e65",
          "details": {
            "name": "modelExportCodeListDictionary",
            "description": "modelExportCodeListDictionaryDesc"
          },
          "frsReference": {
            "name": "modelExportCodeListDictionary",
            "location": {
              "name": "reference360",
              "type": "Project",
              "path": [
                {
                  "frsName": "reference360",
                  "frsType": "Project"
                }
              ]
            }
          },
          "dataSourceConfiguration": {
            "sourceAttributeReferences": [
              {
                "attributeName": "Name"
              },
              {
                "attributeName": "Code"
              }
            ]
          }
        }
      ]
    },
    "dqValidationInfo": {
      "validationConfiguration": {
        "dqLightWeightRuleAssociations": [
          {
            "eClass": "http://informatica.com/mdm/v1/DQI#//
DQLightWeightRuleAssociation",
            "associatedFieldName": {
```

```
                  "$ref": "//@field[name='Name']"
                },
                "dqLightWeightRules": [
                  {
                    "eClass": "http://informatica.com/mdm/v1/DQI#//
DQLightWeightRuleConfiguration",
                    "description": "Minimum Length",
                    "validationMessage": "Enter a minimum of 3 characters.",
                    "downgradeTrustScorePercentage": 100,
                    "errorSeverity": "INFO",
                    "dqRuleName": "Minimum Length",
                    "dqLightWeightRuleId": "MIN_LENGTH",
                    "dqLightWeightRuleParameters": "[{\"min\":3}]",
                    "isEnabled": true,
                    "executeRuleForEmptyFields": false
                  }
                ]
              }
            ],
            "dqRuleAssociations": [
              {
                "ruleId": "5sciIEK0QBKftMdqHqTCeT",
                "statusField": "Validation",
                "statusCode": "Valid number",
                "statusMessageField": "Validation",
                "validationMessage": "",
                "errorSeverity": "ERROR",
                "associatedFieldName": {
                  "$ref": "//@field[name='Name']"
                },
                "id": "Validation_AlternateIdentifier",
                "dqRuleName": "Validation_AlternateIdentifier",
                "description": "Validate if the alternate identifier has a valid format",
                "isEnabled": true,
                "ruleInputFields": [
                  {
                    "accessPath": {
                      "eclass": "http://informatica.com/mdm/v2/Core#//AccessPath",
                      "pathElements": [
                        {
                          "$ref": "//@field[name='Name']"
                        }
                      ]
                    },
                    "ruleFieldName": "value",
                    "downgradeTrustScorePercentage": 100
                  }
                ],
                "ruleOutputFields": []
              }
            ],
            "eClass": "http://informatica.com/mdm/v1/DQI#//DQRuleValidationConfig"
          },
          "enabled": false
        },
        "assetStakeholders": {
          "stakeholders": [
            {}
          ]
        }
      },
      {
        "id": "72b0c8aee6ffc2bbbffb97b5",
        "termId": "64089cd1d1cb171040bc679c",
        "name": "IsoCountry",
        "description": "desc",
        "hierarchical": false,
        "codeValueFields": [
          {
            "name": "Name",
            "labels": [
```

```
      {
        "language": "en",
        "value": "Name"
      }
    ],
    "origin": "TERM",
    "datatype": "String",
    "mandatory": true,
    "position": 0
  },
  {
    "name": "Code",
    "labels": [
      {
        "language": "en",
        "value": "Code"
      }
    ],
    "origin": "TERM",
    "datatype": "String",
    "mandatory": true,
    "position": 0
  }
],
"assetWorkflowConfiguration": {
  "assetApiWorkFlowEnabled": false,
  "assetWorkFlowTasks": [
    {
      "taskName": "TestTaskSendback",
      "taskAction": {
        "approve": "REQUIRED",
        "reject": "REQUIRED",
        "sendBack": "REQUIRED"
      },
      "approvers": {
        "userGroups": [
          {
            "name": "testgroup2"
          }
        ]
      }
    },
    {
      "taskName": "TestTaskReject",
      "taskAction": {
        "approve": "OPTIONAL",
        "sendBack": "REQUIRED"
      },
      "approvers": {
        "userGroups": [
          {
            "name": "testgroup1"
          }
        ]
      }
    },
    {
      "taskName": "TestTakApprove",
      "taskAction": {
        "approve": "REQUIRED",
        "sendBack": "REQUIRED"
      },
      "approvers": {
        "userGroups": [
          {
            "name": "testgroup"
          }
        ]
      }
    }
  ]
```

```
      },
      "rdsName": "rds2",
      "dqValidationInfo": {
        "enabled": false
      },
      "assetStakeholders": {
        "stakeholders": [
          {}
        ]
      }
    }
  ],
  "crosswalks": [
    {
      "id": "368254d346efa33958e38575",
      "internalId": "cw_int",
      "alias": "cw_als",
      "description": "description",
      "status": "status",
      "sourceCodelistId": "72b0c8aee6ffc2bbbffb97b5",
      "targetCodelistId": "b4e6823f84eed45eda84b2c1",
      "dictionaries": {
        "entries": [
          {
            "dictionaryConfigurationId": "5fe35898fe9c74f150c6ef0d_b3c4d798-b514-4d4d-
a909-ca0b148343fd",
            "details": {
              "name": "modelExportCrosswalkDictionary",
              "description": "modelExportCodeListDictionaryDesc"
            },
            "frsReference": {
              "name": "modelExportCrosswalkDictionary",
              "location": {
                "name": "reference360",
                "type": "Project",
                "path": [
                  {
                    "frsName": "reference360",
                    "frsType": "Project"
                  }
                ]
              }
            },
            "dataSourceConfiguration": {
              "sourceAttributeReferences": [
                {
                  "attributeName": "Code",
                  "sourceAttributeAssetIdentity": {
                    "id": "72b0c8aee6ffc2bbbffb97b5",
                    "assetType": "CODELIST"
                  }
                },
                {
                  "attributeName": "Code",
                  "sourceAttributeAssetIdentity": {
                    "id": "b4e6823f84eed45eda84b2c1",
                    "assetType": "CODELIST"
                  }
                }
              ]
            }
          }
        ]
      },
      "assetStakeholders": {
        "stakeholders": [
          {}
        ]
      }
    }
  ],
```

```
"enums": {
  "application": [
    {
      "key": "CRM",
      "label": "CRM"
    }
  ]
},
"hierarchies": [
  {
    "id": "4cde04c990465a2efa8a7e8b",
    "name": "hierarchy1",
    "internalId": "hierarchy1_int",
    "alias": "hierarchy1_als",
    "description": "hierarchy1 desc",
    "codeListRelations": {
      "relations": [
        {
          "parent": {
            "codeListId": "72b0c8aee6ffc2bbbffb97b5",
            "codeListName": "IsoCountry",
            "termId": "64089cd1d1cb171040bc679c",
            "termName": "rds2"
          },
          "child": {
            "codeListId": "d36800d6fcf20b4b91a20f0e",
            "codeListName": "rds1_cl1_name",
            "codeListInternalId": "rds1_cl1_int",
            "termId": "645ab8251c45ce8e99b99daf",
            "termName": "Country"
          }
        }
      ]
    },
    "stakeholderAssignments": {
      "stakeholders": [
        {
          "subject": {
            "name": "rdmUserGroup",
            "type": "USERGROUP"
          }
        }
      ]
    },
    "assetWorkflowConfiguration": {
      "assetApiWorkFlowEnabled": false,
      "assetWorkFlowTasks": [
        {
          "taskName": "TestTaskSendback",
          "taskType": "REVIEW",
          "taskAction": {
            "approve": "REQUIRED",
            "reject": "REQUIRED",
            "sendBack": "REQUIRED"
          },
          "approvers": {
            "userGroups": [
              {
                "name": "testgroup2"
              }
            ]
          }
        },
        {
          "taskName": "TestTaskReject",
          "taskType": "REVIEW",
          "taskAction": {
            "approve": "OPTIONAL",
            "sendBack": "REQUIRED"
          },
          "approvers": {
```

```
            "userGroups": [
              {
                "name": "testgroup1"
              }
            ]
          }
        },
        {
          "taskName": "TestTakApprove",
          "taskType": "FINAL",
          "taskAction": {
            "approve": "REQUIRED",
            "sendBack": "REQUIRED"
          },
          "approvers": {
            "userGroups": [
              {
                "name": "testgroup"
              }
            ]
          }
        }
      ]
    }
  ]
}
```

Note: If the assets have references to any deleted enum entries, the export model API ignores the fields from the exported data.

Import model (v3) for adding new assets and updating existing assets

Imports new assets from a previously exported model and updates the existing assets based on internal ID or alias. The import model API runs an import model job.

The import model REST API uses the identifiers of assets in a sequence. For more information about using identifiers to import assets, see "Migrating reference data assets" on page 98.

Note: When you update an asset in the target organization, the REST API updates only the default value of the fields but does not change the field-level attributes on fields that already exist in the target organization.

POST request

To import only new assets from a previously exported model based on internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v3/model/import?isDeltaImport=true&isDeltaUpdateIgnore=true
```

To import new assets from a previously exported model and update the existing assets, submit a POST request with the following URI:

```
/rdm-service/external/v3/model/import?isDeltaImport=true&isDeltaUpdateIgnore=false
```

Note: Ensure that you use the response from the export model API as the request for the import model API. If you manually modify the exported model, the import process might fail.

The following table describes the parameter in the request:

Parameter	Description
isDeltaImport	Indicates whether to import only the new assets. Value can be `true` or `false`. Default is `true`.
isDeltaUpdateIgnore	Indicates whether to ignore updating the existing assets. Value can be `true` or `false`. Default is `false`.
isAsyncImport	Indicates whether to run an asynchronous job to import the assets. Value can be `true` or `false`. Default is `true`.

Note: The import model accepts only a JSON request body up to 5 MB in size.

The request contains the attributes that the export model API returns. For more information, see "Export model (v3)" on page 210.

The following table describes the attributes in the request:

Field	Type	Description
version	String	Version of the export file. The latest version is `4.1` to reflect the addition of multilingual reference data. Ensure that the export file version is the same as or earlier than the version in the previously imported model. Otherwise, the import fails.
referenceDataSets	-	Includes details about the reference data set.
id	String	IDs of the reference data sets. For more information, see "Asset IDs" on page 199.
name	String	Name of the asset.
description	String	Description of the asset.
hierarchical	Boolean	Indicates whether code lists in the reference data set inherit the hierarchical structure definition.
levels	Number	Optional. The number of hierarchical levels supported in the code lists associated with the reference data set. If hierarchical levels are not provided, the value is `1`. If hierarchical levels are unlimited, the value is `-1`.
internalId	String	Internal ID of the reference data set.
alias	String	Alias of the reference data set.
defaultList	String	ID of the default code list.
domain	String	Optional. Domain of the asset.
confidentiality	String	Optional. Confidentiality of the asset.
priority	String	Optional. Priority of the asset.

Field	Type	Description
status	String	Optional. Status of the asset.
codeValueFields	-	Includes the attribute definition for code values in the reference data set.
name	String	Name of the field.
origin	String	Origin of the definition of the field.
datatype	String	Datatype of the field. Values are `String`, `Integer`, `Decimal`, `Boolean`, `Date`, or `Reference Data`.
mandatory	Boolean	Indicates whether the attribute is required.
localized	Boolean	Indicates whether the attribute is multilingual.
localizationDef	Object	Includes the definition of the target languages that the code list depends on.
targetLanguages	Array	List of preferred additional languages to display the attribute.
dependencyDef	-	Includes the definition of the asset specified as the dependency.
termId	String	ID of the asset specified as the dependency.
assetStakeholders	-	Stakeholders associated with the asset.
codeLists	-	Includes details about the code list.
id	String	ID of the code lists. For more information, see "Asset IDs" on page 199.
termId	String	ID of the reference data set to which the code list is associated.
name	String	Name of the code list.
internalId	String	Internal ID of the code list.
alias	String	Alias of the code list.
description	String	Description of the code list.
hierarchical	Boolean	Indicates whether code lists in the reference data set inherit the hierarchical structure definition.
levels	Number	Number of hierarchical levels supported in the code lists associated with the reference data set. If hierarchical levels are not provided, value is `1`. If hierarchical levels are unlimited, value is `-1`.
codeValueFields	-	Includes the attribute definition for code values in the reference data set.
name	String	Name of the field.
origin	String	Origin of the definition of the field.

Field	Type	Description
datatype	String	Datatype of the field. Values are `String`, `Integer`, `Decimal`, `Boolean`, `Date`, or `Reference Data`.
mandatory	Boolean	Indicates whether the attribute is required.
dependencyDef	-	Includes the definition of the asset specified as the dependency.
termId	String	ID of the asset specified as the dependency.
displayColumns	Array	Display columns used as labels for code values associated with the dependent asset.
dqValidationInfo	-	Includes details about the basic and advanced rule associations of code list attributes.
assetStakeholders	-	Stakeholders associated to the asset.
assetWorkflowConfiguration	-	Workflow configuration of the asset.
dictionaries	-	Includes the details of dictionaries configured for the code list.
dictionaryConfigurationId	String	Identifier of the dictionary.
name	String	Name of the dictionary.
description	String	Description of the dictionary.
location	String	The project or folder within which the dictionary is located.
dataSourceConfiguration	String	Code list attributes that are added as columns in the dictionary.
crosswalks	-	Details about the crosswalk.
id	String	ID of the crosswalk. For more information, see "Asset IDs" on page 199.
internalId	String	Internal ID of the crosswalk.
alias	String	Alias of the crosswalk.
description	String	Description of the crosswalk.
status	String	Status of the crosswalk.
sourceCodelistId	String	ID of the source code list to which the crosswalk is associated.
targetCodelistId	String	ID of the target code list.
assetStakeholders	-	Stakeholders associated to the asset.
enums	-	Includes details about enum groups and entries. For more information, see "enums" on page 529.
key	String	ID of the system reference data value.

Field	Type	Description
label	String	Label for the system reference data value.
dictionaries	-	Includes the details of dictionary configured for the crosswalk.
dictionaryConfigurationId	String	Identifier of the dictionary.
name	String	Name of the dictionary.
description	String	Description of the dictionary.
location	String	The project or folder within which the dictionary is located.
dataSourceConfiguration	String	Value mappings of the crosswalk added as columns in the dictionary.
hierarchies	-	Includes details about the hierarchies.
id	String	ID of the hierarchy. For more information, see "Asset IDs" on page 199.
name	String	Name of the field.
internalId	String	Internal ID of the hierarchy.
alias	String	Alias of the hierarchy.
description	String	Description of the hierarchy.
codeListRelations	-	Code list relations defined by the hierarchies.

POST response

The response contains a detailed report about the import process.

The following table describes the attributes in the response body:

Field	Type	Description
jobId	String	The ID of the import model job.
status	String	Status of the import model job. Value can be `RUNNING`, `SUCCESS`, `FAILED`, or `CANCELLED`.
type	String	Type of the import model job.
createdBy	String	User name of the user who started the import process.
createdDate	String	Date when the user ran the import model job.

POST example

The following sample request imports only the new assets and skips the existing assets:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/model/import?
isDeltaImport=true&isDeltaUpdateIgnore=true HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

```
{
  "version": "4.1",
  "referenceDataSets": [
    {
      "id": "645ab8251c45ce8e99b99daf",
      "name": "Country",
      "internalId": "country_int",
      "alias": "country_als",
      "description": "desc",
      "hierarchical": false,
      "levels": 1,
      "defaultList": "a7d999842c11b52a53865bbe",
      "codeValueFields": [
        {
          "name": "Name",
          "labels": [
            {
              "language": "en",
              "value": "Name"
            }
          ],
          "origin": "TERM",
          "datatype": "String",
          "mandatory": true,
          "position": 0
        },
        {
          "name": "Code",
          "labels": [
            {
              "language": "en",
              "value": "Code"
            }
          ],
          "origin": "TERM",
          "datatype": "String",
          "mandatory": true,
          "position": 0,
          "localized": true
        }
      ],
      "localizationDef": {
        "targetLanguages": [
          "en-US",
          "de-DE",
          "fr-FR"
        ]
      },
      "assetStakeholders": {
        "stakeholders": [
          {}
        ]
      }
    },
    {
      "id": "64089cd1d1cb171040bc679c",
      "name": "rds2",
      "description": "desc",
      "hierarchical": false,
      "levels": 1,
      "defaultList": "b8f577b04efe19d099fde755",
      "codeValueFields": [
        {
          "name": "Name",
          "labels": [
            {
              "language": "en",
              "value": "Name"
            }
          ],
          "origin": "TERM",
```

```
          "datatype": "String",
          "mandatory": true,
          "position": 0
        },
        {
          "name": "Code",
          "labels": [
            {
              "language": "en",
              "value": "Code"
            }
          ],
          "origin": "TERM",
          "datatype": "String",
          "mandatory": true,
          "position": 0
        },
        {
          "name": "fieldDef",
          "origin": "TERM",
          "datatype": "Reference",
          "mandatory": false,
          "relatedTermId": "645ab8251c45ce8e99b99daf",
          "displayColumns": [
            "name"
          ],
          "position": 0
        }
      ],
      "dependencyDef": {
        "termId": "645ab8251c45ce8e99b99daf",
        "displayColumns": [
          "name"
        ]
      },
      "assetStakeholders": {
        "stakeholders": [
          {}
        ]
      }
    }
  ],
  "codeLists": [
    {
      "id": "b4e6823f84eed45eda84b2c1",
      "termId": "64089cd1d1cb171040bc679c",
      "name": "SapCountry",
      "description": "desc",
      "hierarchical": false,
      "codeValueFields": [
        {
          "name": "Name",
          "labels": [
            {
              "language": "en",
              "value": "Name"
            }
          ],
          "origin": "TERM",
          "datatype": "String",
          "mandatory": true,
          "position": 0
        },
        {
          "name": "Code",
          "labels": [
            {
              "language": "en",
              "value": "Code"
            }
          ],
```

```
        "origin": "TERM",
        "datatype": "String",
        "mandatory": true,
        "position": 0
      }
    ],
    "assetWorkflowConfiguration": {
      "assetApiWorkFlowEnabled": false,
      "assetWorkFlowTasks": [
        {
          "taskName": "TestTaskSendback",
          "taskAction": {
            "approve": "REQUIRED",
            "reject": "REQUIRED",
            "sendBack": "REQUIRED"
          },
          "approvers": {
            "userGroups": [
              {
                "name": "testgroup2"
              }
            ]
          }
        },
        {
          "taskName": "TestTaskReject",
          "taskAction": {
            "approve": "OPTIONAL",
            "sendBack": "REQUIRED"
          },
          "approvers": {
            "userGroups": [
              {
                "name": "testgroup1"
              }
            ]
          }
        },
        {
          "taskName": "TestTakApprove",
          "taskAction": {
            "approve": "REQUIRED",
            "sendBack": "REQUIRED"
          },
          "approvers": {
            "userGroups": [
              {
                "name": "testgroup"
              }
            ]
          }
        }
      ]
    },
    "rdsName": "rds2",
    "dqValidationInfo": {
      "enabled": false
    },
    "assetStakeholders": {
      "stakeholders": [
        {}
      ]
    }
  },
  {
    "id": "d36800d6fcf20b4b91a20f0e",
    "termId": "645ab8251c45ce8e99b99daf",
    "name": "rds1_cl1_name",
    "internalId": "rds1_cl1_int",
    "alias": "rds1_cl1_als",
    "description": "desc",
```

```
"hierarchical": false,
"codeValueFields": [
  {
    "name": "Name",
    "labels": [
      {
        "language": "en",
        "value": "Name"
      }
    ],
    "origin": "TERM",
    "datatype": "String",
    "mandatory": true,
    "position": 0
  },
  {
    "name": "Code",
    "labels": [
      {
        "language": "en",
        "value": "Code"
      }
    ],
    "origin": "TERM",
    "datatype": "String",
    "mandatory": true,
    "position": 0
  }
],
"businessIdDef": {
  "type": "ALPHANUMERIC",
  "prefix": "RDM",
  "length": 10,
  "label": "modelExportBusinessId"
},
"assetWorkflowConfiguration": {
  "assetApiWorkFlowEnabled": false,
  "assetWorkFlowTasks": [
    {
      "taskName": "TestTaskSendback",
      "taskAction": {
        "approve": "REQUIRED",
        "reject": "REQUIRED",
        "sendBack": "REQUIRED"
      },
      "approvers": {
        "userGroups": [
          {
            "name": "testgroup2"
          }
        ]
      }
    },
    {
      "taskName": "TestTaskReject",
      "taskAction": {
        "approve": "OPTIONAL",
        "sendBack": "REQUIRED"
      },
      "approvers": {
        "userGroups": [
          {
            "name": "testgroup1"
          }
        ]
      }
    },
    {
      "taskName": "TestTakApprove",
      "taskAction": {
        "approve": "REQUIRED",
```

```
            "sendBack": "REQUIRED"
          },
          "approvers": {
            "userGroups": [
              {
                "name": "testgroup"
              }
            ]
          }
        }
      ]
    },
    "rdsName": "Country",
    "dqRulesFrsLocation": {
      "5sciIEK0QBKftMdqHqTCeT": {
        "name": "RuleSpecName",
        "project": "Default",
        "folder": "ExampleFolder"
      }
    },
    "dictionaries": {
      "entries": [
        {
          "dictionaryConfigurationId": "5fe35898fe9c74f150c6ef0d_b3c4d798_533b7e98-
acd9-4018-9835-dc47ff5d7e65",
          "details": {
            "name": "modelExportCodeListDictionary",
            "description": "modelExportCodeListDictionaryDesc"
          },
          "frsReference": {
            "name": "modelExportCodeListDictionary",
            "location": {
              "name": "reference360",
              "type": "Project",
              "path": [
                {
                  "frsName": "reference360",
                  "frsType": "Project"
                }
              ]
            }
          },
          "dataSourceConfiguration": {
            "sourceAttributeReferences": [
              {
                "attributeName": "Name"
              },
              {
                "attributeName": "Code"
              }
            ]
          }
        }
      ]
    },
    "dqValidationInfo": {
      "validationConfiguration": {
        "dqLightWeightRuleAssociations": [
          {
            "eClass": "http://informatica.com/mdm/v1/DQI#//
DQLightWeightRuleAssociation",
            "associatedFieldName": {
              "$ref": "//@field[name='Name']"
            },
            "dqLightWeightRules": [
              {
                "eClass": "http://informatica.com/mdm/v1/DQI#//
DQLightWeightRuleConfiguration",
                "description": "Minimum Length",
                "validationMessage": "Enter a minimum of 3 characters.",
                "downgradeTrustScorePercentage": 100,
```

```
                "errorSeverity": "INFO",
                "dqRuleName": "Minimum Length",
                "dqLightWeightRuleId": "MIN_LENGTH",
                "dqLightWeightRuleParameters": "[{\"min\":3}]",
                "isEnabled": true,
                "executeRuleForEmptyFields": false
              }
            ]
          }
        ],
        "dqRuleAssociations": [
          {
            "ruleId": "5sciIEK0QBKftMdqHqTCeT",
            "statusField": "Validation",
            "statusCode": "Valid number",
            "statusMessageField": "Validation",
            "validationMessage": "",
            "errorSeverity": "ERROR",
            "associatedFieldName": {
              "$ref": "//@field[name='Name']"
            },
            "id": "Validation_AlternateIdentifier",
            "dqRuleName": "Validation_AlternateIdentifier",
            "description": "Validate if the alternate identifier has a valid format",
            "isEnabled": true,
            "ruleInputFields": [
              {
                "accessPath": {
                  "eclass": "http://informatica.com/mdm/v2/Core#//AccessPath",
                  "pathElements": [
                    {
                      "$ref": "//@field[name='Name']"
                    }
                  ]
                },
                "ruleFieldName": "value",
                "downgradeTrustScorePercentage": 100
              }
            ],
            "ruleOutputFields": []
          }
        ],
        "eClass": "http://informatica.com/mdm/v1/DQI#//DQRuleValidationConfig"
      },
      "enabled": false
    },
    "assetStakeholders": {
      "stakeholders": [
        {}
      ]
    }
  },
  {
    "id": "72b0c8aee6ffc2bbbffb97b5",
    "termId": "64089cd1d1cb171040bc679c",
    "name": "IsoCountry",
    "description": "desc",
    "hierarchical": false,
    "codeValueFields": [
      {
        "name": "Name",
        "labels": [
          {
            "language": "en",
            "value": "Name"
          }
        ],
        "origin": "TERM",
        "datatype": "String",
        "mandatory": true,
        "position": 0
```

```
  },
  {
    "name": "Code",
    "labels": [
      {
        "language": "en",
        "value": "Code"
      }
    ],
    "origin": "TERM",
    "datatype": "String",
    "mandatory": true,
    "position": 0
  }
],
"assetWorkflowConfiguration": {
  "assetApiWorkFlowEnabled": false,
  "assetWorkFlowTasks": [
    {
      "taskName": "TestTaskSendback",
      "taskAction": {
        "approve": "REQUIRED",
        "reject": "REQUIRED",
        "sendBack": "REQUIRED"
      },
      "approvers": {
        "userGroups": [
          {
            "name": "testgroup2"
          }
        ]
      }
    },
    {
      "taskName": "TestTaskReject",
      "taskAction": {
        "approve": "OPTIONAL",
        "sendBack": "REQUIRED"
      },
      "approvers": {
        "userGroups": [
          {
            "name": "testgroup1"
          }
        ]
      }
    },
    {
      "taskName": "TestTakApprove",
      "taskAction": {
        "approve": "REQUIRED",
        "sendBack": "REQUIRED"
      },
      "approvers": {
        "userGroups": [
          {
            "name": "testgroup"
          }
        ]
      }
    }
  ]
},
"rdsName": "rds2",
"dqValidationInfo": {
  "enabled": false
},
"assetStakeholders": {
  "stakeholders": [
    {}
  ]
```

```
      }
    }
  ],
  "crosswalks": [
    {
      "id": "368254d346efa33958e38575",
      "internalId": "cw_int",
      "alias": "cw_als",
      "description": "description",
      "status": "status",
      "sourceCodelistId": "72b0c8aee6ffc2bbbffb97b5",
      "targetCodelistId": "b4e6823f84eed45eda84b2c1",
      "dictionaries": {
        "entries": [
          {
            "dictionaryConfigurationId": "5fe35898fe9c74f150c6ef0d_b3c4d798-b514-4d4d-
a909-ca0b148343fd",
            "details": {
              "name": "modelExportCrosswalkDictionary",
              "description": "modelExportCodeListDictionaryDesc"
            },
            "frsReference": {
              "name": "modelExportCrosswalkDictionary",
              "location": {
                "name": "reference360",
                "type": "Project",
                "path": [
                  {
                    "frsName": "reference360",
                    "frsType": "Project"
                  }
                ]
              }
            },
            "dataSourceConfiguration": {
              "sourceAttributeReferences": [
                {
                  "attributeName": "Code",
                  "sourceAttributeAssetIdentity": {
                    "id": "72b0c8aee6ffc2bbbffb97b5",
                    "assetType": "CODELIST"
                  }
                },
                {
                  "attributeName": "Code",
                  "sourceAttributeAssetIdentity": {
                    "id": "b4e6823f84eed45eda84b2c1",
                    "assetType": "CODELIST"
                  }
                }
              ]
            }
          }
        ]
      },
      "assetStakeholders": {
        "stakeholders": [
          {}
        ]
      }
    }
  ],
  "enums": {
    "application": [
      {
        "key": "CRM",
        "label": "CRM"
      }
    ]
  },
  "hierarchies": [
```

```
{
  "id": "4cde04c990465a2efa8a7e8b",
  "name": "hierarchy1",
  "internalId": "hierarchy1_int",
  "alias": "hierarchy1_als",
  "description": "hierarchy1 desc",
  "codeListRelations": {
    "relations": [
      {
        "parent": {
          "codeListId": "72b0c8aee6ffc2bbbffb97b5",
          "codeListName": "IsoCountry",
          "termId": "64089cd1d1cb171040bc679c",
          "termName": "rds2"
        },
        "child": {
          "codeListId": "d36800d6fcf20b4b91a20f0e",
          "codeListName": "rds1_cl1_name",
          "codeListInternalId": "rds1_cl1_int",
          "termId": "645ab8251c45ce8e99b99daf",
          "termName": "Country"
        }
      }
    ]
  },
  "stakeholderAssignments": {
    "stakeholders": [
      {
        "subject": {
          "name": "rdmUserGroup",
          "type": "USERGROUP"
        }
      }
    ]
  },
  "assetWorkflowConfiguration": {
    "assetApiWorkFlowEnabled": false,
    "assetWorkFlowTasks": [
      {
        "taskName": "TestTaskSendback",
        "taskType": "REVIEW",
        "taskAction": {
          "approve": "REQUIRED",
          "reject": "REQUIRED",
          "sendBack": "REQUIRED"
        },
        "approvers": {
          "userGroups": [
            {
              "name": "testgroup2"
            }
          ]
        }
      },
      {
        "taskName": "TestTaskReject",
        "taskType": "REVIEW",
        "taskAction": {
          "approve": "OPTIONAL",
          "sendBack": "REQUIRED"
        },
        "approvers": {
          "userGroups": [
            {
              "name": "testgroup1"
            }
          ]
        }
      },
      {
        "taskName": "TestTakApprove",
```

```
            "taskType": "FINAL",
            "taskAction": {
              "approve": "REQUIRED",
              "sendBack": "REQUIRED"
            },
            "approvers": {
              "userGroups": [
                {
                  "name": "testgroup"
                }
              ]
            }
          }
        ]
      }
    }
  ]
}
```

Note: To arrange an attribute in a specific order, place the attribute and its parameters in the desired location within the attributes list.

The following sample response shows the job ID and status of the import process:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 3436

{
   "jobId":"5247b0161e5a1c4fab80b746",
   "type":"MODEL_IMPORT",
   "status":"RUNNING",
   "createdBy":"userId",
   "createdDate":"2023-01-25T09:54:09.213+00:00"
}
```

Import model (v3) for adding new assets

Imports the new assets from a previously exported model based on internal ID or alias only if the assets are not available in the target organization. By default, the import model API runs an asynchronous import model job.

POST request

To import the new assets from a previously exported model based on internal ID or alias only if the assets are not available in the target organization, submit a POST request with the following URI:

```
/rdm-service/external/v3/model/import?isDeltaImport=false
```

Note: If any of the assets in the exported model matches with the assets in the target organization, the import fails.

The import model REST API uses the identifiers of assets in a sequence. For more information about using identifiers to import assets, see “Migrating reference data assets” on page 98.

The following table describes the parameter in the request:

Parameter	Description
isDeltaImport	Indicates whether to import only the new assets. Value can be `true` or `false`. Default is `true`.
isDeltaUpdateIgnore	Indicates whether to ignore updating the existing assets. Value can be `true` or `false`. Default is `false`.
isAsyncImport	Indicates whether to run an asynchronous job to import the assets. Value can be `true` or `false`. Default is `true`.

The request contains the attributes that the export model API returns. For more information, see "Export model (v3)" on page 210.

The following table describes the attributes in the request:

Field	Type	Description
version	String	Version of the export file. The latest version is `4.0` that supports Data Quality dictionaries. Ensure that the export file version is the same as or earlier than the version in the previously imported model. Otherwise, the import fails.
referenceDataSets	-	Includes details about the reference data set.
id	String	ID of the reference data sets. For more information, see "Asset IDs" on page 199.
name	String	Name of the asset.
description	String	Description of the asset.
hierarchical	Boolean	Indicates whether code lists in the reference data set inherit the hierarchical structure definition.
levels	Number	Optional. Number of hierarchical levels supported in the code lists associated with the reference data set. If hierarchical levels are not provided, value is `1`. If hierarchical levels are unlimited, value is `-1`.
internalId	String	Internal ID of the reference data set.
alias	String	Alias of the reference data set.
defaultList	String	ID of the default code list.
domain	String	Optional. Domain of the asset.
confidentiality	String	Optional. Confidentiality of the asset.
priority	String	Optional. Priority of the asset.
status	String	Optional. Status of the asset.

Field	Type	Description
codeValueFields	-	Includes the attribute definition for code values in the reference data set.
name	String	Name of the field.
origin	String	Origin of the definition of the field.
datatype	String	Datatype of the field. Values are `String`, `Integer`, `Decimal`, `Boolean`, `Date`, or `Reference Data`.
mandatory	Boolean	Indicates whether the attribute is required.
dependencyDef	-	Includes the definition of the asset specified as the dependency.
termId	String	ID of the asset specified as the dependency.
assetStakeholders	-	Stakeholders associated to the asset.
codeLists	-	Includes details about the code list.
id	String	ID of the code lists. For more information, see "Asset IDs" on page 199.
termId	String	ID of the reference data set to which the code list is associated.
name	String	Name of the asset.
internalId	String	Internal ID of the code list.
alias	String	Alias of the code list.
description	String	Description of the asset.
hierarchical	Boolean	Indicates whether code lists in the reference data set inherit the hierarchical structure definition.
levels	Number	Number of hierarchical levels supported in the code lists associated with the reference data set. If hierarchical levels are not provided, value is `1`. If hierarchical levels are unlimited, value is `-1`.
codeValueFields	-	Includes the attribute definition for code values in the reference data set.
name	String	Name of the field.
origin	String	Origin of the definition of the field.
datatype	String	Datatype of the field. Values are `String`, `Integer`, `Decimal`, `Boolean`, `Date`, or `Reference Data`.
mandatory	Boolean	Indicates whether the attribute is required.
businessIdDef	-	Optional. Includes the details of the business IDs configured for the code values.

Field	Type	Description
type	String	Format in which the business IDs are generated. Values are `NUMERIC` or `ALPHANUMERIC`.
prefix	String	Characters added to the beginning of business IDs. Values are `NUMERIC` or `ALPHANUMERIC`.
length	Number	Total length of the business IDs.
offset	String	Starting value for the business IDs that Reference 360 generates. Applicable for numeric business IDs.
label	String	Display name for the business ID field.
dependencyDef	-	Includes the definition of the asset specified as the dependency.
termId	String	ID of the asset specified as the dependency.
displayColumns	Array	Display columns used as labels for code values associated with the dependent asset.
dqValidationInfo	-	Includes details about the basic and advanced rule associations of code list attributes.
assetStakeholders	-	Stakeholders associated with the asset.
assetWorkflowConfiguration	-	Workflow configuration of the asset.
dictionaries	-	Includes the details of dictionaries configured for the code list.
dictionaryConfigurationId	String	Identifier of the dictionary.
name	String	Name of the dictionary.
description	String	Description of the dictionary.
location	String	The project or folder within which the dictionary is located.
dataSourceConfiguration	String	Code list attributes that are added as columns in the dictionary.
crosswalks	-	Details about the crosswalk.
id	String	ID of the crosswalks. For more information, see "Asset IDs" on page 199.
internalId	String	Internal ID of the crosswalk.
alias	String	Alias of the crosswalk.
description	String	Description of the asset.
status	String	Status of the asset.
sourceCodelistId	String	ID of the source code list to which the crosswalk is associated.

Field	Type	Description
targetCodelistId	String	ID of the target code list.
assetStakeholders	-	Stakeholders associated to the asset.
enums	-	Includes details about enum groups and entries. For more information, see "enums" on page 529.
key	String	ID of the system reference data value.
label	String	Label for the system reference data value.
dictionaries	-	Includes the details of dictionary configured for the crosswalk.
dictionaryConfigurationId	String	Identifier of the dictionary.
name	String	Name of the dictionary.
description	String	Description of the dictionary.
location	String	The project or folder within which the dictionary is located.
dataSourceConfiguration	String	Value mappings of the crosswalk added as columns in the dictionary.
hierarchies	-	Includes details about the hierarchies.
id	String	ID of the hierarchy. For more information, see "Asset IDs" on page 199.
name	String	Name of the field.
internalId	String	Internal ID of the hierarchy.
alias	String	Alias of the hierarchy.
description	String	Description of the hierarchy.
codeListRelations	-	Code list relations defined by the hierarchies.

POST response

The response contains a detailed report about the import process.

The following table describes the attributes in the response:

Field	Type	Description
jobId	String	ID of the import model job.
status	String	Status of the import model job. Value can be `RUNNING`, `SUCCESS`, `FAILED`, or `CANCELLED`.
type	String	Type of the import model job.

Field	Type	Description
createdBy	String	User name of the user who started the import process.
createdDate	String	Date when the user ran the import model job.

POST example

The following sample request imports the existing assets with incremental changes:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/model/import?
isDeltaImport=false&isDeltaUpdateIgnore=false HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX


{
   "version":"4.1",
   "referenceDataSets":[
      {
         "id":"645ab8251c45ce8e99b99daf",
         "name":"Country",
         "internalId":"country_int",
         "alias":"country_als",
         "description":"desc",
         "hierarchical":false,
         "levels":1,
         "defaultList":"a7d999842c11b52a53865bbe",
         "codeValueFields":[
            {
               "name":"Name",
               "labels":[
                  {
                     "language":"en",
                     "value":"Name"
                  }
               ],
               "origin":"TERM",
               "datatype":"String",
               "mandatory":true,
               "position":0,
               "localized": true
            },
            {
               "name":"Code",
               "labels":[
                  {
                     "language":"en",
                     "value":"Code"
                  }
               ],
               "origin":"TERM",
               "datatype":"String",
               "mandatory":true,
               "position":0
            }
         ],
         "assetStakeholders":{
            "stakeholders":[
               {

               }
            ]
         }
```

```
    },
    {
       "id":"64089cd1d1cb171040bc679c",
       "name":"rds2",
       "description":"desc",
       "hierarchical":false,
       "levels":1,
       "defaultList":"b8f577b04efe19d099fde755",
       "codeValueFields":[
          {
             "name":"Name",
             "labels":[
                {
                   "language":"en",
                   "value":"Name"
                }
             ],
             "origin":"TERM",
             "datatype":"String",
             "mandatory":true,
             "position":0,
             "localized": true
          },
          {
             "name":"Code",
             "labels":[
                {
                   "language":"en",
                   "value":"Code"
                }
             ],
             "origin":"TERM",
             "datatype":"String",
             "mandatory":true,
             "position":0,
             "localized": false
          },
          {
             "name":"fieldDef",
             "origin":"TERM",
             "datatype":"Reference",
             "mandatory":false,
             "relatedTermId":"645ab8251c45ce8e99b99daf",
             "displayColumns":[
                "name"
             ],
             "position":0
          }
       ],
       "localizationDef": {},
       "platformConfig": {},
       "isFrsAssociated": false,
       "lastUpdateDate": "1790230361414",
       "creationDate": "1790230356255",
       "dependencyDef":{
          "termId":"645ab8251c45ce8e99b99daf",
          "displayColumns":[
             "name"
          ]
       },
       "assetStakeholders":{
          "stakeholders":[
             {

             }
          ]
       }
    }
 ],
 "codeLists":[
    {
```

```
"id":"b4e6823f84eed45eda84b2c1",
"termId":"64089cd1d1cb171040bc679c",
"name":"SapCountry",
"description":"desc",
"hierarchical":false,
"codeValueFields":[
   {
      "name":"Name",
      "labels":[
         {
            "language":"en",
            "value":"Name"
         }
      ],
      "origin":"TERM",
      "datatype":"String",
      "mandatory":true,
      "position":0,
      "localized": true
   },
   {
      "name":"Code",
      "labels":[
         {
            "language":"en",
            "value":"Code"
         }
      ],
      "origin":"TERM",
      "datatype":"String",
      "mandatory":true,
      "position":0
   }
],
"localizationDef": {
       "targetLanguages": [
          "ja-JP"
       ]
},
"assetWorkflowConfiguration":{
   "assetApiWorkFlowEnabled":false,
   "assetWorkFlowTasks":[
      {
         "taskName":"TestTaskSendback",
         "taskAction":{
            "approve":"REQUIRED",
            "reject":"REQUIRED",
            "sendBack":"REQUIRED"
         },
         "approvers":{
            "userGroups":[
               {
                  "name":"testgroup2"
               }
            ]
         }
      },
      {
         "taskName":"TestTaskReject",
         "taskAction":{
            "approve":"OPTIONAL",
            "sendBack":"REQUIRED"
         },
         "approvers":{
            "userGroups":[
               {
                  "name":"testgroup1"
               }
            ]
         }
      },
```

```
            {
               "taskName":"TestTakApprove",
               "taskAction":{
                  "approve":"REQUIRED",
                  "sendBack":"REQUIRED"
               },
               "approvers":{
                  "userGroups":[
                     {
                        "name":"testgroup"
                     }
                  ]
               }
            }
         ]
      },
      "rdsName":"rds2",
      "dqValidationInfo":{
         "enabled":false
      },
      "assetStakeholders":{
         "stakeholders":[
            {

            }
         ]
      }
   },
   {
      "id":"d36800d6fcf20b4b91a20f0e",
      "termId":"645ab8251c45ce8e99b99daf",
      "name":"rds1_cl1_name",
      "internalId":"rds1_cl1_int",
      "alias":"rds1_cl1_als",
      "description":"desc",
      "hierarchical":false,
      "codeValueFields":[
         {
            "name":"Name",
            "labels":[
               {
                  "language":"en",
                  "value":"Name"
               }
            ],
            "origin":"TERM",
            "datatype":"String",
            "mandatory":true,
            "position":0
         },
         {
            "name":"Code",
            "labels":[
               {
                  "language":"en",
                  "value":"Code"
               }
            ],
            "origin":"TERM",
            "datatype":"String",
            "mandatory":true,
            "position":0
         }
      ],
      "businessIdDef":{
         "type":"ALPHANUMERIC",
         "prefix":"RDM",
         "length":10,
         "label":"modelExportBusinessId"
      },
      "assetWorkflowConfiguration":{
```

```
            "assetApiWorkFlowEnabled":false,
            "assetWorkFlowTasks":[
               {
                  "taskName":"TestTaskSendback",
                  "taskAction":{
                     "approve":"REQUIRED",
                     "reject":"REQUIRED",
                     "sendBack":"REQUIRED"
                  },
                  "approvers":{
                     "userGroups":[
                        {
                           "name":"testgroup2"
                        }
                     ]
                  }
               },
               {
                  "taskName":"TestTaskReject",
                  "taskAction":{
                     "approve":"OPTIONAL",
                     "sendBack":"REQUIRED"
                  },
                  "approvers":{
                     "userGroups":[
                        {
                           "name":"testgroup1"
                        }
                     ]
                  }
               },
               {
                  "taskName":"TestTakApprove",
                  "taskAction":{
                     "approve":"REQUIRED",
                     "sendBack":"REQUIRED"
                  },
                  "approvers":{
                     "userGroups":[
                        {
                           "name":"testgroup"
                        }
                     ]
                  }
               }
            ]
         },
         "rdsName":"Country",
         "dqRulesFrsLocation":{
            "5sciIEK0QBKftMdqHqTCeT":{
               "name":"RuleSpecName",
               "project":"Default",
               "folder":"ExampleFolder"
            }
         },
         "dictionaries":{
            "entries":[
               {

"dictionaryConfigurationId":"5fe35898fe9c74f150c6ef0d_b3c4d798_533b7e98-acd9-4018-9835-
dc47ff5d7e65",
                  "details":{
                     "name":"modelExportCodeListDictionary",
                     "description":"modelExportCodeListDictionaryDesc"
                  },
                  "frsReference":{
                     "name":"modelExportCodeListDictionary",
                     "location":{
                        "name":"reference360",
                        "type":"Project",
                        "path":[
```

```
                              {
                                 "frsName":"reference360",
                                 "frsType":"Project"
                              }
                           ]
                        }
                     },
                     "dataSourceConfiguration":{
                        "sourceAttributeReferences":[
                           {
                              "attributeName":"Name"
                           },
                           {
                              "attributeName":"Code"
                           }
                        ]
                     }
                  }
               ]
            },
            "dqValidationInfo":{
               "validationConfiguration":{
                  "dqLightWeightRuleAssociations":[
                     {
                        "eClass":"http://informatica.com/mdm/v1/DQI#//
DQLightWeightRuleAssociation",
                        "associatedFieldName":{
                           "$ref":"//@field[name='Name']"
                        },
                        "dqLightWeightRules":[
                           {
                              "eClass":"http://informatica.com/mdm/v1/DQI#//
DQLightWeightRuleConfiguration",
                              "description":"Minimum Length",
                              "validationMessage":"Enter a minimum of 3 characters.",
                              "downgradeTrustScorePercentage":100,
                              "errorSeverity":"INFO",
                              "dqRuleName":"Minimum Length",
                              "dqLightWeightRuleId":"MIN_LENGTH",
                              "dqLightWeightRuleParameters":"[{\"min\":3}]",
                              "isEnabled":true,
                              "executeRuleForEmptyFields":false
                           }
                        ]
                     }
                  ],
                  "dqRuleAssociations":[
                     {
                        "ruleId":"5sciIEK0QBKftMdqHqTCeT",
                        "statusField":"Validation",
                        "statusCode":"Valid number",
                        "statusMessageField":"Validation",
                        "validationMessage":"",
                        "errorSeverity":"ERROR",
                        "associatedFieldName":{
                           "$ref":"//@field[name='Name']"
                        },
                        "id":"Validation_AlternateIdentifier",
                        "dqRuleName":"Validation_AlternateIdentifier",
                        "description":"Validate if the alternate identifier has a valid
format",
                        "isEnabled":true,
                        "ruleInputFields":[
                           {
                              "accessPath":{
                                 "eclass":"http://informatica.com/mdm/v2/Core#//AccessPath",
                                 "pathElements":[
                                    {
                                       "$ref":"//@field[name='Name']"
                                    }
                                 ]
```

```
                    },
                    "ruleFieldName":"value",
                    "downgradeTrustScorePercentage":100
                 }
              ],
              "ruleOutputFields":[

              ]
           }
        ],
        "eClass":"http://informatica.com/mdm/v1/DQI#//DQRuleValidationConfig"
     },
     "enabled":false
  },
  "assetStakeholders":{
     "stakeholders":[
        {

        }
     ]
  }
},
{
  "id":"72b0c8aee6ffc2bbbffb97b5",
  "termId":"64089cd1d1cb171040bc679c",
  "name":"IsoCountry",
  "description":"desc",
  "hierarchical":false,
  "codeValueFields":[
     {
        "name":"Name",
        "labels":[
           {
              "language":"en",
              "value":"Name"
           }
        ],
        "origin":"TERM",
        "datatype":"String",
        "mandatory":true,
        "position":0
     },
     {
        "name":"Code",
        "labels":[
           {
              "language":"en",
              "value":"Code"
           }
        ],
        "origin":"TERM",
        "datatype":"String",
        "mandatory":true,
        "position":0
     }
  ],
  "assetWorkflowConfiguration":{
     "assetApiWorkFlowEnabled":false,
     "assetWorkFlowTasks":[
        {
           "taskName":"TestTaskSendback",
           "taskAction":{
              "approve":"REQUIRED",
              "reject":"REQUIRED",
              "sendBack":"REQUIRED"
           },
           "approvers":{
              "userGroups":[
                 {
                    "name":"testgroup2"
                 }
```

```
                        ]
                     }
                  },
                  {
                     "taskName":"TestTaskReject",
                     "taskAction":{
                        "approve":"OPTIONAL",
                        "sendBack":"REQUIRED"
                     },
                     "approvers":{
                        "userGroups":[
                           {
                              "name":"testgroup1"
                           }
                        ]
                     }
                  },
                  {
                     "taskName":"TestTakApprove",
                     "taskAction":{
                        "approve":"REQUIRED",
                        "sendBack":"REQUIRED"
                     },
                     "approvers":{
                        "userGroups":[
                           {
                              "name":"testgroup"
                           }
                        ]
                     }
                  }
               ]
            },
            "rdsName":"rds2",
            "dqValidationInfo":{
               "enabled":false
            },
            "assetStakeholders":{
               "stakeholders":[
                  {

                  }
               ]
            }
         }
      ],
      "crosswalks":[
         {
            "id":"368254d346efa33958e38575",
            "internalId":"cw_int",
            "alias":"cw_als",
            "description":"description",
            "status":"status",
            "sourceCodelistId":"72b0c8aee6ffc2bbbffb97b5",
            "targetCodelistId":"b4e6823f84eed45eda84b2c1",
            "dictionaries":{
               "entries":[
                  {
                     "dictionaryConfigurationId":"5fe35898fe9c74f150c6ef0d_b3c4d798-
b514-4d4d-a909-ca0b148343fd",
                     "details":{
                        "name":"modelExportCrosswalkDictionary",
                        "description":"modelExportCodeListDictionaryDesc"
                     },
                     "frsReference":{
                        "name":"modelExportCrosswalkDictionary",
                        "location":{
                           "name":"reference360",
                           "type":"Project",
                           "path":[
                              {
```

```
                        "frsName":"reference360",
                        "frsType":"Project"
                     }
                  ]
               }
            },
            "dataSourceConfiguration":{
               "sourceAttributeReferences":[
                  {
                     "attributeName":"Code",
                     "sourceAttributeAssetIdentity":{
                        "id":"72b0c8aee6ffc2bbbffb97b5",
                        "assetType":"CODELIST"
                     }
                  },
                  {
                     "attributeName":"Code",
                     "sourceAttributeAssetIdentity":{
                        "id":"b4e6823f84eed45eda84b2c1",
                        "assetType":"CODELIST"
                     }
                  }
               ]
            }
         }
      ]
   },
   "assetStakeholders":{
      "stakeholders":[
         {

         }
      ]
   }
}
],
"enums":{
   "application":[
      {
         "key":"CRM",
         "label":"CRM"
      }
   ]
},
"hierarchies":[
   {
      "id":"4cde04c990465a2efa8a7e8b",
      "name":"hierarchy1",
      "internalId":"hierarchy1_int",
      "alias":"hierarchy1_als",
      "description":"hierarchy1 desc",
      "codeListRelations":{
         "relations":[
            {
               "parent":{
                  "codeListId":"72b0c8aee6ffc2bbbffb97b5",
                  "codeListName":"IsoCountry",
                  "termId":"64089cd1d1cb171040bc679c",
                  "termName":"rds2"
               },
               "child":{
                  "codeListId":"d36800d6fcf20b4b91a20f0e",
                  "codeListName":"rds1_cl1_name",
                  "codeListInternalId":"rds1_cl1_int",
                  "termId":"645ab8251c45ce8e99b99daf",
                  "termName":"Country"
               }
            }
         ]
      },
      "stakeholderAssignments":{
```

```
            "stakeholders":[
               {
                  "subject":{
                     "name":"rdmUserGroup",
                     "type":"USERGROUP"
                  }
               }
            ]
         },
         "assetWorkflowConfiguration":{
            "assetApiWorkFlowEnabled":false,
            "assetWorkFlowTasks":[
               {
                  "taskName":"TestTaskSendback",
                  "taskType":"REVIEW",
                  "taskAction":{
                     "approve":"REQUIRED",
                     "reject":"REQUIRED",
                     "sendBack":"REQUIRED"
                  },
                  "approvers":{
                     "userGroups":[
                        {
                           "name":"testgroup2"
                        }
                     ]
                  }
               },
               {
                  "taskName":"TestTaskReject",
                  "taskType":"REVIEW",
                  "taskAction":{
                     "approve":"OPTIONAL",
                     "sendBack":"REQUIRED"
                  },
                  "approvers":{
                     "userGroups":[
                        {
                           "name":"testgroup1"
                        }
                     ]
                  }
               },
               {
                  "taskName":"TestTakApprove",
                  "taskType":"FINAL",
                  "taskAction":{
                     "approve":"REQUIRED",
                     "sendBack":"REQUIRED"
                  },
                  "approvers":{
                     "userGroups":[
                        {
                           "name":"testgroup"
                        }
                     ]
                  }
               }
            ]
         }
      }
   ]
}
```

Note: To arrange an attribute in a specific order, place the attribute and its parameters in the desired location within the attributes list.

The following sample response shows the job ID and status of the import process:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 3436

{
   "jobId":"5247b0161e5a1c4fab80b746",
   "type":"MODEL_IMPORT",
   "status":"RUNNING",
   "createdBy":"userId",
   "createdDate":"2023-01-25T09:54:09.213+00:00"
}
```

Get details of an import model job (v3)

Retrieves details of an import model job.

GET request

To get the details of an import model job, submit a GET request with the following URI:

```
/rdm-service/external/v3/model/import/job/f27f4850bac7ee1fd6f9fa7a
```

GET response

The response contains the details of the import model job and the objects that were ignored during import.

The following table describes the attributes in the response:

Field	Type	Description
jobId	String	ID of the import model job.
status	String	Status of the import model job. Value can be `RUNNING`, `SUCCESS`, `FAILED`, or `CANCELLED`.
createdBy	String	Username of the user who triggered the import model job.
createdDate	String	Date when the user ran the import model job.
detail	-	Includes details about the import model job.
modelImportReport	-	Report for the import model job.
numImportedRds	Number	Number of imported reference data sets.
numImportedCodeList	Number	Number of imported code lists.
numImportedCrosswalk	Number	Number of imported crosswalks.
numImportedHierarchy	Number	Number of imported hierarchies.
ignoredRdsCount	Number	Number of ignored reference data sets.
ignoredCodeListCount	Number	Number of ignored code lists.
ignoredCrosswalkCount	Number	Number of ignored crosswalks.
ignoredHierarchyCount	Number	Number of ignored hierarchies.

Field	Type	Description
importedEntities	-	Includes details about the imported entities.
type	String	Type of imported entity. Value can be `rds`, `codelist`, `crosswalk`, or `hierarchy`.
name	String	Name of the imported entity.
oldId	String	ID of the imported entity in the source organization.
newId	String	ID of the imported entity in the target organization.
ignoredEntities	-	Includes details about the ignored entities.
type	String	Type of ignored entity. Value can be `rds`, `codelist`, `crosswalk`, or `hierarchy`.
name	String	Name of the ignored entity.
oldId	String	ID of the ignored entity in the source organization.
newId	String	ID of the ignored entity in the target organization.
enumImportResult	-	Includes details about the enum entries.
newEntries	Number	Number of new entries imported.
updatedEntries	Number	Number of updated entries.
existingEntries	Number	Number of existing entries that were skipped.
stakeholderImportResult	-	Includes details about the stakeholder import result.
ignoredStakeholderEntries	-	Includes details about the ignored stakeholder entries with reason.
reason	String	The reason for ignoring stakeholder during import.
stakeholder	-	Includes details about the ignored stakeholder record.
subject	-	Includes details about the user group details of the ignored stakeholder record.
name	String	User name or user group name of the ignored stakeholder record.
type	String	User or user group of the ignored stakeholder record.
role	String	User or user group role of the ignored stakeholder record.
assetIdentity	-	Includes details of the asset to which the workflow configuration belongs.
id	String	Identifier of the asset.

Field	Type	Description
assetName	String	Name of the asset.
assetType	String	Type of asset. Value can be `rds`, `codelist`, `crosswalk`, or `hierarchy`.
assetWorkflowConfigurationImportReport	-	Includes details about the imported workflow configuration.
assetWorkflowImportIgnoredTasks	-	Includes details of the workflow tasks ignored during import.
reason	String	The reason for ignoring workflow tasks during import.
taskName	String	Name of the workflow task.
notFoundUserGroups	Array	List of user groups that aren't found.
assetIdentity	-	Includes details of the asset to which the workflow configuration belongs.
id	String	Identifier of the asset.
assetName	String	Name of the asset.
assetType	String	Type of asset. Value can be `rds`, `codelist`, `crosswalk`, or `hierarchy`.
importIgnoredWorkflowTask	-	Includes details about the ignored workflow tasks.
taskName	String	Name of the workflow task.
taskAction	-	Includes details about the workflow task action.
approve	String	Action name of the workflow task.
sendBack	String	Action name of the workflow task.
reject	String	Action name of the workflow task.
approvers	-	Includes details about the workflow task approvers.
userGroups	-	Includes details about user groups of workflow task approvers.
name	String	Names of user groups of workflow task approvers.
assetDictionaryImportReport	-	Includes the summary of imported dictionaries.
assetDictionaryImportDetails	-	Includes details about the imported dictionaries.
assetDictionaryImportItem	-	Includes the properies of an imported dictionary.
dictionaryConfigurationId	String	Identifier of the dictionary.
dictionaryName	String	Name of the dictionary.

Field	Type	Description
frsLocationName	String	The project or folder that contains the dictionary.
importError	-	Includes the error details.
errorCode	String	Error code for the error type.
errorSummary	String	Message that explains why the dictionary isn't imported.
assetIdentity	-	Includes details of the asset to which the dictionaries belong.
id	String	Identifier of the asset.
assetType	String	Type of asset. Value can be `CODELIST` or `CROSSWALK`.
dictionaryState	String	Status of the dictionary. Value can be `VALID` or `INVALID`.

GET example

To get the job details of an import model job, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/model/
import/job/a335550eebe02c6d133898b9 HTTP/1.1
Content-Type: application/json
Accept: application/json
```

The following sample response shows the job details of an import model job:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 4117

{
   "jobId":"a335550eebe02c6d133898b9",
   "type":"IMPORT_MODEL",
   "status":"SUCCESS",
   "createdBy":"userId",
   "createdDate":"2023-01-25T09:54:08.087+00:00",
   "detail":{
      "modelImportReport":{
         "stakeholderImportResult":{
            "ignoredStakeholderEntries":[
               {
                  "reason":"User or UserGroup does not exist to create stakeholder",
                  "assetIdentity":{
                     "id":"fddde0de9f7740721d3ac264",
                     "assetType":"REFERENCEDATASET",
                     "assetName":"Country"
                  },
                  "stakeholder":{

                  }
               },
               {
                  "reason":"User or UserGroup does not exist to create stakeholder",
                  "assetIdentity":{
                     "id":"aaa97c7034473568b09d65f7",
                     "assetType":"REFERENCEDATASET",
                     "assetName":"rds2"
                  },
                  "stakeholder":{

                  }
               },
```

```
                {
                   "reason":"User or UserGroup does not exist to create stakeholder",
                   "assetIdentity":{
                      "id":"db73317aab6d239460fdac9f",
                      "assetType":"CODELIST",
                      "assetName":"rds1_cl1_name"
                   },
                   "stakeholder":{

                   }
                },
                {
                   "reason":"User or UserGroup does not exist to create stakeholder",
                   "assetIdentity":{
                      "id":"fb4c079c4d8376e03f72a73a",
                      "assetType":"CODELIST",
                      "assetName":"SapCountry"
                   },
                   "stakeholder":{

                   }
                },
                {
                   "reason":"User or UserGroup does not exist to create stakeholder",
                   "assetIdentity":{
                      "id":"71e94e4b43e146edb370461a",
                      "assetType":"CODELIST",
                      "assetName":"IsoCountry"
                   },
                   "stakeholder":{

                   }
                },
                {
                   "reason":"User or UserGroup does not exist to create stakeholder",
                   "assetIdentity":{
                      "id":"c984c9006ec3b492c2395e1a",
                      "assetType":"CROSSWALK"
                   },
                   "stakeholder":{

                   }
                },
                {
                   "reason":"The user group does not exists in the system to import as
stakeholder assignment to Hierarchy hierarchy1",
                   "assetIdentity":{
                      "id":"hierarchy1",
                      "assetType":"HIERARCHY",
                      "assetName":"hierarchy1"
                   },
                   "stakeholder":{
                      "subject":{
                         "name":"rdmUserGroup",
                         "type":"USERGROUP"
                      }
                   }
                }
             ]
          },
          "assetWorkflowConfigurationImportReport":{},
          "dqRuleAssociationImportReport":{},
          "assetDictionaryImportReport": {
                 "assetDictionaryImportDetails": [
                    {
                      "assetDictionaryImportItem": {
                           "dictionaryConfigurationId":
"5OaLvg4nuYtb0KnJe8DsSe3627963f0db9497795e20de1374ef7b3",
                           "dictionaryName": "TestCl2407-1",
                           "frsLocationName": "TestImportFolder1407",
                           "assetIdentity": {
```

```
                                "id": "68a72492e7df977e919b2fed",
                                "assetType": "CODELIST"
                            }
                        },
                        "importError": {
                            "errorCode": "RDM.0060058",
                            "errorSummary": "The folder does not exist in DQ in target
system, hence skipping the given dictionary, not to import"
                        }
                    },
                     {
                       "assetDictionaryImportItem": {
                            "dictionaryConfigurationId":
"inoN6scHi8bbMQaYDW0Yue22d5b29a15cb4c40a8943dd718c1c54c",
                            "dictionaryName": "TestCodeCopy",
                            "frsLocationName": "Reference360",
                            "assetIdentity": {
                                "id": "68a72492e7df977e919b2fed",
                                "assetType": "CODELIST"
                            },
                            "dictionaryState": "VALID"
                        }
                    }
                ]
            },
             "mileStonesDuringImport": [
                "The processing of entities
ReferenceDataSets,CodeLists,Crosswalks,Hierarchies completed.",
                "The processing of dictionaries of codeLists completed",
                "The processing of dictionaries of crosswalks completed"
              ],
            "numImportedRds":1,
            "numImportedCodeList":3,
            "numImportedCrosswalk":1,
            "numImportedHierarchy":1,
            "importedEntities":[
               {
                  "type":"rds",
                  "name":"rds2",
                  "oldId":"aaa97c7034473568b09d65f7",
                  "newId":"48a5e3416f1da3e67b7554c5"
               },
               {
                  "type":"codelist",
                  "name":"rds1_cl1_name",
                  "oldId":"db73317aab6d239460fdac9f",
                  "newId":"7dc204fee7d65086c1bdfdaa"
               },
               {
                  "type":"codelist",
                  "name":"SapCountry",
                  "oldId":"fb4c079c4d8376e03f72a73a",
                  "newId":"3a800b90e3fffb1f12b18502"
               },
               {
                  "type":"codelist",
                  "name":"IsoCountry",
                  "oldId":"71e94e4b43e146edb370461a",
                  "newId":"c42bdf7689b6c35e25b22b29"
               },
               {
                  "type":"crosswalk",
                  "name":"description",
                  "oldId":"c984c9006ec3b492c2395e1a",
                  "newId":"fe318acfc4a126b71f0ecba8"
               },
               {
                  "type":"hierarchy",
                  "name":"hierarchy1",
                  "oldId":"hierarchy1",
                  "newId":"847f90ccde87d4674962366d"
```

```
            }
         ],
         "ignoredRdsCount":1,
         "ignoredCodeListCount":0,
         "ignoredCrosswalkCount":0,
         "ignoredHierarchyCount":0,
         "ignoredEntities":[
            {
               "type":"rds",
               "name":"Country",
               "oldId":"fddde0de9f7740721d3ac264",
               "newId":"c77161ce50bce805c788d706"
            }
         ],
         "enumImportResult":{
            "newEntries":2,
            "existingEntries":3,
            "updatedEntries":0
         }
      }
   }
}
```


import version 1

Use this resource to import code values and value mappings, retrieve the status of an import job, and retrieve the detailed report for a failed import job.

Import code values (v1)

Imports code values into a code list.

Note: Validation checks are not performed when you import code values.

POST request

To import code values into a code list, submit a POST request with the following URI:

```
/rdm-service/external/v1/import
```

The request contains form-data with the following parameters:

file

Specify a CSV file that contains the code value attributes. The columns specified depend on your data model. The CSV must start with two header rows, followed by the data rows.

For example, you might have the following code values:

```
Name,Code
Name,Code
Afghanistan,AFG
Aland Islands,ALA
Albania,ALB
Algeria,DZA
American Samoa,ASM
```

Note: You can provide additional information about a code value. For example, you might want to assign a code value to the Approved status. In the `status.key` column for the code value, enter the key value for the system reference data value that you want to assign. For more information about your configured system reference data values, see "Get system reference data values" on page 529.

importSettings

Specify the file specific configuration and container details.

The `importSettings` parameter includes the following attributes:

Field	Type	Description
delimiter	String	Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE`, or `TAB`.
textQualifier	String	Symbol used to indicate where a text begins and ends. Value must be `DOUBLE_QUOTE`, `SINGLE_QUOTE`, or `NO_QUOTE`.
codepage	String	Code page used for the export file. Value must be `UTF8`.
dateFormat	String	Date format to import code values. You can use either the format name or pattern to specify one of the following date formats: - `US_HYPHEN` or MM-dd-yyyy format. - `US_SLASH` or MM/dd/yyyy format. - `ISO` or yyyy-MM-dd format. You can also use the following date formats: - dd-MM-yyyy - dd-MMM-yyyy - dd/MM/yyyy - yyyy/MM/dd **Note:** Ensure that the date formats in the import settings and the CSV file match. Otherwise, the import fails. The date format specified in the import settings takes priority. If you don't specify a date format in the import settings, Reference 360 detects the format from the first 10 rows of the CSV file.
containerType	String	Type of asset that contains code values. Value must be `codelist`.
containerId	String	The ID of the code list to which you want to import code values. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
startingRow	String	Line number from which to start importing data. By default, all rows are imported.

POST response

The response contains the details of the import job.

The response contains the following attributes:

Field	Type	Description
jobId	String	ID of the job.
state	String	Status of the job. Values are `CREATED`, `INPROGRESS`, `COMPLETED`, `SUSPENDED`, `FAILED`, `STOPPED`, `QUEUED` or `WARNING`.
startTime	Number	Time, in milliseconds, when the job started.
numOfRecordsProcessed	Number	Number of records processed.

Field	Type	Description
numOfRecordsFailed	Number	Number of records that failed to be imported.
numOfRecordsSucceeded	Number	Number of records successfully imported.

POST example

To import code values, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/import HTTP/1.1
Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=file; filename=import-code-values.csv

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=importSettings
Content-Type: application/json;charset=UTF-8

{
  "delimiter":"COMMA",
  "textQualifier":"DOUBLE_QUOTE",
  "codepage":"UTF8",
  "dateFormat":"ISO",
  "containerType":"CODELIST",
  "containerId":"9ab3201990a54dcdc86f54cf",
  "startingRow":null
}
--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
```

The CSV file might contain the following header rows and data rows:

```
Name,Code
Name,Code
Afghanistan,AFG
Aland Islands,ALA
Albania,ALB
Algeria,DZA
American Samoa,ASM
```

The following sample response shows the status of the import job:

```
{
  "jobId":"dd1b2018cb47cef99f8d0f42",
  "state":"INPROGRESS",
  "startTime":1561367377428,
  "numOfRecordsProcessed":100,
  "numOfRecordsFailed":25,
  "numOfRecordsSucceeded":75
}
```

Import value mappings (v1)

Imports value mappings into a crosswalk.

POST request

To import value mappings into a crosswalk, submit a POST request with the following URI:

```
/rdm-service/external/v1/import
```

The request contains form-data with two parameters:

file

Specify a CSV file that contains the code value attributes. The columns specified depend on your data model. The CSV starts with two header rows, followed by the data rows.

For example, you might have the following code values:

```
sourcePKey,_from.id.sourcePKey,_to.id.sourcePKey
sourcePKey,_from.id.sourcePKey,_to.id.sourcePKey
AF_AFG,AF,AFG
AL_ALA,AL,ALA
ALB_ALB,ALB,ALB
DZ_DZA,DZ,DZA
AS_ASM,AS,ASM
```

importSettings

Specify the file specific configuration and container details.

The `importSettings` includes the following attributes:

Field	Type	Description
delimiter	String	Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE`, or `TAB`.
textQualifier	String	Symbol used to indicate where a text begins and ends. Value must be `DOUBLE_QUOTE`, `SINGLE_QUOTE`, or `NO_QUOTE`.
codepage	String	Code page used for the export file. Value must be `UTF8`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
containerType	String	Type of asset that contains value mappings. Value must be `crosswalk`.
containerId	String	The ID of the crosswalk to which you want to import value mappings. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
startingRow	String	Line number from which to start importing data. By default, all rows are imported.

POST response

The response contains the details of the import job.

The following table describes the attributes in the response:

Field	Type	Description
jobId	String	ID of the job.
state	String	Status of the job. Values are `CREATED`, `INPROGRESS`, `COMPLETED`, `SUSPENDED`, `FAILED`, `STOPPED`, `QUEUED` or `WARNING`.
startTime	Number	Time, in milliseconds, when the job started.

Field	Type	Description
numOfRecordsProcessed	Number	Number of records processed.
numOfRecordsFailed	Number	Number of records that failed to be imported.
numOfRecordsSucceeded	Number	Number of records successfully imported.

POST example

To import value mappings into a crosswalk, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/import HTTP/1.1
Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=file; filename=import-value-mappings.csv

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=importSettings
Content-Type: application/json;charset=UTF-8

{
  "delimiter":"COMMA",
  "textQualifier":"DOUBLE_QUOTE",
  "codepage":"UTF8",
  "dateFormat":"ISO",
  "containerType":"CROSSWALK",
  "containerId":"9ab3201990a54dcdc86f53AB",
  "startingRow":null
}
--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
```

The CSV file might contain the following header rows and data rows:

```
sourcePKey,_from.id.sourcePKey,_to.id.sourcePKey
sourcePKey,_from.id.sourcePKey,_to.id.sourcePKey
AF_AFG,AF,AFG
AL_ALA,AL,ALA
ALB_ALB,ALB,ALB
DZ_DZA,DZ,DZA
AS_ASM,AS,ASM
```

The following sample response shows the status of the import job:

```
{
    "jobId": "dd1b2018cb47cef99f8d0f43",
    "state": "INPROGRESS",
    "startTime": 1561367377428,
    "numOfRecordsProcessed": 100,
    "numOfRecordsFailed": 25,
    "numOfRecordsSucceeded": 75
}
```

Import hierarchy relationships (v1)

Imports hierarchy relationships for code values and top-level code values into a hierarchy.

POST request

To import hierarchy relationships, submit a POST request with the following URI:

```
/rdm-service/external/v1/import/hierarchy
```

The request contains form-data with two parameters:

file

Specify a CSV file that contains the hierarchy relationships. The CSV file contains one header row with two columns: Code and ParentCode. You can list the related code values below the header.

For example, you might have the following relationships in the CSV file:

```
Code,ParentCode
C1,P1
C2,P2
C1,P3
```

If you import top-level code values, only use the Code column in the header row.

For example, you might have the following top-level code values:

```
Code
P1
P2
P3
```

importSettings

Specify the file specific configuration and container details.

The `importSettings` includes the following attributes:

Field	Type	Description
delimiter	String	Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE`, or `TAB`.
textQualifier	String	Symbol used to indicate where a text begins and ends. Value must be `DOUBLE_QUOTE`, `SINGLE_QUOTE`, or `NO_QUOTE`.
codepage	String	Code page used for the export file. Value must be `UTF8`.
hierarchyId	String	ID of the hierarchy to which you want to import relationships. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
parentCodeListId	String	ID of the parent code list. **Note:** If you import top-level code values, you do not need to provide this attribute.
childCodeListId	String	ID of the child code list.
startingRow	String	Line number from which to start importing data. By default, all rows are imported.

POST response

The response contains details of the import job.

The following table describes the attributes in the response:

Field	Type	Description
jobId	String	ID of the job.
state	String	Status of the job. Values are `CREATED`, `INPROGRESS`, `COMPLETED`, `SUSPENDED`, `FAILED`, `STOPPED`, `QUEUED` or `WARNING`.

Field	Type	Description
startTime	Number	Time, in milliseconds, when the job started.
numOfRecordsProcessed	Number	Number of records processed.
numOfRecordsFailed	Number	Number of records that failed to be imported.
numOfRecordsSucceeded	Number	Number of records successfully imported.

POST example

To import hierarchy relationships into a hierarchy, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/import/
hierarchy HTTP/1.1
Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=file; filename=import.csv
Content-Type: text/plain

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=importSettings
Content-Type: application/json;charset=UTF-8

{
  "delimiter":"COMMA",
  "textQualifier":"DOUBLE_QUOTE",
  "startingRow":0,
  "codepage":"UTF8",
  "hierarchyId":"c79ab91c19b13b11d8d43770",
  "childCodeListId":"96f06071e4aaea81ff203abe",
  "parentCodeListId":"9b300f793470882ca23e6091"
}
--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
```

The CSV file might contain the following header row and data rows:

```
Code,ParentCode
C1,P1
C2,P2
C3,P3
```

To import top-level code values into a hierarchy, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/import/
hierarchy HTTP/1.1
Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=file; filename=import.csv
Content-Type: text/plain

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=importSettings
Content-Type: application/json;charset=UTF-8

{
  "delimiter":"COMMA",
  "textQualifier":"DOUBLE_QUOTE",
  "startingRow":0,
  "codepage":"UTF8",
  "hierarchyId":"c79ab91c19b13b11d8d43770",
  "childCodeListId":"96f06071e4aaea81ff203abe"
```

```
}
--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
```

The CSV file might contain the following header row and data rows:

```
Code
P1
P2
P3
```

The following sample response shows that status of the import job:

```
{
  "jobId":"73580d323feb170be5ec0fd5",
  "state":"INPROGRESS",
  "startTime":1603092643055,
  "numOfRecordsProcessed":100,
  "numOfRecordsFailed":25,
  "numOfRecordsSucceeded":75
}
```

Get import job status (v1)

Retrieves the status of an import job.

GET request

To get the status of an import job, submit a GET request with the following URI:

```
/rdm-service/external/v1/import/job/<job ID>
```

GET response

The response contains the details of the import job, such as the status of the import job, start time, and number of records processed for import.

The following table describes the attributes in the response:

Field	Type	Description
jobId	String	ID of the job.
state	String	Status of the job. Values are `CREATED`, `INPROGRESS`, `COMPLETED`, `SUSPENDED`, `FAILED`, `STOPPED`, `QUEUED` or `WARNING`.
startTime	Number	Time, in milliseconds, when the job started.
numOfRecordsProcessed	Number	Number of records processed.
numOfRecordsFailed	Number	Number of records that failed to be imported.
numOfRecordsSucceeded	Number	Number of records successfully imported.

GET example

To get the status of an import job, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/import/job/
dd1b2018cb47cef99f8d0f42 HTTP/1.1
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the status of an import job:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 193

{
  "jobId":"dd1b2018cb47cef99f8d0f42",
  "state":"INPROGRESS",
  "startTime":1561367376330,
  "numOfRecordsProcessed":100,
  "numOfRecordsFailed":25,
  "numOfRecordsSucceeded":75
}
```

Get failed import job report (v1)

Retrieves an error report for a failed import job.

By default, the error report shows the first 100,000 records. To retrieve more records or to view the next page of records in the error report, use the query parameters.

GET request

To retrieve an error report for a failed import job, submit a GET request with the following URI:

```
/rdm-service/external/v1/import/job/<job Id>/errorDetails
```

To retrieve the paginated error report, submit a GET request with the following query parameters appended to the URI:

```
/rdm-service/external/v1/import/job/<job Id>/errorDetails?pageNum=<page
number>&recordsPerPage=<records per page>
```

GET request query parameters

You can append the query parameters to the URI to retrieve paginated errors.

The following table lists the query parameters:

Parameter	Description
pageNum	Optional. Page number to display. Default value is `0`.
recordsPerPage	Optional. Number of records to display per page. Default value is `100000`.

GET response

The response contains the error details, such as the line where the error occurred and the reason.

The following table describes the attributes in the response:

Field	Type	Description
jobId	String	ID of the job.
entityType	String	Type of entity imported. Value is `BusinessEntity` or `Relationship`.
fileName	String	Name of the file. Value must end with the `.csv` file extension.
entityName	String	Name of the entity.

Field	Type	Description
errorDetails	-	Includes the error details.
lineNumber	Number	Line number where the error occurred.
entitySourcePkey	String	Column identifier.
reasons	Array	Explanation of failure.

GET example

To retrieve an error report for a failed import job, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/import/job/
dae4301e9369c16c08bf0881/errorDetails HTTP/1.1
```

To retrieve the second page of records in a paginated error report with 100 records per page, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/import/job/
dae4301e9369c16c08bf0881/errorDetails?pageNum=2&recordsPerPage=100 HTTP/1.1
```

The following sample response shows the error report for a failed code values import job:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 395
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "jobId":"5984980d317a4b00cfe18880",
  "entityType":"BusinessEntity",
  "fileName":"import.csv",
  "entityName":"rdm.value.be.442ac56e11d5fc9bb11f6a3f",
  "errorDetails":[
    {
      "lineNumber":1,
      "entitySourcePkey":"Code-1101",
      "reasons":[
        "The code value INX does not exist in the picklist that is in the path Country.
Specify a code value from the picklist."
      ]
    }
  ]
}
```

The following sample response shows the error report for a failed value mappings import job:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 354

{
  "jobId":"dae4301e9369c16c08bf0881",
  "entityType":"Relationship",
  "fileName":"import.csv",
  "entityName":"rdm.crosswalk.rel.21ffd6b5f92d10c744acc27c.fc66c441288cf898c6fe5023",
  "errorDetails":[
    {
      "lineNumber":1,
      "entitySourcePkey":"AF_AFG",
      "reasons":[
        "The requested resource with ID 'AFG' does not exist."
      ]
    }
```

```
  ]
}
```

import version 2

Use this resource to import code values, value mappings, and hierarchy relationships.

Import code values (v2)

Imports code values with optional custom headers into a code list.

Note: Validation checks are not performed when you import code values.

POST request

To import code values into a code list, submit a POST request with the following URI:

```
/rdm-service/external/v2/import
```

The request contains form-data with the following parameters:

file

Specify a CSV file that contains the code value attributes. The columns specified depend on your data model. The CSV must contain one or two header rows, followed by the data rows. If the occurrence of header row is one in the import CSV file, you must set the repeatHeaders field to false.

Note: You can provide a single header row in the CSV file because it no longer requires two header rows.

For example, you might have the following code values:

```
CountryName,CountryCode
CountryName,CountryCode
Afghanistan,AFG
Aland Islands,ALA
Albania,ALB
Algeria,DZA
American Samoa,ASM
```

Note: You can provide additional information about a code value. For example, you might want to assign a code value to the Approved status. In the `status.key` column for the code value, enter the key value for the system reference data value that you want to assign. For more information about your configured system reference data values, see "Get system reference data values" on page 529.

If an import file doesn't contain business IDs, Reference 360 automatically generates them based on the business ID configuration in a code list.

importSettings

Specify the file-specific configuration and container details.

The `importSettings` parameter includes the following attributes:

Field	Type	Description
headerLinePresent	Boolean	Indicates whether the header line is present. Value is `true` or `false`. Default is `true`.
nestedFieldSeparator	String	Separator used to separate a nested field. Value is `SEMICOLON` or `DOT`. Default is `DOT`.
headerLineNumber	String	Line number from which the header line starts. Default is `1`.
delimiter	String	Delimiter used to separate values. Value must be `COMMA`, `SEMICOLON`, `SPACE`, or `TAB`.
textQualifier	String	Symbol used to indicate where a text begins and ends. Value must be `DOUBLE_QUOTE`, `SINGLE_QUOTE`, or `NO_QUOTE`.
codepage	String	Code page used for the import file. Value must be `UTF8` or `MS_WINDOWS`.
dateFormat	String	Date format to import code values. You can use either the format name or pattern to specify one of the following date formats: - `US_HYPHEN` or MM-dd-yyyy format. - `US_SLASH` or MM/dd/yyyy format. - `ISO` or yyyy-MM-dd format. You can also use the following date formats: - dd-MM-yyyy - dd-MMM-yyyy - dd/MM/yyyy - yyyy/MM/dd **Note:** Ensure that the date formats in the import settings and the CSV file match. Otherwise, the import fails. The date format specified in the import settings takes priority. If you don't specify a date format in the import settings, Reference 360 detects the format from the first 10 rows of the CSV file.
containerType	String	Type of asset that contains code values. Value must be `codelist`.
containerId	String	The ID of the code list to which you want to import code values. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
startingRow	String	Line number from which to start importing data. By default, all rows are imported.
repeatHeaders	Boolean	Optional. Indicates whether the file has two headers. Value is `true` or `false`. Default is `true`.

Field	Type	Description
mappings	Object	Optional. Maps the column headers in the CSV file with the target attributes of code lists in the key-value format. - You can map the user-defined column name to source field name if required. - You can either choose to ignore the mapping or map all the columns in an import CSV file. **Note:** The key represents the header in the CSV file, and the value represents the field names in Reference 360. For example, if the business ID column in the input CSV is labeled as `sequence`, the following mapping represents the mapping for the business ID column: `"sequence" : "businessID"` If mappings aren't specified in the import settings, Reference 360 selects the `businessID` column by default.
approvalOptions	-	Includes details about the approval parameters.
priority	String	Optional. Defines the priority of the approval request. Value can be `LOW`, `MEDIUM`, `HIGH`, or `CRITICAL`.
dueDate	String	Use the following format: `yyyy-MM-dd`.
Comments	String	Comments that the requester adds to explain the changes.

POST response

The response contains the details of the import job.

The response contains the following attributes:

Field	Type	Description
jobId	String	ID of the job.
state	String	Status of the job. Values are `CREATED`, `INPROGRESS`, `COMPLETED`, `SUSPENDED`, `FAILED`, `STOPPED`, `QUEUED` or `WARNING`.
startTime	Number	Time, in milliseconds, when the job started.
numOfRecordsProcessed	Number	Number of records processed.
numOfRecordsFailed	Number	Number of records that failed to be imported.
numOfRecordsSucceeded	Number	Number of records successfully imported.

POST example

To import code values, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/import HTTP/1.1
Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=file; filename=import-code-values.csv
```

```
{
   "headerLinePresent":true,
   "nestedFieldSeparator":".",
   "headerLineNumber":1,
   "delimiter":"COMMA",
   "textQualifier":"DOUBLE_QUOTE",
   "codepage":"UTF8",
   "dateFormat":"ISO",
   "containerType":"CODELIST",
   "containerId":"c31851709a749e2d53188ec0",
   "startingRow":3,
   "repeatHeaders":false,
   "mappings":{
      "CL_Name":"Name",
      "CL_Code":"Code",
      "CL_Description":"Description"
      "Business_ID":"Business ID"
   },
   "approvalOptions":{
      "priority":"LOW",
      "dueDate":"2024-12-30",
      "comments":"Please approve the request"
   }
}
--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
```

The CSV file might contain the following header rows and data rows:

```
CL_Name,CL_Code,Business_ID
Afghanistan,AFG,RDM001
Aland Islands,ALA,RDM002
Albania,ALB,RDM003
Algeria,DZA,RDM004
American Samoa,ASM,RDM005
```

The following sample response shows the status of the import job:

```
{
   "jobId":"32121c139c84a8edc8696c0c",
   "state":"INPROGRESS",
   "startTime":1631086520325,
   "numOfRecordsProcessed":100,
   "numOfRecordsFailed":25,
   "numOfRecordsSucceeded":75
}
```

Import value mappings (v2)

Imports value mappings into a crosswalk.

POST request

To import value mappings into a crosswalk, submit a POST request with the following URI:

```
/rdm-service/external/v2/import
```

The request contains form-data with two parameters:

file

Specify a CSV file that contains the code value attributes. The columns specified depend on your data model. The CSV must contain one or two header rows, followed by the data rows. If the occurrence of header row is one in the import CSV file, you must set the repeatHeaders field to false.

Note: You can provide a single header row in the CSV file as it no longer requires two header rows.

For example, you might have the following code values:

```
key,from, to
key,from, to
AF_AFG,AF,AFG
AL_ALA,AL,ALA
ALB_ALB,ALB,ALB
DZ_DZA,DZ,DZA
AS_ASM,AS,ASM
```

importSettings

Specify the file-specific configuration and container details.

The `importSettings` includes the following attributes:

Field	Type	Description
delimiter	String	Delimiter used to separate values. Value must be `COMMA`, `SEMICOLON`, `SPACE`, or `TAB`.
textQualifier	String	Symbol used to indicate where a text begins and ends. Value must be `DOUBLE_QUOTE`, `SINGLE_QUOTE`, or `NO_QUOTE`.
codepage	String	Code page used for the import file. Value must be `UTF8` or `MS_WINDOWS`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
containerType	String	Type of asset that contains value mappings. Value must be `crosswalk`.
containerId	String	The ID of the crosswalk to which you want to import value mappings. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
startingRow	String	Line number from which to start importing data. By default, all rows are imported.
repeatHeaders	Boolean	Optional. Indicates whether the file has two headers. Value is `true` or `false`. Default is `true`.
mappings	Object	Optional. Maps the column headers in the CSV file with the target attributes of code lists in the key-value format. - You can map the user-defined column name to source field name if required. - You can either choose to ignore the mapping or map all the columns in an import CSV file. **Note:** The key represents the header in the CSV file, and the value represents the field names in Reference 360.
approvalOptions	-	Includes details about the approval parameters.
priority	String	Optional. Defines the priority of the approval request. Value can be `LOW`, `MEDIUM`, `HIGH`, or `CRITICAL`.
dueDate	String	Use the following format: `yyyy-MM-dd`.
Comments	String	Comments that the requester adds to explain the changes.

POST response

The response contains the details of the import job.

The following table describes the attributes in the response:

Field	Type	Description
jobId	String	ID of the job.
state	String	Status of the job. Values are `CREATED`, `INPROGRESS`, `COMPLETED`, `SUSPENDED`, `FAILED`, `STOPPED`, `QUEUED` or `WARNING`.
startTime	Number	Time, in milliseconds, when the job started.
numOfRecordsProcessed	Number	Number of records processed.
numOfRecordsFailed	Number	Number of records that failed to be imported.
numOfRecordsSucceeded	Number	Number of records successfully imported.

POST example

To import value mappings into a crosswalk, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/import HTTP/1.1
Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=file; filename=import-value-mappings.csv

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=importSettings
Content-Type: application/json;charset=UTF-8

{
   "delimiter":"COMMA",
   "textQualifier":"DOUBLE_QUOTE",
   "startingRow":null,
   "codepage":"UTF8",
   "dateFormat":"ISO",
   "containerType":"CROSSWALK",
   "containerId":"c31851709a749e2d53188ec0",
   "repeatHeaders":true,
   "mappings":{
      "key":"sourcePKey",
      "from":"_from.id.sourcePKey",
      "to":"_to.id.sourcePKey"
   },
   "approvalOptions":{
      "priority":"LOW",
      "dueDate":"2024-12-30",
      "comments":"Please approve the request"
   }
}
--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
```

The CSV file might contain the following header rows and data rows:

```
key,from, to
key,from, to
AF_AFG,AF,AFG
AL_ALA,AL,ALA
ALB_ALB,ALB,ALB
```

```
DZ_DZA,DZ,DZA
AS_ASM,AS,ASM
```

The following sample response shows the status of the import job:

```
{
   "jobId":"32121c139c84a8edc8696c0c",
   "state":"INPROGRESS",
   "startTime":1631086520406,
   "numOfRecordsProcessed":100,
   "numOfRecordsFailed":25,
   "numOfRecordsSucceeded":75
}
```

Import hierarchy relationships (v2)

Imports hierarchy relationships for code values and top-level code values into a hierarchy, with custom headers as optional settings.

POST request

To import hierarchy relationships, submit a POST request with the following URI:

```
/rdm-service/external/v2/import/hierarchy
```

The request contains form-data with two parameters:

file

Specify a CSV file that contains the hierarchy relationships. The CSV file contains one header row with two columns: Code and ParentCode. You can list the related code values below the header.

For example, you might have the following relationships in the CSV file:

```
Code,ParentCode
C1,P1
C2,P2
C1,P3
```

If you import top-level code values, only use the Code column in the header row.

For example, you might have the following top-level code values:

```
Code
P1
P2
P3
```

importSettings

Specify the file specific configuration and container details.

The `importSettings` includes the following attributes:

Field	Type	Description
delimiter	String	Delimiter used to separate values. Value must be `COMMA`, `SEMICOLON`, `SPACE`, or `TAB`.
textQualifier	String	Symbol used to indicate where a text begins and ends. Value must be `DOUBLE_QUOTE`, `SINGLE_QUOTE`, or `NO_QUOTE`.
codepage	String	Code page used for the import file. Value must be `UTF8` or `MS_WINDOWS`.

Field	Type	Description
hierarchyId	String	ID of the hierarchy to which you want to import relationships. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
parentCodeListId	String	ID of the parent code list. **Note:** If you import top-level code values, you do not need to provide this attribute.
childCodeListId	String	ID of the child code list.
startingRow	String	Line number from which to start importing data. By default, all rows are imported.
repeatHeaders	Boolean	Optional. Indicates whether the file has two headers. Value is `true` or `false`. Default is `true`.
mappings	Object	Optional. Maps the column headers in the CSV file with the target attributes of code lists in the key-value format. - You can map the user-defined column name to source field name if required. - You can either choose to ignore the mapping or map all the columns in an import CSV file. **Note:** The key represents the header in the CSV file, and the value represents the field names in Reference 360.

POST response

The response contains details of the import job.

The following table describes the attributes in the response:

Field	Type	Description
jobId	String	ID of the job.
state	String	Status of the job. Values are `CREATED`, `INPROGRESS`, `COMPLETED`, `SUSPENDED`, `FAILED`, `STOPPED`, `QUEUED` or `WARNING`.
startTime	Number	Time, in milliseconds, when the job started.
numOfRecordsProcessed	Number	Number of records processed.
numOfRecordsFailed	Number	Number of records that failed to be imported.
numOfRecordsSucceeded	Number	Number of records successfully imported.

POST example

To import hierarchy relationships into a hierarchy, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/import/
hierarchy HTTP/1.1
Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=file; filename=import.csv
Content-Type: text/plain
```

```
--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=importSettings
Content-Type: application/json;charset=UTF-8

{
   "delimiter":"COMMA",
   "textQualifier":"DOUBLE_QUOTE",
   "startingRow":0,
   "codepage":"UTF8",
   "hierarchyId":"eeb38899dd463330df844b8e",
   "childCodeListId":"3ebff6c89e03c41d5dd08acc",
   "parentCodeListId":"68c3ba1b4c3402a7cc18169a",
   "mappings":{
      "state":"Code",
      "country":"ParentCode"
   }
}--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
```

The CSV file might contain the following header row and data rows:

```
Code,ParentCode
C1,P1
C2,P2
C3,P3
```

To import top-level code values into a hierarchy, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/import/
hierarchy HTTP/1.1
Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=file; filename=import.csv
Content-Type: text/plain

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=importSettings
Content-Type: application/json;charset=UTF-8

{
   "delimiter":"COMMA",
   "textQualifier":"DOUBLE_QUOTE",
   "startingRow":0,
   "codepage":"UTF8",
   "hierarchyId":"eeb38899dd463330df844b8e",
   "childCodeListId":"3ebff6c89e03c41d5dd08acc",
   "parentCodeListId":"68c3ba1b4c3402a7cc18169a",
   "mappings":{
      "state":"Code",
      "country":"ParentCode"
   }
}--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
```

The CSV file might contain the following header row and data rows:

```
Code
P1
P2
P3
```

The following sample response shows that status of the import job:

```
{
   "jobId":"32121c139c84a8edc8696c0c",
   "state":"INPROGRESS",
   "startTime":1631086520353,
   "numOfRecordsProcessed":100,
   "numOfRecordsFailed":25,
   "numOfRecordsSucceeded":75
}
```

import version 3

Use this resource to import code values, value mappings, and hierarchy relationships based on the internal ID or alias.

Import code values (v3)

Imports code values with optional custom headers into a code list based on the internal ID or alias.

Note: Validation checks are not performed when you import code values.

POST request

To import code values into a code list based on an internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v3/import
```

The request contains form-data with the following parameters:

file

Specify a CSV file that contains the code value attributes. The columns specified depend on your data model. The CSV must contain one or two header rows, followed by the data rows. If the occurrence of header row is one in the import CSV file, you must set the repeatHeaders field to false.

Note: You can provide a single header row in the CSV file because it no longer requires two header rows.

For example, you might have the following code values:

```
CountryName,CountryCode
CountryName,CountryCode
Afghanistan,AFG
Aland Islands,ALA
Albania,ALB
Algeria,DZA
American Samoa,ASM
```

The following sample CSV file contains multilingual code values:

```
_language,_localizedContent.Name,Code,Description,dependency.dependency.Code,status.s
tatus.key,effectiveDate
en-US,E1 in EN,E1,,T3,,
en-US,E2 in EN,E2,,T3,,
```

Note: You can provide additional information about a code value. For example, you might want to assign a code value to the Approved status. In the `status.key` column for the code value, enter the key value for the system reference data value that you want to assign. For more information about your configured system reference data values, see "Get system reference data values" on page 529.

If an import file doesn't contain business IDs, Reference 360 automatically generates them based on the business ID configuration in a code list.

importSettings

Specify the file-specific configuration and container details.

The `importSettings` parameter includes the following attributes:

Field	Type	Description
headerLinePresent	Boolean	Indicates whether the header line is present. Value is `true` or `false`. Default is `true`.
nestedFieldSeparator	String	Separator used to separate a nested field. Value is `SEMICOLON` or `DOT`. Default is `DOT`.
headerLineNumber	String	Line number from which the header line starts. Default is `1`.
delimiter	String	Delimiter used to separate values. Value must be `COMMA`, `SEMICOLON`, `SPACE`, or `TAB`.
textQualifier	String	Symbol used to indicate where a text begins and ends. Value must be `DOUBLE_QUOTE`, `SINGLE_QUOTE`, or `NO_QUOTE`.
codepage	String	Code page used for the import file. Value must be `UTF8` or `MS_WINDOWS`.
dateFormat	String	Date format to import code values. You can use either the format name or pattern to specify one of the following date formats: - `US_HYPHEN` or MM-dd-yyyy format. - `US_SLASH` or MM/dd/yyyy format. - `ISO` or yyyy-MM-dd format. You can also use the following date formats: - dd-MM-yyyy - dd-MMM-yyyy - dd/MM/yyyy - yyyy/MM/dd **Note:** Ensure that the date formats in the import settings and the CSV file match. Otherwise, the import fails. The date format specified in the import settings takes priority. If you don't specify a date format in the import settings, Reference 360 detects the format from the first 10 rows of the CSV file.
containerType	String	Type of asset that contains code values. Value must be `codelist`.
containerKey	String	Internal ID or alias of the code list to which you want to import code values.
startingRow	String	Line number from which to start importing data. By default, all rows are imported.
repeatHeaders	Boolean	Optional. Indicates whether the file has two headers. Value is `true` or `false`. Default is `true`.

Field	Type	Description
mappings	Object	Optional. Maps the column headers in the CSV file with the target attributes of code lists in the key-value format. - You can map the user-defined column name to source field name if required. - You can either choose to ignore the mapping or map all the columns in an import CSV file. **Note:** The key represents the header in the CSV file, and the value represents the field names in Reference 360. For example, if the business ID column in the input CSV is labeled as `sequence`, the following mapping represents the mapping for the business ID column: `"sequence" : "businessID"` If mappings aren't specified in the import settings, Reference 360 selects the `businessID` column by default.
approvalOptions	Array	Includes details about the approval parameters.
priority	String	Optional. Defines the priority of the approval request. Value can be `LOW`, `MEDIUM`, `HIGH`, or `CRITICAL`.
dueDate	String	Use the following format: `yyyy-MM-dd`.
Comments	String	Comments that the requester adds to explain the changes.

POST response

The response contains the details of the import job.

The following table describes the attributes in the response body:

Field	Type	Description
jobId	String	ID of the job.
state	String	Status of the job. Values are `CREATED`, `INPROGRESS`, `COMPLETED`, `SUSPENDED`, `FAILED`, `STOPPED`, `QUEUED` or `WARNING`.
startTime	Number	Time, in milliseconds, when the job started.
numOfRecordsProcessed	Number	Number of records processed.
numOfRecordsFailed	Number	Number of records that failed to be imported.
numOfRecordsSucceeded	Number	Number of records successfully imported.

POST example

To import code values based on the internal ID or alias, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/import HTTP/1.1
Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=file; filename=import-code-values.csv
```

```
{
   "headerLinePresent":true,
   "nestedFieldSeparator":".",
   "headerLineNumber":1,
   "delimiter":"COMMA",
   "textQualifier":"DOUBLE_QUOTE",
   "codepage":"UTF8",
   "dateFormat":"ISO",
   "containerType":"CODELIST",
   "containerKey":"ent_country",
   "startingRow":3,
   "repeatHeaders":false,
   "mappings":{
      "CL_Name":"Name",
      "CL_Code":"Code",
      "CL_Description":"Description"
      "Business_ID":"Business ID"
   },
   "approvalOptions":{
      "priority":"LOW",
      "dueDate":"2024-12-30",
      "comments":"Please approve the request"
   }
}
--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
```

The CSV file might contain the following header rows and data rows:

```
CL_Name,CL_Code,Business_ID
Afghanistan,AFG,RDM001
Aland Islands,ALA,RDM002
Albania,ALB,RDM003
Algeria,DZA,RDM004
American Samoa,ASM,RDM005
```

The following sample response shows the status of the import job:

```
{
   "jobId":"32121c139c84a8edc8696c0c",
   "state":"INPROGRESS",
   "startTime":1631086520325,
   "numOfRecordsProcessed":100,
   "numOfRecordsFailed":25,
   "numOfRecordsSucceeded":75
}
```

Import crosswalk values (v3)

Imports value mappings into a crosswalk based on the internal ID or alias.

POST request

To import value mappings into a crosswalk based on an internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v3/import
```

The request contains form-data with two parameters:

file

Specify a CSV file that contains the code value attributes. The columns specified depend on your data model. The CSV must contain one or two header rows, followed by the data rows. If the occurrence of header row is one in the import CSV file, you must set the repeatHeaders field to false.

Note: You can provide a single header row in the CSV file as it no longer requires two header rows.

For example, you might have the following code values:

```
key,from, to
key,from, to
AF_AFG,AF,AFG
AL_ALA,AL,ALA
ALB_ALB,ALB,ALB
DZ_DZA,DZ,DZA
AS_ASM,AS,ASM
```

The following sample CSV file contains multilingual attributes:

```
_language,_localizedContent.Name,Code,Description,dependency.dependency.Code,status.s
tatus.key,effectiveDate
en-US,E1 in EN,E1,,T3,,
en-US,E2 in EN,E2,,T3,,
```

importSettings

Specify the file-specific configuration and container details.

The `importSettings` includes the following attributes:

Field	Type	Description
delimiter	String	Delimiter used to separate values. Value must be `COMMA`, `SEMICOLON`, `SPACE`, or `TAB`.
textQualifier	String	Symbol used to indicate where a text begins and ends. Value must be `DOUBLE_QUOTE`, `SINGLE_QUOTE`, or `NO_QUOTE`.
codepage	String	Code page used for the import file. Value must be `UTF8` or `MS_WINDOWS`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
containerType	String	Type of asset that contains value mappings. Value must be `crosswalk`.
containerKey	String	Internal ID or alias of the crosswalk to which you want to import value mappings.
startingRow	String	Line number from which to start importing data. By default, all rows are imported.
repeatHeaders	Boolean	Optional. Indicates whether the file has two headers. Value is `true` or `false`. Default is `true`.
mappings	Object	Optional. Maps the column headers in the CSV file with the target attributes of code lists in the key-value format. - You can map the user-defined column name to source field name if required. - You can either choose to ignore the mapping or map all the columns in an import CSV file. **Note:** The key represents the header in the CSV file, and the value represents the field names in Reference 360.
approvalOptions	Array	Includes details about the approval parameters.
priority	String	Optional. Defines the priority of the approval request. Value can be `LOW`, `MEDIUM`, `HIGH`, or `CRITICAL`.

Field	Type	Description
dueDate	String	Use the following format: `yyyy-MM-dd`.
Comments	String	Comments that the requester adds to explain the changes.

POST response

The response contains the details of the import job.

The following table describes the attributes in the response body:

Field	Type	Description
jobId	String	ID of the job.
state	String	Status of the job. Values are `CREATED`, `INPROGRESS`, `COMPLETED`, `SUSPENDED`, `FAILED`, `STOPPED`, `QUEUED` or `WARNING`.
startTime	Number	Time, in milliseconds, when the job started.
numOfRecordsProcessed	Number	Number of records processed.
numOfRecordsFailed	Number	Number of records that failed to be imported.
numOfRecordsSucceeded	Number	Number of records successfully imported.

POST example

To import value mappings into a crosswalk based on the internal ID or alias, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/import HTTP/1.1
Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=file; filename=import-value-mappings.csv

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=importSettings
Content-Type: application/json;charset=UTF-8

{
   "delimiter":"COMMA",
   "textQualifier":"DOUBLE_QUOTE",
   "startingRow":null,
   "codepage":"UTF8",
   "dateFormat":"ISO",
   "containerType":"CROSSWALK",
   "containerKey":"ent_country",
   "repeatHeaders":true,
   "mappings":{
      "key":"sourcePKey",
      "from":"_from.id.sourcePKey",
      "to":"_to.id.sourcePKey"
   },
   "approvalOptions":{
      "priority":"LOW",
      "dueDate":"2024-12-30",
      "comments":"Please approve the request"
   }
```

```
}
--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
```

The CSV file might contain the following header rows and data rows:

```
key,from, to
key,from, to
AF_AFG,AF,AFG
AL_ALA,AL,ALA
ALB_ALB,ALB,ALB
DZ_DZA,DZ,DZA
AS_ASM,AS,ASM
```

The following sample response shows the status of the import job:

```
{
   "jobId":"32121c139c84a8edc8696c0c",
   "state":"INPROGRESS",
   "startTime":1631086520406,
   "numOfRecordsProcessed":100,
   "numOfRecordsFailed":25,
   "numOfRecordsSucceeded":75
}
```

Import hierarchy relationships (v3)

Imports hierarchy relationships for code values and top-level code values into a hierarchy, with custom headers as optional settings. You can use this resource to import hierarchy relationships into a hierarchy based on the internal ID or alias.

POST request

To import hierarchy relationships based on an internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v3/import/hierarchy
```

The request contains form-data with two parameters:

file

Specify a CSV file that contains the hierarchy relationships. The CSV file contains one header row with two columns: Code and ParentCode. You can list the related code values below the header.

For example, you might have the following relationships in the CSV file:

```
Code,ParentCode
C1,P1
C2,P2
C1,P3
```

If you import top-level code values, only use the Code column in the header row.

For example, you might have the following top-level code values:

```
Code
P1
P2
P3
```

importSettings

Specify the file specific configuration and container details.

The `importSettings` includes the following attributes:

Field	Type	Description
delimiter	String	Delimiter used to separate values. Value must be `COMMA`, `SEMICOLON`, `SPACE`, or `TAB`.
textQualifier	String	Symbol used to indicate where a text begins and ends. Value must be `DOUBLE_QUOTE`, `SINGLE_QUOTE`, or `NO_QUOTE`.
codepage	String	Code page used for the import file. Value must be `UTF8` or `MS_WINDOWS`.
hierarchyKey	String	Internal ID or alias of the hierarchy to which you want to import relationships.
parentCodeListKey	String	Internal ID or alias of the parent code list. **Note:** If you import top-level code values, you do not need to provide this attribute.
childCodeListKey	String	Internal ID or alias of the child code list.
startingRow	String	Line number from which to start importing data. By default, all rows are imported.
repeatHeaders	Boolean	Optional. Indicates whether the file has two headers. Value is `true` or `false`. Default is `true`.
mappings	Object	Optional. Maps the column headers in the CSV file with the target attributes of code lists in the key-value format. - You can map the user-defined column name to source field name if required. - You can either choose to ignore the mapping or map all the columns in an import CSV file. **Note:** The key represents the header in the CSV file, and the value represents the field names in Reference 360.

POST response

The response contains details of the import job.

The following table describes the attributes in the response body:

Field	Type	Description
jobId	String	ID of the job.
state	String	Status of the job. Values are `CREATED`, `INPROGRESS`, `COMPLETED`, `SUSPENDED`, `FAILED`, `STOPPED`, `QUEUED` or `WARNING`.
startTime	Number	Time, in milliseconds, when the job started.
numOfRecordsProcessed	Number	Number of records processed.
numOfRecordsFailed	Number	Number of records that failed to be imported.
numOfRecordsSucceeded	Number	Number of records successfully imported.

POST example

To import hierarchy relationships into a hierarchy based on the internal ID or alias, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/import/
hierarchy HTTP/1.1
Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=file; filename=import.csv
Content-Type: text/plain

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=importSettings
Content-Type: application/json;charset=UTF-8

{
   "delimiter":"COMMA",
   "textQualifier":"DOUBLE_QUOTE",
   "startingRow":0,
   "codepage":"UTF8",
   "hierarchyId":"eeb38899dd463330df844b8e",
   "childCodeListId":"3ebff6c89e03c41d5dd08acc",
   "parentCodeListId":"68c3ba1b4c3402a7cc18169a",
   "mappings":{
      "state":"Code",
      "country":"ParentCode"
   }
}--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
```

The CSV file might contain the following header row and data rows:

```
Code,ParentCode
C1,P1
C2,P2
C3,P3
```

To import top-level code values into a hierarchy, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/import/
hierarchy HTTP/1.1
Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=file; filename=import.csv
Content-Type: text/plain

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=importSettings
Content-Type: application/json;charset=UTF-8

{
   "delimiter":"COMMA",
   "textQualifier":"DOUBLE_QUOTE",
   "startingRow":0,
   "codepage":"UTF8",
   "hierarchyId":"eeb38899dd463330df844b8e",
   "childCodeListId":"3ebff6c89e03c41d5dd08acc",
   "parentCodeListId":"68c3ba1b4c3402a7cc18169a",
   "mappings":{
      "state":"Code",
      "country":"ParentCode"
   }
}--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
```

The CSV file might contain the following header row and data rows:

```
Code
P1
```

```
P2
P3
```

The following sample response shows that status of the import job:

```
{
   "jobId":"32121c139c84a8edc8696c0c",
   "state":"INPROGRESS",
   "startTime":1631086520353,
   "numOfRecordsProcessed":100,
   "numOfRecordsFailed":25,
   "numOfRecordsSucceeded":75
}
```

Get import job status (v3)

Retrieves the status of an import job based on the internal ID or alias.

GET request

To get the status of an import job based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v3/import/job/<job ID>
```

GET response

The response contains the details of the import job, such as the status of the import job, start time, and number of records processed for import.

The following table describes the attributes in the response body:

Field	Type	Description
jobId	String	ID of the job.
state	String	Status of the job. Values are `CREATED`, `INPROGRESS`, `COMPLETED`, `SUSPENDED`, `FAILED`, `STOPPED`, `QUEUED` or `WARNING`.
startTime	Number	Time, in milliseconds, when the job started.
numOfRecordsProcessed	Number	Number of records processed.
numOfRecordsFailed	Number	Number of records that failed to be imported.
numOfRecordsSucceeded	Number	Number of records successfully imported.

GET example

To get the status of an import job based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/import/job/
dd1b2018cb47cef99f8d0f42 HTTP/1.1
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the status of an import job:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 193

{
  "jobId":"dd1b2018cb47cef99f8d0f42",
  "state":"INPROGRESS",
```

```
  "startTime":1561367376330,
  "numOfRecordsProcessed":100,
  "numOfRecordsFailed":25,
  "numOfRecordsSucceeded":75
}
```

Get failed import job report (v3)

Retrieves an error report for a failed import job based on the internal ID or alias.

By default, the error report shows the first 100,000 records. To retrieve more records or to view the next page of records in the error report, use the query parameters.

GET request

To retrieve an error report for a failed import job based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v3/import/job/<job Id>/errorDetails
```

To retrieve the paginated error report, submit a GET request with the following query parameters appended to the URI:

```
/rdm-service/external/v3/import/job/<job Id>/errorDetails?pageNum=<page
number>&recordsPerPage=<records per page>
```

GET request query parameters

You can append the query parameters to the URI to retrieve paginated errors.

The following table lists the query parameters:

Parameter	Description
jobId	ID of the job.
pageNum	Optional. Page number to display. Default value is `0`.
recordsPerPage	Optional. Number of records to display per page. Default value is `100000`.

GET response

The response contains the error details, such as the line where the error occurred and the reason.

The following table describes the attributes in the response body:

Field	Type	Description
jobId	String	ID of the job.
entityType	String	Type of entity imported. Value is `BusinessEntity` or `Relationship`.
fileName	String	Name of the file. Value must end with the `.csv` file extension.
entityName	String	Name of the entity.
errorDetails	Array	Includes the error details.
lineNumber	Number	Line number where the error occurred.

Field	Type	Description
entitySourcePkey	String	Column identifier.
reasons	Array	Explanation of failure.

GET example

To retrieve an error report for a failed import job based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/import/job/
dae4301e9369c16c08bf0881/errorDetails HTTP/1.1
```

To retrieve the second page of records in a paginated error report with 100 records per page, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/import/job/
dae4301e9369c16c08bf0881/errorDetails?pageNum=2&recordsPerPage=100 HTTP/1.1
```

The following sample response shows the error report for a failed code values import job:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 395
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "jobId":"5984980d317a4b00cfe18880",
  "entityType":"BusinessEntity",
  "fileName":"import.csv",
  "entityName":"rdm.value.be.442ac56e11d5fc9bb11f6a3f",
  "errorDetails":[
    {
      "lineNumber":1,
      "entitySourcePkey":"Code-1101",
      "reasons":[
        "The code value INX does not exist in the picklist that is in the path Country.
Specify a code value from the picklist."
      ]
    }
  ]
}
```

The following sample response shows the error report for a failed value mappings import job:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 354

{
  "jobId":"dae4301e9369c16c08bf0881",
  "entityType":"Relationship",
  "fileName":"import.csv",
  "entityName":"rdm.crosswalk.rel.21ffd6b5f92d10c744acc27c.fc66c441288cf898c6fe5023",
  "errorDetails":[
    {
      "lineNumber":1,
      "entitySourcePkey":"AF_AFG",
      "reasons":[
        "The requested resource with ID 'AFG' does not exist."
      ]
    }
  ]
}
```

export version 1

Use this resource to export data to CSV or JSON format. You can export code values in a code list and value mappings in a crosswalk.

Export code values to a CSV file (v1)

Exports the code values in a code list to a CSV file.

POST request

To export code values in a code list, submit a POST request with the following URI:

```
/rdm-service/external/v1/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the code list to export:

Field	Type	Description
delimiter	String	Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE`, or `TAB`.
codepage	String	Code page used for the export file. Value must be `UTF8`.
decimalSeparator	String	Decimal separator used for numbers. Value must be `COMMA` or `DOT`.
thousandSeparator	String	Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE`, or `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
filename	String	Default name for the CSV file that you can download. The filename parameter is required to map the Content-Disposition header to indicate that you can save the API response as a CSV file.
containerType	String	Type of asset that contains code values. Value must be `codelist`.
containerId	String	The ID of the code list. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
filter	Object	Optional. Filter criteria for exporting filtered code values. For more information, see "Filter criteria" on page 535.
columns	Array	Optional. Attribute columns you want to export. If you don't specify attribute columns, the export includes all attribute columns.
excludeParentId	Boolean	Optional. Indicates whether to include the parent ID of code values in the CSV file. Value is `true` or `false`.

POST response

The response is a CSV file. The CSV file contains two header rows followed by data rows.

The CSV file contains the following columns: `status.key` and `effectiveDate`. Based on your data model, you might have additional columns in the file.

POST example

To export the code values in a code list, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX

{
  "delimiter":"SEMICOLON",
  "codepage":"UTF8",
  "decimalSeparator":"COMMA",
  "thousandSeparator":"DOT",
  "dateFormat":"ISO",
  "filename":"testdata.csv",
  "containerType":"codelist",
  "containerId":"c42dcf614044646d678bf5af",
  "filter":{
    "_and":[
      {
        "Name":{
          "_contains":"Jo"
        }
      }
    ]
  },
  "columns":[
    "Name",
    "Code"
  ],
  "excludeParentId":false
}
```

To export code values with `US` in the Name attribute, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX

{
  "delimiter": "SEMICOLON",
  "codepage": "UTF8",
  "decimalSeparator": "COMMA",
  "thousandSeparator": "DOT",
  "dateFormat": "ISO",
  "filename": "testdata.csv",
  "containerType": "codelist",
  "containerId": "9e42b406d59583f15838adf8",
  "filter": {
    "_and": [
      {
        "Name": {
          "_contains": "US"
        }
      }
    ]
  }
}
```

The following sample response shows the exported data in a CSV file:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
Content-Length: 130

Name,Code
Name0,Code0
Name1,Code1
Name2,Code2
Name3,Code3
Name4,Code4
Name5,Code5
Name6,Code6
Name7,Code7
Name8,Code8
Name9,Code9
```

For more information about exporting filtered code values, see "Exporting filtered code values" on page 542.

Export code values to JSON format (v1)

Exports the code values in a code list to the JSON format.

POST request

To export code values in a code list, submit a POST request with the following URI:

```
/rdm-service/external/v1/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/json`.

Use the following parameters in the request body to specify the code list to export:

Field	Type	Description
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
containerType	String	Type of asset that contains code values. Value must be `codelist`.
containerId	String	The ID of the code list. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

POST response

The response is in JSON format.

POST example

To export the code values in a code list, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/export HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "dateFormat" : "ISO",
  "containerType" : "codelist",
```

```
  "containerId" : "1c03a6555058fdd134f7f417"
}
```

The following sample response shows the exported data in JSON format:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 292

{
  "content":[
    {
      "Code":"UNIVERSITY-001",
      "children":[
        {
          "Code":"STUDENT-1001",
          "fields":{
            "Code":"STUDENT-1001",
            "Name":"BKImL"
          }
        }
      ],
      "fields":{
        "Code":"UNIVERSITY-001",
        "Name":"ABC University"
      }
    }
  ]
}
```

Export value mappings to a CSV file (v1)

Exports the value mappings in a crosswalk to a CSV file.

POST request

To export value mappings in a crosswalk, submit a POST request with the following URI:

```
/rdm-service/external/v1/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the crosswalk to export:

Field	Type	Description
delimiter	String	Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE`, or `TAB`.
codepage	String	Code page used for the export file. Value must be `UTF8`.
decimalSeparator	String	Decimal separator used for numbers. Value must be `COMMA` or `DOT`.
thousandSeparator	String	Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE`, or `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.

Field	Type	Description
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
containerType	String	Type of asset that contains value mappings. Value must be `crosswalk`.
containerId	String	The ID of the crosswalk. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

POST response

The response is a CSV file. The CSV file contains two header rows followed by data rows.

The CSV file uses the following columns: `status.key` and `effectiveDate`. Based on your data model, you might have additional columns in the file.

POST example

To export the value mappings in a crosswalk, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "delimiter" : "SEMICOLON",
  "codepage" : "UTF8",
  "decimalSeparator" : "COMMA",
  "thousandSeparator" : "DOT",
  "dateFormat" : "ISO",
  "filename" : "testdata.csv",
  "containerType" : "crosswalk",
  "containerId" : "5d123f6e4077c700010d59e4"
}
```

The following sample response shows the exported data in a CSV file:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
Content-Length: 578

sourcePKey,_from.id.sourcePKey,_to.id.sourcePKey
sourcePKey,_from.id.sourcePKey,_to.id.sourcePKey
sourceCode0_targetCode0,sourceCode0,targetCode0
sourceCode1_targetCode1,sourceCode1,targetCode1
sourceCode2_targetCode2,sourceCode2,targetCode2
sourceCode3_targetCode3,sourceCode3,targetCode3
sourceCode4_targetCode4,sourceCode4,targetCode4
sourceCode5_targetCode5,sourceCode5,targetCode5
sourceCode6_targetCode6,sourceCode6,targetCode6
sourceCode7_targetCode7,sourceCode7,targetCode7
sourceCode8_targetCode8,sourceCode8,targetCode8
sourceCode9_targetCode9,sourceCode9,targetCode9
```

Export value mappings to JSON format (v1)

Exports the value mappings in a crosswalk to the JSON format.

POST request

To export value mappings in a crosswalk, submit a POST request with the following URI:

```
/rdm-service/external/v1/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/json`.

Use the following parameters in the request body to specify the crosswalk to export:

Field	Type	Description
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
containerType	String	Type of asset that contains code values. Value must be `crosswalk`.
containerId	String	The ID of the crosswalk. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

POST response

The response is in JSON format.

POST example

To export the value mappings in a crosswalk, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/export HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "dateFormat" : "ISO",
  "containerType" : "crosswalk",
  "containerId" : "1c03a6555058fdd134f7f417"
}
```

The following sample response shows the exported data:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 234

{
  "content":[
    {
      "fields":{
        "toCode":"DEU",
        "fromCode":"DE"
      },
      "sourcePKey":"DE_DEU"
    },
    {
      "fields":{
        "toCode":"AFG",
        "fromCode":"AF"
```

```
        },
        "sourcePKey":"AF_AFG"
      }
    ]
}
```

export version 2

Use this resource to export data to CSV or JSON format. You can export code values in a code list, value mappings in a crosswalk, or relationships in a hierarchy.

Export code values to a CSV file (v2)

Exports the code values in a code list to a CSV file.

POST request

To export code values in a code list, submit a POST request with the following URI:

```
/rdm-service/external/v2/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the code list to export:

Field	Type	Description
delimiter	String	Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE`, or `TAB`.
codepage	String	Code page used for the export file. Value must be `UTF8`.
decimalSeparator	String	Decimal separator used for numbers. Value must be `COMMA` or `DOT`.
thousandSeparator	String	Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE`, or `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
containerType	String	Type of asset that contains code values. Value must be `codelist`.
containerId	String	The ID of the code list. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
filter	Object	Optional. Filter criteria for exporting filtered code values. For more information, see "Filter criteria for export version 2" on page 306.

Field	Type	Description
columns	Array	Optional. Attribute columns you want to export. If you don't specify attribute columns, the export includes all attribute columns.
fieldName	String	Optional. Name of attribute column to include in the exported file.
excludeParentId	Boolean	Optional. Indicates whether to include the parent ID of code values in the CSV file. Value is `true` or `false`.

POST response

The response is a CSV file. The CSV file contains two header rows followed by data rows.

The CSV file contains the following columns: `status.key` and `effectiveDate`. Based on your data model, you might have additional columns in the file.

POST example

To export the code values in a code list, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "delimiter":"SEMICOLON",
  "codepage":"UTF8",
  "decimalSeparator":"COMMA",
  "thousandSeparator":"DOT",
  "dateFormat":"ISO",
  "filename":"testdata.csv",
  "containerType":"codelist",
  "containerId":"dd318e233414acf05ffa451b",
  "columns":[
    {
      "fieldName":"Name"
    },
    {
      "fieldName":"Code"
    }
  ],
  "excludeParentId":false
}
```

To export code values with `Jo` in the Name attribute, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "delimiter":"SEMICOLON",
  "codepage":"UTF8",
  "decimalSeparator":"COMMA",
  "thousandSeparator":"DOT",
  "dateFormat":"ISO",
  "filename":"testdata.csv",
  "containerType":"codelist",
  "containerId":"dd318e233414acf05ffa451b",
  "filter":{
    "_and":[
      {
        "Name":{
          "_contains":"Jo"
```

```
        }
      }
    ]
  },
  "columns":[
    {
      "fieldName":"Name"
    },
    {
      "fieldName":"Code"
    }
  ],
  "excludeParentId":false
}
```

The following sample response shows the exported data in a CSV file:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
Content-Length: 130

Name,Code
Name0,Code0
Name1,Code1
Name2,Code2
Name3,Code3
Name4,Code4
Name5,Code5
Name6,Code6
Name7,Code7
Name8,Code8
Name9,Code9
```

For more information about exporting filtered code values, see "Exporting filtered code values" on page 542.

Export code values to JSON format (v2)

Exports the code values in a code list to the JSON format.

POST request

To export code values in a code list, submit a POST request with the following URI:

```
/rdm-service/external/v2/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/json`.

Use the following parameters in the request body to specify the code list to export:

Field	Type	Description
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
containerType	String	Type of asset that contains code values. Value must be `codelist`.

Field	Type	Description
containerId	String	The ID of the code list. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
filter	Object	Optional. Filter criteria for exporting filtered code values. For more information, see "Filter criteria for export version 2" on page 306.

POST response

The response is in JSON format.

POST example

To export the code values in a code list, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/export HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
   "dateFormat":"ISO",
   "containerType":"codelist",
   "containerId":"1c03a6555058fdd134f7f417",
   "filter":{
      "_and":[
         {
            "Name":{
               "_equals":"ABC University"
            }
         }
      ]
   }
}
```

The following sample response shows the exported data in JSON format:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 292

{
  "content":[
    {
      "Code":"UNIVERSITY-001",
      "children":[
        {
          "Code":"STUDENT-1001",
          "fields":{
            "Code":"STUDENT-1001",
            "Name":"BKImL"
          }
        }
      ],
      "fields":{
        "Code":"UNIVERSITY-001",
        "Name":"ABC University"
      }
    }
  ]
}
```

Export value mappings to a CSV file (v2)

Exports the value mappings in a crosswalk to a CSV file.

POST request

To export value mappings in a crosswalk, submit a POST request with the following URI:

```
/rdm-service/external/v2/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the crosswalk to export:

Field	Type	Description
delimiter	String	Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE`, or `TAB`.
codepage	String	Code page used for the export file. Value must be `UTF8`.
decimalSeparator	String	Decimal separator used for numbers. Value must be `COMMA` or `DOT`.
thousandSeparator	String	Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE`, or `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
containerType	String	Type of asset that contains value mappings. Value must be `crosswalk`.
containerId	String	The ID of the crosswalk. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

POST response

The response is a CSV file. The CSV file contains two header rows followed by data rows.

The CSV file uses the following columns: `status.key` and `effectiveDate`. Based on your data model, you might have additional columns in the file.

POST example

To export the value mappings in a crosswalk, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "delimiter" : "SEMICOLON",
  "codepage" : "UTF8",
  "decimalSeparator" : "COMMA",
  "thousandSeparator" : "DOT",
  "dateFormat" : "ISO",
  "filename" : "testdata.csv",
```

```
  "containerType" : "crosswalk",
  "containerId" : "5d123f6e4077c700010d59e4"
}
```

The following sample response shows the exported data in a CSV file:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
Content-Length: 578

sourcePKey,_from.id.sourcePKey,_to.id.sourcePKey
sourcePKey,_from.id.sourcePKey,_to.id.sourcePKey
sourceCode0_targetCode0,sourceCode0,targetCode0
sourceCode1_targetCode1,sourceCode1,targetCode1
sourceCode2_targetCode2,sourceCode2,targetCode2
sourceCode3_targetCode3,sourceCode3,targetCode3
sourceCode4_targetCode4,sourceCode4,targetCode4
sourceCode5_targetCode5,sourceCode5,targetCode5
sourceCode6_targetCode6,sourceCode6,targetCode6
sourceCode7_targetCode7,sourceCode7,targetCode7
sourceCode8_targetCode8,sourceCode8,targetCode8
sourceCode9_targetCode9,sourceCode9,targetCode9
```

Export value mappings to JSON format (v2)

Exports the value mappings in a crosswalk to the JSON format.

POST request

To export value mappings in a crosswalk, submit a POST request with the following URI:

```
/rdm-service/external/v2/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/json`.

Use the following parameters in the request body to specify the crosswalk to export:

Field	Type	Description
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
containerType	String	Type of asset that contains code values. Value must be `crosswalk`.
containerId	String	The ID of the crosswalk. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

POST response

The response is in JSON format.

POST example

To export the value mappings in a crosswalk, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/export HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

```
{
  "dateFormat" : "ISO",
  "containerType" : "crosswalk",
  "containerId" : "1c03a6555058fdd134f7f417"
}
```

The following sample response shows the exported data:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 234

{
  "content":[
    {
      "fields":{
        "toCode":"DEU",
        "fromCode":"DE"
      },
      "sourcePKey":"DE_DEU"
    },
    {
      "fields":{
        "toCode":"AFG",
        "fromCode":"AF"
      },
      "sourcePKey":"AF_AFG"
    }
  ]
}
```

Export hierarchies to a CSV file (v2)

Exports a hierarchy to a CSV file.

POST request

To export a hierarchy, submit a POST request with the following URI:

```
/rdm-service/external/v2/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the hierarchy to export:

Field	Type	Description
delimiter	String	Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE`, or `TAB`.
codepage	String	Code page used for the export file. Value must be `UTF8`.
decimalSeparator	String	Decimal separator used for numbers. Value must be `COMMA` or `DOT`.
thousandSeparator	String	Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE`, or `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.

Field	Type	Description
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
containerType	String	Type of asset that contains code values. Value must be `hierarchy`.
containerId	String	The ID of the hierarchy. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
filter	Object	Optional. Filter criteria for exporting filtered code values. For more information, see "Filter criteria for export version 2" on page 306.
columns	Array	Optional. Attribute columns you want to export. If you don't specify attribute columns, the export includes all attribute columns.
fieldName	String	Optional. Name of attribute column to include in the exported file.
codeListId	String	The ID of the code list associated to the attribute.
excludeParentId	Boolean	Optional. Indicates whether to include the parent ID of code values in the CSV file. Value is `true` or `false`.

POST response

The response is a CSV file. The CSV file contains two header rows followed by data rows.

The CSV file uses the following columns: `status.key` and `effectiveDate`. Based on your data model, you might have additional columns in the file.

POST example

To export the code values in a hierarchy, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
Content-Length: 456
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "delimiter":"SEMICOLON",
  "codepage":"UTF8",
  "decimalSeparator":"COMMA",
  "thousandSeparator":"DOT",
  "dateFormat":"ISO",
  "filename":"testdata.csv",
  "containerType":"hierarchy",
  "containerId":"7681b69b2f02a8f81389307d",
  "columns":[
    {
      "fieldName":"Name",
      "codeListId":"5495ed772814d1a29c588599"
    },
    {
      "fieldName":"Code",
      "codeListId":"5495ed772814d1a29c588599"
    }
  ],
  "excludeParentId":false
}
```

To export code values with `US` in the Name attribute, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
Content-Length: 346
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "delimiter":"SEMICOLON",
  "codepage":"UTF8",
  "decimalSeparator":"COMMA",
  "thousandSeparator":"DOT",
  "dateFormat":"ISO",
  "filename":"testdata.csv",
  "containerType":"hierarchy",
  "containerId":"350e0fcf85d589f5578226e9",
  "filter": {
    "_and": [
      {
        "Name": {
          "_contains": "US"
        }
      }
    ]
  }
}
```

The following sample response shows the exported data in a CSV file:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
Content-Length: 140

Name,Code
Name,Code
Name0,Code0
Name1,Code1
Name2,Code2
Name3,Code3
Name4,Code4
Name5,Code5
Name6,Code6
Name7,Code7
Name8,Code8
Name9,Code9
```

For more information about exporting filtered code values, see "Exporting filtered code values" on page 542.

Export hierarchies to JSON format (v2)

Exports a hierarchy to the JSON format.

POST request

To export a hierarchy, submit a POST request with the following URI:

```
/rdm-service/external/v2/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/json`.

Use the following parameters in the request body to specify the hierarchy to export:

Field	Type	Description
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
containerType	String	Type of asset that contains code values. Value must be `hierarchy`.
containerId	String	The ID of the code list. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
excludeParentId	Boolean	Optional. Indicates whether to include the parent ID of code values in the CSV file. Value is `true` or `false`.
includeAssetName	Boolean	Optional. Indicates whether to include the Name field of code lists in the JSON file. Value can be `true` or `false`. Default is `false`.

POST response

The response is in JSON format.

POST example

To export a hierarchy, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/export HTTP/1.1
Content-Type: application/json
Accept: application/json
Host: localhost:8080
Content-Length: 163
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
   "dateFormat":"ISO",
   "containerType":"hierarchy",
   "containerId":"cbf658b0c0cb82c8323ed9d5",
   "excludeParentId":false,
   "includeAssetName":true
}
```

The following sample response shows the exported data in JSON format:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 1114

{
   "content":[
      {
         "Code":"root1",
         "children":[
            {
               "Code":"child1_1",
               "codeListId":"Id of Child 1",
               "codeListName":"Name of Child 1",
               "fields":{
                  "Name":"Name of child 1.1",
                  "Code":"child1_1"
               }
            },
            {
```

```
                "Code":"child1_2",
                "codeListId":"Id of Child 1",
                "codeListName":"Name of Child 1",
                "fields":{
                   "Name":"Name of child 1.2",
                   "Code":"child1_2"
                }
             }
          ],
          "fields":{
             "Name":"Name of root 1",
             "Code":"root1"
          }
       },
       {
          "Code":"root2",
          "children":[
             {
                "Code":"child2_1",
                "codeListId":"Id of Child 2",
                "codeListName":"Name of Child 2",
                "fields":{
                   "Name":"Name of child 2.1",
                   "Code":"child2_1"
                }
             },
             {
                "Code":"child2_2",
                "codeListId":"Id of Child 2",
                "codeListName":"Name of Child 2",
                "fields":{
                   "Name":"Name of child 2.2",
                   "Code":"child2_2"
                }
             }
          ],
          "fields":{
             "Name":"Name of root 2",
             "Code":"root2"
          }
       }
    ]
}
```


Filter criteria for export version 2

You can filter code values in a code list and export the results.

You can filter values for the code attribute or the attributes that are configured as display attributes. For example, you can filter values for the code attribute with `001`.

Supported filter operators and filter values for fields

The filter operators depend on the field type of the attribute.

The following table describes the filter operators that you can use for each field type:

Field Type	Supported Filter Operators	Filter Values
Boolean	_equals _notEquals _isEmpty	Boolean
Decimal or Integer	_equals _notEquals _isEmpty _greaterThan _greaterThanEquals _lessThan _lessThanEquals	Number
String	_equals _notEquals _isEmpty _startsWith _endsWith _contains _doesNotContain	Text
Date	_equals _notEquals _isEmpty _from> _to _range	ISO 8601 date For example, `2019-12-24`.
DateTime	_equals _notEquals _isEmpty _from _to _range	ISO 8601 date and time For example, `2019-12-15T14:17:04Z`.
Reference Data	_equals _notEquals _isEmpty _in	Values in the Code attribute or values in the display attributes for the reference data.

Filter examples

To filter assets with fields that are equal to NY, you can use the following filter operator:

```
{
   "state":{
      "_equals":"NY"
   }
}
```

To filter assets with fields that are not equal to NY, you can use the following filter operator:

```
{
   "state":{
      "_notEquals:":"NY"
   }
}
```

To filter assets with numeric fields that are greater than 68, you can use the following filter operator:

```
{
   "age":{
      "_greaterThan":68
   }
}
```

To filter assets with numeric fields that are greater than or equal to 68, you can use the following filter operator:

```
{
   "age":{
      "_greaterThanEquals":68
   }
}
```

To filter assets with numeric fields that are less than 68, you can use the following filter operator:

```
{
   "age":{
      "_lessThan":68
   }
}
```

To filter assets with numeric fields that are less than or equal to 68, you can use the following filter operator:

```
{
   "age":{
      "_lessThanEquals":68
   }
}
```

To filter assets with fields that match any of the specified values, you can use the following filter operator:

```
{
   "state":{
      "_in":[
         "NY",
         "CA"
      ]
   }
}
```

To use multiple field operators, you can use the _and or _or operators.

To filter assets with fields that match all the specified values, you can use the following filter operator:

```
{
   "_and":[
      {
         "state":"CA"
      },
      {
         "age":68
      }
   ]
}
```

To filter assets with fields that match at least one of the specified values, you can use the following filter operator:

```
{
   "_or":[
      {
```

```
            "state":"CA"
         },
         {
            "age":68
         }
      ]
   }
```

When you use a comma in the code, the comma acts like an `_and` operator. For example, the following code filters the state "NY" and people of age "68":

```
   {
      {
         "state":"NY"
      },
      {
         "age":68
      }
   }
```

export version 3

Use this resource to export code values in a code list and value mappings in a crosswalk at a point in time to JSON and CSV formats.

Export code lists at a point in time to JSON format (v3)

Exports the code values in a code list at a point in time to the JSON format.

POST request

To export code values in a code list, submit a POST request with the following URI:

```
/rdm-service/external/v3/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/json`.

Use the following parameters in the request body to specify the code list to export:

Field	Type	Description
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
containerType	String	Type of asset that contains code values. Value must be `codelist`.
containerId	String	The ID of the code list. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
filter	Object	Optional. Filter criteria for exporting filtered code values. For more information, see "Filter criteria for export version 3 and 4" on page 326.

Field	Type	Description
excludeParentId	Boolean	Optional. Indicates whether to include the parent ID of code values in the JSON file. Value is `true` or `false`. Default is `false`.
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.
page	Number	Optional. Page number to display. Default is `0`.
sort	Array	Optional. Sort criteria for exporting sorted code values. Values must be comma-separated field names prefixed with their sort directions. The plus symbol (+) indicates ascending order, and the minus symbol (-) indicates descending order. For example, `"_sort": ["+Name","-Code"]` indicates to sort the names in the ascending order and codes in the descending order.
pit	String	Optional. Date or date and time to retrieve the point in time information about the code list. Use the following ISO formats for date, and date and time: - Date. yyyy-mm-dd - Date and time. yyyy-mm-dd HH-mm-ss
addLabelsForReferenceAttribute	Boolean	Optional. Displays values for reference and dependent attributes based on the display attributes of reference and dependent code lists, respectively.
includeMetadataAttributes	Boolean	Optional. Indicates whether to include metadata of the code list in the JSON format. Default is `false`.

POST response

The response is in the JSON format.

Note: When you export code values in a code list, the API response does not include the code values that do not contain relationships. The number of exported code values might result in mismatch with the number of imported code values.

POST example

To export the code values in a code list at a point in time, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/export HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
Content-Length: 356

{
  "dateFormat": "ISO",
  "containerType": "codelist",
  "containerId": "3dac5bc1b65697fba1ccf8f4",
  "filter": {
    "_and": [
      {
        "Name": {
          "_eq": "Global Business Services"
        }
      }
    ]
  },
  "excludeParentId": false,
  "pageSize": 10000,
```

```
  "page": 0,
  "sort": [
    "+Name",
    "+Code"
  ],
  "pit": "2025-05-05 12-00-00",
  "repeatHeaders": false,
  "addLabelsForReferenceAttribute": true,
  "includeMetadataAttributes": true
}
```

The following sample response shows the exported data in the JSON format:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 426

{
  "pageSize": 10,
  "page": 0,
  "totalNumberOfElements": 1,
  "numberOfElements": 1,
  "lastPage": true,
  "firstPage": true,
  "content": [
    {
      "Code": "GBS",
      "_meta": {
        "lastUpdateDate": "2025-05-05",
        "creationDate": "2024-09-02",
        "parentRelUpdatedBy": "",
        "parentRelLastUpdateDate": "",
        "createdBy": "user_10_24",
        "updatedBy": "user_10_24"
      },
      "fields": {
        "Description": "Description for GBS",
        "GeoRefAttr": "GER",
        "GeoRefAttr.label": "GERMANY-GER-GERMANYDESC",
        "Code": "GBS",
        "Name": "Global Business Services",
        "Business ID": "RDM0000123"
      }
    }
  ]
}
```

Export code lists at a point in time to CSV format (v3)

Exports the code values in a code list at a point in time to the CSV format.

POST request

To export code values in a code list, submit a POST request with the following URI:

```
/rdm-service/external/v3/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the code list to export:

Field	Type	Description
delimiter	String	Optional. Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE` or `TAB`. Default is `COMMA`.
codepage	String	Optional. Code page used for the export file. Value must be `UTF8` or `MS_WINDOWS`. Default is `UTF8`.
decimalSeparator	String	Optional. Decimal separator used for numbers. Value must be `COMMA` or `DOT`. Default is `DOT`.
thousandSeparator	String	Optional. Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE` or `NONE`. Default is `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
containerType	String	Type of asset that contains code values. Value must be `codelist`.
containerId	String	The ID of the code list. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
filter	Object	Optional. Filter criteria for exporting filtered code values. For more information, see "Filter criteria for export version 3 and 4" on page 326.
columns	Array	Optional. Attribute columns you want to export. If you do not specify attribute columns, the export includes all the attribute columns.
fieldName	String	Optional. Name of attribute column to include in the exported file.
excludeParentId	Boolean	Optional. Indicates whether to include the parent ID of code values in the CSV file. Value is `true` or `false`. Default is `false`.
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.
page	Number	Optional. Page number to display. Default is `0`.
sort	Array	Optional. Sort criteria for exporting sorted code values. Values must be comma-separated field names prefixed with their sort directions. The plus symbol (+) indicates ascending order, and the minus symbol (-) indicates descending order. For example, `"_sort": ["+Name","-Code"]` indicates to sort the names in the ascending order and codes in the descending order.

Field	Type	Description
pit	String	Optional. Date or date and time to retrieve the point in time information about the code list. Use the following ISO formats for date, and date and time: - Date. yyyy-mm-dd - Date and time. yyyy-mm-dd HH-mm-ss
repeatHeaders	Boolean	Optional. Indicates whether to populate headers in the CSV response. Value is `true` or `false`. Default is `true`.
addLabelsForReferenceAttribute	Boolean	Optional. Displays values for reference and dependent attributes based on the display attributes of reference and dependent code lists, respectively.
includeMetadataAttributes	Boolean	Optional. Indicates whether to include metadata of the code list in the CSV format. Default is `false`.

POST response

The response is a CSV file.

Note: When you export code values in a code list, the API response does not include the code values that do not contain relationships. The number of exported code values might result in mismatch with the number of imported code values.

POST example

To export the code values in a code list at a point in time, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
Content-Length: 580

{
   "delimiter":"COMMA",
   "codepage":"UTF8",
   "decimalSeparator":"COMMA",
   "thousandSeparator":"DOT",
   "dateFormat":"ISO",
   "filename":"testdata.csv",
   "containerType":"codelist",
   "containerId":"f6770b657b381038f9e9db3c",
   "filter":{
      "_and":[
         {
            "Name":{
               "_eq":"ABC University"
            }
         }
      ]
   },
   "columns":[
      {
         "fieldName":"Name"
      },
      {
         "fieldName":"Code"
      },
      {
         "fieldName":"Business ID"
      }
   ],
```

```
   "excludeParentId":false,
   "pageSize":10000,
   "page":0,
   "sort":[
      "+Name",
      "+Code"
   ],
   "pit":"2025-05-05 12-00-00",
   "repeatHeaders":false,
   "addLabelsForReferenceAttribute":true,
   "includeMetadataAttributes": false
}
```

The following sample response shows the exported data in the CSV format:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
RDM-PAGE: 0
RDM-TOTAL-NUMBER-OF-ELEMENTS: 10
RDM-NUMBER-OF-ELEMENTS: 10
RDM-PAGE-SIZE: 10000
RDM-FIRST-PAGE: true
RDM-LAST-PAGE: true
Content-Length: 950

status.key;effectiveDate;Name;Code;Description;Business ID
status.status.key;effectiveDate;Name;Code;Description;Business ID
;;ABC University;ABC;Description for ABC;RDM0000123
```

Export value mappings at a point in time to JSON format (v3)

Exports the value mappings in a crosswalk at a point in time to the JSON format.

POST request

To export value mappings in a crosswalk, submit a POST request with the following URI:

```
/rdm-service/external/v3/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/json`.

Use the following parameters in the request body to specify the crosswalk to export:

Field	Type	Description
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
containerType	String	Type of asset that contains code values. Value must be `crosswalk`.
containerId	String	ID of the crosswalk. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.

Field	Type	Description
page	Number	Optional. Page number to display. Default is `0`.
pit	String	Date to retrieve the point in time information about the crosswalk. Use the ISO format: yyyy-mm-dd.

POST response

The response is in the JSON format.

POST example

To export the value mappings in a crosswalk at a point in time, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/export HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
Content-Length: 220

{
   "dateFormat":"ISO",
   "containerType":"crosswalk",
   "containerId":"c27e43cad990f4e57361ae60",
   "pageSize":10000,
   "page":0,
   "pit":"2021-05-21",
   "repeatHeaders":false
}
```

The following sample response shows the exported data in the JSON format:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 366

{
   "pageSize":2,
   "page":0,
   "totalNumberOfElements":2,
   "numberOfElements":2,
   "lastPage":true,
   "firstPage":true,
   "content":[
      {
         "fields":{
            "toCode":"DEU",
            "fromCode":"DE"
         },
         "sourcePKey":"DE_DEU"
      },
      {
         "fields":{
            "toCode":"AFG",
            "fromCode":"AF"
         },
         "sourcePKey":"AF_AFG"
      }
   ]
}
```

Export value mappings at a point in time to CSV format (v3)

Exports the value mappings in a crosswalk at a point in time to the CSV format.

POST request

To export value mappings in a crosswalk, submit a POST request with the following URI:

```
/rdm-service/external/v3/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the crosswalk to export:

Field	Type	Description
delimiter	String	Optional. Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE` or `TAB`. Default is `COMMA`.
codepage	String	Optional. Code page used for the export file. Value must be `UTF8` or `MS_WINDOWS`. Default is `UTF8`.
decimalSeparator	String	Optional. Decimal separator used for numbers. Value must be `COMMA` or `DOT`. Default is `DOT`.
thousandSeparator	String	Optional. Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE` or `NONE`. Default is `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
filename	String	File name for the exported file. Value must end with the .csv file extension.
containerType	String	Type of asset that contains code values. Value must be `crosswalk`.
containerId	String	ID of the crosswalk. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

Field	Type	Description
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.
page	Number	Optional. Page number to display. Default is `0`.
pit	String	Date to retrieve the point in time information about the crosswalk. Use the ISO format: yyyy-mm-dd.
repeatHeaders	Boolean	Indicates whether to populate headers in the CSV response.

POST response

The response is in the CSV format.

POST example

To export the value mappings in a crosswalk at a point in time, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
Content-Length: 338

{
  "delimiter" : "SEMICOLON",
  "codepage" : "UTF8",
  "decimalSeparator" : "COMMA",
  "thousandSeparator" : "DOT",
  "dateFormat" : "ISO",
  "filename" : "testdata.csv",
  "containerType" : "crosswalk",
  "containerId" : "2029dbbcd5596d905d15fa83",
  "pageSize" : 10000,
  "page" : 0,
  "pit" : "2021-06-11",
  "repeatHeaders" : false
}
```

The following sample response shows the exported data in the CSV format:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
RDM-PAGE: 0
RDM-TOTAL-NUMBER-OF-ELEMENTS: 10
RDM-NUMBER-OF-ELEMENTS: 10
RDM-PAGE-SIZE: 10000
RDM-FIRST-PAGE: true
RDM-LAST-PAGE: true
Content-Length: 578

sourcePKey,_from.id.sourcePKey,_to.id.sourcePKey
sourcePKey,_from.id.sourcePKey,_to.id.sourcePKey
sourceCode0_targetCode0,sourceCode0,targetCode0
sourceCode1_targetCode1,sourceCode1,targetCode1
sourceCode2_targetCode2,sourceCode2,targetCode2
sourceCode3_targetCode3,sourceCode3,targetCode3
sourceCode4_targetCode4,sourceCode4,targetCode4
sourceCode5_targetCode5,sourceCode5,targetCode5
```

```
sourceCode6_targetCode6,sourceCode6,targetCode6
sourceCode7_targetCode7,sourceCode7,targetCode7
sourceCode8_targetCode8,sourceCode8,targetCode8
sourceCode9_targetCode9,sourceCode9,targetCode9
```

Export incoming crosswalk mappings to CSV format (v3)

Exports incoming crosswalk value mappings to CSV format.

POST request

To export incoming crosswalk value mappings, submit a POST request with the following URI:

```
/rdm-service/external/v3/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the incoming crosswalk value mappings to export:

Field	Type	Description
Delimiter	String	Optional. Delimiter used to separate values that belong to a mapping. Value must be `COMMA`, `SEMICOLON`, `SPACE`, `TAB`, `PIPE`, `COLON`, `SECTION`, `CIRCUMFLEX`, or `ASTERISK`. Default is `COMMA`.
codepage	String	Code page used for the export file. Value must be `UTF8` or `MS_WINDOWS`. Default is `UTF8`.
decimalSeparator	String	Optional. Decimal separator used for numbers. Value must be `COMMA` or `DOT`. Default is `DOT`.
thousandSeparator	String	Optional. Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE` or `NONE`. Default is `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US`. For mm/dd/yyyy format.
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
containerType	String	Type of asset that contains incoming value mappings from a codelist. Value must be `codelist_crosswalk`.
containerId	String	ID of the codelist from which the incoming crosswalks must be exported. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
columns	Array	Optional. Attribute columns you want to export. If you do not specify attribute columns, the export includes all the attribute columns.
fieldName	String	Optional. Name of attribute column to include in the exported file.
excludeParentId	Boolean	Optional. Indicates whether to include the parent ID of value mappings in the CSV file. Value is `true` or `false`. Default is `false`.

Field	Type	Description
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.
page	Number	Optional. Page number to display. Default is `0`.
pit	String	Optional. Date to retrieve the point in time information about the crosswalk. Use the ISO format: yyyy-mm-dd. Default value is the current timestamp in yyyy-mm-dd format.
repeatHeaders	Boolean	Optional. Indicates whether to populate headers in the CSV response. Value is `true` or `false`. Default is `true`.
direction	String	Use `incoming` to export incoming crosswalks.
mappingDelimiter	String	Optional. Delimiter used to separate values from different mappings. Value must be `COMMA`, `SEMICOLON`, `SPACE`, `TAB`, `PIPE`, `COLON`, `SECTION`, `CIRCUMFLEX`, `HYPHEN`, or `ASTERISK`. Default is `COMMA`.

POST response

The response is a CSV file.

POST example

To export the incoming crosswalk value mappings, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
Content-Length: 517

{
   "delimiter":"COMMA",
   "codepage":"UTF8",
   "decimalSeparator":"COMMA",
   "thousandSeparator":"DOT",
   "dateFormat":"ISO",
   "filename":"testdata.csv",
   "containerType":"codelist_crosswalk",
   "containerId":"9a34f404a1e836f3a16af3e9",
   "columns":[
      {
         "fieldName":"Name"
      },
      {
         "fieldName":"Code"
      }
   ],
   "excludeParentId":false,
   "pageSize":10000,
   "page":0,
   "pit":"2021-10-21",
   "repeatHeaders":false,
   "direction":"incoming",
   "mappingDelimiter":"PIPE"
}
```

The following sample response shows the exported data in the CSV format:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
```

```
Content-Length: 888

CodelistName,IncomingCodelist1Name,IncomingCodelist2Name
codelistCode0,incomingCodelist1Code0|incomingCodelist1Code1,incomingCodelist2Code0
codelistCode1,incomingCodelist1Code1|incomingCodelist1Code2,incomingCodelist2Code1
codelistCode2,incomingCodelist1Code2|incomingCodelist1Code3,incomingCodelist2Code2
codelistCode3,incomingCodelist1Code3|incomingCodelist1Code4,incomingCodelist2Code3
codelistCode4,incomingCodelist1Code4|incomingCodelist1Code5,incomingCodelist2Code4
codelistCode5,incomingCodelist1Code5|incomingCodelist1Code6,incomingCodelist2Code5
codelistCode6,incomingCodelist1Code6|incomingCodelist1Code7,incomingCodelist2Code6
codelistCode7,incomingCodelist1Code7|incomingCodelist1Code8,incomingCodelist2Code7
codelistCode8,incomingCodelist1Code8|incomingCodelist1Code9,incomingCodelist2Code8
codelistCode9,incomingCodelist1Code9|incomingCodelist1Code10,incomingCodelist2Code9
```

Export outgoing crosswalk mappings to CSV format (v3)

Exports outgoing crosswalk value mappings to CSV format.

POST request

To export outgoing crosswalk value mappings, submit a POST request with the following URI:

```
/rdm-service/external/v3/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the outgoing crosswalk value mappings to export:

Field	Type	Description
Delimiter	String	Optional. Delimiter used to separate values that belong to a mapping. Value must be `COMMA`, `SEMICOLON`, `SPACE`, `TAB`, `PIPE`, `COLON`, `SECTION`, `CIRCUMFLEX`, or `ASTERISK`. Default is `COMMA`.
codepage	String	Code page used for the export file. Value must be `UTF8` or `MS_WINDOWS`. Default is `UTF8`.
decimalSeparator	String	Optional. Decimal separator used for numbers. Value must be `COMMA` or `DOT`. Default is `DOT`.
thousandSeparator	String	Optional. Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE` or `NONE`. Default is `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US`. For mm/dd/yyyy format.
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
containerType	String	Type of asset that contains outgoing value mappings from a codelist. Value must be `codelist_crosswalk`.
containerId	String	ID of the codelist from which the outgoing crosswalks must be exported. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

Field	Type	Description
columns	Array	Optional. Attribute columns you want to export. If you do not specify attribute columns, the export includes all the attribute columns.
fieldName	String	Optional. Name of attribute column to include in the exported file.
excludeParentId	Boolean	Optional. Indicates whether to include the parent ID of value mappings in the CSV file. Value is `true` or `false`. Default is `false`.
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.
page	Number	Optional. Page number to display. Default is `0`.
pit	String	Optional. Date to retrieve the point in time information about the crosswalk. Use the ISO format: yyyy-mm-dd. Default value is the current timestamp in yyyy-mm-dd format.
repeatHeaders	Boolean	Optional. Indicates whether to populate headers in the CSV response. Value is `true` or `false`. Default is `true`.
direction	String	Use `outgoing` to export outgoing crosswalks.
mappingDelimiter	String	Optional. Delimiter used to separate values from different mappings. Value must be `COMMA`, `SEMICOLON`, `SPACE`, `TAB`, `PIPE`, `COLON`, `SECTION`, `CIRCUMFLEX`, `HYPHEN`, or `ASTERISK`. Default is `COMMA`.

POST response

The response is a CSV file.

POST example

To export the outgoing crosswalk value mappings, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
Content-Length: 888

{
   "delimiter":"COMMA",
   "codepage":"UTF8",
   "decimalSeparator":"COMMA",
   "thousandSeparator":"DOT",
   "dateFormat":"ISO",
   "filename":"testdata.csv",
   "containerType":"codelist_crosswalk",
   "containerId":"26ab763d543b157a4b12097f",
   "columns":[
      {
         "fieldName":"Name"
      },
      {
         "fieldName":"Code"
      }
   ],
   "excludeParentId":false,
   "pageSize":10000,
   "page":0,
   "pit":"2021-10-13",
```

```
   "repeatHeaders":false,
   "direction":"outgoing",
   "mappingDelimiter":"PIPE"
}
```

The following sample response shows the exported data in the CSV format:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
Content-Length: 888

CodelistName,OutgoingCodelist1Name,OutgoingCodelist2Name
codelistCode0,outgoingCodelist1Code0|outgoingCodelist1Code1,outgoingCodelist2Code0
codelistCode1,outgoingCodelist1Code1|outgoingCodelist1Code2,outgoingCodelist2Code1
codelistCode2,outgoingCodelist1Code2|outgoingCodelist1Code3,outgoingCodelist2Code2
codelistCode3,outgoingCodelist1Code3|outgoingCodelist1Code4,outgoingCodelist2Code3
codelistCode4,outgoingCodelist1Code4|outgoingCodelist1Code5,outgoingCodelist2Code4
codelistCode5,outgoingCodelist1Code5|outgoingCodelist1Code6,outgoingCodelist2Code5
codelistCode6,outgoingCodelist1Code6|outgoingCodelist1Code7,outgoingCodelist2Code6
codelistCode7,outgoingCodelist1Code7|outgoingCodelist1Code8,outgoingCodelist2Code7
codelistCode8,outgoingCodelist1Code8|outgoingCodelist1Code9,outgoingCodelist2Code8
codelistCode9,outgoingCodelist1Code9|outgoingCodelist1Code10,outgoingCodelist2Code9
```

Export value mappings to CSV format with display attributes in the header (v3)

Exports the value mappings in a crosswalk to CSV format with display attributes instead of codes, in the header.

POST request

To export value mappings in a crosswalk with display attributes in the header, submit a POST request with the following URI:

```
/rdm-service/external/v3/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the crosswalk value mappings to export:

Field	Type	Description
Delimiter	String	Optional. Delimiter used to separate values. Value must be `COMMA`, `SEMICOLON`, `SPACE`, `TAB`, `PIPE`, `COLON`, `SECTION`, `CIRCUMFLEX`, or `ASTERISK`. Default is `COMMA`.
codepage	String	Code page used for the export file. Value must be `UTF8` or `MS_WINDOWS`. Default is `UTF8`.
decimalSeparator	String	Optional. Decimal separator used for numbers. Value must be `COMMA` or `DOT`. Default is `DOT`.
thousandSeparator	String	Optional. Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE` or `NONE`. Default is `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US`. For mm/dd/yyyy format.

Field	Type	Description
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
containerType	String	Type of asset that contains value mappings. Value must be `crosswalk`.
containerId	String	ID of the crosswalk. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.
page	Number	Optional. Page number to display. Default is `0`.
pit	String	Optional. Date to retrieve the point in time information about the crosswalk. Use the ISO format: yyyy-mm-dd. Default value is the current timestamp in yyyy-mm-dd format.
repeatHeaders	Boolean	Optional. Indicates whether to populate headers in the CSV response for the requested page. Value is `true` or `false`. Default is `true`.
useDisplayAttributes	Boolean	Indicates to use display attributes in the header of the CSV file. Value must be `true`.
codelistNameAndFieldDelimiter	String	Optional. Delimiter used to separate the codelist name and the display attribute name in the header. Value must be `COMMA`, `SEMICOLON`, `SPACE`, `TAB`, `PIPE`, `COLON`, `SECTION`, `CIRCUMFLEX`, or `ASTERISK`. Default is `SPACE`.

POST response

The response is a CSV file.

POST example

To export value mappings in a crosswalk with display attributes in the header, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
Content-Length: 454

{
   "delimiter":"SEMICOLON",
   "codepage":"UTF8",
   "decimalSeparator":"COMMA",
   "thousandSeparator":"DOT",
   "dateFormat":"ISO",
   "filename":"testdata.csv",
   "containerType":"crosswalk",
   "containerId":"a8ef483f4f448a6ec273d547",
   "pageSize":10000,
   "page":0,
   "pit":"2021-11-09",
   "repeatHeaders":false,
   "useDisplayAttributes":true,
   "codelistNameAndFieldDelimiter":"SPACE"
}
```

The following sample response shows the exported data in the CSV format:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
RDM-PAGE: 0
RDM-TOTAL-NUMBER-OF-ELEMENTS: 10
RDM-NUMBER-OF-ELEMENTS: 10
RDM-PAGE-SIZE: 10000
RDM-FIRST-PAGE: true
RDM-LAST-PAGE: true
Content-Length: 560

SourceCodelist Name;SourceCodelist Code;TargetCodelist Name;TargetCodelist Code
sourceName0;sourceCode0;targetName0;targetCode0
sourceName1;sourceCode1;targetName1;targetCode1
sourceName2;sourceCode2;targetName2;targetCode2
sourceName3;sourceCode3;targetName3;targetCode3
sourceName4;sourceCode4;targetName4;targetCode4
sourceName5;sourceCode5;targetName5;targetCode5
sourceName6;sourceCode6;targetName6;targetCode6
sourceName7;sourceCode7;targetName7;targetCode7
sourceName8;sourceCode8;targetName8;targetCode8
sourceName9;sourceCode9;targetName9;targetCode9
```

Export value mappings to CSV format with code list names in the header (v3)

Export value mappings to CSV format with code list names in the header.

POST request

To export value mappings in a crosswalk with code list names in the header, submit a POST request with the following URI:

```
/rdm-service/external/v3/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the crosswalk value mappings to export:

Field	Type	Description
Delimiter	String	Optional. Delimiter used to separate values. Value must be `COMMA`, `SEMICOLON`, `SPACE`, `TAB`, `PIPE`, `COLON`, `SECTION`, `CIRCUMFLEX`, or `ASTERISK`. Default is `COMMA`.
codepage	String	Code page used for the export file. Value must be `UTF8` or `MS_WINDOWS`. Default is `UTF8`.
decimalSeparator	String	Optional. Decimal separator used for numbers. Value must be `COMMA` or `DOT`. Default is `DOT`.
thousandSeparator	String	Optional. Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE` or `NONE`. Default is `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US`. For mm/dd/yyyy format.
filename	String	File name for the exported file. Value must end with the `.csv` file extension.

Field	Type	Description
containerType	String	Type of asset that contains value mappings. Value must be `crosswalk`.
containerId	String	ID of the crosswalk. **Note:** You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.
page	Number	Optional. Page number to display. Default is `0`.
pit	String	Optional. Date to retrieve the point in time information about the crosswalk. Use the ISO format: yyyy-mm-dd. Default value is the current timestamp in yyyy-mm-dd format.
repeatHeaders	Boolean	Optional. Indicates whether to populate headers in the CSV response for the requested page. Value is `true` or `false`. Default is `true`.
useAssetNameInHeader	Boolean	Indicates to use the AssetName attribute in the header. Value must be `true`.

POST response

The response is a CSV file.

POST example

To export value mappings in a crosswalk with code list names in the header, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
Content-Length: 454

{
   "delimiter":"SEMICOLON",
   "codepage":"UTF8",
   "decimalSeparator":"COMMA",
   "thousandSeparator":"DOT",
   "dateFormat":"ISO",
   "filename":"testdata.csv",
   "containerType":"crosswalk",
   "containerId":"a8ef483f4f448a6ec273d547",
   "pageSize":10000,
   "page":0,
   "pit":"2021-11-09",
   "repeatHeaders":false,
   "useAssetNameInHeader":true,
}
```

The following sample response shows the exported data in the CSV format:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
RDM-PAGE: 0
RDM-TOTAL-NUMBER-OF-ELEMENTS: 10
RDM-NUMBER-OF-ELEMENTS: 10
RDM-PAGE-SIZE: 10000
RDM-FIRST-PAGE: true
RDM-LAST-PAGE: true
Content-Length: 560
```

```
SourceCodelist;TargetCodelist
sourceCode0;targetCode0
sourceCode1;targetCode1
sourceCode2;targetCode2
sourceCode3;targetCode3
sourceCode4;targetCode4
sourceCode5;targetCode5
sourceCode6;targetCode6
sourceCode7;targetCode7
sourceCode8;targetCode8
sourceCode9;targetCode9
```

Filter criteria for export version 3 and 4

You can export a filtered set of code values in a code list.

You can only filter values in the code attribute or values in attributes that are configured as display attributes. For example, you might want to filter values with `001` in the code attribute.

Supported filter operators and filter values for different field types

The filter operators depend on the field type of the attribute.

The following table describes the filter operators supported for each field type:

Field Type	Supported Filter Operators	Filter Values
Boolean	_eq (equal to) _ne (not equal to) _or (or) _and (and)	Boolean
Decimal or Integer	_eq (equal to) _ne (not equal to) _gt (greater than) _gte (greater than or equal to) _lt (less than) _lte (less than or equal to) _or (or) _and (and)	Number
String	_eq (equal to) _ne (not equal to) _or (or) _and (and)	Text

Field Type	Supported Filter Operators	Filter Values
Date	_eq (equal to) _ne (not equal to) _or (or) _and (and)	ISO 8601 date or date and time For example, `2019-12-24` or `2019-12-15T14:17:04Z`.
Reference Data	_eq (equal to) _ne (not equal to) _in (in) _or (or) _and (and)	Values in the Code attribute or values in the display attributes for the reference data.

Filter examples

To filter assets with fields that are equal to NY, you can use the following filter operator:

```
{
   "state":{
      "_eq":"NY"
   }
}
```

To filter assets with fields that are not equal to NY, you can use the following filter operator:

```
{
   "state":{
      "_ne":"NY"
   }
}
```

To filter assets with number fields that are greater than 68, you can use the following filter operator:

```
{
   "age":{
      "_gt":68
   }
}
```

To filter assets with number fields that are greater than or equal to 68, you can use the following filter operator:

```
{
   "age":{
      "_gte":68
   }
}
```

To filter assets with number fields that are less than 68, you can use the following filter operator:

```
{
   "age":{
      "_lt":68
   }
}
```

To filter assets with number fields that are less than or equal to 68, you can use the following filter operator:

```
{
   "age":{
      "_lte":68
   }
}
```

To filter assets with fields that match any of the specified values, you can use the following filter operator:

```
{
   "state":{
      "_in":[
         "NY",
         "CA"
      ]
   }
}
```

To filter assets with fields that match all the specified values, you can use the following filter operator:

```
{
   "_and":[
      {
         "state":"CA"
      },
      {
         "age":68
      }
   ]
}
```

To filter assets with fields that match at least one of the specified values, you can use the following filter operator:

```
{
   "_or":[
      {
         "state":"CA"
      },
      {
         "age":68
      }
   ]
}
```

When you use a comma in the code, the comma acts like an `_and` operator. For example, the following code filters the state "NY" and people of age "68":

```
{
   {
      "state":"NY"
   },
   {
      "age":68
   }
}
```

export version 4

Use this resource to export code values in a code list and value mappings in a crosswalk at a point in time to JSON and CSV formats based on its internal ID or alias.

Export code lists at point in time to JSON format (v4)

Exports the code values in a code list at a point in time to the JSON format based on the internal ID or alias of the code list.

POST request

To export code values in a code list based on the internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v4/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/json`.

Use the following parameters in the request body to specify the code list to export:

Field	Type	Description
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
delimiter	String	Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE`, or `TAB`.
textQualifier	String	Symbol used to indicate where a text begins and ends. Value must be `DOUBLE_QUOTE`, `SINGLE_QUOTE`, or `NO_QUOTE`.
startingRow	String	Line number from which to start importing data. By default, all rows are imported.
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
codepage	String	Code page used for the export file. Value must be `UTF8`.
containerType	String	Type of asset that contains code values. Value must be `codelist`.
containerKey	String	Internal ID or alias of a code list.
filter	Object	Optional. Filter criteria for exporting filtered code values. For more information, see "Filter criteria for export version 3 and 4" on page 326.
excludeParentId	Boolean	Optional. Indicates whether to include the parent ID of code values in the JSON file. Value is `true` or `false`. Default is `false`.
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.

Field	Type	Description
page	Number	Optional. Page number to display. Default is `0`.
sort	Array	Optional. Sort criteria for exporting sorted code values. Values must be comma-separated field names prefixed with their sort directions. The plus symbol (+) indicates ascending order, and the minus symbol (-) indicates descending order. For example, `"_sort": ["+Name","-Code"]` indicates to sort the names in the ascending order and codes in the descending order.
pit	String	Optional. Date or date and time to retrieve the point in time information about the code list. Use the following ISO formats for date, and date and time: - Date. yyyy-mm-dd - Date and time. yyyy-mm-dd HH-mm-ss
addLabelsForReferenceAttribute	Boolean	Optional. Displays values for reference and dependent attributes based on the display attributes of reference and dependent code lists, respectively.
includeMetadataAttributes	Boolean	Optional. Indicates whether to include metadata of the code list in the JSON format. Default is `false`.

POST response

The response is in the JSON format.

Note: When you export code values in a code list, the API response does not include the code values that don't contain relationships. The number of exported code values might result in mismatch with the number of imported code values.

POST example

To export the code values in a code list at a point in time based on the internal ID or alias, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v4/export HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
Content-Length: 356

{
  "delimiter": "SEMICOLON",
  "textQualifier": "DOUBLE_QUOTE",
  "startingRow": "0",
  "codepage": "UTF8",
  "dateFormat": "ISO",
  "filename": "export.json",
  "containerType": "CODELIST",
  "containerKey": "ent_country",
  "pageSize": 3,
  "page": 0,
  "pit": "2025-06-18 22-30-24"
}
```

The following sample response shows the exported data in the JSON format:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 426
{
  "pageSize": 3,
```

```
"page": 0,
"totalNumberOfElements": 251,
"numberOfElements": 3,
"lastPage": false,
"firstPage": true,
"content": [
  {
      "Code": "AC",
      "fields": {
        "_localizedContent": [
          {
            "Name": "l'Ascension",
            "_language": {
              "Code": "fr-FR"
            }
          },
          {
            "Name": "Ascension",
            "_language": {
              "Code": "en-US"
            }
          },
          {
            "Name": "Ascensão",
            "_language": {
              "Code": "pt-BR"
            }
          }
        ],
        "Code": "AC"
      }
    },
    {
      "Code": "AD",
      "fields": {
        "_localizedContent": [
          {
            "Name": "L'Andorre",
            "_language": {
              "Code": "fr-FR"
            }
          },
          {
            "Name": "Andorra",
            "_language": {
              "Code": "en-US"
            }
          },
          {
            "Name": "Andorra",
            "_language": {
              "Code": "pt-BR"
            }
          }
        ],
        "Code": "AD"
      }
    },
    {
      "Code": "AE",
      "fields": {
        "_localizedContent": [
          {
            "Name": "Émirats arabes unis",
            "_language": {
              "Code": "fr-FR"
            }
          },
          {
            "Name": "United Arab Emirates",
            "_language": {
```

```
            "Code": "en-US"
          }
        },
        {
          "Name": "Emirados Árabes Unidos",
          "_language": {
            "Code": "pt-BR"
          }
        }
      ],
      "Code": "AE"
    }
  }
 ]
}
```

Export code lists at point in time to CSV format (v4)

Exports the code values in a code list at a point in time to the CSV format based on the internal ID or alias.

POST request

To export code values in a code list based on the internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v4/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the code list to export:

Field	Type	Description
delimiter	String	Optional. Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE` or `TAB`. Default is `COMMA`.
codepage	String	Optional. Code page used for the export file. Value must be `UTF8` or `MS_WINDOWS`. Default is `UTF8`.
decimalSeparator	String	Optional. Decimal separator used for numbers. Value must be `COMMA` or `DOT`. Default is `DOT`.
thousandSeparator	String	Optional. Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE` or `NONE`. Default is `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
containerType	String	Type of asset that contains code values. Value must be `codelist`.
containerKey	String	Internal ID or alias of a code list.

Field	Type	Description
filter	Object	Optional. Filter criteria for exporting filtered code values. For more information, see "Filter criteria for export version 3 and 4" on page 326.
columns	Array	Optional. Attribute columns you want to export. If you don't specify attribute columns, the export includes all the attribute columns.
fieldName	String	Optional. Name of attribute column to include in the exported file.
excludeParentId	Boolean	Optional. Indicates whether to include the parent ID of code values in the CSV file. Value is `true` or `false`. Default is `false`.
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.
page	Number	Optional. Page number to display. Default is `0`.
sort	Array	Optional. Sort criteria for exporting sorted code values. Values must be comma-separated field names prefixed with their sort directions. The plus symbol (+) indicates ascending order, and the minus symbol (-) indicates descending order. For example, `"_sort": ["+Name","-Code"]` indicates to sort the names in the ascending order and codes in the descending order.
pit	String	Optional. Date or date and time to retrieve the point in time information about the code list. Use the following ISO formats for date, and date and time: - Date. yyyy-mm-dd - Date and time. yyyy-mm-dd HH-mm-ss
repeatHeaders	Boolean	Optional. Indicates whether to populate headers in the CSV response. Value is `true` or `false`. Default is `true`.
addLabelsForReferenceAttribute	Boolean	Optional. Displays values for reference and dependent attributes based on the display attributes of reference and dependent code lists, respectively.
includeMetadataAttributes	Boolean	Optional. Indicates whether to include metadata of the code list in the CSV format. Default is `false`.

POST response

The response is a CSV file.

Note: When you export code values in a code list, the API response does not include the code values that don't contain relationships. The number of exported code values might result in mismatch with the number of imported code values.

POST example

To export the code values in a code list at a point in time based on the internal ID or alias, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v4/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
Content-Length: 580
```

```
{
   "delimiter":"COMMA",
   "codepage":"UTF8",
   "decimalSeparator":"COMMA",
   "thousandSeparator":"DOT",
   "dateFormat":"ISO",
   "filename":"testdata.csv",
   "containerType":"codelist",
   "containerKey":"ABC_Int",
   "filter":{
      "_and":[
         {
            "Name":{
               "_eq":"ABC University"
            }
         }
      ]
   },
   "columns":[
      {
         "fieldName":"Name"
      },
      {
         "fieldName":"Code"
      },
      {
         "fieldName":"Business ID"
      }
   ],
   "excludeParentId":false,
   "pageSize":10000,
   "page":0,
   "sort":[
      "+Name",
      "+Code"
   ],
   "pit":"2025-05-05 12-00-00",
   "repeatHeaders":false,
   "addLabelsForReferenceAttribute":true,
   "includeMetadataAttributes": false
}
```

The following sample response shows the exported data in the CSV format:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
RDM-PAGE: 0
RDM-TOTAL-NUMBER-OF-ELEMENTS: 10
RDM-NUMBER-OF-ELEMENTS: 10
RDM-PAGE-SIZE: 10000
RDM-FIRST-PAGE: true
RDM-LAST-PAGE: true
Content-Length: 950

_language;status.key;effectiveDate;_localizedContent.Name;Code;Description
_language;status.status.key;effectiveDate;_localizedContent.Name;Code;Description
fr-FR;;;l'Ascension;AC;
en-US;;;Ascension;AC;
pt-BR;;;Ascensão;AC;
fr-FR;;;L'Andorre;AD;
en-US;;;Andorra;AD;
pt-BR;;;Andorra;AD;
fr-FR;;;Émirats arabes unis;AE;
en-US;;;United Arab Emirates;AE;
fr-FR;;;Emirados Árabes Unidos;AE;
```

Export value mappings at point in time to JSON format (v4)

Exports the value mappings in a crosswalk at a point in time to the JSON format based on the internal ID or alias.

POST request

To export value mappings in a crosswalk based on the internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v4/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/json`.

Use the following parameters in the request body to specify the crosswalk to export:

Field	Type	Description
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
containerType	String	Type of asset that contains code values. Value must be `crosswalk`.
containerKey	String	Internal ID or alias of a crosswalk.
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.
page	Number	Optional. Page number to display. Default is `0`.
pit	String	Date to retrieve the point in time information about the crosswalk. Use the ISO format: yyyy-mm-dd.

POST response

The response is in the JSON format.

POST example

To export the value mappings in a crosswalk at a point in time based on the internal ID or alias, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v4/export HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
Content-Length: 220

{
  "delimiter": "SEMICOLON",
  "textQualifier": "DOUBLE_QUOTE",
  "startingRow": "0",
  "codepage": "UTF8",
  "dateFormat": "ISO",
  "filename": "export.json",
  "containerType": "crosswalk",
  "containerKey": "salesforce_country_enterprise_country",
  "pageSize": 10000,
  "page": 0,
  "pit": "2025-06-18 22-40-24",
```

```
  "repeatHeaders": false
}
```

The following sample response shows the exported data in the JSON format:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 366

{
  "pageSize": 10000,
  "page": 0,
  "totalNumberOfElements": 3,
  "numberOfElements": 3,
  "lastPage": true,
  "firstPage": true,
  "content": [
    {
      "fields": {
        "toCode": "FR",
        "fromCode": "SF_FR"
      },
      "sourcePKey": "SF_FR_FR"
    },
    {
      "fields": {
        "toCode": "IN",
        "fromCode": "SF_IN"
      },
      "sourcePKey": "SF_IN_IN"
    },
    {
      "fields": {
        "toCode": "US",
        "fromCode": "SF_US"
      },
      "sourcePKey": "SF_US_US"
    }
  ]
}
```

Export value mappings at point in time to CSV format (v4)

Exports the value mappings in a crosswalk at a point in time to the CSV format based on the internal ID or alias.

POST request

To export value mappings in a crosswalk based on the internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v4/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the crosswalk to export:

Field	Type	Description
delimiter	String	Optional. Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE` or `TAB`. Default is `COMMA`.
codepage	String	Optional. Code page used for the export file. Value must be `UTF8` or `MS_WINDOWS`. Default is `UTF8`.
decimalSeparator	String	Optional. Decimal separator used for numbers. Value must be `COMMA` or `DOT`. Default is `DOT`.
thousandSeparator	String	Optional. Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE` or `NONE`. Default is `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
filename	String	File name for the exported file. Value must end with the .csv file extension.
containerType	String	Type of asset that contains code values. Value must be `crosswalk`.
containerKey	String	Internal ID or alias of a crosswalk.
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.
page	Number	Optional. Page number to display. Default is `0`.
pit	String	Date to retrieve the point in time information about the crosswalk. Use the ISO format: yyyy-mm-dd.
repeatHeaders	Boolean	Indicates whether to populate headers in the CSV response.

POST response

The response is in the CSV format.

POST example

To export the value mappings in a crosswalk at a point in time based on the internal ID or alias, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v4/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX
Content-Length: 338

{
  "delimiter": "SEMICOLON",
  "textQualifier": "DOUBLE_QUOTE",
  "startingRow": "0",
  "codepage": "UTF8",
  "dateFormat": "ISO",
  "filename": "export.csv",
  "containerType": "crosswalk",
  "containerKey": "salesforce_country_enterprise_country",
  "pageSize": 10000,
  "page": 0,
  "pit": "2025-06-18 22-40-24",
  "repeatHeaders": false
}
```

The following sample response shows the exported data in the CSV format:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
RDM-PAGE: 0
RDM-TOTAL-NUMBER-OF-ELEMENTS: 10
RDM-NUMBER-OF-ELEMENTS: 10
RDM-PAGE-SIZE: 10000
RDM-FIRST-PAGE: true
RDM-LAST-PAGE: true
Content-Length: 178

sourcePKey,_from.id.sourcePKey,_to.id.sourcePKey
sourcePKey,_from.id.sourcePKey,_to.id.sourcePKey
SF_FR;FR
SF_IN;IN
SF_US;US
```

Export incoming crosswalk mappings (v4)

Exports incoming crosswalk value mappings to CSV format based on the internal ID or alias.

POST request

To export incoming crosswalk value mappings based on the internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v4/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the incoming crosswalk value mappings to export:

Field	Type	Description
Delimiter	String	Optional. Delimiter used to separate values that belong to a mapping. Value must be `COMMA`, `SEMICOLON`, `SPACE`, `TAB`, `PIPE`, `COLON`, `SECTION`, `CIRCUMFLEX`, or `ASTERISK`. Default is `COMMA`.
codepage	String	Code page used for the export file. Value must be `UTF8` or `MS_WINDOWS`. Default is `UTF8`.
decimalSeparator	String	Optional. Decimal separator used for numbers. Value must be `COMMA` or `DOT`. Default is `DOT`.
thousandSeparator	String	Optional. Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE` or `NONE`. Default is `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US`. For mm/dd/yyyy format.
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
containerType	String	Type of asset that contains incoming value mappings from a code list. Value must be `codelist_crosswalk`.
containerKey	String	Internal ID or alias of a code list from which the incoming crosswalks must be exported.
columns	Array	Optional. Attribute columns you want to export. If you don't specify attribute columns, the export includes all the attribute columns.
fieldName	String	Optional. Name of attribute column to include in the exported file.
excludeParentId	Boolean	Optional. Indicates whether to include the parent ID of value mappings in the CSV file. Value is `true` or `false`. Default is `false`.
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.
page	Number	Optional. Page number to display. Default is `0`.
pit	String	Optional. Date to retrieve the point in time information about the crosswalk. Use the ISO format: yyyy-mm-dd. Default value is the current timestamp in yyyy-mm-dd format.
repeatHeaders	Boolean	Optional. Indicates whether to populate headers in the CSV response. Value is `true` or `false`. Default is `true`.
direction	String	Use `incoming` to export incoming crosswalks.
mappingDelimiter	String	Optional. Delimiter used to separate values from different mappings. Value must be `COMMA`, `SEMICOLON`, `SPACE`, `TAB`, `PIPE`, `COLON`, `SECTION`, `CIRCUMFLEX`, `HYPHEN`, or `ASTERISK`. Default is `COMMA`.

POST response

The response is a CSV file.

POST example

To export the incoming crosswalk value mappings based on an internal ID or alias, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v4/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
Content-Length: 517

{
  "delimiter": "SEMICOLON",
  "textQualifier": "DOUBLE_QUOTE",
  "startingRow": "0",
  "pageSize": "5",
  "codepage": "UTF8",
  "dateFormat": "ISO",
  "filename": "export.csv",
  "containerType": "codelist_crosswalk",
  "containerKey": "ent_country",
  "direction": "INCOMING"
}
```

The following sample response shows the exported data in the CSV format:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
Content-Length: 888

CodelistName,IncomingCodelist1Name,IncomingCodelist2Name
Enterprise Country;ISO Country Codes;Salesforce Country
IN;ISO_IN;SF_IN
DE;ISO_DE;SF_DE
FR;ISO_FR;SF_FR
CN;;ISO_CN;SF_CN
```

Export outgoing crosswalk mappings (v4)

Exports outgoing crosswalk value mappings to CSV format based on the internal ID or alias.

POST request

To export outgoing crosswalk value mappings based on the internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v4/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the outgoing crosswalk value mappings to export:

Field	Type	Description
Delimiter	String	Optional. Delimiter used to separate values that belong to a mapping. Value must be `COMMA`, `SEMICOLON`, `SPACE`, `TAB`, `PIPE`, `COLON`, `SECTION`, `CIRCUMFLEX`, or `ASTERISK`. Default is `COMMA`.
codepage	String	Code page used for the export file. Value must be `UTF8` or `MS_WINDOWS`. Default is `UTF8`.
decimalSeparator	String	Optional. Decimal separator used for numbers. Value must be `COMMA` or `DOT`. Default is `DOT`.
thousandSeparator	String	Optional. Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE` or `NONE`. Default is `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US`. For mm/dd/yyyy format.
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
containerType	String	Type of asset that contains outgoing value mappings from a code list. Value must be `codelist_crosswalk`.
containerKey	String	Internal ID or alias of a code list from which the outgoing crosswalks must be exported.
columns	Array	Optional. Attribute columns you want to export. If you don't specify attribute columns, the export includes all the attribute columns.
fieldName	String	Optional. Name of attribute column to include in the exported file.
excludeParentId	Boolean	Optional. Indicates whether to include the parent ID of value mappings in the CSV file. Value is `true` or `false`. Default is `false`.
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.
page	Number	Optional. Page number to display. Default is `0`.
pit	String	Optional. Date to retrieve the point in time information about the crosswalk. Use the ISO format: yyyy-mm-dd. Default value is the current timestamp in yyyy-mm-dd format.
repeatHeaders	Boolean	Optional. Indicates whether to populate headers in the CSV response. Value is `true` or `false`. Default is `true`.
direction	String	Use `outgoing` to export outgoing crosswalks.
mappingDelimiter	String	Optional. Delimiter used to separate values from different mappings. Value must be `COMMA`, `SEMICOLON`, `SPACE`, `TAB`, `PIPE`, `COLON`, `SECTION`, `CIRCUMFLEX`, `HYPHEN`, or `ASTERISK`. Default is `COMMA`.

POST response

The response is a CSV file.

POST example

To export the outgoing crosswalk value mappings based on the internal ID or alias, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v4/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
Content-Length: 888
{
  "delimiter": "SEMICOLON",
  "textQualifier": "DOUBLE_QUOTE",
  "startingRow": "0",
  "pageSize": "5",
  "codepage": "UTF8",
  "dateFormat": "ISO",
  "filename": "export.csv",
  "containerType": "codelist_crosswalk",
  "containerKey": "sf_country",
  "direction": "OUTGOING"
}
```

The following sample response shows the exported data in the CSV format:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
Content-Length: 888

CodelistName,OutgoingCodelist1Name,OutgoingCodelist2Name
Salesforce Country;Enterprise Country
SF_IN;IN
SF_US;US
SF_FR;FR
```

Export value mappings to CSV format with display attributes in the header (v4)

Exports the value mappings in a crosswalk to CSV format with display attributes instead of codes, in the header. Use this resource to export value mappings in a crosswalk with display attributes in the header based on the internal ID or alias.

POST request

To export value mappings in a crosswalk with display attributes in the header based on the internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v4/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the crosswalk value mappings to export:

Field	Type	Description
Delimiter	String	Optional. Delimiter used to separate values. Value must be `COMMA`, `SEMICOLON`, `SPACE`, `TAB`, `PIPE`, `COLON`, `SECTION`, `CIRCUMFLEX`, or `ASTERISK`. Default is `COMMA`.
codepage	String	Code page used for the export file. Value must be `UTF8` or `MS_WINDOWS`. Default is `UTF8`.
decimalSeparator	String	Optional. Decimal separator used for numbers. Value must be `COMMA` or `DOT`. Default is `DOT`.
thousandSeparator	String	Optional. Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE` or `NONE`. Default is `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US`. For mm/dd/yyyy format.
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
containerType	String	Type of asset that contains value mappings. Value must be `crosswalk`.
containerKey	String	Internal ID or alias of a crosswalk.
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.
page	Number	Optional. Page number to display. Default is `0`.
pit	String	Optional. Date to retrieve the point in time information about the crosswalk. Use the ISO format: yyyy-mm-dd. Default value is the current timestamp in yyyy-mm-dd format.
repeatHeaders	Boolean	Optional. Indicates whether to populate headers in the CSV response for the requested page. Value is `true` or `false`. Default is `true`.
useDisplayAttributes	Boolean	Indicates to use display attributes in the header of the CSV file. Value must be `true`.
codelistNameAndFieldDelimiter	String	Optional. Delimiter used to separate the codelist name and the display attribute name in the header. Value must be `COMMA`, `SEMICOLON`, `SPACE`, `TAB`, `PIPE`, `COLON`, `SECTION`, `CIRCUMFLEX`, or `ASTERISK`. Default is `SPACE`.

POST response

The response is a CSV file.

POST example

To export value mappings in a crosswalk with display attributes in the header based on the internal ID or alias, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v4/export HTTP/1.1
Content-Type: application/json
```

```
Accept: application/octet-stream
Content-Length: 454

{
  "delimiter": "SEMICOLON",
  "textQualifier": "DOUBLE_QUOTE",
  "startingRow": "0",
  "codepage": "UTF8",
  "dateFormat": "ISO",
  "filename": "export.csv",
  "containerType": "crosswalk",
  "containerKey": "salesforce_country_enterprise_country",
  "pageSize": 10000,
  "page": 0,
  "repeatHeaders": true,
  "useDisplayAttributes": true,
  "useAssetNameInHeader": false
}
```

The following sample response shows the exported data in the CSV format:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
RDM-PAGE: 0
RDM-TOTAL-NUMBER-OF-ELEMENTS: 10
RDM-NUMBER-OF-ELEMENTS: 10
RDM-PAGE-SIZE: 10000
RDM-FIRST-PAGE: true
RDM-LAST-PAGE: true
Content-Length: 560

SourceCodelist Name;TargetCodelist Name
Salesforce Country Name;Enterprise Country Name
SF France;France
SF India;India
SF United States;United States
```

Export value mappings to CSV format with code list names in CSV header (v4)

Export value mappings to CSV format with code list names in the header based on the internal ID or alias.

POST request

To export value mappings in a crosswalk with code list names in the header based on the internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v4/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the crosswalk value mappings to export:

Field	Type	Description
delimiter	String	Optional. Delimiter used to separate values. Value must be `COMMA`, `SEMICOLON`, `SPACE`, `TAB`, `PIPE`, `COLON`, `SECTION`, `CIRCUMFLEX`, or `ASTERISK`. Default is `COMMA`.
codepage	String	Code page used for the export file. Value must be `UTF8` or `MS_WINDOWS`. Default is `UTF8`.

Field	Type	Description
decimalSeparator	String	Optional. Decimal separator used for numbers. Value must be `COMMA` or `DOT`. Default is `DOT`.
thousandSeparator	String	Optional. Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE` or `NONE`. Default is `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US`. For mm/dd/yyyy format.
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
containerType	String	Type of asset that contains value mappings. Value must be `crosswalk`.
containerKey	String	Internal ID or alias of a crosswalk.
pageSize	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.
page	Number	Optional. Page number to display. Default is `0`.
pit	String	Optional. Date to retrieve the point in time information about the crosswalk. Use the ISO format: yyyy-mm-dd. Default value is the current timestamp in yyyy-mm-dd format.
repeatHeaders	Boolean	Optional. Indicates whether to populate headers in the CSV response for the requested page. Value is `true` or `false`. Default is `true`.
useAssetNameInHeader	Boolean	Indicates to use the AssetName attribute in the header. Value must be `true`.

POST response

The response is a CSV file.

POST example

To export value mappings in a crosswalk with code list names in the header based on the internal ID or alias, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v4/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
Content-Length: 454

{
  "delimiter": "SEMICOLON",
  "textQualifier": "DOUBLE_QUOTE",
  "startingRow": "0",
  "codepage": "UTF8",
  "dateFormat": "ISO",
  "filename": "export.csv",
  "containerType": "crosswalk",
  "containerKey": "salesforce_country_enterprise_country",
  "pageSize": 10000,
  "page": 0,
  "repeatHeaders": true,
  "useDisplayAttributes": false,
  "useAssetNameInHeader": true
}
```

The following sample response shows the exported data in the CSV format:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
RDM-PAGE: 0
RDM-TOTAL-NUMBER-OF-ELEMENTS: 10
RDM-NUMBER-OF-ELEMENTS: 10
RDM-PAGE-SIZE: 10000
RDM-FIRST-PAGE: true
RDM-LAST-PAGE: true
Content-Length: 560

SourceCodelist;TargetCodelist
Salesforce Country;Enterprise Country
SF_FR;FR
SF_IN;IN
SF_US;US
```

Export hierarchies to a CSV file (v4)

Exports a hierarchy to a CSV file based on the internal ID or alias.

POST request

To export a hierarchy based on the internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v4/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request body to specify the hierarchy to export:

Field	Type	Description
delimiter	String	Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE`, or `TAB`.
codepage	String	Code page used for the export file. Value must be `UTF8`.
decimalSeparator	String	Decimal separator used for numbers. Value must be `COMMA` or `DOT`.
thousandSeparator	String	Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE`, or `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
containerType	String	Type of asset that contains code values. Value must be `hierarchy`.
containerKey	String	Internal ID or alias of a hierarchy.
filter	Object	Optional. Filter criteria for exporting filtered code values. For more information, see "Filter criteria for export version 3 and 4" on page 326.

Field	Type	Description
columns	Array	Optional. Attribute columns you want to export. If you don't specify attribute columns, the export includes all attribute columns.
fieldName	String	Optional. Name of attribute column to include in the exported file.
codeListId	String	The ID of the code list associated to the attribute.
excludeParentId	Boolean	Optional. Indicates whether to include the parent ID of code values in the CSV file. Value is `true` or `false`.

POST response

The response is a CSV file. The CSV file contains two header rows followed by data rows.

The CSV file uses the following columns: `status.key` and `effectiveDate`. Based on your data model, you might have additional columns in the file.

POST example

To export the code values in a hierarchy based on an internal ID or alias, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v4/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
Content-Length: 456
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "delimiter": "SEMICOLON",
  "textQualifier": "DOUBLE_QUOTE",
  "startingRow": "0",
  "codepage": "UTF8",
  "dateFormat": "ISO",
  "filename": "export.csv",
  "containerType": "hierarchy",
  "containerKey": "colour_hierarchy"
}
```

To export code values with `US` in the Name attribute, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v4/export HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
Content-Length: 346
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "delimiter":"SEMICOLON",
  "codepage":"UTF8",
  "decimalSeparator":"COMMA",
  "thousandSeparator":"DOT",
  "dateFormat":"ISO",
  "filename":"export.csv",
  "containerType":"hierarchy",
  "containerKey":"country_hierarchy",
  "filter": {
    "_and": [
      {
        "Name": {
          "_contains": "EN"
        }
      }
    ]
```

```
  }
}
```

The following sample response shows the exported data in a CSV file:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
Content-Length: 140
_language,_localizedContent.Name,Name,Code,_localizedContent.check2,Description,_localize
dContent.check1,colorCode1,colorCode2,_localizedContent.str2,refattr3,reffattr4,refattr5,
Ref1,_localizedContent.Str1,dependency,status,effectiveDate,parentCode
_language,_localizedContent.Name,Name,Code,_localizedContent.check2,Description,_localize
dContent.check1,refattr1,refattr2,_localizedContent.str2,refattr3,reffattr4,refattr5,Ref1
,_localizedContent.Str1,dependency,status,effectiveDate,parentCode
en-US,Black,,S1,,Black,Five,T1,T3,,,,,,T1,,,
fr-FR,Noir,,S1,,Black,Cinq,T1,T3,,,,,,T1,,,
en-US,Blue,,S4,Blue,,Blue,T1,T1,,,,,,,T1,,,
en-US,Green,,S1,,Green,,S1,S4,,,,,,,S1,,,S4
fr-FR,Noir,,S1,,Black,,S1,S4,couleur,,,,,,S1,,,S4
en-US,Brown,,S2,,,,S1,S1,,,,,,,S4,,,S4
fr-FR,jaune,,S3,,,,S1,S1,couleur,,,,,,S5,,,S4
de-DE,weiß,,S7,,White,,S1,S1,couleur,,,,,,S5,,,S4
en-US,White,,S7,,White,,S1,S1,couleur,,,,,,S5,,,S4
en-US,Violet,,S11,,,,S1,S4,,,,,,,S1,,,S4
```

For more information about exporting filtered code values, see "Exporting filtered code values" on page 542.

Export hierarchies to JSON format (v4)

Exports a hierarchy to the JSON format based on the internal ID or alias.

POST request

To export a hierarchy based on the internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v4/export
```

Note: In the request header, you must specify the `Accept` attribute to `application/json`.

Use the following parameters in the request body to specify the hierarchy to export:

Field	Type	Description
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US_HYPHEN`. For mm-dd-yyyy format. - `US_SLASH`. For mm/dd/yyyy format.
containerType	String	Type of asset that contains code values. Value must be `hierarchy`.
containerKey	String	Internal ID or alias of a code list.
excludeParentId	Boolean	Optional. Indicates whether to include the parent ID of code values in the CSV file. Value is `true` or `false`.
includeAssetName	Boolean	Optional. Indicates whether to include the Name field of code lists in the JSON file. Value can be `true` or `false`. Default is `false`.

POST response

The response is in JSON format.

POST example

To export a hierarchy based on the internal ID or alias, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v4/export HTTP/1.1
Content-Type: application/json
Accept: application/json
Host: localhost:8080
Content-Length: 163
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX

{
  "delimiter": "SEMICOLON",
  "textQualifier": "DOUBLE_QUOTE",
  "startingRow": "0",
  "codepage": "UTF8",
  "dateFormat": "ISO",
  "filename": "export.json",
  "containerType": "hierarchy",
  "containerKey": "color_hierarchy",
  "includeAssetName": "false"
}
```

The following sample response shows the exported data in JSON format:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 1114
{
  "content": [
    {
      "Code": "B",
      "children": [
        {
          "Code": "Black",
          "codeListDetails": {
            "id": "66d975cba667801d60523bef"
          },
          "fields": {
            "Code": "Black",
            "Name": "Black"
          }
        },
        {
          "Code": "Blue",
          "codeListDetails": {
            "id": "66d975cba667801d60523bef"
          },
          "fields": {
            "Code": "Blue",
            "Name": "Blue"
          }
        }
      ],
      "codeListDetails": {
        "id": "673ec36b545db931eb1b9b8b",
        "internalId": "ent_alphabet",
        "alias": "enterprise_alpha"
      },
      "fields": {
        "Description": "Black",
        "dependency": "G",
        "Position": "W",
        "_localizedContent": [
          {
            "_language": {
              "Code": "en-US"
            },
            "Name": "Black",
            "Palette Number": "Five"
          },
```

```
            {
              "_language": {
                "Code": "fr-FR"
              },
              "Name": "Noir",
              "Palette Number": "Cinq"
            }
          ],
          "Shadow": "G",
          "Code": "B"
        }
      }
    ]
}
```

audit trail version 1

Use this resource to retrieve the history of specified assets.

Get history of specified assets by time range (v1)

Retrieves all the change events of specified assets for a specific time range.

GET request

To retrieve all the change events of specified assets for a specific time range, submit a GET request with the following URI:

```
/rdm-service/external/v1/assets/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the query parameters:

Parameter	Description
from	Start date and time of a time range. The start date and time is inclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2023-01-01T00:00:00Z`.
to	End date and time of a time range. The end date and time is exclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2024-01-01T00:00:00Z`.
page	Optional. Page number to display. Default is 0.
pageSize	Optional. The number of records to display on each page. Default is 100. Maximum is 10000.
assetType	Optional. Type of asset. Set the value to REFERENCEDATASET, CODELIST, CROSSWALK, or HIERARCHY.
modificationType	Optional. Type of modification. Value can be CREATE, UPDATE, or DELETE.

GET response

The response contains the change events of specified assets for a time range.

The following table describes the attributes in the response:

Attribute	Type	Description
pageSize	Number	The number of records displayed on each page.
page	Number	Page number displayed.
numberOfElements	Number	The number of events.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of modification events.
asset	Object	Includes details about the assets.
modificationType	String	The type of change made to the assets.
fieldChanges	Object	Displays the previous and new values of the fields with changes.
attributeChanges	Object	Displays the previous and new values of the attributes with changes.
eventTime	String	The date and time the assets were last updated.
userName	String	The user name of the user who initiated the modification.

GET example

To retrieve all the change events of specified assets for a specific time range, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/assets/audit?
from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of the specified assets from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 2752

{
   "pageSize":10,
   "page":0,
   "numberOfElements":10,
   "lastPage":false,
   "firstPage":true,
   "content":[
      {
         "asset":{
            "id":"640707a46b19964cc917d0c8",
            "name":"Enterprise Country to ISO 3166-1 numeric",
            "assetType":"CROSSWALK"
         },
         "modificationType":"UPDATE",
```

```
      "eventTime":"2023-03-07T09:45:11Z",
      "userName":"integration-test-admin"
   },
   {
      "asset":{
         "id":"640707a46b19964cc917d0c8",
         "name":"Enterprise Country to ISO 3166-1 numeric",
         "assetType":"CROSSWALK"
      },
      "modificationType":"CREATE",
      "eventTime":"2023-03-07T09:45:08Z",
      "userName":"integration-test-admin"
   },
   {
      "asset":{
         "id":"6407004f5b243e32c4b02bb8",
         "name":"Enterprise Country",
         "assetType":"CODELIST"
      },
      "modificationType":"UPDATE",
      "eventTime":"2023-03-07T09:28:38Z",
      "userName":"integration-test-admin"
   },
   {
      "asset":{
         "id":"6407004f5b243e32c4b02bb8",
         "name":"Enterprise Country",
         "assetType":"CODELIST"
      },
      "modificationType":"UPDATE",
      "eventTime":"2023-03-07T09:28:37Z",
      "userName":"integration-test-admin"
   },
   {
      "asset":{
         "id":"6407004f6b19964cc917c7bf",
         "name":"Country",
         "assetType":"REFERENCEDATASET"
      },
      "modificationType":"UPDATE",
      "eventTime":"2023-03-07T09:28:08Z",
      "userName":"integration-test-admin"
   },
   {
      "asset":{
         "id":"6407004f5b243e32c4b02bb8",
         "name":"Enterprise Countary",
         "assetType":"CODELIST"
      },
      "modificationType":"UPDATE",
      "eventTime":"2023-03-07T09:27:36Z",
      "userName":"integration-test-admin"
   },
   {
      "asset":{
         "id":"6407004f5b243e32c4b02bb8",
         "name":"Enterprise Countary",
         "assetType":"CODELIST"
      },
      "modificationType":"UPDATE",
      "eventTime":"2023-03-07T09:27:35Z",
      "userName":"integration-test-admin"
   },
   {
      "asset":{
         "id":"6407004f5b243e32c4b02bb8",
         "name":"Enterprise Countary",
         "assetType":"CODELIST"
      },
      "modificationType":"UPDATE",
      "eventTime":"2023-03-07T09:24:55Z",
```

```
            "userName":"integration-test-admin"
         },
         {
            "asset":{
               "id":"6407004f5b243e32c4b02bb8",
               "name":"Enterprise Countary",
               "assetType":"CODELIST"
            },
            "modificationType":"UPDATE",
            "eventTime":"2023-03-07T09:24:53Z",
            "userName":"integration-test-admin"
         },
         {
            "asset":{
               "id":"6407004f5b243e32c4b02bb8",
               "name":"Enterprise Countary",
               "assetType":"CODELIST"
            },
            "modificationType":"UPDATE",
            "eventTime":"2023-03-07T09:24:48Z",
            "userName":"integration-test-admin"
         }
      ]
   }
```


audit trail version 2

Use this resource to retrieve the history of specified assets based on the internal ID or alias.

Get history of specified assets by time range (v2)

Retrieves all the change events of specified assets for a specific time range based on the internal ID or alias.

GET request

To retrieve all the change events of specified assets for a specific time range based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/assets/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the query parameters:

Parameter	Description
from	Start date and time of the time range, which is inclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-12T14:04:04Z`.
to	End date and time of the time range, which is exclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-15T14:04:04Z`.

Parameter	Description
page	Optional. Page number to display. Default is `0`.
pageSize	Optional. Number of records to display per page. Default value is `100`. Maximum value is `10000`.
assetType	Optional. Type of asset. Set the value to REFERENCEDATASET, CODELIST, CROSSWALK, HIERARCHY, or ALL.
modificationType	Optional. Type of modification. Value can be CREATE, UPDATE, or DELETE.

GET response

The response contains the change events of specified assets for a time range.

The following table describes the attributes in the response body:

Attribute	Type	Description
pageSize	Number	Number of records displayed on each page.
page	Number	Page number that is displayed.
numberOfElements	Number	Total number of assets found.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of modification events.
asset	Object	Includes details about the assets.
id	String	ID of the asset. For more information, see "Asset IDs" on page 199.
name	String	Name of the asset.
internalId	String	Unique identifier of the asset.
alias	String	Alternative identifier of the asset.
assetType	String	Type of the asset.
modificationType	String	Type of change made to the assets. Value can be CREATE, UPDATE, or DELETE.
fieldChanges	Object	Includes details about the previous and new values of the fields with changes.
attributeChanges	Object	Includes details about the previous and new values of the attributes with changes.
eventTime	String	Date and time when the assets were last updated.
userName	String	User name of the user who initiated the modification.

GET example

To retrieve all the change events of specified assets for a specific time range based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/assets/audit?
from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of the specified assets from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 2752

{
   "pageSize":10,
   "page":0,
   "numberOfElements":10,
   "lastPage":false,
   "firstPage":true,
   "content":[
      {
         "asset":{
            "id":"640707a46b19964cc917d0c8",
            "name":"Enterprise Country to ISO 3166-1 numeric",
            "internalId": "cw_country",
            "alias": "cw_als_country",
            "assetType":"CROSSWALK"
         },
         "modificationType":"UPDATE",
         "eventTime":"2023-03-07T09:45:11Z",
         "userName":"integration-test-admin"
      },
      {
         "asset":{
            "id":"640707a46b19964cc917d0c8",
            "name":"Enterprise Country to ISO 3166-1 numeric",
            "internalId": "cw_country",
            "alias": "cw_als_country",
            "assetType":"CROSSWALK"
         },
         "modificationType":"CREATE",
         "eventTime":"2023-03-07T09:45:08Z",
         "userName":"integration-test-admin"
      },
      {
         "asset":{
            "id":"6407004f5b243e32c4b02bb8",
            "name":"Enterprise Country",
            "internalId": "cl_country",
            "alias": "cl_als_country",
            "assetType":"CODELIST"
         },
         "modificationType":"UPDATE",
         "eventTime":"2023-03-07T09:28:38Z",
         "userName":"integration-test-admin"
      },
      {
         "asset":{
            "id":"6407004f5b243e32c4b02bb8",
            "name":"Enterprise Country",
            "internalId": "cl_country",
            "alias": "cl_als_country",
            "assetType":"CODELIST"
         },
         "modificationType":"UPDATE",
         "eventTime":"2023-03-07T09:28:37Z",
         "userName":"integration-test-admin"
```

```
},
{
   "asset":{
      "id":"6407004f6b19964cc917c7bf",
      "name":"Country",
      "internalId": "rds_country",
      "alias": "rds_als_country",
      "assetType":"REFERENCEDATASET"
   },
   "modificationType":"UPDATE",
   "eventTime":"2023-03-07T09:28:08Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407004f5b243e32c4b02bb8",
      "name":"Enterprise Country",
      "internalId": "cl_country",
      "alias": "cl_als_country",
      "assetType":"CODELIST"
   },
   "modificationType":"UPDATE",
   "eventTime":"2023-03-07T09:27:36Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407004f5b243e32c4b02bb8",
      "name":"Enterprise Country",
      "internalId": "cl_country",
      "alias": "cl_als_country",
      "assetType":"CODELIST"
   },
   "modificationType":"UPDATE",
   "eventTime":"2023-03-07T09:27:35Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407004f5b243e32c4b02bb8",
      "name":"Enterprise Country",
      "internalId": "cl_country",
      "alias": "cl_als_country",
      "assetType":"CODELIST"
   },
   "modificationType":"UPDATE",
   "eventTime":"2023-03-07T09:24:55Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407004f5b243e32c4b02bb8",
      "name":"Enterprise Country",
      "internalId": "cl_country",
      "alias": "cl_als_country",
      "assetType":"CODELIST"
   },
   "modificationType":"UPDATE",
   "eventTime":"2023-03-07T09:24:53Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407004f5b243e32c4b02bb8",
      "name":"Enterprise Country",
      "internalId": "cl_country",
      "alias": "cl_als_country",
      "assetType":"CODELIST"
   },
   "modificationType":"UPDATE",
   "eventTime":"2023-03-07T09:24:48Z",
```

```
            "userName":"integration-test-admin"
        }
    ]
}
```

rds version 1

Use this resource to list all reference data sets, get the details for a reference data set, and list code lists in a reference data set.

Get reference data sets (v1)

Retrieves all the reference data sets.

GET request

To retrieve all the reference data sets, submit a GET request with the following URI:

```
/rdm-service/external/v1/rds
```

GET response

The response contains information about each reference data sets.

The following table describes the attributes in the response:

Field	Type	Description
id	String	ID of the asset. Assets include reference data sets, code lists, crosswalks, and hierarchies. For more information, see "Asset IDs" on page 199.
name	String	Name of the asset.
description	String	Optional. Description of asset.
hierarchical	Boolean	Optional. Indicates whether code lists in the reference data set inherit the hierarchical structure definition.
levels	Number	Optional. Number of hierarchical levels supported in the code lists associated with the reference data set. If hierarchical is false or levels aren't provided, value is `1`. If levels are unlimited, value is `-1`.
domain	String	Optional. Domain of the asset.
confidentiality	String	Optional. Confidentiality of the asset.
priority	String	Optional. Priority of the asset.
status	String	Optional. Status of the asset.
effectiveDate	String	Optional. Date the asset became effective.

Field	Type	Description
approvedById	String	Optional. ID of the approver of the asset.
approvedByName	String	Optional. User name of the user who approved the asset.
approvedOn	String	Optional. Date the asset was approved.
displayColumns	String	Optional. List of display columns used as labels for code values. Default is `name`.

GET example

To retrieve all the reference data sets, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/rds HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the reference data sets:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 514

[
  {
    "id":"15d106a7905bd02a580d2b8d",
    "name":"Country",
    "description":"A Business Term named Country",
    "hierarchical":false,
    "levels":1,
    "displayColumns":[
      "Name"
    ]
  },
  {
    "id":"655df89e349a7fc1c0cd5f33",
    "name":"Currency",
    "hierarchical":false,
    "levels":1,
    "domain":"International standards",
    "confidentiality":"private",
    "priority":"Prio1",
    "status":"Draft",
    "effectiveDate":"2017-04-01",
    "approvedOn":"2017-03-01",
    "displayColumns":[
      "Name"
    ]
  }
]
```

Get reference data set details (v1)

Retrieves the details of a reference data set, such as the properties, status, structure definition, and attributes.

GET request

To retrieve the details of a reference data set, submit a GET request with the following URI:

```
/rdm-service/external/v1/rds/<reference data set ID>
```

Note: You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

GET response

The response contains the summary information and definition of the reference data set.

The following table describes the attributes in the response:

Field	Type	Description
id	String	ID of the asset. Assets include reference data sets, code lists, crosswalks, and hierarchies. For more information, see "Asset IDs" on page 199.
name	String	Name of the asset.
description	String	Optional. Description of asset.
hierarchical	Boolean	Optional. Indicates whether code lists in the reference data set inherit the hierarchical structure definition.
levels	Number	Optional. Number of hierarchical levels supported in the code lists associated with the reference data set. If hierarchical is false or levels aren't provided, value is `1`. If levels are unlimited, value is `-1`.
domain	String	Optional. Domain of the asset.
confidentiality	String	Optional. Confidentiality of the asset.
priority	String	Optional. Priority of the asset.
status	String	Optional. Status of the asset.
effectiveDate	String	Optional. Date the asset became effective.
approvedById	String	Optional. ID of the approver of the asset.
approvedByName	String	Optional. User name of the user who approved the asset.
approvedOn	String	Optional. Date the asset was approved.
defaultList	String	ID of the default code list.
displayColumns	String	Optional. List of display columns used as labels for code values. Default is `name`.
codeValueFields	-	Includes the attribute definition for code values in the reference data set.
name	String	Name of the field.
origin	String	Origin of the definition of the field.
datatype	String	Datatype of the field. Values are `String`, `Integer`, `Decimal`, `Boolean`, `Date`, or `Reference`.
mandatory	Boolean	Indicates whether the attribute is required.

Field	Type	Description
relatedTermId	String	Optional. If the attribute datatype is Reference, lists the ID of the reference data set.
displayColumns	Array	Optional. If the attribute datatype is Reference, lists the display columns.
dependencyDef	-	Optional. Includes the definition of the asset specified as the dependency.
termId	String	Optional. ID of the asset specified as the dependency.
displayColumns	Array	Optional. Display columns used as labels for code values associated with the dependent asset.

GET example

To retrieve the details of a reference data set, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/rds/
15d106a7905bd02a580d2b8d HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the details of a reference data set:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 1265

{
  "id":"15d106a7905bd02a580d2b8d",
  "name":"Country",
  "description":"A Business Term named Country",
  "hierarchical":false,
  "levels":1,
  "defaultList":"fa2a7f11fea1fc38db8d29aa",
  "domain":"International standards",
  "confidentiality":"private",
  "priority":"Prio1",
  "status":"Draft",
  "effectiveDate":"2017-04-01",
  "approvedOn":"2017-03-01",
  "displayColumns":[
    "Name"
  ],
  "codeValueFields":[
    {
      "name":"Name",
      "origin":"TERM",
      "datatype":"String",
      "mandatory":true
    },
    {
      "name":"Code",
      "origin":"TERM",
      "datatype":"String",
      "mandatory":true
    },
    {
      "name":"Description",
      "origin":"TERM",
      "datatype":"String",
      "mandatory":false
    },
    {
      "name":"Alpha2Code",
      "origin":"TERM",
```

```
      "datatype":"String",
      "mandatory":true
    },
    {
      "name":"Alpha3Code",
      "origin":"TERM",
      "datatype":"String",
      "mandatory":false
    },
    {
      "name":"RefField",
      "origin":"TERM",
      "datatype":"Reference",
      "mandatory":true,
      "relatedTermId":"655df89e349a7fc1c0cd5f33",
      "displayColumns":[
        "column1",
        "column2"
      ]
    }
  ],
  "dependencyDef":{
    "termId":"Continent",
    "displayColumns":[
      "Name"
    ]
  }
}
```

Get history of a reference data set by time range (v1)

Retrieves all the change events of a reference data set for a specific time range.

GET request

To retrieve all the change events of a reference data set for a specific time range, submit a GET request with the following URI:

```
/rdm-service/external/v1/rds/<identifier>/summary/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the query parameters:

Parameter	Description
identifier	ID of the reference data set. For more information, see "Asset IDs" on page 199. **Note:** When you provide an invalid identifier in the request URL, the rds REST API returns an empty content without an error.
from	Start date and time of a time range. The start date and time is inclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2023-01-01T00:00:00Z`.
to	End date and time of a time range. The end date and time is exclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2024-01-01T00:00:00Z`.
page	Optional. Page number to display. Default is 0.

Parameter	Description
pageSize	Optional. The number of records to display on each page. Default is 100. Maximum is 10000.
modificationType	Optional. Type of modification. Value can be CREATE, UPDATE, or DELETE.

GET response

The response contains the change events of a reference data set for a specific time range.

The following table describes the attributes in the response:

Attribute	Type	Description
pageSize	Number	The number of records displayed on each page.
page	Number	Page number displayed.
numberOfElements	Number	The number of events.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of modification events.
asset	Object	Includes details about the reference data set.
modificationType	String	The type of change made to the reference data set.
fieldChanges	Object	Displays the previous and new values of the fields with changes.
attributeChanges	Object	Displays the previous and new values of the attributes with changes.
eventTime	String	The date and time the reference data set was last updated.
userName	String	The user name of the user who initiated the modification.

GET example

To retrieve all the change events of a reference data set for a specific time range, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/rds/
2cf464206292908e67c9f719/summary/audit?from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z
HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of the reference data set from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 4803

{
  "pageSize":100,
   "page":0,
```

```
"numberOfElements":9,
"lastPage":true,
"firstPage":true,
"content":[
   {
      "asset":{
         "id":"6407004f6b19964cc917c7bf",
         "name":"Country",
         "assetType":"REFERENCEDATASET"
      },
      "modificationType":"UPDATE",
      "fieldChanges":{
         "confidentiality":{
            "newValue":{
               "key":"PublicKey",
               "label":"Public"
            }
         },
         "domain":{
            "newValue":{
               "key":"GeographyKey",
               "label":"Geography"
            }
         },
         "effectiveDate":{
            "newValue":"2023-03-05"
         },
         "priority":{
            "newValue":{
               "key":"Priority3",
               "label":"Medium"
            }
         },
         "status":{
            "newValue":{
               "key":"ActiveKey",
               "label":"Active"
            }
         }
      },
      "eventTime":"2023-03-07T09:28:08Z",
      "userName":"integration-test-admin"
   },
   {
      "asset":{
         "id":"6407004f6b19964cc917c7bf",
         "name":"Country",
         "assetType":"REFERENCEDATASET"
      },
      "modificationType":"UPDATE",
      "fieldChanges":{
         "name":{
            "previousValue":"Countary",
            "newValue":"Country"
         }
      },
      "eventTime":"2023-03-07T09:17:46Z",
      "userName":"integration-test-admin"
   },
   {
      "asset":{
         "id":"6407004f6b19964cc917c7bf",
         "name":"Countary",
         "assetType":"REFERENCEDATASET"
      },
      "modificationType":"UPDATE",
      "eventTime":"2023-03-07T09:15:28Z",
      "userName":"integration-test-admin"
   },
   {
      "asset":{
```

```
      "id":"6407004f6b19964cc917c7bf",
      "name":"Countary",
      "assetType":"REFERENCEDATASET"
   },
   "modificationType":"UPDATE",
   "attributeChanges":{
      "Alpha3Code":{
         "name":{
            "newValue":"Alpha3Code"
         },
         "type":{
            "newValue":"String"
         },
         "required":{
            "newValue":"false"
         }
      }
   },
   "eventTime":"2023-03-07T09:14:47Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407004f6b19964cc917c7bf",
      "name":"Countary",
      "assetType":"REFERENCEDATASET"
   },
   "modificationType":"UPDATE",
   "attributeChanges":{
      "Alpha2Code":{
         "name":{
            "newValue":"Alpha2Code"
         },
         "type":{
            "newValue":"String"
         },
         "required":{
            "newValue":"false"
         }
      }
   },
   "eventTime":"2023-03-07T09:14:37Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407004f6b19964cc917c7bf",
      "name":"Countary",
      "assetType":"REFERENCEDATASET"
   },
   "modificationType":"UPDATE",
   "eventTime":"2023-03-07T09:13:55Z",
   "userName":"mdm-rdm-service"
},
{
   "asset":{
      "id":"6407004f6b19964cc917c7bf",
      "name":"Countary",
      "assetType":"REFERENCEDATASET"
   },
   "modificationType":"UPDATE",
   "eventTime":"2023-03-07T09:13:54Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407004f6b19964cc917c7bf",
      "name":"Countary",
      "assetType":"REFERENCEDATASET"
   },
   "modificationType":"UPDATE",
```

```
   "fieldChanges":{
      "defaultList":{
         "newValue":"6407004f5b243e32c4b02bb8"
      }
   },
   "eventTime":"2023-03-07T09:13:54Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407004f6b19964cc917c7bf",
      "name":"Countary",
      "assetType":"REFERENCEDATASET"
   },
   "modificationType":"CREATE",
   "fieldChanges":{
      "description":{
         "newValue":"A Business Term named Country"
      },
      "displayColumns":{
         "newValue":[
            "Name"
         ]
      },
      "hierarchical":{
         "newValue":"false"
      },
      "levels":{
         "newValue":"1"
      },
      "name":{
         "newValue":"Countary"
      }
   },
   "attributeChanges":{
      "Code":{
         "name":{
            "newValue":"Code"
         },
         "type":{
            "newValue":"String"
         },
         "required":{
            "newValue":"true"
         }
      },
      "Description":{
         "name":{
            "newValue":"Description"
         },
         "type":{
            "newValue":"String"
         },
         "required":{
            "newValue":"false"
         }
      },
      "Name":{
         "name":{
            "newValue":"Name"
         },
         "type":{
            "newValue":"String"
         },
         "required":{
            "newValue":"true"
         }
      }
   },
   "eventTime":"2023-03-07T09:13:51Z",
   "userName":"integration-test-admin"
```

```
        }
    ]
}
```

Get code lists (v1)

Retrieves all the code lists in a reference data set.

GET request

To retrieve all the code lists in a reference data set, submit a GET request with the following URI:

```
/rdm-service/external/v1/rds/<reference data set ID>/codelists
```

Note: You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

GET response

The response contains information about the code lists in the specified reference data set.

The following table describes the attributes in the response:

Field	Type	Definition
id	String	ID of the asset. Assets include reference data sets, code lists, crosswalks, and hierarchies. For more information, see "Asset IDs" on page 199.
termId	String	ID of the reference data set to which the code list is associated.
name	String	Name of the asset.
description	String	Optional. Description of asset.
displayColumns	String	Optional. List of display columns used as labels for code values. Default is `name`.
version	String	Optional. Version of the code list.
application	String	Optional. Application that uses the code list.
hierarchical	Boolean	Optional. Indicates whether code lists in the reference data set inherit the hierarchical structure definition.
levels	Number	Optional. Number of hierarchical levels supported in the code lists associated with the reference data set. If hierarchical is false or levels aren't provided, value is `1`. If levels are unlimited, value is `-1`.
domain	String	Optional. Domain of the asset.
confidentiality	String	Optional. Confidentiality of the asset.
priority	String	Optional. Priority of the asset.
status	String	Optional. Status of the asset.
effectiveDate	String	Optional. Date the asset became effective.

Field	Type	Definition
approvedById	String	Optional. ID of the approver of the asset.
approvedByName	String	Optional. User name of the user who approved the asset.
approvedOn	String	Optional. Date the asset was approved.

GET example

To retrieve all the code lists in a reference data set, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/rds/
15d106a7905bd02a580d2b8d/codelists HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the code lists in a reference data set:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 747

[
  {
    "id":"863efc094436e68bc4299204",
    "termId":"c0af1cd77e7a1a3d658883ad",
    "name":"Units",
    "description":"Code list for units",
    "version":"2.0",
    "application":"UN recommendation 20",
    "hierarchical":false,
    "levels":1,
    "displayColumns":[
      "Name"
    ]
  },
  {
    "id":"17f60eb76ebcc75fec93dadc",
    "termId":"c0af1cd77e7a1a3d658883ad",
    "name":"SAP Units",
    "description":"Code list for SAP units",
    "version":"1.1",
    "application":"SAP",
    "hierarchical":true,
    "levels":10,
    "domain":"International standards",
    "confidentiality":"private",
    "priority":"Prio1",
    "status":"Draft",
    "effectiveDate":"2017-04-01",
    "approvedOn":"2017-03-01",
    "displayColumns":[
      "Name",
      "Code"
    ]
  }
]
```

rds version 2

Use this resource to list all reference data sets, get the details for a reference data set, and list code lists in a reference data set based on its internal ID or alias.

Get reference data sets (v2)

Retrieves all the reference data sets based on the internal ID or alias.

GET request

To retrieve all the reference data sets based on the internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/rds
```

GET response

The response contains information about each reference data sets.

The following table describes the attributes in the response body:

Field	Type	Description
id	String	ID of the reference data set. For more information, see "Asset IDs" on page 199.
name	String	Name of the reference data set.
description	String	Optional. Description of reference data set.
hierarchical	Boolean	Optional. Indicates whether code lists in the reference data set inherit the hierarchical structure definition.
levels	Number	Optional. Number of hierarchical levels supported in the code lists associated with the reference data set. If hierarchical is false or levels aren't provided, value is `1`. If levels are unlimited, value is `-1`.
internalId	String	Unique identifier of the reference data set.
alias	String	Alternative identifier of the reference data set.
defaultListDetails	Object	Includes details of the default code list for the reference data set.
id	String	ID of the default code list.
internalId	String	Unique identifier of the default code list for the reference data set.
alias	String	Alternative identifier of the default code list for the reference data set.
domain	String	Optional. Domain of the reference data set.
confidentiality	String	Optional. Confidentiality of the reference data set.
priority	String	Optional. Priority of the reference data set.

Field	Type	Description
status	String	Optional. Status of the reference data set.
effectiveDate	String	Optional. Date the reference data set became effective.
approvedById	String	Optional. ID of the approver of the reference data set.
approvedByName	String	Optional. Username of the user who approved the reference data set.
approvedOn	String	Optional. Date the reference data set was approved.
displayColumns	Array	Optional. List of display columns used as labels for code values. Default is `name`.

GET example

To retrieve all the reference data sets based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/rds HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the reference data sets:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 514

[
  {
    "id": "fba77ab1fdb64d78fc52efb1",
    "name": "Country",
    "description": "A Business Term named Country",
    "hierarchical": false,
    "levels": 1,
    "internalId": "rds_units",
    "alias": "units_rds",
    "defaultListDetails": {
      "id": "045e33bb684b4fcfa58c4e06",
      "internalId": "iso_units",
      "alias": "isounits"
    },
    "displayColumns": [
      "Name"
    ]
  },
  {
    "id":"655df89e349a7fc1c0cd5f33",
    "name":"Currency",
    "hierarchical":false,
    "levels":1,
    "internalId": "rds_currency",
    "alias": "currency_rds",
    "domain":"International standards",
    "confidentiality":"private",
    "priority":"Prio1",
    "status":"Draft",
    "effectiveDate":"2017-04-01",
    "approvedOn":"2017-03-01",
    "displayColumns":[
      "Name"
    ]
  }
]
```

Get reference data set details (v2)

Retrieves the details of a reference data set, such as the properties, status, structure definition, and attributes. You can retrieve the details of a reference data set based on the internal ID or alias.

GET request

To retrieve the details of a reference data set based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/rds/<rdsKey>
```

GET request query parameter

The following table describes the query parameter:

Parameter	Description
rdsKey	Internal ID or alias of the reference data set.

GET response

The response contains the summary information and definition of the reference data set.

The following table describes the attributes in the response body:

Field	Type	Description
id	String	ID of the reference data set. For more information, see "Asset IDs" on page 199.
name	String	Name of the reference data set.
description	String	Optional. Description of reference data set.
hierarchical	Boolean	Optional. Indicates whether code lists in the reference data set inherit the hierarchical structure definition.
levels	Number	Optional. Number of hierarchical levels supported in the code lists associated with the reference data set. If hierarchical is false or levels are not provided, value is `1`. If levels are unlimited, value is `-1`.
internalId	String	Unique identifier of the reference data set.
alias	String	Alternative identifier of the reference data set.
defaultListDetails	Object	Includes details of the default code list for the reference data set.
id	String	ID of the default code list for the reference data set. For more information, see "Asset IDs" on page 199.
internalId	String	Unique identifier of the default code list for the reference data set.
alias	String	Alternative identifier of the default code list for the reference data set.
domain	String	Optional. Domain of the reference data set.

Field	Type	Description
confidentiality	String	Optional. Confidentiality of the reference data set.
priority	String	Optional. Priority of the reference data set.
status	String	Optional. Status of the reference data set.
effectiveDate	String	Optional. Date the reference data set became effective.
approvedById	String	Optional. ID of the approver of the reference data set.
approvedByName	String	Optional. Username of the user who approved the reference data set.
approvedOn	String	Optional. Date the reference data set was approved.
displayColumns	String	Optional. List of display columns used as labels for code values. Default is `name`.
codeValueFields	Object	Includes the attribute definition for code values in the reference data set.
name	String	Name of the field.
origin	String	Origin of the definition of the field.
datatype	String	Datatype of the field. Values are `String`, `Integer`, `Decimal`, `Boolean`, `Date`, or `Reference`.
mandatory	Boolean	Indicates whether the attribute is required.
relatedTermId	String	Optional. If the attribute datatype is Reference, lists the ID of the reference data set.
displayColumns	Array	Optional. If the attribute datatype is Reference, lists the display columns.
dependencyDef	Object	Optional. Includes the definition of the asset specified as the dependency.
rdsDetails	Object	Includes details for the reference data set.
id	String	ID of the reference data set. For more information, see "Asset IDs" on page 199.
internalID	String	Unique identifier of the reference data set.
alias	String	Alternative identifier of the reference data set.
displayColumns	Array	Optional. Display columns used as labels for code values associated with the dependent asset.

GET example

To retrieve the details of a reference data set based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/rds/rds_units
HTTP/1.1Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the details of a reference data set:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
```

```
Content-Length: 2752
{
  "id": "fba77ab1fdb64d78fc52efb1",
  "name": "Country",
  "description": "A Business Term named Country",
  "hierarchical": false,
  "levels": 1,
  "internalId": "rds_units",
  "alias": "units_rds",
  "defaultListDetails": {
    "id": "045e33bb684b4fcfa58c4e06",
    "internalId": "iso_units",
    "alias": "isounits"
  },
  "domain": "International standards",
  "confidentiality": "private",
  "priority": "Prio1",
  "status": "Draft",
  "effectiveDate": "2017-04-01",
  "approvedOn": "2017-03-01",
  "displayColumns": [
    "Name"
  ],
  "codeValueFields": [
    {
      "name": "Name",
      "origin": "TERM",
      "datatype": "String",
      "mandatory": true,
      "labels": [
        {
          "language": "en",
          "value": "Name"
        }
      ]
    },
    {
      "name": "Code",
      "origin": "TERM",
      "datatype": "String",
      "mandatory": true,
      "labels": [
        {
          "language": "en",
          "value": "Code"
        }
      ]
    },
    {
      "name": "Description",
      "origin": "TERM",
      "datatype": "String",
      "mandatory": false,
      "labels": [
        {
          "language": "en",
          "value": "Description"
        }
      ]
    },
    {
      "name": "Alpha2Code",
      "origin": "TERM",
      "datatype": "String",
      "mandatory": true,
      "labels": [
        {
          "language": "en",
          "value": "Alpha2Code"
        }
      ]
```

```
    },
    {
      "name": "Alpha3Code",
      "origin": "TERM",
      "datatype": "String",
      "mandatory": false,
      "labels": [
        {
          "language": "en",
          "value": "Alpha3Code"
        }
      ]
    },
    {
      "name": "RefField",
      "origin": "TERM",
      "datatype": "Reference",
      "mandatory": true,
      "relatedRdsDetails": {
        "id": "93d5f260efa741436ba60591",
        "internalId": "testexternalrdsv2_2",
        "alias": "testexternalrds_2"
      },
      "displayColumns": [
        "column1",
        "column2"
      ],
      "labels": [
        {
          "language": "en",
          "value": "RefField"
        }
      ]
    }
  ],
  "dependencyDef": {
    "rdsDetails": {
      "id": "Continent",
      "internalId": "testexternalrdsv2",
      "alias": "testexternalrds_1"
    },
    "displayColumns": [
      "Name"
    ]
  }
}
```

Get history of a reference data set by time range (v2)

Retrieves all the change events of a reference data set for a specific time range based on the internal ID or alias.

GET request

To retrieve all the change events of a reference data set for a specific time range based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/rds/{assetKey}/summary/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the query parameters:

Parameter	Description
assetKey	Internal ID or alias of the reference data set. **Note:** When you provide an invalid assetKey in the request URL, the rds version 2 REST API returns an error if the request contains invalid internal IDs or aliases.
from	Start date and time of the time range, which is inclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-12T14:04:04Z`.
to	End date and time of the time range, which is exclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-15T14:04:04Z`.
page	Optional. Page number to display. Default is `0`.
pageSize	Optional. Number of records to display per page. Default value is `100`. Maximum value is `10000`.
modificationType	Optional. Type of modification. Value can be CREATE, UPDATE, or DELETE.

GET response

The response contains the change events of a reference data set for a specific time range.

The following table describes the attributes in the response body:

Attribute	Type	Description
pageSize	Number	The number of records displayed on each page.
page	Number	Page number displayed.
numberOfElements	Number	Total number of reference data sets found.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of modification events.
asset	Object	Includes details about the reference data set.
id	String	ID of the reference data set.
name	String	Name of the reference data set.
internalId	String	Unique identifier of the reference data set.
alias	String	Alternative identifier of the reference data set.
assetType	String	Type of reference data asset.
modificationType	String	The type of change made to the reference data set. Value can be CREATE, UPDATE, or DELETE.

Attribute	Type	Description
fieldChanges	Object	Includes details about the previous and new values of the fields with changes.
attributeChanges	Object	Includes details about the previous and new values of the attributes with changes.
eventTime	String	Date and time when the reference data set was last updated.
userName	String	User name of the user who initiated the modification.

GET example

To retrieve all the change events of a reference data set for a specific time range based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/rds/country/
summary/audit?from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of the reference data set from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 4803
{
  "pageSize": 100,
  "page": 0,
  "numberOfElements": 9,
  "lastPage": true,
  "firstPage": true,
  "content": [
    {
      "asset": {
        "id": "6407004f6b19964cc917c7bf",
        "name": "Country",
        "internalId": "country",
        "alias": "als_country",
        "assetType": "REFERENCEDATASET"
      },
      "modificationType": "UPDATE",
      "fieldChanges": {
        "confidentiality": {
          "newValue": {
            "key": "PublicKey",
            "label": "Public"
          }
        },
        "domain": {
          "newValue": {
            "key": "GeographyKey",
            "label": "Geography"
          }
        },
        "effectiveDate": {
          "newValue": "2023-03-05"
        },
        "priority": {
          "newValue": {
            "key": "Priority3",
            "label": "Medium"
          }
        },
        "status": {
          "newValue": {
```

```
        "key": "ActiveKey",
        "label": "Active"
      }
    }
  },
  "eventTime": "2023-03-07T09:28:08Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "6407004f6b19964cc917c7bf",
    "name": "Country",
    "internalId": "country",
    "alias": "als_country",
    "assetType": "REFERENCEDATASET"
  },
  "modificationType": "UPDATE",
  "fieldChanges": {
    "name": {
      "previousValue": "Countary",
      "newValue": "Country"
    }
  },
  "eventTime": "2023-03-07T09:17:46Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "6407004f6b19964cc917c7bf",
    "name": "Countary",
    "internalId": "country",
    "alias": "als_country",
    "assetType": "REFERENCEDATASET"
  },
  "modificationType": "UPDATE",
  "eventTime": "2023-03-07T09:15:28Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "6407004f6b19964cc917c7bf",
    "name": "Countary",
    "internalId": "country",
    "alias": "als_country",
    "assetType": "REFERENCEDATASET"
  },
  "modificationType": "UPDATE",
  "attributeChanges": {
    "Alpha3Code": {
      "name": {
        "newValue": "Alpha3Code"
      },
      "type": {
        "newValue": "String"
      },
      "required": {
        "newValue": "false"
      }
    }
  },
  "eventTime": "2023-03-07T09:14:47Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "6407004f6b19964cc917c7bf",
    "name": "Countary",
    "internalId": "country",
    "alias": "als_country",
    "assetType": "REFERENCEDATASET"
  },
```

```
  "modificationType": "UPDATE",
  "attributeChanges": {
    "Alpha2Code": {
      "name": {
        "newValue": "Alpha2Code"
      },
      "type": {
        "newValue": "String"
      },
      "required": {
        "newValue": "false"
      }
    }
  },
  "eventTime": "2023-03-07T09:14:37Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "6407004f6b19964cc917c7bf",
    "name": "Countary",
    "internalId": "country",
    "alias": "als_country",
    "assetType": "REFERENCEDATASET"
  },
  "modificationType": "UPDATE",
  "eventTime": "2023-03-07T09:13:55Z",
  "userName": "mdm-rdm-service"
},
{
  "asset": {
    "id": "6407004f6b19964cc917c7bf",
    "name": "Countary",
    "internalId": "country",
    "alias": "als_country",
    "assetType": "REFERENCEDATASET"
  },
  "modificationType": "UPDATE",
  "eventTime": "2023-03-07T09:13:54Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "6407004f6b19964cc917c7bf",
    "name": "Countary",
    "internalId": "country",
    "alias": "als_country",
    "assetType": "REFERENCEDATASET"
  },
  "modificationType": "UPDATE",
  "fieldChanges": {
    "defaultList": {
      "newValue": "6407004f5b243e32c4b02bb8"
    }
  },
  "eventTime": "2023-03-07T09:13:54Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "6407004f6b19964cc917c7bf",
    "name": "Countary",
    "internalId": "country",
    "alias": "als_country",
    "assetType": "REFERENCEDATASET"
  },
  "modificationType": "CREATE",
  "fieldChanges": {
    "description": {
      "newValue": "A Business Term named Country"
    },
```

```
        "displayColumns": {
          "newValue": [
            "Name"
          ]
        },
        "hierarchical": {
          "newValue": "false"
        },
        "levels": {
          "newValue": "1"
        },
        "name": {
          "newValue": "Countary"
        }
      },
      "attributeChanges": {
        "Code": {
          "name": {
            "newValue": "Code"
          },
          "type": {
            "newValue": "String"
          },
          "required": {
            "newValue": "true"
          }
        },
        "Description": {
          "name": {
            "newValue": "Description"
          },
          "type": {
            "newValue": "String"
          },
          "required": {
            "newValue": "false"
          }
        },
        "Name": {
          "name": {
            "newValue": "Name"
          },
          "type": {
            "newValue": "String"
          },
          "required": {
            "newValue": "true"
          }
        }
      },
      "eventTime": "2023-03-07T09:13:51Z",
      "userName": "integration-test-admin"
    }
  ]
}
```


Get code lists (v2)

Retrieves all the code lists in a reference data set based on the internal ID or alias.

GET request

To retrieve all the code lists in a reference data set based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/rds/<rdsKey>/codelists
```

GET request query parameter

The following table describes the query parameter:

Parameter	Description
rdsKey	Internal ID or alias of the reference data set.

GET response

The response contains information about the code lists in the specified reference data set.

The following table describes the attributes in the response body:

id	String	ID of the code lists in a reference data set. For more information, see "Asset IDs" on page 199.
rdsDetails	Object	Includes details for the reference data set.
id	String	ID of the reference data set.
internalID	String	Unique identifier of the reference data set.
alias	String	Alternative identifier of the reference data set.
name	String	Name of the reference data set.
name	String	Name of the code list.
description	String	Optional. Description of asset.
version	String	Optional. Version of the code list.
application	String	Optional. Application that uses the code list.
hierarchical	Boolean	Optional. Indicates whether code lists in the reference data set inherit the hierarchical structure definition.
levels	Number	Optional. Number of hierarchical levels supported in the code lists associated with the reference data set. If hierarchical is false or levels aren't provided, value is `1`. If levels are unlimited, value is `-1`.
domain	String	Optional. Domain of the code list in the reference data set.
confidentiality	String	Optional. Confidentiality of the code list in the reference data set.
priority	String	Optional. Priority of the code list in the reference data set.
status	String	Optional. Status of the code list in the reference data set.
effectiveDate	String	Optional. Date the code list in the reference data set became effective.

approvedById	String	Optional. ID of the approver of the code list in the reference data set.
approvedByName	String	Optional. User name of the user who approved the code list in the reference data set.
approvedOn	String	Optional. Date the code list in the reference data set was approved.
displayColumns	String	Optional. List of display columns used as labels for code values. Default is `name`.
name	String	Name of the field.
origin	String	Origin of the definition of the field.
datatype	String	Datatype of the field. Values are `String`, `Integer`, `Decimal`, `Boolean`, `Date`, or `Reference`.
mandatory	Boolean	Indicates whether the attribute is required.
relatedTermId	String	Optional. If the attribute datatype is Reference, lists the ID of the reference data set.
displayColumns	Array	Optional. If the attribute datatype is Reference, lists the display columns.

Note: You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

GET example

To retrieve all the code lists in a reference data set based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/rds/rds_units/
codelists HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the code lists in a reference data set:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 747
[
  {
    "id": "e36aebf55a48d54133b9e79a",
    "rdsDetails": {
      "id": "86845d3822371e96a4dbbcbb",
      "internalId": "rds_units",
      "alias": "units",
      "name": "rds_units"
    },
    "name": "Units",
    "description": "Code list for units",
    "version": "2.0",
    "application": "UN recommendation 20",
    "hierarchical": false,
    "levels": 1,
    "displayColumns": [
      "Name"
    ]
  },
  {
    "id": "fae288a8796d3664ed7b3ffd",
    "rdsDetails": {
```

```
      "id": "86845d3822371e96a4dbbcbb",
      "internalId": "rds_units",
      "alias": "units",
      "name": "rds_units"
    },
    "name": "SAP Units",
    "description": "Code list for SAP units",
    "version": "1.1",
    "application": "SAP",
    "hierarchical": true,
    "levels": 10,
    "domain": "International standards",
    "confidentiality": "private",
    "priority": "Prio1",
    "status": "Draft",
    "effectiveDate": "2017-04-01",
    "approvedOn": "2017-03-01",
    "displayColumns": [
      "Name",
      "Code"
    ]
  }
]
```

codelists version 1

Use this resource to retrieve the details of a code list, the details of a code value, and the crosswalks associated to a code list based on its name. You can also unlock code lists locked by other users.

Get code list details (v1)

Retrieves the details of a code list, such as the properties, status, structure definition, and attributes.

GET request

To retrieve the details of a code list, submit a GET request with the following URI:

```
/rdm-service/external/v1/codelists/<code list ID>
```

Note: You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

GET response

The response contains the details of the specified code list.

The following table describes the attributes in the response:

Field	Type	Description
id	String	ID of the asset. Assets include reference data sets, code lists, crosswalks, and hierarchies. For more information, see "Asset IDs" on page 199.
termId	String	ID of the reference data set to which the code list is associated.
name	String	Name of the asset.

Field	Type	Description
description	String	Optional. Description of asset.
hierarchical	Boolean	Optional. Indicates whether code lists in the reference data set inherit the hierarchical structure definition.
levels	Number	Optional. Number of hierarchical levels supported in the code lists associated with the reference data set. If hierarchical is false or levels aren't provided, value is `1`. If levels are unlimited, value is `-1`.
domain	String	Optional. Domain of the asset.
confidentiality	String	Optional. Confidentiality of the asset.
priority	String	Optional. Priority of the asset.
status	String	Optional. Status of the asset.
effectiveDate	String	Optional. Date the asset became effective.
approvedById	String	Optional. ID of the approver of the asset.
approvedByName	String	Optional. User name of the user who approved the asset.
approvedOn	String	Optional. Date the asset was approved.
defaultList	String	ID of the default code list.
displayColumns	String	Optional. List of display columns used as labels for code values. Default is `name`.
codeValueFields	-	Includes the attribute definition for code values in the reference data set.
name	String	Name of the field.
origin	String	Origin of the definition of the field.
datatype	String	Datatype of the field. Values are `String`, `Integer`, `Decimal`, `Boolean`, `Date`, or `Reference`.
mandatory	Boolean	Indicates whether the attribute is required.
relatedTermId	String	Optional. If the attribute datatype is Reference, lists the ID of the reference data set.
displayColumns	Array	Optional. If the attribute datatype is Reference, lists the display columns.
dependencyDef	-	Optional. Includes the definition of the asset specified as the dependency.
termId	String	Optional. ID of the asset specified as the dependency.
displayColumns	Array	Optional. Display columns used as labels for code values associated with the dependent asset.

GET example

To retrieve the details of a code list, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
4fb1356728272974bd46945f HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the details of a code list:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 1284

{
  "id":"4fb1356728272974bd46945f",
  "termId":"Unit Term",
  "name":"Units",
  "description":"Code list for units",
  "version":"2.0",
  "application":"UN recommendation 20",
  "hierarchical":false,
  "levels":1,
  "domain":"International standards",
  "confidentiality":"private",
  "priority":"Prio1",
  "status":"Draft",
  "effectiveDate":"2017-04-01",
  "approvedOn":"2017-03-01",
  "displayColumns":[
    "Name",
    "Code"
  ],
  "codeValueFields":[
    {
      "name":"Name",
      "origin":"TERM",
      "datatype":"String",
      "mandatory":false
    },
    {
      "name":"Code",
      "origin":"TERM",
      "datatype":"String",
      "mandatory":true
    },
    {
      "name":"Description",
      "origin":"TERM",
      "datatype":"String",
      "mandatory":false
    },
    {
      "name":"achronym",
      "origin":"CODELIST",
      "datatype":"String",
      "mandatory":false
    },
    {
      "name":"refField2",
      "origin":"CODELIST",
      "datatype":"Reference",
      "mandatory":true,
      "relatedTermId":"b02c86d02ac7de3a688353dc",
      "relatedListId":"dc245266d5a61ce4d0535f74",
      "displayColumns":[
        "column5"
      ]
    }
  ],
```

```
  "dependencyDef":{
    "termId":"UnitSystem Term",
    "codelistId":"UnitSystems",
    "displayColumns":[
      "Name"
    ]
  }
}
```

Get history of a code list by time range (v1)

Retrieves all the change events of a code list for a specific time range.

GET request

To retrieve all the change events of a code list for a specific time range, submit a GET request with the following URI:

```
/rdm-service/external/v1/codelists/<listIdentifier>/summary/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the query parameters:

Parameter	Description
listIdentifier	ID of the code list. For more information, see "Asset IDs" on page 199. **Note:** When you provide an invalid asset identifier in the request URL, the codelist REST API returns an empty content without an error.
from	Start date and time of a time range. The start date and time is inclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2023-01-01T00:00:00Z`.
to	End date and time of a time range. The end date and time is exclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2024-01-01T00:00:00Z`.
page	Optional. Page number to display. Default is 0.
pageSize	Optional. The number of records to display on each page. Default is 100. Maximum is 10000.
modificationType	Optional. Type of modification. Value can be CREATE, UPDATE, or DELETE.

GET response

The response contains the change events of a code list for a specific time range.

The following table describes the attributes in the response:

Attribute	Type	Description
pageSize	Number	The number of records displayed on each page.
page	Number	Page number displayed.

Attribute	Type	Description
numberOfElements	Number	The number of events.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of modification events.
asset	Object	Includes details about the code list.
modificationType	String	The type of change made to the code list.
fieldChanges	Object	Displays the previous and new values of the fields with changes.
attributeChanges	Object	Displays the previous and new values of the attributes with changes.
eventTime	String	The date and time the code list was last updated.
userName	String	The user name of the user who initiated the modification.

GET example

To retrieve all the change events of a code list for a specific time range, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
9ae8fcf6b8f0327a7bdb7f2c/summary/audit?from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z
HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of the code list from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 4991

{
   "pageSize":100,
   "page":0,
   "numberOfElements":8,
   "lastPage":true,
   "firstPage":true,
   "content":[
      {
         "asset":{
            "id":"6407014f5b243e32c4b02ce3",
            "name":"ISO 3166-1 numeric",
            "assetType":"CODELIST"
         },
         "modificationType":"UPDATE",
         "eventTime":"2023-03-07T09:23:35Z",
         "userName":"integration-test-admin"
      },
      {
         "asset":{
            "id":"6407014f5b243e32c4b02ce3",
            "name":"ISO 3166-1 numeric",
            "assetType":"CODELIST"
         },
         "modificationType":"UPDATE",
         "fieldChanges":{
```

```
      "additionalDescription":{
         "newValue":"(ISO 3166-1)"
      },
      "application":{
         "newValue":{
            "key":"EnterpriseKey",
            "label":"Enterprise"
         }
      },
      "approvedById":{
         "newValue":"2L0DXgAOiCObZDT7Yql02R"
      },
      "approvedByName":{
         "newValue":"integration-test-planner"
      },
      "approvedOn":{
         "newValue":"2023-03-08"
      },
      "description":{
         "newValue":"Numeric Codes for Countries"
      },
      "effectiveDate":{
         "newValue":"2023-03-06"
      },
      "status":{
         "newValue":{
            "key":"ActiveKey",
            "label":"Active"
         }
      },
      "version":{
         "newValue":"1"
      }
   },
   "eventTime":"2023-03-07T09:23:34Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407014f5b243e32c4b02ce3",
      "name":"ISO 3166-1 numeric",
      "assetType":"CODELIST"
   },
   "modificationType":"UPDATE",
   "attributeChanges":{
      "NumericCode":{
         "name":{
            "newValue":"NumericCode"
         },
         "type":{
            "newValue":"String"
         },
         "required":{
            "newValue":"false"
         }
      }
   },
   "eventTime":"2023-03-07T09:19:29Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407014f5b243e32c4b02ce3",
      "name":"ISO 3166-1 numeric",
      "assetType":"CODELIST"
   },
   "modificationType":"UPDATE",
   "attributeChanges":{
      "NumericCode":{
         "name":{
            "newValue":"NumericCode"
```

```
         },
         "type":{
            "newValue":"String"
         },
         "required":{
            "newValue":"false"
         }
      }
   },
   "eventTime":"2023-03-07T09:19:27Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407014f5b243e32c4b02ce3",
      "name":"ISO 3166-1 numeric",
      "assetType":"CODELIST"
   },
   "modificationType":"UPDATE",
   "eventTime":"2023-03-07T09:18:19Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407014f5b243e32c4b02ce3",
      "name":"ISO 3166-1 numeric",
      "assetType":"CODELIST"
   },
   "modificationType":"UPDATE",
   "eventTime":"2023-03-07T09:18:17Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407014f5b243e32c4b02ce3",
      "name":"ISO 3166-1 numeric",
      "assetType":"CODELIST"
   },
   "modificationType":"UPDATE",
   "eventTime":"2023-03-07T09:18:09Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407014f5b243e32c4b02ce3",
      "name":"ISO 3166-1 numeric",
      "assetType":"CODELIST"
   },
   "modificationType":"CREATE",
   "fieldChanges":{
      "displayColumns":{
         "newValue":[
            "Name"
         ]
      },
      "hierarchical":{
         "newValue":"false"
      },
      "levels":{
         "newValue":"1"
      },
      "name":{
         "newValue":"ISO 3166-1 numeric"
      },
      "termId":{
         "newValue":"6407004f6b19964cc917c7bf"
      }
   },
   "attributeChanges":{
      "Alpha2Code":{
         "name":{
```

```
                        "newValue":"Alpha2Code"
                     },
                     "type":{
                        "newValue":"String"
                     },
                     "required":{
                        "newValue":"false"
                     }
                  },
                  "Alpha3Code":{
                     "name":{
                        "newValue":"Alpha3Code"
                     },
                     "type":{
                        "newValue":"String"
                     },
                     "required":{
                        "newValue":"false"
                     }
                  },
                  "Code":{
                     "name":{
                        "newValue":"Code"
                     },
                     "type":{
                        "newValue":"String"
                     },
                     "required":{
                        "newValue":"true"
                     }
                  },
                  "Description":{
                     "name":{
                        "newValue":"Description"
                     },
                     "type":{
                        "newValue":"String"
                     },
                     "required":{
                        "newValue":"false"
                     }
                  },
                  "Name":{
                     "name":{
                        "newValue":"Name"
                     },
                     "type":{
                        "newValue":"String"
                     },
                     "required":{
                        "newValue":"true"
                     }
                  }
               },
               "eventTime":"2023-03-07T09:18:07Z",
               "userName":"integration-test-admin"
            }
         ]
      }
```

Enable auto-generation of Code field values for a code list (v1)

Enables auto-generation of Code field values when you import code values to a code list that's created prior to the April 2025 release, if the Code field is configured with a rule association.

POST request

To enable auto-generation of Code field values when you import code values to a code list with a rule association configured for the Code field, submit a POST request with the following URI:

```
/rdm-service/external/v1/codelists/<code list ID>/spkey/population/migrate
```

POST response

The response is in the JSON format.

The following table describes the attributes in the response:

Field	Type	Description
status	String	Status of the request. The request can return one of the following statuses: - MIGRATION_SUCCESSFUL. Indicates that the request is successful. - MIGRATION_NOT_POSSIBLE. Indicates that the code list contains Code field values that don't match their source primary keys. For more information, contact Informatica Global Customer Support. - ALREADY_MIGRATED. Indicates that the request is successful and the code list is already enabled with auto-generation of Code field values. - NOT_APPLICABLE. Indicates that the Code field of the code list doesn't contain a rule association.
spKeyPopulationStatus.before	String	Status of the code list before running the request. The code list can have one of the following statuses: - NOT_AVAILABLE. Indicates that the Code field of the code list doesn't contain a rule association. - UNKNOWN. Indicates that the request failed because of internal errors. For more information, contact Informatica Global Customer Support. - NOT_READY_FOR_MIGRATION. Indicates that the Code field of the code list contains a rule association but can't be enabled for auto-generation of Code field values because the code list contains Code field values that don't match their source primary keys. For more information, contact Informatica Global Customer Support. - READY_FOR_MIGRATION. Indicates that the code list is eligible for auto-generation of Code field values when you import code values to the code list. - MIGRATED. Indicates that the code list is already enabled for auto-generation of Code field values.

Field	Type	Description
spKeyPopulationStatus.after	String	Status of the code list after running the request. The code list can have one of the following statuses: - NOT_AVAILABLE. Indicates that the Code field of the code list doesn't contain a rule association. - UNKNOWN. Indicates that the request failed because of internal errors. For more information, contact Informatica Global Customer Support. - NOT_READY_FOR_MIGRATION. Indicates that the Code field of the code list contains a rule association but can't be enabled for auto-generation of Code field values because the code list contains Code field values that don't match their source primary keys. For more information, contact Informatica Global Customer Support. - MIGRATED. Indicates that the code list is enabled for auto-generation of Code field values.
invalidCodes	Object	Displays the details of invalid source primary keys and Code field values only if the status of the request is `MIGRATION_NOT_POSSIBLE`.

POST example

To enable auto-generation of Code field values when you import code values to a code list with a rule association configured for the Code field, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
34cea9471fe977f7decef5f5/spkey/population/migrate HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows a successful request that enabled auto-generation of Code field values for a code list:

```
{
   "status":"MIGRATION_SUCCESSFUL",
   "spKeyPopulationStatus":{
      "before":"READY_FOR_MIGRATION",
      "after":"MIGRATED"
   }
}
```

Get the status of a code list if it's enabled with auto-generation of Code field values (v1)

Retrieves the status of a code list if it's enabled with auto-generation of Code field values.

GET request

To check whether a code list is enabled with auto-generation of Code field values, submit a GET request with the following URI:

```
/rdm-service/external/v1/codelists/<code list ID>/spkey/population/status
```

GET response

The response contains the status of a code list if it's enabled with auto-generation of Code field values.

The response can contain one of the following statuses:

- NOT_AVAILABLE. Indicates that the Code field of the code list doesn't contain a rule association.
- UNKNOWN. Indicates that the request failed because of internal errors. For more information, contact Informatica Global Customer Support.

- NOT_READY_FOR_MIGRATION. Indicates that the Code field of the code list contains a rule association but can't be enabled for auto-generation of Code field values because the code list contains Code field values that don't match their source primary keys. For more information, contact Informatica Global Customer Support.
- READY_FOR_MIGRATION. Indicates that the code list is eligible for auto-generation of Code field values when you import code values to the code list.
- MIGRATED. Indicates that the code list is already enabled for auto-generation of Code field values.

GET example

To get the status of a code list if it's enabled with auto-generation of Code field values, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
34cea9471fe977f7decef5f5/spkey/population/status HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the status of the code list:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 20

"READY_FOR_MIGRATION"
```

Get code value details (v1)

Retrieves the details of a code value.

You identify the code value that you want to retrieve the details for by specifying the value in the Code attribute.

Note: You can't use the + symbol inside the code field value.

GET request

To retrieve the details of a code value, submit a GET request with the following URI and specify the code:

```
/rdm-service/external/v1/codelists/{code list ID}/codevalues?Code={code}
```

GET response

The response contains the details of the code value.

The following table describes the attributes in the response:

Field	Type	Description
codelistId	String	ID of the code list that the code values belong to.
status	String	Optional. Status of the code value. **Note:** To retrieve a list of statuses, use the List enum entries API. For more information, see "Get system reference data values" on page 529.
effectiveDate	String	Optional. Date the code value became effective.
codeValueFields	Object	Includes the attribute field values for the code value.

GET example

To retrieve the details of a code value, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
591c302d8af18b0001b1fac2/codevalues?Code=AR HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the details of a code value:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 237

[
  {
    "codelistId":"591c302d8af18b0001b1fac2",
    "status":"Draft",
    "effectiveDate":"2019-09-20",
    "codeValueFields":{
      "Name":"Argentina",
      "Code":"AR",
      "description":"The EU country code for Argentina"
    }
  }
]
```

Get history of code values of a code list (v1)

Retrieves all the change events of code values of a code list for a specific time range.

GET request

To retrieve all the change events of code values of a code list for a specific time range, submit a GET request with the following URI:

```
/rdm-service/external/v1/codelists/<code list ID>/codevalues/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the query parameters:

Parameter	Description
from	Start date and time of a time range. The start date and time is inclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2023-01-01T00:00:00Z`.
to	End date and time of a time range. The end date and time is exclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2024-01-01T00:00:00Z`.
page	Optional. Page number to display. Default is 0.
pageSize	Optional. The number of records to display on each page. Default is 100. Maximum is 10000.
modificationType	Optional. Type of modification. Value can be CREATE or UPDATE_AND_DELETE.

GET response

The response contains the change event of code values of a code list for a specific time range.

The following table describes the attributes in the response:

Attribute	Type	Description
pageSize	Number	The number of records displayed on each page.
page	Number	Page number displayed.
numberOfElements	Number	The number of events.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of modification events.
asset	Object	Includes details about the code values of the code list.
modificationType	String	The type of change made to the code values of the code list.
changedAttributes	Object	Displays the previous and new values of the attributes with changes.
code	String	The code field of the code values.
codeValueId	String	Internal ID of the code values.
_localizedContent[<language code>].<attribute name>	Object	The previous and current values of an attribute in the specified language. The parameter combines the BCP 47 language code and the attribute name. For example, _localizedContent[en-us].Description includes the change to the Description attribute in English (United States). The object contains the previous and new values of the attribute in that language.
previousValue	String	Previous value of the attribute in the specified language. If the value is newly created, the response doesn't include this parameter.
newValue	String	Current value of the attribute in the specified language.
eventTime	String	The date and time the code values were last updated.
userName	String	The user name of the user who initiated the modification.

GET example

To retrieve all the change events of code values of a code list for a specific time range, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
c2f9323dbec86b2cfe8a8101/codevalues/audit?
from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of a code list from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
```

```
Content-Length: 3441

{
  "pageSize": 100,
  "page": 0,
  "numberOfElements": 6,
  "lastPage": true,
  "firstPage": true,
  "content": [
    {
      "asset": {
        "id": "6407014f5b243e32c4b02ce3",
        "assetType": "CODELIST"
      },
      "codeValueId": "640701e45b243e32c4b02d40",
      "code": "Antarctica",
      "modificationType": "UPDATE",
      "changedAttributes": {
        "_localizedContent[en-us].Description": {
          "previousValue": "Numeric Code of Antarctica",
          "newValue": "Numeric Code of Antarctica (ISO 3166-1)"
        },
        "_localizedContent[de].Description": {
          "newValue": "Numerischer Code der Antarktis (ISO 3166-1)"
        }
      },
      "eventTime": "2023-03-07T09:21:56Z",
      "userName": "integration-test-admin"
    },
    {
      "asset": {
        "id": "6407014f5b243e32c4b02ce3",
        "assetType": "CODELIST"
      },
      "codeValueId": "640701d25b243e32c4b02d35",
      "code": "Albania",
      "modificationType": "UPDATE",
      "changedAttributes": {
        "_localizedContent[en-us].Description": {
          "previousValue": "Numeric Code of Albania",
          "newValue": "Numeric Code of Albania (ISO 3166-1)"
        },
        "_localizedContent[de].Description": {
          "newValue": "Numerischer Code von Albanien (ISO 3166-1)"
        }
      },
      "eventTime": "2023-03-07T09:21:39Z",
      "userName": "integration-test-admin"
    },
    {
      "asset": {
        "id": "6407014f5b243e32c4b02ce3",
        "assetType": "CODELIST"
      },
      "codeValueId": "640701c35b243e32c4b02d21",
      "code": "Afghanistan",
      "modificationType": "UPDATE",
      "changedAttributes": {
        "_localizedContent[en-us].Description": {
          "previousValue": "Numeric Code for Afghanistan",
          "newValue": "Numeric Code for Afghanistan (ISO 3166-1)"
        },
        "_localizedContent[de].Description": {
          "newValue": "Numerischer Code für Afghanistan (ISO 3166-1)"
        }
      },
      "eventTime": "2023-03-07T09:21:19Z",
      "userName": "integration-test-admin"
    },
    {
      "asset": {
```

```
    "id": "6407014f5b243e32c4b02ce3",
    "assetType": "CODELIST"
  },
  "codeValueId": "640701c35b243e32c4b02d21",
  "code": "Afghanistan",
  "modificationType": "CREATE",
  "changedAttributes": {
    "Code": {
      "newValue": "Afghanistan"
    },
    "_localizedContent[en-us].Description": {
      "newValue": "Numeric Code for Afghanistan"
    },
    "_localizedContent[en-us].Name": {
      "newValue": "Afghanistan"
    },
    "_localizedContent[de].Name": {
      "newValue": "Afghanistan"
    },
    "NumericCode": {
      "newValue": "004"
    }
  },
  "eventTime": "2023-03-07T09:20:43Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "6407014f5b243e32c4b02ce3",
    "assetType": "CODELIST"
  },
  "codeValueId": "640701d25b243e32c4b02d35",
  "code": "Albania",
  "modificationType": "CREATE",
  "changedAttributes": {
    "Code": {
      "newValue": "Albania"
    },
    "_localizedContent[en-us].Description": {
      "newValue": "Numeric Code of Albania"
    },
    "_localizedContent[en-us].Name": {
      "newValue": "Albania"
    },
    "_localizedContent[de].Name": {
      "newValue": "Albanien"
    },
    "NumericCode": {
      "newValue": "008"
    }
  },
  "eventTime": "2023-03-07T09:20:43Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "6407014f5b243e32c4b02ce3",
    "assetType": "CODELIST"
  },
  "codeValueId": "640701e45b243e32c4b02d40",
  "code": "Antarctica",
  "modificationType": "CREATE",
  "changedAttributes": {
    "Code": {
      "newValue": "Antarctica"
    },
    "_localizedContent[en-us].Description": {
      "newValue": "Numeric Code of Antarctica"
    },
    "_localizedContent[en-us].Name": {
      "newValue": "Antarctica"
```

```
        },
        "_localizedContent[de].Name": {
          "newValue": "Antarktis"
        },
        "NumericCode": {
          "newValue": "010"
        }
      },
      "eventTime": "2023-03-07T09:20:43Z",
      "userName": "integration-test-admin"
    }
  ]
}
```

Move a code value (v1)

Moves a code value to another node within the same hierarchical code list without locking the code list.

POST request

To move a code value to another node within the same hierarchical code list, submit a POST request with the following URI:

```
/rdm-service/external/v1/codelists/{listIdentifier}/codevalues/move
```

Use the following parameters in the request body to specify the code fields of the code value to move and the target node:

Field	Type	Description
Code	String	The code field of the code value to move.
TargetParentCode	String	The code field of the target node to which you want to move the code value. If you don't specify a code, the code value is moved to the top level.

POST response

The response contains a success message.

POST example

To move the code value to another node within the same hierarchical code list, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
833861786fb48aa467602980/codevalues/move HTTP/1.1
Content-Type: application/json
Content-Length: 48
Host: localhost:8080

{
  "Code" : "DE",
  "TargetParentCode" : "EU"
}
```

The following sample response shows the success message:

```
HTTP/1.1 200 OK
```

Create a code value (v1)

Creates a code value in a code list.

PATCH request

To create a code value in a code list, submit a PATCH request with the following URI:

```
/rdm-service/external/v1/codelists/{listIdentifier}/codevalues
```

The following table describes the parameter in the request:

Parameter	Type	Description
listIdentifier	String	ID of the code list that the code values belong to.

The following table describes the attributes in the request:

Field	Type	Description
action	String	Action to perform on the code value. Value is `CREATE`.
records	Array	List of code values to create. **Note:** Currently, you can include only one code value in the body.
Name	String	Name of the field.
Code	String	The code field of the code value to create.
Description	String	Description of the code value.
status	String	Optional. Status of the code value.
parentCode	String	Code value of the parent node.

When you specify attributes in the request body, consider the following guidelines:

- The attribute names are case-sensitive.
- Use camel case for name and code attributes and lowercase for other fields, such as dependency.
- Use string data type for a decimal attribute.
- When you add a node to a parent code value in a hierarchical code list, use the parentCode attribute.

When you create a code value with data quality rule associations assigned to a Code attribute and set the value of the code attribute within a code list, the rule statement overrides the value.

PATCH response

The response generates a report of code values that were created.

The following table describes the attributes in the response:

Field	Type	Description
successfulRecords	-	Lists the code values that were created successfully and describes the details of the code values.
Code	String	Code attribute value for the created code value.
label	String	Display attribute value for the created code value.
failedRecords	-	Lists the code values that weren't created and describes the reasons.
label	String	Display attribute value for the code value that wasn't created.
Code	String	Code attribute value for the code value that wasn't created.
errorCauses	-	Error details for the code values that couldn't be created.
errorCode	String	Error code for the error type.
errorSummary	String	A short summary that explains why the code value wasn't created.
localizedErrorSummary	String	A short summary that explains the error in the user locale.
errorParameter	String	Parameter the provides details of an error.
successfulRecordsCount	String	Number of code values that were created successfully.
failedRecordsCount	String	Number of code values that weren't created.

PATCH example

To create a code value in a code list, you might use the following request:

```
    PATCH https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/
codelists/b32a4e91e42dee8fb6d7e92d/codevalues HTTP/1.1
    Content-Type: application/json
    Accept: application/json
    IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX

    {
        "action": "CREATE",
        "records": [
            {
                "Name": "NAME-A",
                "Code": "CODE-A",
                "Description": "DESCRIPTION-A",
                "status": "ActiveKey"
                                                                "parentCode": "STEM"
            }
        ]
    }
}
```

The following sample response shows the error report for a failed code value creation:

```
    HTTP/1.1 201 Created
    Location: /subset/71253be5deb0e7d4974935e0/cursor
    Content-Type: application/json
    Content-Length: 39
```

```
{
 "successfulRecords": [],
 "failedRecords": [
     {
         "label": "NAME-A",
         "errorCauses": [
             {
                 "errorCode": "RDM.0010045",
                 "errorSummary": "The code value already exists in the code list.",
                 "errorParameter": {
                     "Code": "CODE-A"
                 }
             }
         ],
         "Code": "CODE-A"
     }
 ],
 "successfulRecordsCount": 0,
 "failedRecordsCount": 1
}
```


Update a code value (v1)

Updates a code value in a code list.

PATCH request

To update a code value in a code list, submit a PATCH request with the following URI:

```
/rdm-service/external/v1/codelists/{listIdentifier}/codevalues
```

The following table describes the parameter in the request:

Parameter	Type	Description
listIdentifier	String	ID of the code list that the code values belong to.

The following table describes the attributes in the request:

Field	Type	Description
action	String	Action to perform on code values. Value is `UPDATE`.
records	Array	List of code values to update. **Note:** Currently, you can include only one code value in the body.
Name	String	Name of the field.
Code	String	The code field of the code value to create.
Description	String	Description of the code value.
status	String	Optional. Status of the code value.

When you specify attributes in the request body, consider the following guidelines:

- The attribute names are case-sensitive.

- Use camel case for name and code attributes and lowercase for other fields, such as dependency.
- Use string data type for a decimal attribute.
- Ensure that you include values for all the fields of an existing code value. If you specify only the field values that you want to update, the REST API updates the specified field values and sets the other field values to null.

Note: You can't update code values that are defined as a parent code value in a hierarchical code list. Use the code value REST API to move a code value to another node within the same hierarchical code list.

To update a code value, ensure that you use the Code attribute value. If you don't use the Code attribute value, the following error appears:

```
The specified code value does not exist
```

When you update a code value in a hierarchical code list, you don't need to use the parentCode attribute.

PATCH response

The response generates a report of code values that were updated.

The following table describes the attributes in the response:

Field	Type	Description
successfulRecords	-	Lists the code values that were updated successfully and describes the details of the code values.
Code	String	Code attribute value for the updated code value.
label	String	Display attribute value for the updated code value.
failedRecords	-	Lists the code values that weren't updated and describes the reasons.
label	String	Display attribute value for the code value that wasn't updated.
Code	String	Code attribute value for the code value that wasn't updated.
errorCauses	-	Error details for the code values that couldn't be updated.
errorCode	String	Error code for the error type.
errorSummary	String	A short summary that explains why the code value wasn't updated.
localizedErrorSummary	String	A short summary that explains the error in the user locale.
errorParameter	String	Parameter that provides details of an error.
successfulRecordsCount	String	Number of code values that were updated successfully.
failedRecordsCount	String	Number of code values that weren't updated.

PATCH example

To update a code value in a code list, you might use the following request:

```
                PATCH https://use4-mdm.dm-us.informaticacloud.com/rdm-service/
external/v1/codelists/b32a4e91e42dee8fb6d7e92d/codevalues HTTP/1.1
    Content-Type: application/json
```

```
    Accept: application/json
    IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

    {
        "action": "UPDATE",
        "records": [
            {
                "Name": "NAME-A",
                "Code": "CODE-A",
                "Description": "DESCRIPTION-A",
                "status": "ActiveKey"
            }
        ]
    }
}
```

The following sample response shows the error report for a failed code value update:

```
                HTTP/1.1 201 Updated
    Location: /subset/71253be5deb0e7d4974935e0/cursor
    Content-Type: application/json
    Content-Length: 39

{
    "successfulRecords": [],
    "failedRecords": [
        {
            "label": "NAME-A",
            "errorCauses": [
                {
                    "errorCode": "RDM.001027674",
                    "errorSummary": "The specified code value does not exist.",
                    "errorParameter": {
                        "Code": "CODE-A"
                    }
                }
            ],
            "Code": "CODE-A"
        }
    ],
    "successfulRecordsCount": 0,
    "failedRecordsCount": 1
}
```

Delete code values (v1)

Deletes one or multiple code values that you no longer need. When you delete code values using the Delete code values API, you directly delete code values without creating a draft changes or sending your changes for approval.

You can't delete the following types of code values:

- Code values that are defined as a parent code value in a hierarchical code list
- Code values that are used as a Reference Data attribute for another code list
- Code values that are used as a dependency in another code list
- Code values that are part of a value mapping in a crosswalk
- Code values that are part of a hierarchy asset

DELETE request

To delete a code value, submit a DELETE request with the following URI:

```
/rdm-service/external/v1/codelists/<code list ID>/codevalues
```

The following table describes the attributes in the request:

Field	Type	Description
Codes	String	Comma-separated list of code values by the value in the Code attribute.

DELETE response

The response contains the deletion report. If the delete failed, the report provides failure reasons.

The following table describes the attributes in the response:

Field	Type	Description
numberOfRecordsDeleted	Number	Number of code values that were deleted successfully.
numberOfRecordsFailed	Number	Number of code values that weren't deleted.
deletedRecords	-	Lists the code values that were deleted successfully.
Code	String	Code attribute value for deleted code value.
Label	String	Display attribute value for deleted code value.
failedRecords	-	Lists the code values that weren't deleted and describes the reasons.
Code	String	Code attribute value for the code value.
label	String	Display attribute value for the code value.
errorCode	String	Error code for the error type.
errorSummary	String	Explains why the code value wasn't deleted.

The display attribute value represents the code value when the code value appears in other assets in Reference 360. For more information, see "Display attributes" on page 31.

DELETE example

To delete code values, you might use the following request:

```
DELETE https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
34cea9471fe977f7decef5f5/codevalues HTTP/1.1
Content-Type: application/json
Content-Length: 30
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX


{
  "Codes":[
    "DE",
    "EN"
  ]
}
```

The following sample response shows the deletion report:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 345

{
```

```
  "deletedRecords":[
    {
      "Code":"DE",
      "label":"DELabel"
    }
  ],
  "failedRecords":[
    {
      "Code":"EN",
      "label":"EnLabel",
      "errorCauses":[
        {
          "errorCode":"RDM.0010209",
          "errorSummary":"The code value is used in a Hierarchy"
        }
      ]
    }
  ],
  "numberOfRecordsDeleted":1,
  "numberOfRecordsFailed":1
}
```

Search code values (v1)

Use the search API to search for code values in all the code lists.

POST request

To search for code values in all the code lists, submit a POST request with the following URI:

```
/rdm-service/external/v1/search
```

Use the following parameters in the search request body:

Field	Type	Description
page	Integer	Optional. Page number to display. Default is 0.
pageSize	Integer	Optional. Number of code values that you want to display on each page. Default is 100. You can specify up to 10000.
search	String	Search string that must contain a minimum of two characters. For example, you can specify the search string as `"IND."` You can use only the values of string data type attributes, such as name, code, description, and other custom string attributes. **Note:** You can't use the values of integer, date, boolean, and decimal data type attributes in the search string. To retrieve all the code values, use an asterisk (*). To retrieve the exact code value that you search for, specify the search string in the following format: `"\"<search_string>\""` For example, to search for the exact code value `AUSTRIA`, specify `"\"AUSTRIA\""` as search string.

POST response

The request response returns the search results.

The response contains the following attributes:

Field	Type	Description
pageSize	Integer	Number of code values displayed on each page.
page	Integer	Number of the page that you retrieved.
totalNumberOfElements	Integer	Total number of code values found.
numberOfElements	Integer	Total number of code values on the current page.
lastPage	Boolean	Indicates whether the current page is the last page of the search results.
firstPage	Boolean	Indicates whether the current page is the first page of the search results.
content	Object	Includes the list of code values.
<entity prefix>._localizedContent	Array	Lists the multilingual attribute values of a code value. For example, rdmvaluebeent_vehiclebrand._localizedContent contains the Name and Description values of the AUDI code value in English (United States), French (France), Urdu, and Japanese (Japan).
_id	String	BCP 47 language code of the values in the object, such as en-US, fr-FR, or ja-JP. The language code matches the code value in the Localization Languages code list.
Name	String	Display name of the attribute in the language.
Description	String	Description of the attribute in the language.
<entity prefix>.Name	String	Display name of the attribute in the display language. For example, rdmvaluebeent_vehiclebrand.Name returns アウディ because the display language is Japanese (Japan). If the attribute has no value in the display language, the response returns the value in the default language.
<entity prefix>.Description	String	Description of the attribute in the display language. If the attribute has no value in the display language, the response returns the value in the default language.
<entity prefix>.Code	String	Code of the code value.
appType	String	Application that the record belongs to.
appEntityType	String	Type of the record. The value CODEVALUE indicates that the record is a code value.
codeListId	String	ID of the code list that contains the code value. For more information, see "Asset IDs" on page 199.
codelistName	String	Name of the code list that contains the code value.
referenceDataSetName	String	Name of the reference data set that the code list belongs to.

POST example

To search for code values in all the code lists, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/search HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
{
   "page":1,
   "pageSize":1,
   "search":"IN"
}
```

The following sample response displays the code values that contain Audi in all the code lists:

```
{
  "pageSize": 4,
  "page": 4,
  "totalNumberOfElements": 20,
  "numberOfElements": 1,
  "lastPage": false,
  "firstPage": true,
  "content": [
    {
  "rdmvaluebeent_vehiclebrand._localizedContent": [
    {
      "_id": "en-US",
      "Name": "AUDI",
      "Description": "Automotive brand"
    },
    {
      "_id": "fr-FR",
      "Name": "AUDI",
      "Description": "une marque automobile"
    },
    {
      "_id": "ur",
      "Name": "آڈی",
      "Description": "آڈی کی تفصیل"
    },
    {
      "_id": "ja-JP",
      "Name": "アウディ",
      "Description": "アウディの説明"
    }
  ],
  "rdmvaluebeent_vehiclebrand.Name": "アウディ",
  "rdmvaluebeent_vehiclebrand.Description": "アウディの説明",
  "rdmvaluebeent_vehiclebrand.Code": "AUDI",
  "appType": "rdm",
  "appEntityType": "CODEVALUE",
  "codeListId": "68a58ccd3ed14f1174b67c33",
  "codelistName": "Enterprise VehicleBrand",
  "referenceDataSetName": "VehicleBrand"
   }
  ]
}
```

Synchronize Data Quality dictionaries with code values of a code list

Synchronizes Data Quality dictionaries with code values of a code list based on its internal ID or alias.

PUT request

To synchronize Data Quality dictionaries with code values of a code list based on its internal ID or alias, submit a PUT request with the following URI:

```
/rdm-service/external/v1/codelists/<list Identifier>/dictionaries/data
```

The following table describes the query parameter:

Field	Type	Description	
list Identifier		String	Internal ID or alias of the code list.

PUT response

The PUT response returns HTTP 200 success code in JSON format.

PUT example

To synchronize Data Quality dictionaries with code values of a code list based on its internal ID or alias, you might use the following request:

```
PUT https://qa-pod1-mdm.rel.infaqa.com/rdm-service/external/v1/codelists/
enterprise_country/dictionaries/data
Accept: application/json
```

The following sample response shows the dictionary synchronization details:

```
HTTP/1.1 200 OK
Content-Type: application/json
{
   "dictionarySynchronizationDetails":[
      {
         "dictionaryConfigurationIdentityAndName":{
            "dictionaryName":"Country Dictionary"
         }
      }
   ]
}
```

Export a subset of code values in a hierarchical code list (v1)

Use this resource to export a subset of code values from a starting node and its ancestors and descendants with specified level in a hierarchical code list.

Create a subset ID for code values in a hierarchical code list (v1)

Creates a subset ID for code values in a hierarchical code list.

POST request

To create a subset ID for code values in a hierarchical code list, submit a POST request with the following URI:

```
/rdm-service/external/v1/codelists/{codelistId}/codevalues/subset
```

Use the following parameters in the request body to specify the code value hierarchy:

Field	Type	Description
type	String	Type of the code values subset. Value must be `ANCESTORS_AND_DESCENDANTS`.
ancestors	-	The ancestors of the code value that you want to export.
depth	Number	Optional. The levels of ancestors of the code value that you want to export.
include	Boolean	The flag to indicate inclusion of ancestors of the code value.
descendants	-	The descendants of the code value that you want to export.
depth	Number	Optional. The levels of descendants of the code value that you want to export.
include	Boolean	The flag to indicate inclusion of descendants of the code value that you want to export.
startNode	-	The starting node of the code value that you want to export.
code	String	The code field of the starting node of the code value that you want to export.
cursorSettings	-	Includes details about the cursor settings.
repeatHeaders	Boolean	Optional. Indicates whether to populate headers in the CSV response. Value is `true` or `false`. Default is `false`.
addLabelsForReferenceAttribute	Boolean	Optional. Displays values for reference and dependent attributes based on the display attributes of reference and dependent code lists, respectively.
delimiter	String	Optional. Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE`, or `TAB`. Default is `COMMA`.
codepage	String	Optional. Code page used for the export file. Value can be `UTF8` or `MS_WINDOWS`. Default is `UTF8`.
decimalSeparator	String	Optional. Decimal separator used for numbers. Value must be `COMMA` or `DOT`. Default is `DOT`.
thousandSeparator	String	Optional. Grouping separator used for numbers. Value can be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE` or `NONE`. Default is `NONE`.
filename	String	File name for the exported file. Value must end with the `.csv` file extension.

Field	Type	Description
dateFormat	String	Format used for dates. Use one of the following formats: - DE. For dd.mm.yyyy format. - ISO. For yyyy-mm-dd format. - US. For mm/dd/yyyy format. Default is ISO.
columns	Array	Optional. Attribute columns that you want to export. If you don't specify attribute columns, the export includes all the attribute columns.
fieldName	String	Optional. Name of attribute column to include in the exported file.
codeListId	String	ID of the code list.
noOfRecordsPerPage	Number	Optional. Number of records to display on each page. The maximum value is 10000. Default is 10000.

POST response

The response contains the subset ID of the code value hierarchy in a hierarchical code list.

POST example

To create a subset ID for code value hierarchy in a hierarchical code list, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
5d352cbb8de0392f8c0710d3/codevalues/subset HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX

{
   "type":"ANCESTORS_AND_DESCENDANTS",
   "ancestors":{
      "include":true,
      "depth":2
   },
   "descendants":{
      "include":true,
      "depth":2
   },
   "startNode":{
      "code":"desiredCodeValueName"
   },
   "cursorSettings":{
      "filename":"codeValueHierarchyExportFile.csv",
      "repeatHeaders":true,
      "delimiter":"COMMA",
      "decimalSeparator":"COMMA",
      "thousandSeparator":"DOT",
      "dateFormat":"DE",
      "addLabelsForReferenceAttribute":true,
      "codepage":"UTF8",
      "columns":[
         {
            "fieldName":"columnName",
            "codeListId":"5d352cbb8de0392f8c0710d3"
         }
      ],
      "noOfRecordsPerPage":10000
   }
}
```

The following sample response shows the subset ID for a code value hierarchy in a hierarchical code list:

```
HTTP/1.1 201 Created
Location: /subset/71253be5deb0e7d4974935e0/cursor
Content-Type: application/json
Content-Length: 39

{
  "id" : "71253be5deb0e7d4974935e0"
}
```

Update cursor settings of the subset of code values (v1)

Updates the cursor settings of the subset of code values.

PUT request

To update the cursor settings of the subset of code values, submit a PUT request with the following URI:

```
/rdm-service/external/v1/codelists/{codelistId}/codevalues/subset/{subsetId}/cursor
```

Note: When you update the cursor setting, the page is reset to zero. The export API exports the subset of code values from the starting page.

Field	Type	Description
filename	String	File name for the exported file. Value must end with the `.csv` file extension.
repeatHeaders	Boolean	Optional. Indicates whether to populate headers in the CSV response. Value is `true` or `false`. Default is `false`.
delimiter	String	Optional. Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE` or `TAB`. Default is `COMMA`.
decimalSeparator	String	Optional. Decimal separator used for numbers. Value must be `COMMA` or `DOT`. Default is `DOT`.
thousandSeparator	String	Optional. Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE` or `NONE`. Default is `NONE`.
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US`. For mm/dd/yyyy format. Default is `ISO`.
addLabelsForReferenceAttribute	Boolean	Optional. Displays values for reference and dependent attributes based on the display attributes of reference and dependent code lists, respectively.
codepage	String	Optional. Code page used for the export file. Value must be `UTF8` or `MS_WINDOWS`. Default is `UTF8`.

Field	Type	Description
columns	Array	Optional. Attribute columns you want to export. If you don't specify attribute columns, the export includes all the attribute columns.
fieldName	String	Optional. Name of attribute column to include in the exported file.
codeListId	String	ID of the code list.
noOfRecordsPerPage	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.

PUT response

A 204 no content response is returned.

PUT example

To update the cursor settings of the subset of code values, you might use the following request:

```
PUT https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
5d352cbb8de0392f8c0710d3/codevalues/subset/71253be5deb0e7d4974935e0/cursor HTTP/1.1
Content-Type: application/json
Accept: application/json
Content-Length: 391

{
   "filename":"codeValueHierarchyExportFile.csv",
   "repeatHeaders":true,
   "delimiter":"COMMA",
   "decimalSeparator":"COMMA",
   "thousandSeparator":"DOT",
   "dateFormat":"DE",
   "addLabelsForReferenceAttribute":true,
   "codepage":"UTF8",
   "columns":[
      {
         "fieldName":"columnName",
         "codeListId":"5d352cbb8de0392f8c0710d3"
      }
   ],
   "noOfRecordsPerPage":10000
}
```

The following sample response shows the no content response:

```
HTTP/1.1 204 No Content
```

Export the subset of code values to a CSV file (v1)

Exports the subset of code values in a hierarchical code list to a CSV file.

GET request

To export the subset of code values in a hierarchical code list, submit a GET request with the following URI:

```
/rdm-service/external/v1/codelists/{codelistId}/codevalues/subset/{subsetId}/cursor?
page={page}
```

Note: This API supports only incremental pagination starting from page `0`. In the request header, you must specify the `Accept` attribute to `application/octet-stream`.

Use the following parameters in the request query to specify the subset of code values to export:

Field	Type	Description
page	Number	Page number to display. Default value is `0`.
limit	Number	Optional. Number of records to display on each page. Default value is `10000`.

GET response

The response is a CSV file.

GET example

To export the subset of code values in a hierarchical code list, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
5d352cbb8de0392f8c0710d3/codevalues/subset/71253be5deb0e7d4974935e0/cursor?page=0
HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the exported subset of code values in a CSV file:

```
HTTP/1.1 200 OK
Content-Type: application/octet-stream
Content-Disposition: attachment;filename=codeValueHierarchyExportFile.csv
RDM-PAGE: 0
RDM-NUMBER-OF-ELEMENTS: 10
RDM-PAGE-SIZE: 1000
Content-Length: 130

Name,Code
Name0,Code0
Name1,Code1
Name2,Code2
Name3,Code3
Name4,Code4
Name5,Code5
Name6,Code6
Name7,Code7
Name8,Code8
Name9,Code9
```

Export the subset of code values to a JSON file (v1)

Exports the subset of code values in a hierarchical code list to a JSON file.

GET request

To export the subset of code values in a hierarchical code list, submit a GET request with the following URI:

```
/rdm-service/external/v1/codelists/{codelistId}/codevalues/subset/{subsetId}/cursor?
page={page}
```

Note: This API supports only incremental pagination starting from page `0`. In the request header, you must specify the `Accept` attribute to `application/json`.

Use the following parameters in the request query to specify the subset of code values to export:

Field	Type	Description
page	Number	Page number to display. Default value is `0`.
limit	Number	Optional. Number of records to display on each page. Default value is `10000`.

GET response

The response is in JSON file.

GET example

To export the subset of code values in a hierarchical code list, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
5d352cbb8de0392f8c0710d3/codevalues/subset/71253be5deb0e7d4974935e0/cursor?page=0
HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the exported subset of code values in the JSON file:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 352

{
   "pageSize":2,
   "page":0,
   "numberOfElements":2,
   "content":[
      {
         "Code":"STUDENT-1001",
         "fields":{
            "Code":"STUDENT-1001",
            "Name":"ppniK"
         },
         "parentId":"UNIVERSITY-001"
      },
      {
         "Code":"UNIVERSITY-001",
         "fields":{
            "Code":"UNIVERSITY-001",
            "Name":"ABC University"
         }
      }
   ]
}
```

Get history of code value hierarchies by time range (v1)

Retrieves all the change events of code value hierarchies for a specific time range.

GET request

To retrieve all the change events of code value hierarchies for a specific time range, submit a GET request with the following URI:

```
/rdm-service/external/v1/codelists/<code list ID>/codevalues/hierarchy/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the parameters in the request:

Parameter	Description
from	Start date and time of a time range. The start date and time is inclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2023-01-01T00:00:00Z`.
to	End date and time of a time range. The end date and time is exclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2024-01-01T00:00:00Z`.
page	Optional. Page number to display. Default is 0.
pageSize	Optional. The number of records to display on each page. Default is 100. Maximum is 10000.
modificationType	Optional. Type of modification. Value can be CREATE or DELETE.

GET response

The response contains the change events of code value hierarchies for a specific time range.

The following table describes the attributes in the response:

Attribute	Type	Description
pageSize	Number	The number of records displayed on each page.
page	Number	Page number displayed.
numberOfElements	Number	The number of events.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of modification events.
asset	Object	Includes details about the code values of the code list.
modificationType	String	The type of change applied to the code values of the code list.
eventTime	String	The date and time the code values were last updated.
userName	String	The user name of the user who initiated the modification.

GET example

To retrieve all the change events of code value hierarchies for a specific time range, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
c2f9323dbec86b2cfe8a8101/codevalues/hierarchy/audit?
from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z HTTP/1.1
```

```
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of the code value hierarchies from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 3441

{
   "pageSize":100,
   "page":0,
   "numberOfElements":6,
   "lastPage":true,
   "firstPage":true,
   "content":[
      {
         "asset":{
            "id":"6407014f5b243e32c4b02ce3",
            "assetType":"CODELIST"
         },
         "codeValueId":"640701e45b243e32c4b02d40",
         "code":"Antarctica",
         "modificationType":"UPDATE",
         "changedAttributes":[
            {
               "attribute":"Description",
               "previousValue":"Numeric Code of Antarctica",
               "newValue":"Numeric Code of Antarctica (ISO 3166-1)"
            }
         ],
         "eventTime":"2023-03-07T09:21:56Z",
         "userName":"integration-test-admin"
      },
      {
         "asset":{
            "id":"6407014f5b243e32c4b02ce3",
            "assetType":"CODELIST"
         },
         "codeValueId":"640701d25b243e32c4b02d35",
         "code":"Albania",
         "modificationType":"UPDATE",
         "changedAttributes":[
            {
               "attribute":"Description",
               "previousValue":"Numeric Code of Albania",
               "newValue":"Numeric Code of Albania (ISO 3166-1)"
            }
         ],
         "eventTime":"2023-03-07T09:21:39Z",
         "userName":"integration-test-admin"
      },
      {
         "asset":{
            "id":"6407014f5b243e32c4b02ce3",
            "assetType":"CODELIST"
         },
         "codeValueId":"640701c35b243e32c4b02d21",
         "code":"Afghanistan",
         "modificationType":"UPDATE",
         "changedAttributes":[
            {
               "attribute":"Description",
               "previousValue":"Numeric Code for Afghanistan",
               "newValue":"Numeric Code for Afghanistan (ISO 3166-1)"
            }
         ],
         "eventTime":"2023-03-07T09:21:19Z",
         "userName":"integration-test-admin"
      },
```

```
{
   "asset":{
      "id":"6407014f5b243e32c4b02ce3",
      "assetType":"CODELIST"
   },
   "codeValueId":"640701c35b243e32c4b02d21",
   "code":"Afghanistan",
   "modificationType":"CREATE",
   "changedAttributes":[
      {
         "attribute":"Code",
         "newValue":"Afghanistan"
      },
      {
         "attribute":"Description",
         "newValue":"Numeric Code for Afghanistan"
      },
      {
         "attribute":"Name",
         "newValue":"Afganistan"
      },
      {
         "attribute":"NumericCode",
         "newValue":"004"
      }
   ],
   "eventTime":"2023-03-07T09:20:43Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407014f5b243e32c4b02ce3",
      "assetType":"CODELIST"
   },
   "codeValueId":"640701d25b243e32c4b02d35",
   "code":"Albania",
   "modificationType":"CREATE",
   "changedAttributes":[
      {
         "attribute":"Code",
         "newValue":"Albania"
      },
      {
         "attribute":"Description",
         "newValue":"Numeric Code of Albania"
      },
      {
         "attribute":"Name",
         "newValue":"Albania"
      },
      {
         "attribute":"NumericCode",
         "newValue":"008"
      }
   ],
   "eventTime":"2023-03-07T09:20:43Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"6407014f5b243e32c4b02ce3",
      "assetType":"CODELIST"
   },
   "codeValueId":"640701e45b243e32c4b02d40",
   "code":"Antarctica",
   "modificationType":"CREATE",
   "changedAttributes":[
      {
         "attribute":"Code",
         "newValue":"Antarctica"
      },
```

```
                {
                   "attribute":"Description",
                   "newValue":"Numeric Code of Antarctica"
                },
                {
                   "attribute":"Name",
                   "newValue":"Antarctica"
                },
                {
                   "attribute":"NumericCode",
                   "newValue":"010"
                }
             ],
             "eventTime":"2023-03-07T09:20:43Z",
             "userName":"integration-test-admin"
          }
       ]
    }
```

Get crosswalks for a code list (v1)

Retrieves the crosswalks associated to a code list and the summary information for each crosswalk.

GET request

To retrieve all crosswalks associated to a code list, submit a GET request with the following URI:

```
/rdm-service/external/v1/codelists/<code list ID>/crosswalks
```

Note: You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

GET response

The response contains the crosswalks associated to the code list.

The following table describes the attributes in the response:

Field	Type	Description
id	String	ID of the asset. Assets include reference data sets, code lists, crosswalks, and hierarchies. For more information, see "Asset IDs" on page 199.
sourceCodelistId	String	ID of the source code list to which the crosswalk is associated.
targetCodelistId	String	ID of the target code list.
description	String	Optional. Description of asset.
status	String	Optional. Status of the asset.
confidentiality	String	Optional. Confidentiality of the asset.
sourceApplication	String	Optional. Application of the source code list.
targetApplication	String	Optional. Application of the target code list.
effectiveDate	String	Optional. Date the asset became effective.
approvedById	String	Optional. ID of the approver of the asset.

Field	Type	Description
approvedByName	String	Optional. User name of the user who approved the asset.
approvedOn	String	Optional. Date the asset was approved.

GET example

To retrieve the crosswalks associated to a code list, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
eff26036111cbb1e9c03f21f/crosswalks HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the crosswalks associated to a code list:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 514

[
  {
    "id":"242a0d14fc1149843cf00626",
    "description":"Sample Crosswalk 1",
    "status":"active",
    "sourceCodelistId":"eff26036111cbb1e9c03f21f",
    "targetCodelistId":"6253315d698f95a216e15a5e"
  },
  {
    "id":"cd11cae0400cf0eaaedfd711",
    "description":"Sample Crosswalk 2",
    "status":"inactive",
    "sourceCodelistId":"eff26036111cbb1e9c03f21f",
    "targetCodelistId":"1bf325b44008a5ba4034c23c",
    "confidentiality":"private",
    "effectiveDate":"2007-04-01",
    "approvedOn":"2017-03-01"
  }
]
```

Get latest modified code values by time range (v1)

Retrieves the latest modified code values in a code list for a time range.

By default, the request returns the first 100 records. To retrieve more records or to view the next page of records, use the query parameters. You can retrieve a maximum of 10000 records in a request.

GET request

To retrieve the latest modified code values in a code list for a time range, submit a GET request with the following URI:

```
/rdm-service/external/v1/codelists/<code list ID>/codevalues/modifications?
from=<from>&to=<to>
```

To specify the page number and page size, submit a GET request with the following query parameters appended to the URI:

```
/rdm-service/external/v1/codelists/<code list ID>/codevalues/modifications?
from=<from>&to=<to>&pageSize=<page size>&page=<page number>
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, and page size.

The following table lists the query parameters:

Parameter	Description
from	Start date and time of the time range, which is inclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-12T14:04:04Z`.
to	End date and time of the time range, which is exclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-15T14:04:04Z`.
page	Optional. Page number to display. Default is `0`.
pageSize	Optional. Number of records to display per page. Default value is `100`. Maximum value is `10000`.

GET response

The response contains details about the latest modified code values.

The following table describes the attributes in the response:

Field	Type	Description
page	Number	Page from which the response was retrieved.
pageSize	Number	Number of records displayed per page.
totalNumberOfElements	Number	Total number of records found.
numberOfElements	Number	Number of records returned in the current page.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of code values.
status	String	Optional. Status of the code value.
effectiveDate	String	Optional. Date code value becomes effective.
codeValueFields	Object	Contains the field values for the code value.
lastUpdateDate	String	Date when the code value was last updated.
changeType	Object	Type of change made to the code value. Values are `MODIFIED` or `DELETED`. **Note:** The MODIFIED change type appears for both new and updated code values.

GET example

To retrieve the first page of the latest modified code values for a time range, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
ce63c85efb9791eb49c4baa3/codevalues/modifications?
from=2019-12-11T13:29:55Z&to=2019-12-12T13:29:55Z&page=0&pageSize=100 HTTP/1.1
Accept: application/json
```

The following sample response shows the first page of the latest modified code values:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 412

{
  "pageSize":100,
  "page":0,
  "totalNumberOfElements":1000,
  "numberOfElements":1,
  "lastPage":false,
  "firstPage":true,
  "content":[
    {
      "status":"Draft",
      "dependency":"f9b48b61fbeb3b491a69bd44",
      "lastUpdateDate":"2019-12-11T13:29:55Z",
      "changeType":"MODIFIED",
      "effectiveDate":"2017-04-01",
      "codeValueFields":{
        "field1":"Some value"
      }
    }
  ]
}
```

Known limitations

- If the code list is in draft state, some code values might appear in the API response even though the code values have not changed in the specified time range. For example, if you delete a code value in a draft code list, the last update date of the code value updates to the date the code value was deleted in the draft. This means that the deleted code value in the draft code list might meet the specified time range criteria now.

Get modified code value relationships in hierarchy by time range (v1)

Retrieves the modified code value relationships in a hierarchy for a time range. Use the API to retrieve changes to code value relationships, such as code values that moved in the hierarchy and code values added under a parent code value.

By default, the request returns the first 100 records. To retrieve more records or to view the next page of records, use the query parameters.

GET request

To retrieve the modified code values in a hierarchical code list for a time range, submit a GET request with the following URI:

```
/rdm-service/external/v1/codelists/<code list ID>/codevalues/hierarchy/modifications?
from=<from>&to=<to>
```

To specify the page number and page size, submit a GET request with the following query parameters appended to the URI:

```
/rdm-service/external/v1/codelists/<code list ID>/codevalues/hierarchy/modifications?
from=<from>&to=<to>&pageSize=<page size>&page=<page number>
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, and page size.

The following table lists the query parameters:

Parameter	Description
from	Start date and time of the time range, which is inclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-12T14:04:04Z`.
to	End date and time of the time range, which is exclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-15T14:04:04Z`.
page	Optional. Page number to display. Default is `0`.
pageSize	Optional. Number of records to display per page. Default value is `100`. Maximum value is `10000`.

GET response

The response contains data about the modified code value relationships for the specified time range.

The following table describes the attributes in the response:

Field	Type	Description
page	Number	Page from which the response was retrieved.
pageSize	Number	Number of records displayed per page.
totalNumberOfElements	Number	Total number of records found.
numberOfElements	Number	Number of records returned in the current page.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of code values.
lastUpdateDate	String	Date when the code value was last updated.
parentCode	String	Code value of the parent node.
childCode	String	Code value of the child node.

GET example

To retrieve the first page of modified code values relationships for a time range, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/123456/
codevalues/hierarchy/modifications?
```

```
from=2019-12-11T13:29:55Z&to=2019-12-12T13:29:55Z&page=0&pageSize=100 HTTP/1.1
Accept: application/json
Host: localhost:8080
```

The following sample response shows the first page of modified code value relationships:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 261

{
  "pageSize":100,
  "page":0,
  "totalNumberOfElements":1000,
  "numberOfElements":1,
  "lastPage":false,
  "firstPage":true,
  "content":[
    {
      "lastUpdateDate":"2019-12-11T13:29:55Z",
      "parentCode":"USA",
      "childCode":"NY"
    }
  ]
}
```

Known limitations

- If the code list is in draft state, some code value relationships might appear in the API response even though the code value relationships have not changed in the specified time range. For example, if you move a code value in a draft code list, the last update date of the code value updates to the date the code value was moved in the draft. This means that the moved code value in draft code list might meet the specified time range criteria now.

Get modified code lists by time range (v1)

Retrieves the modified code lists for a time range. Modified code lists include code lists that users published or code values that users directly imported during the time range.

By default, the request returns the first 100 records. To retrieve more records or to view the next page of records, use the query parameters. You can retrieve a maximum of 10000 records in a request.

GET request

To retrieve the modified code lists for a time range, submit a GET request with the following URI:

```
/rdm-service/external/v1/codelists/modifications?from=<from>&to=<to>
```

To specify the page number and page size, submit a GET request with the following query parameters appended to the URI:

```
/rdm-service/external/v1/codelists/modifications?from=<from>&to=<to>&pageSize=<page
size>&page=<page number>
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, and page size.

Parameter	Description
from	Start date and time of the time range, which is inclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-12T14:04:04Z`.
to	End date and time of the time range, which is exclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-15T14:04:04Z`.
page	Optional. Page number to display. Default is `0`.
pageSize	Optional. Number of records to display per page. Default value is `100`. Maximum value is `10000`.

GET response

The response contains data about the modified code lists.

The following table describes the attributes in the response:

Field	Type	Description
page	Number	Page from which the response was retrieved.
pageSize	Number	Number of records displayed per page.
totalNumberOfElements	Number	Total number of records found.
numberOfElements	Number	Number of records returned in the current page.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of code values.
id	String	Included in the content object. ID of the code list.
updateDate	String	Included in the content object. Last updated date of the change.
changeType	String	Included in the content object. Type of change made to the code list. Values are `DIRECT_IMPORT` and `PUBLISH_DRAFT`.

GET example

To retrieve the first page of modified code lists for a time range, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/codelists/
modifications?from=2019-12-11T13:29:55Z&to=2019-12-12T13:29:55Z&page=0&pageSize=100
HTTP/1.1
Accept: application/json
```

The following sample response shows the first page of modified code lists:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 282

{
  "pageSize":100,
  "page":0,
  "totalNumberOfElements":1000,
  "numberOfElements":1,
  "lastPage":true,
  "firstPage":true,
  "content":[
    {
      "id":"3f23f14a44bae0f6e2bb8aa2",
      "updateDate":"2019-12-11T13:29:55Z",
      "changeType":"DIRECT_IMPORT"
    }
  ]
}
```

codelists version 2

Use this resource to retrieve the details of a code list, a code value, and the crosswalks associated to a code list based on its internal ID or alias. You can also unlock code lists locked by other users.

Note: To identify a code list based on its name, use the codelists version 1 REST APIs.

Unlock a code list (v2)

Unlocks a code list that's locked by another user.

Note: To use the API, you require the IDMC Reference 360 Administrator role.

PUT request

To unlock a code list, submit a PUT request with the following URI:

```
/rdm-service/external/v2/codelists/<code list ID>/unlock
```

Note: You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

PUT response

A 202 accepted response is returned.

PUT example

To unlock a code list, you might use the following request:

```
PUT https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
653a20e37659bf7eceecafa5/unlock HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the accepted response:

```
HTTP/1.1 202 Accepted
```

Create a code value (v2)

Creates a code value in a code list that has an internal ID or alias configured. You can directly create code values without creating draft changes or sending your changes for approval.

PATCH request

To create a code value in a code list, submit a PATCH request with the following URI:

```
/rdm-service/external/v2/codelists/{codeListKey}/codevalues
```

The following table describes the parameter in the request:

Parameter	Type	Description
codeListKey	String	Internal ID or alias of the code list to which you want to add code values.

The following table describes the attributes in the request body:

Attributes	Type	Description
action	String	Action to perform on code values. Set to `CREATE`.
records	Array	Details of the code value to create. **Note:** Currently, you can include only one code value in the body.
Name	String	Name for the code value.
Code	String	Code attribute value for the code value.
Description	String	Optional. Short description for the code value.
status	String	Optional. Status of the code value in the life cycle.
parentCode	String	Optional. Code value of the top-level parent node to which you want to add the new code value a child node. To add a node to a top-level parent code value in a hierarchical code list, use the parentCode attribute. When you don't specify the parentCode attribute, Reference 360 adds the specified code values to the top-level parent node of the hierarchy.
dependency	String	Optional. Code value of a code list that is used as a dependency in another code list. To add a code value to a dependent code list, use the dependency attribute.

When you specify attributes in the request body, consider the following guidelines:

- The attribute names are case-sensitive.
- Use camel case for name and code attributes and lowercase for other fields, such as dependency.
- Use string data type for a decimal attribute.

When you create a code value with data quality rule associations assigned to a Code attribute and set the value of the code attribute within a code list, the rule statement overrides the value.

PATCH response

The response generates a report of code values that were created.

The following table describes the attributes in the response body:

Attributes	Type	Description
successfulRecords	Array	An array that lists the successfully created code values and their details.
Code	String	Code attribute value for the created code value.
label	String	Display attribute value for the created code value.
failedRecords	Array	An array that lists the code values that weren't created and describes the reasons.
label	String	Display attribute value for the code value that wasn't created.
Code	String	Code attribute value for the code value that wasn't created.
errorCauses	-	Error details for the code values that weren't created.
errorCode	String	Error code for the error type.
errorSummary	String	A short summary that explains why the code values weren't created.
localizedErrorSummary	String	A short summary that explains the error in the user locale.
errorParameter	String	Parameter that provides details of an error.
successfulRecordsCount	String	Number of code values that were created successfully.
failedRecordsCount	String	Number of code values that weren't created.

PATCH example

To create a code value in a hierarchical code list based on its internal ID `country`, you might use the following request:

```
    PATCH https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
country/codevalues HTTP/1.1
    Content-Type: application/json
    Accept: application/json
    IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX

    {
  "action": "CREATE",
  "records": [
    {
      "Name": "NAME-A",
      "Code": "CODE-A",
      "Description": "DESCRIPTION-A",
      "status": "ActiveKey",
      "parentCode": "STEM"
    }
  ]
}
```

To create a code value in a dependent code list based on its internal ID `_ent_state`, you might use the following request:

```
    PATCH https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
_ent_state/codevalues HTTP/1.1
    Content-Type: application/json
```

```
  Accept: application/json
  IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

  {
"action": "CREATE",
"records": [
  {
    "Name": "Karnataka",
    "Code": "KA",
    "Description": "Karnataka - India",
    "dependency": "IN"
  }
]
}
```

The following sample response shows the report for a successful code value creation:

```
  HTTP/1.1 200 OK
  Content-Type: application/json
  Content-Length: 39

 {
"successfulRecords": [
  {
    "Code": "CODE-A",
    "label": "NAME-A"
  }
],
"failedRecords": [],
"successfulRecordsCount": 1,
"failedRecordsCount": 0
}
```

Update a code value (v2)

Updates a code value in a code list based on the internal ID or alias of the code list. You can directly update code values without creating draft changes or sending your changes for approval.

You can't update code values that are defined as a parent code value in a hierarchical code list. To move a code value to another node within the same hierarchical code list, see Move a code valueon page 428.

When you update a code value in a hierarchical code list, you don't need to use the parentCode attribute.

PATCH request

To update a code value in a code list, submit a PATCH request with the following URI:

```
/rdm-service/external/v2/codelists/{codeListKey}/codevalues
```

The following table describes the parameter in the request:

Parameter	Type	Description
codeListKey	String	Internal ID or alias of the code list to which you want to update the code values.

The following table describes the attributes in the request body:

Field	Type	Description
action	String	Action to perform on code values. Set to `UPDATE`.
records	Array	Details of the code value to update. **Note:** Currently, you can include only one code value in the body.
Name	String	Name for the code value.
Code	String	Code attribute value for the code value.
Description	String	Short description for the code value.
status	String	Optional. Status of the code value in the life cycle.

When you specify attributes in the request body, consider the following guidelines:

- The attribute names are case-sensitive.
- Use camel case for name and code attributes and lowercase for other fields, such as dependency.
- Use string data type for a decimal attribute.
- Ensure that you include values for all the fields of an existing code value. If you specify only the field values that you want to update, the REST API updates the specified field values and sets the other field values to null.

To update a code value, ensure that you use the Code attribute value. If you don't use the Code attribute value, the following error appears:

```
The specified code value does not exist
```

PATCH response

The response generates a report of code values that were updated.

The following table describes the attributes in the response:

Field	Type	Description
successfulRecords	Array	An array that lists the successfully updated code values and their details.
Code	String	Code attribute value for the updated code value.
label	String	Display attribute value for the updated code value.
failedRecords	Array	An array that lists the code values that weren't updated and describes the reasons.
label	String	Display attribute value for the code value that wasn't updated.
Code	String	Code attribute value for the code value that wasn't updated.
errorCauses	-	Error details for the code values that weren't updated.
errorCode	String	Error code for the error type.

Field	Type	Description
errorSummary	String	A short summary that explains why the code values weren't updated.
localizedErrorSummary	String	A short summary that explains the error in the user locale.
errorParameter	String	Parameter that provides details of an error.
successfulRecordsCount	String	Number of code values that were updated successfully.
failedRecordsCount	String	Number of code values that weren't updated.

PATCH example

To update a code value in a code list based on its internal ID `country`, you might use the following request:

```
PATCH https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
country/codevalues HTTP/1.1
    Content-Type: application/json
    Accept: application/json
    IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

    {
  "action": "UPDATE",
  "records": [
    {
      "Name": "NAME-A",
      "Code": "CODE-A",
      "Description": "DESCRIPTION-A",
      "status": "ActiveKey"
    }
  ]
}
```

The following sample response shows the report for a successful code value update:

```
    HTTP/1.1 200 OK
    Content-Type: application/json
    Content-Length: 39
    {
  "successfulRecords": [
    {
      "Code": "CODE-A",
      "label": "NAME-A"
    }
  ],
  "failedRecords": [],
  "successfulRecordsCount": 1,
  "failedRecordsCount": 0
```

Move a code value (v2)

Moves a code value to another node within a hierarchical code list without locking the code list. You can directly move code values without creating draft changes or sending your changes for approval based on the internal ID or alias.

PATCH request

To move a code value to another node within the same hierarchical code list, submit a PATCH request with the following URI:

```
/rdm-service/external/v2/codelists/{codeListKey}/codevalues
```

The following table describes the parameter in the request:

Parameter	Type	Description
codeListKey	String	Internal ID or alias of a hierarchical code list that you want to update.

The following table describes the attributes in the request body:

Field	Type	Description
action	String	Action to perform on code values. Set to `MOVE`.
records	Array	Details of the code value to move.
Code	String	Code attribute value for the code value.
parentCode	String	Optional. Code value of the parent node to which you want to move the code value. By default, Reference 360 adds the specified code values to the parent node of the hierarchy.

PATCH response

The response generates a report of code values that were moved.

The following table describes the attributes in the response body:

Field	Type	Description
successfulRecords	Array	An array that lists the successfully moved code values and their details.
Code	String	Code attribute value for the moved code value.
label	String	Display attribute value for the moved code value.
failedRecords	Array	An array that lists the code values that weren't moved and describes the reasons.
Code	String	Code attribute value for the code value.
label	String	Display attribute value for the code value.
errorCauses	-	Error details for the code values that weren't moved.
errorCode	String	Error code for the error type.
errorSummary	String	A short summary that explains why the code value wasn't moved.
localizedErrorSummary	String	A short summary that explains the error in the user locale.
errorParameter	String	Parameter the provides details of an error.
successfulRecordsCount	String	Number of code values that were moved successfully.
failedRecordsCount	String	Number of code values that weren't moved.

PATCH example

To move a code value to another node within a hierarchical code list based on its internal ID `country`, you might use the following request:

```
PATCH https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
country/codevalues HTTP/1.1
    Content-Type: application/json
    Accept: application/json
    IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

    {
        "action": "MOVE",
        "records": [
            {
                "Code": "CODE-A",
                "parentCode": "CODE-B"
            }
        ]
    }
}
```

The following sample response shows the report for moved code values:

```
    HTTP/1.1 200 OK
    Location: /subset/71253be5deb0e7d4974935e0/cursor
    Content-Type: application/json
    Content-Length: 39

   {
  "successfulRecords": [
    {
      "Code": "CODE-A"
    }
  ],
  "failedRecords": [],
  "successfulRecordsCount": 1,
  "failedRecordsCount": 0
}
```

Delete code values (v2)

Deletes code values of a code list that you no longer need based on the internal ID or alias of the code list. You can directly delete code values without creating draft changes or sending your changes for approval.

You can't delete the following types of code values:

- Code values that are defined as a parent code value in a hierarchical code list
- Code values that are used as a Reference Data attribute in another code list
- Code values that are used as a dependency in another code list
- Code values that are part of a value mapping in a crosswalk
- Code values that are part of a hierarchy asset

PATCH request

To delete a code value, submit a PATCH request with the following URI:

```
/rdm-service/external/v2/codelists/{codeListKey}/codevalues
```

The following table describes the parameter in the request:

Parameter	Type	Description
codeListKey	String	Internal ID or alias of the code list that contains the code values to delete.

The following table describes the attributes in the request body:

Field	Type	Description
action	String	Action to perform on code values. Set to `DELETE`.
records	Array	List of code values to delete. **Note:** You can delete 50 code values at a time.
Code	String	The code field for the code value.

PATCH response

The response generates a report of code values that were deleted.

The following table describes the attributes in the response body:

Field	Type	Description
successfulRecords	Array	An array that lists the code values that were deleted successfully.
Code	String	Code attribute value for deleted code value.
label	String	Display attribute value for deleted code value.
failedRecords	Array	An array that lists the code values that weren't deleted and describes the reasons.
Code	String	Code attribute value for the code value.
label	String	Display attribute value for the code value.
errorCauses	-	Error details for the code values that weren't deleted.
errorCode	String	Error code for the error type.
errorSummary	String	Explains why the code value wasn't deleted.
localizedErrorSummary	String	A short summary that explains the error in the user locale.
errorParameter	String	Parameter that provides details of an error.
successfulRecordsCount	String	Number of code values that were deleted successfully.
failedRecordsCount	String	Number of code values that weren't deleted.

PATCH example

To delete code values in a code list based on its internal ID `country`, you might use the following request body:

```
PATCH https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
country/codevalues HTTP/1.1
Content-Type: application/json
Content-Length: 30
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
{
        "action": "DELETE",
        "records": [
            {
                "Code": "CODE-A"
            }
        ]
    }
}
```

The following sample response shows the deletion report:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 34

{
  "successfulRecords": [
    {
      "Code": "CODE-A"
    }
  ],
  "failedRecords": [],
  "successfulRecordsCount": 1,
  "failedRecordsCount": 0
}
```

Get code value details (v2)

Retrieves the details of a code value in a code list based on the internal ID or alias of the code list.

GET request

To retrieve the details of a code value, submit a GET request with the following URI and specify the code:

```
/rdm-service/external/v2/codelists/{codeListKey}/codevalues?Code={code}
```

The following table describes the parameter in the request body:

Parameter	Type	Description
codeListKey	String	Internal ID or alias of the code list that contains the code value.
Code	String	The code field for the code value.

GET response

The response contains the details of the code value.

The following table describes the attributes in the response body:

Field	Type	Description
codelistId	String	ID of the code list that contains the code value.
internalId	String	Internal ID of the code list that contains the code value.
alias	String	Alias of the code list that contains the code value.
businessId		Business ID of the code value.
codeValueFields	Array	Lists the attribute field values for the code value.
Code	String	The code field for the code value.
Name	String	The name field for the code value.

GET example

To retrieve the first page of code values in a code list based on its internal ID `country`, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
country/codevalues?Code=CODE-A HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the details of a code value:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 237

[
  {
    "codelistId": "67e2590c022698651e1964c8",
    "internalId": "code1",
    "alias": "alias1",
    "businessId": "RDM0000123",
    "codeValueFields": {
      "Code": "CODE-A",
      "Name": "NAME-A"
    }
  }
]
```

Get latest modified code values by time range (v2)

Retrieves the latest modified code values in a code list that were created or updated during the specified period based on the internal ID or alias.

By default, the request returns the first 100 code values. To retrieve more code values or to view the next page of code values, use the query parameters. You can retrieve a maximum of 10,000 code values in a request.

GET request

To retrieve the code values in a code list that were created or updated during the specified period based on the internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/codelists/{codeListKey}/modifications?
from=<from>&to=<to>[&pageSize=<page size>&page=<page number>]
```

The following table describes the parameter in the URI:

Parameter	Type	Description
codeListKey	String	Internal ID or alias of the code list from which you are retrieving the code values.
from	String	Start date and time of the time range, which is inclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-12T14:04:04Z`.
to	String	End date and time of the time range, which is exclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-15T14:04:04Z`.
page	Integer	Optional. Page number to display. Default is `0`.
pageSize	Integer	Optional. Number of records to display per page. Default value is `100`. Maximum value is `10000`.

GET response

The response contains data about the created or updated code lists.

The following table describes the attributes in the response body:

Field	Type	Description
page	Number	Page from which the response was retrieved.
pageSize	Number	Number of records displayed per page.
totalNumberOfElements	Number	Total number of records found.
numberOfElements	Number	Number of records returned in the current page.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of code values.
status	String	Status of the code list.
dependency	String	ID of the code value specified as the dependency.
lastUpdateDate	String	Date when the code value was last updated.
changeType	String	Type of change made to the code list. Values are `MODIFIED` and `DELETED`.
effectiveDate	String	Date when the code list became effective.
codeValueFields	Object	Includes the attribute definition for code values.

GET example

To retrieve the first page of code values in a code list that were created or updated during a specific time period based on its internal ID `country` , you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
country/codevalues/modifications?
from=2019-12-11T13:29:55Z&to=2019-12-12T13:29:55Z&page=0&pageSize=100 HTTP/1.1
Accept: application/json
```

The following sample response shows the first page of modified code lists:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 282
{
  "pageSize": 1,
  "page": 0,
  "totalNumberOfElements": 1000,
  "numberOfElements": 1,
  "lastPage": false,
  "firstPage": true,
  "content": [
    {
      "status": "Draft",
      "dependency": "870d2e1b6cd2249aa48dc5db",
      "lastUpdateDate": "2019-12-11T13:29:55Z",
      "changeType": "MODIFIED",
      "effectiveDate": "2017-04-01",
      "codeValueFields": {
        "Name": "NAME-A"
      }
    }
  ]
}
```

Get modified code value relationships in hierarchy by time range (v2)

Retrieves code value relationships in a hierarchy that were created or updated during the specified period based on the internal ID or alias. Use the API to retrieve changes to code value relationships, such as code values that moved in the hierarchy and code values added under a parent code value for code lists with internal IDs or aliases.

By default, the request returns the first 100 code values. To retrieve more code values or to view the next page of code values, use the query parameters.

GET request

To retrieve the modified code values in a hierarchical code list for a time range, submit a GET request with the following URI:

```
/rdm-service/external/v2/codelists/{codeListKey}/codevalues/hierarchy/modifications?
from=<from>&to=<to>[&pageSize=<page size>&page=<page number>
]
```

The following table lists the query parameters in the URI:

Parameter	Type	Description
codeListKey	String	Internal ID or alias of the code list from which you are retrieving the code values.
from	String	Start date and time of the time range, which is inclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-12T14:04:04Z`.
to	String	End date and time of the time range, which is exclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-15T14:04:04Z`.
page	Integer	Optional. Page number to display. Default is `0`.
pageSize	Integer	Optional. Number of records to display per page. Default value is `100`. Maximum value is `10000`.

GET response

The response contains data about the created or updated code value relationships for the specified time range.

The following table describes the attributes in the response body:

Field	Type	Description
page	Number	Page from which the response was retrieved.
pageSize	Number	Number of records displayed per page.
totalNumberOfElements	Number	Total number of records found.
numberOfElements	Number	Number of records returned in the current page.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of code values.
lastUpdateDate	String	Date when the code value was last updated.
parentCode	String	Code value of the parent node.
childCode	String	Code value of the child node.

GET example

To retrieve the first page of code values relationships that were created or updated during a specific time period based on its internal ID `country`, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
country/codevalues/hierarchy/modifications?
from=2019-12-11T13:29:55Z&to=2019-12-12T13:29:55Z&page=0&pageSize=100 HTTP/1.1
Accept: application/json
Host: localhost:8080
```

The following sample response shows the first page of code value relationships that were created or updated during a specific time period:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 261

{
  "pageSize":1,
  "page":0,
  "totalNumberOfElements":1000,
  "numberOfElements":1,
  "lastPage":false,
  "firstPage":true,
  "content":[
    {
      "lastUpdateDate":"2019-12-11T13:29:55Z",
      "parentCode":"USA",
      "childCode":"NY"
    }
  ]
}
```

Note: If the code list is in draft state, some code value relationships might appear in the API response even though the code value relationships have not changed in the specified time range. For example, if you move a code value in a draft code list, the last update date of the code value updates to the date on which the code value was moved in the draft code list. The moved code value in draft code list might now meet the specified time range criteria.

Get modified code lists by time range (v2)

Retrieves the modified code lists for a time range based on the internal ID or alias. Modified code lists include code lists that users published or code values that users directly imported during the time range.

By default, the request returns the first 100 records. To retrieve more records or to view the next page of records, use the query parameters. You can retrieve a maximum of 10000 records in a request.

GET request

To retrieve the modified code lists for a time range based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/codelists/modifications?from=<from>&to=<to>
```

To specify the page number and page size, submit a GET request with the following query parameters appended to the URI:

```
/rdm-service/external/v2/codelists/modifications?from=<from>&to=<to>&pageSize=<page
size>&page=<page number>
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, and page size.

Parameter	Description
from	Start date and time of the time range, which is inclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-12T14:04:04Z`.
to	End date and time of the time range, which is exclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-15T14:04:04Z`.

Parameter	Description
page	Optional. Page number to display. Default is `0`.
pageSize	Optional. Number of records to display per page. Default value is `100`. Maximum value is `10000`.

GET response

The response contains data about the modified code lists.

The following table describes the attributes in the response body:

Field	Type	Description
page	Number	Page from which the response was retrieved.
pageSize	Number	Number of records displayed per page.
totalNumberOfElements	Number	Total number of records found.
numberOfElements	Number	Number of records returned in the current page.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of code values.
id	String	Included in the content object. ID of the code list.
updateDate	String	Included in the content object. Last updated date of the change.
changeType	String	Included in the content object. Type of change made to the code list. Values are `DIRECT_IMPORT` and `PUBLISH_DRAFT`.
internalId	String	Unique identifier of the code list.
alias	String	Alternative identifier of the code list.

GET example

To retrieve the first page of modified code lists for a time range based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
modifications?from=2019-12-11T13:29:55Z&to=2019-12-12T13:29:55Z&page=0&pageSize=100
HTTP/1.1
Accept: application/json
```

The following sample response shows the first page of modified code lists:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 282
```

```
{
  "pageSize":100,
  "page":0,
  "totalNumberOfElements":1000,
  "numberOfElements":1,
  "lastPage":true,
  "firstPage":true,
  "content":[
    {
      "id":"3f23f14a44bae0f6e2bb8aa2",
      "updateDate":"2019-12-11T13:29:55Z",
      "changeType":"DIRECT_IMPORT"
      "internalId": "isocountry",
      "alias": "al_isocountry"
    }
  ]
}
```


Get code list details (v2)

Retrieves the details of a code list, such as the properties, status, structure definition, and attributes based on the internal ID or alias.

GET request

To retrieve the details of a code list based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/codelists/{codeListKey}
```

GET response

The response contains the details of the specified code list.

The following table describes the attributes in the response body:

Field	Type	Description
id	String	ID of the asset. For more information, see "Asset IDs" on page 199.
internalId	String	Unique identifier of the code list.
alias	String	Alternative identifier of the code list.
rdsDetails	-	Includes details for the reference data set.
id	String	ID of the reference data set.
internalID	String	Unique identifier of the reference data set.
alias	String	Alternative identifier of the reference data set.
name	String	Name of the code list.
description	String	Optional. Description of the code list.
version	String	Optional. Version of the code list.
application	String	Optional. Application that uses the code list.

Field	Type	Description
hierarchical	Boolean	Optional. Indicates whether code lists in the reference data set inherit the hierarchical structure definition.
levels	Number	Optional. Number of hierarchical levels supported in the code lists associated with the reference data set. If hierarchical is false or levels aren't provided, value is `1`. If levels are unlimited, value is `-1`.
domain	String	Optional. Domain of the asset.
confidentiality	String	Optional. Confidentiality of the asset.
priority	String	Optional. Priority of the asset.
status	String	Optional. Status of the asset.
effectiveDate	String	Optional. Date the asset became effective.
approvedById	String	Optional. ID of the approver of the asset.
approvedByName	String	Optional. User name of the user who approved the asset.
approvedOn	String	Optional. Date the asset was approved.
displayColumns	String	Optional. List of display columns used as labels for code values. Default is `name`.
codeValueFields	-	Includes the attribute definition for code values in the reference data set.
name	String	Name of the field.
origin	String	Origin of the definition of the field.
datatype	String	Datatype of the field. Values are `String`, `Integer`, `Decimal`, `Boolean`, `Date`, or `Reference`.
mandatory	Boolean	Indicates whether the attribute is required.
labels	Array	Includes list of display names for the fields from the target code list.
language	String	Language in which the field name is displayed.
value	String	Value for the field.
relatedRdsDetails	Array	Includes details of the related reference data set.
id	String	ID of the reference data set. For more information, see "Asset IDs" on page 199.
internalId	String	Unique identifier of the reference data set.
alias	String	Alternative identifier of the reference data set.
relatedListDetails	Array	Includes details of the related code list.

Field	Type	Description
id	String	ID of the code list. For more information, see "Asset IDs" on page 199.
internalId	String	Unique identifier of the code list.
alias	String	Alternative identifier of the code list.
displayColumns		Optional. List of display fields used as labels for the code list.
language	String	Language in which the field name is displayed.
value	String	Value for the field.
dependencyDef	-	Optional. Includes the definition of the asset specified as the dependency.
rdsDetails	-	Includes details for the reference data set.
id	String	Asset ID of the reference data set. For more information, see "Asset IDs" on page 199.
internalID	String	Unique identifier of the reference data set.
externalContacts	Array	Includes details of the external contacts to send notifications.
contactName	String	Name of the contact.
contactType	String	Type of contact, such as email or SMS.
contactDetail	String	Details of contact, such as email address or phone number.
externalUrls	Array	Includes details of the external URLs of the code list.
urlName	String	Name of the external URL.
urlLink	String	Link for the external URL.

Note: You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

GET example

To retrieve the details of a code list based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
sap_country HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the details of a code list:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 1284

{
  "id": "e0f3e8e73466a98e71b1731b",
  "internalId": "sap_country",
  "rdsDetails": {
    "id": "Unit Term",
    "internalId": "unit_term",
    "alias": "units"
```

```
},
"name": "Units",
"description": "Code list for units",
"version": "2.0",
"application": "UN recommendation 20",
"hierarchical": false,
"levels": 1,
"domain": "International standards",
"confidentiality": "private",
"priority": "Prio1",
"status": "Draft",
"effectiveDate": "2017-04-01",
"approvedOn": "2017-03-01",
"displayColumns": [
  "Name",
  "Code"
],
"codeValueFields": [
  {
    "name": "Name",
    "origin": "TERM",
    "datatype": "String",
    "mandatory": false,
    "labels": [
      {
        "language": "en",
        "value": "Name"
      }
    ]
  },
  {
    "name": "Code",
    "origin": "TERM",
    "datatype": "String",
    "mandatory": true,
    "labels": [
      {
        "language": "en",
        "value": "Code"
      }
    ]
  },
  {
    "name": "Description",
    "origin": "TERM",
    "datatype": "String",
    "mandatory": false,
    "labels": [
      {
        "language": "en",
        "value": "Description"
      }
    ]
  },
  {
    "name": "refField2",
    "origin": "CODELIST",
    "datatype": "Reference",
    "mandatory": true,
    "relatedRdsDetails": {
      "id": "907cea2cca037950cd1ad67a",
      "internalId": "relSetInternalId2",
      "alias": "relSetAliasId2"
    },
    "relatedListDetails": {
      "id": "5449217342ac9d487f389b47",
      "internalId": "relListInternalId2",
      "alias": "relListAliasId2"
    },
    "displayColumns": [
      "column5"
```

```
      ],
      "labels": [
        {
          "language": "en",
          "value": "refField2"
        }
      ]
    },
    {
      "name": "achronym",
      "origin": "CODELIST",
      "datatype": "String",
      "mandatory": false,
      "labels": [
        {
          "language": "en",
          "value": "achronym"
        }
      ]
    }
  ],
  "dependencyDef": {
    "rdsDetails": {
      "id": "UnitSystemTerm",
      "internalId": "unitsystemterm",
      "alias": "unit_system_term"
    },
    "codeListDetails": {
      "id": "UnitSystems",
      "internalId": "unitsystems",
      "alias": "unit_systems"
    },
    "displayColumns": [
      "Name"
    ]
  },
  "externalContacts": [
    {
      "contactName": "Test Name",
      "contactType": "SMS",
      "contactDetail": "+49 - 7867564543"
    },
    {
      "contactName": "Test Name1",
      "contactType": "EMAIL",
      "contactDetail": "test@info.com"
    }
  ],
  "externalUrls": [
    {
      "urlName": "TestUrlName",
      "urlLink": "https://abc.com"
    },
    {
      "urlName": "TestUrlName1",
      "urlLink": "https://abc1.com"
    }
  ]
}
```

Get history of a code list by time range (v2)

Retrieves all the change events of a code list for a specific time range based on the internal ID or alias.

GET request

To retrieve all the change events of a code list for a specific time range based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/codelists/{assetKey}/summary/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the request query parameters:

Parameter	Description
assetKey	Internal ID or alias of a code list. **Note:** When you provide an invalid assetKey in the request URL, the codelist version 2 REST API returns an error if the request contains invalid internal IDs or aliases.
from	Start date and time of the time range, which is inclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-12T14:04:04Z`.
to	End date and time of the time range, which is exclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-15T14:04:04Z`.
page	Optional. Page number to display. Default is `0`.
pageSize	Optional. Number of records to display per page. Default value is `100`. Maximum value is `10000`.
modificationType	Optional. Type of modification. Value can be CREATE, UPDATE, or DELETE.

GET response

The response contains the change events of a code list for a specific time range.

The following table describes the attributes in the response body:

Attribute	Type	Description
pageSize	Number	Number of records displayed per page.
page	Number	Page from which the response was retrieved.
numberOfElements	Number	Number of records returned in the current page.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of modification events.

Attribute	Type	Description
asset	Object	Includes details about the code list.
id	String	ID of the code list.
name	String	Name of the code list.
internalId	String	Unique identifier of the code list.
alias	String	Alternative identifier of the code list.
assetType	String	Type of the code list.
modificationType	String	The type of change made to the code list.
fieldChanges	Object	Includes details about the previous and new values of the fields with changes.
attributeChanges	Object	Includes details about the previous and new values of the attributes with changes.
eventTime	String	The date and time when the code list was last updated.
userName	String	The user name of the user who initiated the modification.
application	String	Optional. Application that uses the code list.
approvedById	String	Optional. ID of the approver of the asset.
approvedByName	String	Optional. User name of the user who approved the asset.
approvedOn	String	Optional. Date the asset was approved.
description	String	Optional. Description of asset.
effectiveDate	String	Optional. Date the asset became effective.
status	String	Optional. Status of the asset.
version	String	Optional. Version of the code list.

GET example

To retrieve all the change events of a code list for a specific time range based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
ent_numeric/summary/audit?from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of the code list from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 4991

{
  "pageSize": 100,
```

```
"page": 0,
"numberOfElements": 8,
"lastPage": true,
"firstPage": true,
"content": [
  {
    "asset": {
      "id": "6407014f5b243e32c4b02ce3",
      "name": "ISO 3166-1 numeric",
      "internalId": "ent_numeric",
      "alias": "als_ent_numeric",
      "assetType": "CODELIST"
    },
    "modificationType": "UPDATE",
    "eventTime": "2023-03-07T09:23:35Z",
    "userName": "integration-test-admin"
  },
  {
    "asset": {
      "id": "6407014f5b243e32c4b02ce3",
      "name": "ISO 3166-1 numeric",
      "internalId": "ent_numeric",
      "alias": "als_ent_numeric",
      "assetType": "CODELIST"
    },
    "modificationType": "UPDATE",
    "fieldChanges": {
      "additionalDescription": {
        "newValue": "(ISO 3166-1)"
      },
      "application": {
        "newValue": {
          "key": "EnterpriseKey",
          "label": "Enterprise"
        }
      },
      "approvedById": {
        "newValue": "2L0DXgAOiCObZDT7Yql02R"
      },
      "approvedByName": {
        "newValue": "integration-test-planner"
      },
      "approvedOn": {
        "newValue": "2023-03-08"
      },
      "description": {
        "newValue": "Numeric Codes for Countries"
      },
      "effectiveDate": {
        "newValue": "2023-03-06"
      },
      "status": {
        "newValue": {
          "key": "ActiveKey",
          "label": "Active"
        }
      },
      "version": {
        "newValue": "1"
      }
    },
    "eventTime": "2023-03-07T09:23:34Z",
    "userName": "integration-test-admin"
  },
  {
    "asset": {
      "id": "6407014f5b243e32c4b02ce3",
      "name": "ISO 3166-1 numeric",
      "internalId": "ent_numeric",
      "alias": "als_ent_numeric",
      "assetType": "CODELIST"
```

```
  },
  "modificationType": "UPDATE",
  "attributeChanges": {
    "NumericCode": {
      "name": {
        "newValue": "NumericCode"
      },
      "type": {
        "newValue": "String"
      },
      "required": {
        "newValue": "false"
      }
    }
  },
  "eventTime": "2023-03-07T09:19:29Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "6407014f5b243e32c4b02ce3",
    "name": "ISO 3166-1 numeric",
    "internalId": "ent_numeric",
    "alias": "als_ent_numeric",
    "assetType": "CODELIST"
  },
  "modificationType": "UPDATE",
  "attributeChanges": {
    "NumericCode": {
      "name": {
        "newValue": "NumericCode"
      },
      "type": {
        "newValue": "String"
      },
      "required": {
        "newValue": "false"
      }
    }
  },
  "eventTime": "2023-03-07T09:19:27Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "6407014f5b243e32c4b02ce3",
    "name": "ISO 3166-1 numeric",
    "internalId": "ent_numeric",
    "alias": "als_ent_numeric",
    "assetType": "CODELIST"
  },
  "modificationType": "UPDATE",
  "eventTime": "2023-03-07T09:18:19Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "6407014f5b243e32c4b02ce3",
    "name": "ISO 3166-1 numeric",
    "internalId": "ent_numeric",
    "alias": "als_ent_numeric",
    "assetType": "CODELIST"
  },
  "modificationType": "UPDATE",
  "eventTime": "2023-03-07T09:18:17Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "6407014f5b243e32c4b02ce3",
    "name": "ISO 3166-1 numeric",
```

```
    "internalId": "ent_numeric",
    "alias": "als_ent_numeric",
    "assetType": "CODELIST"
  },
  "modificationType": "UPDATE",
  "eventTime": "2023-03-07T09:18:09Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "6407014f5b243e32c4b02ce3",
    "name": "ISO 3166-1 numeric",
    "internalId": "ent_numeric",
    "alias": "als_ent_numeric",
    "assetType": "CODELIST"
  },
  "modificationType": "CREATE",
  "fieldChanges": {
    "displayColumns": {
      "newValue": [
        "Name"
      ]
    },
    "hierarchical": {
      "newValue": "false"
    },
    "levels": {
      "newValue": "1"
    },
    "name": {
      "newValue": "ISO 3166-1 numeric"
    },
    "termId": {
      "newValue": "6407004f6b19964cc917c7bf"
    }
  },
  "attributeChanges": {
    "Alpha2Code": {
      "name": {
        "newValue": "Alpha2Code"
      },
      "type": {
        "newValue": "String"
      },
      "required": {
        "newValue": "false"
      }
    },
    "Alpha3Code": {
      "name": {
        "newValue": "Alpha3Code"
      },
      "type": {
        "newValue": "String"
      },
      "required": {
        "newValue": "false"
      }
    },
    "Code": {
      "name": {
        "newValue": "Code"
      },
      "type": {
        "newValue": "String"
      },
      "required": {
        "newValue": "true"
      }
    },
    "Description": {
```

```
            "name": {
              "newValue": "Description"
            },
            "type": {
              "newValue": "String"
            },
            "required": {
              "newValue": "false"
            }
          },
          "Name": {
            "name": {
              "newValue": "Name"
            },
            "type": {
              "newValue": "String"
            },
            "required": {
              "newValue": "true"
            }
          }
        },
        "eventTime": "2023-03-07T09:18:07Z",
        "userName": "integration-test-admin"
      }
    ]
}
```


Get crosswalks for a code list (v2)

Retrieves the crosswalks associated to a code list and the summary information for each crosswalk based on the internal ID or alias.

GET request

To retrieve all crosswalks associated to a code list based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/codelists/{codeListKey}/crosswalks
```

GET request query parameters

The following table describes the parameter in the request:

Parameter	Type	Description
codeListKey	String	Internal ID or alias of a code list of the crosswalk.

GET response

The response contains the crosswalks associated to the code list.

The following table describes the attributes in the response body:

Field	Type	Description
id	String	ID of the code list. For more information, see "Asset IDs" on page 199.
internalId	String	Unique identifier of the code list.
description	String	Optional. Description of the code list.

Field	Type	Description
status	String	Optional. Status of the code list.
confidentiality	String	Optional. Confidentiality of the code list.
effectiveDate	String	Optional. Date when the code list became effective.
approvedOn	String	Optional. Date when the code list was approved.
sourceCodeListDetails	Object	Includes details of the source code list to which the crosswalk is associated.
id	String	ID of the source code list to which the crosswalk is associated.
targetCodeListDetails	Object	Includes details of the target code list to which the crosswalk is associated.
id	String	ID of the target code list to which the crosswalk is associated.

Note: You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

GET example

To retrieve the crosswalks associated to a code list based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
internalId/crosswalks HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the crosswalks associated to a code list:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 514
[
  {
    "id": "af5dc4d0d7ada6609985ab33",
    "internalId": "internalId",
    "description": "Sample Crosswalk 1",
    "status": "active",
    "sourceCodeListDetails": {
      "id": "85812f48d51aac9513b7df4e"
    },
    "targetCodeListDetails": {
      "id": "3dd16657bab79b6d60eab2ff"
    }
  },
  {
    "id": "3536aeb4364a4afabca8eadd",
    "internalId": "internalId2",
    "description": "Sample Crosswalk 2",
    "status": "inactive",
    "confidentiality": "private",
    "effectiveDate": "2007-04-01",
    "approvedOn": "2017-03-01",
    "sourceCodeListDetails": {
      "id": "85812f48d51aac9513b7df4e"
    },
    "targetCodeListDetails": {
      "id": "c4a1387ef17d161d3a3c2575"
    }
```

```
    }
]
```

Get history of code value hierarchies by time range (v2)

Retrieves all the change events of code value hierarchies for a specific time range based on the internal ID or alias.

GET request

To retrieve all the change events of code value hierarchies for a specific time range based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/codelists/{codeListKey}/codevalues/hierarchy/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the query parameters:

Parameter	Description
codeListKey	Internal ID or alias of the code list from which you are retrieving the code values.
from	Start date and time of the time range, which is inclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-12T14:04:04Z`.
to	End date and time of the time range, which is exclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-15T14:04:04Z`.
page	Optional. Page number to display. Default is `0`.
pageSize	Optional. Number of records to display per page. Default value is `100`. Maximum value is `10000`.
modificationType	Optional. Type of modification. Value can be CREATE or DELETE.

GET response

The response contains the change events of code value hierarchies for a specific time range.

The following table describes the attributes in the response body:

Attribute	Type	Description
page	Number	Page from which the response was retrieved.
pageSize	Number	Number of records displayed per page.
numberOfElements	Number	Number of records returned in the current page.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.

Attribute	Type	Description
content	Array	Includes the list of modification events.
asset	Object	Includes details about the code values of the code list.
id	String	ID of the code list. For more information, see "Asset IDs" on page 199.
internalId	String	Unique identifier of the code list.
alias	String	Alternative identifier of the code list.
assetType	String	Type of the asset. The value is `CODELIST`.
modificationType	String	The type of change made to the code values of the code list.
eventTime	String	The date and time the code values were last updated.
userName	String	The user name of the user who initiated the modification.

GET example

To retrieve all the change events of code value hierarchies for a specific time range based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
cl_country/codevalues/hierarchy/audit?from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z
HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of the code value hierarchies from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 3441

{
  "pageSize": 100,
  "page": 0,
  "numberOfElements": 12,
  "lastPage": true,
  "firstPage": true,
  "content": [
    {
      "asset": {
        "id": "64071b105b243e32c4b04caa",
        "internalId": "cl_country",
        "alias": "als_cl_country",
        "assetType": "CODELIST"
      },
      "modificationType": "DELETE",
      "from": {
        "previousValue": "France"
      },
      "to": {
        "previousValue": "Europe"
      },
      "eventTime": "2023-03-07T11:11:06Z",
      "userName": "integration-test-admin"
    },
    {
      "asset": {
```

```
    "id": "64071b105b243e32c4b04caa",
    "internalId": "cl_country",
    "alias": "als_cl_country",
    "assetType": "CODELIST"
  },
  "modificationType": "DELETE",
  "from": {
    "previousValue": "Paris"
  },
  "to": {
    "previousValue": "France"
  },
  "eventTime": "2023-03-07T11:11:06Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "64071b105b243e32c4b04caa",
    "internalId": "cl_country",
    "alias": "als_cl_country",
    "assetType": "CODELIST"
  },
  "modificationType": "CREATE",
  "from": {
    "newValue": "America"
  },
  "eventTime": "2023-03-07T11:10:14Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "64071b105b243e32c4b04caa",
    "internalId": "cl_country",
    "alias": "als_cl_country",
    "assetType": "CODELIST"
  },
  "modificationType": "CREATE",
  "from": {
    "newValue": "Europe"
  },
  "eventTime": "2023-03-07T11:10:14Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "64071b105b243e32c4b04caa",
    "internalId": "cl_country",
    "alias": "als_cl_country",
    "assetType": "CODELIST"
  },
  "modificationType": "CREATE",
  "from": {
    "newValue": "USA"
  },
  "to": {
    "newValue": "America"
  },
  "eventTime": "2023-03-07T11:10:14Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "64071b105b243e32c4b04caa",
    "internalId": "cl_country",
    "alias": "als_cl_country",
    "assetType": "CODELIST"
  },
  "modificationType": "CREATE",
  "from": {
    "newValue": "New York"
  },
```

```
  "to": {
    "newValue": "USA"
  },
  "eventTime": "2023-03-07T11:10:14Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "64071b105b243e32c4b04caa",
    "internalId": "cl_country",
    "alias": "als_cl_country",
    "assetType": "CODELIST"
  },
  "modificationType": "CREATE",
  "from": {
    "newValue": "Canada"
  },
  "to": {
    "newValue": "America"
  },
  "eventTime": "2023-03-07T11:10:14Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "64071b105b243e32c4b04caa",
    "internalId": "cl_country",
    "alias": "als_cl_country",
    "assetType": "CODELIST"
  },
  "modificationType": "CREATE",
  "from": {
    "newValue": "Toronto"
  },
  "to": {
    "newValue": "Canada"
  },
  "eventTime": "2023-03-07T11:10:14Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "64071b105b243e32c4b04caa",
    "internalId": "cl_country",
    "alias": "als_cl_country",
    "assetType": "CODELIST"
  },
  "modificationType": "CREATE",
  "from": {
    "newValue": "Germany"
  },
  "to": {
    "newValue": "Europe"
  },
  "eventTime": "2023-03-07T11:10:14Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "64071b105b243e32c4b04caa",
    "internalId": "cl_country",
    "alias": "als_cl_country",
    "assetType": "CODELIST"
  },
  "modificationType": "CREATE",
  "from": {
    "newValue": "Berlin"
  },
  "to": {
    "newValue": "Germany"
  },
```

```
      "eventTime": "2023-03-07T11:10:14Z",
      "userName": "integration-test-admin"
    },
    {
      "asset": {
        "id": "64071b105b243e32c4b04caa",
        "internalId": "cl_country",
        "alias": "als_cl_country",
        "assetType": "CODELIST"
      },
      "modificationType": "CREATE",
      "from": {
        "newValue": "France"
      },
      "to": {
        "newValue": "Europe"
      },
      "eventTime": "2023-03-07T11:10:14Z",
      "userName": "integration-test-admin"
    },
    {
      "asset": {
        "id": "64071b105b243e32c4b04caa",
        "internalId": "cl_country",
        "alias": "als_cl_country",
        "assetType": "CODELIST"
      },
      "modificationType": "CREATE",
      "from": {
        "newValue": "Paris"
      },
      "to": {
        "newValue": "France"
      },
      "eventTime": "2023-03-07T11:10:14Z",
      "userName": "integration-test-admin"
    }
  ]
}
```


Get history of code values of a code list (v2)

Retrieves all the change events of code values of a code list for a specific time range based on the internal ID or alias.

GET request

To retrieve all the change events of code values of a code list for a specific time range based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/codelists/{codeListKey}/codevalues/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the parameters in the request:

Parameter	Description
codeListKey	Internal ID or alias of the code list from which you are retrieving the code values.
from	Start date and time of the time range, which is inclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-12T14:04:04Z`.
to	End date and time of the time range, which is exclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-15T14:04:04Z`.
page	Optional. Page number to display. Default is `0`.
pageSize	Optional. Number of records to display per page. Default value is `100`. Maximum value is `10000`.
modificationType	Optional. Type of modification. Value can be CREATE or UPDATE_AND_DELETE.

GET response

The response contains the change event of code values of a code list for a specific time range.

The following table describes the attributes in the response body:

Attribute	Type	Description
pageSize	Number	Number of records displayed per page.
page	Number	Page from which the response was retrieved.
numberOfElements	Number	Number of records returned in the current page.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
content	Array	Includes the list of modification events.
asset	Object	Includes details about the code values of the code list.
id	String	ID of the code list. For more information, see "Asset IDs" on page 199.
internalId	String	Unique identifier of the code list.
alias	String	Alternative identifier of the code list.
assetType	String	Type of the asset. The value is `CODELIST`.
modificationType	String	Type of change made to the code values of the code list.
changedAttributes	Object	Includes details about the previous and new values of the attributes with changes.
code	String	Code field of the code values.

Attribute	Type	Description
codeValueId	String	Internal ID of the code values.
_localizedContent[<language code>].<attribute name>	Object	The previous and current values of an attribute in the specified language. The parameter combines the BCP 47 language code and the attribute name. For example, _localizedContent[en-us].Description includes the change to the Description attribute in English (United States). The object contains the previous and new values of the attribute in that language.
previousValue	String	Previous value of the attribute in the specified language. If the value is newly created, the response doesn't include this parameter.
newValue	String	Current value of the attribute in the specified language.
eventTime	String	Date and time when the code values were last updated.
userName	String	User name of the user who initiated the modification.

GET example

To retrieve all the change events of code values of a code list for a specific time range based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
cl_country/codevalues/audit?from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of a code list from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 3441

{
  "pageSize": 100,
  "page": 0,
  "numberOfElements": 6,
  "lastPage": true,
  "firstPage": true,
  "content": [
    {
      "asset": {
        "id": "64071b105b243e32c4b04caa",
        "internalId": "cl_country",
        "alias": "als_cl_country",
        "assetType": "CODELIST"
      },
      "codeValueId": "640701e45b243e32c4b02d40",
      "code": "Antarctica",
      "modificationType": "UPDATE",
      "changedAttributes": {
        "_localizedContent[en-us].Description": {
          "previousValue": "Numeric Code of Antarctica",
          "newValue": "Numeric Code of Antarctica (ISO 3166-1)"
        },
        "_localizedContent[de].Description": {
          "newValue": "Numerischer Code der Antarktis (ISO 3166-1)"
```

```
    }
  },
  "eventTime": "2023-03-07T09:21:56Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "64071b105b243e32c4b04caa",
    "internalId": "cl_country",
    "alias": "als_cl_country",
    "assetType": "CODELIST"
  },
  "codeValueId": "640701d25b243e32c4b02d35",
  "code": "Albania",
  "modificationType": "UPDATE",
  "changedAttributes": {
    "_localizedContent[en-us].Description": {
      "previousValue": "Numeric Code of Albania",
      "newValue": "Numeric Code of Albania (ISO 3166-1)"
    },
    "_localizedContent[de].Description": {
      "newValue": "Numerischer Code von Albanien (ISO 3166-1)"
    }
  },
  "eventTime": "2023-03-07T09:21:39Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "64071b105b243e32c4b04caa",
    "internalId": "cl_country",
    "alias": "als_cl_country",
    "assetType": "CODELIST"
  },
  "codeValueId": "640701c35b243e32c4b02d21",
  "code": "Afghanistan",
  "modificationType": "UPDATE",
  "changedAttributes": {
    "_localizedContent[en-us].Description": {
      "previousValue": "Numeric Code for Afghanistan",
      "newValue": "Numeric Code for Afghanistan (ISO 3166-1)"
    },
    "_localizedContent[de].Description": {
      "newValue": "Numerischer Code für Afghanistan (ISO 3166-1)"
    }
  },
  "eventTime": "2023-03-07T09:21:19Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "64071b105b243e32c4b04caa",
    "internalId": "cl_country",
    "alias": "als_cl_country",
    "assetType": "CODELIST"
  },
  "codeValueId": "640701c35b243e32c4b02d21",
  "code": "Afghanistan",
  "modificationType": "CREATE",
  "changedAttributes": {
    "Code": {
      "newValue": "Afghanistan"
    },
    "_localizedContent[en-us].Description": {
      "newValue": "Numeric Code for Afghanistan"
    },
    "_localizedContent[en-us].Name": {
      "newValue": "Afghanistan"
    },
    "_localizedContent[de].Name": {
      "newValue": "Afghanistan"
```

```
        },
        "NumericCode": {
          "newValue": "004"
        }
      },
      "eventTime": "2023-03-07T09:20:43Z",
      "userName": "integration-test-admin"
    },
    {
      "asset": {
        "id": "64071b105b243e32c4b04caa",
        "internalId": "cl_country",
        "alias": "als_cl_country",
        "assetType": "CODELIST"
      },
      "codeValueId": "640701d25b243e32c4b02d35",
      "code": "Albania",
      "modificationType": "CREATE",
      "changedAttributes": {
        "Code": {
          "newValue": "Albania"
        },
        "_localizedContent[en-us].Description": {
          "newValue": "Numeric Code of Albania"
        },
        "_localizedContent[en-us].Name": {
          "newValue": "Albania"
        },
        "_localizedContent[de].Name": {
          "newValue": "Albanien"
        },
        "NumericCode": {
          "newValue": "008"
        }
      },
      "eventTime": "2023-03-07T09:20:43Z",
      "userName": "integration-test-admin"
    },
    {
      "asset": {
        "id": "64071b105b243e32c4b04caa",
        "internalId": "cl_country",
        "alias": "als_cl_country",
        "assetType": "CODELIST"
      },
      "codeValueId": "640701e45b243e32c4b02d40",
      "code": "Antarctica",
      "modificationType": "CREATE",
      "changedAttributes": {
        "Code": {
          "newValue": "Antarctica"
        },
        "_localizedContent[en-us].Description": {
          "newValue": "Numeric Code of Antarctica"
        },
        "_localizedContent[en-us].Name": {
          "newValue": "Antarctica"
        },
        "_localizedContent[de].Name": {
          "newValue": "Antarktis"
        },
        "NumericCode": {
          "newValue": "010"
        }
      },
      "eventTime": "2023-03-07T09:20:43Z",
      "userName": "integration-test-admin"
    }
  ]
}
```

Search code values (v2)

Use the search API to search for code values in all the code lists based on the internal ID or alias.

Reference 360 search results display the multilingual attributes of an asset in the language determined by your browser locale. If a value isn't available in that language, Reference 360 displays the value in the default language of the organization.

POST request

To search for code values in all the code lists based on an internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v2/search
```

The following table describes the parameters in the search request:

Parameter	Type	Description
showAssetDetails	Boolean	Indicates whether the search result displays the details of assets. Value can be `true` or `false`. Default is `false`.
page	Integer	Optional. Page number to display. Default is 0.
pageSize	Integer	Optional. Number of code values that you want to display on each page. Default is 100. You can specify up to 10000.
search	String	Search string that must contain a minimum of two characters. For example, you can enter the search string as `"IND."` You can use only the values of string data type attributes, such as name, code, description, and other custom string attributes. **Note:** You can't use the values of integer, date, boolean, and decimal data type attributes in the search string. To retrieve all the code values, use an asterisk (*). To retrieve the exact code value that you search for, enter the search string in the following format: `"\"<search_string>\""` For example, to search for the exact code value `AUSTRIA`, specify `"\"AUSTRIA\""` as search string.

POST response

The response returns the search results.

The following table describes the attributes in the response body:

Field	Type	Description
pageSize	Integer	Number of code values displayed on each page.
page	Integer	Number of the page that you retrieved.
totalNumberOfElements	Integer	Total number of code values found.
numberOfElements	Integer	Total number of code values on the current page.

Field	Type	Description
lastPage	Boolean	Indicates whether the current page is the last page of the search results.
firstPage	Boolean	Indicates whether the current page is the first page of the search results.
content	Object	Includes the list of code values.
<entity prefix>._localizedContent	Array	Lists the multilingual attribute values of a code value. For example, rdmvaluebeent_vehiclebrand._localizedContent contains the Name and Description values of the AUDI code value in English (United States), French (France), Urdu, and Japanese (Japan).
_id	String	BCP 47 language code of the values in the object, such as en-US, fr-FR, or ja-JP. The language code matches the code value in the Localization Languages code list.
Name	String	Display name of the attribute in the language.
Description	String	Description of the attribute in the language.
<entity prefix>.Name	String	Display name of the attribute in the display language. For example, rdmvaluebeent_vehiclebrand.Name returns アウディ because the display language is Japanese (Japan). If the attribute has no value in the display language, the response returns the value in the default language.
<entity prefix>.Description	String	Description of the attribute in the display language. If the attribute has no value in the display language, the response returns the value in the default language.
<entity prefix>.Code	String	Code of the code value.
appType	String	Application that the record belongs to.
appEntityType	String	Type of the record. The value CODEVALUE indicates that the record is a code value.
codeListId	String	ID of the code list that contains the code value. For more information, see "Asset IDs" on page 199.
codelistName	String	Name of the code list that contains the code value.
referenceDataSetName	String	Name of the reference data set that the code list belongs to.

POST example

To search for code values in all the code lists based on the internal ID or alias, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/search?
showAssetDetails=true' HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
{
   "page":4,
   "pageSize":4,
```

```
   "search":"*"
}
```

The following sample response displays the code values that contain Audi in all the code lists:

```
{
  "pageSize": 4,
  "page": 4,
  "totalNumberOfElements": 20,
  "numberOfElements": 1,
  "lastPage": false,
  "firstPage": true,
  "content": [
    {
  "rdmvaluebeent_vehiclebrand._localizedContent": [
    {
      "_id": "en-US",
      "Name": "AUDI",
      "Description": "Automotive brand"
    },
    {
      "_id": "fr-FR",
      "Name": "AUDI",
      "Description": "une marque automobile"
    },
    {
      "_id": "ur",
      "Name": "آڈی",
      "Description": "آڈی کی تفصیل"
    },
    {
      "_id": "ja-JP",
      "Name": "アウディ",
      "Description": "アウディの説明"
    }
  ],
  "rdmvaluebeent_vehiclebrand.Name": "アウディ",
  "rdmvaluebeent_vehiclebrand.Description": "アウディの説明",
  "rdmvaluebeent_vehiclebrand.Code": "AUDI",
  "appType": "rdm",
  "appEntityType": "CODEVALUE",
  "codeListId": "68a58ccd3ed14f1174b67c33",
  "codelistName": "Enterprise VehicleBrand",
  "referenceDataSetName": "VehicleBrand"
   }
  ]
}
```

Export code value hierarchy to JSON format (v2)

Exports the subset of code values in a hierarchical code list to a JSON file based on the internal ID or alias.

GET request

To export the subset of code values in a hierarchical code list based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/codelists/{codeListKey}/codevalues/subset/{subsetId}/cursor?
page={page}
```

Note: This API supports only incremental pagination starting from page `0`. In the request header, you must specify the `Accept` attribute to `application/json`.

Use the following parameters in the request query to specify the subset of code values to export:

Field	Type	Description	
codeListKey		String	Internal ID or alias of the hierarchical code list.
subSetIdentifier		String	ID of the subset.
page	Number	Page number to display. Default value is `0`.	
limit	Number	Optional. Number of records to display on each page. Default value is `10000`.	

GET response

The response is in JSON file.

GET example

To export the subset of code values in a hierarchical code list based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
ent_country/codevalues/subset/71253be5deb0e7d4974935e0/cursor?page=0 HTTP/1.1
Content-Type: application/json
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the exported subset of code values in the JSON file:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 352

{
   "pageSize":2,
   "page":0,
   "numberOfElements":2,
   "content":[
      {
         "Code":"STUDENT-1001",
         "fields":{
            "Code":"STUDENT-1001",
            "Name":"ppniK"
         },
         "parentId":"UNIVERSITY-001"
      },
      {
         "Code":"UNIVERSITY-001",
         "fields":{
            "Code":"UNIVERSITY-001",
            "Name":"ABC University"
         }
      }
   ]
}
```

Update cursor settings of the subset of code values (v2)

Updates the cursor settings of the subset of code values based on the internal ID or alias.

PUT request

To update the cursor settings of the subset of code values based on an internal ID or alias, submit a PUT request with the following URI:

```
/rdm-service/external/v2/codelists/{codeListKey}/codevalues/subset/{subSetIdentifier}/cursor
```

Note: When you update the cursor setting, the page is reset to zero. The export API exports the subset of code values from the starting page.

The following table describes the query parameters:

Field	Type	Description	
codeListKey		String	Internal ID or alias of the code list.
subSetIdentifier		String	ID of the subset.
filename	String	File name for the exported file. Value must end with the `.csv` file extension.	
repeatHeaders	Boolean	Optional. Indicates whether to populate headers in the CSV response. Value is `true` or `false`. Default is `false`.	
delimiter	String	Optional. Delimiter used to separate values. Value must be `ASTERISK`, `CIRCUMFLEX`, `COLON`, `COMMA`, `PIPE`, `SECTION`, `SEMICOLON`, `SPACE` or `TAB`. Default is `COMMA`.	
decimalSeparator	String	Optional. Decimal separator used for numbers. Value must be `COMMA` or `DOT`. Default is `DOT`.	
thousandSeparator	String	Optional. Grouping separator used for numbers. Value must be `COMMA`, `DOT`, `SPACE`, `SINGLEQUOTE` or `NONE`. Default is `NONE`.	
dateFormat	String	Format used for dates. Use one of the following formats: - `DE`. For dd.mm.yyyy format. - `ISO`. For yyyy-mm-dd format. - `US`. For mm/dd/yyyy format. Default is `ISO`.	
addLabelsForReferenceAttribute	Boolean	Optional. Displays values for reference and dependent attributes based on the display attributes of reference and dependent code lists, respectively.	

Field	Type	Description
codepage	String	Optional. Code page used for the export file. Value must be `UTF8` or `MS_WINDOWS`. Default is `UTF8`.
columns	Array	Optional. Attribute columns you want to export. If you don't specify attribute columns, the export includes all the attribute columns.
fieldName	String	Optional. Name of attribute column to include in the exported file.
codeListId	String	ID of the code list.
noOfRecordsPerPage	Number	Optional. Number of records to display on each page. The maximum value is `10000`. Default is `10000`.

PUT response

A 204 no content response is returned.

PUT example

To update the cursor settings of the subset of code values based on the internal ID or alias, you might use the following request:

```
PUT https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/codelists/
country/codevalues/subset/71253be5deb0e7d4974935e0/cursor HTTP/1.1
Content-Type: application/json
Accept: application/json
Content-Length: 391

{
   "filename":"codeValueHierarchyExportFile.csv",
   "repeatHeaders":true,
   "delimiter":"COMMA",
   "decimalSeparator":"COMMA",
   "thousandSeparator":"DOT",
   "dateFormat":"DE",
   "addLabelsForReferenceAttribute":true,
   "codepage":"UTF8",
   "columns":[
      {
         "fieldName":"columnName",
         "codeListId":"5d352cbb8de0392f8c0710d3"
      }
   ],
   "noOfRecordsPerPage":10000
}
```

The following sample response shows the no content response:

```
HTTP/1.1 204 No Content
```

codelists version 3

Use this resource to unlock code lists locked by other users based on its internal ID or alias.

Note: To identify a code list based on its name, use the codelists version 1 REST APIs.

Unlock a code list (v3)

Unlocks a code list that's locked by another user based on the internal ID or alias.

Note: To use the API, you require the IDMC Reference 360 Administrator role.

PUT request

To unlock a code list based on an internal ID or alias, submit a PUT request with the following URI:

```
/rdm-service/external/v3/codelists/{codeListKey}/unlock
```

PUT request query parameters

The following table describes the parameter in the request:

Parameter	Type	Description
codeListKey	String	Internal ID or alias of a code list that you want to unlock.

PUT response

A 202 accepted response is returned.

PUT example

To unlock a code list based on the internal ID or alias, you might use the following request:

```
PUT https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/codelists/
sap_country/unlock HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the accepted response:

```
HTTP/1.1 202 Accepted
```

crosswalks version 1

Use this resource to retrieve the details of a crosswalk, such as the properties, status, source code list, and target code list. You can also retrieve the value mappings for a code value.

Unlock a crosswalk (v1)

Unlocks a crosswalk that's locked by another user.

Note: To use the API, you require the IDMC Reference 360 Administrator role.

PUT request

To unlock a crosswalk, submit a PUT request with the following URI:

```
/rdm-service/external/v1/crosswalks/<crosswalk ID>/unlock
```

Note: You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

PUT response

A 202 accepted response is returned.

PUT example

To unlock a crosswalk, you might use the following request:

```
PUT https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/crosswalks/
653a21df0649f17aba8134d5/unlock HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the accepted response:

```
HTTP/1.1 202 Accepted
```

Get crosswalk (v1)

Retrieves the details of a crosswalk.

GET request

To retrieve the details of a crosswalk, submit a GET request with the following URI:

```
/rdm-service/external/v1/crosswalks/<crosswalk ID>
```

Note: You can find the ID of assets in Reference 360 or use REST APIs to retrieve the IDs. For more information, see "Asset IDs" on page 199.

GET response

The response contains the details of a crosswalk.

The following table describes the attributes in the response:

Field	Type	Description
id	String	ID of the asset. Assets include reference data sets, code lists, crosswalks, and hierarchies. For more information, see "Asset IDs" on page 199.
sourceCodelistId	String	ID of the source code list to which the crosswalk is associated.
targetCodelistId	String	ID of the target code list.
description	String	Optional. Description of asset.
status	String	Optional. Status of the asset.
confidentiality	String	Optional. Confidentiality of the asset.
sourceApplication	String	Optional. Application of the source code list.

Field	Type	Description
targetApplication	String	Optional. Application of the target code list.
effectiveDate	String	Optional. Date the asset became effective.
approvedById	String	Optional. ID of the approver of the asset.
approvedByName	String	Optional. User name of the user who approved the asset.
approvedOn	String	Optional. Date the asset was approved.

GET example

To retrieve the details of a crosswalk, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/crosswalks/
20c85dde693051cf8037f1eb HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the details of a crosswalk:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 300

{
  "id" : "20c85dde693051cf8037f1eb",
  "description" : "Sample Crosswalk",
  "status" : "active",
  "sourceCodelistId" : "f6821570be0fd451934dff86",
  "targetCodelistId" : "628234c5ba9d033c33ff0284",
  "confidentiality" : "private",
  "effectiveDate" : "2007-04-01",
  "approvedOn" : "2017-03-01"
}
```

Get history of a crosswalk by time range (v1)

Retrieves all the change events of a crosswalk for a specific time range.

GET request

To retrieve all the change events of a crosswalk for a specific time range, submit a GET request with the following URI:

```
/rdm-service/external/v2/crosswalks/<crosswalkIdentifier>/summary/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the query parameters:

Parameter	Description
crosswalkIdentifier	ID of the crosswalk. For more information, see "Asset IDs" on page 199. **Note:** When you provide an invalid asset identifier in the request URL, the crosswalk REST API returns an empty content without an error.
from	Start date and time of a time range. The start date and time is inclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2023-01-01T00:00:00Z`.
to	End date and time of a time range. The end date and time is exclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2024-01-01T00:00:00Z`.
page	Optional. Page number to display. Default is 0.
pageSize	Optional. The number of records to display on each page. Default is 100. Maximum is 10000.
modificationType	Optional. Type of modification. Value can be CREATE, UPDATE, or DELETE.

GET response

The response contains the change events of a crosswalk for a specific time range.

The following table describes the attributes in the response:

Attributes	Type	Description
pageSize	Number	The number of records displayed on each page.
page	Number	Page number displayed.
numberOfElements	Number	The number of events.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of modification events.
asset	Object	Includes details about the crosswalk.
modificationType	String	The type of change made to the crosswalk.
fieldChanges	Object	Displays the previous and new values of the fields with changes.
attributeChanges	Object	Displays the previous and new values of the attributes with changes.
eventTime	String	The date and time the crosswalk was last modified.
userName	String	The user name of the user who initiated the modification.

GET example

To retrieve all the change events of a crosswalk for a specific time range, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/crosswalks/
cf4f97026e010a95e19c9e79/summary/audit?from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z
HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of the crosswalk from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 1247

{
   "pageSize":100,
   "page":0,
   "numberOfElements":3,
   "lastPage":true,
   "firstPage":true,
   "content":[
      {
         "asset":{
            "id":"640707a46b19964cc917d0c8",
            "name":"Enterprise Country to ISO 3166-1 numeric",
            "assetType":"CROSSWALK"
         },
         "modificationType":"UPDATE",
         "fieldChanges":{
            "description":{
               "newValue":"Mapping from ISO 3166-1 alpha2 and ISO 3166-1 alpha3 to ISO
3166-1 numeric"
            },
            "status":{
               "newValue":{
                  "key":"ActiveKey",
                  "label":"Active"
               }
            }
         },
         "eventTime":"2023-03-07T09:45:37Z",
         "userName":"integration-test-admin"
      },
      {
         "asset":{
            "id":"640707a46b19964cc917d0c8",
            "name":"Enterprise Country to ISO 3166-1 numeric",
            "assetType":"CROSSWALK"
         },
         "modificationType":"UPDATE",
         "eventTime":"2023-03-07T09:45:11Z",
         "userName":"integration-test-admin"
      },
      {
         "asset":{
            "id":"640707a46b19964cc917d0c8",
            "name":"Enterprise Country to ISO 3166-1 numeric",
            "assetType":"CROSSWALK"
         },
         "modificationType":"CREATE",
         "eventTime":"2023-03-07T09:45:08Z",
         "userName":"integration-test-admin"
      }
   ]
}
```

Get value mappings for a code value (v1)

Retrieves the value mappings for a code value.

You can identify the code value that you want to retrieve value mappings for by specifying the value in the Code attribute.

Note: You can't use the + symbol inside the code field value.

GET request

To retrieve the value mappings for a code value, submit a GET request with the following URI and specify the code:

```
/rdm-service/external/v1/crosswalks/{crosswalk ID}/mappings?Code={code}
```

GET response

The response contains the value mappings for the code value.

The following table describes the attributes in the response:

Field	Type	Description
Code	String	Value in the Code attribute of the source code value.
mappings	Array	Includes the values in the Code attribute of the target code values.

GET example

To retrieve the value mappings for the code value, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/crosswalks/
69d08b8c300e9d1aed32a777/mappings?Code=DE HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows value mappings:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 92

[
  {
    "Code":"DE",
    "mappings":[
      "Code 1",
      "Code 2",
      "Code 3",
      "Code 4",
      "Code 5"
    ]
  }
]
```

Delete duplicate mappings of a crosswalk (v1)

The delete request runs a crosswalk cleanser job that scans all mappings of a crosswalk and removes the duplicate mappings.

DELETE request

To delete duplicate mappings of a crosswalk, submit a DELETE request with the following URI:

```
/rdm-service/external/v1/crosswalks/{crosswalkIdentifier}/mappings/duplicates
```

Note: You must execute this API to run the crosswalk cleanser job only if you find duplicate crosswalk mappings in the exported CSV file. The CSV file must be exported using export v3 API at a point in time.

DELETE response

A 202 accepted response is returned.

DELETE example

To delete duplicate mappings of a crosswalk, you might use the following request:

```
DELETE https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/crosswalks/
12102ba6924d0279d40bf5b1/mappings/duplicates HTTP/1.1
Content-Type: application/json
```

The following sample response shows the accepted response:

```
HTTP/1.1 202 Accepted
```

Get job details of a crosswalk cleanser job (v1)

Retrieves job details of a crosswalk cleanser job.

GET request

To get the job details of a crosswalk cleanser job, submit a GET request with the following URI:

```
/rdm-service/external/v1/crosswalks/{crosswalkIdentifier}/mappings/duplicates/status
```

GET response

The response contains the details of the crosswalk cleanser job, such as the status of the crosswalk cleanser job, number of records processed for import, and the error details.

The following table describes the attributes in the response:

Field	Type	Description
crosswalkId	String	Identifier of the crosswalk for which the crosswalk cleanser job was triggered.
createdBy	String	User name of the user who triggered the crosswalk cleanser job.
createdDate	String	Date when the crosswalk cleanser job was triggered.
status	String	Status of the crosswalk cleanser job.
jobDetails	Object	Details of the crosswalk cleanser job.
initialNumberOfMappings	Number	Number of existing crosswalk mappings.

Field	Type	Description
invalidSourcePKeyReport	Object	Details of invalid mappings.
numberOfRecords	Number	Number of invalid records.
numberOfSuccessRecords	Number	Number of invalid records that were successfully deleted.
numberOfFailedRecords	Number	Number of invalid records that weren't deleted.
errors	Array	Details of the errors.
errorCode	String	Error code for the error type.
errorSummary	String	Message that explains why the invalid records aren't deleted.
errorParameter	Object	Error parameter.

GET example

To get the job details of a crosswalk cleanser job, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/crosswalks/
12102ba6924d0279d40bf5b1/mappings/duplicates/status HTTP/1.1
Accept: application/json
```

The following sample response shows the job details of a crosswalk cleanser job:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 555

{
   "crosswalkId":"12102ba6924d0279d40bf5b1",
   "createdBy":"12bb7c6b10e23be958e8b270",
   "createdDate":"2021-12-10T11:29:34.174+00:00",
   "status":"RUNNING",
   "jobDetails":{
      "initialNumberOfMappings":10,
      "invalidSourcePKeyReport":{
         "numberOfRecords":5,
         "numberOfSuccessRecords":4,
         "numberOfFailedRecords":1
      }
   },
   "errors":[
      {
         "errorCode":"RDM.0010000",
         "errorSummary":"Unable to fix the sourcePkey",
         "errorParameter":{
            "sourcePkey":"619cd13b57eb9948fa585d65"
         }
      }
   ]
}
```

Known limitation

If the crosswalk cleanser job fails to remove duplicate mappings of a crosswalk, the API response returns the `RDM.0010269` error code.

To avoid duplicate mappings in the crosswalk when the crosswalk cleanser job fails, perform the following actions:

1. In Reference 360, open the specific crosswalk.
2. Click **Export mappings** to export the value mappings of the crosswalk.
3. Delete the crosswalk.
4. Create another crosswalk between the same source and target code lists.
5. Open the crosswalk and click **Import Mappings** to import the exported CSV file to the crosswalk.

Synchronize a Data Quality dictionary with value mappings of a crosswalk

Synchronizes a Data Quality dictionary with value mappings of a crosswalk based on its internal ID or alias.

PUT request

To synchronize a Data Quality dictionary with value mappings of a crosswalk based on its internal ID or alias, submit a PUT request with the following URI:

```
/rdm-service/external/v1/crosswalks/<crosswalk Identifier>/dictionaries/data
```

The following table describes the query parameter:

Field	Type	Description	
crosswalk Identifier		String	Internal ID or alias of the crosswalk.

PUT response

The PUT response returns HTTP 200 success code in JSON format.

PUT example

To synchronize a Data Quality dictionary with value mappings of a crosswalk based on its internal ID or alias, you might use the following request:

```
PUT https://qa-pod1-mdm.rel.infaqa.com/rdm-service/external/v1/crosswalks/
enterprise_country_currencycodes/dictionaries/data
Accept: application/json
```

The following sample response shows the dictionary synchronization details:

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "dictionarySynchronizationDetails": [
    {
      "dictionaryConfigurationIdentityAndName": {
        "dictionaryName": "Dictionary"
      }
    }
  ]
}
```

crosswalks version 2

Use this resource to retrieve the details of a crosswalk, such as the properties, status, source code list, and target code list based on its internal ID or alias. You can also retrieve the value mappings for a code value.

Unlock a crosswalk (v2)

Unlocks a crosswalk that's locked by another user based on the internal ID or alias.

Note: To use the API, you require the IDMC Reference 360 Administrator role.

PUT request

To unlock a crosswalk based on an internal ID or alias, submit a PUT request with the following URI:

```
/rdm-service/external/v2/crosswalks/{crosswalkKey}/unlock
```

PUT request query parameters

The following table describes the parameter in the request:

Parameter	Type	Description
crosswalkKey	String	Internal ID or alias of a crosswalk that you want to unlock.

PUT response

A 202 accepted response is returned.

PUT example

To unlock a crosswalk based on the internal ID or alias, you might use the following request:

```
PUT https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/crosswalks/
sap_country_to_iso_country/unlock HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the accepted response:

```
HTTP/1.1 202 Accepted
```

Get crosswalk details (v2)

Retrieves the details of a crosswalk based on the internal ID or alias.

GET request

To retrieve the details of a crosswalk based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v1/crosswalks/<crosswalkKey>
```

GET request query parameters

The following table describes the parameter in the request:

Parameter	Type	Description
crosswalkKey	String	Internal ID or alias of a crosswalk.

GET response

The response contains the details of a crosswalk.

The following table describes the attributes in the response body:

Field	Type	Description
id	String	ID of the crosswalk. For more information, see "Asset IDs" on page 199.
internalId	String	Unique identifier of the crosswalk.
alias	String	Alternative identifier of the crosswalk.
sourceCodeListDetails	Object	Includes the details of the source code list.
sourceCodelistId	String	ID of the source code list to which the crosswalk is associated.
internalId	String	Unique identifier of the source code list.
alias	String	Alternative identifier of the target code list.
targetCodeListDetails	Object	Includes the details of the target code list.
targetCodelistId	String	ID of the target code list to which the crosswalk is associated.
internalId	String	Unique identifier of the target code list.
alias	String	Alternative identifier of the target code list.
description	String	Optional. Description of asset.
status	String	Optional. Status of the asset.
confidentiality	String	Optional. Confidentiality of the asset.
sourceApplication	String	Optional. Application of the source code list.
targetApplication	String	Optional. Application of the target code list.
effectiveDate	String	Optional. Date the asset became effective.
approvedById	String	Optional. ID of the approver of the asset.

Field	Type	Description
approvedByName	String	Optional. User name of the user who approved the asset.
approvedOn	String	Optional. Date the asset was approved.

GET example

To retrieve the details of a crosswalk based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/crosswalks/
sap_country_to_iso_country HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the details of a crosswalk:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 300

{
  "id": "67900c57587a205cc9e92834",
  "internalId": "sap_country_to_epm_country",
  "alias": "sapcountry_to_epmcountry",
  "sourceCodeListDetails": {
    "id": "67900c53587a205cc9e92820",
    "internalId": "sap_country",
    "alias": "sapcountry"
  },
  "targetCodeListDetails": {
    "id": "c9abd8c8abe43ad653c3ecd4",
    "internalId": "epm_country",
    "alias": "epmcountry"
  },
  "approvedOn": "2017-03-01",
  "confidentiality": "private",
  "description": "Sample Crosswalk",
  "effectiveDate": "2007-04-01",
  "status": "active"
}
```

Get value mappings for a code value (v2)

Retrieves the value mappings for a code value based on the internal ID or alias.

You can identify the code value that you want to retrieve value mappings for by specifying the value in the Code attribute.

Note: You can't use the + symbol inside the code field value.

GET request

To retrieve the value mappings for a code value, submit a GET request with the following URI and specify the code:

```
/rdm-service/external/v2/crosswalks/{crosswalkIdentifier}/mappings?Code={code}
```

GET request query parameters

The following table describes the parameter in the request:

Parameter	Type	Description
crosswalkIdentifier	String	Internal ID or alias of a crosswalk.
action	String	Action to perform on code values.
page	Integer	Optional. Page number to display. Default is 0.
pageSize	Integer	Optional. The number of records to display in each page. Default is 100. Maximum is 10000.
Code	String	Value in the Code attribute of the source code value.

GET response

The response contains the value mappings for the code value.

The following table describes the attributes in the response body:

Attribute	Type	Description
page	Number	Page from which the response was retrieved.
pageSize	Number	Number of records displayed per page.
totalNumberOfElements	Number	Total number of records found.
numberOfElements	Number	Number of records returned in the current page.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
mappings	Array	Includes the values in the Code attribute of the target code values.
Code	String	Value in the Code attribute of the source code value.
targetCode	String	Value in the Code attribute of the target code value.

GET example

To retrieve the value mappings for the code value, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/crosswalks/
sapcountry_to_isocountry/mappings?Code=DE HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows value mappings:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 92
{
  "pageSize": 100,
  "page": 0,
  "totalNumberOfElements": 5,
  "numberOfElements": 5,
  "lastPage": true,
  "firstPage": true,
  "mappings": [
    {
      "Code": "code1",
      "targetCode": "targetCode1"
    },
    {
      "Code": "code2",
      "targetCode": "targetCode2"
    },
    {
      "Code": "code3",
      "targetCode": "targetCode3"
    },
    {
      "Code": "code4",
      "targetCode": "targetCode4"
    },
    {
      "Code": "code5",
      "targetCode": "targetCode5"
    }
  ]
}
```

Delete duplicate mappings of a crosswalk (v2)

The delete request runs a crosswalk cleanser job that scans all mappings of a crosswalk and removes the duplicate mappings based on the internal ID or alias.

DELETE request

To delete duplicate mappings of a crosswalk based on an internal ID or alias, submit a DELETE request with the following URI:

```
/rdm-service/external/v2/crosswalks/{crosswalkKey}/mappings/duplicates
```

Note: You must execute this API to run the crosswalk cleanser job only if you find duplicate crosswalk mappings in the exported CSV file. The CSV file must be exported using export v3 API at a point in time.

DELETE response

A 202 accepted response is returned.

DELETE example

To delete duplicate mappings of a crosswalk based on the internal ID or alias, you might use the following request:

```
DELETE https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/crosswalks/
sap_country_to_iso_country/mappings/duplicates HTTP/1.1
Content-Type: application/json
```

The following sample response shows the accepted response:

```
HTTP/1.1 202 Accepted
```

Get job details of a crosswalk cleanser job (v2)

Retrieves job details of a crosswalk cleanser job based on the internal ID or alias.

GET request

To get the job details of a crosswalk cleanser job based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/crosswalks/{crosswalkKey}/mappings/duplicates/status
```

GET response

The response contains the details of the crosswalk cleanser job, such as the status of the crosswalk cleanser job, number of records processed for import, and the error details.

The following table describes the attributes in the response body:

Field	Type	Description
id	String	Identifier of the crosswalk for which the crosswalk cleanser job was triggered.
internalId	String	Unique identifier of the crosswalk.
alias	String	Alternative identifier of the crosswalk.
createdBy	String	User name of the user who triggered the crosswalk cleanser job.
createdDate	String	Date when the crosswalk cleanser job was triggered.
status	String	Status of the crosswalk cleanser job.
jobDetails	Object	Details of the crosswalk cleanser job.
initialNumberOfMappings	Number	Number of existing crosswalk mappings.
invalidSourcePKeyReport	Object	Details of invalid mappings.
numberOfRecords	Number	Number of invalid records.
numberOfSuccessRecords	Number	Number of invalid records that were successfully deleted.
numberOfFailedRecords	Number	Number of invalid records that were not deleted.
errors	Array	Details of the errors.
errorCode	String	Error code for the error type.
errorSummary	String	Message that explains why the invalid records are not deleted.
errorParameter	Object	Error parameter.

GET example

To get the job details of a crosswalk cleanser job based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/crosswalks/
sap_country_to_epm_country/mappings/duplicates/status HTTP/1.1
Accept: application/json
```

The following sample response shows the job details of a crosswalk cleanser job:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 555

{
  "id": "572daed0d1a98447ff6e557d",
  "internalId": "sap_country_to_epm_country",
  "alias": "sapcountry_to_epmcountry",
  "createdBy": "5b9d4944f0f0b880eec4068a",
  "createdDate": "2023-09-01T08:15:55.000+00:00",
  "status": "RUNNING",
  "jobDetails": {
    "initialNumberOfMappings": 10,
    "invalidSourcePKeyReport": {
      "numberOfRecords": 5,
      "numberOfSuccessRecords": 4,
      "numberOfFailedRecords": 1
    }
  },
  "errors": [
    {
      "errorCode": "RDM.0010000",
      "errorSummary": "Unable to fix the sourcePkey",
      "errorParameter": {
        "sourcePkey": "619cd13b57eb9948fa585d65"
      }
    }
  ]
}
```

Known limitation

If the crosswalk cleanser job fails to remove duplicate mappings of a crosswalk, the API response returns the `RDM.0010269` error code.

To avoid duplicate mappings in the crosswalk when the crosswalk cleanser job fails, perform the following actions:

1. In Reference 360, open the specific crosswalk.
2. Click **Export mappings** to export the value mappings of the crosswalk.
3. Delete the crosswalk.
4. Create another crosswalk between the same source and target code lists.
5. Open the crosswalk and click **Import Mappings** to import the exported CSV file to the crosswalk.

Get history of value mappings of a crosswalk by time range (v2)

Retrieves all the change events of value mappings of a crosswalk for a specific time range.

GET request

To retrieve all the change events of value mappings of a crosswalk for a specific time range, submit a GET request with the following URI:

```
/rdm-service/external/v2/crosswalks/<crosswalk ID>/mappings/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the query parameters:

Parameter	Description
from	Start date and time of a time range. The start date and time is inclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2023-01-01T00:00:00Z`.
to	End date and time of a time range. The end date and time is exclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2024-01-01T00:00:00Z`.
page	Optional. Page number to display. Default is 0.
pageSize	Optional. The number of records to display on each page. Default is 100. Maximum is 10000.
modificationType	Optional. Type of modification. Value can be CREATE or DELETE.

GET response

The response contains the change events of value mappings of a crosswalk for a specific time range.

The following table describes the attributes in the response:

Attribute	Type	Description
pageSize	Number	The number of records displayed on each page.
page	Number	Page number displayed.
numberOfElements	Number	The number of events.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of modification events.
asset	Object	Includes details about the value mappings of the crosswalk.
modificationType	String	The type of change made to the value mappings of the crosswalk.
from	Object	Displays the previous and new values of the child node in the relationship.

Attribute	Type	Description
to	Object	Displays the previous and new values of the parent node in the relationship.
eventTime	String	The date and time the crosswalk was last modified.
userName	String	The user name of the user who initiated the modification.

GET example

To retrieve all the change events of a crosswalk for a specific time range, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/crosswalks/
cf4f97026e010a95e19c9e79/mappings/audit?
from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of the value mappings of a crosswalk from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 1763

{
   "pageSize":100,
   "page":0,
   "numberOfElements":5,
   "lastPage":true,
   "firstPage":true,
   "content":[
      {
         "asset":{
            "id":"640707a46b19964cc917d0c8",
            "assetType":"CROSSWALK"
         },
         "modificationType":"DELETE",
         "from":{
            "previousValue":"Antarctica"
         },
         "to":{
            "previousValue":"Albania"
         },
         "eventTime":"2023-03-07T09:46:49Z",
         "userName":"integration-test-admin"
      },
      {
         "asset":{
            "id":"640707a46b19964cc917d0c8",
            "assetType":"CROSSWALK"
         },
         "modificationType":"CREATE",
         "from":{
            "newValue":"Antarctica"
         },
         "to":{
            "newValue":"Albania"
         },
         "eventTime":"2023-03-07T09:46:42Z",
         "userName":"integration-test-admin"
      },
      {
         "asset":{
            "id":"640707a46b19964cc917d0c8",
            "assetType":"CROSSWALK"
```

```
            },
            "modificationType":"CREATE",
            "from":{
               "newValue":"Antarctica"
            },
            "to":{
               "newValue":"Antarctica"
            },
            "eventTime":"2023-03-07T09:45:54Z",
            "userName":"integration-test-admin"
         },
         {
            "asset":{
               "id":"640707a46b19964cc917d0c8",
               "assetType":"CROSSWALK"
            },
            "modificationType":"CREATE",
            "from":{
               "newValue":"Albania"
            },
            "to":{
               "newValue":"Albania"
            },
            "eventTime":"2023-03-07T09:45:50Z",
            "userName":"integration-test-admin"
         },
         {
            "asset":{
               "id":"640707a46b19964cc917d0c8",
               "assetType":"CROSSWALK"
            },
            "modificationType":"CREATE",
            "from":{
               "newValue":"Afghanistan"
            },
            "to":{
               "newValue":"Afghanistan"
            },
            "eventTime":"2023-03-07T09:45:46Z",
            "userName":"integration-test-admin"
         }
      ]
   }
```


Delete value mappings of a crosswalk (v2)

To delete value mappings of a crosswalk, you must retrieve the list of all mappings of the crosswalk by crosswalk identifier and then delete value mappings of the crosswalk by mapped source and target code values.

List value mappings of a crosswalk by crosswalk identifier (v2)

Retrieves all value mappings of a crosswalk by crosswalk identifier.

GET request

To retrieve all mappings of a crosswalk by crosswalk identifier, submit a GET request with the following URI:

```
/rdm-service/external/v2/crosswalks/{crosswalkIdentifier}/mappings?
page={page}&pageSize={pageSize}
```

Note: This API also supports crosswalks created from code lists belonging to different reference data sets.

GET request query parameters

You can append query parameters to the URI to specify the page number and page size.

The following table lists the query parameters:

Field	Type	Description
page	Number	Page number to display. Default is 0.
pageSize	Number	Number of records to display on each page. Default is 100.

GET response

The response contains the mappings of a specific crosswalk by crosswalk identifier.

The following table describes the attributes in the response:

Field	Type	Description
mappings	Array	List of mappings for the specified crosswalk.
Code	String	Source code value of a crosswalk mapping.
targetCode	String	Target code value of a crosswalk mapping.
page	Number	Page number to display. Default is 0.
pageSize	Number	Number of records to display on each page. Default is 100.
firstPage	Boolean	Indicates whether the current page is the first page of the total results.
lastPage	Boolean	Indicates whether the current page is the last page of the total results.
numberOfElements	Number	Number of records returned in the current page.
totalNumberOfElements	Number	Total number of records found.

GET example

To retrieve the mappings of a specific crosswalk by crosswalk identifier, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/crosswalks/
daf0cc189e530b6979ac77ce/mappings?page=0&pageSize=100 HTTP/1.1
Accept: application/json
```

The following sample response shows the mappings of a specific crosswalk by crosswalk identifier:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 465

{
   "pageSize":100,
   "page":0,
   "totalNumberOfElements":5,
   "numberOfElements":5,
   "lastPage":true,
   "firstPage":true,
   "mappings":[
      {
         "Code":"code1",
         "targetCode":"targetCode1"
      },
      {
         "Code":"code2",
```

```
            "targetCode":"targetCode2"
         },
         {
            "Code":"code3",
            "targetCode":"targetCode3"
         },
         {
            "Code":"code4",
            "targetCode":"targetCode4"
         },
         {
            "Code":"code5",
            "targetCode":"targetCode5"
         }
      ]
}
```

Delete value mappings of a crosswalk by mapped source and target code values (v2)

Delete value mappings of a crosswalk by mapped source and target code values.

POST request

To delete value mappings of a crosswalk by mapped source and target code values, submit a POST request with the following URI:

```
/rdm-service/external/v2/crosswalks/{crosswalkIdentifier}/mappings
```

Note:

- You can delete a maximum of 100 value mappings with each request.
- This API also supports crosswalks created from code lists belonging to different reference data sets.

Use the following parameters in the request body to specify the value mappings of a crosswalk to delete:

Field	Type	Description
action	String	Action to perform.
mappings	Array	List of mappings for the specified crosswalk.
Code	String	Source code value of a crosswalk mapping.
targetCode	String	Target code value of a crosswalk mapping.

POST response

A 204 no content response is returned.

POST example

To delete value mappings of a crosswalk by mapped source and target code values, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/crosswalks/
daf0cc189e530b6979ac77ce/mappings HTTP/1.1
Content-Type: application/json
Content-Length: 384

{
   "action":"DELETE",
   "mappings":[
      {
```

```
            "Code":"sourceCode1",
            "targetCode":"targetCode1"
        },
        {
            "Code":"sourceCode2",
            "targetCode":"targetCode2"
        },
        {
            "Code":"sourceCode3",
            "targetCode":"targetCode3"
        },
        {
            "Code":"sourceCode4",
            "targetCode":"targetCode4"
        },
        {
            "Code":"sourceCode5",
            "targetCode":"targetCode5"
        }
    ]
}
```

The following sample response shows the no content response:

```
HTTP/1.1 204 No Content
```

crosswalk version 3

Use this resource to retrieve the value mappings for a code value based on its internal ID or alias.

Get history of a crosswalk by time range (v3)

Retrieves all the change events of a crosswalk for a specific time range based on the internal ID or alias.

GET request

To retrieve all the change events of a crosswalk for a specific time range based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v3/crosswalks/{assetKey}/summary/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the query parameters:

Parameter	Description
assetKey	Internal ID or alias of a crosswalk. **Note:** When you provide an invalid assetKey in the request URL, the crosswalk version 3 REST API returns an error if the request contains invalid internal IDs or aliases.
from	Start date and time of the time range, which is inclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-12T14:04:04Z`.

Parameter	Description
to	End date and time of the time range, which is exclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-15T14:04:04Z`.
page	Optional. Page number to display. Default is `0`.
pageSize	Optional. Number of records to display per page. Default value is `100`. Maximum value is `10000`.
modificationType	Optional. Type of modification. Value can be CREATE, UPDATE, or DELETE.

GET response

The response contains the change events of a crosswalk for a specific time range.

The following table describes the attributes in the response body:

Attributes	Type	Description
pageSize	Number	Number of records displayed per page.
page	Number	Page from which the response was retrieved.
numberOfElements	Number	Number of records returned in the current page.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of modification events.
id	String	ID of the crosswalk. For more information, see "Asset IDs" on page 199.
name	String	Name of the crosswalk.
internalId	String	Unique identifier of the crosswalk.
alias	String	Alternative identifier of the crosswalk.
assetType	String	Optional. Type of asset. Set the value to REFERENCEDATASET, CODELIST, CROSSWALK, or HIERARCHY.
asset	Object	Includes details about the crosswalk.
modificationType	String	Type of change made to the crosswalk. Value can be CREATE, UPDATE, or DELETE.
fieldChanges	Object	Displays the previous and new values of the fields with changes.
attributeChanges	Object	Displays the previous and new values of the attributes with changes.
eventTime	String	Date and time when the crosswalk was last modified.
userName	String	User name of the user who initiated the modification.

GET example

To retrieve all the change events of a crosswalk for a specific time range based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/crosswalks/
cl_audit/summary/audit?from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of the crosswalk from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 1247

{
  "pageSize": 100,
  "page": 0,
  "numberOfElements": 4,
  "lastPage": true,
  "firstPage": true,
  "content": [
    {
      "asset": {
        "id": "63eb866c3eceda5c401e22bb",
        "name":
"ZIT_16763797384291632_4196_ExternalCodeListAuditTrailTest_IntTestCodelist1_1",
        "internalId": "cl_audit",
        "alias": "als_cl_audit",
        "assetType": "CODELIST"
      },
      "modificationType": "UPDATE",
      "eventTime": "2023-02-14T13:02:48Z",
      "userName": "integration-test-admin"
    },
    {
      "asset": {
        "id": "63eb866c3eceda5c401e22bb",
        "name":
"ZIT_16763797384291632_4196_ExternalCodeListAuditTrailTest_IntTestCodelist1_1",
        "internalId": "cl_audit",
        "alias": "als_cl_audit",
        "assetType": "CODELIST"
      },
      "modificationType": "UPDATE",
      "fieldChanges": {
        "description": {
          "previousValue": "description 1",
          "newValue": "description 2"
        },
        "version": {
          "previousValue": "version 1",
          "newValue": "version 2"
        }
      },
      "attributeChanges": {
        "ReferenceAttribute": {
          "displayAttributes": {
            "previousValue": [
              "Name",
              "Code"
            ],
            "newValue": [
              "Code"
            ]
          }
        }
      },
      "eventTime": "2023-02-14T13:02:46Z",
```

```
      "userName": "integration-test-admin"
    },
    {
      "asset": {
        "id": "63eb866c3eceda5c401e22bb",
        "name":
"ZIT_16763797384291632_4196_ExternalCodeListAuditTrailTest_IntTestCodelist1_1",
        "internalId": "cl_audit",
        "alias": "als_cl_audit",
        "assetType": "CODELIST"
      },
      "modificationType": "UPDATE",
      "eventTime": "2023-02-14T13:02:40Z",
      "userName": "integration-test-admin"
    },
    {
      "asset": {
        "id": "63eb866c3eceda5c401e22bb",
        "name":
"ZIT_16763797384291632_4196_ExternalCodeListAuditTrailTest_IntTestCodelist1_1",
        "internalId": "cl_audit",
        "alias": "als_cl_audit",
        "assetType": "CODELIST"
      },
      "modificationType": "CREATE",
      "fieldChanges": {
        "description": {
          "newValue": "description 1"
        },
        "displayColumns": {
          "newValue": [
            "Name"
          ]
        },
        "hierarchical": {
          "newValue": "true"
        },
        "levels": {
          "newValue": "6"
        },
        "name": {
          "newValue":
"ZIT_16763797384291632_4196_ExternalCodeListAuditTrailTest_IntTestCodelist1_1"
        },
        "termId": {
          "newValue": "63eb865cf663af1d2a7ef268"
        },
        "version": {
          "newValue": "version 1"
        }
      },
      "attributeChanges": {
        "Code": {
          "name": {
            "newValue": "Code"
          },
          "type": {
            "newValue": "String"
          },
          "required": {
            "newValue": "true"
          }
        },
        "Description": {
          "name": {
            "newValue": "Description"
          },
          "type": {
            "newValue": "String"
          },
          "required": {
```

```
            "newValue": "false"
          }
        },
        "Name": {
          "name": {
            "newValue": "Name"
          },
          "type": {
            "newValue": "String"
          },
          "required": {
            "newValue": "true"
          }
        },
        "ReferenceAttribute": {
          "name": {
            "newValue": "referenceAttribute"
          },
          "type": {
            "newValue": "Reference"
          },
          "required": {
            "newValue": "false"
          },
          "displayAttributes": {
            "newValue": [
              "Name",
              "Code"
            ]
          },
          "referenceDataSetId": {
            "newValue": "63ffb73ba7fc2c03f6273205"
          },
          "codeListId": {
            "newValue": "63ffb73d87e60e6fcd814fbd"
          }
        }
      },
      "dependencyChanges": {
        "displayAttributes": {
          "newValue": [
            "Name",
            "Code"
          ]
        },
        "referenceDataSetId": {
          "newValue": "63ffc0e0a7fc2c03f627323c"
        },
        "codeListId": {
          "newValue": "63ffc0e287e60e6fcd815026"
        }
      },
      "eventTime": "2023-02-14T13:02:35Z",
      "userName": "integration-test-admin"
    }
  ]
}
```


Get value mappings for a code value (v3)

Retrieves the value mappings for a code value based on the internal ID or alias.

You can identify the code value that you want to retrieve value mappings for by specifying the value in the Code attribute.

Note: You can't use the + symbol inside the code field value.

GET request

To retrieve the value mappings for a code value based on an internal ID or alias, submit a GET request with the following URI and specify the code:

```
/rdm-service/external/v3/crosswalks/{crosswalkKey}/mappings?Code={code}
```

GET request query parameters

The following table describes the parameter in the request:

Parameter	Type	Description
crosswalkKey	String	Internal ID or alias of a crosswalk.
page	Number	Optional. Page number to display. Default is 0.
pageSize	Number	Optional. The number of records to display on each page. Default is 100. Maximum is 10000.
Code	String	Value in the Code attribute of the source code value.

GET response

The response contains the value mappings for the code value.

The following table describes the attributes in the response body:

Field	Type	Description
pageSize	Number	Number of records displayed per page.
page	Number	Page from which the response was retrieved.
totalNumberOfElements	Number	Total number of records found.
numberOfElements	Number	Number of records returned in the current page.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
mappings	Array	Includes the values in the Code attribute of the target code values.
Code	String	Source code value of a crosswalk mapping.
targetCode	String	Target code value of a crosswalk mapping.

GET example

To retrieve the value mappings for the code value based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/crosswalks/
sapcountry_to_isocountry/mappings?Code=DE HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows value mappings:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 92

{
  "pageSize": 100,
  "page": 0,
  "totalNumberOfElements": 5,
  "numberOfElements": 5,
  "lastPage": true,
  "firstPage": true,
  "mappings": [
    {
      "Code": "code1",
      "targetCode": "targetCode1"
    },
    {
      "Code": "code2",
      "targetCode": "targetCode2"
    },
    {
      "Code": "code3",
      "targetCode": "targetCode3"
    },
    {
      "Code": "code4",
      "targetCode": "targetCode4"
    },
    {
      "Code": "code5",
      "targetCode": "targetCode5"
    }
  ]
}
```

Delete value mappings of a crosswalk by mapped source and target code values (v3)

Delete value mappings of a crosswalk by mapped source and target code values based on the internal ID or alias.

POST request

To delete value mappings of a crosswalk by mapped source and target code values based on an internal ID or alias, submit a POST request with the following URI:

```
/rdm-service/external/v3/crosswalks/{crosswalkKey}/mappings
```

Note:

- You can delete a maximum of 100 value mappings with each request.
- This API also supports crosswalks created from code lists belonging to different reference data sets.

Use the following parameters in the request body to specify the value mappings of a crosswalk to delete:

Field	Type	Description
crosswalkKey	String	Internal ID or alias of a crosswalk.
action	String	Action to perform.

Field	Type	Description
mappings	Array	List of mappings for the specified crosswalk.
Code	String	Source code value of a crosswalk mapping.
targetCode	String	Target code value of a crosswalk mapping.

POST response

A 204 no content response is returned.

POST example

To delete value mappings of a crosswalk by mapped source and target code values based on the internal ID or alias, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v3/crosswalks/
sapcountry_to_isocountry/mappings HTTP/1.1
Content-Type: application/json
Content-Length: 384

{
   "action":"DELETE",
   "mappings":[
      {
         "Code":"sourceCode1",
         "targetCode":"targetCode1"
      },
      {
         "Code":"sourceCode2",
         "targetCode":"targetCode2"
      },
      {
         "Code":"sourceCode3",
         "targetCode":"targetCode3"
      },
      {
         "Code":"sourceCode4",
         "targetCode":"targetCode4"
      },
      {
         "Code":"sourceCode5",
         "targetCode":"targetCode5"
      }
   ]
}
```

The following sample response shows the no content response:

```
HTTP/1.1 204 No Content
```

hierarchies version 1

Use this resource to retrieve hierarchies, hierarchy details, and hierarchy model relationships.

Get hierarchies (v1)

Retrieves a list of all hierarchies.

GET request

To retrieve a list of all hierarchies, submit a GET request with the following URI:

```
/rdm-service/external/v1/hierarchies
```

GET response

The response contains information about each hierarchy.

The following table describes the attributes in the response:

Field	Type	Definition
id	String	ID of the asset. Assets include reference data sets, code lists, crosswalks, and hierarchies. For more information, see "Asset IDs" on page 199.
name	String	Name of the asset.
description	String	Optional. Description of asset.
version	String	Optional. Version of the code list.
domain	String	Optional. Domain of the asset.
confidentiality	String	Optional. Confidentiality of the asset.
priority	String	Optional. Priority of the asset.
application	String	Optional. Application that uses the code list.
priority	String	Optional. Priority of the asset.
status	String	Optional. Status of the asset.
effectiveDate	String	Optional. Date the asset became effective.
approvedById	String	Optional. ID of the approver of the asset.
approvedByName	String	Optional. User name of the user who approved the asset.
approvedOn	String	Optional. Date the asset was approved.

GET example

To retrieve the list of all hierarchies, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/hierarchies
HTTP/1.1
```

```
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows hierarchies:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 452

[
  {
    "id":"6e7dd28fc13b417c5c19d1fb",
    "name":"Cost centers",
    "description":"Cost center hierarchy",
    "version":"v1",
    "domain":"International standards",
    "confidentiality":"private",
    "priority":"Prio1",
    "status":"Draft",
    "effectiveDate":"2017-04-01",
    "approvedOn":"2017-03-01"
  },
  {
    "id":"e3e57e39ea8623f258013c43",
    "name":"Profit centers",
    "description":"Profit center hierarchy",
    "version":"v1"
  }
]
```

Get hierarchy details (v1)

Retrieves the details of a hierarchy, such as the properties and status.

GET request

To retrieve the details of a hierarchy, submit a GET request with the following URI:

```
/rdm-service/external/v1/hierarchies/<hierarchy ID>
```

GET response

The response contains the details of the hierarchy.

The following table describes the attributes in the response:

Field	Type	Definition
id	String	ID of the asset. Assets include reference data sets, code lists, crosswalks, and hierarchies. For more information, see "Asset IDs" on page 199.
name	String	Name of the asset.
description	String	Optional. Description of asset.
version	String	Optional. Version of the code list.
domain	String	Optional. Domain of the asset.
confidentiality	String	Optional. Confidentiality of the asset.
priority	String	Optional. Priority of the asset.

Field	Type	Definition
application	String	Optional. Application that uses the code list.
priority	String	Optional. Priority of the asset.
status	String	Optional. Status of the asset.
effectiveDate	String	Optional. Date the asset became effective.
approvedById	String	Optional. ID of the approver of the asset.
approvedByName	String	Optional. User name of the user who approved the asset.
approvedOn	String	Optional. Date the asset was approved.

GET example

To retrieve the hierarchy details, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/hierarchies/
6e7dd28fc13b417c5c19d1fb HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows hierarchies:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 334

{
  "id":"6e7dd28fc13b417c5c19d1fb",
  "name":"Cost centers",
  "description":"Cost center hierarchy",
  "version":"v1",
  "application":"App1",
  "domain":"Internal standards",
  "confidentiality":"private",
  "priority":"Prio1",
  "status":"Draft",
  "effectiveDate":"2017-04-01",
  "approvedOn":"2020-03-01"
}
```

Get hierarchy model relationships (v1)

Retrieves the relationships in a hierarchy model.

GET request

To retrieve the relationships in a hierarchy model, submit a GET request with the following URI:

```
/rdm-service/external/v1/hierarchies/<hierarchy ID>/relations
```

GET response

The response contains the relationships and the code lists in each relationship. The `child.codeListId` attribute contains the top-level node relationship.

The following table describes the attributes in the response:

Field	Type	Definition
relations	-	Lists the code list in the relationship.
child	-	Contains information about the child code list.
parent	-	Contains information about the parent code list.
codeListId	String	ID of the code list.
codeListName	String	Name of the code list.
termId	String	ID of the reference data set to which the code list is associated.
termName	String	Name of the reference data set.

GET example

To retrieve the relationships in a hierarchy model, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/hierarchies/
6e7dd28fc13b417c5c19d1fb/relations HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows hierarchies:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 452

{
  "relations":[
    {
      "child":{
        "codeListId":"8bc955e614df2040968e9d85",
        "codeListName":"Parent Codelist",
        "termId":"28a1320fe7f63cd25b58bef4",
        "termName":"Reference Data Set"
      }
    },
    {
      "parent":{
        "codeListId":"8bc955e614df2040968e9d85",
        "codeListName":"Parent Codelist",
        "termId":"28a1320fe7f63cd25b58bef4",
        "termName":"Reference Data Set"
      },
      "child":{
        "codeListId":"81a714a66863f954a9b60045",
        "codeListName":"First Level Codelist",
        "termId":"28a1320fe7f63cd25b58bef4",
        "termName":"Reference Data Set"
      }
    }
  ]
}
```

The `child.codeListId` attribute contains the top-level node relationship. For example, the first relationship in the example is the top-level node relationship.

Get history of a hierarchy by time range (v1)

Retrieves all the change events of a hierarchy for a specific time range.

GET request

To retrieve all the change events of a hierarchy for a specific time range, submit a GET request with the following URI:

```
/rdm-service/external/v1/hierarchies/<identifier>/summary/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the query parameters:

Parameter	Description
identifier	ID of the hierarchy. For more information, see "Asset IDs" on page 199. **Note:** When you provide an invalid identifier in the request URL, the hierarchies REST API returns an empty content without an error.
from	Start date and time of a time range. The start date and time is inclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2023-01-01T00:00:00Z`.
to	End date and time of a time range. The end date and time is exclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2024-01-01T00:00:00Z`.
page	Optional. Page number to display. Default is 0.
pageSize	Optional. The number of records to display in each page. Default is 100. Maximum is 10000.
modificationType	Optional. Type of modification. Value can be CREATE, UPDATE, or DELETE.

GET response

The response contains the change events of a hierarchy for a specific time range.

The following table describes the attributes in the response:

Attribute	Type	Description
pageSize	Number	The number of records displayed on each page.
page	Number	Page number displayed.
numberOfElements	Number	The number of events.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.

Attribute	Type	Description
content	Array	Includes the list of modification events.
asset	Object	Includes details about the hierarchy.
modificationType	String	The type of change made to the hierarchy.
fieldChanges	Object	Displays the previous and new values of the fields with changes.
attributeChanges	Object	Displays the previous and new values of the attributes with changes.
eventTime	String	The date and time the hierarchy was last updated.
userName	String	The user name of the user who initiated the modification.

GET example

To retrieve all the change events of a hierarchy for a specific time range, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/hierarchies/
a5bfc1f71d31ae1149dae8ef/summary/audit?from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z
HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of the hierarchy from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 2552

{
   "pageSize":100,
   "page":0,
   "numberOfElements":6,
   "lastPage":true,
   "firstPage":true,
   "content":[
      {
         "asset":{
            "id":"640717485b243e32c4b048f9",
            "name":"Country ISO 3166-1",
            "assetType":"HIERARCHY"
         },
         "modificationType":"UPDATE",
         "eventTime":"2023-03-07T10:53:29Z",
         "userName":"mdm-rdm-service"
      },
      {
         "asset":{
            "id":"640717485b243e32c4b048f9",
            "name":"Country ISO 3166-1",
            "assetType":"HIERARCHY"
         },
         "modificationType":"UPDATE",
```

```
   "fieldChanges":{
      "application":{
         "newValue":{
            "key":"EnterpriseKey",
            "label":"Enterprise"
         }
      },
      "confidentiality":{
         "newValue":{
            "key":"PublicKey",
            "label":"Public"
         }
      },
      "description":{
         "newValue":"Hierarchy for Country Codes"
      },
      "domain":{
         "newValue":{
            "key":"GeographyKey",
            "label":"Geography"
         }
      },
      "priority":{
         "newValue":{
            "key":"Priority4",
            "label":"Low"
         }
      },
      "status":{
         "newValue":{
            "key":"ActiveKey",
            "label":"Active"
         }
      },
      "version":{
         "newValue":"1"
      }
   },
   "eventTime":"2023-03-07T10:53:28Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"640717485b243e32c4b048f9",
      "name":"Country ISO 3166-1",
      "assetType":"HIERARCHY"
   },
   "modificationType":"UPDATE",
   "eventTime":"2023-03-07T10:52:15Z",
   "userName":"mdm-rdm-service"
},
{
   "asset":{
      "id":"640717485b243e32c4b048f9",
      "name":"Country ISO 3166-1",
      "assetType":"HIERARCHY"
   },
   "modificationType":"UPDATE",
   "eventTime":"2023-03-07T10:52:14Z",
   "userName":"integration-test-admin"
},
{
   "asset":{
      "id":"640717485b243e32c4b048f9",
      "name":"Country ISO 3166-1",
      "assetType":"HIERARCHY"
   },
   "modificationType":"UPDATE",
   "eventTime":"2023-03-07T10:51:53Z",
   "userName":"mdm-rdm-service"
},
```

```
        {
           "asset":{
              "id":"640717485b243e32c4b048f9",
              "name":"Country ISO 3166-1",
              "assetType":"HIERARCHY"
           },
           "modificationType":"CREATE",
           "fieldChanges":{
              "name":{
                 "newValue":"Country ISO 3166-1"
              }
           },
           "eventTime":"2023-03-07T10:51:52Z",
           "userName":"integration-test-admin"
        }
     ]
  }
```

Get history of hierarchy relationships by time range (v1)

Retrieves all the change events of hierarchy relationships for a specific time range.

GET request

To retrieve all the change events of hierarchy relationships for a specific time range, submit a GET request with the following URI:

```
/rdm-service/external/v1/hierarchies/<hierarchy ID>/mappings/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the query parameters:

Parameter	Description
from	Start date and time of a time range. The start date and time is inclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2023-01-01T00:00:00Z`.
to	End date and time of a time range. The end date and time is exclusive. Set the time range in the ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you can set `2024-01-01T00:00:00Z`.
page	Optional. Page number to display. Default is 0.
pageSize	Optional. The number of records to display on each page. Default is 100. Maximum is 10000.
modificationType	Optional. Type of modification. Value can be CREATE or DELETE.

GET response

The response contains the change events of hierarchy relationships for a specific time range.

The following table describes the attributes in the response:

Attribute	Type	Description
pageSize	Number	The number of records displayed on each page.
page	Number	Page number displayed.
numberOfElements	Number	The number of events.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of modification events.
asset	Object	Includes details about the hierarchy.
modificationType	String	The type of change made to the hierarchy.
from	Object	Displays the previous and new values of the child node in the relationship.
to	Object	Displays the previous and new values of the parent node in the relationship.
eventTime	String	The date and time the hierarchy was last updated.
userName	String	The user name of the user who initiated the modification.

GET example

To retrieve all the change events of a hierarchy for a specific time range, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/hierarchies/
a34cf6171f8db153e41977d0/mappings/audit?
from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of the hierarchy from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 1929

{
   "pageSize":100,
   "page":0,
   "numberOfElements":6,
   "lastPage":true,
   "firstPage":true,
```

```
"content":[
   {
      "asset":{
         "id":"640717485b243e32c4b048f9",
         "assetType":"HIERARCHY"
      },
      "modificationType":"CREATE",
      "from":{
         "newValue":"Antarctica"
      },
      "to":{
         "newValue":"Antarctica"
      },
      "eventTime":"2023-03-07T10:53:00Z",
      "userName":"integration-test-admin"
   },
   {
      "asset":{
         "id":"640717485b243e32c4b048f9",
         "assetType":"HIERARCHY"
      },
      "modificationType":"CREATE",
      "from":{
         "newValue":"Albania"
      },
      "to":{
         "newValue":"Albania"
      },
      "eventTime":"2023-03-07T10:52:52Z",
      "userName":"integration-test-admin"
   },
   {
      "asset":{
         "id":"640717485b243e32c4b048f9",
         "assetType":"HIERARCHY"
      },
      "modificationType":"CREATE",
      "from":{
         "newValue":"Afghanistan"
      },
      "to":{
         "newValue":"Afghanistan"
      },
      "eventTime":"2023-03-07T10:52:44Z",
      "userName":"integration-test-admin"
   },
   {
      "asset":{
         "id":"640717485b243e32c4b048f9",
         "assetType":"HIERARCHY"
      },
      "modificationType":"CREATE",
      "from":{
         "newValue":"Antarctica"
      },
      "eventTime":"2023-03-07T10:52:35Z",
      "userName":"integration-test-admin"
   },
   {
      "asset":{
         "id":"640717485b243e32c4b048f9",
         "assetType":"HIERARCHY"
      },
      "modificationType":"CREATE",
      "from":{
         "newValue":"Albania"
      },
      "eventTime":"2023-03-07T10:52:34Z",
      "userName":"integration-test-admin"
   },
   {
```

```
            "asset":{
               "id":"640717485b243e32c4b048f9",
               "assetType":"HIERARCHY"
            },
            "modificationType":"CREATE",
            "from":{
               "newValue":"Afghanistan"
            },
            "eventTime":"2023-03-07T10:52:34Z",
            "userName":"integration-test-admin"
         }
      ]
   }
```


hierarchies version 2

Use this resource to retrieve hierarchies, hierarchy details, and hierarchy model relationships based on its internal ID or alias.

Get hierarchy details (v2)

Retrieves the details of a hierarchy based on the internal ID or alias.

GET request

To retrieve the details of a hierarchy based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/hierarchies/{key}
```

GET request query parameters

The following table describes the parameter in the request:

Parameter	Type	Description
key	String	Internal ID or alias of the hierarchy.

GET response

The response contains the details of the hierarchy.

The following table describes the attributes in the response body:

Field	Type	Description
id	String	ID of the hierarchy. For more information, see "Asset IDs" on page 199.
name	String	Name of the hierarchy.
internalId	String	Unique identifier of the code list.
alias	String	Alternative identifier of the code list.

Field	Type	Description
description	String	Optional. Description of the hierarchy.
version	String	Optional. Version of the hierarchy.
application	String	Optional. Application that uses the hierarchy.
domain	String	Optional. Domain of the hierarchy.
confidentiality	String	Optional. Confidentiality of the hierarchy.
priority	String	Optional. Priority of the hierarchy.
status	String	Optional. Status of the hierarchy.
effectiveDate	String	Optional. Date the hierarchy became effective.
approvedOn	String	Optional. Date the hierarchy was approved.

GET example

To retrieve the details of a hierarchy based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/hierarchies/
hierarchy_internal_id HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the details of the hierarchy:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 334

{
  "id": "640717485b243e32c4b048f9",
  "name": "Cost centers",
  "internalId": "hierarchy_internal_id",
  "alias": "hierarchy_alias",
  "description": "Cost center hierarchy",
  "version": "v1",
  "application": "App1",
  "domain": "Internal standards",
  "confidentiality": "private",
  "priority": "Prio1",
  "status": "Draft",
  "effectiveDate": "2017-04-01",
  "approvedOn": "2020-03-01"
}
```

Get hierarchy model relationships (v2)

Retrieves the relationships in a hierarchy model based on the internal ID or alias.

GET request

To retrieve the relationships in a hierarchy model based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/hierarchies/{key}/relations
```

GET request query parameters

The following table describes the parameter in the request:

Parameter	Type	Description
key	String	Internal ID or alias of the hierarchy.

GET response

The response contains the relationships and the code lists in each relationship. The `child.codeListDetails` attribute contains the top-level node relationship.

The following table describes the attributes in the response body:

Field	Type	Definition
relations	Array	Lists the code list in the relationship.
child	Object	Contains information about the child in the hierarchy model.
codeListDetails	Object	Includes details for the child code list.
id	String	ID of the child code list.
internalId	String	Unique identifier of the child code list.
alias	String	Alternative identifier of the child code list.
name	String	Name of the child code list.
rdsDetails	Object	Includes details for the child reference data set.
id	String	ID of the child reference data set. For more information, see "Asset IDs" on page 199.
internalID	String	Unique identifier of the child reference data set.
alias	String	Alternative identifier of the child reference data set.
name	String	Name of the child reference data set.
parent	Object	Contains information about the parent in the hierarchy model.
codeListDetails	Object	Includes details for the parent code list.
id	String	ID of the parent code list.
internalId	String	Unique identifier of the parent code list.
alias	String	Alternative identifier of the parent code list.
name	String	Name of the parent code list.
rdsDetails	Object	Includes details for the parent reference data set.

Field	Type	Definition
id	String	ID of the parent reference data set. For more information, see "Asset IDs" on page 199.
internalID	String	Unique identifier of the parent reference data set.
alias	String	Alternative identifier of the parent reference data set.
name	String	Name of the parent reference data set.

GET example

To retrieve the relationships in a hierarchy model based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/hierarchies/
parent_codelist/relations HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows hierarchies:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 452
{
  "relations": [
    {
      "child": {
        "codeListDetails": {
          "id": "b9270cec5293af42eb60fa00",
          "internalId": "parent_codelist",
          "alias": "parentcodelist",
          "name": "Parent Codelist"
        },
        "rdsDetails": {
          "id": "fa9c90dfdd269c354581807f",
          "internalId": "parent_rds",
          "alias": "parentrds",
          "name": "Reference Data Set"
        }
      }
    },
    {
      "parent": {
        "codeListDetails": {
          "id": "b9270cec5293af42eb60fa00",
          "internalId": "parent_codelist",
          "alias": "parentcodelist",
          "name": "Parent Codelist"
        },
        "rdsDetails": {
          "id": "fa9c90dfdd269c354581807f",
          "internalId": "parent_rds",
          "alias": "parentrds",
          "name": "Reference Data Set"
        }
      },
      "child": {
        "codeListDetails": {
          "id": "d1cf0227c95f4668ffcfaeda",
          "internalId": "first_level_codelist",
          "alias": "firstlevelcodelist",
          "name": "First Level Codelist"
        },
        "rdsDetails": {
          "id": "fa9c90dfdd269c354581807f",
```

```
            "internalId": "parent_rds",
            "alias": "parentrds",
            "name": "Reference Data Set"
          }
        }
      }
    ]
  }
```

The `child.codeListDetails` attribute contains the top-level node relationship. For example, the first relationship in the example is the top-level node relationship.

Get history of a hierarchy by time range (v2)

Retrieves all the change events of a hierarchy for a specific time range based on the internal ID or alias.

GET request

To retrieve all the change events of a hierarchy for a specific time range based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/hierarchies/{assetKey}/summary/audit
```

GET request query parameters

The following table describes the parameter in the request body:

Parameter	Description
assetKey	Internal ID or alias of the hierarchy. **Note:** When you provide an invalid assetKey in the request URL, the hierarchies version 2 REST API returns an error if the request contains invalid internal IDs or aliases.
from	Start date and time of the time range, which is inclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-12T14:04:04Z`.
to	End date and time of the time range, which is exclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-15T14:04:04Z`.
page	Optional. Page number to display. Default is `0`.
pageSize	Optional. Number of records to display per page. Default value is `100`. Maximum value is `10000`.
modificationType	Optional. Type of modification. Value can be CREATE, UPDATE, or DELETE.

GET response

The response contains the change events of a hierarchy for a specific time range.

The following table describes the attributes in the response body:

Attribute	Type	Description
pageSize	Number	Number of records displayed per page.
page	Number	Page from which the response was retrieved.
numberOfElements	Number	Number of records returned in the current page.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
firstPage	Boolean	Indicates whether the current page is the first page in the total result.
content	Array	Includes the list of modification events.
asset	Object	Includes details about the asset.
id	String	ID of the hierarchy. For more information, see "Asset IDs" on page 199.
name	String	Name of the hierarchy.
internalId	String	Unique identifier of the hierarchy.
alias	String	Alternative identifier of the hierarchy.
assetType	String	Type of the asset. The value is `hierarchy`.
modificationType	String	Type of change made to the hierarchy. Value can be CREATE, UPDATE, or DELETE.
fieldChanges	Object	Includes details of the previous and new values of the fields with changes.
confidentiality	String	Optional. Confidentiality of the hierarchy.
description	String	Optional. Description of the hierarchy.
domain	String	Optional. Domain of the hierarchy.
priority	String	Optional. Priority of the hierarchy.
status	String	Optional. Status of the hierarchy.

Attribute	Type	Description
version	String	Optional. Version of the hierarchy.
eventTime	String	Date and time when the hierarchy was last updated.
userName	String	User name of the user who initiated the modification.

GET example

To retrieve all the change events of a hierarchy for a specific time range based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/hierarchies/
country_iso/summary/audit?from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of the hierarchy from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 2552

{
  "pageSize": 100,
  "page": 0,
  "numberOfElements": 6,
  "lastPage": true,
  "firstPage": true,
  "content": [
    {
      "asset": {
        "id": "640717485b243e32c4b048f9",
        "name": "Country ISO 3166-1",
        "internalId": "country_iso",
        "alias": "als_country_iso",
        "assetType": "HIERARCHY"
      },
      "modificationType": "UPDATE",
      "eventTime": "2023-03-07T10:53:29Z",
      "userName": "mdm-rdm-service"
    },
    {
      "asset": {
        "id": "640717485b243e32c4b048f9",
        "name": "Country ISO 3166-1",
        "internalId": "country_iso",
        "alias": "als_country_iso",
        "assetType": "HIERARCHY"
      },
      "modificationType": "UPDATE",
      "fieldChanges": {
        "application": {
          "newValue": {
            "key": "EnterpriseKey",
            "label": "Enterprise"
          }
        },
        "confidentiality": {
          "newValue": {
            "key": "PublicKey",
            "label": "Public"
```

```
      }
    },
    "description": {
      "newValue": "Hierarchy for Country Codes"
    },
    "domain": {
      "newValue": {
        "key": "GeographyKey",
        "label": "Geography"
      }
    },
    "priority": {
      "newValue": {
        "key": "Priority4",
        "label": "Low"
      }
    },
    "status": {
      "newValue": {
        "key": "ActiveKey",
        "label": "Active"
      }
    },
    "version": {
      "newValue": "1"
    }
  },
  "eventTime": "2023-03-07T10:53:28Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "640717485b243e32c4b048f9",
    "name": "Country ISO 3166-1",
    "internalId": "country_iso",
    "alias": "als_country_iso",
    "assetType": "HIERARCHY"
  },
  "modificationType": "UPDATE",
  "eventTime": "2023-03-07T10:52:15Z",
  "userName": "mdm-rdm-service"
},
{
  "asset": {
    "id": "640717485b243e32c4b048f9",
    "name": "Country ISO 3166-1",
    "internalId": "country_iso",
    "alias": "als_country_iso",
    "assetType": "HIERARCHY"
  },
  "modificationType": "UPDATE",
  "eventTime": "2023-03-07T10:52:14Z",
  "userName": "integration-test-admin"
},
{
  "asset": {
    "id": "640717485b243e32c4b048f9",
    "name": "Country ISO 3166-1",
    "assetType": "HIERARCHY"
  },
  "modificationType": "UPDATE",
  "eventTime": "2023-03-07T10:51:53Z",
  "userName": "mdm-rdm-service"
},
{
  "asset": {
    "id": "640717485b243e32c4b048f9",
    "name": "Country ISO 3166-1",
    "assetType": "HIERARCHY"
  },
  "modificationType": "CREATE",
```

```
        "fieldChanges": {
          "name": {
            "newValue": "Country ISO 3166-1"
          }
        },
        "eventTime": "2023-03-07T10:51:52Z",
        "userName": "integration-test-admin"
      }
    ]
}
```

Get history of hierarchy relationships by time range (v2)

Retrieves all the change events of hierarchy relationships for a specific time range based on the internal ID or alias.

GET request

To retrieve all the change events of hierarchy relationships for a specific time range based on an internal ID or alias, submit a GET request with the following URI:

```
/rdm-service/external/v2/hierarchies/{assetKey}/mappings/audit
```

GET request query parameters

You can append query parameters to the URI to specify the time range, page number, page size, and modification type.

The following table describes the query parameters:

Parameter	Description
assetKey	Internal ID or alias of the hierarchy.
from	Start date and time of the time range, which is inclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-12T14:04:04Z`.
to	End date and time of the time range, which is exclusive. Time range must be in the following ISO-8601 format: yyyy-MM-dd'T'HH:mm:ss'Z'. For example, you might use `2019-12-15T14:04:04Z`.
page	Optional. Page number to display. Default is `0`.
pageSize	Optional. Number of records to display per page. Default value is `100`. Maximum value is `10000`.
modificationType	Optional. Type of modification. Value can be CREATE, UPDATE, or DELETE.

GET response

The response contains the change events of hierarchy relationships for a specific time range.

The following table describes the attributes in the response body:

Attribute	Type	Description
pageSize	Number	Number of records displayed per page.
page	Number	Page from which the response was retrieved.
numberOfElements	Number	Number of records returned in the current page.
lastPage	Boolean	Indicates whether the current page is the last page in the total result.
content	Array	Includes the list of modification events.
asset	Object	Includes details about the hierarchy.
id	String	ID of the hierarchy. For more information, see "Asset IDs" on page 199.
internalId	String	Unique identifier of the hierarchy.
alias	String	Alternative identifier of the hierarchy.
assetType	String	Type of the asset. The value is `hierarchy`.
modificationType	String	Type of change made to the hierarchy. Value can be CREATE, UPDATE, or DELETE.
from	Object	Includes details of the previous and new values of the child node in the relationship.
to	Object	Displays the previous and new values of the parent node in the relationship.
eventTime	String	Date and time when the hierarchy was last updated.
userName	String	User name of the user who initiated the modification.

GET example

To retrieve all the change events of a hierarchy for a specific time range based on the internal ID or alias, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v2/hierarchies/
hr_hierarchy/mappings/audit?from=2023-01-01T00:00:00Z&to=2024-01-01T00:00:00Z HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the change events of the hierarchy from 2023-01-01T00:00:00Z to 2024-01-01T00:00:00Z:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 1929

{
  "pageSize": 100,
  "page": 0,
  "numberOfElements": 6,
  "lastPage": true,
  "firstPage": true,
  "content": [
    {
      "asset": {
        "id": "640717485b243e32c4b048f9",
        "internalId": "hr_hierarchy",
        "alias": "als_hr_hierarchy",
        "assetType": "HIERARCHY"
      },
      "modificationType": "CREATE",
      "from": {
        "newValue": "Antarctica"
      },
      "to": {
        "newValue": "Antarctica"
      },
      "eventTime": "2023-03-07T10:53:00Z",
      "userName": "integration-test-admin"
    },
    {
      "asset": {
        "id": "640717485b243e32c4b048f9",
        "internalId": "hr_hierarchy",
        "alias": "als_hr_hierarchy",
        "assetType": "HIERARCHY"
      },
      "modificationType": "CREATE",
      "from": {
        "newValue": "Albania"
      },
      "to": {
        "newValue": "Albania"
      },
      "eventTime": "2023-03-07T10:52:52Z",
      "userName": "integration-test-admin"
    },
    {
      "asset": {
        "id": "640717485b243e32c4b048f9",
        "internalId": "hr_hierarchy",
        "alias": "als_hr_hierarchy",
        "assetType": "HIERARCHY"
      },
      "modificationType": "CREATE",
      "from": {
        "newValue": "Afghanistan"
      },
      "to": {
        "newValue": "Afghanistan"
      },
      "eventTime": "2023-03-07T10:52:44Z",
      "userName": "integration-test-admin"
    },
    {
      "asset": {
        "id": "640717485b243e32c4b048f9",
        "internalId": "hr_hierarchy",
        "alias": "als_hr_hierarchy",
        "assetType": "HIERARCHY"
```

```
      },
      "modificationType": "CREATE",
      "from": {
        "newValue": "Antarctica"
      },
      "eventTime": "2023-03-07T10:52:35Z",
      "userName": "integration-test-admin"
    },
    {
      "asset": {
        "id": "640717485b243e32c4b048f9",
        "internalId": "hr_hierarchy",
        "alias": "als_hr_hierarchy",
        "assetType": "HIERARCHY"
      },
      "modificationType": "CREATE",
      "from": {
        "newValue": "Albania"
      },
      "eventTime": "2023-03-07T10:52:34Z",
      "userName": "integration-test-admin"
    },
    {
      "asset": {
        "id": "640717485b243e32c4b048f9",
        "internalId": "hr_hierarchy",
        "alias": "als_hr_hierarchy",
        "assetType": "HIERARCHY"
      },
      "modificationType": "CREATE",
      "from": {
        "newValue": "Afghanistan"
      },
      "eventTime": "2023-03-07T10:52:34Z",
      "userName": "integration-test-admin"
    }
  ]
}
```


external localization

Use this resource to export a list of languages as code values from the Localization Languages code list, and import them into it. This list of languages represents the languages your organization supports for localization.

The predefined Localization Languages code list identifies each language by a BCP 47 language code and specifies the default language for your organization.

You can export the list of language code values fom the Localization Languages code list in a development organization and then import it into the Localization Languages code list in a production organization. You can also view the details of code values that were skipped in the import language code values report.

Preview Notice: Effective in the October 2026 release, the ability to manage multilingual assets is available for preview.

Preview functionality is supported for evaluation purposes but is unwarranted and is not supported in production environments or any environment that you plan to push to production. Informatica intends to include the preview functionality in an upcoming release for production use, but might choose not to in accordance with changing market or technical circumstances. For more information, contact Informatica Global Customer Support.

Export localization languages

Exports a list of languages as code values from the Localization Languages code list.

GET request

To export the localization language code values, submit a GET request with the following URI:

```
/rdm-service/external/v1/localization/export
```

GET response

The response contains the details of the localization language code values.

The following table describes the attributes in the response body:

Field	Type	Description
defaultLanguage	String	Locale code of the default language for the organization, such as de-DE. If an attribute value isn't available in a preferred language, then Reference 360 uses the default language to display the attributes and their values.
content	Array	List of languages exported from the Localization Languages code list. Each language includes a language code and multilingual attributes for a code value.
codeValueFields	Object	Contains the attributes of a code value, its identifying code and the multilingual values of each attribute.
_localizedContent	Array	List of localized versions of the attributes of the code value.
_id	String	Locale code that identifies a language, such as pt-BR.
Name	String	Localized display name of the language.
_language	Object	Contains the details of the language code value.
Code	String	Locale code of the language code value, such as de-DE.
targetLanguages	Array	List of locale codes of the additional languages in which the organization stores multilingual content, such as en-US, de-DE, and pt-BR.

GET example

To get the details of the localization language code values to be exported, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/localization/
export HTTP/1.1
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the details of the localization language code values exported:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "defaultLanguage": "de-DE",
  "content": [
    {
      "codeValueFields": {
        "_localizedContent": [
          {
```

```
          "_id": "de-DE",
          "Name": "Englisch US",
          "_language": {
            "Code": "de-DE"
          }
        },
        {
          "_id": "pt-BR",
          "Name": "Inglês dos EUA",
          "_language": {
            "Code": "pt-BR"
          }
        },
        {
          "_id": "en-US",
          "Name": "English United States",
          "_language": {
            "Code": "en-US"
          }
        }
      ],
      "Code": "en-US"
    }
  },
  {
    "codeValueFields": {
      "_localizedContent": [
        {
          "_id": "de-DE",
          "Name": "Deutsch",
          "_language": {
            "Code": "de-DE"
          }
        },
        {
          "_id": "pt-BR",
          "Name": "Alemã",
          "_language": {
            "Code": "pt-BR"
          }
        },
        {
          "_id": "en-US",
          "Name": "German",
          "_language": {
            "Code": "en-US"
          }
        }
      ],
      "Code": "de-DE"
    }
  },
  {
    "codeValueFields": {
      "_localizedContent": [
        {
          "_id": "de-DE",
          "Name": "Portugiesisch",
          "_language": {
            "Code": "de-DE"
          }
        },
        {
          "_id": "pt-BR",
          "Name": "Português",
          "_language": {
            "Code": "pt-BR"
          }
        },
        {
          "_id": "en-US",
```

```
            "Name": "Portuguese",
            "_language": {
              "Code": "en-US"
            }
          }
        ],
        "Code": "pt-BR"
      }
    }
  ],
  "targetLanguages": [
    "en-US",
    "de-DE",
    "pt-BR"
  ]
}
```

Import localization languages

Imports language code values into the Localization Languages code list.

POST request

To import localization language code values into the Localization Languages code list, submit a POST request with the following URI:

```
/rdm-service/external/v1/localization/import
```

The following table describes the parameter in the request body:

Field	Type	Description
defaultLanguage	String	Locale code of the default language for the organization, such as de-DE. If an attribute value isn't available in a preferred language, then Reference 360 uses the default language to display the attributes and their values.
content	Array	List of languages to import into the Localization Languages code list. Each language includes a language code and multilingual attributes for a code value.
codeValueFields	Object	Contains the attributes of a code value, its identifying code and the multilingual values of each attribute.
_localizedContent	Array	List of localized versions of the attributes of the code value.
_id	String	Locale code that identifies a language, such as pt-BR.
Name	String	Localized display name of the language.
_language	Object	Contains the details of the language code value.
Code	String	Locale code of the language code value, such as de-DE.
targetLanguages	Array	List of locale codes of the additional languages in which the organization stores multilingual content, such as en-US, de-DE, and pt-BR.

POST response

The response contains the details of the multilingual code values that were imported.

The following table describes the attributes in the response body:

Field	Type	Description
createdCodeValueIds	Array	IDs of the code value for which multilingual values was created.
updatedCodeValueIds	Array	IDs of the code value for which existing multilingual values was updated.
excludedAndFailedCodeValueIds	Object	IDs of code value that were excluded or failed to import
codeValueId	Array	List of errors that caused the code value to be excluded or to fail.
errorCode	String	Code to identify the error.
errorSummary	String	A short summary that explains why the code value was excluded or failed.
errorParameter	String	Parameter the provides details of an error.
fieldname	String	Display attribute value for the code value that was excluded or failed.

POST example

To import code values, you might use the following request:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/localization/
import HTTP/1.1
Content-Type: application/json;charset=UTF-8
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "defaultLanguage": "de-DE",
  "content": [
    {
      "codeValueFields": {
        "_localizedContent": [
          {
            "_id": "de-DE",
            "Name": "Englisch US",
            "_language": {
              "Code": "de-DE"
            }
          },
          {
            "_id": "pt-BR",
            "Name": "Inglês dos EUA",
            "_language": {
              "Code": "pt-BR"
            }
          },
          {
            "_id": "en-US",
            "Name": "English United States",
            "_language": {
              "Code": "en-US"
            }
          }
        ],
        "Code": "en-US"
      }
    },
    {
      "codeValueFields": {
        "_localizedContent": [
          {
```

```
            "_id": "de-DE",
            "Name": "Deutsch",
            "_language": {
              "Code": "de-DE"
            }
          },
          {
            "_id": "pt-BR",
            "Name": "Alemã",
            "_language": {
              "Code": "pt-BR"
            }
          },
          {
            "_id": "en-US",
            "Name": "German",
            "_language": {
              "Code": "en-US"
            }
          }
        ],
        "Code": "de-DE"
      }
    },
    {
      "codeValueFields": {
        "_localizedContent": [
          {
            "_id": "de-DE",
            "Name": "Portugiesisch",
            "_language": {
              "Code": "de-DE"
            }
          },
          {
            "_id": "pt-BR",
            "Name": "Português",
            "_language": {
              "Code": "pt-BR"
            }
          },
          {
            "_id": "en-US",
            "Name": "Portuguese",
            "_language": {
              "Code": "en-US"
            }
          }
        ],
        "Code": "pt-BR"
      }
    }
  ],
  "targetLanguages": [
    "en-US",
    "de-DE",
    "pt-BR"
  ]
}
```

The following sample response shows the details of multilingual code values that were imported:

```
{
  "createdCodeValueIds": [
    "68417a0b8fb05a12ea3448ed",
    "68417a0b8fb05a12ea3448ee",
    "68417a0b8fb05a12ea3448ef"
  ],
  "updatedCodeValueIds": [
    "68417a0b8fb05a12ea34559d",
    "68417a0b8fb05a12ea3559ee",
```

```
    "68417a0b8fb05a12ea35598ef"
  ],
  "excludedAndFailedCodeValueIds": {
    "68417a0c8fb05a12ea344900": [
      {
        "errorCode": "RDM.0010001",
        "errorSummary": "Mandatory field is missing or empty",
        "errorParameter": {
          "fieldname": "Name"
        }
      }
    ],
    "68471b52e47799288730f733": [
      {
        "errorCode": "RDM.0010001",
        "errorSummary": "Mandatory field is missing or empty",
        "errorParameter": {
          "fieldname": "Name"
        }
      }
    ]
  }
}
```

asset optimization modelling

Use this resource to enable asset optimization option on the user interface. You can use the asset optimization modelling REST API to optimize the existing code lists and hierarchies. You can also retrieve the status of an asset optimization modelling job.

Note: To use the asset optimization modelling resource, you must be assigned the Reference 360 Administrator role.

When the volume of reference data is high, use asset optimization once for your organization.

Note: The loading time and performance improvement might depend on your environment and network latency.

Enabling the asset optimization option is a one-time activity for an organization and you can't revert the optimization. After you optimize assets in your organization, you can't enable the multilingual property on any reference data assets in Reference 360.

Enable the asset optimization option

By default, the asset optimization option isn't available in the user interface. You must enable it by running the asset optimization modeling REST API. The REST API also analyzes the number of assets to be optimized and sets them up for optimization. When you run the asset optimization modeling REST API, it triggers an asset optimization modeling job.

Enabling the asset optimization option is a one-time activity for an organization and you can't revert the optimization.

POST request

To enable the asset optimization option and to run an asset optimization modeling job, submit a POST request with the following URI:

```
/rdm-service/external/v1/asset/optimization/modelling
```

POST response

The response is in the JSON format.

The following table describes the attributes in the response:

Field	Type	Description
jobId	String	ID of the asset optimization modeling job.
type	String	Type of the job, such as asset optimization modeling.
status	String	Status of the asset optimization modeling job. Value can be `RUNNING`, `SUCCESS`, `FAILED`, or `CANCELLED`.
createdBy	String	User name of the user who triggered the asset optimization modeling job.
createdDate	String	Date when the user ran the asset optimization modeling job.

POST example

To enable the asset optimization option, you might use the following request:

```
{
   "method":"POST",
   "url":"https://qa-pod1-mdm.mrel.infaqa.com/rdm-service/external/v1/asset/optimization/
modelling",
   "headers":{
      "accept":"application/json",
      "Session ID":"XXXXXXXXXXXXXXXXXXXXXX"
   },
   "data":""
}
```

The following sample response shows the details of an asset optimization modeling report:

```
{
  "jobId": "663a68d9ec6cff7ea71b48ed",
  "type": "ASSET_OPTIMIZATION_MODELLING",
  "status": "RUNNING",
  "createdBy": "cLuE2VONraVl6EjvWO9Hw5",
  "createdDate": "2024-05-07T17:46:01.768+00:00"
}
```

Get asset modeling job status

Retrieves the status of an asset optimization modeling job and the number of assets that the job processed.

GET request

To get the status of an asset optimization modeling job, submit a GET request with the following URI:

```
/rdm-service/external/v1/asset/optimization/modelling/job/{jobId}
```

GET response

The response contains the details of an asset optimization modeling job, such as status of the job, created date, and number of code lists and hierarchies that can be optimized.

The following table describes the attributes in the response:

Field	Type	Description
jobId	String	ID of the asset optimization modeling job.
type	String	Type of the job, such as asset optimization modeling.
status	String	Status of the asset optimization modeling job. Values can be `RUNNING`, `SUCCESS`, `FAILED`, or `CANCELLED`.
createdBy	String	User name of the user who triggered the asset optimization modeling job.
createdDate	String	Date when the user ran the asset optimization modeling job.
detail	-	Details of a job at a specific time.
numberOfCodelistModelled	Integer	Number of code lists that are ready to be optimized.
numberOfHierarchyModelled	Integer	Number of hierarchies that are ready to be optimized.
errors	-	Details of the error that the REST API request returns.
errorCode	String	Error code for the error type.
errorSummary	String	Message that explains why the invalid records couldn't be set up for optimization.
errorParameter	String	Parameter that provides details of an error.
errorCauses	String	Causes of the error to explain why the assets couldn't be set up for optimization.

GET example

To get the status of an asset optimization modeling job, you might use the following request:

```
{
    "method": "GET",
    "url": "https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/asset/
optimization/modelling/job/97d517fc22a78aea89646b1na",
    "protocol": "HTTP/1.1",
    "headers": {
        "IDS-SESSION-ID": "XXXXXXXXXXXXXXXXXXXXXX"
    }
}
```

The following sample response shows the status of an asset optimization modeling job:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 193

{
   "jobId":"663a68d9ec6cff7ea71b48ed",
   "type":"ASSET_OPTIMIZATION_MODELLING",
   "status":"SUCCESS",
   "createdBy":"cLuE2VONraVl6EjvWO9Hw5",
   "createdDate":"2024-05-07T17:46:01.768+00:00",
   "detail":{
      "numberOfCodelistModelled":0,
      "numberOfHierarchyModelled":0
```

```
    }
}
```

alias generation

Use this resource to generate aliases for existing predefined assets that you created before the April 2025 release based on their names. These assets don't have internal IDs configured, so you can use this resource to create aliases that function as alternative identifiers for these assets. The alias generation API runs an alias generation job. You can retrieve the status of the alias generation job.

When you migrate assets that don't have internal IDs, Reference 360 uses aliases to identify them. Additionally, generation of alias identifies existing predefined assets created before the April 2025 release that don't contain internal IDs.

You can't generate aliases for assets that are in the draft state. Before you run the alias generation REST API, ensure that you publish these assets.

To run the REST API, a user with the Reference Data Management Basic Edition license must be assigned the MDM Designer role.

To run the REST API, a user with the Reference Data Management full license must be assigned one of the following roles:

- Reference 360 Administrator.
- Reference 360 Primary Owner to update reference data sets and code lists.
- Reference 360 Business Steward to update crosswalks.
- Reference 360 Planner to update hierarchies.

Generate aliases for reference data assets

Generates aliases for all the existing predefined assets that don't have internal IDs by running an alias generation job.

POST request

To generate aliases for existing predefined assets, submit a POST request with the following URI:

```
/rdm-service/external/v1/assets/generateAliases?overwriteExisting=false
```

The following table describes the parameter in the request:

Parameter	Type	Description
overwriteExisting	Boolean	Optional. Indicates whether the job overwrites existing alias values. Set to `false` to skip overwriting the existing alias values and to generate alias only for predefined assets with no internal ID. Set to `true` to overwrite the existing alias values and to generate alias only for predefined assets with no internal ID. Default is `false`.

POST response

The response is in the JSON format.

The following table describes the attributes in the response body:

Field	Type	Description
jobId	String	ID of the alias generation job.
type	String	Type of the job. The supported value is `ALIAS_GENERATION`.
status	String	Status of the alias generation job. Value can be `RUNNING`, `SUCCESS`, `FAILED`, or `CANCELLED`.
createdBy	String	User name of the user who ran the alias generation job.
createdDate	String	Date when the user ran the alias generation job.

POST example

To generate aliases for existing predefined assets, you might use the following request:

```
{
   POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/assets/
generateAliases?overwriteExisting=false",
   HTTP/1.1
   Accept: application/json
   IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the details of an alias generation job:

```
{
  "jobId": "663a68d9ec6cff7ea71edcba",
  "type": "ALIAS_GENERATION",
  "status": "RUNNING",
  "createdBy": "cLuE2VONraVl6EjvWO9Hw5",
  "createdDate": "2024-10-23T13:00:44.702+00:00"
}
```

Get alias generation job status

Retrieves the status of an alias generation job and the list of assets that the job processed with their status.

GET request

To retrieve the details of an alias generation job, submit a GET request with the following URI:

```
/rdm-service/external/v1/assets/generateAliases/job/{jobId}
```

The following table describes the query parameter:

Field	Type	Description
jobId	String	ID of the alias generation job.

GET response

The response contains the details of an alias generation job, such as status of the job, created date, and array of assets for which aliases were generated.

The following table describes the attributes in the response body:

Field	Type	Description
jobId	String	ID of the alias generation job.
type	String	Type of the job. The supported value is `ALIAS_GENERATION`.
status	String	Status of the alias generation job. Values can be `RUNNING`, `SUCCESS`, `FAILED`, or `CANCELLED`.
createdBy	String	User name of the user who triggered the alias generation job.
createdDate	String	Date when the user ran the alias generation job.
detail	-	Details of the alias generation job at a specific time.
referenceDataSets	-	Includes alias generation details for reference data sets.
SUCCESS	Array	An array that lists the reference data sets for which aliases were generated.
FAILED	Array	An array that lists the reference data sets for which aliases weren't generated.
SKIPPED	Array	An array that lists the reference data sets for which aliases exist and the input overwriteExisting parameter was `false`.
codelists	-	Includes alias generation details for code lists.
SUCCESS	Array	An array that lists the code lists for which aliases were generated.
FAILED	Array	An array that lists the code lists for which aliases weren't generated.
SKIPPED	Array	An array that lists the code lists for which aliases exist and the input overwriteExisting parameter was `false`.
errors	-	Details of the error that the REST API request returns.
errorCode	String	Error code for the error type.
errorSummary	String	Message that explains why aliases weren't generated for the assets.
errorParameter	String	Parameter that provides details of an error.
errorCauses	String	Causes of the error to explain why aliases weren't generated for the assets.

GET example

To retrieve the status of an alias generation job, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/assets/
generateAliases/job/663a68d9ec6cff7ea71edcba
HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the status of an alias generation job:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 20
```

```
{
  "jobId": "663a68d9ec6cff7ea71edcba",
  "type": "ALIAS_GENERATION",
  "status": "SUCCESS",
  "createdBy": "cLuE2VONraVl6EjvWO9Hw5",
  "createdDate": "2024-10-23T13:00:44.702+00:00",
  "detail": {
    "referenceDataSets": {
      "SUCCESS": [
        "ZIT_17031562157755360_9505_ModelDeltaImportTest_IntTestRDS2"
      ],
      "FAILED": [
        "CaseTest1",
        "ABC RDA",
        "CaseTest",
        "aaaa",
        "RdsCheckCorruption"
      ],
      "SKIPPED": [
        "PaymentMethodType",
        "SupplierType",
        "Color",
        "EmploymentStatus",
        "AccountType",
        "PurchaseOrgCodeType",
        "QualificationType",
        "PersonStatus",
        "MeasuringUnit",
        "18355088358"
      ]
    },
    "codelists": {
      "FAILED": [
        "Enterprise ZIT_17183546558623610_4072_ReferenceHierarchyTest_IntTestRDS4",
        "Enterprise ZIT_17183546558623610_4072_ReferenceHierarchyTest_IntTestRDS3",
        "Enterprise ZIT_17183546558623610_4072_ReferenceHierarchyTest_IntTestRDS2",
        "Enterprise ZIT_17183546558623610_4072_ReferenceHierarchyTest_IntTestRDS1",
        "Enterprise AAAA",
        "Enterprise CaseTest1",
        "Dep2",
        "Enterprise Rds_master_dep_del",
        "Enterprise Country",
        "Alt Codelist - Unused",
        "Alt Codelist - With SrcSystem",
        "Enterprise CaseTest",
        "Enterprise Color"
      ],
      "SKIPPED": [
        "Enterprise PaymentMethodType",
        "Enterprise SupplierType",
        "Enterprise AddressUsageType",
        "Enterprise PhoneUsageType",
        "Enterprise EmploymentStatus",
        "Enterprise SocialHandleType",
        "Enterprise AccountType",
        "Enterprise Activités RH"
      ]
    },
    "crosswalks": {
      "SKIPPED": [
        "Enterprise Language to Enterprise Country",
        "Enterprise Test-Rds to Enterprise Test-Rds2",
        "Enterprise AccountType to Enterprise AddressEnrichedIndicator",
        "Enterprise AddressEnrichedIndicator to Enterprise AccountType",
        "Enterprise State to Enterprise Country",
        "Enterprise AccountType to Enterprise AddressStatus",
        "Enterprise Dep Area to Enterprise AddressUsageType",
        "Enterprise TestRDA to SAP RDA",
        "Enterprise A RDA Test to A RDA ORC",
        "Enterprise A RDA Test to A RDA SAP",
        "Enterprise PermTest to PermTest2",
```

```
            "Enterprise RDA-XYZ to XYZ",
            "Enterprise FakeChangelistTest to Codelist21",
            "Enterprise ABC  to ABC Alt",
            "Enterprise ABC  to Alt Unused",
            "DataEZ Country to Enterprise Country",
            "IQVIA Country to Enterprise Country",
            "Salesforce Country to Enterprise Country",
            "Country_ORA_AP to Enterprise Country",
            "Enterprise Country to VeevaCRM Country",
            "Enterprise Country to DataEZ Country",
            "Enterprise AddressType to VeevaCRM AddressType",
            "Enterprise Country to IQVIA Country",
            "Enterprise AddressType to IQVIA Addresstype",
            "Enterprise Beta to ISO Beta",
            "Enterprise Beta to SAP Beta",
            "SAP Beta to Enterprise Beta",
            "Enterprise Country to ISO Country Codes",
            "PeopleSoft AddressType to Enterprise AddressType"
          ]
        },
        "hierarchies": {
          "SKIPPED": [
            "CountryLanguage",
            "LanguageCountry",
            "New hier",
            "CountryStatus",
            "LanguageStatus",
            "ColorHier",
            "ColorHier1",
            "Kite13"
          ]
        }
      }
    }
```

enums

Use this resource to list system reference data values and add system reference data values.

You can retrieve system reference data values or add values to the following system reference data:

- Application
- Confidentiality
- Domain
- Priority
- Status

Get system reference data values

Retrieves values of the system reference data.

GET request

To retrieve system reference data values, submit a GET request with the following URI:

```
/rdm-service/external/v1/enums
```

GET response

The response contains the values of all the system reference data.

The following table describes attributes in the response:

Field	Type	Description
key	String	ID of the system reference data value. In Reference 360, when you sort assets in the **Explore** panel by priority, the assets are sorted based on the key value. **Note:** When you import code values, you might want to provide additional information about the data. For example, you might want to assign a code value to the Approved status. You use the key value to assign the appropriate system reference data value.
label	String	Label for the system reference data value. The labels appear in Reference 360. In Reference 360, the values appear in alphanumeric order based on the label.

GET example

To retrieve the values of all the system reference data, you might use the following request:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/enums HTTP/1.1
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX
```

The following sample response shows the values for each system reference data:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 368

{
   "Priority":[
      {
         "key":"0",
         "label":"Low"
      },
      {
         "key":"1",
         "label":"Medium"
      },
      {
         "key":"2",
         "label":"High"
      },
      {
         "key":"9",
         "label":"Critical"
      }
   ],
   "Domain":[
      {
         "key":"0",
         "label":"Finance"
      },
      {
         "key":"1",
         "label":"Geography"
      },
      {
         "key":"2",
         "label":"Social"
      }
   ]
}
```

Add system reference data values

Adds values to the system reference data.

PATCH request

To add values to the system reference data, submit a PATCH request with the following URI:

```
/rdm-service/external/v1/enums
```

Use the following attributes in the request body to specify the new values:

Field	Type	Description
key	String	ID of the system reference data value. In Reference 360, when you sort assets in the **Explore** panel by priority, the assets are sorted based on the key value. **Note:** When you import code values, you might want to provide additional information about the data. For example, you might want to assign a code value to the Approved status. You use the key value to assign the appropriate system reference data value.
label	String	Label for the system reference data value. The labels appear in Reference 360. In Reference 360, the values appear in alphanumeric order based on the label.

Note: You can use the same value for the `key` and `label` parameters.

PATCH response

The response shows the number of values added.

The following table describes the attributes in the response:

Field	Type	Description
newEntries	Number	Number of values added.
existingEntries	Number	Number of existing values that were skipped.

PATCH example

To add values to the system reference data, you might use the following request:

```
PATCH https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/enums HTTP/1.1
Content-Type: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "Priority":[
    {
      "key":"0",
      "label":"Low"
    },
    {
      "key":"1",
      "label":"Medium"
    },
    {
      "key":"2",
      "label":"High"
    },
    {
      "key":"9",
```

```
        "label":"Critical"
      }
    ],
    "Domain":[
      {
        "key":"0",
        "label":"Finance"
      },
      {
        "key":"1",
        "label":"Geography"
      },
      {
        "key":"2",
        "label":"Social"
      }
    ]
}
```

The following sample response shows the number of values added:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 47

{
  "newEntries" : 5,
  "existingEntries" : 2
}
```

Update system reference data values

Updates the values of the system reference data.

PATCH request

To update the values of the system reference data, submit a PATCH request with the following URI:

```
/rdm-service/external/v1/enums
```

The following table describes the parameter in the request:

Parameter	Description
Overwrite	Optional. Indicates whether to update the existing system reference data values with same keys. Value is `true` or `false`. Default is `false`.

The following table describes the attributes in the request:

Field	Type	Description
key	String	ID of the system reference data value. In Reference 360, when you sort assets in the **Explore** panel by priority, the assets are sorted based on the key value. **Note:** When you import code values, you can provide additional information about the data. For example, you can assign a code value to the Approved status. You can use the key value to assign the appropriate system reference data value.
label	String	Label for the system reference data value. In Reference 360, the values appear in alphanumeric order based on the label.

PATCH response

The response shows the number of values updated.

The following table describes the attributes in the response:

Field	Type	Description
newEntries	Number	Number of values added.
existingEntries	Number	Number of existing values that are skipped.
updatedEntries	Number	Number of existing values that are updated.

PATCH example

To update values of the system reference data, you can use the following request:

```
PATCH https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/enums?
overwrite=true HTTP/1.1
Content-Type: application/json
Host: localhost:8080
Content-Length: 368

{
   "Priority":[
      {
         "key":"0",
         "label":"Low"
      },
      {
         "key":"1",
         "label":"Medium"
      },
      {
         "key":"2",
         "label":"High"
      },
      {
         "key":"9",
         "label":"Critical"
      }
   ],
   "Domain":[
      {
         "key":"0",
         "label":"Finance"
      },
      {
         "key":"1",
         "label":"Geography"
      },
      {
         "key":"2",
         "label":"Social"
      }
   ]
}
```

The following sample response shows the number of values updated:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 71

{
   "newEntries":1,
```

```
   "existingEntries":4,
   "updatedEntries":2
}
```

Delete system reference data values

Deletes values of the system reference data.

DELETE request

To delete a value of the system reference data, submit a DELETE request with the following URI:

```
/rdm-service/external/v1/enum/{enumType}/{enumKey}
```

Note: You can delete only one system reference data value at a time.

The following table describes the attributes in the request:

Field	Type	Description
enumType	String	Type of system reference data.
enumKey	String	ID of the system reference data value.

DELETE response

A 204 no content response is returned.

DELETE example

To delete a value of the system reference data, you might use the following request:

```
DELETE https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/enums/status/
statusKey HTTP/1.1
Host: localhost:8080
```

The following sample response shows the no content response:

```
HTTP/1.1 204 No Content
```

Using REST APIs to import and export

You can use the Reference 360 REST APIs to import or export code values and value mappings in bulk.

REST APIs for import and export

You can use a set of REST APIs to import or export data, retrieve the status of an import job, or retrieve a failed import job report.

The following table describes the REST APIs for importing and exporting data:

REST API	Description
Import code values	Import code values into a code list.
Import value mappings	Import value mappings into a crosswalk.
Get import job status	Retrieve the status of an import job.
Get failed import job report	Retrieve an error report for a failed import job.
Export code values	Export the code values in a code list.
Export value mappings	Export the value mappings in a crosswalk.

Filter criteria

You can filter the reference data that you want to export. For example, you can export a filtered set of code values in a code list.

When you filter code values, you can only filter values in the Code attribute or values in attributes that are configured as display attributes. For example, you might want a filter to include values with `001` in the Code attribute.

Field types

The filter operators available depends on the field type of the attribute.

The following table describes the filter operators supported for each field type:

Field Type	Supported Filter Operators	Filter Values
Boolean	_equals _notEquals _isEmpty	Boolean
Decimal or Integer	_equals _notEquals _isEmpty _greaterThan _greaterThanEquals _lessThan _lessThanEquals	Number

Field Type	Supported Filter Operators	Filter Values
String	_equals _notEquals _isEmpty _startsWith _endsWith _contains _notContains	Text
Date	_equals _notEquals _isEmpty _from, _to, _range	ISO 8601 date or date and time For example, `2019-12-24` or `1969-12-15T14:17:04Z`.
Reference Data	_equals _notEquals _isEmpty _in	Values in the Code attribute or values in the display attributes for the reference data.

Filter examples

To filter assets with text fields that are empty, you might use the following filter operator:

```
{
   "textField":{
      "_isEmpty":true
   }
}
```

To filter assets with boolean fields that are equal to true, you might use the following filter operator:

```
{
   "booleanField":{
      "_equals":true
   }
}
```

To filter assets with number fields that are greater than 1 and less than 2, you might use the following filter operators:

```
{
   "numberField":{
      "_greaterThan":1,
      "_lessThan":3
   }
}
```

To filter assets with date fields between specified dates, you might use the following filter operators:

```
{
   "dateField":{
      "_from":"2019-01-01",
      "_to":"2019-06-15"
   }
}
```

To filter assets with date fields for a time range based on a reference date or time, you might use the following filter operators:

```
{
   "dateField":{
```

```
      "_range":{
         "_months":6,
         "_reference":"2020-01-01"
      }
   }
}
```

To filter assets with reference data attribute fields, you might use the following filter operators:

```
{
   "referenceDataAttributeField.Name":{
      "_in":[
         "EUR",
         "USD"
      ]
   }
}
```

To use multiple field operators, you might use the `_and` or `_or` operators like the following example:

```
{
  "_and": [
    {
      "_or": [
        {
          "name": {
            "_startsWith": "G"
          }
        },
        {
          "name": {
            "_endsWith": "g"
          }
        }
      ]
    },
    {
      "currencyLookup.Name": [
        "EUR"
      ]
    },
    {
      "founded": {
        "_to": "1951-08-29"
      }
    },
    {
      "population": {
        "_greaterThan": 8e7
      }
    },
    {
      "monarchic": false
    }
  ]
}
```

When you use a comma to separate operators inside a field, the comma acts like an `_and` operator. For example, the following examples filters for a name that starts with "Ger" and ends with "many", or equals "Japan":

```
{
  "_or": [
    {
      "name": {
        "_startsWith": "Ger","_endsWith": "many"
      }
    },
    {
      "name": {
        "_equals": "Japan"
```

```
        }
      }
    ]
}
```

Related Topics:

- "Exporting filtered code values" on page 542

Importing code values

You can import code values into a code list. After you start an import job, you can check the status of the import job. If the import job fails, you can retrieve an error report.

Before you begin, you must get a session ID and identify the asset ID. The session ID authenticates your requests. The asset ID is the ID of the code list to which you want to import code values. For more information, see "Session IDs" on page 198 and "Asset IDs" on page 199.

1. To import code values into a code list, use the Import code values REST API.

   For more information about the Import code values REST API, see "Import code values (v1)" on page 259.

   For example, the following request imports code values:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/import
HTTP/1.1
Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=file; filename=import-code-values.csv


--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=importSettings
Content-Type: application/json;charset=UTF-8

{
  "delimiter":"COMMA",
  "textQualifier":"DOUBLE_QUOTE",
  "codepage":"UTF8",
  "dateFormat":"ISO",
  "containerType":"CODELIST",
  "containerId":"9ab3201990a54dcdc86f54cf",
  "startingRow":null
}
--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
```

   The CSV file might contain the following header rows and data rows:

```
Name,Code
Name,Code
Afghanistan,AFG
Aland Islands,ALA
Albania,ALB
Algeria,DZA
American Samoa,ASM
```

   Note: The `containerId` attribute is the ID of the code list to which you want to import code values.

   For example, the Import code values REST API returns the following job ID and details about the import job:

```
{
  "jobId":"dd1b2018cb47cef99f8d0f42",
  "state":"INPROGRESS",
  "startTime":1561367377428,
```

```
  "numOfRecordsProcessed":100,
  "numOfRecordsFailed":25,
  "numOfRecordsSucceeded":75
}
```

2. To check the status of an import job, use the Get import job status REST API.

   For more information about the Get import job status REST API, see "Get import job status (v1)" on page 266.

   For example, the following request retrieves the status of an import job:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/import/job/
dd1b2018cb47cef99f8d0f42 HTTP/1.1
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

   For example, the Get import job status REST API returns the following status of the import job:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 193
{
  "jobId":"dd1b2018cb47cef99f8d0f42",
  "state":"INPROGRESS",
  "startTime":1561367376330,
  "numOfRecordsProcessed":100,
  "numOfRecordsFailed":25,
  "numOfRecordsSucceeded":75
}
```

3. To retrieve an error report for a failed import job, use the Get failed import job report REST API.

   For more information about the Get failed import job report REST API, see "Get failed import job report (v1)" on page 267.

   For example, the following request retrieves the error report for a failed import job:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/import/job/
dd1b2018cb47cef99f8d0f42/errorDetails HTTP/1.1
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

   For example, the Get failed import job report REST API returns the following details of the failed import job:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 354

{
  "jobId":"dd1b2018cb47cef99f8d0f42",
  "entityType":"Relationship",
  "fileName":"import.csv",
  "entityName":"rdm.crosswalk.rel.21ffd6b5f92d10c744acc27c.fc66c441288cf898c6fe5023",
  "errorDetails":[
    {
      "lineNumber":1,
      "entitySourcePkey":"AF_AFG",
      "reasons":[
        "The requested resource with ID 'AFG' does not exist."
      ]
    }
  ]
}
```

Importing value mappings

You can import value mappings into a crosswalk. After you start an import job, you can check the status of the import job. If the import job fails, you can retrieve an error report.

Before you begin, you must get a session ID and identify the asset ID. The session ID authenticates your requests. The asset ID is the ID of the crosswalk to which you want to import value mappings. For more information, see "Session IDs" on page 198 and "Asset IDs" on page 199.

1. To import value mappings into a crosswalk, use the Import value mappings REST API.

   For more information about the Import value mappings REST API, see "Import value mappings (v1)" on page 261.

   For example, the following request imports value mappings:

   ```
   POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/import
   HTTP/1.1
   Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
   IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

   --6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
   Content-Disposition: form-data; name=file; filename=import-value-mappings.csv

   --6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
   Content-Disposition: form-data; name=importSettings
   Content-Type: application/json;charset=UTF-8

   {
     "delimiter":"COMMA",
     "textQualifier":"DOUBLE_QUOTE",
     "codepage":"UTF8",
     "dateFormat":"ISO",
     "containerType":"CROSSWALK",
     "containerId":"9ab3201990a54dcdc86f53AB",
     "startingRow":null
   }
   --6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
   ```

   The CSV file might contain the following header rows and data rows:

   ```
   sourcePKey,_from.id.sourcePKey,_to.id.sourcePKey
   sourcePKey,_from.id.sourcePKey,_to.id.sourcePKey
   AF_AFG,AF,AFG
   AL_ALA,AL,ALA
   ALB_ALB,ALB,ALB
   DZ_DZA,DZ,DZA
   AS_ASM,AS,ASM
   ```

   Note: The `containerId` attribute is the ID of the code list to which you want to import value mappings.

   For example, the Import value mappings REST API returns the following job ID and import job information:

   ```
   {
     "jobId":"dd1b2018cb47cef99f8d0f43",
     "state":"INPROGRESS",
     "startTime":1561367377428,
     "numOfRecordsProcessed":100,
     "numOfRecordsFailed":25,
     "numOfRecordsSucceeded":75
   }
   ```

   Note: You use the job ID to check the status of an import job.

2. To check the status of an import job, use the Get import job status REST API.

   For more information about the Get import job status REST API, see "Get import job status (v1)" on page 266.

For example, the following request retrieves the status of an import job:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/import/job/
dd1b2018cb47cef99f8d0f43 HTTP/1.1
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX
```

For example, the Get import job status REST API returns the following status of the import job:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 193
{
  "jobId":"dd1b2018cb47cef99f8d0f43",
  "state":"INPROGRESS",
  "startTime":1561367376330,
  "numOfRecordsProcessed":100,
  "numOfRecordsFailed":25,
  "numOfRecordsSucceeded":75
}
```

3. To retrieve an error report for a failed import job, use the Get error report for failed import job REST API.

   For more information about the Get failed import job report REST API, see "Get failed import job report (v1)" on page 267.

   For example, the following request retrieves the error report for a failed import job:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/import/job/
dd1b2018cb47cef99f8d0f43/errorDetails HTTP/1.1
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX
```

   For example, the Get error report for failed import job REST API returns the following error report:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 354

{
  "jobId":"dd1b2018cb47cef99f8d0f43",
  "entityType":"Relationship",
  "fileName":"import.csv",
  "entityName":"rdm.crosswalk.rel.21ffd6b5f92d10c744acc27c.fc66c441288cf898c6fe5023",
  "errorDetails":[
    {
      "lineNumber":1,
      "entitySourcePkey":"AF_AFG",
      "reasons":[
        "The requested resource with ID 'AFG' does not exist."
      ]
    }
  ]
}
```

Exporting code values

Export code values in a code list.

Before you begin, you must get a session ID and identify the asset ID. The session ID authenticates your requests. The asset ID is the ID of the code list that contains the code values that you want to export. For more information, see "Session IDs" on page 198 and "Asset IDs" on page 199.

- To export code values in a code list, use the Export code values REST API.

  For more information about the Export code values REST API, see "Exporting code values" on page 541.

  For example, the following request exports code values:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/export
HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
```

```
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX

{
  "delimiter" : "SEMICOLON",
  "codepage" : "UTF8",
  "decimalSeparator" : "COMMA",
  "thousandSeparator" : "DOT",
  "dateFormat" : "ISO",
  "filename" : "testdata.csv",
  "containerType" : "codelist",
  "containerId" : "1989aae96bdaa4c2b8768fcc"
}
```

Note: The `containerId` attribute is the ID of the code list that contains the code values that you want to export.

For example, the Export code values REST API exports the following CSV file with the data:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
Content-Length: 124

status.key;effectiveDate;approvedOn
status.status.key;effectiveDate;approvedOn
ActiveStatus;myEffectiveDate;myApprovedOn
ActiveStatus;myEffectiveDate;myApprovedOn
```

Exporting filtered code values

Export filtered code values based on filter criteria. You can filter code values based on values in attributes or reference data attributes.

Before you begin, you must get a session ID and identify the asset ID. The session ID authenticates your requests. The asset ID is the ID of the code list that contains the code values that you want to export. For more information, see "Session IDs" on page 198 and "Asset IDs" on page 199.

1. To export code values that contain a status, use the Export code values API with a filter operator for the status field.

   For more information about the Export code values REST API, see "Exporting code values" on page 541.

   For example, the following request exports code values with any status:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/export
HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXX

{
  "delimiter":"SEMICOLON",
  "codepage":"UTF8",
  "decimalSeparator":"COMMA",
  "thousandSeparator":"DOT",
  "dateFormat":"ISO",
  "filename":"testdata.csv",
  "containerType":"codelist",
  "containerId":"1989aae96bdaa4c2b8768fcc",
  "filter":{
    "_and":[
      {
        "Status":{
          "_isEmpty":false
        }
      }
    ]
  }
}
```

Note: The `containerId` attribute is the ID of the code list that contains the code values that you want to export.

For example, the Export code values REST API exports the following CSV file with the data:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
Content-Length: 124

status.key;effectiveDate;Name;Code;Description
status.status.key;effectiveDate;Name;Code;Description
Active;;US;001;United States of America
Active;;CAN;002;Canada
```

2. To export filtered code values based on a display attribute for a reference data attribute and an attribute, use the Export code values API with multiple filter operators.

   For more information about the Export code values REST API, see "Exporting code values" on page 541.

   For example, the following request exports filtered code values that contain `Dollar` in the Name display attribute for the Currency reference data attribute and `00` in the Code attribute:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/export
HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "delimiter":"SEMICOLON",
  "codepage":"UTF8",
  "decimalSeparator":"COMMA",
  "thousandSeparator":"DOT",
  "dateFormat":"ISO",
  "filename":"testdata.csv",
  "containerType":"codelist",
  "containerId":"1989aae96bdaa4c2b8768fcc",
  "filter":{
    "_and":[
      {
        "Currency.Name":{
          "_contains":"Dollar"
        }
      },
      {
        "Code":{
          "_contains":"00"
        }
      }
    ]
  }
}
```

   Note: The `containerId` attribute is the ID of the code list that contains the code values that you want to export.

   For example, the Export code values REST API exports the following CSV file with the data:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
Content-Length: 124

status.key;effectiveDate;Name;Code;Description;Currency.Code
status.status.key;effectiveDate;Name;Code;Currency.Currency.Code
Active;;US;001;United States of America;USD
Active;;CAN;002;Canada;CAD
```

   Note: When you filter on a specific display attribute for a reference data attribute, the filtered code values appear in the CSV file, but the code value is represented by the Code attribute.

3. To export filtered code values based on a reference data attribute and an attribute, use the Export code values API with multiple filter operators.

   For more information about the Export code values REST API, see "Exporting code values" on page 541.

   For example, the following request exports filtered code values with `EUR` in the Currency reference data attribute and `an` in the Name attribute:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/export
HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "delimiter":"SEMICOLON",
  "codepage":"UTF8",
  "decimalSeparator":"COMMA",
  "thousandSeparator":"DOT",
  "dateFormat":"ISO",
  "filename":"testdata.csv",
  "containerType":"codelist",
  "containerId":"1989aae96bdaa4c2b8768fcc",
  "filter":{
    "_and":[
      {
        "Currency":"EUR"
      },
      {
        "Name":{
          "_contains":"an"
        }
      }
    ]
  }
}
```

   Note: The `containerId` attribute is the ID of the code list that contains the code values that you want to export.

   For example, the Export code values REST API exports the following CSV file with the data:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
Content-Length: 124

status.key;effectiveDate;Name;Code;Description;Currency.Code
status.status.key;effectiveDate;Name;Code;Description;Currency.Code
;;Netherlands;NLD;Netherlands;;EUR
;;Germany;DEU;Germany;;EUR
;;Ireland;IRL;Ireland;;EUR
;;Finland;FIN;Finland;;EUR
```

   Note: When you filter on values in a reference data attribute without specifying a display attribute to filter on, the filter applies on values in the Code attribute.

Related Topics:

- "Filter criteria" on page 535
- "Display attributes" on page 31
- "Reference data attributes" on page 28

Exporting value mappings

Export value mappings in a crosswalk.

Before you begin, you must get a session ID and identify the asset ID. The session ID authenticates your requests. The asset ID is the ID of the crosswalk that contains the value mappings that you want to export. For more information, see "Session IDs" on page 198 and "Asset IDs" on page 199.

- To export value mappings in a crosswalk, use the Export value mappings REST API.

  For more information about the Import value mappings REST API, see "Export value mappings to a CSV file (v1)" on page 293.

  For example, the following request exports value mappings:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/export
HTTP/1.1
Content-Type: application/json
Accept: application/octet-stream
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

{
  "delimiter" : "SEMICOLON",
  "codepage" : "UTF8",
  "decimalSeparator" : "COMMA",
  "thousandSeparator" : "DOT",
  "dateFormat" : "ISO",
  "filename" : "testdata.csv",
  "containerType" : "crosswalk",
  "containerId" : "5d123f6e4077c700010d59e4"
}
```

  Note: The `containerId` attribute is the ID of the crosswalk that contains the value mappings that you want to export.

  For example, the Export value mappings REST API exports the following CSV file with the data:

```
HTTP/1.1 200 OK
Content-Disposition: attachment;filename=testdata.csv
Content-Type: application/octet-stream
Content-Length: 124

status.key;effectiveDate;approvedOn
status.status.key;effectiveDate;approvedOn
ActiveStatus;myEffectiveDate;myApprovedOn
ActiveStatus;myEffectiveDate;myApprovedOn
```

Using REST APIs to manage hierarchies

You can use the Reference 360 REST APIs to manage hierarchies.

REST APIs to manage hierarchies

You can use a set of REST APIs to list hierarchies, list hierarchy details, list hierarchy model relationships, and import hierarchy relationships.

The following table describes the REST APIs for managing hierarchies:

REST API	Description
List hierarchies	Retrieves all hierarchies.
List hierarchy details	Retrieves the details of a hierarchy, such as the properties and status.
List hierarchy model relationships	Retrieves the relationships in a hierarchy model.
List hierarchy relationships	Imports top-level code values and relationships into a hierarchy.

Managing hierarchies

You can create hierarchies to show hierarchical relationships between code values in multiple code lists. A hierarchy consists of two components: the hierarchy model and the hierarchy tree. In the hierarchy model, you define the top-level code list and add relationships to other code lists. Then based on the hierarchy model, you can create the hierarchy tree and define relationships between the code values in the code lists.

For example, you might create a location hierarchy. First, you define the hierarchy model. You define the Region code list as the top-level code list. Then you create a parent-child relationship from the Region code list to the Enterprise Country Codes code list. Based on this hierarchy model, in the hierarchy, you create a hierarchy relationship from the North America code value to the United States code value. You create a hierarchy relationship from the North America code value to the Canada code value.

To create and manage hierarchies, perform the following actions:

1. In Reference 360, create the hierarchy asset and define the hierarchy model. For more information, see "Creating hierarchy models" on page 159.
2. Use the List hierarchies REST API to retrieve all hierarchies.
3. Use the List hierarchy model relationships REST API to retrieve the relationships in a hierarchy model.
4. Use the Import hierarchy relationships REST API to import relationships in a hierarchy tree.

Step 1. List hierarchies

You can retrieve all hierarchies in Reference 360.

Before you begin, you must get a session ID. The session ID authenticates your requests. For more information, see "Session IDs" on page 198.

1. To retrieve all hierarchies, use the List hierarchies REST API.

   For more information about the List hierarchies REST API, see "Get hierarchies (v1)" on page 495.

   For example, the following request retrieves all hierarchies:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/hierarchies
HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

For example, the List hierarchies REST API returns the following hierarchies:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 452
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

[
  {
    "id":"6e7dd28fc13b417c5c19d1fb",
    "name":"Cost centers",
    "description":"Cost center hierarchy",
    "version":"v1",
    "domain":"International standards",
    "confidentiality":"private",
    "priority":"Prio1",
    "status":"Draft",
    "effectiveDate":"2017-04-01",
    "approvedOn":"2017-03-01"
  },
  {
    "id":"e3e57e39ea8623f258013c43",
    "name":"Profit centers",
    "description":"Profit center hierarchy",
    "version":"v1"
  }
]
```

Note: The `id` attribute is the ID of the hierarchy, which you require for other hierarchy REST APIs.

2. Optionally, to retrieve the details of a hierarchy, use the List hierarchy details REST API.

   For more information about the List hierarchy details REST API, see "Get hierarchy details (v1)" on page 496.

   For example, the following request retrieves the details of a hierarchy:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/
hierarchies/6e7dd28fc13b417c5c19d1fb HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

   For example, the List hierarchy details REST API returns the following hierarchy details:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 334

{
  "id":"6e7dd28fc13b417c5c19d1fb",
  "name":"Cost centers",
  "description":"Cost center hierarchy",
  "version":"v1",
  "application":"App1",
  "domain":"Internal standards",
  "confidentiality":"private",
  "priority":"Prio1",
  "status":"Draft",
  "effectiveDate":"2017-04-01",
  "approvedOn":"2020-03-01"
}
```

Step 2. List hierarchy model relationships

You can retrieve the top-level code list and the relationships between code lists in a hierarchy model. Then, based on the hierarchy model, you can import hierarchy relationships between code values in the code lists.

- To retrieve the relationships in a hierarchy model, use the List hierarchy model relationships REST API.

For more information about the List hierarchy model relationships REST API, see "Get hierarchy model relationships (v1)" on page 497.

For example, the following request retrieves the relationships in a hierarchy model:

```
GET https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/
hierarchies/6e7dd28fc13b417c5c19d1fb/relations HTTP/1.1
Accept: application/json
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX
```

For example, the List hierarchy model relationship REST API returns the following relationships:

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Length: 452

{
  "relations":[
    {
      "child":{
        "codeListId":"8bc955e614df2040968e9d85",
        "codeListName":"Parent Codelist",
        "termId":"28a1320fe7f63cd25b58bef4",
        "termName":"Reference Data Set"
      }
    },
    {
      "parent":{
        "codeListId":"8bc955e614df2040968e9d85",
        "codeListName":"Parent Codelist",
        "termId":"28a1320fe7f63cd25b58bef4",
        "termName":"Reference Data Set"
      },
      "child":{
        "codeListId":"81a714a66863f954a9b60045",
        "codeListName":"First Level Codelist",
        "termId":"28a1320fe7f63cd25b58bef4",
        "termName":"Reference Data Set"
      }
    }
  ]
}
```

The `child.codeListId` attribute contains the top-level node relationship. For example, the first relationship in the example is the top-level node relationship.

Step 3. Import hierarchy relationships

You can import top-level code values and parent-child relationships into a hierarchy. For example, in a locations hierarchy, you might define the North America code value as a top-level code value. Then you define a relationship from the North America code value to the United States code value. You might also define a relationship from the North America code value to the Canada code value.

1. To import top-level code values into a hierarchy, use the Import hierarchy relationships REST API to specify the CSV file that contains the code values.

   For more information about the Import hierarchy relationships REST API, see "Step 3. Import hierarchy relationships" on page 548.

   For example, the following request imports top-level code values:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/import/
hierarchy HTTP/1.1
Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=file; filename=import.csv
```

```
Content-Type: text/plain

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=importSettings
Content-Type: application/json;charset=UTF-8

{
  "delimiter":"COMMA",
  "textQualifier":"DOUBLE_QUOTE",
  "startingRow":0,
  "codepage":"UTF8",
  "hierarchyId":"c79ab91c19b13b11d8d43770",
  "childCodeListId":"96f06071e4aaea81ff203abe"
}
--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
```

The CSV file might contain the following header row and data rows:

```
Code
P1
P2
P3
```

For example, the Import hierarchy relationships REST API returns the following response:

```
{
  "jobId":"73580d323feb170be5ec0fd5",
  "state":"INPROGRESS",
  "startTime":1603092643055,
  "numOfRecordsProcessed":100,
  "numOfRecordsFailed":25,
  "numOfRecordsSucceeded":75
}
```

2. To import relationships, use the Import hierarchy relationships REST API to specify the CSV file that contains the relationships.

   For more information about the Import hierarchy relationships REST API, see "Step 3. Import hierarchy relationships" on page 548.

   For example, the following request imports relationships:

```
POST https://use4-mdm.dm-us.informaticacloud.com/rdm-service/external/v1/import/
hierarchy HTTP/1.1
Content-Type: multipart/form-data; boundary=6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
IDS-SESSION-ID: XXXXXXXXXXXXXXXXXXXXXX

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=file; filename=import.csv
Content-Type: text/plain

--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm
Content-Disposition: form-data; name=importSettings
Content-Type: application/json;charset=UTF-8

{
  "delimiter":"COMMA",
  "textQualifier":"DOUBLE_QUOTE",
  "startingRow":0,
  "codepage":"UTF8",
  "hierarchyId":"c79ab91c19b13b11d8d43770",
  "childCodeListId":"96f06071e4aaea81ff203abe",
  "parentCodeListId":"9b300f793470882ca23e6091"
}
--6o2knFse3p53ty9dmcQvWAIx1zInP11uCfbm--
```

   The CSV file might contain the following header row and data rows:

```
Code,ParentCode
C1,P1
C2,P2
C3,P3
```

For example, the Import hierarchy relationships REST API returns the following response:

```
{
  "jobId":"73580d323feb170be5ec0fd5",
  "state":"INPROGRESS",
  "startTime":1603092643055,
  "numOfRecordsProcessed":100,
  "numOfRecordsFailed":25,
  "numOfRecordsSucceeded":75
}
```

CHAPTER 16

Glossary

attribute

A component of a code value. Code values have two or more attributes.

code list

A grouping of reference data that comes from the same application, industry standard list, or internal list. You organize code lists into reference data sets.

code value

A unique value, such as a business term. A code value is the lowest representation of reference data. You create code values in code lists.

crosswalk

A visual representation of a one-way relationship between code values in a pair of code lists. Crosswalks exist between a pair of code lists in the same reference data set.

dependent code list

A code list that contains code values that depend on code values from a code list in a different reference data set.

dependent reference data set

A reference data set that depends on code values in another reference data set. Code lists inherit the dependency from the reference data set.

hierarchical code lists

A code list that supports hierarchical data structures. You can arrange its code values into levels to create hierarchies.

hierarchical reference data set

A reference data set that supports hierarchical data structures. Code lists inherit the hierarchical structure from the reference data set. You can arrange code values in a hierarchical reference data set into levels to create hierarchies.

hierarchy

An arrangement of code values from multiple code lists below, above, or at the same level as other code values. The code values that you can arrange depend on the hierarchy model.

For example, you might have a hierarchy model with the Region code list defined as the top level node and a relationship from the Region code list to the Country code list. Then in the hierarchy, you can create a hierarchy relationship from the North America code value to the United States code value. You might also create a hierarchy relationship from the North America code value to the Canada code value.

hierarchy model

A model in which nodes are organized into a tree-like structure. A hierarchy model contains a top-level node, child nodes, a relationship from the top-level node to a child node, and relationships from child nodes to other child nodes. Each node in the model corresponds to a code list.

For example, you might create a location hierarchy. In the hierarchy model, you define the Region code list as the top-level node. You add the Country code list as a child node and then create a hierarchy model relationship from the Region node to the Country node.

reference data set

A logical grouping of reference data. A reference data set represents a category of reference data and acts as a template and container for code lists.

value mapping

The process of creating a one-way relationship between code values in the source code list and code values in the target code list. Value mappings provide a way to translate code values in the source code list to code values in the target code list.

INDEX

A

AssociateCodeListAttributeswithCDGCAssets 119
attributes
  about 26
  defining 103, 111, 130

C

child code values
  creating 126
code lists
  about 19
  creating 109, 110, 147
  deleting 122, 142
  stakeholders 105, 116, 138, 162
  viewinghistory 117
code value
  viewinghistory 128
code values
  about 23
  creating 123
  editing 127
crosswalks
  about 23
  creating 135
  stakeholders 105, 116, 138, 162
  value mapping 137
  viewinghistory 139
custom attributes
  about 27

D

Data Quality dictionaries associated with code lists
  managing 120
Data Quality dictionary associated with crosswalks
  managing 141
defining 111, 130
dependent code lists
  about 22
  defining 111, 130
dependent reference data sets
  about 18
dictionaries in a code list
  creating 120
  synchronizing data 120, 121, 142
dictionaries in a crosswalk
  creating 141
dictionary in a crosswalk
  synchronizing data 142
display attributes
  about 31
display settings
  defining 103, 111, 130

G

glossary 551

H

hierarchical code lists
  about 21
  defining 111, 130
hierarchical reference data sets
  about 17
  defining 103
    dependent reference data sets
      defining 103
hierarchies
  manage 159
  viewinghistory 165
history
  about 37

I

Informatica Global Customer Support
  contact information 14
Informatica Intelligent Data Management Cloud
  web site 13

M

maintenance outages 14
managecode lists
  about 109

N

notifications
  about 33

R

Reference 360
  Reference data configuration process 96
Reference 360 roles
  about 50
reference data attributes
  about 28

reference data comparisons
about 35
reference data sets
about 16
creating 102
defining 103
managing 102
stakeholders 105, 116, 138, 162
viewinghistory 106
resource
model version 3 210
REST API versions 200
resources
audit trail 350
audit trail (v2) 353
codelists 381, 423
codelists (v3 466
crosswalks 466
crosswalks (v2) 475
crosswalks (v3) 487
Delete value mappings 484
enum 529
export subset of code value hierarchy 406
hierarchies 495
hierarchies (v2) 505
import 259
import version 2 269
import version 3 278
rds 357
rds (v2) 368
version 1 export 290
version 2 export 296
version 3 export 309
REST APIs
add system reference data values 531
create code value 397, 424
create code value hierarchy subset 406
delete code values 401, 430
delete duplicate mappings of a crosswalk 472
delete duplicate mappings of a crosswalk (v2) 479
delete system reference data values 534
export code values at point in time to CSV (version 3) 311
export code values at point in time to CSV (version 4) 332
export code values at point in time to JSON (version 3) 309
export code values at point in time to JSON (version 4) 329
export code values to a CSV file (v2) 296
export code values to a CSV file (version 1) 290
export code values to JSON (version 1) 292
export code values to JSON FORMAT (v2) 298
export hierarchies to a CSV file 346
export hierarchies to a CSV file (v2) 302
export hierarchies to JSON 348
export hierarchies to JSON (v2) 304
export incoming crosswalk mappings 318, 338
export model version 3 210
export model version 4 329
export outgoing crosswalk mappings 340
export outgoing crosswalk mappings (version3) 320
export subset of code values in a hierarchical code list to a CSV file 410
export subset of code values in a hierarchical code list to JSON 411
export subset of code values in a hierarchical code list to JSON (v2) 462
export value mappings at point in time to JSON (version 3) 314, 335
export value mappings to a CSV file (v2) 300
export value mappings to a CSV file (version 1) 293
export value mappings to JSON (v2) 301
export value mappings to JSON (version 1) 295
REST APIs *(continued)*
get asset modeling job status 523
get code value details 391, 432
get error report for failed import job (v1) 267
get error report for failed import job(v3) 288
get history of a code list by time range 384
get history of a code list by time range (v2) 444
get history of a crosswalk by time range 468
get history of a crosswalk by time range (v3) 487
get history of a hierarchy by time range 499
get history of a hierarchy by time range (v2) 509
get history of a reference data set by time range 361
get history of a reference data set by time range (v2) 373
get history of code value hierarchies by time range 412
get history of code value hierarchies by time range (v2) 451
get history of code values of a code list by time range 392, 455
get history of hierarchy relationships by time range 502
get history of hierarchy relationships by time range (v2 513
get history of specified assets by time range 350
get history of specified assets by time range (v2) 353
get history of value mappings of a crosswalk by time range 482
get import job status (v1) 266
get import job status(v3) 287
get job details of a crosswalk cleanser job 472
get job details of a crosswalk cleanser job (v2) 480
get job details of an import model job (v3) 253
get value mappings for a code value 471
get value mappings for a code value (v3) 477, 491
import code values (v1) 259
import code values (v2) 269
import code values (v3) 278
import hierarchy relationships (v1) 263
import hierarchy relationships (v2) 275
import hierarchy relationships (v3) 284
import model version 3 for existing assets with incremental changes 238
import model version 3 for new assets 224
import value mappings (v1) 261
import value mappings (v2) 272
import value mappings (v3) 281
list code list details 381
list code list details (v2) 439
list code lists 366
list code lists (v2) 378
list crosswalk details 467
list crosswalk details (v2) 475
list crosswalks for a code list 416
list crosswalks for a code list (v2) 449
list hierarchies 495
list hierarchy details 496
list hierarchy details (v2) 505
list hierarchy model relationships 497
list hierarchy model relationships (v2) 506
list mappings of a crosswalk by crosswalkid 484
list reference data set details 358
list reference data set details (v2) 370
list reference data sets 357
list reference data sets (v2) 368
list system reference data values 529
move a code value 396
unlock a code list (v2) 423
unlock a code list (v3) 466
unlock a crosswalk 466, 475
update code value 399, 426
update system reference data values 532

S

SAML single sign-on
  additional attribute mapping properties 93
  configuration steps 91
  configuring 88
  creating users 87
  deleting users 87
  identity provider configuration properties 92
  overview 85
  registering a Secure Agent 87
  requirements 87
  restrictions 87
  SAML attribute mapping properties 93
  SAML group mapping properties 95
  SAML role mapping properties 95
  service provider metadata 95
  service provider settings 93
  user credentials storage 87
  with trusted IP ranges 87
search
  about 40
search code values (v1) 403
search code values (v2) 460
search REST API 403, 460
stakeholder roles
  about 53
stakeholders
  about 53
  assigning to code lists 105, 116, 138, 162
  assigning to crosswalks 105, 116, 138, 162
  assigning to reference data sets 105, 116, 138, 162
status
  IDMC 14
system status 14

T

tasks
  about 34
  reviewing 178
trust site
  description 14
trusted IP ranges
  with SAML single sign-on 87

U

upgrade notifications 14
user groups
  creating 57
users
  creating 56

V

value mappings
  configuring 137

W

web site 13
workflows
  about 33
  configuring 177
  manage 176